hushuqi 算法竞赛模板 (下)

ICPC/CCPC Algorithm Templates

目录

9. 线性代数与数值算法 1
9.1 矩阵 . . . 1
9.1.001 普通矩阵与取模矩阵 . . . 1
9.1.002 矩阵快速幂 . . . 3
9.1.003 Frobenius 标准形矩阵快速幂（高级版） . . . 4
9.1.004 `+min` / `+max` 半环矩阵 . . . 12
9.1.005 矩阵求逆（质数模数 / 任意模数） . . . 14
9.1.006 行列式（质数模数 / 任意模数） . . . 18
9.1.007 矩阵特征多项式 . . . 22
9.1.008 稀疏矩阵 . . . 27
9.1.009 Strassen 矩阵乘法 . . . 28
9.1.010 Woodbury 矩阵恒等式 . . . 30
9.1.011 伴随矩阵 . . . 34
9.1.012 Hafnian 与一般图完美匹配计数 . . . 37
9.1.013 Permanent / Ryser . . . 39
9.1.014 GF(2) 位集高斯消元 . . . 40
9.2 线性方程与规划 . . . 43
9.2.001 高斯消元：唯一解、无解、多解 . . . 43
9.2.002 高斯消元：列主元 . . . 48
9.2.003 高斯消元最小范数解 . . . 49
9.2.004 线性基（参见数据结构） . . . 52
9.2.005 Simplex 单纯形 . . . 52
9.2.006 0-1 分数规划 . . . 56
9.2.007 追赶法解三对角/周期方程 . . . 58
9.2.008 Pfaffian . . . 60
9.2.009 对称矩阵 LDL^T 分解 . . . 61
9.3 数值计算 . . . 64
9.3.001 自适应 Simpson . . . 64
9.3.002 Romberg 积分 . . . 65
9.3.003 Newton 多项式求根 . . . 66
9.3.004 一元三次方程 . . . 67
9.3.005 线性插值与幂和 . . . 70
9.3.006 指数乘多项式有限和 . . . 72
9.3.007 指数乘多项式无穷和 . . . 74
9.3.008 LLL 浮点实用版 . . . 75

10. 多项式、变换与线性递推 78
10.1 卷积与变换 . . . 78
10.1.001 DFT / IDFT . . . 78
10.1.002 FFT . . . 78
10.1.003 NTT . . . 81
10.1.004 MTT 任意模数卷积 . . . 93

10.1.005 二元 NTT / 二元多项式卷积 . . . 99
10.1.006 FWT / FST . . . 102
10.1.007 偶模 FWT 与光速乘 . . . 109
10.1.008 K 进制 FWT . . . 111
10.1.009 全在线卷积：基准版与 NTT 版 . . . 113
10.1.010 整除关系卷积 . . . 114
10.1.011 子集卷积 . . . 116
10.1.012 Min-Plus 卷积 . . . 118
10.1.013 集合形式幂级数 . . . 121
10.1.014 多元卷积 . . . 126
10.1.015 模 2^n 乘法卷积 . . . 130
10.1.016 $GF(2^{64})$ 卷积 . . . 133
10.1.017 模素数乘法幺半群卷积 . . . 140
10.1.018 模 2^{64} 精确整数卷积 . . . 142
10.2 多项式基础运算 . . . 143
10.2.001 多项式封装 . . . 143
10.2.002 乘法、除法、取模 . . . 146
10.2.003 乘法逆元 . . . 147
10.2.004 导数、积分 . . . 150
10.2.005 `ln`、`exp`、开根、快速幂 . . . 151
10.2.006 稀疏多项式除法 . . . 158
10.2.007 多点求值 . . . 160
10.2.008 拉格朗日单点求值、多点插值 . . . 164
10.2.009 连续点值插值与点值平移 . . . 165
10.2.010 快速阶乘 . . . 168
10.2.011 Power Projection . . . 171
10.2.012 多项式复合 . . . 174
10.2.013 下降幂多项式乘法 . . . 176
10.2.014 稀疏 FPS 运算 . . . 177
10.2.015 有限域多项式求根与因式分解 . . . 183
10.2.016 等差、等比节点求值与插值 . . . 194
10.2.017 二元形式幂级数逆 . . . 197
10.2.018 多项式序列乘积 . . . 199
10.2.019 单项式基转 Newton 基 . . . 200
10.2.020 多项式 half-GCD 与模逆 . . . 202
10.2.021 拉格朗日非连续点单点求值 . . . 209
10.2.022 计数版子集和生成函数 . . . 210
10.3 生成函数 . . . 211
10.3.001 普通生成函数 OGF . . . 211
10.3.002 指数生成函数 EGF . . . 212
10.3.003 q-int / q-Binomial . . . 213
10.3.004 二项式型生成函数 . . . 214
10.3.005 拉格朗日反演 . . . 214
10.3.006 两边界间非降序列计数 . . . 217

10.4 线性递推 . . . 222
10.4.001 Berlekamp-Massey（BM / 最小线性递推） . . . 222
10.4.002 最短递推式 . . . 224
10.4.003 Linear Recurrence . . . 224
10.4.004 递推式转分式 . . . 226
10.4.005 Bostan-Mori / 分式第 N 项 . . . 227
10.4.006 有理生成函数远处连续区间 . . . 228

11. 计算几何 232
11.1 二维基础对象 . . . 232
11.1.001 浮点误差与精确整数几何 . . . 232
11.1.002 Point / Vector . . . 232
11.1.003 点积、叉积、长度、夹角、旋转 . . . 235
11.1.004 极角排序与象限分类 . . . 236
11.1.005 Line / Ray / Segment . . . 237
11.1.006 平行、垂直、共线、同侧/异侧 . . . 238
11.1.007 直线交点、投影、对称点 . . . 239
11.1.008 点到直线/线段距离 . . . 240
11.1.009 线段相交与距离 . . . 241
11.1.010 线段中垂线 . . . 243
11.1.011 四点组成矩形 / 正方形判定 . . . 243
11.1.012 三角形内心、外心、垂心与旁心 . . . 244
11.2 多边形 . . . 245
11.2.001 多边形面积、周长、重心 . . . 245
11.2.002 点在多边形内与有符号绕数 . . . 246
11.2.003 线段是否在多边形内 . . . 247
11.2.004 多边形切割 . . . 249
11.2.005 多边形的核 . . . 249
11.2.006 Ear Clipping 三角剖分 . . . 250
11.2.007 多边形交、并 . . . 252
11.2.008 多边形与圆面积交 . . . 256
11.2.009 Pick 定理与格点多边形 . . . 258
11.2.010 三角形点集批量计数 . . . 259
11.2.011 多个简单多边形的覆盖层面积 . . . 261
11.2.012 直线平面嵌入的面枚举与对偶图 . . . 267
11.3 凸包 . . . 272
11.3.001 Andrew / Graham 静态凸包 . . . 272
11.3.002 单点插入动态凸包 . . . 273
11.3.003 点在凸包内 . . . 275
11.3.004 凸包极点、切点、最近点查询 . . . 276
11.3.005 凸包切线 . . . 279
11.3.006 旋转卡壳、凸包直径 . . . 280
11.3.007 最大面积三角形、四边形 . . . 281
11.3.008 闵可夫斯基和 . . . 283
11.3.009 半平面交 . . . 284

11.3.010 整数半平面交 . . . 289
11.3.011 Delaunay 三角剖分 / Voronoi 图 . . . 293
11.3.012 三角形四心（内心、外心、垂心、费马点） . . . 302
11.3.013 凸包分层 . . . 303
11.3.014 点集最小 / 最大面积三角形 . . . 308
11.4 圆 . . . 312
11.4.001 点、线、圆的位置关系 . . . 312
11.4.002 直线与圆、两圆交点 . . . 314
11.4.003 两圆相交面积 . . . 315
11.4.004 圆的切线与两圆公切线 . . . 315
11.4.005 三点外接圆、三角形内切圆 . . . 317
11.4.006 最小圆覆盖 . . . 318
11.4.007 固定半径圆覆盖最多点 . . . 321
11.4.008 圆反演与阿波罗尼斯圆 . . . 323
11.4.009 圆并与多圆覆盖层面积 . . . 324
11.4.010 最近圆对 . . . 328
11.4.011 圆弧表示与圆弧重心 . . . 329
11.4.012 圆上整点 . . . 330
11.5 距离、扫描线与特殊坐标 . . . 331
11.5.001 平面最近点对 . . . 331
11.5.002 最远曼哈顿距离 . . . 334
11.5.003 曼哈顿距离与切比雪夫距离转换 . . . 334
11.5.004 矩形面积并、周长并 . . . 335
11.5.005 六边形坐标系 . . . 338
11.5.006 球面经纬度与球面距离 . . . 339
11.5.007 多线段任意相交对 . . . 340
11.6 三维几何 . . . 345
11.6.001 三维点、向量、直线、平面 . . . 345
11.6.002 共线、共面、平行、垂直 . . . 346
11.6.003 直线与平面、两平面求交 . . . 349
11.6.004 点线面距离与夹角 . . . 350
11.6.005 空间线段与三角形相交 . . . 351
11.6.006 三维凸包 . . . 353
11.6.007 最小覆盖球 . . . 355
11.6.008 四面体体积与内切球 . . . 358
11.6.009 高维球体积、超球坐标与立体角 . . . 358
11.6.010 投影、最近点与空间距离 . . . 359
11.6.011 射线与 AABB 查询 . . . 363
11.6.012 空间三角形与平面裁剪 . . . 367
11.6.013 球、空间圆与球冠 / 球带 . . . 372
11.6.014 共面基底、凸包与网格截面 . . . 377
11.6.015 反射、二面角与球面多边形 . . . 382

12. 动态规划与优化 386
12.1 背包 . . . 386

12.1.001 01 背包 . . . 386
12.1.002 完全背包 . . . 386
12.1.003 多重背包 . . . 387
12.1.004 混合背包 . . . 388
12.1.005 二维费用背包 . . . 389
12.1.006 分组背包 . . . 390
12.1.007 有依赖背包 . . . 391
12.1.008 背包方案数与具体方案 . . . 392
12.1.009 树上背包及单调队列优化 . . . 393
12.1.010 布尔母函数 / 有界子集和可达性 . . . 395
12.1.011 基础区间 DP . . . 396
12.2 常见 DP . . . 399
12.2.001 数位 DP . . . 399
12.2.002 状压 DP . . . 402
12.2.003 棋盘分割 . . . 403
12.2.004 两台机器工作调度 . . . 405
12.2.005 找硬币问题 . . . 406
12.2.006 汉诺塔 . . . 407
12.2.007 最长上升/公共子序列 . . . 407
12.2.008 子集和 Bitset 优化 . . . 408
12.2.009 树形 DP 与动态 DP . . . 410
12.2.010 不同子序列计数 . . . 411
12.2.011 插头 DP . . . 412
12.3 DP 优化 . . . 416
12.3.001 单调队列优化 . . . 416
12.3.002 四边形不等式优化 . . . 417
12.3.003 分治优化 . . . 418
12.3.004 斜率优化 / 李超树优化 . . . 420
12.3.005 WQS 二分 . . . 422
12.3.006 SMAWK 全单调矩阵优化 . . . 424
12.3.007 Slope Trick . . . 425

13. 博弈论、概率与杂项数学 429
13.1 博弈论 . . . 429
13.1.001 Bash 与扩展 Bash . . . 429
13.1.002 Nim、Anti-Nim、Nim-K . . . 429
13.1.003 阶梯 Nim . . . 430
13.1.004 SG、Anti-SG、Multi-SG、Every-SG . . . 431
13.1.005 Wythoff 博弈 . . . 432
13.1.006 Fibonacci 博弈 . . . 433
13.1.007 树上删边、无向图删边游戏 . . . 434
13.1.008 取石子游戏 . . . 437
13.1.009 64 位 nim-product . . . 438
13.1.010 有环博弈的胜负平逆推 . . . 440
13.1.011 无向图删点行走与匹配博弈 . . . 441

13.1.012 二分图博弈与全部必胜起点 . . . 442
13.2 概率与分布 . . . 444
13.2.001 常见概率公式 . . . 444
13.2.002 常见概率分布 . . . 445
13.2.003 随机变量期望与方差 . . . 446
13.2.004 随机算法与 Xorshift . . . 447
13.2.005 期望 DP 与随机游走 . . . 448
13.2.006 概率界与集中不等式 . . . 449
13.2.007 指示变量、尾和公式与集齐问题 . . . 450
13.3 常用公式表 . . . 451
13.3.001 三角恒等式、导数与不定积分表 . . . 451
13.3.002 调和数与幂和 . . . 456
13.3.003 三角形、四边形与圆公式 . . . 458
13.3.004 常用面积、体积公式 . . . 459
13.3.005 日期公式：Zeller / 基姆拉尔森 . . . 460
13.3.006 常用常数与 NTT / 哈希素数表 . . . 460
13.3.007 数论数量级与极值表 . . . 462
13.3.008 裂项相消、有限差分与几何级数 . . . 464
13.3.009 泛型复数与三次单位根代数 . . . 466

14. 其他实用模板 471
14.1 其他实用模板 . . . 471
14.1.001 高精度整数（BigInt） . . . 471
14.1.002 分数类与有理数还原 . . . 479
14.1.003 表达式解析：中缀转后缀、表达式树 . . . 480
14.1.004 德州扑克牌型判断（五张牌与多手牌最佳五张） . . . 484
14.1.005 日期与时间换算 . . . 487
14.1.006 高精度进制转换与快速幂 . . . 489
14.1.007 9×9 数独字典序最小解 . . . 490
14.1.008 装箱问题 . . . 492
14.1.009 选数与网格路径计数 . . . 494
14.1.010 坐标类 . . . 495
14.1.011 Python：基础语法、容器、字符串、数学库 . . . 495
14.1.012 Python：并查集、线段树、最大流、费用流、FFT、NTT . . . 497
14.1.013 Python：Splay . . . 503
14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO . . . 507
14.1.015 完全图部分边染色的 1-因子扩展尝试 . . . 508
14.1.016 高精度整数平方根 . . . 511
14.1.017 一元二次方程实根 . . . 514
14.1.018 N 皇后构造解 . . . 515
14.1.019 十六进制百万位大整数 . . . 517
14.1.020 任意精度定点与十进制数 . . . 531
14.1.021 高精度圆周率：Machin 与 Chudnovsky . . . 540

15. ACM 常用英文单词 543

15.1 按字母索引 543
15.1.001 A 543
15.1.002 B 544
15.1.003 C 544
15.1.004 D 545
15.1.005 E 546
15.1.006 F 547
15.1.007 G 547
15.1.008 H 548
15.1.009 I 548
15.1.010 J 549
15.1.011 K 549
15.1.012 L 549
15.1.013 M 550
15.1.014 N 551
15.1.015 O 551
15.1.016 P 552
15.1.017 Q 552
15.1.018 R 553
15.1.019 S 553
15.1.020 T 555
15.1.021 U 555
15.1.022 V 556
15.1.023 W 556

9. 线性代数与数值算法

9.1 矩阵

9.1.001 普通矩阵与取模矩阵

矩阵 A 记作行数 n、列数 m 的二维数组，`Mat<T>` 把元素类型 T 参数化，只要 T 支持所在环的加法与乘法就能复用同一套加乘与快速幂。乘积 $C = AB$ 定义为 $C_{ij} = \sum_k A_{ik}B_{kj}$，要求左矩阵列数等于右矩阵行数，一般不交换；实现里三重循环特意按 i, k, j 顺序累加，使最内层对 `x[i]`、`y[k]` 都连续访问，比教科书的 i, j, k 更缓存友好。`eye(n)` 造的 n 阶单位矩阵既是快速幂的初值，也是空乘积的单位元。元素类型若用原生整数，连乘会溢出；需要取模时把 `T` 换成模整数类型（每次乘加后自动约减），本条不内置取模、由 `T` 承担。

```
template <class T> struct Mat
{
    // n、m 是行数与列数；a 始终应有 n 行，每行恰有 m 个元素。
    int n, m;
    vector<vector<T>> a;

    Mat(int n = 0, int m = 0) : n(n), m(m), a(n, vector<T>(m))
    {
        // 建立 n 行 m 列的零矩阵；不要直接改变 a 的行数或行宽。
    }

    static Mat eye(int n)
    {
        // n 是矩阵阶数；返回 n 阶单位矩阵。
        Mat a(n, n);
        for (int i = 0; i < n; i++)
        {
            a[i][i] = 1;
        }
        return a;
    }

    vector<T> &operator[](int x)
    {
        // x 是行下标；返回该行的可修改引用。
        return a[x];
    }
    const vector<T> &operator[](int x) const
    {
        // x 是行下标；返回该行的只读引用。
        return a[x];
    }

    friend Mat operator*(const Mat &x, const Mat &y)
    {
        // x、y 是乘法左右两侧的矩阵；返回维数相容时的普通乘积。
```

```
        assert((int)x.a.size() == x.n && (int)y.a.size() == y.n && x.m == y.n); // 调试检查，可删。
        // 调试检查，可删。
        for (const auto &r : x.a)
        {
            assert((int)r.size() == x.m);
        }
        // 调试检查，可删。
        for (const auto &r : y.a)
        {
            assert((int)r.size() == y.m);
        }
        Mat z(x.n, y.m);
        for (int i = 0; i < x.n; i++)
        {
            for (int k = 0; k < x.m; k++)
            {
                for (int j = 0; j < y.m; j++)
                {
                    z[i][j] += x[i][k] * y[k][j];
                }
            }
        }
        return z;
    }

    Mat pow(unsigned long long b) const
    {
        // b 是非负指数；返回当前方阵的 b 次幂。
        assert(n == m); // 调试检查，可删。
        Mat x = *this;
        Mat ans = eye(n);
        while (b)
        {
            if (b & 1)
            {
                ans = ans * x;
            }
            b >>= 1;
            if (b)
            {
                x = x * x;
            }
        }
        return ans;
    }
};
```

以 `Mat<T>(n,m)` 建零矩阵、`Mat<T>::eye(n)` 建单位阵，`operator*` 要求维数相容，`pow(b)` 要求方阵，$b=0$ 时返回单位阵。由于 `a` 是公开成员，若直接改坏其行数或行宽，后续运算不再合法；代码中的断言只用于调试。$n\times m$ 乘 $m\times k$ 是 $O(nmk)$ 时间、$O(nk)$ 结果空间；n 阶方阵的 b 次幂是 $O(n^3\log(b+1))$ 时间、$O(n^2)$ 空间。转移是普通加乘时本条即够；只需裸函数时用 9.1.002 矩阵快速幂；转移是 min-加、

max-加等半环运算时改用 9.1.004 +min / +max 半环矩阵；非零元远少于 n^2 时用 9.1.008 稀疏矩阵；要行列式或逆矩阵则转 9.1.006 行列式（质数模数 / 任意模数） 与 9.1.005 矩阵求逆（质数模数 / 任意模数）。

9.1.002 矩阵快速幂

常系数线性递推 $f_n = \sum_i c_i f_{n-i}$ 与固定状态转移都能写成向量左乘同一个转移方阵 M，第 b 步的状态即 M^b 乘初始向量，于是把递推的 $O(b)$ 步压成对 M 求幂。快速幂把指数按二进制拆开：$M^b = \prod_{b\text{ 的置位}i} M^{2^i}$，边平方边在对应位乘入答案。只有方阵能做普通幂，答案初值必须是同阶单位矩阵。本条给出不依赖 9.1.001 普通矩阵与取模矩阵 类封装的裸函数版，`T` 只需支持加法、乘法与从 0、1 构造。

```cpp
template <class T> vector<vector<T>> matPow(vector<vector<T>> a, unsigned long long b)
{
    // a 是方阵，b 是非负指数；返回 a 的 b 次幂，T 必须支持加法、乘法和从 0、1 构造。
    int n = a.size();
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector<vector<T>> ans(n, vector<T>(n));
    for (int i = 0; i < n; i++)
    {
        ans[i][i] = 1;
    }
    auto mul = [&](const vector<vector<T>> &x, const vector<vector<T>> &y)
    {
        // x、y 是同阶方阵；返回普通矩阵乘积。
        vector<vector<T>> z(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                for (int j = 0; j < n; j++)
                {
                    z[i][j] += x[i][k] * y[k][j];
                }
            }
        }
        return z;
    };
    while (b)
    {
        if (b & 1)
        {
            ans = mul(ans, a);
        }
        b >>= 1;
        if (b)
        {
            a = mul(a, a);
```

```
        }
    }
    return ans;
}
```

调用 `matPow(a,b)`，`a` 必须是方阵，$b = 0$ 时返回单位矩阵；斐波那契转移 $\begin{pmatrix}1 & 1\\ 1 & 0\end{pmatrix}$ 的 10 次幂应为 $\begin{pmatrix}89 & 55\\ 55 & 34\end{pmatrix}$。$O(n^3 \log(b+1))$ 时间、$O(n^2)$ 额外空间。转移是 min-加或 max-加运算时改用 9.1.004 `+min` / `+max` 半环矩阵；需要携带行列信息或下标运算符时用 9.1.001 普通矩阵与取模矩阵 的 `Mat<T>::pow`。

下面只调用本条的裸函数，检查手算三次幂、零次幂、单位阵与 Fibonacci 转移：

```
signed main()
{
    vector<vector<long long>> a{{1, 2}, {3, 4}};
    vector<vector<long long>> cube{{37, 54}, {81, 118}};
    assert(matPow(a, 3) == cube);
    assert(matPow(a, 0) == vector<vector<long long>>({{1, 0}, {0, 1}}));
    assert(matPow(vector<vector<long long>>{{1, 0}, {0, 1}}, 7) == vector<vector<long
    ↪  long>>({{1, 0}, {0, 1}}));
    auto f = matPow(vector<vector<long long>>{{1, 1}, {1, 0}}, 10);
    cout << f[0][0] << ' ' << f[0][1] << ' ' << f[1][0] << ' ' << f[1][1] << '\n'; // 期望输出:
    ↪  89 55 55 34
    return 0;
}
```

9.1.003 Frobenius 标准形矩阵快速幂（高级版）

基础二进制矩阵幂每个指数位都做一次 n^3 乘法。若指数很大且矩阵阶数也高，可先随机选 Krylov 向量，把空间拆成循环子空间，使 $P^{-1}AP$ 成为若干伴随块；每个 d 阶块上的幂只需算 x^k 对块多项式取模，最后再乘 P 与 P^{-1} 还原。构造基底时会验证新块多项式对旧块的整除关系；验证失败就丢弃本轮随机向量并重建，因此这是 Las Vegas 算法：运行时间随机，返回结果不带错误概率。

```
template <class T> struct NaivePolyMul
{
    vector<T> operator()(const vector<T> &a, const vector<T> &b) const
    {
        // a、b 是系数从低次到高次排列的多项式；返回普通卷积。
        if (a.empty() || b.empty())
        {
            return {};
        }
        vector<T> c(a.size() + b.size() - 1);
        for (int i = 0; i < (int)a.size(); i++)
        {
            for (int j = 0; j < (int)b.size(); j++)
            {
                c[i + j] += a[i] * b[j];
```

```
            }
        }
        return c;
    }
};

template <class T> struct FieldBasis
{
    // n 是向量维数，rk 是当前秩。
    int n, rk = 0;
    // basis 保存插入的原向量；row 是按首个非零位置存放的阶梯基，way 给出 row 关于 basis 的系数。
    vector<vector<T>> basis, row, way;

    FieldBasis(int n) : n(n), row(n, vector<T>(n)), way(n, vector<T>(n))
    {
        // 建立 n 维空线性基。
    }

    pair<bool, vector<T>> solveOrAdd(vector<T> x)
    {
        // x 是待查询向量；可表示时返回 true 及其关于 basis 的系数，否则插入 x 并返回 false。
        vector<T> original = x;
        vector<T> coef(n);
        for (int i = 0; i < n; i++)
        {
            if (row[i][i] == T(1))
            {
                coef[i] = x[i];
                for (int j = i; j < n; j++)
                {
                    x[j] -= coef[i] * row[i][j];
                }
            }
        }
        int p = 0;
        while (p < n && x[p] == T(0))
        {
            p++;
        }
        if (p == n)
        {
            vector<T> ans(rk);
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < rk; j++)
                {
                    ans[j] += coef[i] * way[i][j];
                }
            }
            return {true, ans};
        }
        T z = T(1) / x[p];
```

```
        for (int j = p; j < n; j++)
        {
            x[j] *= z;
        }
        for (auto &v : coef)
        {
            v *= z;
        }
        basis.push_back(move(original));
        row[p] = move(x);
        way[p][rk] = z;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < rk; j++)
            {
                way[p][j] -= coef[i] * way[i][j];
            }
        }
        rk++;
        return {false, {}};
    }

    void rollback(int r)
    {
        // r 是此前的秩；撤销最近插入的向量，直到秩恢复为 r。
        while (rk > r)
        {
            rk--;
            basis.pop_back();
            for (int i = 0; i < n; i++)
            {
                if (way[i][rk] != T(0))
                {
                    fill(row[i].begin(), row[i].end(), T(0));
                    fill(way[i].begin(), way[i].end(), T(0));
                }
            }
        }
    }
};

template <class T, class Conv = NaivePolyMul<T>> struct FrobPower
{
    using Poly = vector<T>;
    using Matrix = vector<vector<T>>;

    // n 是矩阵阶数；a 是原矩阵，p 的列是 Krylov 基，ip 是 p 的逆矩阵。
    int n;
    Matrix a, p, ip;
    // start 保存各循环子空间的起始向量；relation 保存对应伴随块的递推系数。
    vector<Poly> start, relation;
    // conv 计算多项式卷积，rng 产生试探向量，trials 记录完整重建次数。
```

```
    Conv conv;
    mt19937_64 rng;
    int trials = 0;

    FrobPower(Matrix a, Conv conv = Conv(), unsigned long long seed = 0)
        : n(a.size()), a(move(a)), conv(move(conv)), rng(seed)
    {
        // a 是域上的方阵，conv 是卷积器，seed 是固定随机种子；构造时完成 Frobenius 分解。
        // 调试检查，可删。
        for (const auto &r : this->a)
        {
            assert((int)r.size() == n);
        }
        do
        {
            trials++;
        } while (!trial());
    }

    Matrix pow(unsigned long long b)
    {
        // b 是非负指数；返回原矩阵的 b 次幂。
        Matrix x(n, vector<T>(n));
        int s = 0;
        for (const auto &g : relation)
        {
            int d = g.size();
            Poly f = polyPowMod(g, b);
            for (int j = 0; j < d; j++)
            {
                for (int i = 0; i < (int)f.size(); i++)
                {
                    x[s + i][s + j] = f[i];
                }
                if (j + 1 < d)
                {
                    f.insert(f.begin(), T(0));
                    divmod(f, g);
                }
            }
            s += d;
        }
        return matrixMul(matrixMul(p, x), ip);
    }

private:
    static void trim(Poly &f)
    {
        // f 的系数按低次到高次排列；原地删除高次零项。
        while (!f.empty() && f.back() == T(0))
        {
            f.pop_back();
```

```
        }
    }

    Poly divmod(Poly &f, const Poly &g)
    {
        // f 是被除式，g 表示 x^d-sum(g[i]x^i)；原地留下余数并返回商。
        trim(f);
        int d = g.size();
        Poly q;
        for (int i = (int)f.size() - 1; i >= d; i--)
        {
            q.push_back(f[i]);
            for (int j = 0; j < d; j++)
            {
                f[i - d + j] += f[i] * g[j];
            }
            f[i] = T(0);
        }
        trim(f);
        reverse(q.begin(), q.end());
        return q;
    }

    Poly polyMulMod(const Poly &x, const Poly &y, const Poly &g)
    {
        // x、y 是被乘多项式，g 表示模多项式；返回乘积的余式。
        Poly z = conv(x, y);
        divmod(z, g);
        return z;
    }

    Poly polyPowMod(const Poly &g, unsigned long long b)
    {
        // g 表示模多项式，b 是非负指数；返回 x^b 的余式。
        Poly ans{T(1)};
        Poly x{T(0), T(1)};
        divmod(x, g);
        while (b)
        {
            if (b & 1)
            {
                ans = polyMulMod(ans, x, g);
            }
            b >>= 1;
            if (b)
            {
                x = polyMulMod(x, x, g);
            }
        }
        return ans;
    }

```

```
    Matrix matrixMul(const Matrix &x, const Matrix &y)
    {
        // x、y 是 n 阶方阵；返回普通矩阵乘积。
        Matrix z(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                for (int j = 0; j < n; j++)
                {
                    z[i][j] += x[i][k] * y[k][j];
                }
            }
        }
        return z;
    }

    optional<Matrix> matrixInv(Matrix x)
    {
        // x 是 n 阶方阵；可逆时返回逆矩阵，否则返回空。
        Matrix y(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            y[i][i] = T(1);
        }
        for (int col = 0; col < n; col++)
        {
            int row = col;
            while (row < n && x[row][col] == T(0))
            {
                row++;
            }
            if (row == n)
            {
                return nullopt;
            }
            swap(x[row], x[col]);
            swap(y[row], y[col]);
            T z = T(1) / x[col][col];
            for (int j = 0; j < n; j++)
            {
                x[col][j] *= z;
                y[col][j] *= z;
            }
            for (int i = 0; i < n; i++)
            {
                if (i == col)
                {
                    continue;
                }
                T q = x[i][col];
                for (int j = 0; j < n; j++)
```

```
                {
                    x[i][j] -= q * x[col][j];
                    y[i][j] -= q * y[col][j];
                }
            }
        }
        return y;
    }

    vector<T> apply(const vector<T> &v)
    {
        // v 是 n 维列向量；返回 a*v。
        vector<T> w(n);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                w[i] += a[i][j] * v[j];
            }
        }
        return w;
    }

    vector<T> randomVector()
    {
        // 返回由 rng 生成的 n 维试探向量。
        vector<T> v(n);
        for (auto &x : v)
        {
            x = T((long long)(rng() & 0x7FFFFFFFFFFFFFFFULL));
        }
        return v;
    }

    bool trial()
    {
        // 从空基底尝试构造一次 Frobenius 分解；成功时写入 p、ip 与 relation。
        start.clear();
        relation.clear();
        FieldBasis<T> s(n);
        while (s.rk < n)
        {
            int oldRank = s.rk;
            vector<T> v = randomVector();
            start.push_back(v);
            while (true)
            {
                auto [solved, coef] = s.solveOrAdd(v);
                if (!solved)
                {
                    v = apply(v);
                    continue;
```

```
                }
                Poly f(coef.begin() + oldRank, coef.end());
                if (f.empty())
                {
                    return false;
                }
                relation.push_back(f);
                if (start.size() == 1)
                {
                    break;
                }
                v = start.back();
                int pos = 0;
                for (int k = 0; k + 1 < (int)start.size(); k++)
                {
                    int d = relation[k].size();
                    Poly rem(coef.begin() + pos, coef.begin() + pos + d);
                    Poly q = divmod(rem, f);
                    if (!rem.empty())
                    {
                        return false;
                    }
                    for (int i = 0; i < (int)q.size(); i++)
                    {
                        for (int j = 0; j < n; j++)
                        {
                            v[j] -= q[i] * s.basis[pos + i][j];
                        }
                    }
                    pos += d;
                }
                s.rollback(oldRank);
                start.back() = v;
                for (int i = 0; i < (int)f.size(); i++)
                {
                    if (s.solveOrAdd(v).first)
                    {
                        return false;
                    }
                    if (i + 1 < (int)f.size())
                    {
                        v = apply(v);
                    }
                }
                break;
            }
        }
        p.assign(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
```

```
                p[i][j] = s.basis[j][i];
            }
        }
        auto inv = matrixInv(p);
        if (!inv)
        {
            return false;
        }
        ip = move(*inv);
        return true;
    }
};
```

`FrobPower<T> fp(a,{},seed)` 构造后用 `fp.pow(k)`。`T` 必须是域；每轮分解与求逆为 $O(n^3)$，失败后会从空基底重试，`trials` 可查看实际轮数。一次 `pow` 的块内求幂为 $O(\sum_i d_i^2 \log(k+1))$，乘 P、P^{-1} 还原为 $O(n^3)$，峰值空间为 $O(n^2)$。第二模板参数只替换卷积器；当前 `divmod` 仍是二次取模，因此仅接入快速卷积只能减小卷积部分的常数。若 $M(d)$ 表示长度 d 的卷积复杂度，要达到 $O(\sum_i M(d_i)\log(k+1))$ 还必须同时换成快速多项式取模。有限域大小为 p 时单轮失败概率为 $O(n/p)$。它依赖域除法、卷积和随机 Krylov 分解，只适合大矩阵或超大指数；一般情况优先使用 9.1.002 矩阵快速幂。

固定查验时可在模 998244353 的域上取 `seed=0` 与 $A=\begin{pmatrix}1&1&0\\0&1&1\\0&0&1\end{pmatrix}$；`fp.pow(5)` 应为 $\begin{pmatrix}1&5&10\\0&1&5\\0&0&1\end{pmatrix}$。再分别对零矩阵、单位阵和两个 Krylov 块组成的分块矩阵核对指数 $0,1,2,17$ 的预先算定结果，可覆盖空乘积、不可逆矩阵和多块分解；完整随机检验放在本地测试文件。

9.1.004 `+min` / `+max` 半环矩阵

普通矩阵乘法 $C_{ij}=\sum_k A_{ik}B_{kj}$ 只要求标量加法与乘法构成半环——满足结合律、乘法对加法分配、有各自单位元——就能整套照搬。把 $(+,\times)$ 换成 $(\min,+)$ 得 min-plus（热带）半环，此时 M^b 的 (i,j) 项恰是 $i\to j$ 恰好 b 条边的最短路，正是 Floyd 分治与定长最短路的代数形式；换成 $(\max,+)$ 则给定长最长路。两者的加法单位元分别是 $+\infty$、$-\infty$ 哨兵，乘法单位元都是 0。`semiringMul` 与 `semiringPow` 把 `add`、`mul` 与单位元 `zero`、`one` 抽成回调，因此同一份幂运算能服务任意半环，不必为每种运算重写。

```
template <class T, class Add, class Mul>
vector<vector<T>> semiringMul(const vector<vector<T>> &a, const vector<vector<T>> &b, T zero,
 ↪  Add add, Mul mul)
{
    // a、b 是待乘矩阵，zero 是加法单位元；add、mul 定义半环加法与乘法。
    int n = a.size();
    int m = b.size();
    int k = m ? b[0].size() : 0;
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == m);
    }
    // 调试检查，可删。
```

```
    for (const auto &r : b)
    {
        assert((int)r.size() == k);
    }
    vector c(n, vector<T>(k, zero));
    for (int i = 0; i < n; i++)
    {
        for (int z = 0; z < m; z++)
        {
            for (int j = 0; j < k; j++)
            {
                c[i][j] = add(c[i][j], mul(a[i][z], b[z][j]));
            }
        }
    }
    return c;
}

template <class T, class Add, class Mul>
vector<vector<T>> semiringPow(vector<vector<T>> a, unsigned long long b, T zero, T one, Add
 ↪ add, Mul mul)
{
    // a 是方阵，b 是非负指数；zero、one 是加法与乘法单位元。
    int n = a.size();
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector ans(n, vector<T>(n, zero));
    for (int i = 0; i < n; i++)
    {
        ans[i][i] = one;
    }
    while (b)
    {
        if (b & 1)
        {
            ans = semiringMul(ans, a, zero, add, mul);
        }
        b >>= 1;
        if (b)
        {
            a = semiringMul(a, a, zero, add, mul);
        }
    }
    return ans;
}
```

`semiringMul(a,b,zero,add,mul)` 做一次半环乘法，`semiringPow(a,b,zero,one,add,mul)` 对方阵求 b 次幂。min-plus 不能把普通 `+` 直接作为 `mul`：不可达哨兵与负边相加后会变成有限值。安全回调可写成：

```cpp
signed main()
{
    // 0->1 权 2, 1->2 权 3, 0->2 权 10; 求恰好走 2 条边的最短路。
    vector<vector<int>> g{{INF, 2, 10}, {INF, INF, 3}, {INF, INF, INF}};
    auto minAdd = [](int x, int y)
    {
        return min(x, y);
    };
    auto minMul = [](int x, int y)
    {
        return x == INF || y == INF ? INF : x + y;
    };
    auto d = semiringPow(g, 2, INF, 0LL, minAdd, minMul);
    assert(d[0][2] == 5 && d[0][1] == INF && d[1][1] == INF);
}
```

这里还要求所有有限距离及其加法都在 `long long` 范围内且不等于 `INF`。max-plus 同理使用 `-INF`，只要任一因子为 `-INF` 就返回 `-INF`。$n\times m$ 与 $m\times k$ 的半环乘法是 $O(nmk)$，n 阶方阵快速幂是 $O(n^3\log(b+1))$，空间 $O(n^2)$。转移是普通加乘时回到 9.1.001 普通矩阵与取模矩阵、9.1.002 矩阵快速幂；若求任意步数最短路而非恰好 k 条边，用图论章的 Floyd 或 Bellman-Ford 更直接。

9.1.005 矩阵求逆（质数模数 / 任意模数）

求方阵 A 的模逆矩阵，即找 B 使 $AB\equiv I$。素数模 p 下每个非零元都可逆，直接 Gauss-Jordan 消元即可；难点在合数模：此时非零元未必可逆，不能随便拿主元求逆。关键事实是 A 模 p^k 可逆当且仅当 $\det(A)$ 不被 p 整除，等价于消元时每一列都能找到模 p 非零的主元（模 p^k 的单位）。于是把模数分解成互质的质数幂 $\prod p_i^{k_i}$，`matInvPP` 在每个 $p_i^{k_i}$ 上单独消元（主元用扩欧求 `invInt` 得到模 p^k 逆元），再对每个位置用 CRT 把各分量唯一合并回模 mod。该接口的模数在运行期确定且允许合数，不能使用固定素数域 `MInt<P>`，宽乘法显式复用 7.1.001 快速幂的 `mulMod`。任一分量不可逆则整体不可逆，返回空。

```cpp
// 依赖 7.1.001 快速幂
int invInt(int a, int mod)
{
    // a 与 mod 互质且 mod>1; 返回 a 模 mod 的最小非负逆元。
    assert(mod > 1); // 调试检查，可删。
    a %= mod;
    if (a < 0)
    {
        a += mod;
    }
    int b = mod;
    __int128 x = 1;
    __int128 y = 0;
    while (b)
    {
        int q = a / b;
        a -= q * b;
        swap(a, b);
        x -= q * y;
```

```
        swap(x, y);
    }
    assert(a == 1); // 调试检查，可删。
    x %= mod;
    if (x < 0)
    {
        x += mod;
    }
    return (int)x;
}

optional<vector<vector<int>>> matInvPP(vector<vector<int>> a, int p, int pk)
{
    // a 是方阵，p 是质数，pk 是 p 的正次幂；返回模 pk 的逆矩阵，不可逆时返回空。
    int n = a.size();
    assert(p >= 2 && pk >= p); // 调试检查，可删。
    for (auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
        for (auto &x : row)
        {
            x %= pk;
            if (x < 0)
            {
                x += pk;
            }
        }
    }
    vector b(n, vector<int>(n));
    for (int i = 0; i < n; i++)
    {
        b[i][i] = 1;
    }
    for (int c = 0; c < n; c++)
    {
        int r = c;
        while (r < n && a[r][c] % p == 0)
        {
            r++;
        }
        if (r == n)
        {
            return nullopt;
        }
        swap(a[r], a[c]);
        swap(b[r], b[c]);
        int z = invInt(a[c][c], pk);
        for (int j = 0; j < n; j++)
        {
            a[c][j] = mulMod(a[c][j], z, pk);
            b[c][j] = mulMod(b[c][j], z, pk);
        }
```

```
        for (int i = 0; i < n; i++)
        {
            if (i == c)
            {
                continue;
            }
            int q = a[i][c];
            for (int j = 0; j < n; j++)
            {
                a[i][j] = (a[i][j] - mulMod(q, a[c][j], pk)) % pk;
                b[i][j] = (b[i][j] - mulMod(q, b[c][j], pk)) % pk;
                if (a[i][j] < 0)
                {
                    a[i][j] += pk;
                }
                if (b[i][j] < 0)
                {
                    b[i][j] += pk;
                }
            }
        }
    }
    return b;
}

optional<vector<vector<int>>> matInvAny(const vector<vector<int>> &a, int mod)
{
    // a 是方阵，mod>1 可为素数或合数；返回模 mod 的逆矩阵，不存在时返回空。
    assert(mod > 1); // 调试检查，可删。
    vector<pair<int, int>> fac;
    int x = mod;
    for (int p = 2; p <= x / p; p++)
    {
        if (x % p)
        {
            continue;
        }
        int pk = 1;
        while (x % p == 0)
        {
            x /= p;
            pk *= p;
        }
        fac.push_back({p, pk});
    }
    if (x > 1)
    {
        fac.push_back({x, x});
    }
    int n = a.size();
    int cur = 1;
    vector ans(n, vector<int>(n));
```

```
    for (auto [p, pk] : fac)
    {
        auto b = matInvPP(a, p, pk);
        if (!b)
        {
            return nullopt;
        }
        int z = invInt(cur % pk, pk);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int t = ((*b)[i][j] - ans[i][j]) % pk;
                if (t < 0)
                {
                    t += pk;
                }
                t = mulMod(t, z, pk);
                // 已处理模数 cur 与当前 pk 的乘积不超过总模数，故 cur*t 不溢出。
                ans[i][j] += cur * t;
            }
        }
        cur *= pk;
    }
    return ans;
}
```

调用 `matInvAny(a,mod)`，`a` 是方阵、`mod>1`，返回空表示不可逆；素数模也走同一接口，此时只有一个质数幂分量。矩阵 $\begin{pmatrix}2&3\\3&2\end{pmatrix}$ 模 6 可逆且逆恰为自身，$\begin{pmatrix}2&0\\0&1\end{pmatrix}$ 模 6 不可逆。若记模数为 M，试除分解最坏为 $O(\sqrt{M})$，随后是 $O(\omega(M)n^3)$ 时间、$O(n^2)$ 空间，其中 $\omega(M)$ 是 M 的不同质因子数。只要行列式而不需要整个逆矩阵时用 9.1.006 行列式（质数模数 / 任意模数）；解线性方程组时用 9.2.001 高斯消元：唯一解、无解、多解，避免多算 n 列。

下面直接检查两组已知逆矩阵和一组不可逆矩阵；不可逆样例的行列式为 2，与模数 6 不互质：

```
signed main()
{
    auto a = matInvAny({{1, 2}, {3, 4}}, 5);
    assert(a && *a == vector<vector<int>>({{3, 1}, {4, 2}}));
    auto b = matInvAny({{2, 3}, {3, 2}}, 6);
    assert(b && *b == vector<vector<int>>({{2, 3}, {3, 2}}));
    assert(!matInvAny({{2, 0}, {0, 1}}, 6));
    cout << (*b)[0][0] << ' ' << (*b)[0][1] << ' ' << (*b)[1][0] << ' ' << (*b)[1][1] << '\n';
    // 期望输出：2 3 3 2
    return 0;
}
```

9.1.006 行列式（质数模数 / 任意模数）

行列式在行初等变换下的行为完全确定：交换两行变号，某行乘常数 c 则整体乘 c，某行加另一行的倍数不变。把矩阵消成上三角后行列式就是对角线乘积（乘上累计的交换符号）。素数模下 `detPrime` 直接取主元逆元（费马小定理求 a^{mod-2}）消去下方各行。合数模不能求逆，`detAny` 改用辗转相除式消元：对同列的两行反复取商作整倍数相减再交换，就像对这两个元素跑欧几里得算法，把下方元素辗转消成 0，全程只用整数乘减、绝不请求模逆——代价是每列多花 $O(\log mod)$ 轮相减。任一列主元最终为 0 则行列式为 0。

```cpp
// 依赖 7.1.001 快速幂
int detPrime(vector<vector<int>> a, int mod)
{
    // a 是方阵，mod 是素数；返回行列式对 mod 的最小非负余数。
    assert(mod > 1); // 调试检查，可删。
    int n = a.size();
    int ans = 1;
    for (auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    for (int c = 0; c < n; c++)
    {
        int r = c;
        while (r < n && a[r][c] == 0)
        {
            r++;
        }
        if (r == n)
        {
            return 0;
        }
        if (r != c)
        {
            swap(a[r], a[c]);
            ans = (mod - ans) % mod;
        }
        ans = mulMod(ans, a[c][c], mod);
        int z = 1;
        int b = mod - 2;
        int x = a[c][c];
        while (b)
        {
            if (b & 1)
```

```
            {
                z = mulMod(z, x, mod);
            }
            b >>= 1;
            if (b)
            {
                x = mulMod(x, x, mod);
            }
        }
        for (int i = c + 1; i < n; i++)
        {
            int q = mulMod(a[i][c], z, mod);
            for (int j = c; j < n; j++)
            {
                a[i][j] = (a[i][j] - mulMod(q, a[c][j], mod)) % mod;
                if (a[i][j] < 0)
                {
                    a[i][j] += mod;
                }
            }
        }
    }
    return ans;
}

int detAny(vector<vector<int>> a, int mod)
{
    // a 是方阵，mod>1 可为合数；返回行列式对 mod 的最小非负余数，全程不做模逆。
    assert(mod > 1); // 调试检查，可删。
    int n = a.size();
    int ans = 1;
    for (auto &row : a)
    {
        assert((int)row.size() == n); // 调试检查，可删。
        for (auto &x : row)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    for (int c = 0; c < n; c++)
    {
        for (int i = c + 1; i < n; i++)
        {
            while (a[i][c])
            {
                int q = a[c][c] / a[i][c];
                for (int j = c; j < n; j++)
                {
```

```
                    a[c][j] = (a[c][j] - mulMod(q, a[i][j], mod)) % mod;
                    if (a[c][j] < 0)
                    {
                        a[c][j] += mod;
                    }
                }
                swap(a[c], a[i]);
                ans = (mod - ans) % mod;
            }
        }
        if (a[c][c] == 0)
        {
            return 0;
        }
        ans = mulMod(ans, a[c][c], mod);
    }
    return ans;
}
```

模数是编译期已知且小于 2^{30} 的素数时，热点可改用 `detPrimeFast<P>`：矩阵改成连续 64 位数组，每行累计 8 次消元后才统一取模，避免内层每个元素一次 `%`。核心消元如下，完整初始化、换行与收尾见对应头文件：

```
template <int P> int detPrimeFast(const vector<vector<long long>> &src)
{
    // P 是小于 2^30 的素数，src 是方阵；返回 det(src) mod P，每行累计 8 次消元后统一取模。
    static_assert(1 < P && P < (1U << 30));
    int n = src.size();
    vector<unsigned long long> a((size_t)n * n);
    vector<int> cnt(n);
    for (int i = 0; i < n; i++)
    {
        assert(src[i].size() == n); // 调试检查，可删
        for (int j = 0; j < n; j++)
        {
            long long x = src[i][j] % (long long)P;
            if (x < 0)
            {
                x += P;
            }
            a[(size_t)i * n + j] = x;
        }
    }
    auto power = [](unsigned long long x, int b)
    {
        unsigned long long ans = 1;
        for (; b; b >>= 1, x = x * x % P)
        {
            if (b & 1)
            {
                ans = ans * x % P;
```

```
            }
        }
        return (int)ans;
    };
    bool neg = false;
    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n; j++)
        {
            a[(size_t)j * n + i] %= P;
        }
        int p = i;
        while (p < n && !a[(size_t)p * n + i])
        {
            p++;
        }
        if (p == n)
        {
            return 0;
        }
        if (p != i)
        {
            for (int j = i; j < n; j++)
            {
                swap(a[(size_t)i * n + j], a[(size_t)p * n + j]);
            }
            swap(cnt[i], cnt[p]);
            neg = !neg;
        }
        auto row = a.data() + (size_t)i * n;
        for (int j = i; j < n; j++)
        {
            row[j] %= P;
        }
        int iv = power(row[i], P - 2);
        for (int k = i + 1; k < n; k++)
        {
            auto cur = a.data() + (size_t)k * n;
            int q = cur[i] ? P - (unsigned long long)iv * cur[i] % P : 0;
            int j = i;
            for (; j + 4 <= n; j += 4)
            {
                cur[j] += (unsigned long long)q * row[j];
                cur[j + 1] += (unsigned long long)q * row[j + 1];
                cur[j + 2] += (unsigned long long)q * row[j + 2];
                cur[j + 3] += (unsigned long long)q * row[j + 3];
            }
            for (; j < n; j++)
            {
                cur[j] += (unsigned long long)q * row[j];
            }
            if (++cnt[k] == 8)
```

```
            {
                cnt[k] = 0;
                for (int j = i; j < n; j++)
                {
                    cur[j] %= P;
                }
            }
        }
    }
    unsigned long long ans = neg ? P - 1 : 1;
    for (int i = 0; i < n; i++)
    {
        ans = ans * (a[(size_t)i * n + i] % P) % P;
    }
    return ans;
}
```

已知 mod 为素数时调用 `detPrime(a,mod)`，可能为合数时调用 `detAny(a,mod)`；两者都接受负元素并返回最小非负余数。固定小素数且矩阵较大时调用 `detPrimeFast<P>(a)`；它仍是便携 C++，未绑定 AVX2，但要求 $P < 2^{30}$ 以证明 8 轮延迟取模不溢出 64 位。矩阵 $\begin{pmatrix} 2 & 3 \\ 3 & 2 \end{pmatrix}$ 的整数行列式为 -5，故 `detPrime(a,7)` 返回 2、`detAny(a,6)` 返回 1。三版渐进复杂度不变；本机 400×400 随机矩阵的 5 轮中位数约为 `8.2ms`，逐元素取模版约为 `25.8ms`。运行时模数或大模数不能用固定版。若记模数为 M，素数模版本为 $O(n^3 + n \log M)$，任意模版本为 $O(n^3 \log M)$ 次基本算术，空间均为 $O(n^2)$。需要在模意义下解方程或求逆而非只要行列式时用 9.2.001 高斯消元：唯一解、无解、多解、9.1.005 矩阵求逆（质数模数 / 任意模数）；要 $\det(xI - A)$ 的整条特征多项式时用 9.1.007 矩阵特征多项式。

下面用两个手算行列式的固定矩阵检查素数模、合数模、负元素和奇异矩阵；只保留一个 3×3 例，不做大规模随机对拍：

```
signed main()
{
    vector<vector<int>> a{{2, 3}, {3, 2}};
    assert(detPrime(a, 7) == 2);
    assert(detAny(a, 6) == 1);
    vector<vector<int>> b{{1, 2, 3}, {0, 1, 4}, {5, 6, 0}};
    assert(detPrime(b, 7) == 1 && detAny(b, 6) == 1);
    assert(detAny({{1, 2}, {2, 4}}, 6) == 0);
    cout << detAny(a, 6) << ' ' << detPrime(b, 7) << '\n'; // 期望输出：1 1
    return 0;
}
```

9.1.007 矩阵特征多项式

特征多项式 $p(x) = \det(xI - A)$ 的系数装着行列式、迹等谱信息，也是 Cayley-Hamilton（$p(A) = 0$）与常系数矩阵递推降幂的基础。`charPoly` 先用相似变换把 A 化成上 Hessenberg 形 H（$i > j + 1$ 时 $h_{i,j} = 0$）：对第 j 列在第 $j + 1$ 行以下选非零主元，交换行与对应列，再用行变换 $r_i \leftarrow r_i - q r_{j+1}$ 消去下方元素，并同步做列变换 $c_{j+1} \leftarrow c_{j+1} + q c_i$，保持相似从而特征多项式不变。之后记 p_i 为 H 左上

i 阶主子阵的特征多项式，$p_0 = 1$，沿最后一列展开（下标从 1 开始）得

$$p_i(x) = (x - h_{i,i})\,p_{i-1}(x) - \sum_{m=1}^{i-1} h_{i-m,i}\Big(\prod_{t=0}^{m-1} h_{i-t,i-t-1}\Big)p_{i-m-1}(x),$$

p_n 即所求。消元要做除法，`T` 必须是域（模素数的取模类或浮点）。返回顺序是从常数项到最高次项：`c[n]` 恒为 1，`c[0]` 是 $(-1)^n \det A$。`detAPlusBz(a,b)` 在同一框架下求 $\det(A + Bz)$。

```cpp
template <class T> vector<T> charPoly(const vector<vector<T>> &a)
{
    // a 是域上的方阵；返回 det(xI-a) 从常数项到最高次项的系数。
    int n = a.size();
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector h = a;
    for (int col = 0; col + 2 < n; col++)
    {
        int p = col + 1;
        while (p < n && h[p][col] == T(0))
        {
            p++;
        }
        if (p == n) continue;
        if (p != col + 1)
        {
            swap(h[p], h[col + 1]);
            for (int i = 0; i < n; i++)
            {
                swap(h[i][p], h[i][col + 1]);
            }
        }
        T inv = T(1) / h[col + 1][col];
        for (int row = col + 2; row < n; row++)
        {
            if (h[row][col] == T(0)) continue;
            T q = h[row][col] * inv;
            for (int j = col; j < n; j++)
            {
                h[row][j] -= q * h[col + 1][j];
            }
            for (int i = 0; i < n; i++)
            {
                h[i][col + 1] += q * h[i][row];
            }
        }
    }
    vector<vector<T>> p(n + 1);
    p[0] = {T(1)};
    for (int i = 1; i <= n; i++)
```

```
    {
        p[i].assign(i + 1, T(0));
        for (int j = 0; j < i; j++)
        {
            p[i][j] -= p[i - 1][j] * h[i - 1][i - 1];
            p[i][j + 1] += p[i - 1][j];
        }
        T prod = 1;
        for (int len = 1; len < i; len++)
        {
            prod *= h[i - len][i - len - 1];
            T q = prod * h[i - len - 1][i - 1];
            for (int j = 0; j <= i - len - 1; j++)
            {
                p[i][j] -= q * p[i - len - 1][j];
            }
        }
    }
    return p[n];
}

template <class T> vector<T> detAPlusBz(vector<vector<T>> a,
                                        vector<vector<T>> b)
{
    // a、b 是域上的同阶方阵；返回 det(a+bz) 从常数项到最高次项的系数，允许 b 奇异。
    int n = a.size(), shift = 0;
    assert((int)b.size() == n); // 调试检查，可删
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n && (int)b[i].size() == n); // 调试检查，可删
    }
    T scale = T(1);
    for (int col = 0; col < n; col++)
    {
        while (true)
        {
            for (int p = 0; p < col; p++)
            {
                T w = b[p][col];
                for (int i = 0; i < n; i++)
                {
                    a[i][col] -= w * a[i][p];
                }
                b[p][col] = T(0);
            }
            int p = col;
            while (p < n && b[p][col] == T(0))
            {
                p++;
            }
            if (p < n) break;
            if (++shift > n) return vector<T>(n + 1);
```

```
            for (int i = 0; i < n; i++)
            {
                b[i][col] = a[i][col], a[i][col] = T(0);
            }
        }
        int p = col;
        while (b[p][col] == T(0))
        {
            p++;
        }
        if (p != col) scale = -scale, swap(a[p], a[col]), swap(b[p], b[col]);
        scale *= b[col][col];
        T inv = T(1) / b[col][col];
        for (int j = 0; j < n; j++)
        {
            a[col][j] *= inv, b[col][j] *= inv;
        }
        for (int i = col + 1; i < n; i++)
        {
            T w = b[i][col];
            for (int j = 0; j < n; j++)
            {
                a[i][j] -= w * a[col][j], b[i][j] -= w * b[col][j];
            }
        }
    }
    for (auto &row : a)
    {
        for (T &x : row)
        {
            x = -x;
        }
    }
    for (int col = 0; col + 1 < n; col++)
    {
        int p = col + 1;
        while (p < n && a[p][col] == T(0))
        {
            p++;
        }
        if (p == n) continue;
        swap(a[p], a[col + 1]);
        for (int i = 0; i < n; i++)
        {
            swap(a[i][p], a[i][col + 1]);
        }
        T inv = T(1) / a[col + 1][col];
        for (int i = col + 2; i < n; i++)
        {
            T w = a[i][col] * inv;
            for (int j = 0; j < n; j++)
            {
```

```
                a[i][j] -= w * a[col + 1][j], a[j][col + 1] += w * a[j][i];
            }
        }
    }
    vector<vector<T>> f(n + 1);
    f[0] = {T(1)};
    for (int m = 1; m <= n; m++)
    {
        f[m].assign(m + 1, T(0));
        for (int i = 0; i < m; i++)
        {
            T w = a[i][m - 1];
            for (int k = i + 1; k < m; k++)
            {
                w *= a[k][k - 1];
            }
            for (int d = 0; d <= i; d++)
            {
                f[m][d] -= w * f[i][d];
            }
        }
        for (int d = 0; d < m; d++)
        {
            f[m][d + 1] += f[m - 1][d];
        }
    }
    vector<T> ans(n + 1);
    for (int d = 0; d + shift <= n; d++)
    {
        ans[d] = f[n][d + shift] * scale;
    }
    return ans;
}
```

调用 `charPoly(a)` 传入域上的方阵，返回长度 $n+1$ 的 $\det(xI-A)$ 系数向量；Hessenberg 相似变换与递推共 $O(n^3)$ 时间、$O(n^2)$ 空间。`detAPlusBz(a,b)` 返回 $\det(A+Bz)$，允许 B 奇异并自动处理最低非零次数的平移，复杂度同为 $O(n^3)$。只要行列式时仍使用 9.1.006 行列式（质数模数 / 任意模数）。

下面直接核对空矩阵、对角矩阵与一个非对角矩阵的已知特征多项式：

```
signed main()
{
    auto same = [](const vector<long double> &a, const vector<long double> &b)
    {
        if (a.size() != b.size())
        {
            return false;
        }
        for (int i = 0; i < (int)a.size(); i++)
        {
            if (fabsl(a[i] - b[i]) > 1E-10L)
            {
```

```
                return false;
            }
        }
        return true;
    };
    assert(same(charPoly<long double>({}), {1}));
    assert(same(charPoly<long double>({{1, 0}, {0, 2}}), {2, -3, 1})); // 即 x^2-3x+2。
    auto c = charPoly<long double>({{1, 2}, {3, 4}});
    assert(same(c, {-2, -5, 1}));                                      // 即 x^2-5x-2。
    return 0;
}
```

9.1.008 稀疏矩阵

当矩阵非零元个数 nnz 远少于 $n \times m$ 时，稠密存储既费内存又让每次乘法白算大量零。`SparseMat` 按行存（列下标，值），只保留非零项，矩阵乘向量 $y = Ax$ 就退化为遍历每行的非零表累加 $y_i \mathrel{+}= v \cdot x_j$。它专为幂法、共轭梯度等只需反复算 Ax 的线性迭代设计；若做稀疏乘稀疏且结果趋于稠密，或反复自乘导致填充增长，稀疏表的优势会消失，此时应转回稠密矩阵。`add(i,j,x)` 中 `i`、`j` 分别是行、列下标，同一位置可加入多项并在乘法时累加。

```
template <class T> struct SparseMat
{
    // n、m 是行数与列数，a[i] 存第 i 行的 (列下标, 值)。
    int n, m;
    vector<vector<pair<int, T>>> a;

    SparseMat(int n, int m) : n(n), m(m), a(n)
    {
        // 建立 n 行 m 列的零矩阵。
    }

    void add(int i, int j, T x)
    {
        // 在 (i,j) 加一项 x；重复位置会在乘法时累加，零项应由调用方过滤。
        assert(0 <= i && i < n && 0 <= j && j < m); // 调试检查，可删。
        a[i].push_back({j, x});
    }

    vector<T> operator*(const vector<T> &x) const
    {
        // x 是 m 维列向量；返回 A*x。
        assert((int)x.size() == m); // 调试检查，可删。
        vector<T> y(n);
        for (int i = 0; i < n; i++)
        {
            for (auto [j, v] : a[i])
            {
                y[i] += v * x[j];
            }
```

```
        }
        return y;
    }
};
```

以 `SparseMat<T> s(n,m)` 建 $n \times m$ 零矩阵，`s.add(i,j,x)` 只加入非零项，`s*x` 返回长度 n 的乘积向量。同一位置可加入多次，乘法时各项自然相加；若把零值也放入表中，算法仍正确但会浪费时间和空间。单次矩阵乘向量为 $O(\mathrm{nnz})$，存储为 $O(n + \mathrm{nnz})$。矩阵稠密时回到 9.1.001 普通矩阵与取模矩阵；固定步数的半环最短路用 9.1.004 `+min` / `+max` 半环矩阵，那是稠密表示。

9.1.009 Strassen 矩阵乘法

把两个 $n \times n$ 矩阵各切成四块 $m \times m (m = n/2)$, 朴素分块乘法要算八次子块乘积。Strassen 用七次线性组合乘积 $p_1, \dots, p_7$ 拼出全部四块结果,把递归式从 $T(n) = 8T(n/2)$ 降到 $T(n) = 7T(n/2) + O(n^2)$, 复杂度为 $O(n^{\log_2 7})$。本实现在规模 ≤ 32 时退回朴素乘法；入口先把行数、公共维数和列数补齐到同一个二次幂，算完再裁回原形。实现放在 `strassen_mul` 命名空间内，避免其中的矩阵别名与 9.1.001 普通矩阵与取模矩阵 的 `Mat` 结构体重名。

```
namespace strassen_mul
{
template <class T> using Mat = vector<vector<T>>;

template <class T> Mat<T> matAdd(const Mat<T> &a, const Mat<T> &b, int sgn = 1)
{
    // a、b 是同阶方阵；sgn=1 时相加，sgn=-1 时相减。
    int n = a.size();
    Mat<T> c(n, vector<T>(n));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            c[i][j] = a[i][j] + (sgn == 1 ? b[i][j] : -b[i][j]);
        }
    }
    return c;
}

template <class T> Mat<T> strassenRec(const Mat<T> &a, const Mat<T> &b)
{
    // a、b 是同阶二次幂方阵；返回其乘积。
    int n = a.size();
    if (n <= 32)
    {
        Mat<T> c(n, vector<T>(n));
        for (int i = 0; i < n; i++)
        {
            for (int k = 0; k < n; k++)
            {
                for (int j = 0; j < n; j++)
```

```
                {
                    c[i][j] += a[i][k] * b[k][j];
                }
            }
        }
        return c;
    }
    int m = n / 2;
    Mat<T> x[2][2], y[2][2];
    for (int p = 0; p < 2; p++)
    {
        for (int q = 0; q < 2; q++)
        {
            x[p][q].assign(m, vector<T>(m));
            y[p][q].assign(m, vector<T>(m));
            for (int i = 0; i < m; i++)
            {
                copy_n(a[i + p * m].begin() + q * m, m, x[p][q][i].begin());
                copy_n(b[i + p * m].begin() + q * m, m, y[p][q][i].begin());
            }
        }
    }
    auto p1 = strassenRec(matAdd(x[0][0], x[1][1]), matAdd(y[0][0], y[1][1]));
    auto p2 = strassenRec(matAdd(x[1][0], x[1][1]), y[0][0]);
    auto p3 = strassenRec(x[0][0], matAdd(y[0][1], y[1][1], -1));
    auto p4 = strassenRec(x[1][1], matAdd(y[1][0], y[0][0], -1));
    auto p5 = strassenRec(matAdd(x[0][0], x[0][1]), y[1][1]);
    auto p6 = strassenRec(matAdd(x[1][0], x[0][0], -1), matAdd(y[0][0], y[0][1]));
    auto p7 = strassenRec(matAdd(x[0][1], x[1][1], -1), matAdd(y[1][0], y[1][1]));
    Mat<T> c(n, vector<T>(n));
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < m; j++)
        {
            c[i][j] = p1[i][j] + p4[i][j] - p5[i][j] + p7[i][j];
            c[i][j + m] = p3[i][j] + p5[i][j];
            c[i + m][j] = p2[i][j] + p4[i][j];
            c[i + m][j + m] = p1[i][j] - p2[i][j] + p3[i][j] + p6[i][j];
        }
    }
    return c;
}

template <class T> Mat<T> strassen(const Mat<T> &a, const Mat<T> &b)
{
    // a 是 n*m 矩阵，b 是 m*k 矩阵；返回 n*k 乘积。
    int n = a.size();
    int m = b.size();
    int k = m ? b[0].size() : 0;
    // 调试检查，可删。
    for (const auto &r : a)
    {
```

```
        assert((int)r.size() == m);
    }
    // 调试检查，可删。
    for (const auto &r : b)
    {
        assert((int)r.size() == k);
    }
    if (!n || !m || !k)
    {
        return Mat<T>(n, vector<T>(k));
    }
    int z = (int)bit_ceil((unsigned long long)max({n, m, k}));
    Mat<T> x(z, vector<T>(z));
    Mat<T> y(z, vector<T>(z));
    for (int i = 0; i < n; i++)
    {
        copy(a[i].begin(), a[i].end(), x[i].begin());
    }
    for (int i = 0; i < m; i++)
    {
        copy(b[i].begin(), b[i].end(), y[i].begin());
    }
    x = strassenRec(x, y);
    x.resize(n);
    for (auto &v : x)
    {
        v.resize(k);
    }
    return x;
}
}
```

调用 `strassen_mul::strassen(a,b)`，`a` 是 $n\times m$、`b` 是 $m\times k$，返回 $n\times k$ 乘积；零行或零列矩阵返回对应零形。补零尺寸用 64 位无符号整数求 $N=\text{bit_ceil}(\max(n,m,k))$，要求该尺寸和对应矩阵均可分配。元素类型 `T` 必须同时容纳最终乘积与递归中的块加减，最终答案可表示并不能单独保证 Strassen 的临时值可表示。时间 $O(N^{\log_2 7})$、峰值空间 $O(N^2)$；常规规模或模矩阵仍优先用 9.1.001 普通矩阵与取模矩阵。

9.1.010 Woodbury 矩阵恒等式

已知 A^{-1}，若 A 又叠加一个低秩修正变成 $A+UCV$（U 是 $n\times k$、C 是 $k\times k$、V 是 $k\times n$，$k\ll n$），从头重算逆是 $O(n^3)$，太浪费。Woodbury 恒等式 $(A+UCV)^{-1}=A^{-1}-A^{-1}U(C^{-1}+VA^{-1}U)^{-1}VA^{-1}$ 把它转成只对一个 $k\times k$ 矩阵 $C^{-1}+VA^{-1}U$ 求逆，其余都是矩阵乘法。要求 A、C 以及那个 $k\times k$ 中间矩阵都可逆。$k=1$ 的特例即 Sherman–Morrison。下面 `inv` 用带部分选主元的高斯消元求逆（`long double`），`woodbury(ai,u,ci,v)` 直接吃 A^{-1}、C^{-1} 与低秩因子。

```
namespace woodbury
{
using Mat = vector<vector<long double>>;
```

```cpp

Mat mul(const Mat &a, const Mat &b)
{
    // a、b 是维数相容矩阵；返回矩阵乘积。
    if (a.empty())
    {
        return {};
    }
    if (b.empty())
    {
        for (const auto &r : a)
        {
            assert(r.empty()); // 调试检查，可删。
        }
        return Mat(a.size());
    }
    int n = a.size();
    int k = b.size();
    int m = b[0].size();
    // 调试检查，可删。
    for (const auto &r : b)
    {
        assert((int)r.size() == m);
    }
    Mat c(n, vector<long double>(m));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == k); // 调试检查，可删。
        for (int p = 0; p < k; p++)
        {
            for (int j = 0; j < m; j++)
            {
                c[i][j] += a[i][p] * b[p][j];
            }
        }
    }
    return c;
}

Mat inv(Mat a)
{
    // a 是可逆方阵；返回高斯消元求得的逆矩阵。
    int n = a.size();
    Mat b(n, vector<long double>(2 * n));
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n); // 调试检查，可删。
        copy(a[i].begin(), a[i].end(), b[i].begin());
        b[i][n + i] = 1;
    }
    for (int j = 0; j < n; j++)
    {
```

```
        int p = j;
        for (int i = j + 1; i < n; i++)
        {
            if (abs(b[i][j]) > abs(b[p][j]))
            {
                p = i;
            }
        }
        assert(abs(b[p][j]) > 1E-18L); // 调试检查，可删。
        swap(b[p], b[j]);
        long double x = b[j][j];
        for (auto &v : b[j])
        {
            v /= x;
        }
        for (int i = 0; i < n; i++)
        {
            if (i == j)
            {
                continue;
            }
            x = b[i][j];
            for (int k = 0; k < 2 * n; k++)
            {
                b[i][k] -= x * b[j][k];
            }
        }
    }
    Mat ans(n, vector<long double>(n));
    for (int i = 0; i < n; i++)
    {
        copy(b[i].begin() + n, b[i].end(), ans[i].begin());
    }
    return ans;
}

Mat woodbury(const Mat &ai, const Mat &u, const Mat &ci, const Mat &v)
{
    // ai、ci 分别是 A 与 C 的逆，u、v 是低秩因子；返回 (A+UCV) 的逆。
    int n = ai.size();
    int k = ci.size();
    assert((int)u.size() == n && (int)v.size() == k); // 调试检查，可删。
    // 调试检查，可删。
    for (const auto &r : ai)
    {
        assert((int)r.size() == n);
    }
    // 调试检查，可删。
    for (const auto &r : u)
    {
        assert((int)r.size() == k);
    }
```

```
    // 调试检查，可删。
    for (const auto &r : ci)
    {
        assert((int)r.size() == k);
    }
    // 调试检查，可删。
    for (const auto &r : v)
    {
        assert((int)r.size() == n);
    }
    if (n == 0)
    {
        return {};
    }
    if (k == 0)
    {
        return ai;
    }
    Mat left = mul(ai, u);
    Mat right = mul(v, ai);
    Mat mid = ci;
    Mat vaiu = mul(right, u);
    for (int i = 0; i < (int)mid.size(); i++)
    {
        for (int j = 0; j < (int)mid.size(); j++)
        {
            mid[i][j] += vaiu[i][j];
        }
    }
    Mat sub = mul(mul(left, inv(mid)), right);
    Mat ans = ai;
    for (int i = 0; i < (int)ans.size(); i++)
    {
        for (int j = 0; j < (int)ans.size(); j++)
        {
            ans[i][j] -= sub[i][j];
        }
    }
    return ans;
}
}
```

调用 `woodbury::woodbury(ai,u,ci,v)`，其中 `ai=A^{-1}`、`ci=C^{-1}` 需调用方预先备好（或用同命名空间的 `inv` 现算）。`k=0` 是合法的零秩修正，此时 `ci`、`v` 为空、`u` 每行为空，函数直接返回 `ai`。低秩为 k、原维数为 n 时 $O(n^2k+nk^2+k^3)$ 时间、$O(n^2+k^2)$ 空间，当 $k \ll n$ 时远优于重新求逆。若修正不是低秩、或没有现成 A^{-1}，直接对 $A+UCV$ 用 9.1.005 矩阵求逆（质数模数 / 任意模数）（模意义）或本条 `inv`（浮点）从头求逆。

9.1.011 伴随矩阵

伴随矩阵满足 $A\,\mathrm{adj}(A)=\det(A)I$。满秩时直接用 $\mathrm{adj}(A)=\det(A)A^{-1}$；秩为 $n-1$ 时，右零空间与左零空间都为一维，伴随矩阵是两个零向量外积乘一个标量，标量由一个 $(n-1)\times(n-1)$ 余子式确定；秩至多 $n-2$ 时所有余子式都为零，伴随矩阵就是零矩阵。下面实现只要求素数模，按行最简形确定秩和零向量，整个过程是确定性的，不依赖随机增广。

```cpp
// 依赖 7.1.001 快速幂
// 依赖 9.1.005 矩阵求逆（质数模数 / 任意模数）
// 依赖 9.1.006 行列式（质数模数 / 任意模数）
vector<vector<int>> adjugateP(vector<vector<int>> a, int mod)
{
    // a 是方阵，mod 是素数；返回 A 的伴随矩阵。
    assert(mod > 1); // 调试检查，可删。
    int n = a.size();
    for (auto &r : a)
    {
        assert((int)r.size() == n); // 调试检查，可删。
        for (auto &x : r)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    if (n == 0)
    {
        return {};
    }
    auto power = [&](int x, int b)
    {
        // x 是非零剩余，b 是非负指数；返回 x 的 b 次幂对 mod 的余数。
        int ans = 1;
        while (b)
        {
            if (b & 1)
            {
                ans = mulMod(ans, x, mod);
            }
            x = mulMod(x, x, mod);
            b >>= 1;
        }
        return ans;
    };
    auto reduce = [&](vector<vector<int>> m)
    {
        // m 是 n 阶方阵；返回秩、行最简形和每列对应的主元行，非主元列记为 -1。
        vector<int> pivot(n, -1);
        int rk = 0;
```

```
        for (int col = 0; col < n && rk < n; col++)
        {
            int row = rk;
            while (row < n && m[row][col] == 0)
            {
                row++;
            }
            if (row == n)
            {
                continue;
            }
            swap(m[row], m[rk]);
            int z = power(m[rk][col], mod - 2);
            for (int j = 0; j < n; j++)
            {
                m[rk][j] = mulMod(m[rk][j], z, mod);
            }
            for (int i = 0; i < n; i++)
            {
                if (i == rk)
                {
                    continue;
                }
                int q = m[i][col];
                for (int j = 0; j < n; j++)
                {
                    m[i][j] = (m[i][j] - mulMod(q, m[rk][j], mod)) % mod;
                    if (m[i][j] < 0)
                    {
                        m[i][j] += mod;
                    }
                }
            }
            pivot[col] = rk++;
        }
        return tuple<int, vector<vector<int>>, vector<int>>(rk, move(m), move(pivot));
    };
    auto [rk, r, pivot] = reduce(a);
    vector ans(n, vector<int>(n));
    if (rk == n)
    {
        auto inv = matInvPP(a, mod, mod);
        int det = detPrime(a, mod);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                ans[i][j] = mulMod(det, (*inv)[i][j], mod);
            }
        }
        return ans;
    }
```

```
    if (rk <= n - 2)
    {
        return ans;
    }
    int fx = find(pivot.begin(), pivot.end(), -1) - pivot.begin();
    vector<int> x(n);
    x[fx] = 1;
    for (int col = 0; col < n; col++)
    {
        if (pivot[col] != -1)
        {
            x[col] = (mod - r[pivot[col]][fx]) % mod;
        }
    }
    vector<vector<int>> at(n, vector<int>(n));
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            at[i][j] = a[j][i];
        }
    }
    auto [rk2, rt, pivot2] = reduce(at);
    (void)rk2;
    int fy = find(pivot2.begin(), pivot2.end(), -1) - pivot2.begin();
    vector<int> y(n);
    y[fy] = 1;
    for (int col = 0; col < n; col++)
    {
        if (pivot2[col] != -1)
        {
            y[col] = (mod - rt[pivot2[col]][fy]) % mod;
        }
    }
    int i0 = 0, j0 = 0;
    while (x[i0] == 0)
    {
        i0++;
    }
    while (y[j0] == 0)
    {
        j0++;
    }
    vector<vector<int>> minor;
    for (int i = 0; i < n; i++)
    {
        if (i == j0)
        {
            continue;
        }
        vector<int> row;
        for (int j = 0; j < n; j++)
```

```
        {
            if (j != i0)
            {
                row.push_back(a[i][j]);
            }
        }
        minor.push_back(move(row));
    }
    int cof = detPrime(minor, mod);
    if ((i0 + j0) & 1 && cof)
    {
        cof = mod - cof;
    }
    int scale = mulMod(cof, power(x[i0], mod - 2), mod);
    scale = mulMod(scale, power(y[j0], mod - 2), mod);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            ans[i][j] = mulMod(mulMod(scale, x[i], mod), y[j], mod);
        }
    }
    return ans;
}
```

`adjugateP(a,mod)` 覆盖满秩、秩 $n-1$ 和秩至多 $n-2$ 三种情况，返回值总在 $[0, mod)$。时间为 $O(n^3)$，空间为 $O(n^2)$；满秩分支复用本章的 9.1.005 矩阵求逆（质数模数 / 任意模数） 和 9.1.006 行列式（质数模数 / 任意模数）。只要逆矩阵而不要伴随矩阵时不要额外乘行列式。

下面分别检查三种秩：

```
signed main()
{
    assert(adjugateP({{1, 2}, {3, 4}}, 5) == vector<vector<int>>({{4, 3}, {2, 1}}));
    assert(adjugateP({{1, 2}, {2, 4}}, 5) == vector<vector<int>>({{4, 3}, {3, 1}}));
    assert(adjugateP({{0, 0}, {0, 0}}, 5) == vector<vector<int>>({{0, 0}, {0, 0}}));
    assert(adjugateP({{0}}, 5) == vector<vector<int>>({{1}}));
    cout << "OK\n";
    return 0;
}
```

9.1.012 Hafnian 与一般图完美匹配计数

对 $2n \times 2n$ 对称矩阵 A，Hafnian 是所有完美匹配边权乘积之和：$\mathrm{haf}(A) = \sum_{M} \prod_{(i,j)\in M} A_{ij}$。邻接矩阵取 $0/1$ 时，它就是一般图完美匹配数。直接枚举匹配约有 $(2n-1)!!$ 项；Bjorklund 型算法把顶点预先两两分组，把交错环贡献写成集合函数 `cyc`，再调用 10.1.013 集合形式幂级数 的 `setExp`。子集卷积与集合指数的表示、工作区和边界完全适用，因此本条只保留 Hafnian 自己的状态转移。

```cpp
// 依赖 10.1.013 集合形式幂级数
template <class T> T hafnian(const vector<vector<T>> &a)
{
    // a 是 2n 阶对称矩阵；返回其 Hafnian。
    int N = a.size();
    assert(N % 2 == 0); // 调试检查，可删。
    int n = N / 2;
    assert(n <= 20); // 调试检查，可删。
    // 调试检查，可删。
    for (int i = 0; i < N; i++)
    {
        assert((int)a[i].size() == N);
        for (int j = 0; j < N; j++)
        {
            assert(a[i][j] == a[j][i]);
        }
    }
    vector<T> cyc(1LL << n);
    for (int i = 0; i < n; i++)
    {
        int u = 2 * i;
        int v = u + 1;
        int w = 2 * i;
        cyc[1LL << i] += a[u][v];
        vector<T> dp(w * (1LL << i));
        for (int j = 0; j < i; j++)
        {
            int x = 2 * j;
            int y = x + 1;
            dp[w * (1LL << j) + x] += a[u][y];
            dp[w * (1LL << j) + y] += a[u][x];
        }
        for (int mask = 0; mask < (1LL << i); mask++)
        {
            for (int j = 0; j < i; j++)
            {
                int x = 2 * j;
                int y = x + 1;
                cyc[mask | (1LL << i)] += dp[w * mask + x] * a[v][x];
                cyc[mask | (1LL << i)] += dp[w * mask + y] * a[v][y];
                int rest = ((1LL << i) - 1) ^ mask;
                while (rest)
                {
                    int k = countr_zero((unsigned)rest);
                    rest &= rest - 1;
                    int p = 2 * k;
                    int q = p + 1;
                    int to = mask | (1LL << k);
                    dp[w * to + p] += dp[w * mask + x] * a[x][q];
                    dp[w * to + p] += dp[w * mask + y] * a[y][q];
                    dp[w * to + q] += dp[w * mask + x] * a[x][p];
                    dp[w * to + q] += dp[w * mask + y] * a[y][p];
```

```
                }
            }
        }
    }
    return setExp(cyc).back();
}
```

`hafnian(a)` 要求矩阵阶数为 $2n$ 且对称，主对角线不会参与答案；`T` 必须构成交换环，空矩阵的 Hafnian 按空乘积定义为 1。本条显式依赖第 10 章的 `setExp`，拼接或单独使用头文件时都要带上该公共头。按子集大小分层计数，时间为 $O(n^2 2^n)$，峰值空间为 $O(n2^n)$，因此代码把 `n` 限为 20。它计算的是一般图完美匹配的带权和；二分图完美匹配计数对应行列式或 permanent 的其他专用算法，不能混用。

下面用 4×4 矩阵直接核对三种配对之和 $1\cdot 6+2\cdot 5+3\cdot 4=28$：

```
signed main()
{
    vector<vector<long long>> a{{0, 1, 2, 3}, {1, 0, 4, 5}, {2, 4, 0, 6}, {3, 5, 6, 0}};
    assert(hafnian(vector<vector<long long>>{}) == 1);
    assert(hafnian(vector<vector<long long>>{{0, 7}, {7, 0}}) == 7);
    cout << hafnian(a) << '\n'; // 期望输出：28
    return 0;
}
```

9.1.013 Permanent / Ryser

n 阶方阵的积和式 $\operatorname{perm}(A)=\sum_{\sigma}\prod_{i}a_{i,\sigma(i)}$ 与行列式只差符号，却没有已知的多项式算法（#P 完全）。Ryser 公式用容斥写成

$$\operatorname{perm}(A)=\sum_{S\subseteq\{0,\dots,n-1\}}(-1)^{n-|S|}\prod_{i=0}^{n-1}\sum_{j\in S}a_{i,j},$$

右边展开后每一项是每行各选一列的乘积，只有所选列恰好覆盖全部列（即构成排列）的项系数为 1，其余项被容斥抵消。按 Gray 码枚举 S，相邻两个集合只差一列，每行的列和可以 $O(1)$ 更新，所以每个集合只需 $O(n)$。

```
int permanent(const vector<vector<int>> &a, int mod)
{
    // a 是 n<=24 的方阵，mod>0；返回积和式 perm(a) 模 mod 的非负值。
    // Ryser 公式：perm = sum_{S} (-1)^{n-|S|} prod_i sum_{j in S} a[i][j]，按格雷码每次改一列。
    int n = a.size();
    assert(mod > 0 && n <= 24); // 调试检查，可删。
    vector<int> row(n); // row[i] 是当前列集合 S 在第 i 行上的元素和（模 mod）。
    int ans = 0;
    for (int t = 1; t < (1LL << n); t++)
    {
        int s = t ^ (t >> 1), b = __builtin_ctzll(t), sg = s >> b & 1 ? 1 : -1;
        int prod = 1 % mod;
        for (int i = 0; i < n; i++)
        {
            row[i] = (((__int128)row[i] + (__int128)sg * a[i][b]) % mod + mod) % mod;
```

```
            prod = (__int128)prod * row[i] % mod;
        }
        if ((n - __builtin_popcountll(s)) & 1)
        {
            prod = (mod - prod) % mod;
        }
        ans = ((__int128)ans + prod) % mod;
    }
    return n ? ans : 1 % mod;
}
```

`permanent(a,mod)` 要求方阵、$n \le 24$、`mod>0`，元素可为任意整数，返回 $\operatorname{perm}(A) \bmod mod$ 的非负值；空矩阵的积和式为 1。时间 $O(2^n n)$，额外空间 $O(n)$；中间量用 `__int128`，`mod` 可以取到 `LLONG_MAX`。

0/1 矩阵的积和式等于二分图完美匹配数；一般图完美匹配计数用 9.1.012 Hafnian 与一般图完美匹配计数，行列式见 9.1.006 行列式（质数模数 / 任意模数）。

```
signed main()
{
    vector<vector<int>> a{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    assert(permanent(a, 1000000007) == 450);
    assert(permanent(vector<vector<int>>(10, vector<int>(10, 1)), 1000000007) == 3628800); //
    ↪ 全 1 矩阵的积和式是 10!。
    assert(permanent({}, 7) == 1);
}
```

9.1.014 GF(2) 位集高斯消元

GF(2) 上的线性方程组 $Ax = b$ 中加法是异或；每行压成 64 位字的位集后，消去一行只需 $\lfloor \frac{n+64}{64} \rfloor$ 次字异或。按列找主元做 Gauss-Jordan 消元，得到简化阶梯形后：

- 某个全零系数行的右端为 1 时方程组无解；
- 自由元全取 0 时，主元变量等于所在行的右端，得到一组特解；
- 令某个自由元 f 为 1、其余自由元为 0，主元变量等于所在行第 f 位，得到齐次方程组的一个基向量。

解集是特解加上基向量张成的空间，共 $2^{n-\mathrm{rank}}$ 个解。

```
namespace gf2
{
using Row = vector<unsigned long long>; // Row 是压位的 0/1 行，第 i 位存在第 i/64 个字的第 i%64
↪ 位。

bool bit(const Row &r, int i)
{
    // 返回 r 的第 i 位。
    return r[i / 64] >> (i % 64) & 1;
}

void flip(Row &r, int i)
{
```

```cpp
    // 翻转 r 的第 i 位。
    r[i / 64] ^= 1ULL << (i % 64);
}

struct Result
{
    bool consistent; // consistent 表示方程组有解。
    int rank, free;  // rank 是系数矩阵的秩，free = n - rank 是自由元个数。
    Row solution;    // solution 是一组特解（自由元取 0），无解时无意义。
    vector<Row> basis; // basis 是齐次方程组解空间的一组基，共 free 个。
};

Result solve(vector<Row> a, int n)
{
    // a 是 GF(2) 上的增广矩阵，每行第 0..n-1 位是系数，第 n 位是右端，行长须为 (n+64)/64 个字；
    // 返回可解性、秩、特解与齐次解基。复杂度 O(m n^2 / 64)。
    int rank = 0, m = a.size();
    vector<int> where(n, -1); // where[c] 是以第 c 列为主元的行号，自由元为 -1。
    for (int c = 0; c < n; c++)
    {
        int p = rank;
        while (p < m && !bit(a[p], c))
        {
            p++;
        }
        if (p == m)
        {
            continue;
        }
        swap(a[p], a[rank]);
        where[c] = rank;
        for (int i = 0; i < m; i++)
        {
            if (i != rank && bit(a[i], c))
            {
                for (int j = 0; j < (int)a[i].size(); j++)
                {
                    a[i][j] ^= a[rank][j];
                }
            }
        }
        rank++;
    }
    Result res{true, rank, n - rank, Row((n + 63) / 64), {}};
    for (int i = rank; i < m; i++)
    {
        if (bit(a[i], n))
        {
            res.consistent = false;
            return res;
        }
    }
```

```
    // 消成简化阶梯形后，主元变量 = 右端 xor (该行上自由元的取值)。
    for (int c = 0; c < n; c++)
    {
        if (where[c] >= 0 && bit(a[where[c]], n))
        {
            flip(res.solution, c);
        }
    }
    for (int f = 0; f < n; f++)
    {
        if (where[f] < 0)
        {
            // 令自由元 f 为 1、其余自由元为 0，得到一个齐次解。
            Row v((n + 63) / 64);
            flip(v, f);
            for (int c = 0; c < n; c++)
            {
                if (where[c] >= 0 && bit(a[where[c]], f))
                {
                    flip(v, c);
                }
            }
            res.basis.push_back(move(v));
        }
    }
    return res;
}
}
```

`gf2::solve(a,n)` 中 `a` 是 m 行增广矩阵，每行的第 $0\dots n-1$ 位是系数、第 n 位是右端，行长为 $\lfloor\frac{n+64}{64}\rfloor$ 个字；`bit`、`flip` 读写单个位。返回的 `Result` 含可解性、秩、自由元个数、特解与齐次解基，特解和基向量的行长为 $\lceil\frac{n}{64}\rceil$ 个字。时间 $O(\frac{mn^2}{64})$，空间 $O(\frac{mn}{64})$。

一般模数下的消元见 9.1.005 矩阵求逆（质数模数 / 任意模数） 与 9.1.006 行列式（质数模数 / 任意模数）。下面解 $x_0\oplus x_1=1$、$x_1\oplus x_2=0$：

```
signed main()
{
    int n = 3;
    vector<gf2::Row> a(2, gf2::Row((n + 64) / 64));
    a[0][0] = 0b1011; // x0^x1=1，第 3 位是右端。
    a[1][0] = 0b0110; // 方程 x1^x2=0。
    auto r = gf2::solve(a, n);
    assert(r.consistent && r.rank == 2 && r.free == 1);
    assert(r.solution[0] == 0b001 && r.basis[0][0] == 0b111);
}
```

9.2 线性方程与规划

9.2.001 高斯消元：唯一解、无解、多解

在素数模域 $\mathbb{F}_p$ 上解 $Ax \equiv b$，并完整区分无解、唯一解与多解。Gauss-Jordan 消元逐列找非零主元、用主元逆元把该列化成单位列（主元行系数变 1、同列其他行清零），处理完得到行最简形。判定规则：若某行左侧系数全零而右端非零，方程 $0 \equiv c \neq 0$ 矛盾即无解；否则每个主元列的变量由该行右端唯一定出（特解），每个非主元列是自由变量，对应一条齐次解基——把该自由变量置 1、其余自由变量置 0，主元变量取相应负系数，就得到 $Ax \equiv 0$ 的一个基向量。通解即 “特解 + 齐次基的任意线性组合”。`basis` 为空即唯一解。

```cpp
// 依赖 7.1.001 快速幂
struct ModLinearSol
{
    // part 是一组特解，basis 是齐次解空间基；全部值在 [0,mod) 内。
    vector<int> part;
    vector<vector<int>> basis;
};

optional<ModLinearSol> gaussMod(vector<vector<int>> a, int n, int mod)
{
    // a 有 n+1 列且最后一列是右端，mod 是素数；返回特解与齐次基，无解时返回空。
    assert(n >= 0 && mod > 1); // 调试检查，可删。
    for (auto &r : a)
    {
        assert((int)r.size() == n + 1); // 调试检查，可删。
        for (auto &x : r)
        {
            x %= mod;
            if (x < 0)
            {
                x += mod;
            }
        }
    }
    int m = a.size();
    int rk = 0;
    vector<int> pivot(n, -1);
    auto power = [&](int x, int b)
    {
        // x 是非零剩余，b 是非负指数；返回 x^b mod mod。
        int ans = 1;
        while (b)
        {
            if (b & 1)
            {
                ans = mulMod(ans, x, mod);
            }
            x = mulMod(x, x, mod);
            b >>= 1;
```

```
        }
        return ans;
    };
    for (int col = 0; col < n && rk < m; col++)
    {
        int row = rk;
        while (row < m && a[row][col] == 0)
        {
            row++;
        }
        if (row == m)
        {
            continue;
        }
        swap(a[row], a[rk]);
        int inv = power(a[rk][col], mod - 2);
        for (int j = col; j <= n; j++)
        {
            a[rk][j] = mulMod(a[rk][j], inv, mod);
        }
        for (int i = 0; i < m; i++)
        {
            if (i == rk)
            {
                continue;
            }
            int q = a[i][col];
            for (int j = col; j <= n; j++)
            {
                a[i][j] -= mulMod(q, a[rk][j], mod);
                if (a[i][j] < 0)
                {
                    a[i][j] += mod;
                }
            }
        }
        pivot[col] = rk++;
    }
    for (int i = rk; i < m; i++)
    {
        if (a[i][n])
        {
            return nullopt;
        }
    }
    ModLinearSol sol{vector<int>(n), {}};
    for (int col = 0; col < n; col++)
    {
        if (pivot[col] != -1)
        {
            sol.part[col] = a[pivot[col]][n];
        }
```

```
    }
    for (int free = 0; free < n; free++)
    {
        if (pivot[free] != -1)
        {
            continue;
        }
        vector<int> v(n);
        v[free] = 1;
        for (int col = 0; col < n; col++)
        {
            if (pivot[col] != -1)
            {
                v[col] = (mod - a[pivot[col]][free]) % mod;
            }
        }
        sol.basis.push_back(move(v));
    }
    return sol;
}
```

调用 `gaussMod(a,n,mod)`，`a` 是 m 行 $n+1$ 列增广矩阵（最后一列是右端），`mod` 必须是素数；显式传 `n` 是为了让 `m=0` 时仍能表示变量数。返回空即无解，`sol.basis.empty()` 即唯一解，齐次基数量恒为 `n-rk`。零条方程会返回全零特解和 n 个单位基向量；零个变量时，每个合法方程只剩右端，全部为零才相容。连同输入归一化和齐次基输出，时间为 $O(m(n+1)+mn\min(m,n)+n^2)$，空间为 $O(m(n+1)+n^2)$。实数方程用 9.2.002 高斯消元：列主元 或 9.2.003 高斯消元最小范数解；异或方程参见 9.2.004 线性基（参见数据结构）；三对角结构用 9.2.007 追赶法解三对角/周期方程。

下面用五个固定系统覆盖唯一解、多解、无解、零方程和零变量，并直接核对齐次基维数：

```
signed main()
{
    auto one = gaussMod({{1, 1, 3}, {1, -1, 1}}, 2, 5);
    assert(one && one->part == vector<int>({2, 1}) && one->basis.empty());
    auto many = gaussMod({{1, 1, 1}}, 2, 5);
    assert(many && many->basis.size() == 1);
    assert(many->basis[0] == vector<int>({4, 1}));
    assert(!gaussMod({{1, 1}, {1, 2}}, 1, 5));
    auto none = gaussMod({}, 3, 5);
    assert(none && none->part == vector<int>(3, 0) && none->basis.size() == 3);
    assert(gaussMod({{0}}, 0, 5) && !gaussMod({{1}}, 0, 5));
    cout << one->part[0] << ' ' << one->part[1] << ' ' << none->basis.size() << '\n'; // 期望输
    ↪ 出: 2 1 3
    return 0;
}
```

异或方程组（$\mathbb{F}_2$）直接用 9.1.014 GF(2) 位集高斯消元 的 `gf2::solve`，它同时给出特解与齐次解基。三元域专用接口 `solveMod3(mat,vars)` 用两个机器位集记录系数 $1,2$，返回任意一组解，无解返回空。

三元域位并行实现如下：

```cpp
struct Mod3Row
{
    // n 是列数；one、two 分别记录系数为 1、2 的列，未记录列系数为 0。
    int n = 0;
    vector<unsigned long long> one, two;

    Mod3Row(int n = 0) : n(n), one((n + 63) / 64), two((n + 63) / 64) {}

    int get(int i) const
    {
        // i 是列下标；返回该列在 F3 中的系数。
        return one[i >> 6] >> (i & 63) & 1ULL ? 1 :
               (two[i >> 6] >> (i & 63) & 1ULL ? 2 : 0);
    }

    void set(int i, int x)
    {
        // i 是列下标，x 是 0、1 或 2；设置该列系数。
        unsigned long long b = 1ULL << (i & 63);
        one[i >> 6] &= ~b, two[i >> 6] &= ~b;
        if (x == 1) one[i >> 6] |= b;
        if (x == 2) two[i >> 6] |= b;
    }

    void add(const Mod3Row &b)
    {
        // b 是同宽行；把 b 加到当前行，系数按模 3 运算。
        assert(n == b.n);
        for (int i = 0; i < (int)one.size(); i++)
        {
            auto az = ~(one[i] | two[i]), bz = ~(b.one[i] | b.two[i]);
            auto o = (one[i] & bz) | (two[i] & b.two[i]) | (az & b.one[i]);
            auto t = (one[i] & b.one[i]) | (two[i] & bz) | (az & b.two[i]);
            one[i] = o, two[i] = t;
        }
    }

    void sub(const Mod3Row &b)
    {
        // b 是同宽行；从当前行减去 b，系数按模 3 运算。
        assert(n == b.n);
        for (int i = 0; i < (int)one.size(); i++)
        {
            auto az = ~(one[i] | two[i]), bz = ~(b.one[i] | b.two[i]);
            auto o = (one[i] & bz) | (two[i] & b.one[i]) | (az & b.two[i]);
            auto t = (one[i] & b.two[i]) | (two[i] & bz) | (az & b.one[i]);
            one[i] = o, two[i] = t;
        }
    }
};

inline optional<vector<int>> solveMod3(const vector<vector<int>> &mat, int vars)
```

```
{
    // mat 每行含 vars 个系数和一个右端，vars 是变量数；返回任意一组 F3 解，无解时返回空。
    assert(vars >= 0); // 调试检查，可删
    int rows = mat.size(), rk = 0;
    vector<Mod3Row> a(rows, Mod3Row(vars + 1));
    for (int i = 0; i < rows; i++)
    {
        assert((int)mat[i].size() == vars + 1); // 调试检查，可删
        for (int j = 0; j <= vars; j++)
        {
            int x = mat[i][j] % 3;
            if (x < 0) x += 3;
            a[i].set(j, x);
        }
    }
    vector<int> pivot(vars, -1);
    for (int col = 0; col < vars && rk < rows; col++)
    {
        int p = rk;
        while (p < rows && !a[p].get(col))
        {
            p++;
        }
        if (p == rows) continue;
        swap(a[p], a[rk]);
        int lead = a[rk].get(col);
        for (int i = 0; i < rows; i++)
        {
            if (i == rk || !a[i].get(col)) continue;
            if (a[i].get(col) == lead) a[i].sub(a[rk]);
            else a[i].add(a[rk]);
        }
        pivot[col] = rk++;
    }
    for (int i = rk; i < rows; i++)
    {
        if (a[i].get(vars)) return nullopt;
    }
    vector<int> ans(vars);
    for (int col = 0; col < vars; col++)
    {
        if (pivot[col] != -1)
ans[col] = a[pivot[col]].get(col) * a[pivot[col]].get(vars) % 3;
    }
    return ans;
}
```

`solveMod3(mat,vars)` 中每行前 `vars` 个数是系数、最后一个是右端，任意整数先归一到 $\{0,1,2\}$；自由变量取 0，返回任意一组 $\mathbb{F}_3$ 解，无解返回空。每个系数用两个位集按位存储，消去一行只需常数次字运算，时间为 $O\left(\frac{nm\cdot\min(n,m)}{w}\right)$（$n$ 为方程数、m 为变量数、w 为机器字位数），空间 $O\left(\frac{nm}{w}\right)$。

9.2.002 高斯消元：列主元

浮点消元里若拿一个绝对值极小的数当主元（除数），舍入误差会被放大很多倍，最终解可能面目全非。列主元（部分选主元）法在当前列的候选行中挑绝对值最大者交换到主元位，让除数尽量大、误差最小。这里用它求浮点线性方程的唯一解：消元后统计秩，秩等于变量数才有唯一解并回代；否则（出现矛盾行或秩不足）返回空，把多解、最小范数解交给专门接口处理。判零阈值取公共前导里的 `eps`。

```
optional<vector<long double>> gaussPivot(vector<vector<long double>> a, int n)
{
    // a 有 n+1 列且最后一列是右端；唯一解时返回解，否则返回空。
    assert(n >= 0); // 调试检查，可删。
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n + 1);
    }
    int m = a.size();
    int rk = 0;
    vector<int> pivot(n, -1);
    for (int col = 0; col < n && rk < m; col++)
    {
        int row = rk;
        for (int i = rk + 1; i < m; i++)
        {
            if (fabsl(a[i][col]) > fabsl(a[row][col]))
            {
                row = i;
            }
        }
        if (fabsl(a[row][col]) <= eps)
        {
            continue;
        }
        swap(a[row], a[rk]);
        for (int j = n; j >= col; j--)
        {
            a[rk][j] /= a[rk][col];
        }
        for (int i = 0; i < m; i++)
        {
            if (i == rk)
            {
                continue;
            }
            long double q = a[i][col];
            for (int j = col; j <= n; j++)
            {
                a[i][j] -= q * a[rk][j];
            }
        }
        pivot[col] = rk++;
```

```
    }
    for (int i = rk; i < m; i++)
    {
        if (fabsl(a[i][n]) > eps)
        {
            return nullopt;
        }
    }
    if (rk < n)
    {
        return nullopt;
    }
    vector<long double> x(n);
    for (int col = 0; col < n; col++)
    {
        x[col] = a[pivot[col]][n];
    }
    return x;
}
```

调用 `gaussPivot(a,n)`，`a` 是 m 行 $n+1$ 列的增广矩阵；显式变量数使空方程组也有确定含义：$m=n=0$ 有唯一空解，$m=0<n$ 时是多解。函数只用于求浮点唯一解，返回空可能是无解也可能是多解。连同输入检查，时间为 $O(m(n+1)+mn\min(m,n))$，空间为 $O(m(n+1))$。素数模方程用 9.2.001 高斯消元：唯一解、无解、多解；相容多解的最小范数解用 9.2.003 高斯消元最小范数解；三对角结构用 9.2.007 追赶法解三对角/周期方程。

下面固定检查唯一解、需要换行的主元、多解、不相容与空系统：

```
signed main()
{
    auto x = gaussPivot({{0, 1, 2}, {1, 1, 3}}, 2);
    assert(x && fabsl((*x)[0] - 1) < 1E-10L && fabsl((*x)[1] - 2) < 1E-10L);
    assert(!gaussPivot({{1, 1, 2}}, 2));
    assert(!gaussPivot({{1, 1}, {1, 2}}, 1));
    assert(gaussPivot({}, 0) && !gaussPivot({}, 2));
    cout << llround((*x)[0]) << ' ' << llround((*x)[1]) << '\n'; // 期望输出：1 2
    return 0;
}
```

9.2.003 高斯消元最小范数解

当实数约束 $Ax=b$ 相容但欠定（解不唯一）时，把自由变量随手置零得到的解依赖消元顺序、没有几何意义。这里求的是欧氏范数 $\|x\|_2$ 最小的那个唯一解。它是带约束极小化 $\min\|x\|^2$ s.t. $Ax=b$，拉格朗日 KKT 条件给出 $x+A^T\lambda=0$、$Ax=b$，把 x 与乘子 λ 拼成未知量就是一个对称增广线性系统 $\begin{pmatrix} I & A^T \\ A & 0 \end{pmatrix}\begin{pmatrix} x \\ \lambda \end{pmatrix}=\begin{pmatrix} 0 \\ b \end{pmatrix}$。用列主元消元解这个 $(n+m)$ 维系统，约束相关只让 λ 不唯一，x 部分仍唯一确定，取出前 n 维即最小范数解。

```cpp
optional<vector<long double>> minNormSolve(const vector<vector<long double>> &a, const vector<long double> &b, int n)
{
    // a 是 m*n 系数矩阵，b 是 m 维右端；返回 Ax=b 的最小二范数解，不相容时返回空。
    int m = a.size();
    int s = n + m;
    assert(n >= 0); // 调试检查，可删。
    assert((int)b.size() == m); // 调试检查，可删。
    // 调试检查，可删。
    for (const auto &r : a)
    {
        assert((int)r.size() == n);
    }
    vector mat(s, vector<long double>(s + 1));
    for (int j = 0; j < n; j++)
    {
        mat[j][j] = 1;
        for (int i = 0; i < m; i++)
        {
            mat[j][n + i] = a[i][j];
        }
    }
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            mat[n + i][j] = a[i][j];
        }
        mat[n + i][s] = b[i];
    }
    int rk = 0;
    vector<int> pivot(s, -1);
    for (int col = 0; col < s && rk < s; col++)
    {
        int row = rk;
        for (int i = rk + 1; i < s; i++)
        {
            if (fabsl(mat[i][col]) > fabsl(mat[row][col]))
            {
                row = i;
            }
        }
        if (fabsl(mat[row][col]) <= eps)
        {
            continue;
        }
        swap(mat[row], mat[rk]);
        for (int j = s; j >= col; j--)
        {
            mat[rk][j] /= mat[rk][col];
        }
        for (int i = 0; i < s; i++)
```

```
        {
            if (i == rk)
            {
                continue;
            }
            long double q = mat[i][col];
            for (int j = col; j <= s; j++)
            {
                mat[i][j] -= q * mat[rk][j];
            }
        }
        pivot[col] = rk++;
    }
    for (int i = rk; i < s; i++)
    {
        if (fabsl(mat[i][s]) > 1E-10L)
        {
            return nullopt;
        }
    }
    vector<long double> z(s);
    for (int col = 0; col < s; col++)
    {
        if (pivot[col] != -1)
        {
            z[col] = mat[pivot[col]][s];
        }
    }
    z.resize(n);
    return z;
}
```

调用 `minNormSolve(a,b,n)`，`a` 各行长度为 `n`、`b` 长度等于行数，返回空表示约束不相容。显式传 `n` 后，零条约束会正确返回 n 维零向量。单个方程 $x+y=1$ 的最小范数解是 $(0.5,0.5)$，重复约束不改变结果。复杂度为 $O((m+n)^3)$，空间为 $O((m+n)^2)$。这里解的是 KKT 增广系统，不是正规方程；它不会把条件数直接平方，但矩阵不定且在冗余约束下奇异，病态问题仍应换 QR 或 SVD。唯一解用 9.2.002 高斯消元：列主元 更省；素数模多解结构用 9.2.001 高斯消元：唯一解、无解、多解 的齐次基表示。

下面检查单约束、重复约束、不相容约束和零约束：

```
signed main()
{
    auto x = minNormSolve({{1, 1}}, {1}, 2);
    assert(x && fabsl((*x)[0] - 0.5L) < 1E-10L && fabsl((*x)[1] - 0.5L) < 1E-10L);
    auto y = minNormSolve({{1, 1}, {1, 1}}, {1, 1}, 2);
    assert(y && fabsl((*y)[0] - 0.5L) < 1E-10L && fabsl((*y)[1] - 0.5L) < 1E-10L);
    assert(!minNormSolve({{1, 1}, {1, 1}}, {1, 2}, 2));
    auto z = minNormSolve({}, {}, 2);
    assert(z && *z == vector<long double>({0, 0}));
    cout << fixed << setprecision(1) << (*x)[0] << ' ' << (*x)[1] << '\n'; // 期望输出：0.5 0.5
    return 0;
```

```
}
```

9.2.004 线性基（参见数据结构）

整数按二进制看成 $\mathbb{F}_2$（二元域）上的向量，异或就是向量加法，一组数的所有异或子集和正是这些向量张成的线性子空间。线性基按最高位存一组极大无关向量：插入 x 时从高位向低位消元，若最终在空档留下非零主元则秩加一，否则它可由现有基表示。`has` 判可表示性，`maxXor` 贪心得最大异或值，`rk` 是秩，`kth` 在化简后的基上取第 k 小可表示值。本条只给出线性代数视角的索引，代表实现与增强接口统一放在数据结构章，避免同一代码维护两份。

本条不重复打印代码；请直接使用 3.8.001 线性基的代表实现。模板参数 `W` 是位宽，`add(x)` 返回是否使秩增加，`has(x)`、`maxXor(x)`、`kth(k)`、`merge(o)` 的语义和边界检查均以该条为准。`add`、`has`、`maxXor` 均 $O(W)$，`kth` 需一次 $O(W^2)$ 化简，空间 $O(W)$。解异或方程组也使用同一按位消元思路；系数不在 $\mathbb{F}_2$ 而在素数模域时改用 9.2.001 高斯消元：唯一解、无解、多解。

9.2.005 Simplex 单纯形

线性规划求 $\max c^T x$ 使 $Ax \le b$、$x \ge 0$。可行域是若干半空间的交；若有限最优值存在，就能在某个顶点取到。单纯形维护一个基可行解，每轮选一个可改进目标的非基变量入基，再按最小比值法则选基变量出基。退化换基时目标值可以不变，直到没有可改进方向才得到最优解。初始基未必可行，故先引入一个人工变量，第一阶段把它压到 0 以找到可行基，再在第二阶段优化原目标。三种结局必须分清——不可行（第一阶段无法把人工变量压到 0）、无界（存在改进方向却没有正系数约束它）、最优。本实现按 Bland 规则选编号最小的可改进入基变量，最小比值相等时选编号最小的离基变量；精确算术下这能避免退化循环，浮点版仍由成员容差 `e` 决定何时视为相等。

```
struct Simplex
{
    // m 是约束数，n 是变量数。
    int m, n;
    // b、z 分别记录各行的基变量和各列的非基变量编号。
    vector<int> b, z;
    // a 是单纯形表。
    vector<vector<long double>> a;
    // e 是浮点比较容差，linf 标记无界或不可行。
    static constexpr long double e = 1E-12L;
    static constexpr long double linf = numeric_limits<long double>::infinity();

    Simplex(const vector<vector<long double>> &A, const vector<long double> &B, const
    ↪ vector<long double> &C)
        : m(B.size()), n(C.size()), b(m), z(n + 1), a(m + 2, vector<long double>(n + 2))
    {
        // A、B 给出 Ax<=B，C 是最大化目标系数；隐含 x>=0。
        assert((int)A.size() == m); // 调试检查，可删。
        for (int i = 0; i < m; i++)
        {
            assert((int)A[i].size() == n); // 调试检查，可删。
            for (int j = 0; j < n; j++)
```

```
            {
                a[i][j] = A[i][j];
            }
        }
        for (int i = 0; i < m; i++)
        {
            b[i] = n + i;
            a[i][n] = -1;
            a[i][n + 1] = B[i];
        }
        for (int j = 0; j < n; j++)
        {
            z[j] = j;
            a[m][j] = -C[j];
        }
        z[n] = -1;
        a[m + 1][n] = 1;
    }

private:
    void pivot(int r, int s)
    {
        // r 是离基变量所在行，s 是入基变量所在列；用 a[r][s] 换基并更新整张表。
        long double v = 1 / a[r][s];
        for (int i = 0; i < m + 2; i++)
        {
            if (i != r)
            {
                for (int j = 0; j < n + 2; j++)
                {
                    if (j != s)
                    {
                        a[i][j] -= a[r][j] * a[i][s] * v;
                    }
                }
            }
        }
        for (int j = 0; j < n + 2; j++)
        {
            if (j != s)
            {
                a[r][j] *= v;
            }
        }
        for (int i = 0; i < m + 2; i++)
        {
            if (i != r)
            {
                a[i][s] *= -v;
            }
        }
        a[r][s] = v;
```

```
        swap(b[r], z[s]);
    }

    bool phase(int p)
    {
        // p=1 时找可行基，p=2 时优化原目标；返回 false 表示存在无界改进方向。
        int x = p == 1 ? m + 1 : m;
        while (true)
        {
            int s = -1;
            for (int j = 0; j <= n; j++)
            {
                if (z[j] == -1 || a[x][j] >= -e)
                {
                    continue;
                }
                if (s < 0 || z[j] < z[s])
                {
                    s = j;
                }
            }
            if (s < 0)
            {
                return true;
            }
            int r = -1;
            for (int i = 0; i < m; i++)
            {
                if (a[i][s] <= e)
                {
                    continue;
                }
                if (r < 0 || a[i][n + 1] / a[i][s] < a[r][n + 1] / a[r][s] - e ||
                    (fabsl(a[i][n + 1] / a[i][s] - a[r][n + 1] / a[r][s]) <= e && b[i] < b[r]))
                {
                    r = i;
                }
            }
            if (r < 0)
            {
                return false;
            }
            pivot(r, s);
        }
    }

public:
    pair<long double, vector<long double>> solve()
    {
        // 返回最优值与变量；不可行返回 -linf，无界返回 linf，二者的解向量均为空。
        if (m == 0)
        {
```

```
            for (int j = 0; j < n; j++)
            {
                if (a[m][j] < -e)
                {
                    return {linf, {}};
                }
            }
            return {0, vector<long double>(n)};
        }
        int r = min_element(a.begin(),
                            a.begin() + m,
                            [&](auto &x, auto &y)
                            {
                                return x[n + 1] < y[n + 1];
                            }) -
                a.begin();
        if (a[r][n + 1] < -e)
        {
            pivot(r, n);
            if (!phase(1) || a[m + 1][n + 1] < -e)
            {
                return {-linf, {}};
            }
            if (fabsl(a[m + 1][n + 1]) > e)
            {
                return {-linf, {}};
            }
            auto it = find(b.begin(), b.end(), -1);
            if (it != b.end())
            {
                r = it - b.begin();
                int s = 0;
                for (int j = 1; j <= n; j++)
                {
                    if (fabsl(a[r][j]) > fabsl(a[r][s]))
                    {
                        s = j;
                    }
                }
                if (fabsl(a[r][s]) > e)
                {
                    pivot(r, s);
                }
            }
        }
        if (!phase(2))
        {
            return {linf, {}};
        }
        vector<long double> x(n);
        for (int i = 0; i < m; i++)
        {
```

```
            if (b[i] < n)
            {
                x[b[i]] = a[i][n + 1];
            }
        }
        return {a[m][n + 1], x};
    }
};
```

以 `Simplex(A,B,C)` 构造（A 是 $m \times n$ 约束系数、B 是 m 维右端、C 是 n 维目标系数，隐含 $x \geq 0$），`solve()` 返回最优值与解向量：不可行时值为 $-\infty$、无界时为 $+\infty$，均配空向量。每次换基为 $O(mn)$，最坏换基次数是指数级，空间为 $O(mn)$。等式约束要拆成两个不等式；纯粹解方程用 9.2.001 高斯消元：唯一解、无解、多解 或 9.2.002 高斯消元：列主元；分式目标的恰选问题用 9.2.006 0-1 分数规划。

下面固定检查普通最优、第一阶段、不可行、无界，再用经典退化循环例检查 Bland 选元：

```
signed main()
{
    auto [val, x] = Simplex({{1, 1}, {1, 0}, {0, 1}}, {4, 2, 3}, {3, 2}).solve();
    assert(fabsl(val - 10) < 1E-10L && x.size() == 2);
    auto [phaseOne, y] = Simplex({{-1}}, {-1}, {-1}).solve();
    assert(fabsl(phaseOne + 1) < 1E-10L && y.size() == 1);
    auto [infeasible, p] = Simplex({{1}, {-1}}, {0, -1}, {1}).solve();
    assert(isinf(infeasible) && infeasible < 0 && p.empty());
    auto [unbounded, q] = Simplex({}, {}, {1}).solve();
    assert(isinf(unbounded) && unbounded > 0 && q.empty());
    auto [degenerate, z] = Simplex({{0.5L, -5.5L, -2.5L, 9},
                                    {0.5L, -1.5L, -0.5L, 1},
                                    {1, 0, 0, 0}},
                                   {0, 0, 1}, {10, -57, -9, -24})
                               .solve();
    assert(fabsl(degenerate - 1) < 1E-10L && z.size() == 4);
    cout << llround(val) << ' ' << llround(phaseOne) << '\n'; // 期望输出：10 -1
    return 0;
}
```

9.2.006 0-1 分数规划

要在若干物品里恰选 k 个，最大化比值 $\dfrac{\sum a_i}{\sum b_i}$（$b_i > 0$）。分式不能直接贪心，但可以参数化：比值 $\geq r$ 当且仅当 $\sum a_i - r\sum b_i = \sum(a_i - rb_i) \geq 0$。固定 r 后 $a_i - rb_i$ 就是每个物品的独立权重，取最大的 k 个之和判它是否非负即可——这个判定关于 r 单调，于是对 r 二分。所有分母 b_i 必须为正，否则单调性不成立。（另一路线是 Dinkelbach 迭代，收敛更快但实现更绕，这里用固定轮数二分足够。）

```
namespace frac_prog
{
long double maxRatio(const vector<long double> &a, const vector<long double> &b, int k)
{
    // a、b 是分子分母贡献且 b[i] 为正，k 是恰选数量；返回最大比值。
    int n = a.size();
```

```
    assert((int)b.size() == n && 1 <= k && k <= n); // 调试检查，可删。
    long double l = numeric_limits<long double>::infinity();
    long double r = -l;
    for (int i = 0; i < n; i++)
    {
        assert(b[i] > 0); // 调试检查，可删。
        l = min(l, a[i] / b[i]);
        r = max(r, a[i] / b[i]);
    }
    vector<long double> c(n);
    for (int it = 0; it < 100; it++)
    {
        long double mid = (l + r) / 2;
        for (int i = 0; i < n; i++)
        {
            c[i] = a[i] - mid * b[i];
        }
        nth_element(c.begin(), c.end() - k, c.end());
        long double s = accumulate(c.end() - k, c.end(), 0.0L);
        if (s >= 0)
        {
            l = mid;
        }
        else
        {
            r = mid;
        }
    }
    return (l + r) / 2;
}
}
```

调用 `frac_prog::maxRatio(a,b,k)`，`a`、`b` 是各物品分子分母贡献（`b[i]>0`）、`k` 是恰选数量，返回最大比值。上下界取所有单项比值的最小值与最大值，因此最优值为负时也正确。固定 100 轮二分，每轮用 `nth_element` 在线性期望时间取前 k 大，总时间为 $O(n)$ 期望、空间为 $O(n)$。复杂可行性约束可把内层判定换成对应最优化；连续线性规划用 9.2.005 Simplex 单纯形。

下面用负最优值和普通正值各检查一次：

```
signed main()
{
    long double x = frac_prog::maxRatio({-5, -2, -9}, {1, 1, 1}, 2);
    long double y = frac_prog::maxRatio({1, 8}, {1, 2}, 1);
    assert(fabsl(x + 3.5L) < 1E-10L && fabsl(y - 4) < 1E-10L);
    cout << fixed << setprecision(1) << x << ' ' << y << '\n'; // 期望输出：-3.5 4.0
    return 0;
}
```

9.2.007 追赶法解三对角/周期方程

三对角矩阵每行只有主对角 b_i 与相邻的下对角 a_i、上对角 c_i 三个非零元。对它做一般高斯消元是 $O(n^3)$ 的浪费，因为消去下对角后不会产生新的填充。追赶法（Thomas 算法）就是这个专门化：正向“追”一遍，用 a_i/b_{i-1} 把每行下对角消掉、同步更新主对角与右端；再反向“赶”一遍回代求解，总共 $O(n)$。周期三对角在左下角 $A_{n-1,0}$ 与右上角 $A_{0,n-1}$ 多两个耦合元素，破坏了带状结构；这里用 Sherman-Morrison 把周期矩阵写成“三对角 + 秩一修正”，解两次普通三对角方程（一次对原右端、一次对修正向量）再线性组合得到答案。所有主元 b_{i-1} 消元时必须非零可除（对角占优时天然成立）。

```cpp
template <class T> optional<vector<T>> tridiagonal(vector<T> a, vector<T> b, vector<T> c,
    vector<T> d)
{
    // a、b、c 是下、主、上对角线，d 是右端；零主元时返回空。
    int n = b.size();
    assert((int)a.size() == n && (int)c.size() == n && (int)d.size() == n); // 调试检查，可删。
    if (n == 0)
    {
        return vector<T>{};
    }
    for (int i = 1; i < n; i++)
    {
        if (b[i - 1] == T(0))
        {
            return nullopt;
        }
        T q = a[i] / b[i - 1];
        b[i] -= q * c[i - 1];
        d[i] -= q * d[i - 1];
    }
    if (b[n - 1] == T(0))
    {
        return nullopt;
    }
    vector<T> x(n);
    x[n - 1] = d[n - 1] / b[n - 1];
    for (int i = n - 2; i >= 0; i--)
    {
        if (b[i] == T(0))
        {
            return nullopt;
        }
        x[i] = (d[i] - c[i] * x[i + 1]) / b[i];
    }
    return x;
}

template <class T>
optional<vector<T>> cyclicTri(vector<T> a, vector<T> b, vector<T> c, T tr, T bl, const
    vector<T> &d)
{
    // tr、bl 是右上角与左下角系数；追赶法或秩一修正失败时返回空。
```

```
    int n = b.size();
    assert(n >= 3); // 调试检查，可删。
    assert((int)a.size() == n && (int)c.size() == n && (int)d.size() == n); // 调试检查，可删。
    T g = b[0] == T(0) ? T(1) : -b[0];
    b[0] -= g;
    b[n - 1] -= tr * bl / g;
    vector<T> u(n);
    u[0] = g;
    u[n - 1] = bl;
    auto x = tridiagonal(a, b, c, d);
    auto z = tridiagonal(a, b, c, u);
    if (!x || !z)
    {
        return nullopt;
    }
    T den = T(1) + (*z)[0] + tr * (*z)[n - 1] / g;
    if (den == T(0))
    {
        return nullopt;
    }
    T q = ((*x)[0] + tr * (*x)[n - 1] / g) / den;
    for (int i = 0; i < n; i++)
    {
        (*x)[i] -= q * (*z)[i];
    }
    return x;
}
```

普通三对角方程调用 `tridiagonal(a,b,c,d)`，约定 `a[0]`、`c[n-1]` 不使用；空系统返回一个存在但长度为零的解。周期方程调用 `cyclicTri(a,b,c,tr,bl,d)`，要求 $n \geq 3$。两者返回 `nullopt` 表示无主元交换的追赶过程失败，并不等价于原矩阵一定奇异。`T` 必须提供域上的除法；若 `T` 是整数，则每一步除法都必须整除，否则应改用有理数或模整数。两者均为 $O(n)$ 时间、$O(n)$ 空间。一般矩阵用 9.2.002 高斯消元：列主元；模意义下用 9.2.001 高斯消元：唯一解、无解、多解。

下面检查空系统、普通三对角、周期三对角与非零行列式但首主元为零的失败分支：

```
signed main()
{
    auto empty = tridiagonal<long double>({}, {}, {}, {});
    assert(empty && empty->empty());
    auto x = tridiagonal<long double>({0, -1, -1}, {2, 2, 2}, {-1, -1, 0}, {1, 0, 1});
    assert(x);
    for (auto v : *x)
    {
        assert(fabsl(v - 1) < 1E-10L);
    }
    auto y = cyclicTri<long double>({0, -1, -1, -1}, {4, 4, 4, 4},
                                    {-1, -1, -1, 0}, -1, -1, {2, 2, 2, 2});
    assert(y);
    for (auto v : *y)
    {
```

```
        assert(fabsl(v - 1) < 1E-10L);
    }
    assert(!tridiagonal<long double>({0, 1}, {0, 1}, {1, 0}, {1, 2}));
}
```

9.2.008 Pfaffian

偶数阶斜对称矩阵 $A^T = -A$ 的 Pfaffian 满足 $\mathrm{pf}(A)^2 = \det(A)$。对前两个下标选非零配对主元后，其余部分做一次斜对称 Schur 补，Pfaffian 就乘上该主元；若交换一个下标到主元位置，必须同步交换对应行列并让答案变号。重复成对消元即可在 $O(n^3)$ 内求值，不需要先算行列式再开平方。

```
template <class T> T pfaffian(vector<vector<T>> a)
{
    // a 是偶数阶斜对称矩阵，T 是域；返回 Pfaffian。
    int n = a.size();
    assert(n % 2 == 0); // 调试检查，可删。
    // 调试检查，可删。
    for (int i = 0; i < n; i++)
    {
        assert((int)a[i].size() == n && a[i][i] == T(0));
        for (int j = 0; j < n; j++)
        {
            assert(a[i][j] == -a[j][i]);
        }
    }
    T ans = 1;
    for (int k = 0; k < n; k += 2)
    {
        int p = k + 1;
        while (p < n && a[k][p] == T(0))
        {
            p++;
        }
        if (p == n)
        {
            return T(0);
        }
        if (p != k + 1)
        {
            swap(a[p], a[k + 1]);
            for (int i = 0; i < n; i++)
            {
                swap(a[i][p], a[i][k + 1]);
            }
            ans = -ans;
        }
        T v = a[k][k + 1];
        ans *= v;
        for (int i = k + 2; i < n; i++)
        {
```

```
            for (int j = i + 1; j < n; j++)
            {
                a[i][j] -= (a[k][i] * a[k + 1][j] - a[k][j] * a[k + 1][i]) / v;
                a[j][i] = -a[i][j];
            }
        }
    }
    return ans;
}
```

`pfaffian(a)` 要求偶数阶斜对称矩阵，`T` 必须是可除的域；模意义下通常使用奇素数模的模整数。空矩阵返回空乘积 1。时间为 $O(n^3)$，按值传入的矩阵副本占 $O(n^2)$ 空间，除此之外只需 $O(1)$ 工作空间。Pfaffian 是带符号的配对和；不带符号的一般图完美匹配计数应使用 9.1.012 Hafnian 与一般图完美匹配计数，二者不能互换。

下面用 4×4 闭式 $a_{01}a_{23}-a_{02}a_{13}+a_{03}a_{12}$ 检查普通主元与交换主元：

```
signed main()
{
    vector<vector<long double>> a{{0, 1, 2, 3}, {-1, 0, 4, 5}, {-2, -4, 0, 6}, {-3, -5, -6, 0}};
    vector<vector<long double>> b{{0, 0, 2, 3}, {0, 0, 4, 5}, {-2, -4, 0, 6}, {-3, -5, -6, 0}};
    assert(fabsl(pfaffian(vector<vector<long double>>{}) - 1) < 1E-10L);
    assert(fabsl(pfaffian(a) - 8) < 1E-10L);
    cout << llround(pfaffian(b)) << '\n'; // 期望输出：2
    return 0;
}
```

9.2.009 对称矩阵 LDL^T 分解

对称矩阵在不交换主元且各步枢轴非零时可分解为 $A=LDL^T$，其中 L 是单位下三角矩阵、$D=\operatorname{diag}(d_0,\dots,d_{n-1})$。比较等式两侧第 (j,j) 与 (i,j) 项可依次得到

$$d_j=a_{jj}-\sum_{k<j}l_{jk}^2d_k,$$
$$l_{ij}=\frac{a_{ij}-\sum_{k<j}l_{ik}d_kl_{jk}}{d_j}\qquad (i>j),$$

因此按列递推即可原地确定全部因子。解 $Ax=b$ 时依次做单位下三角前代 $Ly=b$、对角求解 $Dz=y$ 和单位上三角回代 $L^Tx=z$。与 Cholesky 不同，这个分解允许 D 中出现负数，因而能处理一部分不定矩阵；但当前版本不做对称主元交换，某个 d_j 为零时即使 A 可逆也会失败。

```
struct LDLT
{
    // l 是单位下三角因子，满足 A=l*diag(d)*l^T；只使用其下三角部分。
    vector<vector<long double>> l;
    // d 是对角因子；每个元素都必须远离零枢轴。
    vector<long double> d;
    // eps 是判断对称误差和零枢轴的绝对容差。
```

```
    long double eps;
    // valid 表示构造时是否成功完成无主元交换的分解。
    bool valid;

    explicit LDLT(const vector<vector<long double>> &a, long double tol = 1E-12L)
        : l(a.size(), vector<long double>(a.size())), d(a.size()), eps(tol), valid(true)
    {
        // a 是方阵且应对称，tol 是非负的对称性与枢轴判定容差；保存 A 的 LDL^T 分解。
        assert(tol >= 0); // 调试检查，可删。
        int n = (int)a.size();
        for (int i = 0; i < n; i++)
        {
            assert((int)a[i].size() == n); // 调试检查，可删。
        }
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                assert(fabsl(a[i][j] - a[j][i]) <= eps); // 调试检查，可删。
            }
            l[i][i] = 1;
        }
        for (int j = 0; j < n && valid; j++)
        {
            long double diag = a[j][j];
            for (int k = 0; k < j; k++)
            {
                diag -= l[j][k] * l[j][k] * d[k];
            }
            if (fabsl(diag) <= eps)
            {
                valid = false;
                break;
            }
            d[j] = diag;
            for (int i = j + 1; i < n; i++)
            {
                long double value = a[i][j];
                for (int k = 0; k < j; k++)
                {
                    value -= l[i][k] * d[k] * l[j][k];
                }
                l[i][j] = value / d[j];
            }
        }
    }

    optional<vector<long double>> solve(const vector<long double> &b) const
    {
        // b 是长度等于矩阵阶数的右端向量；返回 Ax=b 的解，无效分解时返回空。
        assert((int)b.size() == (int)d.size()); // 调试检查，可删。
        if (!valid)
```

```
        {
            return nullopt;
        }
        int n = (int)d.size();
        vector<long double> y = b;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < i; j++)
            {
                y[i] -= l[i][j] * y[j];
            }
        }
        vector<long double> z(n);
        for (int i = 0; i < n; i++)
        {
            z[i] = y[i] / d[i];
        }
        vector<long double> x = z;
        for (int i = n - 1; i >= 0; i--)
        {
            for (int j = i + 1; j < n; j++)
            {
                x[i] -= l[j][i] * x[j];
            }
        }
        return x;
    }
};
```

构造 `LDLT fact(a,e)` 后先检查 `fact.valid`，再可对多个右端重复调用 `fact.solve(b)`；输入必须是 $n\times n$ 对称矩阵，`e` 是同时用于对称性断言与零枢轴判断的绝对容差，调用者应按数据尺度选取。空矩阵有唯一空分解和空解；失败只说明当前顺序下无主元分解不可用，不等价于矩阵奇异，此时改用 9.2.002 高斯消元：列主元，病态对称系统则应使用带对称主元交换的 LDL^T 或 QR。分解耗时 $O(n^3)$、空间 $O(n^2)$，每个右端的前代与回代耗时 $O(n^2)$、额外空间 $O(n)$。

下面用一个 D 含负数的不定矩阵核对解 $(1,-2,3)$，并检查空矩阵和 “矩阵可逆但首枢轴为零” 的失败分支：

```
signed main()
{
    LDLT fact({{4, 8, -4}, {8, 21, 7}, {-4, 7, 47}});
    auto x = fact.solve({-24, -13, 123});
    assert(fact.valid && x);
    assert(fabsl((*x)[0] - 1) < 1E-10L);
    assert(fabsl((*x)[1] + 2) < 1E-10L);
    assert(fabsl((*x)[2] - 3) < 1E-10L);
    auto empty = LDLT({}).solve({});
    assert(empty && empty->empty());
    LDLT zeroPivot({{0, 1}, {1, 0}});
    assert(!zeroPivot.valid && !zeroPivot.solve({1, 1}));
```

```
    cout << llround((*x)[0]) << ' ' << llround((*x)[1]) << ' ' << llround((*x)[2]) << '\n'; // 期望输出：1 -2 3
    return 0;
}
```

9.3 数值计算

9.3.001 自适应 Simpson

Simpson 公式用一条抛物线拟合三点近似定积分：$\int_l^r f \approx \frac{r-l}{6}\left(f(l)+4f(\frac{l+r}{2})+f(r)\right)$，对三次以内多项式精确。自适应细分是它的实用化：把区间二分后左右两段的 Simpson 和 S_{lr} 与整段 S 相比；对足够光滑的函数，两半的总误差约为整段误差的 $1/16$，故差值 $S_{lr}-S$ 能估计误差。当 $|S_{lr}-S| \le 15\varepsilon$ 时，用 $S_{lr}+\frac{S_{lr}-S}{15}$ 做 Richardson 外推；否则把误差预算 ε 均分给两半递归。变化平缓处自然少分、剧烈处自动加密，代码中的 `err` 就是 ε。

```
template <class F>
long double simpson(F f, long double l, long double r, long double err = 1E-12L, int dep = 24)
{
    // f 是被积函数，[l,r] 是区间，err 是绝对误差目标，dep 是递归上限；返回定积分近似值。
    assert(err > 0 && dep >= 0); // 调试检查，可删。
    auto simpson = [](long double ql, long double qr, long double vl, long double vm, long double vr)
    {
        // ql、qr 是端点，vl、vm、vr 是左端、中点、右端函数值；返回一次 Simpson 近似。
        return (vl + 4 * vm + vr) * (qr - ql) / 6;
    };
    long double m = (l + r) / 2;
    long double fl = f(l);
    long double fm = f(m);
    long double fr = f(r);
    auto dfs = [&](auto &&self,
                   long double ql,
                   long double qr,
                   long double vl,
                   long double vm,
                   long double vr,
                   long double whole,
                   long double tol,
                   int depth) -> long double
    {
        // self 是递归入口，whole 是当前段近似，tol 是误差预算，depth 是剩余层数；返回当前段积分值。
        long double mid = (ql + qr) / 2;
        long double x = (ql + mid) / 2;
        long double y = (mid + qr) / 2;
        long double fx = f(x);
        long double fy = f(y);
        long double a = simpson(ql, mid, vl, fx, vm);
        long double b = simpson(mid, qr, vm, fy, vr);
        if (!depth || fabsl(a + b - whole) <= 15 * tol)
```

```
        {
            return a + b + (a + b - whole) / 15;
        }
        return self(self, ql, mid, vl, fx, vm, a, tol / 2, depth - 1) +
               self(self, mid, qr, vm, fy, vr, b, tol / 2, depth - 1);
    };
    return dfs(dfs, l, r, fl, fm, fr, simpson(l, r, fl, fm, fr), err, dep);
}
```

调用 `simpson(f,l,r,err,dep)`，要求 `err>0`、`dep>=0`，且 `f` 在采样点处处有定义；对 $f(x)=x^4$ 在 $[0,1]$ 积分结果应在 10^{-10} 内逼近 0.2。间断、尖峰或强振荡函数要在奇点处主动分段，`dep` 只是递归上限、并非误差保证。若实际细分出 K 个区间，则有 $O(K)$ 次函数求值，递归栈不超过 `dep` 层。被积函数很光滑、想要更高阶收敛时用 9.3.002 Romberg 的外推；本条的优势在于对局部剧烈变化的函数按需加密、实现简短。

```
signed main()
{
    auto f = [](long double x) { return x * x * x * x; };
    assert(fabsl(simpson(f, 0, 1) - .2L) < 1E-10L);
}
```

9.3.002 Romberg 积分

复合梯形公式的误差按步长有 $h^2,h^4,h^6,\dots$ 的偶次幂展开（Euler–Maclaurin），这正好适合 Richardson 外推逐阶消去。Romberg 先用梯形法在步长 $h,h/2,h/4,\dots$ 上算出第一列 $R_{i,0}$——第 i 层复用上一层的值、只补新加入的中点，不重算旧点；再按 $R_{i,j}=\dfrac{4^jR_{i,j-1}-R_{i-1,j-1}}{4^j-1}$ 组合相邻两层，每做一次 j 就消掉一阶误差，对光滑函数收敛极快。本实现只滚动保存一维数组，原地从高层往低层外推。

```
template <class F> long double romberg(F f, long double l, long double r, int levels = 15)
{
    // f 是被积函数，[l,r] 是区间，levels 是外推层数；返回 Romberg 积分近似值。
    assert(1 <= levels && levels <= 25); // 调试检查，可删。
    vector<long double> a(levels);
    a[0] = (f(l) + f(r)) * (r - l) / 2;
    for (int i = 1; i < levels; i++)
    {
        unsigned long long cnt = 1ULL << i;
        long double h = (r - l) / cnt;
        long double sum = 0;
        for (unsigned long long j = 1; j < cnt; j += 2)
        {
            sum += f(l + j * h);
        }
        a[i] = a[i - 1] / 2 + h * sum;
        long double p = 4;
        for (int j = i - 1; j >= 0; j--)
        {
            a[j] = (p * a[j + 1] - a[j]) / (p - 1);
```

```
            p *= 4;
        }
    }
    return a[0];
}
```

调用 `romberg(f,l,r,levels)`，`levels` 每加 1 采样点约翻倍，限制为 `1<=levels<=25`；最高层约有 2^{24} 个小区间，继续放宽通常已没有竞赛实用性。外推因子使用 `long double`，不会发生整数连乘溢出。对 $f(x)=x^4$ 在 $[0,1]$ 应收敛到 0.2。若外推层数为 L，函数求值次数为 $O(2^L)$，空间为 $O(L)$。端点有可积奇异时须先变量代换；函数有间断或尖峰时用 9.3.001 自适应 Simpson 更合适。

```
signed main()
{
    auto f = [](long double x) { return x * x * x * x; };
    assert(fabsl(romberg(f, 0, 1, 8) - .2L) < 1E-12L);
}
```

9.3.003 Newton 多项式求根

Newton 迭代用当前点的切线零点逼近函数零点：把 f 在 x 处一阶泰勒展开 $f(x)+f'(x)\Delta\approx 0$ 解出步长，得 $x\leftarrow x-\dfrac{f(x)}{f'(x)}$。在单根附近它二次收敛（有效位数每步约翻倍），但对任意初值并不保证收敛：初值离根远、或迭代路过导数为零的驻点时步长会爆炸甚至发散。本实现把零导数与非有限数判为失败，用相对位移 $|y-x|\le\varepsilon\max(1,|y|)$ 判收敛；耗尽迭代上限仍未满足该条件就返回空。这里不对导数或函数值使用固定绝对阈值，因而把 f 整体乘一个很小的非零常数不会改变判定。

```
template <class F, class G> optional<long double> newton(F f, G df, long double x, int it = 80)
{
    // f、df 是函数及导数，x 是初值，it 是迭代上限；收敛时返回一个根，导数为零或发散返回空。
    assert(it >= 0); // 调试检查，可删。
    for (int i = 0; i < it; i++)
    {
        long double d = df(x);
        if (d == 0 || !isfinite(d))
        {
            return nullopt;
        }
        long double y = x - f(x) / d;
        if (!isfinite(y))
        {
            return nullopt;
        }
        if (fabsl(y - x) <= eps * max(1.L, fabsl(y)))
        {
            return y;
        }
        x = y;
    }
    return nullopt;
```

```
}
```

调用 `newton(f,df,x0)` 传函数、导数与初值，要求 `it>=0`；返回空表示当前初值未能可靠收敛。若实际迭代 I 轮，就有 $O(I)$ 次函数与导数求值和 $O(1)$ 空间；二次收敛阶不等于最坏迭代次数保证。没有好初值但已知一个异号区间 $[l,r]$（$f(l)f(r)<0$）时，用有界二分更稳、必定收敛；专门求实系数三次方程全部实根时用 9.3.004 一元三次方程，它先用导数临界点定位单调段再二分，不依赖初值。

```
signed main()
{
    auto f = [](long double x) { return x * x - 2; };
    auto df = [](long double x) { return 2 * x; };
    auto root = newton(f, df, 1.L);
    assert(root && fabsl(*root * *root - 2) < 1E-12L);
}
```

9.3.004 一元三次方程

求实系数三次方程 $ax^3+bx^2+cx+d=0$ 的全部互异实根，思路是“分段单调 + 二分”。次数按归一化前的精确零判断；随后把系数除以最大绝对值，要求原来非零的最高有效系数归一化后仍非零，这一比例范围取决于平台的 `long double`。三次函数的导数至多两个实根，它们把实轴切成至多三段单调区间，再由 Cauchy 根界包住全部实根并逐段二分。

```
optional<vector<long double>> cubicRoots(long double a, long double b, long double c, long
↪ double d)
{
    // a、b、c、d 是有限实系数；返回升序互异实根，恒等式返回空 optional，无实根返回存在的空向量。
    assert(isfinite(a) && isfinite(b) && isfinite(c) && isfinite(d)); // 调试检查，可删。
    bool hasA = a != 0;
    bool hasB = b != 0;
    bool hasC = c != 0;
    long double sc = max({fabsl(a), fabsl(b), fabsl(c), fabsl(d)});
    if (sc == 0)
    {
        return nullopt;
    }
    a /= sc;
    b /= sc;
    c /= sc;
    d /= sc;
    assert((!hasA || a != 0) && (!hasB || b != 0) && (!hasC || c != 0)); // 调试检查，可删。
    if (!hasA)
    {
        if (!hasB)
        {
            if (!hasC)
            {
                return vector<long double>{};
            }
            return vector<long double>{-d / c};
```

```
        }
        long double det = c * c - 4 * b * d;
        if (det < -eps)
        {
            return vector<long double>{};
        }
        det = max(0.L, det);
        if (det == 0)
        {
            return vector<long double>{-c / (2 * b)};
        }
        long double s = sqrtl(det);
        long double q = -0.5L * (c + copysignl(s, c));
        vector<long double> ans{q / b, d / q};
        sort(ans.begin(), ans.end());
        if (ans[0] == ans[1])
        {
            ans.pop_back();
        }
        return ans;
    }
    auto f = [&](long double x)
    {
        return ((a * x + b) * x + c) * x + d;
    };
    long double lim = 1 + max({fabsl(b), fabsl(c), fabsl(d)}) / fabsl(a);
    vector<long double> key{-lim, lim};
    long double det = b * b - 3 * a * c;
    if (det >= -eps)
    {
        det = sqrtl(max(0.L, det));
        long double x = (-b - det) / (3 * a);
        long double y = (-b + det) / (3 * a);
        if (-lim < x && x < lim)
        {
            key.push_back(x);
        }
        if (-lim < y && y < lim)
        {
            key.push_back(y);
        }
    }
    sort(key.begin(), key.end());
    vector<long double> ans;
    for (auto x : key)
    {
        if (fabsl(f(x)) <= eps)
        {
            ans.push_back(x);
        }
    }
    for (int i = 0; i + 1 < (int)key.size(); i++)
```

```
    {
        long double l = key[i];
        long double r = key[i + 1];
        long double fl = f(l);
        long double fr = f(r);
        if (fl == 0 || fr == 0 || (fl < 0) == (fr < 0))
        {
            continue;
        }
        for (int t = 0; t < 100; t++)
        {
            long double m = (l + r) / 2;
            long double fm = f(m);
            if ((fl < 0) == (fm < 0))
            {
                l = m;
                fl = fm;
            }
            else
            {
                r = m;
                fr = fm;
            }
        }
        ans.push_back((l + r) / 2);
    }
    sort(ans.begin(), ans.end());
    ans.erase(unique(ans.begin(),
                     ans.end(),
                     [&](auto x, auto y)
                     {
                         return fabsl(x - y) <= 100 * eps * max({1.L, fabsl(x), fabsl(y)});
                     }),
              ans.end());
    return ans;
}
```

调用 `cubicRoots(a,b,c,d)`：四个系数必须有限，且原来非零的最高有效系数除以最大绝对值后仍可表示为非零；MSVC 的 `long double` 通常等同 `double`，范围小于 GCC x86 的扩展精度。返回 `nullopt` 表示恒等式，存在但为空的向量表示没有实根，否则返回升序互异实根。原始系数精确零只用于次数分流，归一化判别式用相对其各项量级的容差，根去重使用相对根大小的 `100*eps`，三者不是同一零判断。内部二分固定 100 轮，时间、空间均为 $O(1)$。

下面检查三个实根、退化二次、无实根、重根与恒等式：

```
signed main()
{
    auto a = cubicRoots(1, -6, 11, -6);
    assert(a && a->size() == 3);
    auto b = cubicRoots(0, -1, 3, -2);
    assert(b && b->size() == 2 && fabsl((*b)[0] - 1) < 1E-10L && fabsl((*b)[1] - 2) < 1E-10L);
```

```
    auto wide = cubicRoots(0, 1, 1E16L, 1);
    assert(wide && wide->size() == 2 && fabsl((*wide)[1] + 1E-16L) < 1E-20L);
    auto none = cubicRoots(0, 1, 0, 1);
    assert(none && none->empty());
    auto repeated = cubicRoots(1, 0, 0, 0);
    assert(repeated && repeated->size() == 1 && fabsl((*repeated)[0]) < 1E-10L);
    assert(!cubicRoots(0, 0, 0, 0));
    cout << llround((*a)[0]) << ' ' << llround((*a)[1]) << ' ' << llround((*a)[2]) << '\n'; // 期望输出：1 2 3
    return 0;
}
```

9.3.005 线性插值与幂和

很多计数题的答案关于某个变量 x 是次数已知的多项式（典型如幂和 $\sum_{i=1}^{x} i^k$ 是 x 的 $k+1$ 次多项式），却只能算出前若干个点值。给定 $f(0), f(1), \dots, f(n-1)$ 这 n 个等距样本，f 的次数小于 n，就能用拉格朗日插值唯一恢复 $f(x)$。等距点让公式大幅简化：$f(x) = \sum_i y_i \prod_{j \neq i} \frac{x-j}{i-j}$ 中，分母 $\prod_{j \neq i}(i-j) = (-1)^{n-1-i}\, i!\,(n-1-i)!$ 可用阶乘逆元 $O(1)$ 得到，分子的连乘用前缀积 `pre` 与后缀积 `suf` 拆开，于是整体 $O(n)$。模数须为素数以保证阶乘可逆。`interpSeq` 里 $x < n$ 直接返回样本值。

```
// 依赖 7.1.001 快速幂
inline long long interpSeq(const vector<long long> &y, long long x, long long mod)
{
    // y[i]=f(i)，x 是查询点，mod 是素数；返回 f(x) 对 mod 的最小非负余数。
    int n = y.size();
    assert(n > 0 && n < mod); // 调试检查，可删。
    x %= mod;
    if (x < 0)
    {
        x += mod;
    }
    if (x < n)
    {
        long long ans = y[x] % mod;
        return ans < 0 ? ans + mod : ans;
    }
    vector<long long> fac(n, 1), ifac(n, 1), pre(n + 1, 1), suf(n + 1, 1);
    for (int i = 1; i < n; ++i)
    {
        fac[i] = mulMod(fac[i - 1], i, mod);
    }
    ifac[n - 1] = powerMod(fac[n - 1], mod - 2, mod);
    for (int i = n - 1; i; --i)
    {
        ifac[i - 1] = mulMod(ifac[i], i, mod);
    }
    for (int i = 0; i < n; ++i)
    {
        long long d = x >= i ? x - i : mod - (i - x);
```

```
        pre[i + 1] = mulMod(pre[i], d, mod);
    }
    for (int i = n - 1; i >= 0; --i)
    {
        long long d = x >= i ? x - i : mod - (i - x);
        suf[i] = mulMod(suf[i + 1], d, mod);
    }
    long long ans = 0;
    for (int i = 0; i < n; ++i)
    {
        long long v = y[i] % mod;
        if (v < 0)
        {
            v += mod;
        }
        long long term = mulMod(v, pre[i], mod);
        term = mulMod(term, suf[i + 1], mod);
        term = mulMod(term, ifac[i], mod);
        term = mulMod(term, ifac[n - 1 - i], mod);
        if ((n - 1 - i) & 1)
        {
            term = term ? mod - term : 0;
        }
        ans = ans >= mod - term ? ans - (mod - term) : ans + term;
    }
    return ans;
}
```

调用 `interpSeq(values,x,mod)`，要求样本非空、f 次数小于样本数、`mod` 为素数且 `values.size()<mod`；空样本不能唯一确定多项式，属于不满足前提。`x` 和样本值可以为负，函数统一返回最小非负余数。该接口保留运行时素数模并支持接近 `long long` 上界的查询，因此不能实例化固定 `MInt<P>`；模乘和模幂统一复用 7.1.001 快速幂，末尾模加用减法比较避免溢出。若模数为 M，预处理与单次求值为 $O(n+\log M)$ 时间、$O(n)$ 空间。求幂和 $\sum_{i=0}^{x} i^k$ 时先算 $k+2$ 个前缀样本再插值；固定常用素数且多次调用时，优先用 1.1.008 静态取模类改写元素类型。非等距样本改用一般拉格朗日插值或第 10 章的多点求值。

下面用 $f(x)=x^2$ 检查区间外正查询和负查询，再检查负样本的直接返回：

```
signed main()
{
    assert(interpSeq({0, 1, 4, 9}, 5, 998244353) == 25);
    constexpr long long mod = 9'223'372'036'854'775'783LL;
    assert(interpSeq({0, 1, 4}, mod - 1, mod) == 1);
    assert(interpSeq({0, 1, 4}, -1, 7) == 1);
    assert(interpSeq({0, -1}, 1, 5) == 4);
    cout << interpSeq({0, 1, 4, 9}, 5, 998244353) << '\n'; // 期望输出：25
    return 0;
}
```

9.3.006 指数乘多项式有限和

给定次数不超过 d 的多项式 f 在 $0,1,\dots,d$ 的值，求 $\sum_{i=0}^{n-1} r^i f(i)$。当 $r=1$ 时，前缀和本身是 $d+1$ 次多项式，算出前 $d+2$ 个值后等距插值即可；当 $r\neq 0,1$ 时，先求形式无穷和常数 $C=\sum_{i\geq 0} r^i f(i)$，则 $r^{-n}(S_n-C)$ 是关于 n 的 d 次多项式，同样由前 $d+2$ 个值插值得到。`r=0` 必须单独处理，因为一般分支会请求它的逆元。

```cpp
template <class T> T fieldPow(T a, unsigned long long b)
{
    // a 是域元素，b 是非负指数；返回 a 的 b 次幂。
    T ans = 1;
    while (b)
    {
        if (b & 1)
        {
            ans *= a;
        }
        b >>= 1;
        if (b)
        {
            a *= a;
        }
    }
    return ans;
}

template <class T> T interpIota(const vector<T> &y, long long x)
{
    // y[i] 是多项式在 i 处的值，x 是非负查询点；返回该多项式在 x 处的值。
    int n = y.size();
    assert(n > 0 && x >= 0); // 调试检查，可删。
    if (x < n)
    {
        return y[x];
    }
    vector<T> fac(n, T(1)), ifac(n, T(1));
    vector<T> pre(n + 1, T(1)), suf(n + 1, T(1));
    for (int i = 1; i < n; i++)
    {
        fac[i] = fac[i - 1] * T(i);
    }
    ifac[n - 1] = T(1) / fac[n - 1];
    for (int i = n - 1; i; i--)
    {
        ifac[i - 1] = ifac[i] * T(i);
    }
    for (int i = 0; i < n; i++)
    {
        pre[i + 1] = pre[i] * T(x - i);
    }
    for (int i = n - 1; i >= 0; i--)
```

```
    {
        suf[i] = suf[i + 1] * T(x - i);
    }
    T ans = 0;
    for (int i = 0; i < n; i++)
    {
        T v = y[i] * pre[i] * suf[i + 1] * ifac[i] * ifac[n - 1 - i];
        ans += (n - 1 - i) & 1 ? -v : v;
    }
    return ans;
}

template <class T> T sumExpPLim(T r, vector<T> f)
{
    // f[i] 是多项式在 i 处的值且 r≠1；返回生成函数给出的形式无穷和。
    assert(!f.empty() && r != T(1)); // 调试检查，可删。
    T q = r / (T(1) - r);
    T term = T(1) / (T(1) - r);
    T ans = 0;
    for (int k = 0; k < (int)f.size(); k++)
    {
        ans += f[0] * term;
        for (int i = 0; i + 1 < (int)f.size() - k; i++)
        {
            f[i] = f[i + 1] - f[i];
        }
        term *= q;
    }
    return ans;
}

template <class T> T sumExpPoly(T r, const vector<T> &f, long long n)
{
    // f[i] 是多项式在 i 处的值，n 是项数；返回前 n 项的指数加权和。
    assert(!f.empty() && n >= 0); // 调试检查，可删。
    if (n == 0)
    {
        return T(0);
    }
    if (r == T(0))
    {
        return f[0];
    }
    int d = f.size() - 1;
    vector<T> s(d + 2);
    T pw = 1;
    for (int i = 0; i <= d; i++)
    {
        s[i + 1] = s[i] + pw * f[i];
        pw *= r;
    }
    if (r == T(1))
```

```
    {
        return interpIota(s, n);
    }
    T limit = sumExpPLim(r, f);
    T inv = T(1) / r;
    T ipw = 1;
    for (int i = 0; i <= d + 1; i++)
    {
        s[i] = (s[i] - limit) * ipw;
        ipw *= inv;
    }
    return limit + fieldPow(r, (unsigned long long)n) * interpIota(s, n);
}
```

`sumExpPoly(r,f,n)` 要求 `f[i]` 是次数小于 `f.size()` 的多项式在连续整数点的值，`n>=0`。`T` 必须是域，且 $1, 2, \dots, d+1$ 可逆；素数模下通常要求 `d+1<mod`。有限差分占 $O(d^2)$ 时间，其他步骤为 $O(d + \log n)$，空间为 $O(d)$。若已有快速卷积，可用二项式变换加速各阶差分的计算；普通竞赛次数下本版更短。

下面覆盖 `r=0`、`r=1` 与一般分支：

```
signed main()
{
    assert(fabsl(sumExpPoly(2.L, vector<long double>{0, 1}, 4) - 34) < 1E-10L);
    assert(fabsl(sumExpPoly(1.L, vector<long double>{0, 1}, 5) - 10) < 1E-10L);
    assert(fabsl(sumExpPoly(0.L, vector<long double>{7, 8}, 100) - 7) < 1E-10L);
    cout << llround(sumExpPoly(2.L, vector<long double>{0, 1}, 4)) << '\n'; // 期望输出：34
    return 0;
}
```

9.3.007 指数乘多项式无穷和

把 $f(i)$ 写成 Newton 展开 $f(i) = \sum_k \Delta^k f(0) \binom{i}{k}$，再用 $\sum_{i \ge 0} r^i \binom{i}{k} = \dfrac{r^k}{(1-r)^{k+1}}$，即可得到上条 `sumExpPLim(r,f)`。调用时 `f[i]` 必须是次数不超过 d 的多项式在 $i = 0, 1, \dots, d$ 处的值，`r=1` 非法，`r=0` 合法且结果为 `f(0)`。这里的 “无穷和” 是在域上按生成函数得到的有理式；当 `T` 是实数时，只有 $|r| < 1$ 才同时等于通常意义下的收敛级数。时间为 $O(d^2)$，空间为 $O(d)$；实现与有限和共享，故不重复打印。

```
// 依赖 9.3.006 指数乘多项式有限和
signed main()
{
    long double a = sumExpPLim(0.5L, vector<long double>{1});
    long double b = sumExpPLim(0.5L, vector<long double>{0, 1});
    assert(fabsl(a - 2) < 1E-10L && fabsl(b - 2) < 1E-10L);
    assert(fabsl(sumExpPLim(0.L, vector<long double>{7, 8}) - 7) < 1E-10L);
    cout << fixed << setprecision(1) << a << ' ' << b << '\n'; // 期望输出：2.0 2.0
    return 0;
}
```

9.3.008 LLL 浮点实用版

LLL 算法把整数格的一组基 $b_0,\dots,b_{n-1}$ 变换成同一格中较短、较接近正交的基。设 b_i^* 是 Gram-Schmidt 正交化向量，$\mu_{i,j}=\frac{\langle b_i,b_j^*\rangle}{\langle b_j^*,b_j^*\rangle}$。算法维护下标 k（初值 1），反复执行：

1. 尺寸约简：对 $j=k-1,\dots,0$，令 $q=\operatorname{round}(\mu_{k,j})$，$b_k\leftarrow b_k-qb_j$，使 $|\mu_{k,j}|\le\frac12$；此时正交化向量不变，只需同步更新 $\mu_{k,\cdot}$。
2. Lovász 条件：若 $\|b_k^*\|^2\ge(\delta-\mu_{k,k-1}^2)\|b_{k-1}^*\|^2$，则 $k\leftarrow k+1$；否则交换 b_k,b_{k-1}，重新正交化并令 $k\leftarrow\max(1,k-1)$。

结束时第一个向量满足 $\|b_0\|\le\left(\frac{4}{4\delta-1}\right)^{(n-1)/2}\lambda_1$，$\lambda_1$ 是格中最短非零向量长度。

```cpp
optional<vector<vector<int>>> lll(vector<vector<int>> b, long double delta = 0.75L)
{
    // b 是 n 个长度为 m 的整数向量，须线性无关；delta 取 (1/4,1)，越接近 1 约简越强。
    // 返回同一格的 LLL 约简基（第一个向量长度不超过最短向量的 (4/(4delta-1))^{(n-1)/2} 倍）；
    // 线性相关时返回空。Gram-Schmidt 用 long double 计算，系数很大时可能有精度误差。
    int n = b.size();
    if (!n)
    {
        return b;
    }
    int m = b[0].size();
    assert(0.25L < delta && delta < 1); // 调试检查，可删。
    // st[i] 是 b[i] 的 Gram-Schmidt 正交化向量，nm[i]=|st[i]|^2，mu[i][j]=<b[i],st[j]>/nm[j]。
    vector<vector<long double>> st(n, vector<long double>(m)), mu(n, vector<long double>(n));
    vector<long double> nm(n);
    auto gs = [&]() -> bool
    {
        // 重新做 Gram-Schmidt；出现零向量（线性相关）时返回 false。
        for (int i = 0; i < n; i++)
        {
            for (int t = 0; t < m; t++)
            {
                st[i][t] = b[i][t];
            }
            for (int j = 0; j < i; j++)
            {
                long double dot = 0;
                for (int t = 0; t < m; t++)
                {
                    dot += b[i][t] * st[j][t];
                }
                mu[i][j] = dot / nm[j];
                for (int t = 0; t < m; t++)
                {
                    st[i][t] -= mu[i][j] * st[j][t];
                }
            }
            nm[i] = 0;
            for (auto x : st[i])
```

```
            {
                nm[i] += x * x;
            }
            if (nm[i] < 1e-9L)
            {
                return false;
            }
        }
        return true;
    };
    if (!gs())
    {
        return nullopt;
    }
    int k = 1;
    while (k < n)
    {
        // 尺寸约简：让 |mu[k][j]| <= 1/2。
        for (int j = k - 1; j >= 0; j--)
        {
            int q = llroundl(mu[k][j]);
            if (q)
            {
                for (int t = 0; t < m; t++)
                {
                    b[k][t] -= q * b[j][t];
                }
                // 正交向量不变，只需同步 mu[k][*]。
                for (int i = 0; i < j; i++)
                {
                    mu[k][i] -= q * mu[j][i];
                }
                mu[k][j] -= q;
            }
        }
        // Lovász 条件成立则前进，否则交换相邻两向量并后退。
        if (nm[k] >= (delta - mu[k][k - 1] * mu[k][k - 1]) * nm[k - 1])
        {
            k++;
        }
        else
        {
            swap(b[k], b[k - 1]);
            gs();
            k = max(1LL, k - 1);
        }
    }
    return b;
}
```

`lll(b,delta)` 中 `b` 是 n 个长为 m 的整数向量，须线性无关，$\delta \in (\frac{1}{4}, 1)$，默认 0.75；返回约简后的基，线性相关时返回空。Gram-Schmidt 用 `long double`，坐标很大时可能因精度问题得不到严格的约简基，

也可能在 `long long` 坐标上溢出，需要严格结果时应改用有理数或整数版本。交换次数关于输入规模是多项式级，每次交换重新正交化 $O(n^2 m)$。

典型用途是背包型密码、求小整数关系与丢番图逼近；二维格的最短向量用 Gauss 约简更简单。下面约简一个三维格，变换前后行列式的绝对值都是 3：

```
signed main()
{
    auto b = lll({{1, 1, 1}, {-1, 0, 2}, {3, 5, 6}});
    assert(b && *b == vector<vector<int>>({{0, 1, 0}, {1, 0, 1}, {-2, 0, 1}}));
    assert(!lll({{1, 1}, {2, 2}}));
}
```

10. 多项式、变换与线性递推

10.1 卷积与变换

10.1.001 DFT / IDFT

DFT 把长度 n 的序列映到 n 个单位根上的点值 $A_k = \sum_{j=0}^{n-1} a_j \omega_n^{jk}$，其中 $\omega_n = e^{2\pi i/n}$；逆变换换成共轭单位根 ω_n^{-jk} 再除以 n，单位根正交性 $\sum_k \omega_n^{k(i-j)} = n\,[i=j]$ 保证两次变换互逆。这份朴素双重循环只用于小规模验算和理解定义，不是快速卷积主力。

```cpp
using Complex = complex<long double>;

vector<Complex> dftNaive(const vector<Complex> &a, bool inv = false)
{
    // a 是时域或频域序列，inv=false/true 表示 DFT/IDFT；返回变换结果，逆变换已除以长度。
    int n = a.size();
    vector<Complex> b(n);
    long double pi = acosl(-1);
    for (int k = 0; k < n; k++)
    {
        for (int j = 0; j < n; j++)
        {
            long double ang = 2 * pi * j * k / n * (inv ? -1 : 1);
            b[k] += a[j] * Complex(cosl(ang), sinl(ang));
        }
        if (inv)
        {
            b[k] /= n;
        }
    }
    return b;
}
```

`dftNaive(a,false)` 得频域点值，`dftNaive(A,true)` 还原时域，浮点结果按误差比较；序列 `{1,2,3,4}` 正逆变换后实部在 10^{-12} 内还原、虚部量级约 10^{-19}。$O(n^2)$ 时间、$O(n)$ 输出空间。规模一大必须换 10.1.002 FFT 的 $O(n \log n)$ 蝶形；模意义精确卷积用 10.1.003 NTT，目标模数不支持则用 10.1.004 MTT 任意模数卷积。

10.1.002 FFT

FFT 用分治蝴蝶在 $O(n \log n)$ 内算 DFT/IDFT：按下标奇偶把 $A_k = E_k + \omega_n^k O_k$、$A_{k+n/2} = E_k - \omega_n^k O_k$ 展开，迭代版先做位逆序置换再自底向上合并各层蝶形，逆变换用共轭角度并整体除以长度。整数卷积把系数放进复数实部，变换后逐点相乘再逆变换、结果四舍五入取整；`long double` 精度下系数与长度不过大时误差可控（本机随机 ± 500、长度百级时最大绝对误差约 3.5×10^{-12}）。

```cpp
using C = complex<long double>;

```

```cpp
void fft(vector<C> &a, bool inv = false)
{
    // a 是待变换序列，inv 表示是否做逆变换；原地完成 FFT，逆变换已除以长度。
    int n = a.size();
    assert(n > 0 && n <= (1 << 23) && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    int j = 0;
    for (int i = 1; i < n; i++)
    {
        int k = n >> 1;
        while (j & k)
        {
            j ^= k;
            k >>= 1;
        }
        j ^= k;
        if (i < j)
        {
            swap(a[i], a[j]);
        }
    }
    const long double pi = acosl(-1);
    for (int len = 2; len <= n; len <<= 1)
    {
        long double ang = 2 * pi / len * (inv ? -1 : 1);
        C wlen(cosl(ang), sinl(ang));
        for (int l = 0; l < n; l += len)
        {
            C w{1.0L, 0.0L};
            for (int j = 0; j < len / 2; j++)
            {
                C x = a[l + j];
                C y = a[l + j + len / 2] * w;
                a[l + j] = x + y;
                a[l + j + len / 2] = x - y;
                w *= wlen;
            }
        }
    }
    if (inv)
    {
        for (C &x : a)
        {
            x /= n;
        }
    }
}

vector<long double> convolution(const vector<long double> &a, const vector<long double> &b)
{
    // a、b 是实系数序列；返回二者的线性卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
```

```
        return {};
    }
    int sz = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)sz);
    vector<C> x(n);
    vector<C> y(n);
    copy(a.begin(), a.end(), x.begin());
    copy(b.begin(), b.end(), y.begin());
    fft(x);
    fft(y);
    for (int i = 0; i < n; i++)
    {
        x[i] *= y[i];
    }
    fft(x, true);
    vector<long double> c(sz);
    for (int i = 0; i < sz; i++)
    {
        c[i] = x[i].real();
    }
    return c;
}
```

`convolution(a,b)` 返回长度 $n+m-1$ 的实系数卷积，内部补齐到 `bit_ceil(n+m-1)`；`fft(a,inv)` 原地变换要求长度是 2 的幂。长度 N 的变换 $O(N\log N)$、卷积 $O((n+m)\log(n+m))$，空间 $O(N)$。系数或长度过大导致浮点误差失控时改用 10.1.003 NTT 的精确整数变换；目标模数不含 NTT 单位根时在此 FFT 上拆系数即 10.1.004 MTT 任意模数卷积。

下面用空输入、单项、负系数和结果长度为 9 的固定例核对补长与舍入；输出 `OK` 即通过：

```
signed main()
{
    assert(convolution({}, {1}).empty());
    auto a = convolution(vector<long double>{3}, vector<long double>{-7});
    assert(a.size() == 1 && llroundl(a[0]) == -21);
    auto b = convolution(vector<long double>{1, 2, 3}, vector<long double>{4, -5});
    assert(b.size() == 4 && llroundl(b[0]) == 4 && llroundl(b[1]) == 3 &&
           llroundl(b[2]) == 2 && llroundl(b[3]) == -15);
    auto c = convolution(vector<long double>{1, -2, 3, -4, 5},
                         vector<long double>{6, 7, -8, 9, -10});
    vector<long long> want{6, -5, -4, 22, -50, 114, -106, 85, -50};
    assert(c.size() == want.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        assert(llroundl(c[i]) == want[i]);
    }
    cout << "OK\n";
    return 0;
}
```

10.1.003 NTT

NTT 在有限域上做整数精确的 DFT。设素模 p、原根 g、变换长度 $N \mid p-1$，取 $\omega_N = g^{(p-1)/N}$，则 $\omega_N^N = 1$ 且 $0 < k < N$ 时 $\omega_N^k \neq 1$。正变换 $A_k = \sum_j a_j \omega_N^{jk}$；由单位根正交性 $\sum_k \omega_N^{k(i-j)} = N\,[i = j]$ 得逆变换 $a_j = N^{-1} \sum_k A_k \omega_N^{-jk}$。按奇偶拆下标并用 $\omega_N^{N/2} = -1$ 得一层蝶形 $A_k = E_k + \omega_N^k O_k$、$A_{k+N/2} = E_k - \omega_N^k O_k$。

`FastNTT` 把相邻两层合成 radix-4 蝶形，用 `rate2`、`rate3` 按块更新旋转因子，不做显式位逆序。长度 N 的结果中相邻位置 `2*i,2*i+1` 对应 $z, -z$，前者与长度 $N/2$ 变换的第 i 个频点对应同一个 z^2；10.2.012 多项式复合 正是复用这个排列契约。`NTT32` 是另一种受限工作接口：它直接处理 `int*`，缓存位逆序表和一块 `unsigned long long` 缓冲区；乘单位根后仍立即取模，只延迟蝶形加减结果的归一化。长度上限固定为 2^{17}，此时尚未归一化的中间值与单位根相乘仍严格落在 64 位无符号范围内；它服务 10.2.005 `ln`、`exp`、开根、快速幂 的 16 叉在线卷积，不能替代最大长度 2^{23} 的通用 `FastNTT`。两者共用本章的 `Z`、模数和单位根计算，没有再复制模整数、卷积或 FPS 基础层。

```
// 依赖 1.1.008 静态取模类
constexpr int NTT_LIM = 1 << 23;

bool nttFit(int n, int m)
{
    // n、m 是两输入序列长度；返回一次不超过 NTT_LIM 的变换能否覆盖其线性卷积。
    assert(n >= 0 && m >= 0); // 调试检查，可删。
    return n == 0 || m == 0 || (__int128)n + m - 1 <= NTT_LIM;
}

bool convFit(int n, int m)
{
    // n、m 是两输入序列长度；返回公共 conv 能否计算其线性卷积。
    // 结果只超上限一至两项时，conv 会拆掉两端最高项再做一次较短卷积。
    assert(n >= 0 && m >= 0); // 调试检查，可删。
    return n == 0 || m == 0 || min(n, m) <= 64 ||
           (__int128)n + m - 1 <= NTT_LIM + 2;
}

class FastNTT
{
    // root[i]、iroot[i] 分别保存 2 的 i 次单位根及其逆元。
    static inline array<int, 24> root;
    static inline array<int, 24> iroot;
    // rate2、irate2 保存二进制蝶形跨块时的旋转因子更新量。
    static inline array<int, 22> rate2;
    static inline array<int, 22> irate2;
    // rate3、irate3 保存四进制蝶形跨块时的旋转因子更新量。
    static inline array<int, 21> rate3;
    static inline array<int, 21> irate3;
    // ready 标记单位根表是否已经初始化。
    static inline bool ready = false;

    static void init()
    {
```

```
        // 无参数；首次调用时预处理各阶单位根和跨块旋转因子，后续调用直接复用。
        if (ready)
        {
            return;
        }
        ready = true;
        root[23] = 31;
        iroot[23] = Z(root[23]).inv().val();
        for (int i = 22; i >= 0; i--)
        {
            root[i] = (unsigned long long)root[i + 1] * root[i + 1] % mod;
            iroot[i] = (unsigned long long)iroot[i + 1] * iroot[i + 1] % mod;
        }
        int prod = 1;
        int iprod = 1;
        for (int i = 0; i <= 21; i++)
        {
            rate2[i] = (unsigned long long)root[i + 2] * prod % mod;
            irate2[i] = (unsigned long long)iroot[i + 2] * iprod % mod;
            prod = (unsigned long long)prod * iroot[i + 2] % mod;
            iprod = (unsigned long long)iprod * root[i + 2] % mod;
        }
        prod = 1;
        iprod = 1;
        for (int i = 0; i <= 20; i++)
        {
            rate3[i] = (unsigned long long)root[i + 3] * prod % mod;
            irate3[i] = (unsigned long long)iroot[i + 3] * iprod % mod;
            prod = (unsigned long long)prod * iroot[i + 3] % mod;
            iprod = (unsigned long long)iprod * root[i + 3] % mod;
        }
    }

public:
    static void dft(Z *a, int n)
    {
        // a 指向长度 n 的连续系数，n 是至多 2^23 的正 2 的幂；原地完成正变换。
        // 长度 n 的相邻位置 2i、2i+1 对应 z、-z；前者与长度 n/2 变换的第 i 个频点对应同一个 z^2。
        assert(n > 0 && has_single_bit((unsigned long long)n) && n <= NTT_LIM); // 调试检查，可
        ↪ 删。
        init();
        int z = n;
        int h = countr_zero((unsigned long long)z);
        int len = 0;
        while (len < h)
        {
            if (h - len == 1)
            {
                int p = 1 << (h - len - 1);
                int rot = 1;
                for (int s = 0; s < (1 << len); s++)
                {
```

```
87                      int off = s << (h - len);
88                      for (int i = 0; i < p; i++)
89                      {
90                          int l = a[off + i].x;
91                          int r = (unsigned long long)a[off + i + p].x * rot % mod;
92                          a[off + i].x = l + r < mod ? l + r : l + r - mod;
93                          a[off + i + p].x = l >= r ? l - r : l + mod - r;
94                      }
95                      if (s + 1 != (1 << len))
96                      {
97                          rot = (unsigned long long)rot * rate2[countr_zero((unsigned long
    ↪ long)~s)] % mod;
98                      }
99                  }
100                 len++;
101             }
102             else
103             {
104                 int p = 1 << (h - len - 2);
105                 int rot = 1;
106                 int imag = root[2];
107                 for (int s = 0; s < (1 << len); s++)
108                 {
109                     int rot2 = (unsigned long long)rot * rot % mod;
110                     int rot3 = (unsigned long long)rot2 * rot % mod;
111                     int off = s << (h - len);
112                     for (int i = 0; i < p; i++)
113                     {
114                         unsigned long long m2 = (unsigned long long)mod * mod;
115                         unsigned long long a0 = a[off + i].x;
116                         unsigned long long a1 = (unsigned long long)a[off + i + p].x * rot;
117                        unsigned long long a2 = (unsigned long long)a[off + i + 2 * p].x * rot2;
118                        unsigned long long a3 = (unsigned long long)a[off + i + 3 * p].x * rot3;
119                        unsigned long long x = (unsigned long long)Z(a1 + m2 - a3).val() * imag;
120                         unsigned long long na2 = m2 - a2;
121                         a[off + i].x = (a0 + a2 + a1 + a3) % mod;
122                         a[off + i + p].x = (a0 + a2 + 2 * m2 - a1 - a3) % mod;
123                         a[off + i + 2 * p].x = (a0 + na2 + x) % mod;
124                         a[off + i + 3 * p].x = (a0 + na2 + m2 - x) % mod;
125                     }
126                     if (s + 1 != (1 << len))
127                     {
128                         rot = (unsigned long long)rot * rate3[countr_zero((unsigned long
    ↪ long)~s)] % mod;
129                     }
130                 }
131                 len += 2;
132             }
133         }
134     }
135
136     static void dft(vector<Z> &a)
```

```
    {
        // a 的长度是至多 2^23 的正 2 的幂；转发到连续内存正变换，无返回值。
        dft(a.data(), a.size());
    }

    static void idft(Z *a, int n)
    {
        // a 指向本类 dft 产生的 n 个频域值；原地逆变换、除以 n 并恢复自然顺序系数。
        assert(n > 0 && has_single_bit((unsigned long long)n) && n <= NTT_LIM); // 调试检查，可删。
        init();
        int z = n;
        int h = countr_zero((unsigned long long)z);
        int len = h;
        while (len)
        {
            if (len == 1)
            {
                int p = 1 << (h - len);
                int rot = 1;
                for (int s = 0; s < (1 << (len - 1)); s++)
                {
                    int off = s << (h - len + 1);
                    for (int i = 0; i < p; i++)
                    {
                        int l = a[off + i].x;
                        int r = a[off + i + p].x;
                        a[off + i].x = l + r < mod ? l + r : l + r - mod;
                        a[off + i + p].x = (unsigned long long)(l + mod - r) * rot % mod;
                    }
                    if (s + 1 != (1 << (len - 1)))
                    {
                        rot = (unsigned long long)rot * irate2[countr_zero((unsigned long long)~s)] % mod;
                    }
                }
                len--;
            }
            else
            {
                int p = 1 << (h - len);
                int rot = 1;
                int imag = iroot[2];
                for (int s = 0; s < (1 << (len - 2)); s++)
                {
                    int rot2 = (unsigned long long)rot * rot % mod;
                    int rot3 = (unsigned long long)rot2 * rot % mod;
                    int off = s << (h - len + 2);
                    for (int i = 0; i < p; i++)
                    {
                        unsigned long long a0 = a[off + i].x;
                        unsigned long long a1 = a[off + i + p].x;
```

```
                        unsigned long long a2 = a[off + i + 2 * p].x;
                        unsigned long long a3 = a[off + i + 3 * p].x;
                        unsigned long long x = Z((mod + a2 - a3) * imag).val();
                        a[off + i].x = (a0 + a1 + a2 + a3) % mod;
                        a[off + i + p].x = (a0 + mod - a1 + x) % mod * rot % mod;
                       a[off + i + 2 * p].x = (a0 + a1 + 2 * mod - a2 - a3) % mod * rot2 % mod;
                        a[off + i + 3 * p].x = (a0 + 2 * mod - a1 - x) % mod * rot3 % mod;
                    }
                    if (s + 1 != (1 << (len - 2)))
                    {
                        rot = (unsigned long long)rot * irate3[countr_zero((unsigned long
    long)~s)] % mod;
                    }
                }
                len -= 2;
            }
        }
        Z iv = Z(n).inv();
        for (int i = 0; i < z; i++)
        {
            a[i] *= iv;
        }
    }

    static void idft(vector<Z> &a)
    {
        // a 必须来自本类 dft；转发到连续内存逆变换，无返回值。
        idft(a.data(), a.size());
    }

    static vector<Z> conv(vector<Z> a, vector<Z> b)
    {
        // a、b 按低次到高次保存系数；返回普通线性卷积，任一输入为空时返回空。
        if (a.empty() || b.empty())
        {
            return {};
        }
        int n = a.size();
        int m = b.size();
        int s = n + m - 1;
        if (min(n, m) <= 64)
        {
            vector<Z> c(s);
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < m; j++)
                {
                    c[i + j] += a[i] * b[j];
                }
            }
            return c;
        }
```

```
        assert(convFit(n, m)); // 调试检查，可删。
        int z = bit_ceil((unsigned long long)s);
        if (s - 2 <= z / 2)
        {
            Z al = a.back();
            Z bl = b.back();
            a.pop_back();
            b.pop_back();
            auto x = a;
            auto y = b;
            auto c = conv(move(a), move(b));
            c.resize(s);
            c[s - 1] = al * bl;
            for (int i = 0; i < n - 1; i++)
            {
                c[i + m - 1] += x[i] * bl;
            }
            for (int i = 0; i < m - 1; i++)
            {
                c[i + n - 1] += y[i] * al;
            }
            return c;
        }
        bool same = a == b;
        a.resize(z);
        b.resize(z);
        dft(a);
        if (same)
        {
            b = a;
        }
        else
        {
            dft(b);
        }
        for (int i = 0; i < z; i++)
        {
            a[i] *= b[i];
        }
        idft(a);
        a.resize(s);
        return a;
    }
};

class NTT32
{
    // 本类服务原始 int 切片，复用位逆序表与 unsigned long long 工作区，并在蝶形间延迟取模。
    // 它与 FastNTT 的 Z 容器接口用途不同；2^17 上限保证最终归一化乘法不会溢出 unsigned long long。
    int mx = 0; // 允许的最大变换长度，所有实际长度必须整除 mx。
    vector<int> w; // w[l..2l) 保存长度 2l 蝶形使用的正单位根。
    vector<unsigned long long> buf; // 蝶形使用的延迟取模工作区。
```

```
290     array<vector<int>, 24> rev; // rev[k] 保存长度 2^k 的二进制翻转下标。
291
292     const vector<int> &getRev(int n)
293     {
294         // n 是不超过 mx 的二次幂；返回对应二进制翻转下标表的引用。
295         int k = countr_zero((unsigned long long)n);
296         auto &r = rev[k];
297         if (r.empty())
298         {
299             r.resize(n);
300             for (int i = 1; i < n; i++)
301             {
302                 r[i] = (r[i >> 1] >> 1) | ((int)(i & 1) << (k - 1));
303             }
304         }
305         return r;
306     }
307
308     void trans(int *a, int n, bool inv)
309     {
310         // a 指向长度 n 且各项在 [0,mod) 内的预分配切片，inv 表示是否逆变换；使用 unsigned long long
            ↪ 工作区原地完成变换。
311         assert(n > 0 && has_single_bit((unsigned long long)n) && n <= mx && mx % n == 0); // 调
            ↪ 试检查，可删。
312         const auto &r = getRev(n);
313         for (int i = 0; i < n; i++)
314         {
315             buf[i] = a[r[i]];
316         }
317         for (int h = 1; h < n; h <<= 1)
318         {
319             const int *o = w.data() + h;
320             for (int l = 0; l < n; l += 2 * h)
321             {
322                 for (int j = 0; j < h; j++)
323                 {
324                     unsigned long long x = buf[l + j];
325                     unsigned long long y = buf[l + j + h] * o[j] % mod;
326                     buf[l + j] = x + y;
327                     buf[l + j + h] = x + mod - y;
328                 }
329             }
330         }
331         if (!inv)
332         {
333             for (int i = 0; i < n; i++)
334             {
335                 a[i] = buf[i] % mod;
336             }
337             return;
338         }
339         int k = countr_zero((unsigned long long)n);
```

```
        int iv = mod - ((mod - 1) >> k);
        a[0] = buf[0] * iv % mod;
        for (int i = 1; i < n; i++)
        {
            a[i] = buf[n - i] * iv % mod;
        }
    }

  public:
    NTT32(int n = 0) : mx(n), w(n), buf(n)
    {
        // n 是最大变换长度，必须为 0 或不超过 2^17 的二次幂；预处理各级连续单位根。
        assert(n == 0 || (has_single_bit((unsigned long long)n) && n <= (1 << 17))); // 调试检查，可删。
        if (n == 0)
        {
            return;
        }
        if (n == 1)
        {
            w[0] = 1;
            return;
        }
        int r = Z(3).pow((mod - 1) / n).val();
        w[n / 2] = 1;
        for (int i = n / 2 + 1; i < n; i++)
        {
            w[i] = (unsigned long long)w[i - 1] * r % mod;
        }
        for (int i = n / 2 - 1; i; i--)
        {
            w[i] = w[i << 1];
        }
    }

    void dft(int *a, int n)
    {
        // a 指向长度 n 且各项在 [0,mod) 内的预分配切片；原地完成自然频点顺序的正变换，无返回值。
        trans(a, n, false);
    }

    void idft(int *a, int n)
    {
        // a 指向本对象 dft 产生的 n 个频域值；原地完成逆变换并除以 n，无返回值。
        trans(a, n, true);
    }
};

void dft(vector<Z> &a)
{
    // a 的长度是至多 2^23 的正 2 的幂；调用卡常正变换，结果满足 FastNTT 的递归配对排列。
    FastNTT::dft(a);
```

```
}

void dft(Z *a, int n)
{
    // a 指向长度 n 的连续系数；调用卡常正变换，结果满足 FastNTT 的递归配对排列。
    FastNTT::dft(a, n);
}

void idft(vector<Z> &a)
{
    // a 必须来自全局 dft；调用卡常逆变换并恢复自然顺序系数。
    FastNTT::idft(a);
}

void idft(Z *a, int n)
{
    // a 指向长度 n 且来自全局 dft 的频域值；调用卡常逆变换并恢复自然顺序系数。
    FastNTT::idft(a, n);
}

vector<Z> conv(vector<Z> a, vector<Z> b)
{
    // a、b 按低次到高次保存系数；返回卡常 NTT 计算的普通线性卷积。
    return FastNTT::conv(move(a), move(b));
}
```

当总长度超过单次 2^{23} NTT 上限时，把每个输入按 `NTT_LIM/2` 分块。每个输入块只做一次正变换；按块下标的和在频域累加乘积，再逐个逆变换输出块。这样设块数为 A, B 时变换次数为 $A + B + AB$（而不是对每个块对重复做两次正变换），峰值空间为 $O((A + B + 1)\, NTT_LIM)$，总时间为 $O((A + B + AB)\, NTT_LIM \log NTT_LIM)$。

```
vector<Z> convLarge(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 的总长可以超过单次 NTT 上限；返回分块线性卷积。
    if (a.empty() || b.empty()) return {};
    if (convFit(a.size(), b.size())) return conv(a, b);
    constexpr int block = NTT_LIM / 2;
    vector<Z> ans(a.size() + b.size() - 1);
    vector<vector<Z>> x, y;
    for (int i = 0; i < (int)a.size(); i += block)
    {
        vector<Z> v(a.begin() + i, a.begin() + min<int>(a.size(), i + block));
        v.resize(NTT_LIM);
        dft(v);
        x.push_back(move(v));
    }
    for (int i = 0; i < (int)b.size(); i += block)
    {
        vector<Z> v(b.begin() + i, b.begin() + min<int>(b.size(), i + block));
        v.resize(NTT_LIM);
        dft(v);
```

```
        y.push_back(move(v));
    }
    for (int s = 0; s + 1 < (int)x.size() + (int)y.size(); s++)
    {
        vector<Z> z(NTT_LIM);
        for (int i = max<int>(0, s - (int)y.size() + 1);
             i <= s && i < (int)x.size(); i++)
        {
            int j = s - i;
            for (int k = 0; k < NTT_LIM; k++)
            {
                z[k] += x[i][k] * y[j][k];
            }
        }
        idft(z);
        int offset = s * block;
        int len = min<int>(NTT_LIM, ans.size() - offset);
        for (int k = 0; k < len; k++)
        {
            ans[offset + k] += z[k];
        }
    }
    return ans;
}
```

下面的普通二进制蝶形版先做位逆序置换、再逐层 radix-2 蝶形，代码直观且频域值是自然顺序，适合调试或需要观察点值的场合。它是固定模数 `998244353` 的独立板子，只使用 `vector<int>`，不依赖 `Z`；系数可以是任意整数，卷积入口会先规约到模数范围。它与卡常版的中间排列不同，两者的正逆变换不能交叉配对。

```
namespace plain_ntt
{
constexpr int mod = 998244353, G = 3;

int qpow(int a, int b)
{
    // a 是底数，b 是非负指数；返回 a^b mod 998244353。
    int r = 1;
    for (; b; b >>= 1, a = (long long)a * a % mod)
    {
        if (b & 1) r = (long long)r * a % mod;
    }
    return r;
}

void dft(vector<int> &a, bool inv = false)
{
    // a 的长度是正的 2 的幂；原地完成自然顺序的正/逆 NTT。
    int n = a.size();
    assert(n > 0 && has_single_bit((unsigned long long)n)); // 调试检查，可删。
    for (int i = 1, j = 0; i < n; i++)
```

```
    {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1)
        {
            j ^= bit;
        }
        if ((j ^= bit) > i) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1)
    {
        int wlen = qpow(G, (mod - 1) / len);
        if (inv) wlen = qpow(wlen, mod - 2);
        for (int i = 0; i < n; i += len)
        {
            int w = 1;
            for (int j = 0; j < len / 2; j++)
            {
                int u = a[i + j], v = (long long)a[i + j + len / 2] * w % mod;
                a[i + j] = u + v < mod ? u + v : u + v - mod;
                a[i + j + len / 2] = u - v < 0 ? u - v + mod : u - v;
                w = (long long)w * wlen % mod;
            }
        }
    }
    if (inv)
    {
        int iv = qpow(n, mod - 2);
        for (int &x : a)
        {
            x = (long long)x * iv % mod;
        }
    }
}

void idft(vector<int> &a)
{
    // a 是自然顺序的频域值；原地完成逆 NTT 并除以长度。
    dft(a, true);
}

vector<int> conv(vector<int> a, vector<int> b)
{
    // a、b 按低次到高次保存系数；返回模 mod 的普通线性卷积。
    if (a.empty() || b.empty()) return {};
    int s = a.size() + b.size() - 1, n = 1;
    while (n < s)
    {
        n <<= 1;
    }
    for (int &x : a)
    {
        x = (x % mod + mod) % mod;
```

```
    }
    for (int &x : b)
    {
        x = (x % mod + mod) % mod;
    }
    a.resize(n), b.resize(n);
    dft(a), dft(b);
    for (int i = 0; i < n; i++)
    {
        a[i] = (long long)a[i] * b[i] % mod;
    }
    idft(a), a.resize(s);
    return a;
}
}
```

在 AMD Ryzen 9 7945HX、g++ 14.2.0、`-O2` 下，两边等长且系数独立随机，取四轮单次卷积的中位耗时（输入长 n 时实际变换长 $2n$）：

每边系数数 n	radix-4 卡常版	radix-2 普通版	加速比
32768	3.91 ms	5.47 ms	1.40x
131072	17.39 ms	25.26 ms	1.45x
262144	35.62 ms	52.69 ms	1.48x

同机、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 下，固定种子 `20260816`，每轮反复执行同样数量的正逆变换并取 7 次中位数；构造根表不计入计时：

变换长度	正逆对数	`FastNTT`	`NTT32`	加速比
64	32768	27.218 ms	15.383 ms	1.77x
256	8192	30.397 ms	18.495 ms	1.64x
1024	2048	35.486 ms	22.294 ms	1.59x
4096	512	41.219 ms	25.659 ms	1.61x
16384	128	46.546 ms	31.552 ms	1.48x
65536	32	52.093 ms	39.809 ms	1.31x
131072	16	54.827 ms	44.594 ms	1.23x

保留普通版是为了调试和自然顺序频域值，不把本机常数差异当成渐进复杂度差异。默认用全局 `dft`、`idft`、`conv`，调试时用 `plain_ntt::dft/idft/conv`；`NTT32` 只在调用方已经采用原始 32 位切片、固定小变换和复用工作区时使用。`FastNTT` 与普通版的变换长度须为正的 2 的幂且不超过 2^{23}，`NTT32` 的实际长度须整除构造时最大长度且不超过 2^{17}。长度 N 的变换为 $O(N \log N)$，卷积为 $O((n+m)\log(n+m))$，工作空间为 $O(N)$。模数不含所需单位根时用 10.1.004 MTT 任意模数卷积；按位卷积用 10.1.006 FWT / FST。

下面核对长度 100000 的确定性系数与滚动校验和：

```cpp
signed main()
{
    int n = 100000;
    vector<Z> a(n), b(n);
    for (int i = 0; i < n; i++)
    {
        a[i] = (i * i + 3 * i + 7) % mod;
        b[i] = (5 * i * i + 11) % mod;
    }
    auto c = conv(a, b);
    assert(c[0].val() == 77 && c[99999].val() == 971178005 && c[199998].val() == 532417205);
    unsigned long long h = 0;
    for (int i = 0; i < (int)c.size(); i++)
    {
        h = h * 1000003 + c[i].val() + i;
    }
    assert(h == 6385078395933506484ULL);
}
```

10.1.004 MTT 任意模数卷积

目标模数不含 NTT 需要的 2 的幂阶单位根时，`convMod` 先规范系数。短边不超过 64 时直接枚举；模数不超过 10^9+7 且变换长度不超过 2^{20} 时，分别在三个 NTT 素数下卷积，再用 Garner 重建后对目标模数规约，三个素数的乘积严格大于该范围内单个卷积系数上界。其余范围仅在误差界允许时用复数 FFT：把每个系数按 $B=2^{15}$ 拆成高低两段，分别卷积后重组；无法证明浮点安全时保守退回朴素实现。

```cpp
namespace arbModConv
{
struct NTT
{
    int mod, root; // mod 是素数模数，root 是其原根。

    int pow(int a, int n) const
    {
        // a 是模 mod 剩余，n 是非负指数；返回 a^n mod mod。
        unsigned long long ans = 1;
        while (n)
        {
            if (n & 1)
            {
                ans = ans * a % mod;
            }
            a = (unsigned long long)a * a % mod;
            n >>= 1;
        }
        return ans;
    }

    void dft(vector<int> &a, bool inv) const
    {
```

```
        // a 长度是支持范围内的 2 的幂；inv 为 false/true 时原地正/逆 NTT。
        int n = a.size();
        for (int i = 1, j = 0; i < n; i++)
        {
            int b = n >> 1;
            for (; j & b; b >>= 1)
            {
                j ^= b;
            }
            j ^= b;
            if (i < j)
            {
                swap(a[i], a[j]);
            }
        }
        for (int len = 2; len <= n; len <<= 1)
        {
            int w = pow(root, (mod - 1) / len);
            if (inv)
            {
                w = pow(w, mod - 2);
            }
            for (int l = 0; l < n; l += len)
            {
                unsigned long long cur = 1;
                for (int i = 0; i < len / 2; i++)
                {
                    int x = a[l + i];
                    int y = cur * a[l + i + len / 2] % mod;
                    a[l + i] = x + y < mod ? x + y : x + y - mod;
                    a[l + i + len / 2] = x >= y ? x - y : x + mod - y;
                    cur = cur * w % mod;
                }
            }
        }
        if (inv)
        {
            int iv = pow(n, mod - 2);
            for (auto &x : a)
            {
                x = (unsigned long long)x * iv % mod;
            }
        }
    }

    vector<int> conv(const vector<unsigned long long> &x,
                     const vector<unsigned long long> &y) const
    {
        // x、y 是普通整数系数；返回它们在 mod 下的线性卷积。
        int need = x.size() + y.size() - 1;
        int n = bit_ceil((unsigned long long)need);
        vector<int> a(n), b(n);
```

```
        for (int i = 0; i < (int)x.size(); i++)
        {
            a[i] = x[i] % mod;
        }
        for (int i = 0; i < (int)y.size(); i++)
        {
            b[i] = y[i] % mod;
        }
        dft(a, false);
        dft(b, false);
        for (int i = 0; i < n; i++)
        {
            a[i] = (unsigned long long)a[i] * b[i] % mod;
        }
        dft(a, true);
        a.resize(need);
        return a;
    }
};

inline int inv(int a, int mod)
{
    // a 与素数 mod 互素；返回 a 在模 mod 下的逆元。
    unsigned long long ans = 1, b = a;
    int e = mod - 2;
    while (e)
    {
        if (e & 1)
        {
            ans = ans * b % mod;
        }
        b = b * b % mod;
        e >>= 1;
    }
    return ans;
}
}

long long normConv(long long x, long long mod)
{
    // mod 是正模数；返回 x 在 [0,mod) 内的代表元。
    x %= mod;
    if (x < 0)
    {
        x += mod;
    }
    return x;
}

vector<long long> convNaiveMod(const vector<long long> &a, const vector<long long> &b,
                               long long mod)
{
```

```cpp
    // a、b 已规范到 [0,mod)，mod 是正模数；用 128 位乘加返回精确模卷积，任一输入为空时返回空。
    assert(mod > 0); // 调试检查，可删。
    if (a.empty() || b.empty())
    {
        return {};
    }
    vector<long long> c(a.size() + b.size() - 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = 0; j < (int)b.size(); j++)
        {
            c[i + j] = ((__int128)c[i + j] + (__int128)a[i] * b[j]) % mod;
        }
    }
    return c;
}

vector<long long> convNTTMod(const vector<long long> &a, const vector<long long> &b,
                             long long mod)
{
    // a、b 已规范到 [0,mod)，mod 不超过 1E9+7；用三个 NTT 素数返回精确模卷积。
    assert(0 < mod && mod <= 1000000007); // 调试检查，可删。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int need = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)need);
    assert(n <= (1 << 20)); // 调试检查，可删；所选三个素数都支持该长度。
    constexpr array<int, 3> prime{998244353, 985661441, 943718401};
    constexpr array<int, 3> root{3, 3, 7};
    vector<unsigned long long> x(a.begin(), a.end()), y(b.begin(), b.end());
    array<vector<int>, 3> rem;
    for (int k = 0; k < 3; k++)
    {
        rem[k] = arbModConv::NTT{prime[k], root[k]}.conv(x, y);
    }
    unsigned long long inv01 = arbModConv::inv(prime[0] % prime[1], prime[1]);
    unsigned long long p01mod2 = (unsigned long long)prime[0] * prime[1] % prime[2];
    unsigned long long inv012 = arbModConv::inv(p01mod2, prime[2]);
    unsigned long long p0mod = prime[0] % mod;
    unsigned long long p01mod = p0mod * (prime[1] % mod) % mod;
    vector<long long> ans(need);
    for (int i = 0; i < need; i++)
    {
        unsigned long long c0 = rem[0][i];
        unsigned long long c1 = (rem[1][i] + prime[1] - c0 % prime[1]) % prime[1] * inv01 %
            prime[1];
        unsigned long long seen2 = (c0 + (unsigned long long)prime[0] * c1) % prime[2];
        unsigned long long c2 = (rem[2][i] + prime[2] - seen2) % prime[2] * inv012 % prime[2];
        ans[i] = (c0 % mod + p0mod * (c1 % mod) + p01mod * (c2 % mod)) % mod;
    }
```

```cpp
    return ans;
}

vector<long long> convMod(vector<long long> a, vector<long long> b, long long mod)
{
    // a、b 是整数系数，mod 是正模数；返回最小非负代表元组成的线性卷积。
    assert(mod > 0); // 调试检查，可删。
    if (a.empty() || b.empty())
    {
        return {};
    }
    for (auto &x : a)
    {
        x = normConv(x, mod);
    }
    for (auto &x : b)
    {
        x = normConv(x, mod);
    }
    if (min(a.size(), b.size()) <= 64)
    {
        return convNaiveMod(a, b, mod);
    }
    int s = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)s);
    if (mod <= 1000000007 && n <= (1 << 20))
    {
        return convNTTMod(a, b, mod);
    }
    const long long B = 1 << 15;
    unsigned __int128 t = min(a.size(), b.size());
    unsigned __int128 hi = (mod - 1) / B;
    unsigned __int128 lim = (unsigned __int128)1 << 48;
    auto bounded = [&](unsigned __int128 x, unsigned __int128 y, unsigned __int128 z)
    {
        if (x && y > lim / x)
        {
            return false;
        }
        x *= y;
        return !x || z <= lim / x;
    };
    bool safe = numeric_limits<long double>::digits >= 64 && n <= (1 << 20) &&
                bounded(t, B - 1, B - 1) && bounded(t, 2 * (B - 1), hi) &&
                bounded(t, hi, hi);
    if (!safe)
    {
        return convNaiveMod(a, b, mod);
    }
    vector<long double> a0(a.size());
    vector<long double> a1(a.size());
    vector<long double> b0(b.size());
```

```
    vector<long double> b1(b.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        a0[i] = a[i] % B;
        a1[i] = a[i] / B;
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        b0[i] = b[i] % B;
        b1[i] = b[i] / B;
    }
    auto c0 = convolution(a0, b0);
    auto c1 = convolution(a0, b1);
    auto q = convolution(a1, b0);
    auto c2 = convolution(a1, b1);
    vector<long long> c(c0.size());
    for (int i = 0; i < (int)c.size(); i++)
    {
        long long x = llroundl(c0[i]) % mod;
        long long y = (llroundl(c1[i]) + llroundl(q[i])) % mod;
        long long z = llroundl(c2[i]) % mod;
        __int128 v = x;
        v += (__int128)y * B % mod;
        v += (__int128)z * B % mod * B % mod;
        c[i] = v % mod;
    }
    return c;
}
```

依赖 10.1.002 FFT 的 `convolution`，调用 `convMod(a,b,p)`；输入允许负数、`p` 须为正，任一输入为空时返回空，否则返回长度 $n+m-1$。三素数路径适用于 $p \le 10^9+7$、补齐长度 $N \le 2^{20}$，时间 $O(N \log N)$、峰值空间 $O(N)$，且因 $N(p-1)^2$ 小于三个素数乘积而能精确重建。其余快速路径要求 `long double` 至少有 64 位尾数，并按 $t=\min(n,m)$、$H=\lfloor (p-1)/2^{15} \rfloor$ 检查三段卷积的绝对系数上界均不超过 2^{48}；无法证明安全时才退回 `__int128` 朴素模卷积的 $O(nm)$。Windows GCC 的 `long double` 可能只有 53 位尾数，但常用十亿级模数仍走精确三素数路径，不会退化为平方复杂度。

下面检查负系数、模数 1、普通卷积和接近 `LLONG_MAX` 时的精确回退：

```
signed main()
{
    assert((convMod({-1, 2}, {3, -4}, 5) == vector<long long>{2, 0, 2}));
    assert((convMod({7, -8}, {9}, 1) == vector<long long>{0, 0}));
    assert((convMod({1, 2, 3}, {4, 5}, 1000000007) == vector<long long>{4, 13, 22, 15}));
    long long big = numeric_limits<long long>::max();
    assert(convMod({big - 1}, {big - 1}, big)[0] == 1);
}
```

10.1.005 二元 NTT / 二元多项式卷积

二维卷积用 Kronecker 打包降成一次一维卷积。令结果列数 $C = c_a + c_b - 1$，把二维下标 (i, j) 映到 $iC + j$；任意乘积项的列指数满足 $j_a + j_b < C$，不会向下一行产生进位，因此一维卷积下标恰好还原为二维次数。`poly2::Mat` 用连续行主序保存二维系数；普通 `poly2::conv(a,b)` 调一次公共 `conv`，三参数重载同时求 `a*c` 与 `b*c`，共享右因子 `c` 的正变换，只需三次正变换和两次逆变换。10.2.011 Power Projection 使用这个真实的第二调用场景，因此该连续矩阵层不是一次性包装。

```cpp
// 依赖 1.1.008 静态取模类
namespace poly2
{
struct Mat
{
    int n = 0; // 二维系数表的行数。
    int m = 0; // 二维系数表的列数。
    vector<Z> a; // 按行连续保存的 n*m 个系数。

    Mat(int r = 0, int c = 0) : n(r), m(c), a(r * c)
    {
        // r、c 是二维系数表的行数与列数；构造全零系数表。
    }

    Z &operator()(int i, int j)
    {
        // i、j 是行列下标；返回对应系数的可修改引用。
        return a[i * m + j];
    }

    const Z &operator()(int i, int j) const
    {
        // i、j 是行列下标；返回对应系数的只读引用。
        return a[i * m + j];
    }
};

vector<Z> pack(const Mat &a, int w)
{
    // a 是行主序二维系数表，w 不小于其列数；返回以 w 为行跨度的 Kronecker 打包结果。
    if (a.n == 0 || a.m == 0)
    {
        return {};
    }
    assert(w >= a.m); // 调试检查，可删。
    vector<Z> x((a.n - 1) * w + a.m);
    for (int i = 0; i < a.n; i++)
    {
        for (int j = 0; j < a.m; j++)
        {
            x[i * w + j] = a(i, j);
        }
    }
```

```cpp
    return x;
}

Mat unpack(const vector<Z> &a, int n, int m)
{
    // a 是行跨度为 m 的 Kronecker 系数，n、m 是目标形状；返回对应行主序二维系数表。
    if (n == 0 || m == 0)
    {
        return {};
    }
    assert((int)a.size() >= n * m); // 调试检查，可删。
    Mat c(n, m);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < m; j++)
        {
            c(i, j) = a[i * m + j];
        }
    }
    return c;
}

Mat conv(const Mat &a, const Mat &b)
{
    // a、b 是行主序二维系数表；用 Kronecker 打包返回二维线性卷积。
    if (a.n == 0 || a.m == 0 || b.n == 0 || b.m == 0)
    {
        return {};
    }
    int n = a.n + b.n - 1;
    int m = a.m + b.m - 1;
    auto c = ::conv(pack(a, m), pack(b, m));
    c.resize(n * m);
    return unpack(c, n, m);
}

array<Mat, 2> conv(const Mat &a, const Mat &b, const Mat &c)
{
    // a、b 是同形左因子，c 是公共右因子；返回 {a*c,b*c}，两次卷积共享 c 的正变换。
    assert(a.n == b.n && a.m == b.m); // 调试检查，可删。
    if (a.n == 0 || a.m == 0 || c.n == 0 || c.m == 0)
    {
        return {};
    }
    int n = a.n + c.n - 1;
    int m = a.m + c.m - 1;
    vector<Z> x = pack(a, m);
    vector<Z> y = pack(b, m);
    vector<Z> z = pack(c, m);
    if (min({x.size(), y.size(), z.size()}) <= 64)
    {
        return {unpack(::conv(x, z), n, m), unpack(::conv(y, z), n, m)};
```

```
    }
    int s = max(x.size() + z.size() - 1, y.size() + z.size() - 1);
    assert(nttFit(x.size(), z.size()) && nttFit(y.size(), z.size())); // 调试检查，可删。
    int len = bit_ceil((unsigned long long)s);
    x.resize(len);
    y.resize(len);
    z.resize(len);
    dft(x);
    dft(y);
    dft(z);
    for (int i = 0; i < len; i++)
    {
        x[i] *= z[i];
        y[i] *= z[i];
    }
    idft(x);
    idft(y);
    x.resize(n * m);
    y.resize(n * m);
    return {unpack(x, n, m), unpack(y, n, m)};
}
}

vector<vector<Z>> conv2(const vector<vector<Z>> &a, const vector<vector<Z>> &b)
{
    // a、b 是矩形二维系数表；返回二维线性卷积，任一维为空时返回空。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int m1 = a[0].size();
    int m2 = b[0].size();
    for (const auto &r : a)
    {
        assert((int)r.size() == m1); // 调试检查，可删。
    }
    for (const auto &r : b)
    {
        assert((int)r.size() == m2); // 调试检查，可删。
    }
    if (m1 == 0 || m2 == 0)
    {
        return {};
    }
    poly2::Mat x(a.size(), m1), y(b.size(), m2);
    for (int i = 0; i < x.n; i++)
    {
        for (int j = 0; j < x.m; j++)
        {
            x(i, j) = a[i][j];
        }
    }
```

```
    for (int i = 0; i < y.n; i++)
    {
        for (int j = 0; j < y.m; j++)
        {
            y(i, j) = b[i][j];
        }
    }
    auto z = poly2::conv(x, y);
    vector<vector<Z>> ans(z.n, vector<Z>(z.m));
    for (int i = 0; i < z.n; i++)
    {
        for (int j = 0; j < z.m; j++)
        {
            ans[i][j] = z(i, j);
        }
    }
    return ans;
}
```

依赖 10.1.003 NTT 的 `conv`、`dft` 和 `idft`。外部接口 `conv2(a,b)` 接受矩形 `vector<vector<Z>>`；内部需要连续布局或共享右因子时使用 `poly2::Mat`。任一输入没有行或列时返回空，否则结果两维各为输入之和减一。设输出矩形为 $R \times C$，打包长度为 $\Theta(RC)$，时间 $O(RC\log(RC))$、额外空间 $O(RC)$；三参数重载与两次独立二维卷积同阶，但少做一次公共右因子的正变换。实际规模仍受一维 NTT 的 2^{23} 长度上限约束。

10.1.006 FWT / FST

设两序列长度同为 $N = 2^k$，按位卷积定义为 $c_s = \sum_{i \circ j = s} a_i b_j$，其中 $\circ$ 为 OR、AND 或 XOR。OR 变换计算子集和 $A_s = \sum_{t \subseteq s} a_t$，AND 变换计算超集和 $A_s = \sum_{t \supseteq s} a_t$；交换求和顺序即得卷积变为逐点乘法。两者的蝶形分别为 $(x, y) \mapsto (x, x + y)$ 与 $(x, y) \mapsto (x + y, y)$，逆变换将加法改为减法。XOR 使用字符 $(-1)^{\mathrm{popcount}(s \& t)}$，由字符乘法对应下标异或得到 Hadamard 蝶形 $(x, y) \mapsto (x + y, x - y)$；矩阵平方为 NI，故逆变换再除以 N。三种变换必须使用各自的逆。

优先调用 `orFwt`、`andFwt`、`xorFwt` 与对应的 `orConv`、`andConv`、`xorConv`。`orFWT` 另支持每个掩码连续保存 d 个通道的批量入口。`BitOp` 与 `bitConv` 保留兼容。OR、AND 只需系数支持加减，卷积还需乘法；XOR 逆变换要求 N 可逆，整数系数则要求最终每项能被 N 整除。整数计算须保证全部中间值不溢出；模整数依赖 1.1.008 静态取模类，其除法接口要求素模。

```
enum class BitOp
{
    Or,
    And,
    Xor
};

template <bool Inv = false, class T> void orFWT(T *a, int n, int d)
{
```

```
    // a 提供 n*d 项连续空间，每行 d 个通道，n 是至多 2^30 的正 2 的幂；Inv 指定逆变换，原地修改，无返回值。
    assert(a && n > 0 && d > 0 && n <= (1ULL << 30) && d <= 2147483647ULL &&
           (size_t)n <= numeric_limits<size_t>::max() / sizeof(T) / (size_t)d &&
           has_single_bit((unsigned long long)n)); // 调试检查，可删。
    int m = n;
    int w = d;
    for (int h = 1; h < m; h <<= 1)
    {
        for (int l = 0; l < m; l += h << 1)
        {
            for (int s = 0; s < h; s++)
            {
                T *x = a + (size_t)(l + s) * w;
                T *y = x + (size_t)h * w;
                if constexpr (!Inv)
                {
                    for (int k = 0; k < w; k++)
                    {
                        y[k] += x[k];
                    }
                }
                else
                {
                    for (int k = 0; k < w; k++)
                    {
                        y[k] -= x[k];
                    }
                }
            }
        }
    }
}

template <class T> void orFWT(T *a, int n, int d, bool inv)
{
    // a 提供 n*d 项连续空间，每行 d 个通道；inv=false/true 指定正/逆 OR 变换，原地修改，无返回值。
    if (inv)
    {
        orFWT<true>(a, n, d);
    }
    else
    {
        orFWT<false>(a, n, d);
    }
}

template <bool Inv = false, class T> void orFWT(vector<T> &a)
{
    // a 按掩码编号，长度是至多 2^30 的正 2 的幂；Inv 指定逆变换，原地修改，无返回值。
    assert(!a.empty() && a.size() <= (1ULL << 30) && has_single_bit(a.size())); // 调试检查，可删。
```

```
    int n = a.size();
    for (int h = 1; h < n; h <<= 1)
    {
        for (int i = 0; i < n; i += h << 1)
        {
            for (int j = 0; j < h; j++)
            {
                if constexpr (!Inv)
                {
                    a[i + j + h] += a[i + j];
                }
                else
                {
                    a[i + j + h] -= a[i + j];
                }
            }
        }
    }
}

template <class T> void fwt(vector<T> &a, BitOp op, bool inv = false)
{
    // a 是按掩码编号的系数，op 指定或、与或异或，inv 指定正变换或逆变换；原地完成变换。
    assert(!a.empty() && a.size() <= (1ULL << 30) && has_single_bit(a.size()));
    int n = a.size();
    if (op == BitOp::Or)
    {
        if (inv)
        {
            orFWT<true>(a);
        }
        else
        {
            orFWT<false>(a);
        }
        return;
    }
    for (int h = 1; h < n; h <<= 1)
    {
        for (int i = 0; i < n; i += h << 1)
        {
            for (int j = 0; j < h; j++)
            {
                T x = a[i + j];
                T y = a[i + j + h];
                if (op == BitOp::Xor)
                {
                    a[i + j] = x + y;
                    a[i + j + h] = x - y;
                }
                else
                {
```

```
                    if (!inv)
                    {
                        a[i + j] += y;
                    }
                    else
                    {
                        a[i + j] -= y;
                    }
                }
            }
        }
    }
    if (op == BitOp::Xor && inv)
    {
        for (T &x : a)
        {
            x /= T(n);
        }
    }
}

// 方向由函数名表达；BitOp 版本保留给旧代码。
template <class T> void orFwt(vector<T> &a, bool inv = false)
{
    // a 按掩码编号，长度是正 2 的幂；inv=false/true 指定正/逆 OR 变换，原地修改，无返回值。
    fwt(a, BitOp::Or, inv);
}
template <class T> void andFwt(vector<T> &a, bool inv = false)
{
    // a 按掩码编号，长度是正 2 的幂；inv=false/true 指定正/逆 AND 变换，原地修改，无返回值。
    fwt(a, BitOp::And, inv);
}
template <class T> void xorFwt(vector<T> &a, bool inv = false)
{
    // a 按掩码编号，长度是正 2 的幂；inv=false/true 指定正/逆 XOR 变换，原地修改，无返回值。
    fwt(a, BitOp::Xor, inv);
}

template <class T> vector<T> bitConv(vector<T> a, vector<T> b, BitOp op)
{
    // a、b 是等长的掩码系数，op 指定卷积下标使用或、与或异或；返回对应的按位卷积。
    assert(!a.empty() && a.size() == b.size()); // 调试检查，可删。
    fwt(a, op);
    fwt(b, op);
    for (int i = 0; i < (int)a.size(); i++)
    {
        a[i] *= b[i];
    }
    fwt(a, op, true);
    return a;
}

```

```cpp
template <class T> vector<T> orConv(vector<T> a, vector<T> b)
{
    // a、b 等长且长度是正 2 的幂；返回按掩码顺序的 OR 卷积，不修改调用者的输入。
    return bitConv(move(a), move(b), BitOp::Or);
}
template <class T> vector<T> andConv(vector<T> a, vector<T> b)
{
    // a、b 等长且长度是正 2 的幂；返回按掩码顺序的 AND 卷积，不修改调用者的输入。
    return bitConv(move(a), move(b), BitOp::And);
}
template <class T> vector<T> xorConv(vector<T> a, vector<T> b)
{
    // a、b 等长且长度是正 2 的幂；返回按掩码顺序的 XOR 卷积，不修改调用者的输入。
    return bitConv(move(a), move(b), BitOp::Xor);
}
```

编译期模数 $1 < P < 2^{31}$ 时，下面的专用内核直接处理归一化代表元，按编译期长度和运算种类实例化蝶形。OR、AND 允许合数与偶数模；XOR 要求 P 为奇数，利用 $u/2 \equiv (u + (u \bmod 2)P)/2 \pmod P$ 逐层逆变换。性能版使用紧凑代表元，打印版使用 `int`，加法与除二前的加宽保持显式；固定模内核不复制模幂或模逆元。

```cpp
template <int P> int fwtAdd(int x, int y)
{
    // x、y 在 [0,P) 内，1<P<2^31；返回 (x+y) mod P，加宽中间和以兼容有符号打印版。
    unsigned long long s = (unsigned long long)x + y;
    return s >= P ? s - P : s;
}

template <int P> int fwtSub(int x, int y)
{
    // x、y 在 [0,P) 内，1<P<2^31；返回 (x-y) mod P，各分支结果均在 [0,P)。
    return x >= y ? x - y : x + (P - y);
}

template <int P, BitOp Op, bool Inv, int N> void fwtRec(int *a)
{
    // a 指向 N 项 [0,P) 代表元；N 是正 2 的幂，Op 指定位运算，Inv 指定逆变换；原地修改，无返回值。
    constexpr int h = N / 2;
    if constexpr (h)
    {
        fwtRec<P, Op, Inv, h>(a);
        fwtRec<P, Op, Inv, h>(a + h);
        for (int i = 0; i < h; i++)
        {
            int x = a[i], y = a[i + h];
            if constexpr (Op == BitOp::Or)
            {
                a[i + h] = Inv ? fwtSub<P>(y, x) : fwtAdd<P>(x, y);
            }
            else if constexpr (Op == BitOp::And)
            {
```

```
                a[i] = Inv ? fwtSub<P>(x, y) : fwtAdd<P>(x, y);
            }
            else
            {
                int s = fwtAdd<P>(x, y);
                int d = fwtSub<P>(x, y);
                if constexpr (Inv)
                {
                    s = ((unsigned long long)s + (s & 1) * P) >> 1;
                    d = ((unsigned long long)d + (d & 1) * P) >> 1;
                }
                a[i] = s;
                a[i + h] = d;
            }
        }
    }
}

template <int P, BitOp Op, bool Inv = false, int N = 1>
void fwtMod(int *a, int n)
{
    // a 指向 n 项 [0,P) 代表元，n 是至多 2^30 的正 2 的幂；Op 指定位运算，Inv 指定逆变换。
    // N 是递归分派起点，调用时保持默认值；原地修改 a，无返回值。
    static_assert(1 < P && P < (1U << 31));
    static_assert(Op != BitOp::Xor || (P & 1));
    static_assert(N > 0 && N <= (1U << 30) && has_single_bit((unsigned long long)N));
    assert(a && n >= N && n <= (1U << 30) && has_single_bit((unsigned long long)n));
    if (n == N)
    {
        fwtRec<P, Op, Inv, N>(a);
    }
    else if constexpr (N < (1U << 30))
    {
        fwtMod<P, Op, Inv, N << 1>(a, n);
    }
}

template <int P, BitOp Op, bool Inv = false> void fwtMod(vector<int> &a)
{
    // a 按掩码保存 [0,P) 代表元；Op 指定位运算，Inv 指定逆变换，原地修改，无返回值。
    assert(!a.empty() && a.size() <= (1ULL << 30) && has_single_bit(a.size()));
    fwtMod<P, Op, Inv>(a.data(), a.size());
}

template <int P, bool Inv = false> void orFwtMod(vector<int> &a)
{
    // a 按掩码保存 [0,P) 代表元；Inv 指定逆 OR 变换，原地修改，无返回值。
    fwtMod<P, BitOp::Or, Inv>(a);
}
template <int P, bool Inv = false> void andFwtMod(vector<int> &a)
{
    // a 按掩码保存 [0,P) 代表元；Inv 指定逆 AND 变换，原地修改，无返回值。
```

```cpp
    fwtMod<P, BitOp::And, Inv>(a);
}
template <int P, bool Inv = false> void xorFwtMod(vector<int> &a)
{
    // a 按掩码保存 [0,P) 代表元；Inv 指定逆 XOR 变换，原地修改，无返回值。
    fwtMod<P, BitOp::Xor, Inv>(a);
}

template <int P, BitOp Op>
vector<int> bitConvMod(vector<int> a, vector<int> b)
{
    // a、b 等长，长度是正 2 的幂，各项在 [0,P)；返回 c[i Op j]+=a[i]*b[j] 的模 P 代表元。
    assert(!a.empty() && a.size() == b.size()); // 调试检查，可删
    fwtMod<P, Op>(a);
    fwtMod<P, Op>(b);
    for (int i = 0; i < a.size(); i++)
    {
        a[i] = (unsigned long long)a[i] * b[i] % P;
    }
    fwtMod<P, Op, true>(a);
    return a;
}

template <int P> vector<int> orConvMod(vector<int> a, vector<int> b)
{
    // a、b 等长，长度是正 2 的幂，各项在 [0,P)；返回模 P 的 OR 卷积，不修改调用者输入。
    return bitConvMod<P, BitOp::Or>(move(a), move(b));
}
template <int P> vector<int> andConvMod(vector<int> a, vector<int> b)
{
    // a、b 等长，长度是正 2 的幂，各项在 [0,P)；返回模 P 的 AND 卷积，不修改调用者输入。
    return bitConvMod<P, BitOp::And>(move(a), move(b));
}
template <int P> vector<int> xorConvMod(vector<int> a, vector<int> b)
{
    // a、b 等长，长度是正 2 的幂，各项在 [0,P)；返回模 P 的 XOR 卷积，不修改调用者输入。
    return bitConvMod<P, BitOp::Xor>(move(a), move(b));
}
```

所有入口要求长度为正二次幂且不超过 2^{30}，不会自动补零。批量入口还要求 $0 < d \le 2^{31}-1$，调用者实际提供 Nd 个连续元素，且总字节数可由 `size_t` 表示。单序列变换耗时 $O(N \log N)$；通用迭代版额外空间 $O(1)$，固定模递归版栈空间 $O(\log N)$。卷积返回按掩码排列的 N 项，耗时 $O(N \log N)$、额外空间 $O(N)$；批量 OR 变换耗时 $O(Nd \log N)$、额外空间 $O(1)$。

性能头另保留 `sparseXor(n,factors)` 处理稀疏因子的 XOR 卷积，固定模数为 `1000000000000125953`，通过局部 Walsh 变换与平方根分解聚合因子；该接口不是上述稠密卷积的别名。

偶数模的 XOR 卷积使用 10.1.007 偶模 FWT 与光速乘；每位超过两种取值用 10.1.008 K 进制 FWT；要求拆分集合不交时用 10.1.011 子集卷积。下面直接核对已知结果，并用正逆变换检查恢复：

```cpp
signed main()
{
    vector<Z> a{1, 2, 3, 4}, b{5, 6, 7, 8};
    assert((xorConv(a, b) == vector<Z>{70, 68, 62, 60}));
    assert((orConv(a, b) == vector<Z>{5, 28, 43, 184}));
    assert((andConv(a, b) == vector<Z>{103, 52, 73, 32}));
    xorFwt(a);
    xorFwt(a, true);
    assert((a == vector<Z>{1, 2, 3, 4}));
    vector<int> x{1, 2, 3, 4}, y{5, 6, 7, 8};
    assert((orConvMod<10>(x, y) == vector<int>{5, 8, 3, 4}));
    assert((xorConvMod<15>(x, y) == vector<int>{10, 8, 2, 0}));
    cout << "OK\n";
    return 0;
}
```

10.1.007 偶模 FWT 与光速乘

偶模数下 2 不可逆，普通异或逆 FWT 末尾的除以 n 会失败。把模数临时提升到 $M = n \cdot mod$，在 $\mathbb{Z}/M\mathbb{Z}$ 上做正 FWT、逐点乘、再做一次正 FWT；异或 FWT 的平方是 nI，所得每项在 $[0, M)$ 内都恰能被 n 整除，先作整数除法再模 mod 即可。下面用 `unsigned __int128` 保存 M，模加减避免先形成可能溢出的和，模乘在 $M > 2^{64}$ 时改用倍增加法。

```cpp
unsigned __int128 addMod128(unsigned __int128 x, unsigned __int128 y,
                            unsigned __int128 mod)
{
    // x、y 均小于正模数 mod；返回 (x+y) mod mod，计算中不发生无符号回绕。
    return x >= mod - y ? x - (mod - y) : x + y;
}

unsigned __int128 subMod128(unsigned __int128 x, unsigned __int128 y,
                            unsigned __int128 mod)
{
    // x、y 均小于正模数 mod；返回 (x-y) mod mod。
    return x >= y ? x - y : mod - (y - x);
}

unsigned __int128 mulMod128(unsigned __int128 x, unsigned __int128 y,
                            unsigned __int128 mod)
{
    // x、y 均小于正模数 mod；返回 (x*y) mod mod。
    if (mod <= ((unsigned __int128)1 << 64))
    {
        return x * y % mod;
    }
    unsigned __int128 r = 0;
    while (y)
    {
        if (y & 1)
        {
```

```
            r = addMod128(r, x, mod);
        }
        y >>= 1;
        if (y)
        {
            x = addMod128(x, x, mod);
        }
    }
    return r;
}

void xorFwt(vector<unsigned __int128> &a, unsigned __int128 mod)
{
    // a 的各项均小于正模数 mod；原地完成异或 FWT。
    int n = a.size();
    for (int len = 1; len < n; len <<= 1)
    {
        for (int l = 0; l < n; l += len << 1)
        {
            for (int i = 0; i < len; i++)
            {
                unsigned __int128 x = a[l + i];
                unsigned __int128 y = a[l + len + i];
                a[l + i] = addMod128(x, y, mod);
                a[l + len + i] = subMod128(x, y, mod);
            }
        }
    }
}

vector<long long> xorConv(const vector<long long> &a, const vector<long long> &b,
                          long long mod)
{
    // a、b 是等长整数序列，mod 是正模数；返回最小非负代表元组成的异或卷积。
    assert(mod > 0 && !a.empty() && a.size() == b.size() &&
           has_single_bit(a.size())); // 调试检查，可删。
    int n = a.size();
    unsigned __int128 lifted = (unsigned __int128)(unsigned long long)mod * n;
    vector<unsigned __int128> x(n);
    vector<unsigned __int128> y(n);
    for (int i = 0; i < n; i++)
    {
        long long u = a[i] % mod;
        long long v = b[i] % mod;
        if (u < 0)
        {
            u += mod;
        }
        if (v < 0)
        {
            v += mod;
        }
```

```
        x[i] = (unsigned long long)u;
        y[i] = (unsigned long long)v;
    }
    xorFwt(x, lifted);
    xorFwt(y, lifted);
    for (int i = 0; i < n; i++)
    {
        x[i] = mulMod128(x[i], y[i], lifted);
    }
    xorFwt(x, lifted);
    vector<long long> c(n);
    for (int i = 0; i < n; i++)
    {
        assert(x[i] % n == 0); // 调试检查，可删。
        c[i] = (long long)(x[i] / n % (unsigned long long)mod);
    }
    return c;
}
```

`xorConv(a,b,mod)` 接受负系数，要求两输入等长、非空且长度为 2 的幂，`mod` 为正的有符号 64 位整数；实现使用提升模数 $M = n \cdot mod$，要求 $M < 2^{128}$。通常 $M \le 2^{64}$，逐点模乘可直接用 `unsigned __int128`，总时间 $O(n \log n)$；更大的 M 用二进制模乘，时间为 $O(n(\log n + \log M))$。额外空间均为 $O(n)$。模数为奇数且长度在模下可逆时直接用 10.1.006 FWT / FST 更短。

下面分别覆盖普通偶模、模数 1 和提升模数超过 2^{64} 的分支：

```
signed main()
{
    assert((xorConv({1, 2, 3, 0}, {3, -1, 2, 1}, 4) ==
            vector<long long>{3, 0, 1, 2}));
    assert((xorConv({7, -8}, {9, 10}, 1) == vector<long long>{0, 0}));
    long long m = numeric_limits<long long>::max() - 1;
    assert((xorConv({m - 1, m - 2}, {m - 3, m - 4}, m) ==
            vector<long long>{11, 10}));
    cout << "OK\n";
    return 0;
}
```

10.1.008 K 进制 FWT

把每位两种取值推广到 k 种：在每一位上做 k 点 DFT，使逐位模 k 加法（无进位）的循环卷积对角化。系数域必须含 k 次单位根 $\omega_k = g^{(P-1)/k}$，即要求 $k \mid P-1$；逆变换用 ω_k^{-1} 并除以总长 n。

```
// 依赖 1.1.008 静态取模类
void kFWT(vector<Z> &a, int k, bool inv = false)
{
    // a 按 k 进制向量编号，k 是每一位的取值数；inv 指定正变换或逆变换，结果写回 a。
    assert(!a.empty() && k > 1 && (Z::mod() - 1) % k == 0); // 调试检查，可删。
    int n = a.size();
    int q = n;
```

```
    while (q % k == 0)
    {
        q /= k;
    }
    assert(q == 1); // 调试检查，可删。
    Z w = Z(3).pow((Z::mod() - 1) / k);
    if (inv)
    {
        w = w.inv();
    }
    vector<Z> pw(k, 1);
    vector<Z> x(k);
    vector<Z> y(k);
    for (int i = 1; i < k; i++)
    {
        pw[i] = pw[i - 1] * w;
    }
    for (int len = 1; len < n; len *= k)
    {
        for (int l = 0; l < n; l += len * k)
        {
            for (int p = 0; p < len; p++)
            {
                for (int i = 0; i < k; i++)
                {
                    x[i] = a[l + i * len + p];
                }
                fill(y.begin(), y.end(), Z(0));
                for (int i = 0; i < k; i++)
                {
                    for (int j = 0; j < k; j++)
                    {
                        y[i] += x[j] * pw[i * j % k];
                    }
                }
                for (int i = 0; i < k; i++)
                {
                    a[l + i * len + p] = y[i];
                }
            }
        }
    }
    if (inv)
    {
        Z q = Z(n).inv();
        for (Z &x : a)
        {
            x *= q;
        }
    }
}
```

本章公共前导固定 $P = 998244353$、原根 3。调用 `kFWT(a,k,inv)` 时长度须为 k 的非负整数次幂且 $k \mid P-1$；正变换后逐点相乘，再以 `inv=true` 逆变换。预存一轮单位根幂后，长度 N 的变换为 $O(kN\log_k N + k\log P)$ 时间、$O(k)$ 临时空间。$k=2$ 即退化为 10.1.006 FWT / FST 的异或卷积。

10.1.009 全在线卷积：基准版与 NTT 版

两序列系数同步逐项到达时，第 n 次加入后要立即得到 $c_n = \sum_{i=0}^{n} a_i b_{n-i}$。`OnlineConv` 直接枚举本项，适合小规模；`OnlineNTT` 按 `p+2` 的最低位把新贡献分成二进制块，复用 10.1.003 NTT 批量更新当前及未来答案，仍在每次 `add` 结束时立即返回 c_p。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
template <class T> struct OnlineConv
{
    vector<T> a; // 第一序列已经加入的系数。
    vector<T> b; // 第二序列已经加入的系数。
    vector<T> c; // 从第 0 项到当前项的卷积系数。

    T add(T x, T y)
    {
        // x、y 分别是两序列新到达的系数；追加二者并返回刚确定的卷积项。
        int n = a.size();
        a.push_back(x);
        b.push_back(y);
        c.push_back({});
        for (int i = 0; i <= n; i++)
        {
            c[n] += a[i] * b[n - i];
        }
        return c[n];
    }
};

struct OnlineNTT
{
    // a、b 是已加入系数，h 是已经批量累加的卷积项。
    vector<Z> a, b, h, x, y;
    // fa[z]、fb[z] 保存长度 2^(z+1) 的前缀变换。
    vector<vector<Z>> fa, fb;
    int p = 0;

    Z add(Z u, Z v)
    {
        // u、v 是下标 p 的新系数；追加后立即返回第 p 项卷积。
        a.push_back(u);
        b.push_back(v);
        int z = countr_zero((unsigned long long)(p + 2));
        int w = 1LL << z;
        int s;
        assert(2 * w <= (1 << 23)); // 调试检查，可删：受当前 NTT 最大长度限制。
```

```
        if (p + 2 == w)
        {
            x = a;
            x.resize(2 * w);
            dft(x);
            fa.emplace_back(x.begin(), x.begin() + w);
            y = b;
            y.resize(2 * w);
            dft(y);
            fb.emplace_back(y.begin(), y.begin() + w);
            for (int i = 0; i < 2 * w; i++)
            {
                x[i] *= y[i];
            }
            s = w - 2;
            h.resize(2 * s + 2);
        }
        else
        {
            x.assign(a.end() - w, a.end());
            x.resize(2 * w);
            dft(x);
            y.assign(b.end() - w, b.end());
            y.resize(2 * w);
            dft(y);
            for (int i = 0; i < 2 * w; i++)
            {
                x[i] = x[i] * fb[z][i] + y[i] * fa[z][i];
            }
            s = w - 1;
        }
        idft(x);
        for (int i = 0; i <= s; i++)
        {
            h[p + i] += x[s + i];
        }
        return h[p++];
    }
};
```

两个版本的 `add(x,y)` 都立即返回新确定的卷积项。基准版第 n 项 $O(n)$、前 n 项累计 $O(n^2)$；NTT 版累计 $O(n\log^2 n)$ 时间，各层缓存长度构成几何级数，故总空间为 $O(n)$。后者仅支持模 998244353 的 `z`，且受当前 2^{23} 阶单位根限制。项数不大时基准版更短、常数也更小。

10.1.010 整除关系卷积

在下标的整除偏序上做卷积。约数方向 Zeta 变换令 $b_n=\sum_{d\mid n}a_d$，Möbius 逆用减法回收；倍数方向令 $b_n=\sum_{n\mid m}a_m$，逆同理。它们是位偏序 FWT 的数论类比，配合逐点乘可算 GCD/LCM 卷积。四个变换都按 “枚举因子—枚举其倍数” 推进，Zeta 与 Möbius 的循环方向相反才能保证就地正确。

```cpp
template <class T> void divisorZeta(vector<T> &a)
{
    // a[n] 是下标 n 的系数；原地改成所有约数系数之和，a[0] 不参与。
    int n = a.size();
    for (int d = n - 1; d >= 1; d--)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[m] += a[d];
        }
    }
}

template <class T> void divMob(vector<T> &a)
{
    // a[n] 是约数 Zeta 变换后的系数；原地恢复原序列，a[0] 不参与。
    int n = a.size();
    for (int d = 1; d < n; d++)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[m] -= a[d];
        }
    }
}

template <class T> void multipleZeta(vector<T> &a)
{
    // a[n] 是下标 n 的系数；原地改成所有倍数系数之和，a[0] 不参与。
    int n = a.size();
    for (int d = 1; d < n; d++)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[d] += a[m];
        }
    }
}

template <class T> void mulMob(vector<T> &a)
{
    // a[n] 是倍数 Zeta 变换后的系数；原地恢复原序列，a[0] 不参与。
    int n = a.size();
    for (int d = n - 1; d >= 1; d--)
    {
        for (int m = d * 2; m < n; m += d)
        {
            a[d] -= a[m];
        }
    }
}
```

`divisorZeta`/`divMob` 是约数方向的正逆变换，`multipleZeta`/`mulMob` 是倍数方向。四者均 $O(n \log n)$ 时间（调和级数求和）、$O(1)$ 额外空间。位集合上的对应变换见 10.1.006 FWT / FST。

10.1.011 子集卷积

子集卷积 $c_S = \sum_{T \subseteq S} a_T b_{S \setminus T}$ 要求两部分不交且并为 S，不能用或卷积逐点乘替代（后者会重复计入有交的拆分）。技巧是引入占位维 popcount：把 a_S 放到第 $|S|$ 层，对每层做或方向子集和（zeta），在每个掩码上按层做加法卷积 $h_k = \sum_i f_i g_{k-i}$，再 Möbius 逆回收；因为不交时 $|T| + |S \setminus T| = |S|$ 恰落在第 $|S|$ 层，取该层即排除所有有交拆分。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.1.006 FWT / FST
template <class T> struct Ranked
{
    int n = 0; // 集合变量数，掩码范围是 [0,2^n)。
    vector<T> a; // 按掩码优先连续保存，每个掩码依次存放 0..n 秩。

    explicit Ranked(int m = 0) : n(m)
    {
        // m 是集合变量数；构造全零秩表。
        assert(0 <= m && m <= 30); // 调试检查，可删。
        a.resize((1LL << m) * (m + 1));
    }

    T &operator()(int s, int k)
    {
        // s 是集合掩码，k 是秩；返回对应元素的可修改引用。
        return a[s * (n + 1) + k];
    }

    const T &operator()(int s, int k) const
    {
        // s 是集合掩码，k 是秩；返回对应元素的只读引用。
        return a[s * (n + 1) + k];
    }
};

template <class T> Ranked<T> rankedZeta(const vector<T> &a)
{
    // a 是长度为 2 的幂的集合函数；返回按子集大小分层后的子集 Zeta 变换。
    assert(!a.empty() && has_single_bit(a.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)a.size());
    int m = a.size();
    int d = n + 1;
    Ranked<T> r(n);
    for (int s = 0; s < m; s++)
    {
        r.a[(size_t)s * d + popcount((unsigned long long)s)] = a[s];
    }
    orFWT<false>(r.a.data(), m, d);
```

```cpp
    return r;
}

template <class T> vector<T> rankedMobius(Ranked<T> r)
{
    // r 是按子集大小分层的 Zeta 变换；返回每个集合对应秩层的 Möbius 逆变换系数。
    int n = r.n;
    assert(0 <= n && n <= 30); // 调试检查，可删。
    int m = 1LL << n;
    assert((int)r.a.size() == m * (n + 1)); // 调试检查，可删。
    orFWT<true>(r.a.data(), m, n + 1);
    vector<T> a(m);
    for (int s = 0; s < (int)m; s++)
    {
        a[s] = r.a[(size_t)s * (n + 1) + popcount((unsigned long long)s)];
    }
    return a;
}

template <class T> vector<T> subsetConv(const vector<T> &a, const vector<T> &b)
{
    // a、b 是等长且长度为 2 的幂的集合函数；返回 c[S]=sum(T subset S) a[T]*b[S\T]。
    assert(a.size() == b.size() && !a.empty() && has_single_bit(a.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)a.size());
    int m = a.size();
    int d = n + 1;
    auto f = rankedZeta(a);
    auto g = rankedZeta(b);
    for (int s = 0; s < m; s++)
    {
        T *x = f.a.data() + (size_t)s * d;
        const T *y = g.a.data() + (size_t)s * d;
        for (int k = n; k >= 0; k--)
        {
            T v{};
            for (int i = 0; i <= k; i++)
            {
                v += x[i] * y[k - i];
            }
            x[k] = v;
        }
    }
    return rankedMobius(move(f));
}
```

`Ranked` 只保存一块按掩码优先的 $(n+1)2^n$ 连续缓冲区，`rankedZeta` 与 `rankedMobius` 统一调用 10.1.006 FWT / FST 的批量公共入口；这里没有第二份私有变换。`subsetConv(a,b)` 的长度须为 2^n 且当前实现要求 $0 \le n \le 30$，`T` 支持加减乘，下标二进制位表示集合；`a={1,2,3,4}`、`b={5,6,7,8}` 得 `{5,16,22,60}`。时间 $O(n^2 2^n)$、额外空间 $O(n2^n)$。或、与、异或卷积不要求两部分不交，分别使用 10.1.006。

下面直接核对空集合和两个二进制位都参与的固定结果：

```
signed main()
{
    assert((subsetConv(vector<Z>{2}, vector<Z>{3}) == vector<Z>{6}));
    assert((subsetConv(vector<Z>{1, 2, 3, 4}, vector<Z>{5, 6, 7, 8}) ==
            vector<Z>{5, 16, 22, 60}));
    cout << "OK\n";
    return 0;
}
```

10.1.012 Min-Plus 卷积

Min-plus 卷积 $c_k = \min_i(a_i + b_{k-i})$ 一般没有 FFT 型加速，只在附加结构下才快。两序列差分都单调不降（离散凸）时，答案差分是两条输入差分的有序归并；一凸一任意时，Monge 性保证最优决策点随 k 单调，可用分治搜索每行最优。无结构时只能枚举所有合法下标对。默认输入、差分和返回值使用 `long long`；可能超过 64 位时显式选择 `Acc=__int128`，不拿有限常量冒充无穷大。

```
template <class F> vector<int> monoMin(int rows, int cols, F better)
{
    // better(r,x,y) 判断第 r 行的候选 y 是否不劣于 x；返回每行的最优列下标。
    assert(rows > 0 && cols > 0); // 调试检查，可删。
    vector<int> p(rows);
    auto dfs = [&](auto &&self, int u, int d, int l, int r) -> void
    {
        if (u == d)
        {
            return;
        }
        int x = (u + d) / 2;
        int y = l;
        for (int j = l + 1; j < r; j++)
        {
            if (better(x, y, j))
            {
                y = j;
            }
        }
        p[x] = y;
        self(self, u, x, l, y + 1);
        self(self, x + 1, d, y, r);
    };
    dfs(dfs, 0, rows, 0, cols);
    return p;
}

template <class Acc = long long>
vector<Acc> minPlus(const vector<Acc> &a, const vector<Acc> &b)
{
    // a、b 是任意整数序列；返回精确的 min-plus 卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
```

```
        return {};
    }
    int n = a.size();
    int m = b.size();
    vector<Acc> c(n + m - 1);
    for (int s = 0; s < n + m - 1; s++)
    {
        int l = max<int>(0, s - m + 1);
        int r = min<int>(n - 1, s);
        Acc best = a[l] + b[s - l];
        for (int i = l + 1; i <= r; i++)
        {
            best = min(best, a[i] + b[s - i]);
        }
        c[s] = best;
    }
    return c;
}

template <class Acc = long long>
vector<Acc> minPlusCC(const vector<Acc> &a, const vector<Acc> &b)
{
    // a、b 的相邻差分均单调不降；返回精确的 min-plus 卷积，任一输入为空时返回空。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int n = a.size();
    int m = b.size();
    vector<Acc> c{a[0] + b[0]};
    int i = 1;
    int j = 1;
    while (i < n || j < m)
    {
        if (j == m || (i < n && a[i] - a[i - 1] <= b[j] - b[j - 1]))
        {
            c.push_back(c.back() + a[i] - a[i - 1]);
            i++;
        }
        else
        {
            c.push_back(c.back() + b[j] - b[j - 1]);
            j++;
        }
    }
    return c;
}

template <class Acc = long long>
vector<Acc> minPlusCA(const vector<Acc> &a, const vector<Acc> &b)
{
    // a 的相邻差分单调不降，b 任意；返回精确的 min-plus 卷积，任一输入为空时返回空。
```

```
    if (a.empty() || b.empty())
    {
        return {};
    }
    int n = b.size();
    int m = a.size();
    auto better = [&](int s, int x, int y)
    {
        auto valid = [&](int j)
        {
            return j <= s && s - j < m;
        };
        if (!valid(y))
        {
            return false;
        }
        if (!valid(x))
        {
            return true;
        }
        return b[x] + a[s - x] >= b[y] + a[s - y];
    };
    vector<int> p = monoMin(n + m - 1, n, better);
    vector<Acc> c(n + m - 1);
    for (int s = 0; s < (int)c.size(); s++)
    {
        c[s] = b[p[s]] + a[s - p[s]];
    }
    return c;
}
```

`minPlus` 是一般 $O(nm)$ 版；`minPlusCC` 要求两序列离散凸，时间 $O(n+m)$；`minPlusCA` 要求 `a` 离散凸、`b` 任意，借 `monoMin` 分治在 $O((n+m)\log(n+m))$ 时间内完成。三者默认返回 `vector<long long>`，必须保证每次加法、相邻差分和结果项都能装入 `Acc`；需要宽结果时显式调用 `minPlus<__int128>`、`minPlusCC<__int128>` 或 `minPlusCA<__int128>` 并传入宽类型序列。三者额外空间均为 $O(n+m)$，每个结果项都有至少一个合法候选，不使用不可达哨兵。它与加法卷积 10.1.003 NTT 无关，是 $(\min,+)$ 半环上的运算，常见于背包合并与凸优化 DP。

直接调用 `monoMin` 时，`better(r,x,y)` 要判断新列 `y` 是否优于旧列 `x`：

```
signed main()
{
    auto better = [](int r, int x, int y)
    {
        return abs(r - y) < abs(r - x);
    };
    assert((monoMin(3, 3, better) == vector<int>{0, 1, 2}));
}
```

下面用两个短例分别核对完整 64 位边界和两种凸优化：

```
signed main()
{
    vector<long long> a{0, 1, 4}, b{5, 0, 6};
    assert((minPlus(vector<long long>{1, 2}, vector<long long>{-1, 3}) ==
            vector<long long>{0, 1, 5}));
    assert((minPlusCC(a, vector<long long>{0, 2, 7}) ==
            vector<long long>{0, 1, 3, 6, 11}));
    assert((minPlusCA(a, b) == vector<long long>{5, 0, 1, 4, 10}));
    vector<__int128> wide{(__int128)6000000000000000000LL, (__int128)7000000000000000000LL};
    auto wc = minPlus<__int128>(wide, vector<__int128>{1, 2});
    assert(wc[0] == (__int128)6000000000000000001LL);
    cout << "OK\n";
    return 0;
}
```

10.1.013 集合形式幂级数

把长度 2^n 的集合函数看成以子集为下标的形式幂级数，乘法取 10.1.011 子集卷积。若 $f_\emptyset = 0$，则 $\exp_*(f) = \sum_{k\ge 0} f^{*k}/k!$；若 $g_\emptyset = 1$，则 $\log_*(g)$ 是它的逆。按最高二进制位拆块后，同一块内任意两项都含该最高位，乘积为零，因此指数新块是输入新块与已知低块指数的子集卷积；对数反向求解 $f_1 * g_0 = g_1$，在 ranked zeta 域按秩作常数项为 1 的三角消元。

`compEgf(c,f)` 计算 $\sum_{k=0}^{n} c_k f^{*k}/k!$。它逐个加入集合变量，在 OR zeta 域维护“已使用的变量数”和秩；`setCompose` 先在 $x = f_\emptyset$ 处求 $p^{(k)}(x)$，再令空集系数归零并调用该公共 EGF 复合，所以允许 $f_\emptyset \neq 0$。`projEgf` 是这条线性映射的转置:对子集卷积算子先把掩码取补集顺序,再调用同一个 `subsetConv`；`setPowerProj` 最后把非零常数项按二项式展开卷回去。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.006 FWT / FST
// 依赖 10.1.011 子集卷积
namespace setfps
{
template <class T> vector<T> compEgf(const vector<T> &c, const vector<T> &f)
{
    // c[k] 是外层 EGF 的第 k 个分子系数，f 是常数项为 0 的集合幂级数；返回 sum c[k]*f^k/k!。
    assert(!c.empty() && !f.empty() && has_single_bit(f.size()) && f[0] == T(0)); // 调试检查，
    ↪ 可删。
    int n = c.size() - 1;
    assert(n <= 30 && (int)f.size() == (1LL << n)); // 调试检查，可删。
    vector<vector<vector<T>>> a(n + 1);
    for (int i = 0; i <= n; i++)
    {
        a[i].assign(i + 1, vector<T>(1LL << i));
        a[i][0][0] = c[n - i];
    }
    for (int v = 0; v < n; v++)
    {
        int z = 1LL << v;
        vector h(v + 1, vector<T>(z));
```

```
        for (int s = 0; s < z; s++)
        {
            h[popcount((unsigned long long)s)][s] = f[z + s];
        }
        for (auto &x : h)
        {
            fwt(x, BitOp::Or);
        }
        for (int i = v + 1; i <= n; i++)
        {
            for (int j = 0; j <= i; j++)
            {
                copy_n(a[i][j].begin(), z, a[i][j].begin() + z);
            }
        }
        for (int i = v; i < n; i++)
        {
            for (int j = 0; j <= v; j++)
            {
                for (int k = 0; j + k <= i; k++)
                {
                    for (int s = 0; s < z; s++)
                    {
                        a[i + 1][j + k + 1][z + s] += h[j][s] * a[i][k][s];
                    }
                }
            }
        }
    }
    for (auto &x : a[n])
    {
        fwt(x, BitOp::Or, true);
    }
    vector<T> ans(f.size());
    for (int s = 0; s < (int)f.size(); s++)
    {
        ans[s] = a[n][popcount((unsigned long long)s)][s];
    }
    return ans;
}

template <class T> vector<T> transConv(const vector<T> &f, vector<T> x)
{
    // f 是固定右因子，x 是输出线性泛函；返回子集卷积算子关于 f 的转置作用结果。
    assert(!f.empty() && f.size() == x.size() && has_single_bit(f.size())); // 调试检查，可删。
    reverse(x.begin(), x.end());
    x = subsetConv(x, f);
    reverse(x.begin(), x.end());
    return x;
}

template <class T> vector<T> projEgf(const vector<T> &f, const vector<T> &w)
```

```
{
    // f 是常数项为 0 的集合幂级数，w 是线性泛函权重；返回 <w,f^k/k!>（0<=k<=n）。
    assert(!f.empty() && f.size() == w.size() && has_single_bit(f.size()) && f[0] == T(0)); // 调试检查，可删。
    int n = countr_zero((unsigned long long)f.size());
    vector<vector<T>> block(n);
    for (int j = 0; j < n; j++)
    {
        block[j] = vector<T>(f.begin() + (1LL << j), f.begin() + (2LL << j));
    }
    vector<T> dp = w;
    vector<T> ans(n + 1);
    ans[0] = w[0];
    for (int i = 0; i < n; i++)
    {
        vector<T> ndp(1LL << (n - i - 1));
        for (int j = 0; j < n - i; j++)
        {
            vector<T> x(dp.begin() + (1LL << j), dp.begin() + (2LL << j));
            x = transConv(block[j], move(x));
            for (int k = 0; k < (int)x.size(); k++)
            {
                ndp[k] += x[k];
            }
        }
        dp = move(ndp);
        ans[i + 1] = dp[0];
    }
    return ans;
}
}

template <class T> vector<T> setExp(const vector<T> &f)
{
    // f[0] 必须为 0；返回子集卷积意义下的 exp(f)。
    assert(!f.empty() && has_single_bit(f.size()) && f[0] == T(0)); // 调试检查，可删。
    int n = countr_zero((unsigned long long)f.size());
    vector<T> g(f.size());
    g[0] = T(1);
    for (int i = 0; i < n; i++)
    {
        int m = 1LL << i;
        vector<T> a(f.begin() + m, f.begin() + 2 * m);
        vector<T> b(g.begin(), g.begin() + m);
        a = subsetConv(a, b);
        copy(a.begin(), a.end(), g.begin() + m);
    }
    return g;
}

template <class T> vector<T> setLog(const vector<T> &g)
{
```

```
    // g[0] 必须为 1；返回子集卷积意义下的 log(g)。
    assert(!g.empty() && has_single_bit(g.size()) && g[0] == T(1)); // 调试检查，可删。
    int n = countr_zero((unsigned long long)g.size());
    vector<T> f(g.size());
    for (int i = n - 1; i >= 0; i--)
    {
        int m = 1LL << i;
        vector<T> a(g.begin() + m, g.begin() + 2 * m);
        vector<T> b(g.begin(), g.begin() + m);
        auto x = rankedZeta(a);
        auto y = rankedZeta(b);
        for (int s = 0; s < m; s++)
        {
            for (int d = 0; d <= i; d++)
            {
                for (int k = 0; k < d; k++)
                {
                    x(s, d) -= x(s, k) * y(s, d - k);
                }
            }
        }
        a = rankedMobius(move(x));
        copy(a.begin(), a.end(), f.begin() + m);
    }
    return f;
}

template <class T> vector<T> setCompose(const vector<T> &p, const vector<T> &f)
{
    // p[k] 是普通多项式系数，f 是集合幂级数；返回普通多项式 p(f)。
    assert(!f.empty() && has_single_bit(f.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)f.size());
    if (p.empty())
    {
        return vector<T>(f.size());
    }
    vector<T> q = p;
    vector<T> c(n + 1);
    T x = f[0];
    for (int k = 0; k <= n && !q.empty(); k++)
    {
        T v{};
        for (auto it = q.rbegin(); it != q.rend(); ++it)
        {
            v = v * x + *it;
        }
        c[k] = v;
        for (int i = 1; i < (int)q.size(); i++)
        {
            q[i - 1] = q[i] * T(i);
        }
        q.pop_back();
```

```cpp
    }
    vector<T> g = f;
    g[0] = T(0);
    return setfps::compEgf(c, g);
}

template <class T> vector<T> setPowerProj(const vector<T> &f, const vector<T> &w, int m)
{
    // f 是集合幂级数，w 是同长线性泛函权重，m 是输出项数；返回 <w,f^k>（0<=k<m）。
    // T 必须是交换域，且 1..m-1 在其中可逆。
    assert(m >= 0 && !f.empty() && f.size() == w.size() && has_single_bit(f.size())); // 调试检查，可删。
    if (m == 0)
    {
        return {};
    }
    T c = f[0];
    vector<T> g = f;
    g[0] = T(0);
    auto q = setfps::projEgf(g, w);
    vector<T> e(m, T(1));
    for (int i = 1; i < m; i++)
    {
        e[i] = e[i - 1] * c / T(i);
    }
    vector<T> ans(m);
    for (int i = 0; i < min(m, (int)q.size()); i++)
    {
        for (int j = 0; i + j < m; j++)
        {
            ans[i + j] += q[i] * e[j];
        }
    }
    T fac = T(1);
    for (int i = 0; i < m; i++)
    {
        if (i)
        {
            fac *= T(i);
        }
        ans[i] *= fac;
    }
    return ans;
}
```

设集合函数长度为 2^n，普通多项式 `p` 的长度为 r，投影输出长度为 m。`setExp`、`setLog`、`compEgf`、`projEgf` 的时间均为 $O(n^2 2^n)$、额外空间为 $O(n2^n)$；`setCompose` 另需 $O(nr)$ 求前 $n+1$ 阶导数值，`setPowerProj` 另需 $O(nm)$ 合并常数项。除 `setPowerProj` 要求 `T` 中 $1,\dots,m-1$ 可逆外，其余接口只使用交换环的加减乘和整数倍。

Library Checker 的 `exp/log` 参考实现都把外层函数转成 `compEgf`，算法同阶；当前主入口保留最高位分块递推，因为在 $n=20$、GCC 14.2.0、`-O2 -pipe` 的同机复测中，模板 core 分别为 `669.447/601.345`

ms，参考为 719.323/712.555 ms，峰值工作集约 97.80/96.27 MiB，参考两项均约 209.00 MiB。这些数值只说明当前实现与环境的取舍，不是跨平台结论。普通下标多项式的 FPS 运算见 10.2 多项式基础运算。

下面在两个元素的全集上分别检查指数、对数与二次复合的固定系数：

```
signed main()
{
    vector<Z> f{0, 2, 3, 5};
    assert(setExp(f) == vector<Z>({1, 2, 3, 11}));
    assert(setLog(vector<Z>{1, 2, 3, 11}) == f);
    assert(setCompose(vector<Z>{7, 11, 13}, f) == vector<Z>({7, 22, 33, 211}));
    assert(setPowerProj(f, vector<Z>{7, 11, 13, 17}, 4) ==
           vector<Z>({7, 146, 204, 0}));
}
```

10.1.014 多元卷积

给定各维长度 $n_0, n_1, \dots, n_{k-1}$，用最低位在前的混合进制把坐标压成下标。目标卷积只保留每一维坐标和都小于对应 n_i 的项，也就是混合进制无进位加法；它不同于把矩阵展平后直接卷积。令 $\chi(x) = \sum_i \lfloor x/(n_0 n_1 \cdots n_i) \rfloor \bmod k$，按 χ 分成 k 组做普通卷积，再只取输出下标所属的组，恰好滤掉发生进位的项。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.1.008 动态取模类
// 依赖 7.4.003 原根与全部原根筛
// 依赖 10.1.003 NTT
// 依赖 10.1.004 MTT 任意模数卷积
vector<Z> multiConv(const vector<int> &base, const vector<Z> &f, const vector<Z> &g)
{
    // base 依次给出各维长度，f、g 按最低维在前展平；返回各维无进位卷积。
    int k = base.size();
    int n = 1;
    for (int x : base)
    {
        assert(x > 0 && n <= (1 << 22) / x); // 调试检查，可删；同时保证 NTT 长度可用。
        n *= x;
    }
    assert((int)f.size() == n && (int)g.size() == n); // 调试检查，可删。
    if (n == 1)
    {
        return {f[0] * g[0]};
    }
    auto cls = [&](int x)
    {
        int c = 0;
        for (int b : base)
        {
            x /= b;
            c += x;
        }
```

```
        return c % k;
    };
    int len = bit_ceil((unsigned long long)(2 * n - 1));
    vector a(k, vector<Z>(len));
    vector b(k, vector<Z>(len));
    for (int i = 0; i < n; i++)
    {
        a[cls(i)][i] = f[i];
        b[cls(i)][i] = g[i];
    }
    for (int i = 0; i < k; i++)
    {
        FastNTT::dft(a[i]);
        FastNTT::dft(b[i]);
    }
    vector c(k, vector<Z>(len));
    for (int i = 0; i < k; i++)
    {
        for (int j = 0; j < k; j++)
        {
            for (int p = 0; p < len; p++)
            {
                c[(i + j) % k][p] += a[i][p] * b[j][p];
            }
        }
    }
    for (auto &v : c)
    {
        FastNTT::idft(v);
    }
    vector<Z> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = c[cls(i)][i];
    }
    return ans;
}
```

`multiConv(base,f,g)` 要求所有维长为正、两数组长度都等于 $N=\prod_i n_i$，并受当前 NTT 的 2^{23} 长度上限约束。若维数为 k，时间 $O(kN\log N+k^2N)$、空间 $O(kN)$；维数很少时常数可接受，所有维长均为 2 时应优先用专门的 10.1.011 子集卷积。

下面用 2×2 系数表检查两维无进位卷积：

```
signed main()
{
    vector<int> base{2, 2};
    vector<Z> a{1, 2, 3, 4};
    vector<Z> b{5, 6, 7, 8};
    assert(multiConv(base, a, b) == vector<Z>({5, 16, 22, 60}));
}
```

题目若允许任意素数模数且要求每一维按 $x_i^{n_i}=1$ 循环折叠，不能调用上面的无进位版本。`multiCycConv` 先在每个维度的几何序列上做 DFT，频域逐点相乘，再按逆根变换并除以 N；长度超过 100 时 `geomDft` 用 10.1.004 任意模卷积 的三素数 NTT + Garner，避免对大模数退化为平方枚举。

```
vector<DynMod> geomDft(vector<DynMod> f, DynMod r)
{
    // f 是低次在前的系数，r 是非零单位根；返回 f(1),f(r),…,f(r^(n-1))。
    int n = f.size();
    if (n <= 100)
    {
        vector<DynMod> ans(n);
        DynMod x = 1;
        for (int i = 0; i < n; i++)
        {
            DynMod pw = 1;
            for (int j = 0; j < n; j++)
            {
                ans[i] += f[j] * pw;
                pw *= x;
            }
            x *= r;
        }
        return ans;
    }
    vector<DynMod> a(2 * n - 1), b(n);
    DynMod pw = 1;
    a[0] = 1;
    for (int i = 1; i < 2 * n - 1; i++)
    {
        a[i] = a[i - 1] * pw;
        pw *= r;
    }
    DynMod ir = r.pow(DynMod::mod() - 2);
    pw = 1;
    b[0] = 1;
    for (int i = 1; i < n; i++)
    {
        b[i] = b[i - 1] * pw;
        pw *= ir;
    }
    for (int i = 0; i < n; i++)
    {
        f[i] *= b[i];
    }
    reverse(f.begin(), f.end());
    vector<long long> x(n), y(2 * n - 1);
    for (int i = 0; i < n; i++)
    {
        x[i] = f[i].x;
    }
    for (int i = 0; i < 2 * n - 1; i++)
    {
```

```
        y[i] = a[i].x;
    }
    auto c = convMod(move(x), move(y), DynMod::mod());
    vector<DynMod> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = DynMod(c[n - 1 + i]) * b[i];
    }
    return ans;
}

vector<long long> multiCycConv(long long p, const vector<int> &base,
                               const vector<long long> &f, const vector<long long> &g)
{
    // p 是不超过 1E9 的素数，各维长度整除 p-1；f、g 最低维在前，返回逐维循环卷积。
    assert(2 <= p && p <= 1000000000); // 调试检查，可删。
    DynMod::setMod(p);
    int n = 1;
    for (int x : base)
    {
        assert(x >= 2 && (p - 1) % x == 0 && n <= 262144 / x); // 调试检查，可删。
        n *= x;
    }
    assert((int)f.size() == n && (int)g.size() == n); // 调试检查，可删。
    vector<DynMod> a(n), b(n);
    for (int i = 0; i < n; i++)
    {
        a[i] = f[i];
        b[i] = g[i];
    }
    DynMod gen = primRoot(p);
    auto transform = [&](vector<DynMod> &v, bool inv)
    {
        int step = 1;
        for (int len : base)
        {
            DynMod root = gen.pow((p - 1) / len);
            if (inv)
            {
                root = root.pow(p - 2);
            }
            for (int block = 0; block < n; block += step * len)
            {
                for (int off = 0; off < step; off++)
                {
                    vector<DynMod> row(len);
                    for (int j = 0; j < len; j++)
                    {
                        row[j] = v[block + off + step * j];
                    }
                    row = geomDft(move(row), root);
                    for (int j = 0; j < len; j++)
```

```
                    {
                        v[block + off + step * j] = row[j];
                    }
                }
            }
            step *= len;
        }
    };
    transform(a, false);
    transform(b, false);
    for (int i = 0; i < n; i++)
    {
        a[i] *= b[i];
    }
    transform(a, true);
    DynMod scale = DynMod(n).pow(p - 2);
    vector<long long> ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = (a[i] * scale).x;
    }
    return ans;
}
```

该循环版要求 p 为素数、$n_i \mid p-1$、$n_i \geq 2$ 且 $N \leq 262144$；输入数组按最低维在前展平。每个维度总共处理 N/n_i 个长度 n_i 的 DFT，几何点求值的总复杂度为 $O(N \sum_i \log n_i)$，三素数卷积的系数重建为常数倍，额外空间为 $O(N + \max n_i)$。当 $N = 1$（$K = 0$）时函数退化为一个标量乘法。

10.1.015 模 2^n 乘法卷积

这里的运算是 $c_k = \sum_{ij \equiv k \pmod{2^n}} a_i b_j$，下标相乘而不是相加。把非零剩余类按 $v_2(x)$ 分层；固定层后，奇数部分属于 $(\mathbb{Z}/2^m\mathbb{Z})^\times$，当 $m \geq 2$ 时可唯一写成符号 ± 1 与 5 的幂。于是每层在一个长度至多 2 的符号轴和一个二进制循环轴上做变换，不同 2-进赋值的层再按加法合并，超过 n 的赋值统一落到下标 0。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
vector<Z> mulMod2n(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 的长度必须同为 2 的幂；返回下标乘法模数组长度的卷积。
    assert(a.size() == b.size() && !a.empty() && has_single_bit(a.size())); // 调试检查，可删。
    int n = countr_zero((unsigned long long)a.size());
    assert(n <= 22); // 调试检查，可删；该上限控制 O(n2^n) 工作量与三组分层缓冲区。
    int mask = (1LL << n) - 1;
    vector<vector<vector<Z>>> x(n + 1), y(n + 1), z(n + 1);
    auto shape = [&](int v)
    {
        int h = n - v >= 2 ? 2 : 1;
        int w = 1LL << max<int>(n - v - 2, 0);
        return pair{h, w};
```

```
    };
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        x[v].assign(h, vector<Z>(w));
        y[v].assign(h, vector<Z>(w));
        z[v].assign(h, vector<Z>(w));
        int p = (1LL << v) & mask;
        for (int j = 0; j < w; j++)
        {
            x[v][0][j] = a[p];
            y[v][0][j] = b[p];
            if (h == 2)
            {
                x[v][1][j] = a[(1LL << n) - p];
                y[v][1][j] = b[(1LL << n) - p];
            }
            p = 5 * p & mask;
        }
    }
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        for (int i = 0; i < h; i++)
        {
            FastNTT::dft(x[v][i]);
            FastNTT::dft(y[v][i]);
        }
        if (h == 2)
        {
            for (int j = 0; j < w; j++)
            {
                Z a0 = x[v][0][j], a1 = x[v][1][j];
                Z b0 = y[v][0][j], b1 = y[v][1][j];
                x[v][0][j] = a0 + a1;
                x[v][1][j] = a0 - a1;
                y[v][0][j] = b0 + b1;
                y[v][1][j] = b0 - b1;
            }
        }
    }
    for (int p = 0; p <= n; p++)
    {
        for (int q = 0; q <= n; q++)
        {
            int v = min(n, p + q);
            auto [h, w] = shape(v);
            for (int i = 0; i < h; i++)
            {
                for (int j = 0; j < w; j++)
                {
                    z[v][i][j] += x[p][i][j] * y[q][i][j];
```

```
                }
            }
        }
    }
    Z inv2 = Z(2).inv();
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        for (int i = 0; i < h; i++)
        {
            FastNTT::idft(z[v][i]);
        }
        if (h == 2)
        {
            for (int j = 0; j < w; j++)
            {
                Z a0 = z[v][0][j], a1 = z[v][1][j];
                z[v][0][j] = (a0 + a1) * inv2;
                z[v][1][j] = (a0 - a1) * inv2;
            }
        }
    }
    vector<Z> c(1LL << n);
    for (int v = 0; v <= n; v++)
    {
        auto [h, w] = shape(v);
        int p = (1LL << v) & mask;
        for (int j = 0; j < w; j++)
        {
            c[p] = z[v][0][j];
            if (h == 2)
            {
                c[(1LL << n) - p] = z[v][1][j];
            }
            p = 5 * p & mask;
        }
    }
    return c;
}
```

`mulMod2n(a,b)` 支持长度 2^n，当前实现要求 $n \le 22$；这个界限制的是 $O(n2^n)$ 工作量和 `x,y,z` 三组各含约 2^n 个 `Z` 的分层缓冲区，不是公共 NTT 的阶数上限。最大单层变换来自 $v=0$，长度仅为 2^{n-2}。总时间 $O(n2^n)$、额外空间 $O(2^n)$。系数仍在模 `998244353` 的 `Z` 中计算，只有下标乘法取模 2^n，不要把它与系数模数混淆。普通加法下标卷积用 10.1.003 NTT。

下面检查长度 1、2、4 的固定下标乘法卷积:

```
signed main()
{
    assert(mulMod2n(vector<Z>{2}, vector<Z>{3}) == vector<Z>({6}));
    assert(mulMod2n(vector<Z>{1, 2}, vector<Z>{3, 4}) == vector<Z>({13, 8}));
```

```
    assert(mulMod2n(vector<Z>{1, 2, 3, 4}, vector<Z>{5, 6, 7, 8}) ==
           vector<Z>({92, 44, 84, 40}));
}
```

10.1.016 $GF(2^{64})$ 卷积

把每个 `unsigned long long` 视为 $GF(2)[x]$ 中次数小于 64 的多项式，并模不可约多项式 $x^{64}+x^4+x^3+x+1$，即可得到 $GF(2^{64})$。加减都是异或；`carrylessMul` 做无进位乘法，`GF64::reduce` 用多项式 Montgomery 约简。乘法群阶 $2^{64}-1$ 没有足够大的二次幂因子，不能套普通 NTT；下面用特征 2 的 additive FFT，在一组 $GF(2)$ 线性无关的基上递归求值。x86-64 的 GCC/Clang 路径使用 PCLMUL 完成常数时间无进位乘法；其他目标保留逐置位的可移植回退。

```
#if defined(__x86_64__) && defined(__GNUC__)
#pragma GCC push_options
#pragma GCC target("pclmul")
#endif

unsigned __int128 carrylessMul(unsigned long long a, unsigned long long b)
{
    // a、b 是按位表示的 GF(2)[x] 多项式；返回 128 位无进位乘积，x86 下用 PCLMULQDQ 指令。
#if defined(__x86_64__) && defined(__GNUC__)
    typedef long long Vec128 __attribute__((vector_size(16)));
    Vec128 z = __builtin_ia32_pclmulqdq128(Vec128{(long long)a, 0},
                                           Vec128{(long long)b, 0}, 0);
    unsigned __int128 r;
    memcpy(&r, &z, sizeof(r));
    return r;
#else
    unsigned __int128 r = 0;
    while (a)
    {
        int k = countr_zero(a);
        r ^= (unsigned __int128)b << k;
        a &= a - 1;
    }
    return r;
#endif
}

constexpr unsigned __int128 clMulConst(unsigned long long a, unsigned long long b)
{
    // a、b 同 carrylessMul；逐位异或的 constexpr 版本，供编译期常量计算。
    unsigned __int128 r = 0;
    for (int i = 0; i < 64; i++)
    {
        if (a >> i & 1)
        {
            r ^= (unsigned __int128)b << i;
        }
```

```
    }
    return r;
}

struct GF64
{
    static constexpr unsigned __int128 poly = ((unsigned __int128)1 << 64) | 0b11011; // poly 是 GF(2^64) 的模多项式 x^64+x^4+x^3+x+1。
    static constexpr unsigned long long inv = []
    {
        unsigned long long a = 1;
        for (int i = 0; i < 6; i++)
        {
            a = clMulConst(a, (unsigned long long)clMulConst(a, (unsigned long long)poly));
        }
        return a;
    }();
    static constexpr unsigned long long r2 = []
    {
        unsigned __int128 r = 1;
        for (int i = 0; i < 128; i++)
        {
            r <<= 1;
            if (r >> 64 & 1)
            {
                r ^= poly;
            }
        }
        return (unsigned long long)r;
    }();

    unsigned long long x = 0; // x 是 Montgomery 表示（v*x^64 mod poly）。

    GF64() = default;
    explicit GF64(unsigned long long v) : x(reduce(carrylessMul(v, r2)))
    {
        // v 是普通表示的域元素；转入 Montgomery 表示。
    }
    static GF64 bit(int x)
    {
        // x 的最低位为 0 或 1；返回域元素 0 或 1。
        return GF64(x & 1);
    }

    static unsigned long long reduce(unsigned __int128 x)
    {
        // x 是 128 位无进位乘积；Montgomery 约减，返回 x*x^{-64} mod poly。
        unsigned long long f = carrylessMul((unsigned long long)x, inv);
        return (unsigned long long)(x >> 64) ^
               (unsigned long long)(carrylessMul(f, (unsigned long long)poly) >> 64) ^ f;
    }

```

```
    GF64 &operator+=(GF64 o)
    {
        x ^= o.x;
        return *this;
    }
    GF64 &operator-=(GF64 o)
    {
        return *this += o;
    }
    GF64 &operator*=(GF64 o)
    {
        x = reduce(carrylessMul(x, o.x));
        return *this;
    }
    friend GF64 operator+(GF64 a, GF64 b)
    {
        // a、b 是域元素；返回 a+b（按位异或）。
        return a += b;
    }
    friend GF64 operator-(GF64 a, GF64 b)
    {
        // a、b 是域元素；返回 a-b，特征 2 下与 a+b 相同。
        return a -= b;
    }
    friend GF64 operator*(GF64 a, GF64 b)
    {
        // a、b 是域元素；返回 a*b。
        return a *= b;
    }

    GF64 pow(unsigned long long n) const
    {
        // n 是指数；返回当前元素的 n 次幂。
        GF64 a = *this, r = bit(1);
        while (n)
        {
            if (n & 1)
            {
                r *= a;
            }
            a *= a;
            n >>= 1;
        }
        return r;
    }
    GF64 invField() const
    {
        // 当前元素非零；返回乘法逆元，即 2^64-2 次幂。
        assert(x != 0); // 调试检查，可删。
        return pow(~(unsigned long long)(0) - 1);
    }
    unsigned long long val() const
```

```
    {
        // 无参数；返回普通表示的 64 位值。
        return reduce(x);
    }
    friend bool operator==(GF64 a, GF64 b)
    {
        return a.x == b.x;
    }
};

vector<GF64> gf64SubSum(const vector<GF64> &b)
{
    // b 是 k 个域元素；返回长 2^k 的表，第 s 项是 s 的二进制位选中的 b 之和。
    vector<GF64> a(1LL << b.size());
    for (int k = 0; k < (int)b.size(); k++)
    {
        for (int s = 0; s < (1LL << k); s++)
        {
            a[(1LL << k) + s] = a[s] + b[k];
        }
    }
    return a;
}

struct GF64FFT
{
    struct Data
    {
        vector<GF64> basis, next, sums, aux; // basis 是本层基，next 是下一层基，sums 是归一化系数的子集和表，aux 是工作区。

        void init()
        {
            // 无参数；由本层 basis 求归一化系数的子集和 sums、下一层基 next，并分配工作区 aux。
            int lg = basis.size();
            aux.resize(1LL << lg);
            GF64 b = basis.back(), ib = b.invField();
            next.resize(lg - 1);
            vector<GF64> g(lg - 1);
            for (int i = 0; i + 1 < lg; i++)
            {
                g[i] = basis[i] * ib;
                next[i] = g[i] * g[i] + g[i];
            }
            sums = gf64SubSum(g);
        }
    };

    vector<Data> d; // d[k] 是长度 2^k 的变换层数据。

    void prepare(int lg)
    {
```

```
        // lg 是变换阶数；尚未准备时用随机元素生成线性无关基并逐层预处理 d[0..lg]。
        if ((int)d.size() > lg)
        {
            return;
        }
        mt19937_64 rng;
        vector<GF64> basis;
        while ((int)basis.size() < lg)
        {
            basis.clear();
            for (GF64 x(rng()); !(x == GF64()); x = x * x + x)
            {
                basis.push_back(x);
            }
        }
        basis.erase(basis.begin(), basis.end() - lg);
        d.resize(lg + 1);
        d[lg].basis = basis;
        for (int k = lg; k > 0; k--)
        {
            d[k].init();
            d[k - 1].basis = d[k].next;
        }
    }

    template <bool inverse = false> void taylor(span<GF64> f)
    {
        for (int n = inverse ? 1 : f.size() / 4;
             inverse ? n * 4 <= (int)f.size() : n >= 1;
             inverse ? n *= 2 : n /= 2)
        {
            for (int s = 0; s < (int)f.size(); s += 4 * n)
            {
                for (int i = 0; i < n; i++)
                {
                    GF64 b = f[s + n + i], c = f[s + 2 * n + i];
                    GF64 z = f[s + 3 * n + i];
                    f[s + n + i] = inverse ? b + c : b + c + z;
                    f[s + 2 * n + i] = c + z;
                }
            }
        }
    }

    template <bool inverse = false> void transform(span<GF64> f)
    {
        int n = f.size();
        if (n == 1)
        {
            return;
        }
        int lg = bit_width((unsigned long long)n) - 1;
```

```
        auto &cur = d[lg];
        if (n == 2)
        {
            f[1] = f[0] + f[1];
            return;
        }
        span<GF64> u(cur.aux.data(), n / 2);
        span<GF64> v(cur.aux.data() + n / 2, n / 2);
        if constexpr (!inverse)
        {
            taylor(f);
            for (int i = 0; i < n / 2; i++)
            {
                u[i] = f[2 * i];
                v[i] = f[2 * i + 1];
            }
            transform(u);
            transform(v);
            for (int i = 0; i < n / 2; i++)
            {
                GF64 a = u[i] + cur.sums[i] * v[i];
                f[i] = a;
                f[i + n / 2] = a + v[i];
            }
        }
        else
        {
            for (int i = 0; i < n / 2; i++)
            {
                v[i] = f[i] + f[n / 2 + i];
                u[i] = f[i] + cur.sums[i] * v[i];
            }
            transform<true>(u);
            transform<true>(v);
            for (int i = 0; i < n / 2; i++)
            {
                f[2 * i] = u[i];
                f[2 * i + 1] = v[i];
            }
            taylor<true>(f);
        }
    }
};

inline GF64FFT gf64FFT;

vector<unsigned long long> convGF64(const vector<unsigned long long> &x, const vector<unsigned
↪ long long> &y)
{
    // x、y 是 GF(2^64) 上多项式的系数（普通表示）；返回乘积系数，规模小时直接相乘，否则用加法 FFT。
    if (x.empty() || y.empty())
    {
```

```
        return {};
    }
    int need = x.size() + y.size() - 1;
    int lg = bit_width((unsigned long long)(need - 1));
    int n = 1LL << lg;
    if ((long long)x.size() * (long long)y.size() <=
        (long long)n * (lg + 1) * (lg + 1))
    {
        vector<GF64> c(need);
        for (int i = 0; i < (int)x.size(); i++)
        {
            for (int j = 0; j < (int)y.size(); j++)
            {
                c[i + j] += GF64(x[i]) * GF64(y[j]);
            }
        }
        vector<unsigned long long> ans(need);
        for (int i = 0; i < need; i++)
        {
            ans[i] = c[i].val();
        }
        return ans;
    }
    if (lg > 3 && need == (1LL << (lg - 1)) + 1)
    {
        vector<unsigned long long> prefix(x.begin(), x.end() - 1);
        vector<unsigned long long> ans = convGF64(prefix, y);
        ans.resize(need);
        GF64 last(x.back());
        for (int j = 0; j < (int)y.size(); j++)
        {
            ans[x.size() - 1 + j] ^= (last * GF64(y[j])).val();
        }
        return ans;
    }
    vector<GF64> a(n), b(n);
    for (int i = 0; i < (int)x.size(); i++)
    {
        a[i] = GF64(x[i]);
    }
    for (int i = 0; i < (int)y.size(); i++)
    {
        b[i] = GF64(y[i]);
    }
    gf64FFT.prepare(lg);
    gf64FFT.transform(span<GF64>(a));
    gf64FFT.transform(span<GF64>(b));
    for (int i = 0; i < n; i++)
    {
        a[i] *= b[i];
    }
    gf64FFT.transform<true>(span<GF64>(a));
```

```
    vector<unsigned long long> ans(need);
    for (int i = 0; i < need; i++)
    {
        ans[i] = a[i].val();
    }
    return ans;
}

#if defined(__x86_64__) && defined(__GNUC__)
#pragma GCC pop_options
#endif
```

`convGF64(a,b)` 返回上述固定域中的线性卷积，空输入返回空；输入和输出都是普通多项式基下的 `unsigned long long`。设补齐长度为 N，每层 additive FFT 的 Taylor 变换还要递归处理半长块，当前实现时间为 $O(N \log^2 N)$ 次域运算、空间为 $O(N \log N)$；当结果长度恰为 2^k+1 时会拆下较长输入的末项，避免补齐长度翻倍。PCLMUL 是本题最大规模下的重要常数优化，非 x86-64 平台的标量回退仍保持相同复杂度但可能无法满足严格时限。模 `998244353` 的整数卷积仍用 10.1.003 NTT。

下面检查域乘的固定值，并用固定种子强制一组 600×620 卷积进入 additive FFT 后核对滚动校验和：

```
signed main()
{
    mt19937_64 rng(0);
    assert((GF64(2) * GF64(3)).val() == 6);
    assert((GF64(1ULL << 63) * GF64(2)).val() == 0x1BULL);
    vector<unsigned long long> a(600), b(620);
    for (auto &x : a)
    {
        x = rng();
    }
    for (auto &x : b)
    {
        x = rng();
    }
    auto c = convGF64(a, b);
    unsigned long long h = 0;
    for (auto x : c)
    {
        h = h * 1000000007ULL + x;
    }
    assert(h == 301456748026672348 9ULL);
}
```

10.1.017 模素数乘法幺半群卷积

下标属于乘法幺半群 $(\mathbb{Z}/P\mathbb{Z}, \times)$ 时，非零元群是循环群。取原根 g，以 g^i 重编号后，非零下标的乘法变为指数模 $P-1$ 的加法，因此先做一次普通线性 NTT 卷积再折回长度 $P-1$。零元不属于该群：积为零恰好是一侧下标为零，需由两数组总和单独累加。

下面显式依赖 10.1.003 NTT 提供的 `Z` 与 `conv`，及 7.4.003 原根的 `primRoot`。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 7.4.003 原根与全部原根筛
vector<Z> mulPrimeConv(int p, const vector<Z> &a, const vector<Z> &b)
{
    // p 是不超过 NTT 长度限制的素数，a、b 按 0..p-1 编号；返回 c[k]=sum_{ij=k(mod p)}a[i]b[j]。
    assert(p >= 2 && (int)a.size() == p && (int)b.size() == p);
    if (p == 2)
    {
        return {a[0] * b[0] + a[0] * b[1] + a[1] * b[0], a[1] * b[1]};
    }
    int g = primRoot(p);
    vector<int> pw(p - 1), lg(p);
    for (int i = 0, x = 1; i < p - 1; i++)
    {
        pw[i] = x;
        lg[x] = i;
        x = x * g % p;
    }
    vector<Z> x(p - 1), y(p - 1);
    for (int i = 1; i < p; i++)
    {
        x[lg[i]] = a[i];
        y[lg[i]] = b[i];
    }
    auto z = conv(x, y);
    for (int i = p - 1; i < (int)z.size(); i++)
    {
        z[i - (p - 1)] += z[i];
    }
    z.resize(p - 1);
    vector<Z> ans(p);
    Z sa = 0, sb = 0;
    for (int i = 0; i < p; i++)
    {
        sa += a[i];
        sb += b[i];
    }
    ans[0] = sa * b[0] + (sb - b[0]) * a[0];
    for (int i = 0; i < p - 1; i++)
    {
        ans[pw[i]] = z[i];
    }
    return ans;
}
```

`mulPrimeConv(p,a,b)` 要求 `p` 为素数、`a,b` 都按余数 $0..p-1$ 存放，并返回同样布局的结果。时间 $O(P\log P)$、空间 $O(P)$，在 LC 中 $P\le 524288$；`p=2` 已单独处理。普通加法下标卷积应直接用 10.1.003 NTT，不要把离散对数变换用于非素数模数或带零元的普通环。

10.1.018 模 2^{64} 精确整数卷积

这里的系数环是整数环 $\mathbb{Z}/2^{64}\mathbb{Z}$，乘法包含进位，和 10.1.016 $GF(2^{64})$ 卷积 的无进位域乘法完全不同。依赖 10.1.004 MTT 任意模数卷积 提供的 `arbModConv::NTT`，选五个互素 NTT 素数分别卷积，每个位置的整数真值小于 $\min(n,m)2^{128}$；LC 的长度上界使五模乘积超过此界。Garner 逐项恢复混合进制系数时，`unsigned long long` 的自然溢出恰实现最终模 2^{64}。

```
vector<unsigned long long> convU64(const vector<unsigned long long> &a,
                                   const vector<unsigned long long> &b)
{
    // a、b 是普通整数系数；返回整数卷积对 2^64 取模的无符号 64 位结果。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int need = a.size() + b.size() - 1;
    int n = bit_ceil((unsigned long long)need);
    assert(n <= (1 << 20)); // 当前五个 NTT 素数均支持这一长度。
    constexpr array<int, 5> mod{998244353, 985661441, 943718401, 935329793, 918552577};
    constexpr array<int, 5> root{3, 3, 7, 3, 5};
    array<vector<int>, 5> rem;
    for (int k = 0; k < 5; k++)
    {
        rem[k] = arbModConv::NTT{mod[k], root[k]}.conv(a, b);
    }
    vector<unsigned long long> ans(need);
    for (int i = 0; i < need; i++)
    {
        array<int, 5> coef{}; // coef[k] 是第 k 个 Garner 系数。
        unsigned long long cur = 0, mul64 = 1;
        for (int k = 0; k < 5; k++)
        {
            unsigned long long seen = 0, mul = 1;
            for (int j = 0; j < k; j++)
            {
                seen = (seen + mul * coef[j]) % mod[k];
                mul = mul * mod[j] % mod[k];
            }
            int delta = (rem[k][i] + mod[k] - seen % mod[k]) % mod[k];
            coef[k] = (unsigned long long)delta * arbModConv::inv((int)mul, mod[k]) % mod[k];
            cur += mul64 * coef[k];
            mul64 *= mod[k];
        }
        ans[i] = cur;
    }
    return ans;
}
```

`convU64(a,b)` 接受无符号 64 位普通整数系数，返回线性卷积后模 2^{64} 的系数；空输入返回空。五个模数都支持长度至多 2^{20} 的 NTT，故在 $|a|,|b|\le 524288$ 下时间 $O((n+m)\log(n+m))$、额外空间 $O(n+m)$。模正奇数时优先用 10.1.004 任意模数卷积，做无进位域运算才使用 10.1.016 $GF(2^{64})$ 卷

积。

10.2 多项式基础运算

10.2.001 多项式封装

`Poly` 是后续形式幂级数的轻量载体，直接继承 `vector<Z>`，系数按 $[x^0, x^1, ...]$ 低次到高次排列，只提供存储、截断 `cut` 和加减乘等基础运算；乘法转交前置的 NTT `conv`，逆元、插值等独立算法留到各自条目。`norm` 删去尾部零系数以规范次数。`midProd(a,b)` 计算 $c_i = \sum_j b_j a_{i+j}$，即只取反转卷积中真正需要的中段，供等比节点插值避免构造无用系数。

```
// 依赖 1.1.008 静态取模类
struct Poly : vector<Z>
{
    using vector<Z>::vector;
    Poly(const vector<Z> &a = {}) : vector<Z>(a)
    {
        // a 按低次到高次给出系数；构造同系数多项式，无返回值。
    }

    Poly(vector<Z> &&a) : vector<Z>(move(a))
    {
        // a 按低次到高次给出系数；移动构造同系数多项式。
    }

    void norm()
    {
        // 无参数；删除末尾多余零系数，无返回值。
        while (!empty() && back() == Z(0))
        {
            pop_back();
        }
    }

    Poly cut(int n) const
    {
        // n 是非负截断长度；返回恰含前 n 项的形式幂级数。
        assert(n >= 0); // 调试检查，可删。
        Poly a(begin(), begin() + min(n, (int)size()));
        a.resize(n);
        return a;
    }

    Poly &operator+=(const Poly &b)
    {
        // b 是要相加的多项式；返回更新后的当前多项式。
        resize(max(size(), b.size()));
        for (int i = 0; i < (int)b.size(); i++)
        {
            (*this)[i] += b[i];
```

```
        }
        return *this;
    }

    Poly &operator-=(const Poly &b)
    {
        // b 是要减去的多项式；返回更新后的当前多项式。
        resize(max(size(), b.size()));
        for (int i = 0; i < (int)b.size(); i++)
        {
            (*this)[i] -= b[i];
        }
        return *this;
    }

    Poly &operator*=(const Poly &b)
    {
        // b 是乘数多项式；返回卷积后的当前多项式。
        if (this == &b)
        {
            Poly a = b;
            return *this = conv(move(a), move(*this));
        }
        return *this = conv(move(*this), b);
    }

    Poly &operator*=(Z k)
    {
        // k 是标量系数；返回所有系数乘 k 后的当前多项式。
        for (auto &x : *this)
        {
            x *= k;
        }
        return *this;
    }

    friend Poly operator+(Poly a, const Poly &b)
    {
        // a、b 是两个多项式；返回系数和。
        return a += b;
    }

    friend Poly operator-(Poly a, const Poly &b)
    {
        // a、b 是两个多项式；返回系数差。
        return a -= b;
    }

    friend Poly operator*(Poly a, const Poly &b)
    {
        // a、b 是两个多项式；返回多项式乘积。
        return a *= b;
```

```
    }

    friend Poly operator*(Poly a, Z k)
    {
        // a 是多项式，k 是标量；返回缩放后的多项式。
        return a *= k;
    }

    friend Poly operator*(Z k, Poly a)
    {
        // k 是标量，a 是多项式；返回缩放后的多项式。
        return a *= k;
    }
};

Poly midProd(const Poly &a, const Poly &b)
{
    // a、b 满足 a.size()>=b.size()>0；返回 c[i]=sum_j b[j]*a[i+j] 的中段乘积。
    assert(!b.empty() && a.size() >= b.size()); // 调试检查，可删。
    int m = a.size() - b.size() + 1;
    if (min((int)b.size(), m) <= 64)
    {
        Poly c(m);
        for (int i = 0; i < m; i++)
        {
            for (int j = 0; j < (int)b.size(); j++)
            {
                c[i] += b[j] * a[i + j];
            }
        }
        return c;
    }
    assert(nttFit(b.size(), m)); // 调试检查，可删。
    int n = bit_ceil((unsigned long long)a.size());
    Poly x = a;
    Poly y(b.rbegin(), b.rend());
    x.resize(n);
    y.resize(n);
    dft(x);
    dft(y);
    for (int i = 0; i < n; i++)
    {
        x[i] *= y[i];
    }
    idft(x);
    return Poly(x.begin() + b.size() - 1, x.begin() + a.size());
}
```

依赖本章公共 `Z` 与 10.1.003 NTT 的 `conv`、`dft`、`idft`，直接用 `a+b`、`a-b`、`a*b`、`a.cut(n)`。加减截断为 $O(\max(n,m))$；乘法与 `midProd` 都是一次补齐长度上的 NTT，时间 $O(N\log N)$、额外空间 $O(N)$。`midProd` 要求 `a.size()>=b.size()>0`，返回长度 `a.size()-b.size()+1`。此后各 FPS 运算都建立在此类型上。

10.2.002 乘法、除法、取模

域上多项式带余除法 $a = bq + r$（$\deg r < \deg b$）商余唯一。快速做法把两式系数反转：$\operatorname{rev}(a) = \operatorname{rev}(b)\operatorname{rev}(q) \pmod{x^{n-m+1}}$，于是 $\operatorname{rev}(q) = \operatorname{rev}(a) \cdot \operatorname{rev}(b)^{-1}$ 截断即可，反转回来得 q，再由 $r = a - bq$ 求余。

```cpp
// 依赖 10.2.003 乘法逆元
Poly fpsInvFast(const Poly &a, int n);

pair<Poly, Poly> polyDivmod(Poly a, Poly b)
{
    // a 是被除式，b 是非零除式；返回 {商，余式}，余式次数严格小于 b。
    a.norm();
    b.norm();
    assert(!b.empty()); // 调试检查，可删。
    if (a.size() < b.size())
    {
        return {{}, a};
    }
    int n = a.size() - b.size() + 1;
    if (a.size() <= 256 || b.size() <= 256)
    {
        Poly q(n);
        Z iv = b.back().inv();
        for (int i = n - 1; i >= 0; i--)
        {
            q[i] = a[i + b.size() - 1] * iv;
            for (int j = 0; j < (int)b.size(); j++)
            {
                a[i + j] -= q[i] * b[j];
            }
        }
        a.resize(b.size() - 1);
        q.norm();
        a.norm();
        return {move(q), move(a)};
    }
    assert(convFit(n, n) && convFit(b.size(), n)); // 调试检查，可删。
    Poly x = a;
    Poly y = b;
    reverse(x.begin(), x.end());
    reverse(y.begin(), y.end());
    Poly q = (x.cut(n) * fpsInvFast(y, n)).cut(n);
    reverse(q.begin(), q.end());
    Poly r = a - b * q;
    r.resize(b.size() - 1);
    q.norm();
    r.norm();
    return {q, r};
}
```

`polyDivmod(a,b)` 返回 `{q,r}`，会删去输入尾零、`b` 不能是零多项式；`{2,3,1}`（$x^2 + 3x + 2$）除 `{1,1}`

（$x+1$）得商 `{2,1}`、余空。被除式或除式长度不超过 256 时使用 $O((\deg q+1)|b|)$ 的长除，否则用 10.2.003 乘法逆元 的 `fpsInvFast`，时间 $O(M(n))$、空间 $O(n)$（$M(n)$ 为 n 项卷积代价）。同机、GCC 14.2.0、`-O2 -DNDEBUG`、固定随机种子与 7 轮中位数基准中，除式长度不超过 256 的规定网格只有被除式长 1000、除式长 256 时 NTT 快约 4%，其余长除更快；该阈值是本机经验值，不是渐进复杂度结论。除式稀疏时用 10.2.006 稀疏多项式除法 的系数递推更省。

下面用整除、带余除法和被除式次数较小三种短例核对 $a=bq+r$：

```
signed main()
{
    auto [q1, r1] = polyDivmod({2, 3, 1}, {1, 1});
    assert((q1 == Poly{2, 1} && r1.empty()));
    auto [q2, r2] = polyDivmod({1, 0, 1}, {1, 1});
    assert((q2 == Poly{-1, 1} && r2 == Poly{2}));
    auto [q3, r3] = polyDivmod({1}, {1, 0, 1});
    assert(q3.empty() && r3 == Poly{1});
    cout << "OK\n";
    return 0;
}
```

10.2.003 乘法逆元

在系数环 R 上，设 $G(Y)$ 是关于未知级数 Y 的一元多项式，或是各次复合均有定义的形式幂级数，精确根 F 满足 $G(F)=0$。目标截断为 x^n 且 $1\le k<n$ 时，若已有 $F^*\equiv F\pmod{x^k}$，令 $m=\min(2k,n)$；G' 表示对 Y 求形式导数而不是对 x 求导。若 $G'(F^*)$ 在 $R[[x]]/(x^m)$ 中是单位，则

$$F_{\text{new}}(x)=F^*(x)-\frac{G(F^*(x))}{G'(F^*(x))}\pmod{x^m}.$$

系数环 R 为域时，上述单位条件等价于 $G'(F^*)$ 的常数项非零；一般环上则要求该常数项本身是 R 中的单位。令 $E=F^*-F\in(x^k)$，代数恒等式 $G(F+E)=G(F)+G'(F)E+E^2H$ 表明校正后误差属于 (x^{2k})，故 $F_{\text{new}}\equiv F\pmod{x^m}$，每轮正确项数翻倍；$G(F^*)$、导数逆和乘积都只需计算到模 x^m。本条取 $G(g)=ag-1$，于是得到常数项非零的 FPS 逆 g 满足 $ag\equiv 1\pmod{x^n}$（不是逐系数求逆），以及迭代 $g\leftarrow g(2-ag)$；$\log n$ 轮到位，总代价由最后一轮卷积主导。基础版直接按该式调用两次通用卷积，便于记忆和修改。

快速版只求每轮新增长出的高半段。若 g 已满足 $ag\equiv 1\pmod{x^m}$，定义 c 使 $ag\equiv 1+x^mc\pmod{x^{2m}}$，则

$$g_{\text{new}}=g-x^m(gc \bmod x^m),\qquad ag_{\text{new}}\equiv 1+x^mc-x^mc\equiv 1\pmod{x^{2m}}.$$

用长度 $2m$ 的循环卷积计算时，$a \bmod x^{2m}$ 与次数小于 m 的 g 相乘，次数至少 $2m$ 的回卷项只会落入 $[0,m)$，所以中间乘积的 $[m,2m)$ 正是 x^mc；清空低半段后再乘一次 g，所需高半段同样不受回卷影响。最后一轮若 $k=\min(2m,n)<2m$，代码只保留 $a \bmod x^k$ 并将误差截到 $[m,k)$；两次乘积最高次数至多 $k+m-2$，回卷后仍只落在低于 m 的位置，因此 $[m,k)$ 不变。这样可复用第一次变换得到的 g 的频域值，每轮只做五次长度 $2m$ 的变换。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Poly fpsInv(const Poly &a, int n)
{
    // a 是输入 FPS，n 是 [0,2^22] 内的截断长度；n>0 时 a[0] 必须非零，返回满足 a*g=1 mod x^n 的 g。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(!a.empty() && a[0] != Z(0)); // 调试检查，可删。
    Poly g{a[0].inv()};
    for (int k = 2; k < 2 * n; k <<= 1)
    {
        g = (g * (Poly{2} - a.cut(k) * g)).cut(k);
    }
    return g.cut(n);
}

Poly fpsInvFast(const Poly &a, int n)
{
    // a 是输入 FPS，n 是 [0,2^23] 内的截断长度；n>0 时 a[0] 必须非零，返回满足 a*g=1 mod x^n 的快速逆元。
    assert(n >= 0 && n <= (1 << 23)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(!a.empty() && a[0] != Z(0)); // 调试检查，可删。

    int m = min<int>(n, 64); // m 是当前已经求出的逆元项数。
    Poly g(m);
    g[0] = a[0].inv();
    for (int i = 1; i < m; i++)
    {
        Z sum = 0;
        for (int j = 1; j <= i && j < (int)a.size(); j++)
        {
            sum += a[j] * g[i - j];
        }
        g[i] = -sum * g[0];
    }

    vector<Z> fg; // fg 保存当前 g 的频域值，供本轮两次乘法复用。
    vector<Z> err; // err 依次保存 a、ag 的高半误差和最终校正乘积。
    int cap = bit_ceil((unsigned long long)n);
    fg.reserve(cap);
    err.reserve(cap);
    while (m < n)
    {
        int k = min(2 * m, n); // k 是本轮要求的逆元项数。
        int z = 2 * m; // z 是本轮循环卷积的 NTT 长度。
```

```
        fg.assign(z, Z(0));
        err.assign(z, Z(0));
        copy(g.begin(), g.end(), fg.begin());
        copy(a.begin(), a.begin() + min(k, (int)a.size()), err.begin());

        dft(fg);
        dft(err);
        for (int i = 0; i < z; i++)
        {
            err[i] *= fg[i];
        }
        idft(err);
        fill(err.begin(), err.begin() + m, Z(0));
        fill(err.begin() + k, err.end(), Z(0));

        dft(err);
        for (int i = 0; i < z; i++)
        {
            err[i] *= fg[i];
        }
        idft(err);

        g.resize(k);
        for (int i = m; i < k; i++)
        {
            g[i] = -err[i];
        }
        m = k;
    }
    return g;
}
```

`fpsInv(a,n)` 与 `fpsInvFast(a,n)` 都把未给出的高次系数视为零；`n=0` 时直接返回空且不限制 `a`，`n>0` 时要求 `a` 非空且 `a[0]≠0`，返回恰 n 项。基础版较短，适合小规模或调试，但两次通用卷积的变换长度更大，在 10.1.003 NTT 的上限下要求 $n \le 2^{22}$；快速版先用 $O(64^2)$ 递推得到前 64 项，再进入高半段迭代，要求 $n \le 2^{23}$。两版均为 $O(M(n))$ 时间、$O(n)$ 额外空间。多项式除法、`ln` 和开根等公共主流程使用 `fpsInvFast`；`fpsInv` 作为更短的 Newton 基准版独立保留。除式稀疏时改用 10.2.014 稀疏 FPS 运算 的递推。

性能参考：在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 且不启用 `-march=native` 的环境中，对种子 `0xC0DEC0FFEE` 生成、常数项非零的稠密随机 FPS，三次预热后取 11 次中位数并重复三个进程，$n = 4096, 65536, 131072$ 时 `fpsInvFast/fpsInv` 分别为 $0.537/1.593$ ms、$11.715/32.639$ ms、$24.563/67.647$ ms，即快 $2.97, 2.79, 2.75$ 倍；$n = 131072$ 时，固定 32 位数组并延迟取模的参考实现为 23.035 ms，只再快 6.6%，但其安全累加界与固定变换上限绑定，故这里不采用。

下面用 $1 + x$ 的交错逆元核对系数，并覆盖零截断：

```
signed main()
{
```

```
    assert(fpsInv({}, 0).empty());
    assert(fpsInvFast({}, 0).empty());
    Poly want(65);
    for (int i = 0; i < 65; i++)
    {
        want[i] = (i & 1) ? -1 : 1;
    }
    assert(fpsInv({1, 1}, 65) == want);
    assert(fpsInvFast({1, 1}, 65) == want);
    Poly constant = fpsInvFast({2}, 65);
    assert(constant[0] == Z(2).inv());
    for (int i = 1; i < 65; i++)
    {
        assert(constant[i] == Z(0));
    }
    cout << "OK\n";
    return 0;
}
```

10.2.004 导数、积分

形式导数把 $a_i x^i$ 变为 $i a_i x^{i-1}$，积分做逆操作 $a_i x^i \mapsto \frac{a_i}{i+1} x^{i+1}$ 并把常数项固定为零，要求 $1..n$ 在模下可逆。积分直接复用 1.1.008 静态取模类的共享 `invTable<mod>`，表只在首次需要更大下标时线性扩展，不逐项调用费马逆元。`batchInv` 用一组前缀积、一次总逆元和一次反向扫描求数组逐项逆元；零项不进入乘积并在答案中保持零。三者是 FPS、插值和点值平移共用的基础零件。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Poly deriv(const Poly &a)
{
    // a 是输入多项式；返回形式导数。
    if (a.empty())
    {
        return {};
    }
    Poly b(a.size() - 1);
    for (int i = 1; i < (int)a.size(); i++)
    {
        b[i - 1] = a[i] * i;
    }
    return b;
}

Poly integr(const Poly &a)
{
    // a 是输入多项式；返回常数项为 0 的形式积分，次数必须小于 mod-1。
    assert(a.size() < mod); // 调试检查，可删。
    Poly b(a.size() + 1);
    const auto &iv = invTable<mod>(a.size());
    for (int i = 0; i < (int)a.size(); i++)
```

```
    {
        b[i + 1] = a[i] * iv[i + 1];
    }
    return b;
}

vector<Z> batchInv(const vector<Z> &a)
{
    // a 是允许含零的数组；返回逐项逆元，零位置仍为零。
    vector<Z> pre(a.size() + 1, 1);
    vector<Z> ans(a.size());
    for (int i = 0; i < (int)a.size(); i++)
    {
        pre[i + 1] = pre[i] * (a[i] == Z(0) ? Z(1) : a[i]);
    }
    Z suf = pre.back().inv();
    for (int i = (int)a.size() - 1; i >= 0; i--)
    {
        if (a[i] != Z(0))
        {
            ans[i] = suf * pre[i];
            suf *= a[i];
        }
    }
    return ans;
}
```

`deriv(a)`、`integr(a)` 均为 $O(n)$ 时间与输出空间；积分长度须小于模数，常数项由调用方按需另补。共享逆元表若需增长，本次调用另做与新增表长成正比的预处理，后续较短积分直接复用。`batchInv(a)` 允许零项，做一次模逆和 $O(n)$ 次乘法。它们直接服务 10.2.005 `ln`、`exp`、开根、快速幂、10.2.008 拉格朗日单点求值、多点插值 和 10.2.009 连续点值插值与点值平移。

10.2.005 `ln`、`exp`、开根、快速幂

$\ln f$ 由 $(\ln f)' = f' f^{-1}$ 积分得到，要求 $f(0) = 1$。令 $g = \exp f$，由 $g' = f' g$ 可得在线递推 $i g_i = \sum_{j=1}^{i} j a_j g_{i-j}$；默认 `fpsExp` 用 16 叉分块 NTT 加速这条递推，另保留 `fpsExpNewton` 的 $g \leftarrow g(1 - \ln g + f)$ 倍增实现。开根用 $g \leftarrow \frac{1}{2}(g + f g^{-1})$：先剥离最低次项，次数必须为偶；`fpsSqrt(a,n,s)` 使用调用方给出的首项平方根，`fpsSqrt(a,n)` 则复用 7.4.004 Tonelli-Shanks 模平方根的 `sqrtMod` 自动选择一组。幂 f^k 先提出最低次 x^p 与首系数 c，对归一化部分做 $\exp(k \ln)$ 再乘回 $c^k x^{pk}$。

```
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.2.001 多项式封装
// 依赖 10.2.003 乘法逆元
// 依赖 10.2.004 导数、积分
Poly fpsLog(const Poly &a, int n)
{
    // a 的常数项必须为 1，n 是不超过 2^22 的截断长度；返回 ln(a) mod x^n。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
```

```
    if (n == 0)
    {
        return {};
    }
    assert(!a.empty() && a[0] == Z(1)); // 调试检查，可删。
    Poly f = a.cut(n);
    return integr((deriv(f) * fpsInvFast(f, n)).cut(n - 1)).cut(n);
}

Poly fpsExpNewton(const Poly &a, int n)
{
    // a 的常数项必须为 0，n 是 [0,2^22] 内的截断长度；用 Newton 迭代返回 exp(a) mod x^n。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(a.empty() || a[0] == Z(0)); // 调试检查，可删。
    Poly g{1};
    for (int k = 2; k < 2 * n; k <<= 1)
    {
        Poly q = a.cut(k) - fpsLog(g, k);
        q[0] += 1;
        g = (g * q).cut(k);
    }
    return g.cut(n);
}

namespace rexp
{
constexpr int B = 16;
constexpr int S = 64;
constexpr int LIM = 1 << 20;

struct RExp
{
    int n; // 实际截断长度。
    int sz; // 补齐后的二次幂长度。
    NTT32 nt; // 预分配切片使用的公共二进制 NTT。
    vector<int> dg; // dg[i]=i*g[i]，是输入指数多项式的导数系数。
    vector<int> f; // 已经求出的 exp(g) 系数。
    vector<int> iv; // iv[i] 是 i 的模逆元，iv[0]=iv[1]=1。
    vector<int> t; // 当前层逆变换的临时数组。
    vector<int> wf; // 各层答案块的预分配频域工作区。
    vector<int> wg; // 各层导数块的预分配频域工作区。
    vector<unsigned long long> sum; // 当前频点的延迟取模乘积和。

    static size_t wsize(int n)
    {
        // n 是当前 16 叉分治区间长度；返回各递归层频域块所需的 int 元素总数。
        size_t ans = 0;
        while (n > S)
```

```
        {
            ans += 2 * n;
            n /= B;
        }
        return ans;
    }

    RExp(const Poly &g, int m)
        : n(m), sz(bit_ceil((unsigned long long)max<int>(1, m))), nt(sz > S ? sz / 8 : 0),
          dg(sz), f(sz), iv(m, 1),
          t(sz > S ? sz / 8 : 0), wf(wsize(sz)), wg(wsize(sz)), sum(t.size())
    {
        // g 是常数项为 0 的输入 FPS，m 是实际截断长度；初始化在线卷积所需全部状态。
        // dg[i]=i*g[i]，使 exp(g)'=g'*exp(g) 成为在线卷积递推。
        for (int i = 1; i < min(m, (int)g.size()); i++)
        {
            dg[i] = (unsigned long long)g[i].x * i % mod;
        }
        f[0] = 1;
        const auto &ziv = invTable<mod>(m - 1);
        for (int i = 1; i < m; i++)
        {
            iv[i] = ziv[i].val();
        }
    }

    void dfs(int l, int w, int *pf, int *pg)
    {
        // l、w 确定当前区间，pf、pg 是本层答案块与导数块的频域工作区；补全该区间答案，无返回值。
        if (w <= S)
        {
            int r = min(n, l + w);
            for (int i = l; i < r; i++)
            {
                unsigned long long v = 0;
                for (int j = l; j < i; j++)
                {
                    v += (unsigned long long)f[j] * dg[i - j];
                    if (((j - l) & 15) == 15)
                    {
                        v %= mod;
                    }
                }
                int x = f[i] + v % mod;
                if (x >= mod)
                {
                    x -= mod;
                }
                f[i] = (unsigned long long)x * iv[i] % mod;
            }
            return;
        }
```

```

        int b = w / B;
        int len = 2 * b;
        array<int *, B - 1> tf, tg;
        for (int i = 0; i < B - 1; i++)
        {
            tf[i] = pf + i * len;
            tg[i] = pg + i * len;
        }

        // 卷积核只与下标差有关，同一层的所有结点复用这些频域块。
        if (l == 0)
        {
            for (int i = 0; i < B - 1 && (i + 1) * b < this->n; i++)
            {
                copy_n(dg.data() + i * b, len, tg[i]);
                nt.dft(tg[i], len);
            }
        }

        for (int i = 0; i < B && l + i * b < this->n; i++)
        {
            if (i)
            {
                fill(sum.begin(), sum.begin() + len, (unsigned long long)(0));
                for (int j = 0; j < i; j++)
                {
                    for (int k = 0; k < len; k++)
                    {
                        sum[k] += (unsigned long long)tf[j][k] * tg[i - j - 1][k];
                    }
                }
                for (int k = 0; k < len; k++)
                {
                    t[k] = sum[k] % mod;
                }
                nt.idft(t.data(), len);
                for (int k = 0; k < b && l + i * b + k < this->n; k++)
                {
                    int &x = f[l + i * b + k];
                    x += t[k + b];
                    if (x >= mod)
                    {
                        x -= mod;
                    }
                }
            }

            dfs(l + i * b, b, pf + 2 * w, pg + 2 * w);
            if (i + 1 < B && l + (i + 1) * b < this->n)
            {
                fill(tf[i], tf[i] + len, (int)0);
```

```
                copy_n(f.data() + l + i * b, b, tf[i]);
                nt.dft(tf[i], len);
            }
        }
        fill(pf, pf + (B - 1) * len, (int)0);
    }

    Poly run()
    {
        // 无参数；执行在线分治并返回输入 FPS 指数的前 n 项。
        dfs(0, sz, wf.data(), wg.data());
        Poly ans(n);
        for (int i = 0; i < n; i++)
        {
            ans[i].x = f[i];
        }
        return ans;
    }
};
}

Poly fpsExpRelax(const Poly &a, int n)
{
    // a 的常数项必须为 0，n 不超过 2^20；用 16 叉在线卷积返回 exp(a) mod x^n。
    assert(n >= 0 && n <= rexp::LIM); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    assert(a.empty() || a[0] == Z(0)); // 调试检查，可删。
    return rexp::RExp(a, n).run();
}

Poly fpsExp(const Poly &a, int n)
{
    // a 的常数项必须为 0，n 是截断长度；默认使用在线分块算法，超长时回退 Newton 迭代。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n <= rexp::LIM)
    {
        return fpsExpRelax(a, n);
    }
    return fpsExpNewton(a, n);
}

optional<Poly> fpsSqrtAt(const Poly &a, int n, int p, Z s)
{
    // a[p] 是前 n 项中的首个非零系数，s 是它的一组平方根；返回对应 FPS 平方根，无解时返回空。
    assert(0 <= p && p < min(n, (int)a.size()) && a[p] != Z(0)); // 调试检查，可删。
    if ((p & 1) || s * s != a[p])
    {
        return nullopt;
    }
```

```
    int m = n - p;
    Poly b(a.begin() + p, a.begin() + min((int)a.size(), p + m));
    b.resize(m);
    Poly g{s};
    Z iv2 = Z(2).inv();
    for (int k = 2; k < 2 * m; k <<= 1)
    {
        g = ((g + (b.cut(k) * fpsInvFast(g, k)).cut(k)) * iv2).cut(k);
    }
    Poly ans(p / 2);
    g = g.cut(n - p / 2);
    ans.insert(ans.end(), g.begin(), g.end());
    return ans.cut(n);
}

optional<Poly> fpsSqrt(const Poly &a, int n, Z s)
{
    // a 是输入 FPS，n 是不超过 2^22 的截断长度，s 是首个非零系数的一组平方根；无平方根时返回空。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return Poly{};
    }
    int p = 0;
    while (p < min(n, (int)a.size()) && a[p] == Z(0))
    {
        p++;
    }
    if (p == min(n, (int)a.size()))
    {
        return Poly(n);
    }
    return fpsSqrtAt(a, n, p, s);
}

optional<Poly> fpsSqrt(const Poly &a, int n)
{
    // a 是输入 FPS，n 是不超过 2^22 的截断长度；自动求首个非零系数平方根，返回一组平方根或空。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return Poly{};
    }
    int p = 0;
    while (p < min(n, (int)a.size()) && a[p] == Z(0))
    {
        p++;
    }
    if (p == min(n, (int)a.size()))
    {
        return Poly(n);
    }
```

```
    if (p & 1)
    {
        return nullopt;
    }
    auto s = sqrtMod(a[p].val(), mod);
    if (!s)
    {
        return nullopt;
    }
    return fpsSqrtAt(a, n, p, Z(*s));
}

Poly fpsPow(const Poly &a, unsigned long long k, int n)
{
    // a 是输入 FPS，k 是非负指数，n 是不超过 2^22 的截断长度；返回 a^k mod x^n。
    assert(n >= 0 && n <= (1 << 22)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    if (k == 0)
    {
        Poly ans(n);
        ans[0] = 1;
        return ans;
    }
    int lim = min(n, (int)a.size());
    int p = 0;
    while (p < lim && a[p] == Z(0))
    {
        p++;
    }
    if (p == lim || (p && k > (unsigned long long)(n - 1) / p))
    {
        return Poly(n);
    }
    Z c = a[p];
    int sh = p * k;
    int m = n - sh;
    Poly b(a.begin() + p, a.begin() + min((int)a.size(), p + m));
    b.resize(m);
    b *= c.inv();
    b = fpsExp(fpsLog(b, m) * Z(k % mod), m) * c.pow(k);
    Poly ans(sh);
    ans.insert(ans.end(), b.begin(), b.end());
    return ans.cut(n);
}
```

`fpsLog(a,n)` 要求 $a_0 = 1$；`fpsExp(a,n)` 与两个显式指数接口要求 $a_0 = 0$；`fpsSqrt(a,n,s)` 要求 s^2 等于首个非零系数，无解返回空。`fpsLog`、`fpsExpNewton`、`fpsSqrt` 均为 $O(M(n))$，`fpsExpRelax` 的 16 叉在线卷积为 $O(n \log^2 n)$；后者渐进复杂度更差，但使用紧凑的 32 位频域数组，在 $n \le 2^{20}$ 时作为默认实现，超过后 `fpsExp` 回退 Newton。当前 NTT 上限下 Newton 路径和依赖基础逆元的接口最多处理

2^{22} 项，各实现峰值空间均为 $O(n)$，`fpsPow` 的复杂度随其调用的指数实现。

性能参考：在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 且不启用 `-march=native` 的环境中，对种子 `20260816` 生成的稠密随机 FPS 交替运行两种指数各 7 次，$n = 100000$ 时 `fpsExpRelax`/`fpsExpNewton` 中位数为 $12.123/79.994$ ms，快 6.60 倍；$n = 131072$ 时为 $14.929/78.355$ ms，快 5.25 倍。因此主接口保留 relaxed 路径，Newton 作为更优渐进复杂度和超长输入的独立实现。Newton 路径建立在 10.2.003 乘法逆元 与 10.2.004 导数、积分 上；输入稀疏时用 10.2.014 稀疏 FPS 运算 的 $O(nk)$ 递推。

下面用 $\log(1+x)$、$\exp(x)$ 和一组固定平方的已知系数，分别检查四个接口：

```cpp
signed main()
{
    int n = 6;
    Poly h{1, 2, 3, 4, 5, 6};
    assert(fpsLog(Poly{1, 1}, n) ==
           Poly({0, 1, -Z(2).inv(), Z(3).inv(), -Z(4).inv(), Z(5).inv()}));
    assert(fpsExp(Poly{0, 1}, n) ==
           Poly({1, 1, Z(2).inv(), Z(6).inv(), Z(24).inv(), Z(120).inv()}));
    assert(fpsPow(h, 3, n) == Poly({1, 6, 21, 56, 126, 252}));
    auto r = fpsSqrt(Poly{1, 4, 10, 20, 35, 56}, n, 1);
    assert(r && *r == h);
    assert(!fpsSqrt(Poly{0, 1}, 2, 1));
    cout << "OK\n";
    return 0;
}
```

10.2.006 稀疏多项式除法

除式只有少量非零项时，把 $a = bq$ 展开成 $a_i = \sum_j b_j q_{i-j}$，从低次到高次解出 $q_i = b_0^{-1}(a_i - \sum_{j\ge 1} b_j q_{i-j})$，内层只枚举 b 的非零项，常数项 b_0 必须可逆。公共 `spNorm` 先把任意次序的 {次数，系数} 排序，合并重复次数并删除相消后的零项；稀疏除法与后续全部稀疏 FPS 接口共用这一入口。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
vector<pair<int, Z>> spNorm(vector<pair<int, Z>> f)
{
    // f 是任意次序的稀疏项；返回按次数递增、合并同次且删除零系数后的稀疏项。
    auto cmp = [](const auto &a, const auto &b)
    {
        return a.first < b.first;
    };
    if (!is_sorted(f.begin(), f.end(), cmp))
    {
        sort(f.begin(), f.end(), cmp);
    }
    vector<pair<int, Z>> g;
    for (auto [i, x] : f)
    {
```

```
        assert(i >= 0); // 调试检查，可删。
        if (!g.empty() && g.back().first == i)
        {
            g.back().second += x;
        }
        else
        {
            g.push_back({i, x});
        }
    }
    g.erase(remove_if(g.begin(), g.end(), [](const auto &x)
    {
        return x.second == Z(0);
    }), g.end());
    return g;
}

Poly sparseDiv(const Poly &a, vector<pair<int, Z>> b, int n)
{
    // a 是稠密分子，b 保存非零的 {次数，系数}，n 是商的截断长度；返回 a/b mod x^n。
    assert(n >= 0); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    b = spNorm(move(b));
    b.erase(lower_bound(b.begin(), b.end(), n, [](const auto &x, int y)
    {
        return x.first < y;
    }), b.end());
    assert(!b.empty() && b[0].first == 0 && b[0].second != Z(0)); // 调试检查，可删。
    Z iv = b[0].second.inv();
    Poly q(n);
    for (int i = 0; i < n; i++)
    {
        q[i] = i < (int)a.size() ? a[i] : Z(0);
        for (int j = 1; j < (int)b.size() && b[j].first <= i; j++)
        {
            q[i] -= b[j].second * q[i - b[j].first];
        }
        q[i] *= iv;
    }
    return q;
}
```

`sparseDiv(a,b,n)` 允许 `b` 乱序、重复次数和显式零系数；`spNorm` 后若 `n>0`，必须留下唯一的非零常数项。次数不小于 `n` 的除式项不会影响截断商并会被删除。分子 `{1}` 除 `{{0,1},{2,-1}}` 取 8 项得 `{1,0,1,0,1,0,1,0}`。设规范化后有效项数为 k，已按次数排列时为 $O(nk)$，乱序时另有 $O(k\log k)$ 排序，空间 $O(n+k)$。除式稠密时用 10.2.002 乘法、除法、取模；稀疏求逆等运算见 10.2.014 稀疏 FPS 运算。

10.2.007 多点求值

一次求多项式在许多横坐标上的全部点值。`multiEval` 把乘积树保存在 NTT 域：叶保存 $X - x_i$ 在二次单位根处的值，内部逐点相乘并用一次倍长变换扩展频点；根结点恢复 $P(X) = \prod_i (X - x_i)$ 的低次系数，再通过反转逆元得到向下传递的初始余式信息。向下时，当前频域值乘另一侧子树的乘积，逆变换后取高半，即得到对应子树的信息，直到线性叶输出点值。

`FastNTT` 的长度 $2k$ 频域排列中，前后两个连续 k 块分别补出原长度 k 频点的两组平方根；`twice` 正是复用 10.1.003 NTT 已写明的排列契约，不另建位逆序 NTT。补齐叶取 $x_i = 0$，只增加末尾无用点值，不改变原点顺序。很小的输入直接逐点 Horner，避免建树常数。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
vector<Z> multiEval(const Poly &f, const vector<Z> &x)
{
    // f 是待求值多项式，x 是求值点；用频域乘积树返回与 x 同序的点值。
    int s = x.size();
    if (s == 0)
    {
        return {};
    }
    if (f.empty())
    {
        return vector<Z>(s);
    }
    if (s == 1 || f.size() <= 64 || (__int128)f.size() * s <= 4096)
    {
        vector<Z> ans(s);
        for (int i = 0; i < s; i++)
        {
            for (auto it = f.rbegin(); it != f.rend(); ++it)
            {
                ans[i] = ans[i] * x[i] + *it;
            }
        }
        return ans;
    }

    int n = bit_ceil((unsigned long long)s);
    assert(n <= NTT_LIM && convFit(f.size(), f.size())); // 调试检查，可删。
    vector<Poly> tr(2 * n);
    for (int i = 0; i < n; i++)
    {
        Z y = i < s ? -x[i] : Z(0);
        tr[n + i] = {y + 1, y - 1};
    }

    auto twice = [&](Poly &a)
    {
        // a 是长度 k 的频域值；原地扩展成同一多项式的长度 2k 频域值。
        int k = a.size();
```

```
        Z w = 31;
        for (int i = 23; i > bit_width((unsigned long long)k); i--)
        {
            w *= w;
        }
        Z v = 1;
        Poly b = a;
        idft(b);
        for (int i = 0; i < k; i++)
        {
            b[i] *= v;
            v *= w;
        }
        dft(b);
        a.insert(a.end(), b.begin(), b.end());
    };

    for (int p = n - 1; p; p--)
    {
        auto &a = tr[p << 1];
        auto &b = tr[p << 1 | 1];
        int k = a.size();
        tr[p].resize(k);
        for (int i = 0; i < k; i++)
        {
            tr[p][i] = a[i] * b[i] - 1;
        }
        if (p == 1)
        {
            continue;
        }
        twice(tr[p]);
        for (int i = 0; i < 2 * k; i++)
        {
            tr[p][i] += i < k ? Z(1) : Z(-1);
        }
    }

    Poly a = tr[1];
    idft(a);
    a.push_back(1);
    reverse(a.begin(), a.end());
    a.resize(f.size());
    a = fpsInvFast(a, f.size());
    reverse(a.begin(), a.end());
    a *= f;
    a.erase(a.begin(), a.begin() + f.size() - 1);
    a.resize(n);

    vector<Z> ans(n);
    auto dfs = [&](auto &&self, int p, Poly cur) -> void
    {
```

```
        if (p >= n)
        {
            ans[p - n] = cur[0];
            return;
        }
        int k = cur.size();
        dft(cur);
        for (int q = p << 1; q <= (p << 1 | 1); q++)
        {
            Poly b = tr[q ^ 1];
            for (int i = 0; i < k; i++)
            {
                b[i] *= cur[i];
            }
            idft(b);
            self(self, q, Poly(b.begin() + k / 2, b.end()));
        }
    };
    dfs(dfs, 1, move(a));
    ans.resize(s);
    return ans;
}
```

`PolyTree` 保留普通系数乘积树。它适合同一批点复用多次，且 10.2.008 拉格朗日单点求值、多点插值 需要直接访问根多项式。建树后先算 $a \bmod t_{\text{root}}$，再自顶向下取模；子树不超过 128 个实际点时用 Horner 直接求当前余式，省去大量小规模逆元与卷积。

```
struct PolyTree
{
    static constexpr int S = 128; // 子树点数不超过 S 时直接逐点 Horner 求值。
    // n 保存求值点个数，z 保存补齐到的 2 的幂叶子数。
    int n;
    int z;
    // x 保存各求值点，t 保存乘积树多项式。
    vector<Z> x;
    vector<Poly> t;

    PolyTree(const vector<Z> &points)
        : n(points.size()), z(bit_ceil((unsigned long long)max<int>(1, n))), x(points)
    {
        // points 是全部求值点；建立乘积树。
        assert(z <= NTT_LIM); // 调试检查，可删。
        t.assign(2 * z, Poly{1});
        for (int i = 0; i < n; i++)
        {
            t[z + i] = {-x[i], 1};
        }
        for (int i = z - 1; i; i--)
        {
            t[i] = t[i << 1] * t[i << 1 | 1];
        }
```

```
    }

    vector<Z> eval(const Poly &a) const
    {
        // a 是待求值多项式；返回 a(x[i])，顺序与构造参数 x 相同。
        if (n == 0)
        {
            return {};
        }
        int m = a.size();
        while (m && a[m - 1] == Z(0))
        {
            m--;
        }
        int b = t[1].size();
        if (m >= b && m > 256 && b > 256)
        {
            int q = m - b + 1;
            assert(convFit(q, q) && convFit(b, q)); // 调试检查，可删。
        }
        vector<Z> ans(n);
        auto dfs = [&](auto &&self, int p, int l, int w, Poly r) -> void
        {
            int cnt = max<int>(0, min(n, l + w) - l);
            if (cnt <= S)
            {
                for (int i = l; i < l + cnt; i++)
                {
                    Z v = 0;
                    for (auto it = r.rbegin(); it != r.rend(); ++it)
                    {
                        v = v * x[i] + *it;
                    }
                    ans[i] = v;
                }
                return;
            }
            if (l + w / 2 >= n)
            {
                self(self, p << 1, l, w / 2,
                     polyDivmod(move(r), t[p << 1]).second);
                return;
            }
            Poly left = polyDivmod(r, t[p << 1]).second;
            Poly right = polyDivmod(move(r), t[p << 1 | 1]).second;
            self(self, p << 1, l, w / 2, move(left));
            self(self, p << 1 | 1, l + w / 2, w / 2, move(right));
        };
        dfs(dfs, 1, 0, z, polyDivmod(a, t[1]).second);
        return ans;
    }
};
```

`multiEval(f,x)` 返回与 `x` 同序的点值，允许重复点、空点集与零多项式。设 $F=|f|$、$N=|x|$，要求补齐点数不超过 `NTT_LIM` 且 `convFit(F,F)`；时间 $O(F\log F+N\log^2 N)$，树占 $O(N\log N)$、根逆元与临时量占 $O(F+N)$ 空间。`PolyTree` 依赖 10.2.002 乘法、除法、取模，建树和单次求值均为 $O(M(N)\log N)$ 量级，树占 $O(N\log N)$；构造 `PolyTree tr(x)` 后可多次调用 `tr.eval(a)`。

在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG -pipe` 且不启用 `-march=native` 的环境中，对种子 `20260824` 生成、$F=N$ 的稠密随机输入交错运行 5 轮并取中位数；$N=4096,16384,65536,131072$ 时，`PolyTree`/`multiEval` 分别为 $7.028/4.845$、$37.503/21.813$、$186.172/100.043$、$414.858/209.294$ ms。单次大规模求值优先 `multiEval`；需要显式根多项式或复用树时使用 `PolyTree`。原 `PolyTree` 的 128 点阈值来自同机 $N=131072$ 的既有基准，保留不变。其逆问题见 10.2.008 拉格朗日单点求值、多点插值。

10.2.008 拉格朗日单点求值、多点插值

两件事。等距样本 $y_i=f(i)$ 求单点 $f(x)$：拉格朗日基 $\prod_{j\neq i}\frac{x-j}{i-j}$ 的分子用前后缀积 $O(n)$ 拼出、分母是两个阶乘之积带符号，免去逐项 $O(n^2)$。一般互异点插值先求 $P'(x_i)$（$P=\prod(x-x_i)$），用一次前缀积与一次总逆元批量得到全部倒数，再递归按 $q=q_Lt_R+q_Rt_L$ 合并；左右中间多项式在返回后立即释放，不再额外保存整棵插值树。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.004 导数、积分
Z lagrangeSeq(const vector<Z> &y, long long x)
{
    // y[i]=f(i) 且 deg(f)<y.size()，x 是求值点；返回 f(x)。
    int n = y.size();
    assert(n > 0 && n <= mod); // 调试检查，可删。
    if (0 <= x && x < n)
    {
        return y[x];
    }
    vector<Z> pre(n + 1, 1);
    vector<Z> suf(n + 1, 1);
    vector<Z> fac(n + 1, 1);
    Z p = x;
    for (int i = 0; i < n; i++)
    {
        pre[i + 1] = pre[i] * (p - Z(i));
        fac[i + 1] = fac[i] * Z(i + 1);
    }
    for (int i = n - 1; i >= 0; i--)
    {
        suf[i] = suf[i + 1] * (p - Z(i));
    }
    Z ans = 0;
    for (int i = 0; i < n; i++)
    {
        Z c = pre[i] * suf[i + 1] / fac[i] / fac[n - 1 - i];
        if ((n - 1 - i) & 1)
```

```
        {
            c = -c;
        }
        ans += y[i] * c;
    }
    return ans;
}

Poly interpPoints(const vector<Z> &x, const vector<Z> &y)
{
    // x 是互不相同的横坐标，y 是对应点值；返回次数小于 x.size() 的唯一插值多项式。
    assert(x.size() == y.size()); // 调试检查，可删。
    if (x.empty())
    {
        return {};
    }
    PolyTree tr(x);
    auto d = tr.eval(deriv(tr.t[1]));
    auto iv = batchInv(d);
    vector<Z> w(tr.n);
    for (int i = 0; i < tr.n; i++)
    {
        assert(d[i] != Z(0)); // 调试检查，可删。
        w[i] = y[i] * iv[i];
    }
    auto dfs = [&](auto &&self, int p) -> Poly
    {
        if (p >= tr.z)
        {
            int i = p - tr.z;
            return i < tr.n ? Poly{w[i]} : Poly{};
        }
        Poly left = self(self, p << 1);
        Poly right = self(self, p << 1 | 1);
        return left * tr.t[p << 1 | 1] + right * tr.t[p << 1];
    };
    return dfs(dfs, 1).cut(tr.n);
}
```

等距用 `lagrangeSeq(y,x)`（要求 `y[i]=f(i)`），一般互异点用 `interpPoints(x,y)`；点 $(0,1),(1,4),(2,9)$ 插得 $1+2x+x^2$，`lagrangeSeq({1,4,9},10)=121`。前者 $O(n)$，后者 $O(M(n)\log n)$ 时间、$O(n\log n)$ 空间。依赖 10.2.007 多点求值 的 `PolyTree`；批量平移等距点值另见 10.2.009 连续点值插值与点值平移。

10.2.009 连续点值插值与点值平移

两种平移。系数平移 $f(x+c)$ 用 $[x^k]f(x+c)=\sum_{i\ge k}a_i\binom{i}{k}c^{i-k}$，把它写成两个数组（$a_i i!$ 与 $\frac{c^j}{j!}$）的一次卷积再乘 $\frac{1}{k!}$。连续点值平移由 $f(0),\dots,f(n-1)$ 求 $f(c),\dots,f(c+m-1)$：拉格朗日分母只依赖下标差、写成卷积，`batchInv` 一次求出所有分母逆元。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.004 导数、积分
Poly taylorShift(const Poly &a, Z c)
{
    // a 保存 f(x) 的系数，c 是平移量；返回 f(x+c) 的系数。
    int n = a.size();
    if (n == 0)
    {
        return {};
    }
    assert(n <= mod && convFit(n, n)); // 调试检查，可删。
    vector<Z> fac(n, 1);
    vector<Z> ifac(n, 1);
    for (int i = 1; i < n; i++)
    {
        fac[i] = fac[i - 1] * i;
    }
    ifac[n - 1] = fac[n - 1].inv();
    for (int i = n - 1; i; i--)
    {
        ifac[i - 1] = ifac[i] * i;
    }
    Poly x(n);
    Poly y(n);
    Z pw = 1;
    for (int i = 0; i < n; i++)
    {
        x[n - 1 - i] = a[i] * fac[i];
        y[i] = pw * ifac[i];
        pw *= c;
    }
    Poly z = x * y;
    Poly ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = z[n - 1 - i] * ifac[i];
    }
    return ans;
}

vector<Z> shiftSamples(const vector<Z> &y, int c, int m)
{
    // y[i]=f(i)，c 是首个整数点，m 是输出长度；返回模 mod 下 f(c)..f(c+m-1)。
    int n = y.size();
    assert(n > 0 && n < mod && 0 <= m && m <= mod); // 调试检查，可删。
    if (m == 0)
    {
        return {};
    }
    assert(convFit(n, n + m - 1)); // 调试检查，可删。
    vector<Z> fac(n + 1, 1);
```

```
    vector<Z> ifac(n + 1, 1);
    for (int i = 1; i <= n; i++)
    {
        fac[i] = fac[i - 1] * i;
    }
    ifac[n] = fac[n].inv();
    for (int i = n; i; i--)
    {
        ifac[i - 1] = ifac[i] * i;
    }
    vector<Z> a(n);
    vector<Z> den(n + m - 1);
    vector<int> xs(m);
    for (int i = 0; i < n; i++)
    {
        a[i] = y[i] * ifac[i] * ifac[n - 1 - i];
        if ((n - 1 - i) & 1)
        {
            a[i] = -a[i];
        }
    }
    Z cur = Z(c) - Z(n - 1);
    for (auto &x : den)
    {
        x = cur;
        cur += 1;
    }
    cur = c;
    for (auto &x : xs)
    {
        x = cur.val();
        cur += 1;
    }
    auto iv = batchInv(den);
    auto sum = conv(a, iv);
    vector<Z> step(m);
    for (int i = 0; i < m; i++)
    {
        int x = xs[i];
        if (x >= n)
        {
            step[i] = x - n;
        }
    }
    auto istep = batchInv(step);
    vector<Z> ans(m);
    Z fall = 0;
    int pre = -2;
    for (int i = 0; i < m; i++)
    {
        int x = xs[i];
        if (x < n)
```

```
        {
            ans[i] = y[x];
            pre = x;
            continue;
        }
        if (pre >= n && x == pre + 1)
        {
            fall *= Z(x) * istep[i];
        }
        else
        {
            fall = 1;
            for (int j = 0; j < n; j++)
            {
                fall *= x - j;
            }
        }
        ans[i] = fall * sum[i + n - 1];
        pre = x;
    }
    return ans;
}
```

系数平移 `taylorShift(a,c)` 中 `{1,2,3}` 平移 $c=4$ 得 `{57,26,3}`；点值平移 `shiftSamples(y,c,m)` 把参数按模 `998244353` 解释，允许输出区间跨过模数回绕。样本 `{1,4,9}` 从 $c=3$ 取 4 项得 `{16,25,36,49}`，从 $c=998244351$ 取 4 项得 `{1,0,1,4}`。前者要求 `convFit(n,n)`，后者要求 `convFit(n,n+m-1)`；这些条件分别覆盖实际构造的两段卷积。Taylor 平移为 $O(M(n))$，点值平移为 $O(M(n+m)+n+m)$，空间线性。两者复用 10.2.004 导数、积分 的 `batchInv` 和 10.1.003 NTT 的 `conv`，本条不再复制批量求逆。

10.2.010 快速阶乘

在多项式友好模域算远小于 P 的 $n!$。取 $B\approx\sqrt{n}$，构造 $F(x)=\prod_{j=1}^{B}(x+j)$，则 $n!$ 的整块部分是 $F(0)F(B)F(2B)\cdots$ 的乘积——用乘积树建 F、多点求值批量取这些块值，再补乘尾部零头。n 接近模数时先用 Wilson 定理 $(P-1)!\equiv-1$ 把 $n!$ 换成较短的 $(P-1-n)!$。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Z fastFact(int n)
{
    // n 满足 0<=n<Z::mod()；返回 n!，使用分块多项式乘积与多点求值降低单次复杂度。
    assert(0 <= n && n < Z::mod()); // 调试检查，可删。
    if (n <= 1)
    {
        return 1;
    }
    if (n > (Z::mod() - 1) / 2)
    {
        int m = Z::mod() - 1 - n;
        Z s = m & 1 ? Z(1) : Z(-1);
```

```
        return s / fastFact(m);
    }
    int b = max<int>(1, sqrtl(n));
    while ((b + 1) * (b + 1) <= n)
    {
        b++;
    }
    while (b * b > n)
    {
        b--;
    }
    int q = n / b;
    vector<Z> roots(b);
    vector<Z> xs(q);
    for (int i = 0; i < b; i++)
    {
        roots[i] = -Z(i + 1);
    }
    for (int i = 0; i < q; i++)
    {
        xs[i] = Z(i * b);
    }
    PolyTree prod(roots);
    PolyTree eval(xs);
    auto val = eval.eval(prod.t[1]);
    Z ans = 1;
    for (auto x : val)
    {
        ans *= x;
    }
    for (int x = q * b + 1; x <= n; x++)
    {
        ans *= x;
    }
    return ans;
}
```

批量查询固定取 $B = 2^{15}$。先预处理所有 $(kB)!$；块内部分把因子叶 $x+1,\dots,x+B-1$ 与对应余数的查询叶交错排列，树上向右传播左侧因子前缀，并对查询点多项式取模。这样每个查询一次得到 $\prod_{j=1}^{r}(kB+j)$，不会对每个 n 从 1 线性乘到 n。

```
vector<Z> factBatch(const vector<int> &q)
{
    // q 是所有待查询的非负阶乘下标；返回与 q 同序的 n!。
    // 先用多点求值预处理整块阶乘，再用一棵乘积树批量求全部块内前缀积。
    if (q.empty()) return {};
    constexpr int LOG_BLOCK = 15;
    constexpr int BLOCK = 1 << LOG_BLOCK;
    static const vector<Z> blockFac = []
    {
        int cnt = (mod - 1) / BLOCK;
```

```
        vector<Z> roots(BLOCK), xs(cnt);
        for (int i = 0; i < BLOCK; i++)
        {
            roots[i] = -Z(i + 1);
        }
        for (int i = 0; i < cnt; i++)
        {
            xs[i] = Z(i * BLOCK);
        }
        Poly f;
        {
            PolyTree prod(roots);
            f = prod.t[1];
        }
        vector<Z> val;
        {
            PolyTree eval(xs);
            val = eval.eval(f);
        }
        vector<Z> fac(cnt + 1, 1);
        for (int i = 0; i < cnt; i++)
        {
            fac[i + 1] = fac[i] * val[i];
        }
        return fac;
    }();

    vector<Z> ans(q.size());
    vector<vector<pair<int, int>>> queries(BLOCK);
    int leaves = BLOCK - 1;
    for (int i = 0; i < (int)q.size(); i++)
    {
        int n = q[i];
        assert(0 <= n && n < mod); // 调试检查，可删。
        int base = n / BLOCK * BLOCK;
        int r = n % BLOCK;
        ans[i] = blockFac[n / BLOCK];
        if (r)
        {
            queries[r].push_back({base, i});
            leaves++;
        }
    }

    int z = bit_ceil((unsigned long long)max<int>(1, leaves));
    vector<int> queryLeaf(q.size(), -1);
    vector<Poly> factors(2 * z, Poly{1}), points(2 * z, Poly{1});
    int at = 0;
    for (int r = 1; r < BLOCK; r++)
    {
        factors[z + at++] = {Z(r), Z(1)};
        for (auto [x, id] : queries[r])
```

```
        {
            queryLeaf[id] = z + at;
            points[z + at++] = {-Z(x), Z(1)};
        }
    }
    for (int p = z - 1; p > 1; p--)
    {
        factors[p] = factors[p << 1] * factors[p << 1 | 1];
        points[p] = points[p << 1] * points[p << 1 | 1];
    }
    points[1] = {Z(1)};
    for (int p = 1; p < z; p++)
    {
        Poly prefix = points[p];
        points[p << 1] = polyDivmod(prefix, points[p << 1]).second;
        points[p << 1 | 1] = polyDivmod(prefix * factors[p << 1],
                                        points[p << 1 | 1]).second;
    }
    for (int i = 0; i < (int)q.size(); i++)
    {
        if (queryLeaf[i] != -1)
        {
            const Poly &r = points[queryLeaf[i]];
            ans[i] *= r.empty() ? Z(0) : r[0];
        }
    }
    return ans;
}
```

依赖 10.2.007 多点求值 的 `PolyTree` 和 10.2.002 多项式除法与取模 的 `polyDivmod`。`fastFact(n)` 要求 $0 \le n < P$，时间 $O(M(\sqrt{n})\log n)$、空间 $O(\sqrt{n}\log n)$；`factBatch(q)` 要求所有下标位于 $0, P)$，令 $T = |q|$，固定 $B = 2^{15}$ 时预处理约 $O(M(B)\log B)$，块内批处理约 $O(M(B+T)\log(B+T))$，空间 $O((B+T)\log(B+T))$。T 很小或 n 较小时直接线性阶乘表更省，见 [8.1.001 阶乘、逆阶乘与组合数。在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`gnu++20 -O2 -pipe` 下，对 $T = 100000$、混合小值/近模数/均匀随机的固定输入预热后交替五轮，模板与受控参考端到端中位数为 $638.6/522.8$ ms，即模板约为参考的 1.221 倍；五轮完整输出哈希一致。该数据只说明当前上限输入通过 1.25 倍工程阈值，不代表模板常数普遍更优。

10.2.011 Power Projection

给定 f 和线性泛函 $L(\sum a_i x^i) = \sum_{i=0}^{n-1} a_i w_i$，要批量求 $L(f^k)$。令 $R(x) = \sum_{i=0}^{n-1} w_{n-1-i} x^i$，则 $L(h) = [x^{n-1}]R(x)h(x)$，从而

$$\sum_{k\ge 0} L(f^k) y^k = [x^{n-1}] \frac{R(x)}{1 - y f(x)}.$$

实现对这个二元有理式做 Bostan–Mori 消元：每轮把分子分母同乘 $Q(-x, y)$，因为 $Q(x,y)Q(-x,y)$ 只含 x 的偶次幂，只需按目标次数奇偶保留一半行并令 $x^2 \mapsto x$。这样 x 方向每轮减半、所需 y 系数窗

口逐轮加倍；二维卷积用 Kronecker 间隔打包成一次一维 NTT。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.003 乘法逆元
vector<Z> powerProj(const Poly &f, const vector<Z> &w, int m)
{
    // f 是输入多项式，w[i] 是线性泛函在 x^i 上的值，m 是输出项数；返回 L(f^k)（0<=k<m）。
    assert(m >= 0 && m <= (1 << 22) && w.size() <= (1 << 20)); // 调试检查，可删。
    if (m == 0)
    {
        return {};
    }
    int n = w.size();
    vector<Z> ans(m);
    if (n == 0)
    {
        return ans;
    }
    if (m == 1)
    {
        return {w[0]};
    }
    if (n <= 32 && m <= 32)
    {
        Poly p{1};
        Poly ff = f.cut(n);
        for (int k = 0; k < m; k++)
        {
            for (int i = 0; i < min(n, (int)p.size()); i++)
            {
                ans[k] += w[i] * p[i];
            }
            p = (p * ff).cut(n);
        }
        return ans;
    }

    // 将 L(h)=sum w[i]h[i] 改写为 [x^(n-1)] reverse(w)*h，
    // 再用 Bostan-Mori 递归同时求出多个 [x^(n-1)] f^k。
    int k = n - 1;
    Poly ff = f.cut(n);
    poly2::Mat p(n, 2), q(n, 2);
    q(0, 0) = 1;
    for (int i = 0; i < n; i++)
    {
        p(i, 0) = w[n - 1 - i];
        q(i, 1) = -ff[i];
    }
    while (k > 0)
    {
        auto r = q;
```

```
        for (int i = 1; i < r.n; i += 2)
        {
            for (int j = 0; j < r.m; j++)
            {
                r(i, j) = -r(i, j);
            }
        }
        auto z = poly2::conv(p, q, r);
        int rn = k / 2 + 1;
        int cm = min(m, z[0].m);
        poly2::Mat np(rn, cm), nq(rn, cm);
        int o = k & 1;
        for (int i = 0; i < rn; i++)
        {
            int pi = 2 * i + o;
            int qi = 2 * i;
            for (int j = 0; j < cm; j++)
            {
                np(i, j) = z[0](pi, j);
                nq(i, j) = z[1](qi, j);
            }
        }
        p = move(np);
        q = move(nq);
        k /= 2;
    }
    Poly a(m), b(m);
    for (int i = 0; i < min(m, p.m); i++)
    {
        a[i] = p(0, i);
        b[i] = q(0, i);
    }
    return (a * fpsInvFast(b, m)).cut(m);
}
```

依赖 10.1.005 二元 NTT / 二元多项式卷积 的连续 `poly2::Mat` 与共享右因子变换，以及 10.2.003 乘法逆元 的 `fpsInvFast`。`powerProj(f,w,m)` 中 `w[i]=L(x^i)`，返回长度 m 的

$$L(f^0),\dots,L(f^{m-1}),$$

其中每次幂只保留前 `w.size()` 项；`m=0` 返回空，`w` 为空时返回 m 个零。当前实现要求 `w.size()<=2^20`、`m<=2^22`，小于等于 32×32 时自动走朴素递推。设 $n=|w|$，一般时间为 $O(M(n)\log n+M(m))$、峰值空间 $O(n+m)$；当 $m=\Theta(n)$ 时即 $O(n\log^2 n)$。取 `f=1+x`、`w={2,3,5,7}` 时得到 `{2,5,13,33}`。

性能参考：在 Ryzen 9 7945HX、Windows 11、GCC 14.2.0、`-std=c++20 -O2 -DNDEBUG` 且不启用 `-march=native` 的环境中，种子 `20260816` 的 $n=m=131072$ 稠密随机输入单次运行约 1090.03 ms；这是包含分配与二维打包常数的本机数据，不代表其他平台的固定耗时。

10.2.012 多项式复合

求 $f(g(x)) \bmod x^n$。当 $g(0)=0$ 时使用 Kinoshita–Li 的 Graeffe 型递归：分母与其负变量版本相乘后，$Q(z)Q(-z)$ 只保留偶次项；公共 `dft` 已保证相邻频点对应 $z,-z$，并与半长变换中的 z^2 保持同一编号，因此逐对相乘即可把当前系数宽度减半、块数加倍，并保持 `b*w` 不变。递归返回时再用同一对频点恢复分子所需切片，全程不显式生成 $1,g,g^2,\dots$，也不另建私有 NTT。若 $g(0)=c\neq 0$，先令 $h=g-c$ 并用 Taylor 平移把外层改写成 $f(x+c)$，再计算平移后外层与 h 的零常数项复合；$n\le 32$ 时保留朴素分支减少常数。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.004 导数、积分
namespace comp
{
void kl(vector<Z> &p, vector<Z> q, int b, int w)
{
    // p 是待更新分子，q 是当前分母，b、w 是二维块数与块宽；原地完成 Kinoshita-Li 递归消元。
    // b*w 保持不变；公共 dft 的相邻配对与半长递归排列保证 q[2i]q[2i+1] 可直接作为半长频域值。
    assert((int)p.size() == 2 * b * w); // 调试检查，可删。
    assert((int)q.size() == 2 * b * w); // 调试检查，可删。
    if (w == 1)
    {
        return;
    }

    q.resize(4 * b * w);
    q[2 * b * w] = 1;
    dft(q);
    if (w > 2)
    {
        vector<Z> nq(2 * b * w);
        for (int i = 0; i < 4 * b * w; i += 2)
        {
            nq[i / 2] = q[i] * q[i + 1];
        }
        idft(nq);
        assert(nq[0] == Z(1)); // 调试检查，可删。
        nq[0] = 0;
        for (int i = 0; i < 2 * b; i++)
        {
            fill(nq.begin() + i * w + w / 2, nq.begin() + (i + 1) * w, Z(0));
        }
        kl(p, move(nq), 2 * b, w / 2);
    }

    dft(p);
    for (int i = 0; i < 4 * b * w; i += 2)
    {
        Z x = q[i];
        q[i] = p[i / 2] * q[i + 1];
        q[i + 1] = p[i / 2] * x;
```

```
    }
    idft(q);
    for (int i = 0; i < b; i++)
    {
        auto dst = p.begin() + 2 * i * w;
        auto src = q.begin() + 2 * (i + b) * w;
        copy_n(src, w, dst);
        fill(dst + w, dst + 2 * w, Z(0));
    }
}

Poly zero(const Poly &f, const Poly &g, int n)
{
    // g[0]=0; 用 Kinoshita-Li 算法返回 f(g(x)) mod x^n。
    assert(n > 0 && (g.empty() || g[0] == Z(0))); // 调试检查，可删。
    int s = bit_ceil((unsigned long long)n);
    assert(s <= (1 << 21)); // 调试检查，可删；递归中的最大 NTT 长度为 4*s。
    vector<Z> p(2 * s), q(2 * s);
    for (int i = 0; i < min({n, s, (int)f.size()}); i++)
    {
        p[2 * i] = f[i];
    }
    for (int i = 0; i < min({n, s, (int)g.size()}); i++)
    {
        q[i] = -g[i];
    }
    kl(p, move(q), 1, s);
    Poly ans(n);
    for (int i = 0; i < n; i++)
    {
        ans[i] = p[i];
    }
    return ans;
}
}

Poly composePoly(const Poly &f, const Poly &g, int n)
{
    // f 是外层多项式，g 是内层多项式，n 是不超过 2^21 的截断项数；返回 f(g(x)) mod x^n。
    assert(n >= 0 && n <= (1 << 21)); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    if (f.empty())
    {
        return {};
    }
    Z c = g.empty() ? Z(0) : g[0];
    Poly b = g.cut(n);
    b[0] -= c;
    if (all_of(b.begin(), b.end(), [](Z x) { return x == Z(0); }))
```

```
    {
        Z v = 0;
        for (auto it = f.rbegin(); it != f.rend(); ++it)
        {
            v = v * c + *it;
        }
        Poly ans{v};
        ans.norm();
        return ans;
    }
    if (c != Z(0))
    {
        assert((int)f.size() <= mod && convFit(f.size(), f.size())); // 调试检查，可删。
    }
    Poly a = c == Z(0) ? f : taylorShift(f, c);
    a.resize(min(n, (int)a.size()));
    if (n <= 32)
    {
        Poly ans(n), p{1};
        for (int k = 0; k < (int)a.size(); k++)
        {
            for (int i = 0; i < (int)p.size(); i++)
            {
                ans[i] += a[k] * p[i];
            }
            p = (p * b).cut(n);
        }
        ans.norm();
        return ans;
    }
    Poly ans = comp::zero(a, b, n);
    ans.norm();
    return ans;
}
```

依赖 10.2.009 连续点值插值与点值平移，并固定在模 `998244353` 的 2^{23} 阶 NTT 上。`composePoly(f,g,n)` 在 `n=0` 或 `f` 为空时返回空；快速分支令 $s=2^{\lceil \log_2 n\rceil}$，要求 $s\le 2^{21}$，因为递归最大变换长度为 $4s\le 2^{23}$。当 $g(0)\neq 0$ 时还会对完整 `f` 做 Taylor 平移，故外层长度也必须落在底层卷积上限内。主递归每层做常数次长度 $O(s)$ 的 NTT，共 $O(n\log^2 n)$ 时间；各层保留分母缓冲区，峰值空间为 $O(n\log n)$，非零常数项另加一次 $O(M(|f|))$ 平移。`f=1+2x+3x^2`、`g=4+5x`、$n=4$ 得 `57+130x+75x^2`；更短但更慢的替代是 Brent–Kung 的 $O(\sqrt{n}\,M(n))$ 实现，复合逆见 10.3.005 拉格朗日反演。

同一环境下，对 $n=131072$ 的稠密随机输入，$g(0)=0$ 与非零常数项两种单次运行分别约 400.43 ms、410.63 ms；连同幂投影、复合逆和一次正向校验的整个基准进程峰值工作集约 45.88 MB。

10.2.013 下降幂多项式乘法

下降幂基 $(x)_i=x(x-1)\cdots(x-i+1)$ 下的乘法。把系数除以阶乘转到 EGF 式表示后，下降幂乘法变成点值意义的乘法，处理完再转回：具体是三次卷积——`a`、`b` 各与 $\frac{1}{i!}$ 卷、逐点乘 $i!$、再与带符号的 $\frac{(-1)^i}{i!}$ 卷。

```cpp
// 依赖 1.1.008 静态取模类
vector<Z> fallMul(const vector<Z> &a, const vector<Z> &b)
{
    // a、b 是下降幂基系数；返回二者乘积的同基系数。
    if (a.empty() || b.empty())
    {
        return {};
    }
    int n = a.size() + b.size() - 1;
    assert(n <= mod); // 调试检查，可删。
    vector<Z> fac(n);
    vector<Z> ifac(n);
    vector<Z> e(n);
    fac[0] = 1;
    for (int i = 1; i < n; i++)
    {
        fac[i] = fac[i - 1] * i;
    }
    ifac[n - 1] = fac[n - 1].inv();
    for (int i = n - 1; i; i--)
    {
        ifac[i - 1] = ifac[i] * i;
    }

    vector<Z> x = conv(a, ifac);
    vector<Z> y = conv(b, ifac);
    x.resize(n);
    y.resize(n);
    for (int i = 0; i < n; i++)
    {
        x[i] *= y[i] * fac[i];
    }
    for (int i = 0; i < n; i++)
    {
        e[i] = i & 1 ? -ifac[i] : ifac[i];
    }
    x = conv(x, e);
    x.resize(n);
    return x;
}
```

依赖 10.1.003 NTT 的 `Z` 与 `conv`。`fallMul(a,b)` 中 `a[i]`、`b[i]` 是下降幂基 $(x)_i$ 的系数，返回同基排列。设 $s=n+m-1$，三次卷积 $O(N\log N)$ 时间、$O(N)$ 空间（$N=2^{\lceil\log_2 s\rceil}$）。普通幂基与下降幂基之间的点值互转另见 10.2.008 拉格朗日单点求值、多点插值 与 10.2.009 连续点值插值与点值平移。

10.2.014 稀疏 FPS 运算

FPS 只有少量非零项时，逆、对数、指数、幂和平方根都能按非零项递推，复杂度依赖有效非零项数 k 而非 n^2。前置 10.2.006 稀疏多项式除法 的 `spNorm` 负责排序、合并同次项和删除零项；各递推共同复用 1.1.008 静态取模类的 `invTable<mod>`，不再维护另一份稀疏专用逆元表。exp 由 $g'=gf'$，log 由

$(\log f)' = f'/f$，pow 由 $g'f = kgf'$；把最低项写成 $f = x^p cu$ 且 $u(0) = 1$ 后，幂先平移 pk 次，平方根要求 p 为偶数并对单位部分使用指数 $\frac{1}{2}$ 的同类递推。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.2.001 多项式封装
// 依赖 10.2.006 稀疏多项式除法
vector<Z> sparseInv(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 是常数项非零的稀疏 FPS；返回 1/f 的前 n 项。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    f = spNorm(move(f));
    assert(!f.empty() && f[0].first == 0 && f[0].second != Z(0)); // 调试检查，可删。
    vector<Z> g(n);
    g[0] = f[0].second.inv();
    for (int i = 1; i < n; i++)
    {
        Z sum = 0;
        for (int j = 1; j < (int)f.size() && f[j].first <= i; j++)
        {
            sum += f[j].second * g[i - f[j].first];
        }
        g[i] = -sum * g[0];
    }
    return g;
}

vector<Z> sparseExp(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 保存常数项为 0 的稀疏 FPS；规范化后返回 exp(f) 的前 n 项。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    f = spNorm(move(f));
    assert(f.empty() || f[0].first > 0 || f[0].second == Z(0)); // 调试检查，可删。
    vector<Z> g(n);
    if (!n)
    {
        return g;
    }
    const auto &iv = invTable<mod>(n - 1);
    g[0] = 1;
    for (int i = 1; i < n; i++)
    {
        for (auto [j, x] : f)
        {
            if (!j)
            {
                continue;
            }
```

```
            if (j > i)
            {
                break;
            }
            g[i] += Z(j) * x * g[i - j];
        }
        g[i] *= iv[i];
    }
    return g;
}

vector<Z> sparseLog(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 保存常数项为 1 的稀疏 FPS；规范化后返回 ln(f) 的前 n 项。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    f = spNorm(move(f));
    assert(!f.empty() && f[0].first == 0 && f[0].second == Z(1)); // 调试检查，可删。
    vector<Z> q(max<int>(0, n - 1));
    vector<Z> g(n);
    const auto &iv = invTable<mod>(n - 1);
    for (int i = 0; i + 1 < n; i++)
    {
        for (auto [j, x] : f)
        {
            if (j == i + 1)
            {
                q[i] += Z(j) * x;
            }
            if (!j)
            {
                continue;
            }
            if (j > i)
            {
                break;
            }
            q[i] -= x * q[i - j];
        }
        g[i + 1] = q[i] * iv[i + 1];
    }
    return g;
}

vector<Z> sparsePow(int n, vector<pair<int, Z>> f, long long k)
{
    // n 是截断长度，f 是可乱序且允许重复次数的稀疏 FPS，k 是非负指数；规范化后返回 f^k 的前 n 项。
    assert(0 <= n && n <= Z::mod() && k >= 0); // 调试检查，可删。
    vector<Z> g(n);
```

```
    if (!n)
    {
        return g;
    }
    if (k == 0)
    {
        g[0] = 1;
        return g;
    }
    auto h = spNorm(move(f));
    if (h.empty())
    {
        return g;
    }
    int p = h[0].first;
    if ((__int128)p * k >= n)
    {
        return g;
    }
    int sh = p * k;
    int m = n - sh;
    Z c = h[0].second;
    vector<pair<int, Z>> q;
    for (auto [i, x] : h)
    {
        if (i - p >= m)
        {
            break;
        }
        q.push_back({i - p, x / c});
    }
    const auto &iv = invTable<mod>(m - 1);
    vector<Z> r(m);
    r[0] = 1;
    long long km = k % mod;
    for (int i = 1; i < m; i++)
    {
        for (int j = 1; j < (int)q.size() && q[j].first <= i; j++)
        {
            int d = q[j].first;
            r[i] += Z((km + 1) * d - i) * q[j].second * r[i - d];
        }
        r[i] *= iv[i];
    }
    Z scale = c.pow(k);
    for (int i = 0; i < m; i++)
    {
        g[sh + i] = scale * r[i];
    }
    return g;
}

```

```cpp
optional<vector<Z>> spSqrt(int n, vector<pair<int, Z>> f, Z root)
{
    // n 是截断长度，f 已按次数规范化，root 是首项的一组平方根；返回对应平方根，无解时返回空。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    vector<Z> ans(n);
    if (n == 0)
    {
        return ans;
    }
    if (f.empty())
    {
        return ans;
    }
    int p = f[0].first;
    if (p >= n)
    {
        return ans;
    }
    if (p & 1)
    {
        return nullopt;
    }
    if (root * root != f[0].second)
    {
        return nullopt;
    }
    int sh = p / 2;
    int m = n - sh;
    Z c = f[0].second;
    vector<pair<int, Z>> q;
    for (auto [i, x] : f)
    {
        if (i - p >= m)
        {
            break;
        }
        q.push_back({i - p, x / c});
    }
    const auto &iv = invTable<mod>(m - 1);
    Z half = Z(2).inv();
    vector<Z> r(m);
    r[0] = 1;
    for (int i = 1; i < m; i++)
    {
        for (int j = 1; j < (int)q.size() && q[j].first <= i; j++)
        {
            int d = q[j].first;
            r[i] += (Z(3 * d) * half - Z(i)) * q[j].second * r[i - d];
        }
        r[i] *= iv[i];
    }
    for (int i = 0; i < m; i++)
```

```
    {
        ans[sh + i] = root * r[i];
    }
    return ans;
}

optional<vector<Z>> sparseSqrt(int n, vector<pair<int, Z>> f, Z root)
{
    // n 是截断长度，f 是可乱序且允许重复次数的稀疏 FPS，root 是首项平方根；返回对应平方根，无解时返回空。
    return spSqrt(n, spNorm(move(f)), root);
}

optional<vector<Z>> sparseSqrt(int n, vector<pair<int, Z>> f)
{
    // n 是截断长度，f 是稀疏 FPS；自动选择首个非零系数的一组平方根，返回对应平方根的前 n 项，无平方根时返回空。
    assert(0 <= n && n <= Z::mod()); // 调试检查，可删。
    auto g = spNorm(move(f));
    if (n == 0 || g.empty() || g[0].first >= n)
    {
        return vector<Z>(n);
    }
    if (g[0].first & 1)
    {
        return nullopt;
    }
    auto root = sqrtMod(g[0].second.val(), mod);
    if (!root)
    {
        return nullopt;
    }
    return spSqrt(n, move(g), Z(*root));
}
```

所有接口要求 $0 \le n \le P$，并都会先调用 `spNorm`，所以输入可以乱序、含重复次数和显式零项。规范化后 `sparseExp` 要求常数项为 0，`sparseLog` 在 `n>0` 时要求常数项为 1，`sparseInv` 要求常数项非零；零截断直接返回空。`sparsePow` 支持任意最低项和首系数，指数 `k` 非负；`sparseSqrt(n,f,root)` 固定首个非零系数的一组平方根，自动重载复用 `sqrtMod` 选择一组，无解返回空。

若最低非零项次数为 $p > 0$，模 x^n 的等式 $g^2 = f$ 只唯一约束 g 的前 $n - p/2$ 项，最后 $p/2$ 项可以任意；实现把缺失的归一化高次系数视为零并返回一个确定代表，显式 `root` 也只固定受约束前缀的首项选择。规范化为 $O(k \log k)$，逆元表为 $O(n)$，其余递推为 $O(nk)$ 时间、$O(n + k)$ 空间。稠密输入用 10.2.005 ln、exp、开根、快速幂。

下面覆盖零截断、任意最低项幂和显式首根平方根：

```
signed main()
{
    assert(sparseInv(0, {}).empty());
    assert(sparseLog(0, {}).empty());
```

```
    assert(sparseInv(5, {{0, 1}, {2, -1}}) ==
           vector<Z>({1, 0, 1, 0, 1}));
    assert(sparsePow(8, {{2, 2}}, 3) ==
           vector<Z>({0, 0, 0, 0, 0, 0, 8, 0}));
    auto r = sparseSqrt(6, {{2, 4}}, 2);
    assert(r && *r == vector<Z>({0, 2, 0, 0, 0, 0}));
}
```

10.2.015 有限域多项式求根与因式分解

固定模 `998244353` 的主版本先用导数与 half-GCD 分离各重数的平方自由部分，再利用 $\gcd(f, x^{p^d} - x)$ 按不可约因子次数分组。固定次数组用 Cantor–Zassenhaus 随机二分：对随机 $a(x)$ 计算 $a^{(p^d-1)/2} \bmod f$，其与 1 的差通常能切出非平凡因子；指数不构造为大整数，而按 a^{p^i} 的 d 个 Frobenius 共轭逐段相乘。只求域内不同根时，先取 $\gcd(f, x^p - x)$ 直接筛出一次因子，再递归拆分，避免完整分解所有高次不可约因子。另有运行时素数模 `factorPrime`，用朴素乘除和 Frobenius 幂处理小次数输入，包含 $p = 2$、导数恒零和重数为特征倍数的情形。

```
// 依赖 7.1.001 快速幂
// 依赖 10.2.001 多项式封装
// 依赖 10.2.002 乘法、除法、取模
// 依赖 10.2.020 多项式 half-GCD 与模逆
vector<pair<Poly, int>> squareFree(Poly f)
{
    // f 是次数小于 mod 的非零多项式；返回全部首一平方自由因子及其重数。
    f = polyMonic(move(f));
    assert(!f.empty() && (int)f.size() <= Z::mod());
    if (f.size() == 1)
    {
        return {};
    }
    Poly d(f.size() - 1);
    for (int i = 1; i < (int)f.size(); i++)
    {
        d[i - 1] = f[i] * i;
    }
    Poly c = polyGcd(f, d);
    Poly w = polyDivmod(f, c).first;
    vector<pair<Poly, int>> ans;
    for (int k = 1; w.size() > 1; k++)
    {
        Poly y = polyGcd(w, c);
        Poly z = polyDivmod(w, y).first;
        if (z.size() > 1)
        {
            ans.push_back({polyMonic(move(z)), k});
        }
        w = move(y);
        c = polyDivmod(c, w).first;
    }
```

```
    return ans;
}

vector<pair<Poly, int>> distinctDeg(Poly f)
{
    // f 是首一平方自由多项式；返回 {同次数不可约因子乘积，不可约因子次数}。
    vector<pair<Poly, int>> ans;
    Poly x{0, 1}, h = polyMod(x, f);
    for (int d = 1; 2 * d <= (int)f.size() - 1; d++)
    {
        h = polyPowMod(h, Z::mod(), f);
        Poly g = polyGcd(f, h - x);
        if (g.size() > 1)
        {
            ans.push_back({g, d});
            f = polyDivmod(f, g).first;
            if (f.size() == 1)
            {
                break;
            }
            h = polyMod(move(h), f);
        }
    }
    if (f.size() > 1)
    {
        ans.push_back({f, (int)f.size() - 1});
    }
    return ans;
}

void splitEqual(Poly f, int d, mt19937_64 &rng, vector<Poly> &out)
{
    // f 的不可约因子次数均为 d，rng 是随机源，out 接收结果；递归加入全部首一不可约因子，无返回值。
    if ((int)f.size() - 1 == d)
    {
        out.push_back(polyMonic(move(f)));
        return;
    }
    while (true)
    {
        Poly a(f.size() - 1);
        for (auto &x : a)
        {
            x = rng() % Z::mod();
        }
        Poly b{1}, y = move(a);
        for (int i = 0; i < d; i++)
        {
            b = polyMod(b * polyPowMod(y, (Z::mod() - 1) / 2, f), f);
            y = polyPowMod(move(y), Z::mod(), f);
        }
        b -= Poly{1};
```

```
        Poly g = polyGcd(f, b);
        if (g.size() == 1 || g.size() == f.size())
        {
            continue;
        }
        Poly q = polyDivmod(move(f), g).first;
        splitEqual(move(g), d, rng, out);
        splitEqual(move(q), d, rng, out);
        return;
    }
}

vector<pair<Poly, int>> factorPoly(Poly f, unsigned long long seed = 0)
{
    // f 是次数小于 mod 的非零多项式，seed 是随机种子；返回 F_mod 上全部首一不可约因子及重数。
    f.norm();
    assert(!f.empty() && (int)f.size() <= Z::mod());
    mt19937_64 rng(seed);
    vector<pair<Poly, int>> ans;
    auto sf = squareFree(polyMonic(move(f)));
    for (auto &[g, k] : sf)
    {
        auto dd = distinctDeg(move(g));
        for (auto &[h, d] : dd)
        {
            vector<Poly> part;
            splitEqual(move(h), d, rng, part);
            for (auto &q : part)
            {
                ans.push_back({move(q), k});
            }
        }
    }
    sort(ans.begin(), ans.end(), [](const auto &a, const auto &b)
    {
        if (a.first.size() != b.first.size())
        {
            return a.first.size() < b.first.size();
        }
        for (int i = a.first.size() - 1; i >= 0; i--)
        {
            if (a.first[i] != b.first[i])
            {
                return a.first[i].val() < b.first[i].val();
            }
        }
        return a.second < b.second;
    });
    return ans;
}

vector<Z> finiteRoots(const Poly &f, unsigned long long seed = 0)
```

```
{
    // f 是非零多项式；用完整因式分解返回所有有限根，根按代数重数重复。
    vector<Z> ans;
    for (auto [g, k] : factorPoly(f, seed))
    {
        if (g.size() == 2)
        {
            Z x = -g[0] / g[1];
            for (int i = 0; i < k; i++)
            {
                ans.push_back(x);
            }
        }
    }
    sort(ans.begin(), ans.end(), [](Z a, Z b) { return a.val() < b.val(); });
    return ans;
}

void splitRoots(Poly f, mt19937_64 &rng, vector<Z> &ans)
{
    // f 是首一且全部因子互异的一次因子积；递归把所有不同根加入 ans。
    int n = f.size() - 1;
    if (n == 1)
    {
        ans.push_back(-f[0]);
        return;
    }
    while (true)
    {
        Poly a(n);
        for (auto &x : a)
        {
            x = rng() % mod;
        }
        Poly b = polyPowMod(move(a), (mod - 1) / 2, f);
        Poly g = polyGcd(f, b - Poly{1});
        if (g.size() == 1 || g.size() == f.size())
        {
            g = polyGcd(f, b);
        }
        if (g.size() == 1 || g.size() == f.size())
        {
            continue;
        }
        Poly q = polyDivmod(move(f), g).first;
        splitRoots(move(g), rng, ans);
        splitRoots(move(q), rng, ans);
        return;
    }
}

vector<Z> distinctRoot(Poly f, unsigned long long seed = 0)
```

```
{
    // f 是非零多项式；返回 F_p 中所有不同有限根，按代表元升序排列且不重复。
    f.norm();
    assert(!f.empty()); // 调试检查，可删。
    if (f.size() == 1)
    {
        return {};
    }
    Poly x{0, 1};
    Poly h = polyPowMod(x, mod, f);
    Poly g = polyGcd(f, h - x);
    if (g.size() == 1)
    {
        return {};
    }
    mt19937_64 rng(seed);
    vector<Z> ans;
    splitRoots(polyMonic(move(g)), rng, ans);
    sort(ans.begin(), ans.end(), [](Z a, Z b)
    {
        return a.val() < b.val();
    });
    ans.erase(unique(ans.begin(), ans.end()), ans.end());
    return ans;
}

namespace dfac
{
using DPoly = vector<int>;

// 公共 Z/NTT 固定模数，不适用于这里的运行时素数；次数至多百级，故保留朴素运算。
// p 是当前模数，state 是确定性随机状态；公开入口会重置二者，因此本实现不可重入。
int p;
unsigned long long state;

int norm(int x)
{
    // x 是任意整数；返回其模 p 的最小非负代表。
    x %= p;
    if (x < 0)
    {
        x += p;
    }
    return x;
}

void trim(DPoly &a)
{
    // a 是低次在前的动态模多项式；原地删除末尾零系数。
    while (!a.empty() && a.back() == 0)
    {
        a.pop_back();
```

```
    }
}

int deg(const DPoly &a)
{
    // a 是动态模多项式；返回忽略末尾零项后的次数，零多项式返回 -1。
    int n = a.size();
    while (n && a[n - 1] == 0)
    {
        n--;
    }
    return n - 1;
}

DPoly monic(DPoly a)
{
    // a 是动态模多项式；返回首一化结果，零多项式仍为空。
    trim(a);
    if (a.empty())
    {
        return a;
    }
    int iv = powerMod(a.back(), p - 2, p);
    for (auto &x : a)
    {
        x = x * iv % p;
    }
    return a;
}

DPoly add(DPoly a, const DPoly &b)
{
    // a、b 是动态模多项式；返回 a+b。
    a.resize(max(a.size(), b.size()));
    for (int i = 0; i < (int)b.size(); i++)
    {
        a[i] += b[i];
        if (a[i] >= p)
        {
            a[i] -= p;
        }
    }
    trim(a);
    return a;
}

DPoly sub(DPoly a, const DPoly &b)
{
    // a、b 是动态模多项式；返回 a-b。
    a.resize(max(a.size(), b.size()));
    for (int i = 0; i < (int)b.size(); i++)
    {
```

```
        a[i] -= b[i];
        if (a[i] < 0)
        {
            a[i] += p;
        }
    }
    trim(a);
    return a;
}

DPoly mul(const DPoly &a, const DPoly &b)
{
    // a、b 是动态模多项式；返回朴素乘积，适用于次数至多 100 的运行时模数题。
    if (a.empty() || b.empty())
    {
        return {};
    }
    DPoly c(a.size() + b.size() - 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = 0; j < (int)b.size(); j++)
        {
            c[i + j] = (c[i + j] + a[i] * b[j]) % p;
        }
    }
    trim(c);
    return c;
}

pair<DPoly, DPoly> divmod(DPoly a, const DPoly &b)
{
    // a 是被除式，b 是非零动态模多项式；返回 {商，余式}。
    int n = deg(a), m = deg(b);
    assert(m >= 0); // 调试检查，可删。
    a.resize(n + 1);
    if (n < m)
    {
        return {{}, a};
    }
    DPoly q(n - m + 1);
    int iv = powerMod(b[m], p - 2, p);
    for (int i = n; i >= m; i--)
    {
        int x = a[i] * iv % p;
        q[i - m] = x;
        for (int j = 0; j <= m; j++)
        {
            a[i - m + j] = norm(a[i - m + j] - x * b[j] % p);
        }
    }
    trim(q);
    trim(a);
```

```
345     return {q, a};
346 }
347 
348 DPoly gcd(DPoly a, DPoly b)
349 {
350     // a、b 是动态模多项式；返回首一化 gcd。
351     trim(a);
352     trim(b);
353     while (!b.empty())
354     {
355         DPoly r = divmod(move(a), b).second;
356         a = move(b);
357         b = move(r);
358     }
359     return monic(move(a));
360 }
361 
362 DPoly mulMod(const DPoly &a, const DPoly &b, const DPoly &m)
363 {
364     // a、b 是动态模多项式，m 是非零模多项式；返回 a*b mod m。
365     return divmod(mul(a, b), m).second;
366 }
367 
368 DPoly powMod(DPoly a, int e, const DPoly &m)
369 {
370     // a 是动态模多项式，e 是非负指数，m 是非零模多项式；返回 a^e mod m。
371     DPoly r = divmod({1}, m).second;
372     a = divmod(move(a), m).second;
373     while (e)
374     {
375         if (e & 1)
376         {
377             r = mulMod(r, a, m);
378         }
379         e >>= 1;
380         if (e)
381         {
382             a = mulMod(a, a, m);
383         }
384     }
385     return r;
386 }
387 
388 unsigned long long rnd()
389 {
390     // 无参数；返回确定性 xorshift 伪随机数并更新 state。
391     state ^= state << 7;
392     state ^= state >> 9;
393     return state;
394 }
395 
396 DPoly rndPoly(int n)
```

```
{
    // n 是严格次数上界；返回次数小于 n 的随机动态模多项式。
    DPoly a(n);
    for (auto &x : a)
    {
        x = rnd() % p;
    }
    trim(a);
    return a;
}

DPoly edfOdd(const DPoly &a, int d, const DPoly &m)
{
    // a 是测试多项式，d 是不可约因子次数，m 是待拆多项式；返回奇素数域的 EDF 分裂值。
    DPoly x = powMod(a, (p - 1) / 2, m);
    DPoly r = divmod({1}, m).second;
    for (int i = 0; i < d; i++)
    {
        r = mulMod(r, x, m);
        if (i + 1 < d)
        {
            x = powMod(move(x), p, m);
        }
    }
    return r;
}

DPoly edfTwo(const DPoly &a, int d, const DPoly &m)
{
    // a 是测试多项式，d 是不可约因子次数，m 是待拆多项式；返回 F_2 的 EDF 迹值。
    DPoly x = divmod(a, m).second;
    DPoly r;
    for (int i = 0; i < d; i++)
    {
        r = add(move(r), x);
        if (i + 1 < d)
        {
            x = mulMod(x, x, m);
        }
    }
    return r;
}

void splitEq(DPoly f, int d, vector<DPoly> &ans)
{
    // f 是首一且全部不可约因子次数为 d 的多项式；递归把全部首一不可约因子加入 ans。
    int n = deg(f);
    if (n == d)
    {
        ans.push_back(move(f));
        return;
    }
```

```
    DPoly g;
    do
    {
        DPoly a = rndPoly(n);
        DPoly h = p == 2 ? edfTwo(a, d, f)
                         : sub(edfOdd(a, d, f), {1});
        g = gcd(f, move(h));
    } while (deg(g) <= 0 || deg(g) == n);
    DPoly q = divmod(move(f), g).first;
    splitEq(move(g), d, ans);
    splitEq(move(q), d, ans);
}

struct Fac
{
    // f 是首一不可约因子，e 是其代数重数。
    DPoly f;
    int e;
};

vector<Fac> factor(DPoly f)
{
    // f 是非零动态模多项式；返回全部首一不可约因子及其重数。
    f = monic(move(f));
    vector<Fac> ans;
    if (deg(f) <= 0)
    {
        return ans;
    }
    DPoly x{0, 1};
    DPoly h = x;
    for (int d = 1; deg(f) > 0 && 2 * d <= deg(f); d++)
    {
        h = powMod(move(h), p, f);
        DPoly g = gcd(f, sub(h, x));
        if (deg(g) <= 0)
        {
            continue;
        }
        vector<DPoly> part;
        splitEq(move(g), d, part);
        for (auto &q : part)
        {
            int e = 0;
            while (deg(f) >= deg(q))
            {
                auto [u, r] = divmod(f, q);
                if (!r.empty())
                {
                    break;
                }
                f = move(u);
```

```
                e++;
            }
            ans.push_back({move(q), e});
        }
    }
    if (deg(f) > 0)
    {
        ans.push_back({monic(move(f)), 1});
    }
    return ans;
}
}

struct DynFactor
{
    // f 是首一不可约因子，e 是其在输入多项式中的代数重数。
    vector<int> f;
    int e;
};

vector<DynFactor> factorPrime(vector<int> f, int p,
                              unsigned long long seed = 88172645463325252ULL)
{
    // f 是低次在前系数，p 是素数模数；返回 F_p 上的首一不可约分解及重数。
    assert(p >= 2); // 调试检查，可删。
    using namespace dfac;
    dfac::p = p;
    state = seed ? seed : 88172645463325252ULL;
    for (auto &x : f)
    {
        x = norm(x);
    }
    trim(f);
    assert(!f.empty()); // 调试检查，可删。
    auto part = factor(move(f));
    vector<DynFactor> ans;
    for (auto &x : part)
    {
        ans.push_back({move(x.f), x.e});
    }
    sort(ans.begin(), ans.end(), [](const auto &a, const auto &b)
    {
        if (a.f.size() != b.f.size())
        {
            return a.f.size() < b.f.size();
        }
        return a.f < b.f;
    });
    return ans;
}
```

`factorPoly(f,seed)` 忽略原多项式的非零常数倍，返回全部首一不可约因子及重数；`finiteRoots` 按代

数重数展开所有一次因子，`distinctRoot` 只返回互异有限根。固定模接口排除零多项式并要求 $\deg f < 998244353$，因此导数恒零时只能是常数；half-GCD 来自 10.2.020 多项式 half-GCD 与模逆。设 $n = \deg f$，完整分解是随机化算法，期望时间约 $O(nM(n)\log p)$，卷积与递归临时空间至少为线性量级；需要复现实验时显式传固定 `seed`。

`factorPrime(f,p,seed)` 的 `p` 必须是素数，系数按低次到高次给出并先规范到 $0, p)$；返回的每个 `DynFactor` 保存首一不可约因子和代数重数。标量模幂直接复用 [7.1.001 基础、位运算与边界安全的 `powerMod`；运行时模数的多项式表示、朴素乘除与 gcd 无法由固定模 `Z/Poly/NTT` 承担，才保留为局部实现。它不用 NTT，适合次数约百以内的运行时模数题；为这一级规模引入动态 NTT 反而增加约束。按朴素乘除、各次数 Frobenius 扫描和随机拆分保守估计，期望时间为 $O(n^4 \log p)$ 次小模运算，空间 $O(n^2)$。该版本通过反复整除记录重数，能覆盖 $p = 2$ 和特征整除重数的情形；大次数固定模仍优先使用 `factorPoly`。

下面检查固定分解的因子、重数与根；实际开发时也应把输出因子乘回原首一式作为证书检查：

```
signed main()
{
    Poly f{72, -84, 14, 17, -8, 1}; // 即 (x-2)^2(x-3)^2(x+2)。
    auto fac = factorPoly(f, 0);
    assert((fac == vector<pair<Poly, int>>({{{2, 1}, 1}, {{-3, 1}, 2}, {{-2, 1}, 2}})));
    assert(finiteRoots(f, 0) == vector<Z>({2, 2, 3, 3, -2}));
    assert(distinctRoot(f, 0) == vector<Z>({2, 3, -2}));
    auto dyn = factorPrime({1, 0, 0, 0, 1}, 2);
    assert(dyn.size() == 1 && dyn[0].f == vector<int>({1, 1}) && dyn[0].e == 4);
}
```

10.2.016 等差、等比节点求值与插值

等差节点 $c, c+1, \ldots$ 已由 10.2.009 连续点值插值与点值平移 的 `shiftSamples` 覆盖；这里补等比节点 $a, ar, ar^2, \ldots$。恒等式 $ij = \binom{i+j}{2} - \binom{i}{2} - \binom{j}{2}$ 把 r^{ij} 拆成三个三角数幂，使全部点值化为一次卷积。插值用等比节点的重心权与 q-二项式系数恢复系数；`triPow(r,n)[i]=r^{i(i-1)/2}` 是两方向共同底座，其中所需卷积中段直接调用 `midProd`，不再先求完整卷积再切片。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.004 导数、积分
vector<Z> triPow(Z r, int n)
{
    // r 是公比，n 是非负长度；返回 a[i]=r^(i(i-1)/2) 的前 n 项。
    assert(n >= 0); // 调试检查，可删。
    vector<Z> a(n);
    if (!n)
    {
        return a;
    }
    a[0] = 1;
    Z pw = 1;
    for (int i = 0; i + 1 < n; i++)
    {
        a[i + 1] = a[i] * pw;
        pw *= r;
```

```
    }
    return a;
}

vector<Z> geomEval(vector<Z> f, Z a, Z r, int m)
{
    // f 是低次在前的多项式，a、r 给出点 a*r^i，m 是点数；返回依次对应的 m 个点值。
    assert(m >= 0);
    if (m == 0)
    {
        return {};
    }
    int n = f.size();
    if (n == 0)
    {
        return vector<Z>(m);
    }
    if (r == Z(0))
    {
        vector<Z> ans(m, f[0]);
        Z x = 1, y = 0;
        for (auto c : f)
        {
            y += c * x;
            x *= a;
        }
        ans[0] = y;
        return ans;
    }
    assert(convFit(n, n + m - 1)); // 调试检查，可删。
    Z pw = 1;
    for (auto &x : f)
    {
        x *= pw;
        pw *= a;
    }
    auto p = triPow(r, n + m - 1);
    auto ip = triPow(r.inv(), max(n, m));
    for (int i = 0; i < n; i++)
    {
        f[i] *= ip[i];
    }
    reverse(f.begin(), f.end());
    auto z = conv(f, p);
    vector<Z> ans(m);
    for (int i = 0; i < m; i++)
    {
        ans[i] = z[i + n - 1] * ip[i];
    }
    return ans;
}

```

```cpp
vector<Z> geomInterp(vector<Z> y, Z a, Z r)
{
    // y[i]=f(a*r^i)，a、r 非零且 r^i≠1(1<=i<y.size())；返回次数小于 y.size() 的 f。
    int n = y.size();
    if (n <= 1)
    {
        return y;
    }
    assert(a != Z(0) && r != Z(0));
    assert(nttFit(n, n)); // 调试检查，可删。
    vector<Z> pw(2 * n - 1);
    pw[0] = 1;
    Z ir = r.inv();
    for (int i = 1; i < 2 * n - 1; i++)
    {
        pw[i] = pw[i - 1] * r;
    }
    auto tpw = triPow(r, 2 * n - 1);
    auto itpw = triPow(ir, 2 * n - 1);
    vector<Z> s(n, 1);
    for (int i = 1; i < n; i++)
    {
        assert(pw[i] != Z(1));
        s[i] = s[i - 1] * (Z(1) - pw[i]);
    }
    auto is = batchInv(s);
    Z sn = s[n - 1] * (Z(1) - pw[n]);
    for (int i = 0; i < n; i++)
    {
        y[i] *= tpw[n - 1 - i] * itpw[n - 1] * is[i] * is[n - 1 - i];
        if (i & 1)
        {
            y[i] = -y[i];
        }
        y[i] *= itpw[i];
    }
    auto f = midProd(Poly(tpw), Poly(y));
    for (int i = 0; i < n; i++)
    {
        f[i] *= itpw[i];
    }
    vector<Z> g(n);
    g[0] = 1;
    for (int i = 1; i < n; i++)
    {
        g[i] = tpw[i] * sn * is[i] * is[n - i];
        if (i & 1)
        {
            g[i] = -g[i];
        }
    }
    f = conv(f, g);
```

```
    f.resize(n);
    reverse(f.begin(), f.end());
    Z ia = a.inv();
    Z ap = 1;
    for (auto &x : f)
    {
        x *= ap;
        ap *= ia;
    }
    return f;
}
```

`geomEval(f,a,r,m)` 返回 $f(ar^0),\dots,f(ar^{m-1})$，允许 $r=0$；非退化分支要求 `convFit(n,n+m-1)`。`geomInterp(y,a,r)` 恢复次数小于 n 的唯一多项式，$n>1$ 时要求 $a,r\neq 0$、$1,r,\dots,r^{n-1}$ 互异且 `nttFit(n,n)`。两者依赖 10.2.001 多项式封装 的 `midProd`、10.2.004 导数、积分 的 `batchInv` 和公共 `conv`，时间分别为 $O(M(n+m))$、$O(M(n))$，空间线性。

下面检查等比节点、$r=0$ 退化和插值复原：

```
signed main()
{
    vector<Z> f{1, 2, 3};
    assert(geomEval(f, 2, 3, 4) == vector<Z>({17, 121, 1009, 8857}));
    assert(geomInterp(geomEval(f, 2, 3, 3), 2, 3) == f);
    assert(geomEval({1, 2, 3}, 7, 0, 4) == vector<Z>({162, 1, 1, 1}));
}
```

10.2.017 二元形式幂级数逆

在矩形截断 $0\le i<n,0\le j<m$ 下求 g 使 $fg=1$。一元 Newton 的 $g\leftarrow 2g-fg^2$ 不能把两维同时翻倍，因为矩形外的项不构成所需理想；改按总次数分层：若 g 已在 $i+j<L$ 正确，则 fg^2 的 $L\le i+j<2L$ 部分取负就是新系数，低次保持不动。每轮仍调用 10.1.005 二元 NTT / 二元多项式卷积 的 `conv2`，输入先按当前总次数清零；第二次卷积前把 g^2 裁到当前矩形，避免把必然不会使用的高行高列继续打包进一维 NTT。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.1.003 NTT
// 依赖 10.1.005 二元 NTT / 二元多项式卷积
vector<vector<Z>> fpsInv2(const vector<vector<Z>> &f, int n, int m)
{
    // f 是二维 FPS，n、m 是两维截断长度；返回满足 f*g=1 mod (x^n,y^m) 的 g。
    assert(n >= 0 && m >= 0);
    if (n == 0 || m == 0)
    {
        return {};
    }
    assert(!f.empty() && !f[0].empty() && f[0][0] != Z(0));
    assert(((__int128)2 * n - 1) * ((__int128)2 * m - 1) <= NTT_LIM + 2); // 调试检查，可删。
    vector<vector<Z>> g(1, vector<Z>(1, f[0][0].inv()));
```

```
    for (int len = 1; len < n + m - 1; len <<= 1)
    {
        int rn = min(n, len * 2);
        int cm = min(m, len * 2);
        vector<vector<Z>> a(rn, vector<Z>(cm));
        for (int i = 0; i < rn && i < (int)f.size(); i++)
        {
            for (int j = 0; j < cm && j < (int)f[i].size(); j++)
            {
                if (i + j < len * 2)
                {
                    a[i][j] = f[i][j];
                }
            }
        }
        auto gg = conv2(g, g);
        gg.resize(rn);
        for (auto &row : gg)
        {
            row.resize(cm);
        }
        auto ggf = conv2(move(gg), move(a));
        g.resize(rn);
        for (auto &row : g)
        {
            row.resize(cm);
        }
        for (int i = 0; i < rn; i++)
        {
            for (int j = 0; j < cm; j++)
            {
                if (len <= i + j && i + j < len * 2)
                {
                    g[i][j] = -ggf[i][j];
                }
            }
        }
    }
    g.resize(n);
    for (auto &row : g)
    {
        row.resize(m);
    }
    return g;
}
```

`fpsInv2(f,n,m)` 要求常数项非零，返回 $n \times m$ 的矩形截断逆；输入可以比目标矩形小，缺项按零处理。最外层两次二维卷积打包后的结果长度不超过 $(2n-1)(2m-1)$，因此还要求该值不超过公共 `conv` 可处理的 `NTT_LIM+2`。总时间为 $O(nm(\log n+\log m))$ 次模域运算、峰值空间 $O(nm)$，常数约为每层两次二维卷积；只做二维乘法直接用 `conv2`，一元逆用 10.2.003 乘法逆元。

下面检查 $1+x+y$ 的 3×3 矩形截断逆：

```cpp
signed main()
{
    vector<vector<Z>> f{{1, 1, 0}, {1, 0, 0}, {0, 0, 0}};
    vector<vector<Z>> inv{{1, -1, 1}, {-1, 2, -3}, {1, -3, 6}};
    assert(fpsInv2(f, 3, 3) == inv);
}
```

10.2.018 多项式序列乘积

给定一组长度差异很大的多项式时，固定层数的完全二叉乘积树会让短多项式过早与长多项式相乘。这里按当前系数长度建立小根堆，每次取最短的两个相乘再放回，等价于对卷积规模做 Huffman 式合并；`conv` 接收移动后的缓冲区，已经合并的中间多项式不会继续占用系数存储。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
Poly productPolys(vector<Poly> a)
{
    // a 是待相乘的多项式序列；返回保持卷积长度的总乘积，空序列返回常数 1。
    Z c = 1;
    vector<Poly> f;
    f.reserve(a.size());
    for (auto &x : a)
    {
        if (x.empty())
        {
            return {};
        }
        if (x.size() == 1)
        {
            c *= x[0];
        }
        else
        {
            f.push_back(move(x));
        }
    }
    if (f.empty())
    {
        return {c};
    }
    vector<Poly> pool;
    pool.reserve(2 * f.size());
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>> q;
    for (auto &x : f)
    {
        int p = pool.size();
        pool.push_back(move(x));
        q.push({pool[p].size(), p});
    }
    while (q.size() > 1)
```

```
    {
        int x = q.top().second;
        q.pop();
        int y = q.top().second;
        q.pop();
        assert(convFit(pool[x].size(), pool[y].size())); // 调试检查，可删。
        vector<Z> z = conv(move(pool[x]), move(pool[y]));
        int p = pool.size();
        pool.emplace_back(move(z));
        q.push({pool[p].size(), p});
    }
    Poly ans = move(pool[q.top().second]);
    ans *= c;
    return ans;
}
```

`productPolys(a)` 返回全部元素的通常乘积，空序列的乘积定义为常数 1，序列中出现零多项式时结果为空多项式。设 k 是多项式个数、$D=1+\sum_i \deg a_i$ 是非零输入总乘积的系数长度，在准线性卷积模型下时间为 $O(M(D)\log k + k\log k)$，峰值额外空间为 $O(D+k)$；规模接近时普通分层乘积树常数相近，规模悬殊时优先队列更稳。

下面检查空积与三个因子的总乘积：

```
signed main()
{
    assert(productPolys({}) == Poly{1});
    assert((productPolys({Poly{1, 1}, Poly{1, -1}, Poly{2}}) ==
            Poly{2, 0, -2}));
}
```

10.2.019 单项式基转 Newton 基

对节点 $p_0,\dots,p_{n-1}$ 定义 $B_0(x)=1$、$B_i(x)=\prod_{j=0}^{i-1}(x-p_j)$。目标是把低次在前的单项式表示 $f(x)$ 转成唯一的 Newton 表示 $f(x)=\sum_{i=0}^{n-1} b_i B_i(x)$；节点允许重复，因为 B_i 始终是次数为 i 的首一多项式。实现先用乘积树构造各段节点多项式，再通过反转多项式与 FPS 逆完成一次批量商信息提取，最后沿树递归恢复全部 b_i。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
vector<Z> monoToNewton(const Poly &f, const vector<Z> &p)
{
    // f 是低次在前的单项式基系数，p 是 Newton 基的节点；两者等长，返回各级前缀乘积的系数。
    int n = f.size();
    assert((int)p.size() == n); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    int z = bit_ceil((unsigned long long)n);
```

```
    assert(z <= NTT_LIM / 2); // 调试检查，可删。
    vector<Poly> prod(2 * z, Poly{1});
    for (int i = 0; i < z; i++)
    {
        Z x = i < n ? p[i] : Z(0);
        prod[z + i] = {-x, 1};
    }
    for (int i = z - 1; i; i--)
    {
        prod[i] = prod[i << 1] * prod[i << 1 | 1];
    }

    Poly a = f.cut(z);
    reverse(a.begin(), a.end());
    Poly root = prod[1];
    reverse(root.begin(), root.end());
    Poly h = (a * fpsInvFast(root, z)).cut(z);
    reverse(h.begin(), h.end());

    vector<Z> ans(n);
    auto dfs = [&](auto &&self, int u, Poly cur) -> void
    {
        if (u >= z)
        {
            int i = u - z;
            if (i < n)
            {
                ans[i] = cur[0];
            }
            return;
        }
        int m = prod[u << 1 | 1].size() - 1;
        Poly q = cur * prod[u << 1 | 1];
        Poly left(q.begin() + m, q.begin() + 2 * m);
        Poly right(cur.begin() + m, cur.end());
        self(self, u << 1, move(left));
        self(self, u << 1 | 1, move(right));
    };
    dfs(dfs, 1, move(h));
    return ans;
}
```

`monoToNewton(f,p)` 要求 `f.size()==p.size()`，返回的第 i 项就是 B_i 的系数；补齐长度 $z=2^{\lceil \log_2 n\rceil}$ 还须满足 $z\le \texttt{NTT_LIM}/2$，从而根节点乘积和反转卷积均落在公共 NTT 可调用域内。p_{n-1} 只参与统一的树形计算，不改变次数小于 n 的最终表示。依赖 10.2.003 乘法逆元，时间 $O(M(n)\log n)$，乘积树占 $O(n\log n)$ 系数空间。若只需在互异节点间做点值插值，直接使用 10.2.008 拉格朗日单点求值、多点插值；这里解决的是基转换，节点不要求互异。

```
signed main()
{
    assert(monoToNewton({11, -13, 6}, {1, 2, 3}) ==
```

```
            vector<Z>({4, 5, 6}));
}
```

10.2.020 多项式 half-GCD 与模逆

普通多项式欧几里得算法每轮都对完整余式做一次除法。half-GCD 只观察高半系数，递归求出一个 2×2 多项式矩阵，把余式对 $(f,g)^T$ 的第二项次数至少降低一半；必要时补一次欧几里得步，再继续递归。`half` 的输入必须已经严格次数递减，因此 `polyGcd` 遇到等次数余式时先做普通 Euclid 一步，不能把等长切片递归下去。矩阵始终表示若干次变换 $(a,b)\mapsto(b,a-qb)$ 的乘积，因此最终首行同时给出 Bézout 系数，满足 $d=xf+yg$。

矩阵乘法的多个卷积共享同一变换长度：本实现直接调用 10.1.003 快速 NTT 的公共 `dft`、`idft`，每个输入只正变换一次，再在频域完成两项点积；没有复制单位根表或蝶形。欧几里得步直接复用 `polyDivmod` 已算出的余式，并按稀疏矩阵公式更新变换矩阵，避免再乘一次原余式对。长度不超过 128 时普通 Euclid 常数更小，因此作为递归叶子。

```
// 依赖 10.2.001 多项式封装
// 依赖 10.2.002 乘法、除法、取模
Poly polyMonic(Poly a)
{
    // a 是输入多项式；返回删除高次零项后的首一化结果，零多项式返回空。
    a.norm();
    if (!a.empty())
    {
        a *= a.back().inv();
    }
    return a;
}

Poly polyMod(Poly a, const Poly &m)
{
    // a 是被除式，m 是非零模多项式；返回 a 除以 m 的余式。
    return polyDivmod(move(a), m).second;
}

Poly polyPowMod(Poly a, long long e, const Poly &m)
{
    // a 是底数多项式，e 是非负指数，m 是次数至少为 1 的模多项式；返回 a^e mod m。
    assert(e >= 0 && m.size() > 1); // 调试检查，可删。
    Poly r{1};
    a = polyMod(move(a), m);
    while (e)
    {
        if (e & 1)
        {
            r = polyMod(r * a, m);
        }
        e >>= 1;
        if (e)
```

```
        {
            a = polyMod(a * a, m);
        }
    }
    return r;
}

namespace hg
{
using Pair = array<Poly, 2>;
using Mat = array<Poly, 4>;

Poly add(Poly a, const Poly &b)
{
    // a、b 是两个多项式；返回去除高次零项后的 a+b。
    a += b;
    a.norm();
    return a;
}

Poly sub(Poly a, const Poly &b)
{
    // a、b 是两个多项式；返回去除高次零项后的 a-b。
    a -= b;
    a.norm();
    return a;
}

int mulLen(const Poly &a, const Poly &b)
{
    // a、b 是两个多项式；返回乘积系数数，任一多项式为零时返回 0。
    return a.empty() || b.empty() ? 0 : a.size() + b.size() - 1;
}

Mat mul(const Mat &a, const Mat &b)
{
    // a、b 是按行展开的 2×2 多项式矩阵；返回矩阵乘积。
    array<int, 4> s{
        max(mulLen(a[0], b[0]), mulLen(a[1], b[2])),
        max(mulLen(a[0], b[1]), mulLen(a[1], b[3])),
        max(mulLen(a[2], b[0]), mulLen(a[3], b[2])),
        max(mulLen(a[2], b[1]), mulLen(a[3], b[3]))}; // s[i] 是结果第 i 项在消去高次零前的长度。
    int n = bit_ceil((unsigned long long)max({1LL, s[0], s[1], s[2], s[3]}));
    bool zero = false; // zero 表示存在零输入，此时逐项卷积更适合稀疏矩阵。
    for (const auto &x : a)
    {
        zero |= x.empty();
    }
    for (const auto &x : b)
    {
        zero |= x.empty();
    }
```

```
    if (zero || n < 64)
    {
        return {add(a[0] * b[0], a[1] * b[2]),
                add(a[0] * b[1], a[1] * b[3]),
                add(a[2] * b[0], a[3] * b[2]),
                add(a[2] * b[1], a[3] * b[3])};
    }
    assert(n <= (1 << 23)); // 调试检查，可删。
    array<Poly, 8> f{a[0], a[1], a[2], a[3], b[0], b[1], b[2], b[3]};
    for (auto &x : f)
    {
        x.resize(n);
        dft(x);
    }
    Mat c{Poly(n), Poly(n), Poly(n), Poly(n)};
    for (int i = 0; i < n; i++)
    {
        c[0][i] = f[0][i] * f[4][i] + f[1][i] * f[6][i];
        c[1][i] = f[0][i] * f[5][i] + f[1][i] * f[7][i];
        c[2][i] = f[2][i] * f[4][i] + f[3][i] * f[6][i];
        c[3][i] = f[2][i] * f[5][i] + f[3][i] * f[7][i];
    }
    for (int i = 0; i < 4; i++)
    {
        idft(c[i]);
        c[i].resize(s[i]);
        c[i].norm();
    }
    return c;
}

Pair mul(const Mat &a, const Pair &b)
{
    // a 是按行展开的 2×2 多项式矩阵，b 是两维列向量；返回矩阵与列向量的乘积。
    int s0 = max(mulLen(a[0], b[0]), mulLen(a[1], b[1]));
    int s1 = max(mulLen(a[2], b[0]), mulLen(a[3], b[1]));
    int n = bit_ceil((unsigned long long)max({1LL, s0, s1}));
    bool zero = b[0].empty() || b[1].empty();
    for (const auto &x : a)
    {
        zero |= x.empty();
    }
    if (zero || n < 128)
    {
        return {add(a[0] * b[0], a[1] * b[1]),
                add(a[2] * b[0], a[3] * b[1])};
    }
    assert(n <= (1 << 23)); // 调试检查，可删。
    array<Poly, 6> f{a[0], a[1], a[2], a[3], b[0], b[1]};
    for (auto &x : f)
    {
        x.resize(n);
```

```
        dft(x);
    }
    Pair c{Poly(n), Poly(n)};
    for (int i = 0; i < n; i++)
    {
        c[0][i] = f[0][i] * f[4][i] + f[1][i] * f[5][i];
        c[1][i] = f[2][i] * f[4][i] + f[3][i] * f[5][i];
    }
    idft(c[0]);
    idft(c[1]);
    c[0].resize(s0);
    c[1].resize(s1);
    c[0].norm();
    c[1].norm();
    return c;
}

Mat id()
{
    // 无参数；返回 2×2 多项式单位矩阵。
    Mat a;
    a[0] = {1};
    a[3] = {1};
    return a;
}

Mat step(const Poly &q)
{
    // q 是一次欧几里得除法的商；返回把 (a,b) 变为 (b,a-qb) 的矩阵。
    Mat a;
    a[1] = {1};
    a[2] = {1};
    a[3] = sub({}, q);
    return a;
}

Mat stepMul(Mat a, const Poly &q)
{
    // a 是变换矩阵，q 是欧几里得商；返回 step(q)*a，并利用 step(q) 的稀疏结构。
    Poly x = q * a[2];
    Poly y = q * a[3];
    return {move(a[2]), move(a[3]),
            sub(move(a[0]), x), sub(move(a[1]), y)};
}

Mat halfSmall(Pair a)
{
    // a 是次数递减且第一项长度不超过 128 的余式对；用普通 Euclid 返回半降矩阵。
    int m = a[0].size() / 2;
    Mat ans = id();
    while (!a[1].empty() && (int)a[1].size() > m)
    {
```

```
        auto [q, r] = polyDivmod(move(a[0]), a[1]);
        ans = stepMul(move(ans), q);
        a = {move(a[1]), move(r)};
    }
    return ans;
}

Mat half(const Pair &a)
{
    // a[0]、a[1] 是次数递减的余式对；返回把第二项次数降到第一项一半以下的变换矩阵。
    int m = a[0].size() / 2;
    if ((int)a[1].size() <= m)
    {
        return id();
    }
    if ((int)a[0].size() <= 128)
    {
        return halfSmall(a);
    }
    Mat ans = half({Poly(a[0].begin() + m, a[0].end()),
                    Poly(a[1].begin() + m, a[1].end())});
    Pair b = mul(ans, a);
    if ((int)b[1].size() <= m)
    {
        return ans;
    }
    auto [q, r] = polyDivmod(move(b[0]), b[1]);
    ans = stepMul(move(ans), q);
    b = {move(b[1]), move(r)};
    if ((int)b[1].size() <= m)
    {
        return ans;
    }
    int cut = 2 * m + 1 - b[0].size();
    Mat suf = half({Poly(b[0].begin() + cut, b[0].end()),
                    Poly(b[1].begin() + cut, b[1].end())});
    return mul(suf, ans);
}

Mat all(const Pair &a)
{
    // a[0]、a[1] 是次数递减的余式对；返回把第二项降为零的完整变换矩阵。
    Mat ans = half(a);
    Pair b = mul(ans, a);
    if (b[1].empty())
    {
        return ans;
    }
    auto [q, r] = polyDivmod(move(b[0]), b[1]);
    ans = stepMul(move(ans), q);
    b = {move(b[1]), move(r)};
    if (b[1].empty())
```

```
    {
        return ans;
    }
    return mul(all(b), ans);
}
}

Poly polyGcd(Poly f, Poly g)
{
    // f、g 是不全为零的多项式；两者非空时长度均不超过 2^22，返回首一 gcd，不计算最终 Bézout 系数。
    f.norm();
    g.norm();
    assert(!f.empty() || !g.empty()); // 调试检查，可删。
    if (f.empty())
    {
        return polyMonic(move(g));
    }
    if (g.empty())
    {
        return polyMonic(move(f));
    }
    assert(max(f.size(), g.size()) <= (1 << 22)); // 调试检查，可删。
    if (f.size() < g.size())
    {
        swap(f, g);
    }
    hg::Pair a{move(f), move(g)};
    while (!a[1].empty())
    {
        if (a[1].size() < a[0].size() &&
            (int)a[1].size() > (int)a[0].size() / 2)
        {
            hg::Mat t = hg::half(a);
            a = hg::mul(t, a);
            if (a[1].empty())
            {
                break;
            }
        }
        Poly r = polyDivmod(move(a[0]), a[1]).second;
        a = {move(a[1]), move(r)};
    }
    return polyMonic(move(a[0]));
}

tuple<Poly, Poly, Poly> polyExtGcd(Poly f, Poly g)
{
    // f、g 是不全为零的多项式；两者非空时长度均不超过 2^22，返回 {d,x,y}，满足 d=xf+yg=gcd(f,g)，d
    ↪ 未首一化。
    f.norm();
    g.norm();
    assert(!f.empty() || !g.empty()); // 调试检查，可删。
```

```
    if (g.empty())
    {
        return {f, Poly{1}, Poly{}};
    }
    if (f.empty())
    {
        return {g, Poly{}, Poly{1}};
    }
    assert(max(f.size(), g.size()) <= (1 << 22)); // 调试检查，可删。
    if (f.size() < g.size())
    {
        auto [d, y, x] = polyExtGcd(move(g), move(f));
        return {move(d), move(x), move(y)};
    }
    using namespace hg;
    auto [q, r] = polyDivmod(f, g);
    Mat ans = step(q);
    Pair rem{g, move(r)};
    if (!rem[1].empty())
    {
        ans = mul(all(rem), ans);
    }
    Poly d = add(f * ans[0], g * ans[1]);
    return {move(d), move(ans[0]), move(ans[1])};
}

optional<Poly> polyInvMod(Poly f, Poly m)
{
    // f 是待求逆多项式，m 是非零模多项式；两者非空且 m 非常数时长度均不超过 2^22，可逆时返回次数小于 m 的逆元，否则返回空。
    f.norm();
    m.norm();
    assert(!m.empty()); // 调试检查，可删。
    if (m.size() == 1)
    {
        return Poly{};
    }
    if (!f.empty())
    {
        assert(max(f.size(), m.size()) <= (1 << 22)); // 调试检查，可删。
    }
    auto [d, x, y] = polyExtGcd(move(f), m);
    if (d.size() != 1)
    {
        return nullopt;
    }
    x *= d[0].inv();
    return polyDivmod(move(x), move(m)).second;
}
```

`polyExtGcd(f,g)` 要求二者不全为零，两者非空时最大长度不超过 2^{22}，返回未首一化的 `{d,x,y}`；同一长度界也用于 `polyGcd`。`polyInvMod(f,m)` 要求 $\deg m \geq 1$，当 `f` 非零时两输入同样不超过 2^{22}；仅当

$\gcd(f,m)$ 为非零常数时返回次数小于 m 的逆元，否则返回空。这个界保证 half-GCD 内部两项点积补齐后的公共 NTT 长度不超过 2^{23}。设 $n=\max(\deg f,\deg g)$，时间 $O(M(n)\log n)$；父层矩阵与余式、子层高半切片以及一次批量变换的长度都按层构成几何级数，峰值额外空间为 $O(n)$。只需要首一 gcd 时由 10.2.015 有限域多项式求根与因式分解 的 `polyGcd` 调用本条；小次数现场实现仍可用普通欧几里得算法换取更短代码。

```
signed main()
{
    auto inv = polyInvMod({0, 1}, {1, 0, 1});
    assert(inv && *inv == Poly({0, -1}));
    assert(!polyInvMod({1, 1}, {1, 2, 1}));
}
```

10.2.021 拉格朗日非连续点单点求值

给定互异横坐标 $x_0,x_1,\dots,x_{n-1}$ 及对应点值 $y_i=f(x_i)$，在一个查询点 t 求次数小于 n 的插值多项式 $f(t)$。直接枚举每个拉格朗日基函数

$$f(t)=\sum_{i=0}^{n-1} y_i\prod_{j\neq i}\frac{t-x_j}{x_i-x_j}$$

即可；它不要求横坐标连续或等距，代码短但时间为 $O(n^2)$。这里使用已有的模整数类 `Z`，因此所有分母必须在模下可逆；若只需一次求值且 n 较小，优先用本条，若要恢复多项式或处理大规模数据则用 10.2.008 拉格朗日单点求值、多点插值 的 `interpPoints`。

```
// 依赖 1.1.008 静态取模类
Z lagrange(const vector<Z> &x, const vector<Z> &y, Z t)
{
    // x 是互异横坐标，y 是对应点值，t 是查询点；返回插值多项式在 t 处的值。
    assert(x.size() == y.size() && !x.empty()); // 调试检查，可删。
    int n = x.size();
    for (int i = 0; i < n; i++)
    {
        if (x[i] == t)
        {
            return y[i];
        }
    }
    Z ans = 0;
    for (int i = 0; i < n; i++)
    {
        Z c = y[i];
        for (int j = 0; j < n; j++)
        {
            if (i != j)
            {
                c *= (t - x[j]) / (x[i] - x[j]);
            }
        }
```

```
        ans += c;
    }
    return ans;
}
```

例如三点 $(0,1),(1,4),(2,9)$ 插值为 $1+2x+x^2$，`lagrange({0,1,2},{1,4,9},10)` 返回 121。横坐标必须两两不同，且每个 x_i-x_j 在模下非零；时间 $O(n^2)$、额外空间 $O(1)$。它只求一个点，不生成多项式系数；连续整数点应使用 `lagrangeSeq`，需要多个任意点时使用 `interpPoints`。

10.2.022 计数版子集和生成函数

给定非负整数多重集 a，对每个 $0\le s\le T$ 求和为 s 的子集个数（模 998244353），即 $\prod_i(1+x^{a_i})$ 的前 $T+1$ 项。把权值相同的元素合并，设权值 v 出现 c_v 次，取对数

$$\ln\prod_i(1+x^{a_i})=\sum_{v\ge1}c_v\ln(1+x^v)=\sum_{v\ge1}c_v\sum_{k\ge1}\frac{(-1)^{k+1}}{k}x^{kv},$$

对每个出现过的 v 只需枚举 $k\le T/v$，总项数不超过调和级数 $T\ln T$；分母 $k\le T<998244353$，模意义下可逆。对这个对数做 `fpsExp` 即得答案；权值为 0 的元素可选可不选，整体乘 2^{c_0}。

```
// 依赖 10.2.005 ln、exp、开根、快速幂
vector<Z> subsetCount(const vector<int> &a, int T)
{
    // a 是非负整数权，0<=T<2^22；返回 f[0..T]，f[s] 是和为 s 的子集个数模 998244353。
    // prod (1+x^v) = exp(sum_v sum_k (-1)^{k+1} x^{kv}/k)，按权值合并后调和级数求 ln，再 exp。
    assert(0 <= T && T < (1LL << 22)); // 调试检查，可删。
    vector<int> cnt(T + 1); // cnt[v] 是权值为 v 的元素个数（只统计 1..T）。
    int zeros = 0;
    for (int x : a)
    {
        assert(x >= 0); // 调试检查，可删。
        if (!x)
        {
            zeros++;
        }
        else if (x <= T)
        {
            cnt[x]++;
        }
    }
    const auto &iv = invTable<mod>(T);
    Poly lg(T + 1);
    for (int v = 1; v <= T; v++)
    {
        if (cnt[v])
        {
            for (int k = 1; k * v <= T; k++)
            {
                lg[k * v] += Z(cnt[v]) * (k & 1 ? iv[k] : -iv[k]);
            }
```

```
        }
    }
    // 权为 0 的元素可选可不选，整体乘 2^zeros。
    Poly f = fpsExp(lg, T + 1);
    Z sc = Z(2).pow(zeros);
    for (auto &x : f)
    {
        x *= sc;
    }
    return f;
}
```

`subsetCount(a,T)` 要求权值非负、$0 \le T < 2^{22}$，返回长 $T+1$ 的 `vector<Z>`，大于 T 的权值不影响结果。时间 $O(T \log T)$，空间 $O(T)$。依赖 10.2.005 `ln、exp、开根、快速幂` 的 `fpsExp` 与第 1 章的逆元表 `invTable`。

只问可达性时用 12.1.010 布尔母函数 / 有界子集和可达性；nT 不大时直接做 01 背包更简单。下面核对多重集 $\{1,2,3,3,0\}$：

```
signed main()
{
    auto f = subsetCount({1, 2, 3, 3, 0}, 6);
    vector<int> want{2, 2, 2, 6, 4, 4, 6}, got;
    for (auto x : f)
    {
        got.push_back(x.val());
    }
    assert(got == want);
}
```

10.3 生成函数

10.3.001 普通生成函数 OGF

普通生成函数 $A(x)=\sum a_n x^n$ 编码无标号序列，x^n 系数直接是大小 n 的对象数；加法对应互斥选择、乘法（卷积）对应有序拼接、$\frac{1}{1-A}$ 对应序列。这段只给最基础的系数乘法示意，要求所有乘加结果均在有符号 64 位范围内；需要取模或大规模计算时改用多项式全家桶。

```
// 依赖 1.1.008 静态取模类
namespace ogf
{
vector<int> ogfProduct(const vector<int> &a, const vector<int> &b)
{
    // a、b 是 OGF 系数；返回乘积系数，即普通卷积。
    if (a.empty() || b.empty())
    {
        return {};
    }
    vector<int> c(a.size() + b.size() - 1);
```

```
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = 0; j < (int)b.size(); j++)
        {
            c[i + j] += a[i] * b[j];
        }
    }
    return c;
}
}
```

无标号计数用 OGF、有标号用 10.3.002 指数生成函数 EGF，两者系数归一方式不同、不能混算。这里的 `ogfProduct` 是 $O(nm)$ 朴素卷积示意；真正计算依赖 10.2.001 多项式封装 与 10.1.003 NTT。

10.3.002 指数生成函数 EGF

指数生成函数 $A(x)=\sum \frac{a_n}{n!}x^n$ 编码有标号对象，乘法 $\frac{a_i}{i!}\cdot\frac{b_j}{j!}$ 自动带上 $\binom{i+j}{i}$，正好表示把标签集合划给两部分，所以有标号结构的不交合并对应 EGF 乘法。这里演示乘 $n!$ 前系数的卷积，要求帕斯卡表和全部乘加中间值均在有符号 64 位范围内。

```
namespace egf
{
vector<int> egfProduct(const vector<int> &a, const vector<int> &b)
{
    // a[n]、b[n] 是乘 n! 前的 EGF 系数；返回有标号乘积系数。
    if (a.empty() || b.empty())
    {
        return {};
    }
    vector<int> ans(a.size() + b.size() - 1);
    vector<int> c(b.size(), 1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (i)
        {
            for (int j = 1; j < (int)b.size(); j++)
            {
                c[j] += c[j - 1];
            }
        }
        for (int j = 0; j < (int)b.size(); j++)
        {
            ans[i + j] += c[j] * a[i] * b[j];
        }
    }
    return ans;
}
}
```

有标号用 EGF、无标号用 10.3.001 普通生成函数 OGF，系数不可直接互换。实现逐行维护 $\binom{i+j}{i}$，对长

度分别为 n, m 的输入用 $O(nm)$ 时间、$O(n+m)$ 空间；实际模意义大规模计算以 10.2.001 多项式封装 承载，指数、对数等运算见 10.2.005 ln、exp、开根、快速幂。

10.3.003 q-int / q-Binomial

q-整数 $[n]_q = 1 + q + \cdots + q^{n-1}$、q-阶乘与高斯二项式 $\binom{n}{k}_q$ 把普通计数按维数或逆序数加权：$\binom{n}{k}_q$ 数 $\mathbb{F}_q$ 上 n 维空间的 k 维子空间，或按逆序对数加权的选择。下面使用递推 $\binom{n}{k}_q = \binom{n-1}{k}_q + q^{n-k}\binom{n-1}{k-1}_q$，$q = 1$ 退化为普通组合数；所有整数系数须在有符号 64 位范围内。

```
namespace gauss_binom
{
vector<int> qBinomial(int n, int k)
{
    // n、k 是高斯二项式参数；返回按 q 次数升序的整数系数。
    if (k < 0 || k > n)
    {
        return {};
    }
    k = min(k, n - k);
    int w = n - k;
    vector<int> a{1};
    for (int i = 1; i <= k; i++)
    {
        vector<int> b(i * w + 1);
        for (int j = 0; j <= i * w; j++)
        {
            if (j < (int)a.size())
            {
                b[j] += a[j];
            }
            if (j >= w + i && j - w - i < (int)a.size())
            {
                b[j] -= a[j - w - i];
            }
            if (j >= i)
            {
                b[j] += b[j - i];
            }
        }
        a = move(b);
    }
    return a;
}
}
```

`qBinomial(n,k)` 返回按 q 次数升序的整数系数多项式。实现先用对称性令 $k \le n-k$，再逐项应用

$$(1-q^i)\binom{n-k+i}{i}_q = (1-q^{n-k+i})\binom{n-k+i-1}{i-1}_q,$$

因此每轮只保留一个系数数组。令 $D=k(n-k)$ 为答案次数，时间 $O(kD)$、空间 $O(D)$；$q=1$ 特化为普通组合数，见 8.1.001 阶乘、逆阶乘与组合数。

10.3.004 二项式型生成函数

二项式变换 $b_n=\sum_k \binom{n}{k}a_k$ 及其反演 $a_n=\sum_k(-1)^{n-k}\binom{n}{k}b_k$，对应 EGF 乘以 e^x / e^{-x}。二项式型序列满足 $p_n(x+y)=\sum_k \binom{n}{k}p_k(x)p_{n-k}(y)$，其 EGF 是某形式幂级数的幂，适合处理卷积型多项式序列。以下整数版要求帕斯卡表和全部乘加中间值均在有符号 64 位范围内。

```cpp
namespace binom_trans
{
vector<int> binomTrans(const vector<int> &a, bool inverse = false)
{
    // a 是原序列，inverse 表示做逆变换；返回二项式变换结果。
    int n = a.size();
    vector<int> b(n), c{1};
    for (int i = 0; i < n; i++)
    {
        if (i)
        {
            c.push_back(0);
        }
        for (int j = i; j >= 1; j--)
        {
            c[j] += c[j - 1];
        }
        for (int j = 0; j <= i; j++)
        {
            int sign = inverse && ((i - j) & 1) ? -1 : 1;
            b[i] += sign * c[j] * a[j];
        }
    }
    return b;
}
}
```

`binomTrans(a,inverse)` 做正/逆二项式变换。实现只滚动保存一行 Pascal 系数，用 $O(n^2)$ 时间、$O(n)$ 空间；写成模意义 EGF 乘 $e^{\pm x}$ 可借 10.2.005 ln、exp、开根、快速幂 降到 $O(n\log n)$。它是容斥的生成函数形式，与 10.3.002 指数生成函数 EGF 同源。

10.3.005 拉格朗日反演

由隐式方程 $F=x\phi(F)$（$\phi(0)\neq 0$）求递归结构计数时，拉格朗日反演给出 $[x^n]F=\frac{1}{n}[x^{n-1}]\phi(x)^n$，更一般地 $[x^n]H(F)=\frac{1}{n}[x^{n-1}]H'(x)\phi(x)^n$。`lagrangeCoef` 直接用一次 FPS 幂计算指定系数。

完整复合逆则把全部系数打包。设 $f(x)=ax+O(x^2)$、$a\neq 0$，先归一化 $F=f/a=x+O(x^2)$，令 $d=n-1$ 并定义

$$A(z)=\sum_{i=1}^{d}\frac{d}{i}[x^d]F(x)^i z^{d-i}.$$

由拉格朗日反演逐项比较可得 $F^{\langle -1\rangle}(z)\equiv zA(z)^{-1/d}\pmod{z^{d+1}}$。因此先用 10.2.011 Power Projection 一次求出全部 $[x^d]F^i$，再做一次单位 FPS 的分数次幂，最后代入 $z=x/a$ 恢复原函数的逆；这避免对每个次数分别计算不同幂。

```cpp
// 依赖 1.1.008 静态取模类
// 依赖 7.4.004 二次剩余
// 依赖 10.1.003 NTT
// 依赖 10.2.001 多项式封装
// 依赖 10.2.003 乘法逆元
// 依赖 10.2.004 导数、积分
// 依赖 10.2.005 ln、exp、开根、快速幂
// 依赖 10.2.011 Power Projection
// 依赖 10.2.012 多项式复合
namespace lagrange
{
int lagrangeCoef(vector<int> phi, int n)
{
    // phi 是 phi(t) 的低次系数，n 为正；返回方程 w=x*phi(w) 中 [x^n]w。
    assert(n > 0 && n < mod && !phi.empty() && phi[0] != 0); // 调试检查，可删。
    Poly f(n);
    for (int i = 0; i < min(n, (int)phi.size()); i++)
    {
        f[i] = phi[i];
    }
    return (fpsPow(f, n, n)[n - 1] / Z(n)).val();
}
}

namespace cinv
{
Poly powUnit(const Poly &f, Z k, int n)
{
    // f[0]=1，k 是模意义指数，n 是截断长度；返回 f^k mod x^n。
    assert(n >= 0 && (n == 0 || (!f.empty() && f[0] == Z(1)))); // 调试检查，可删。
    if (n == 0)
    {
        return {};
    }
    return fpsExp(fpsLog(f, n) * k, n);
}
}

Poly compInverse(const Poly &f, int n)
{
    // f[0]=0、f[1]≠0，n 是截断项数；返回满足 f(g(x))=x mod x^n 的 g。
    assert(n >= 0 && n <= (1 << 20)); // 调试检查，可删。
    if (n == 0)
```

```
    {
        return {};
    }
    if (n == 1)
    {
        return Poly(1);
    }
    assert(!f.empty() && f[0] == Z(0)); // 调试检查，可删。
    assert(f.size() >= 2 && f[1] != Z(0)); // 调试检查，可删。

    Z a = f[1], ia = a.inv();
    Poly h = f.cut(n) * ia;
    int d = n - 1;
    vector<Z> w(d + 1);
    w[d] = 1;
    auto c = powerProj(h, w, d + 1);

    Poly q(d);
    for (int i = 1; i <= d; i++)
    {
        q[d - i] = Z(d) * c[i] / Z(i);
    }
    assert(q[0] == Z(1)); // 调试检查，可删。
    q = cinv::powUnit(q, -Z(d).inv(), d);

    Poly ans(n);
    Z pw = ia;
    for (int i = 1; i < n; i++)
    {
        ans[i] = q[i - 1] * pw;
        pw *= ia;
    }
    return ans;
}
```

`lagrangeCoef(phi,n)` 返回 $w = x\phi(w)$ 中 $[x^n]w$，要求 $\phi(0) \neq 0$、$0 < n < 998244353$；受基础 FPS 逆上限约束，当前实现实际要求 $n \leq 2^{22}$。它的复杂度继承 10.2.005 `ln`、`exp`、开根、快速幂 的 `fpsPow`：默认范围 $n \leq 2^{20}$ 为 $O(n \log^2 n)$，更长时走 Newton 路径。

`compInverse(f,n)` 要求 $f_0 = 0$、$n \leq 2^{20}$，当 $n \geq 2$ 时还要求 $f_1 \neq 0$；返回恰 n 项并满足 $f(g(x)) \equiv x \pmod{x^n}$，`n=0` 返回空、`n=1` 返回零常数项。幂投影和分数次幂合计 $O(n \log^2 n)$ 时间、$O(n)$ 峰值额外空间。正向复合见 10.2.012 多项式复合。同一 Ryzen 9 7945HX / GCC 14.2.0 / `-O2 -DNDEBUG` 环境下，种子 `20260816` 的 $n = 131072$ 稠密随机输入单次求逆约 1162.32 ms。

下面用 $w = x(1+w)^2$ 的前四项 $1, 2, 5, 14$ 快速核对下标与除以 n：

```
signed main()
{
    using namespace lagrange;
    assert(lagrangeCoef({1, 2, 1}, 1) == 1);
    assert(lagrangeCoef({1, 2, 1}, 4) == 14);
```

```
    Poly inv = compInverse({0, 1, 1}, 6);
    assert(inv == Poly({0, 1, -1, 2, -5, 14}));
    assert(composePoly({0, 1, 1}, inv, 6) == Poly({0, 1}));
    cout << "OK\n";
    return 0;
}
```

10.3.006 两边界间非降序列计数

计数满足 $0 \le x_0 \le \cdots \le x_{n-1}$ 且 $A_i \le x_i < B_i$ 的整数序列，输入边界本身不要求单调。先把上界做后缀最小、下界做前缀最大，问题变成上下两条单调边界之间的网格路径计数。`block` 沿中线分割一个单调网格块；跨过分界线的转移是二项式系数和，`enumF`、`enumG` 分别把整段所需值化为一次 NTT 卷积。主过程沿边界交替向上、向右推进，每次把边界差分作为下一块起始值。

```
// 依赖 1.1.008 静态取模类
struct IncSeq
{
    int sz; // 规范化后的边界点数，包含末尾哨兵点。
    bool bad = false; // 是否已经判定不存在合法序列。
    vector<int> lo, hi; // 单调下边界与单调含端点上边界。
    vector<Z> fac, ifac; // 阶乘与逆阶乘表。

    vector<Z> enumF(int n, vector<Z> v, int m)
    {
        // n 是 v 的长度，m 是输出长度；返回 ans[i]=sum(v[j]*C(n-1+i-j,i))。
        reverse(v.begin(), v.end());
        for (int i = 0; i < n; i++)
        {
            v[i] *= ifac[i];
        }
        reverse(v.begin(), v.end());
        assert(convFit(n, n + m - 1)); // 调试检查，可删。
        vector<Z> c(n + m - 1);
        for (int i = 0; i < n + m - 1; i++)
        {
            c[i] = fac[i];
        }
        auto z = conv(v, c);
        vector<Z> ans(m);
        for (int i = 0; i < m; i++)
        {
            ans[i] = z[n - 1 + i] * ifac[i];
        }
        return ans;
    }

    vector<Z> enumG(int n, const vector<Z> &v, int l)
    {
        // n 是 v 与输出的长度，l 是固定下标；返回 ans[i]=sum(v[j]*C(l+i-j,l),j=0..i)。
        assert(convFit(n, n)); // 调试检查，可删。
```

```
        vector<Z> c(n);
        for (int i = 0; i < n; i++)
        {
            c[i] = fac[l + i] * ifac[i];
        }
        auto z = conv(v, c);
        vector<Z> ans(n);
        for (int i = 0; i < n; i++)
        {
            ans[i] = z[i] * ifac[l];
        }
        return ans;
    }

    vector<Z> block(int n, const vector<int> &h, const vector<Z> &st)
    {
        // h 是 n 列单调网格块的高度，st 是各列底边入口值；返回最右边界各高度的路径计数。
        int m = h[n - 1];
        if (n == 1)
        {
            vector<Z> ans(m + 1);
            fill(ans.begin(), ans.end(), st[0]);
            return ans;
        }
        if (n == 2)
        {
            vector<Z> ans(m + 1);
            for (int i = 0; i <= m; i++)
            {
                ans[i] = st[0] * min(i + 1, h[0] + 1) + st[1];
            }
            return ans;
        }

        int mid = n / 2, cut = h[mid];
        vector<int> left(mid + 1), right(n - mid);
        vector<Z> ls(mid + 1), rs(n - mid);
        for (int i = 0; i <= mid; i++)
        {
            left[i] = h[i];
            ls[i] = i == mid ? Z(0) : st[i];
        }
        for (int i = mid; i < n; i++)
        {
            right[i - mid] = h[i] - cut;
            rs[i - mid] = st[i];
        }

        auto lv = block(mid + 1, left, ls);
        for (int i = cut; i >= 1; i--)
        {
            lv[i] -= lv[i - 1];
```

```
        }
        auto lf = enumF(cut + 1, lv, n - mid);
        auto lg = enumG(cut + 1, lv, n - mid - 1);
        auto rf = enumF(n - mid, rs, cut + 1);
        auto rg = enumG(n - mid, rs, cut);
        vector<Z> pre(cut);
        for (int i = 0; i < cut; i++)
        {
            pre[i] = lg[i] + rf[i];
        }
        for (int i = 0; i < n - mid; i++)
        {
            rs[i] = lf[i] + rg[i];
        }
        for (int i = n - mid - 1; i >= 1; i--)
        {
            rs[i] -= rs[i - 1];
        }
        auto rv = block(n - mid, right, rs);
        vector<Z> ans(m + 1);
        copy(pre.begin(), pre.end(), ans.begin());
        for (int i = 0; i < (int)rv.size(); i++)
        {
            ans[i + cut] = rv[i];
        }
        return ans;
    }

    IncSeq(vector<int> a, vector<int> b) : sz(a.size()), lo(move(a)), hi(move(b))
    {
        // a、b 是非空等长的半开下、上界；构造后保存可供 get 计算的单调网格边界。
        assert(sz > 0 && (int)hi.size() == sz); // 调试检查，可删。
        for (int i = 0; i < sz; i++)
        {
            assert(lo[i] >= 0 && hi[i] >= 0); // 调试检查，可删。
            hi[i]--;
            if (lo[i] > hi[i] || (i && hi[i] < lo[i - 1]))
            {
                bad = true;
            }
        }
        for (int i = sz - 2; i >= 0; i--)
        {
            hi[i] = min(hi[i], hi[i + 1]);
        }
        for (int i = 1; i < sz; i++)
        {
            lo[i] = max(lo[i], lo[i - 1]);
        }
        int al = lo[sz - 1], bl = hi[sz - 1];
        for (int i = sz - 1; i >= 1; i--)
        {
```

```
            lo[i] = lo[i - 1];
        }
        int dec = lo[0];
        for (auto &x : lo)
        {
            x -= dec;
        }
        for (auto &x : hi)
        {
            x -= dec;
        }
        sz++;
        lo.push_back(al - dec);
        // 哨兵只需比原末端高一格，使边界游走在原终点后经过唯一的收束竖边。
        hi.push_back(bl + 1 - dec);
        for (int i = 0; i < sz; i++)
        {
            if (lo[i] > hi[i] || (i && hi[i] < lo[i - 1]))
            {
                bad = true;
            }
        }

        if (bad)
        {
            return;
        }
        int m = sz + hi.back() - 1;
        assert(0 < m && m <= mod); // 调试检查，可删。
        fac.assign(m, 1);
        ifac.assign(m, 1);
        for (int i = 1; i < m; i++)
        {
            fac[i] = fac[i - 1] * i;
        }
        ifac[m - 1] = fac[m - 1].inv();
        for (int i = m - 1; i; i--)
        {
            ifac[i - 1] = ifac[i] * i;
        }
    }

    Z get()
    {
        // 无参数；返回原半开边界内非降整数序列数，不可行时返回 0。
        if (bad)
        {
            return 0;
        }
        int dist = upper_bound(lo.begin(), lo.end(), lo[0]) - lo.begin();
        int px = 0, py = lo[0], qx = dist - 1, qy = lo[0];
        if (qx == 0)
```

```
        {
            qy = hi[0];
        }
        vector<Z> cur(abs(qx - px) + abs(qy - py) + 1);
        cur[0] = 1;
        while (qx != sz - 1 || qy != hi[sz - 1])
        {
            for (int i = cur.size() - 1; i >= 1; i--)
            {
                if (i == 1 && px == 0 && py == 0)
                {
                    break;
                }
                cur[i] -= cur[i - 1];
            }
            if (py == qy)
            {
                vector<int> h(qx - px + 1);
                for (int i = 0; i <= qx - px; i++)
                {
                    h[i] = hi[px + i] - py;
                }
                cur = block(qx - px + 1, h, cur);
                px = qx;
                py = qy;
                qy = hi[qx];
            }
            else
            {
                int to = upper_bound(lo.begin(), lo.end(), qy) - lo.begin();
                vector<int> h(qy - py + 1);
                for (int i = 0; i <= qy - py; i++)
                {
                    h[i] = upper_bound(lo.begin(), lo.end(), py + i) - lo.begin() - px;
                }
                for (int i = qy - py; i >= 0; i--)
                {
                    h[i] -= h[0];
                }
                cur = block(qy - py + 1, h, cur);
                px = qx;
                py = qy;
                qx = to - 1;
            }
        }
        return cur.back();
    }
};

Z countIncSeq(const vector<int> &a, const vector<int> &b)
{
    // a、b 是非空等长的半开下、上界；返回其中全部非降整数序列数。
```

```
    return IncSeq(a, b).get();
}
```

`countIncSeq(a,b)` 直接接收半开上界，返回满足 $a_i \le x_i < b_i$ 且 $x_0 \le x_1 \le \cdots \le x_{n-1}$ 的非降整数序列数；数组必须非空、等长且所有端点非负，不可行时返回 0，并在建立阶乘表前结束。末尾哨兵只比原末端上界高一格，使边界游走经过唯一收束竖边。`enumF(n,v,m)` 与 `enumG(n,v,l)` 分别要求 `convFit(n,n+m-1)` 与 `convFit(n,n)`，每个递归块都在调用公共卷积前检查实际长度。两者访问的最大阶乘下标为 `cut+n-mid`，全局恰不超过 `sz+hi.back()-2`，所以表长严格取 `sz+hi.back()-1`，不是经验倍增。`block` 在右递归前只保留长度 `cut` 的左前缀，等右侧返回后才分配完整答案，避免父层高度数组沿递归栈叠成 $\Omega(H\log n)$。令 H 为规范化后最大边界高度，在 $n, H \le 131072$ 的典型约束下，时间约 $O((n+H)\log^2(n+H))$、峰值空间 $O(n+H)$，依赖 10.1.003 NTT 的 `Z` 与 `conv`。边界很小时，$O(nH)$ 前缀和 DP 更短。

下面检查单点、非单调边界、不可行边界和一组 Catalan 型结果：

```
signed main()
{
    assert(countIncSeq({3}, {7}) == Z(4));
    assert(countIncSeq({1, 0, 2}, {4, 3, 5}) == Z(9));
    assert(countIncSeq({2, 0}, {3, 2}) == Z(0));
    assert(countIncSeq({0, 0, 0}, {2, 3, 4}) == Z(14));
}
```

10.4 线性递推

10.4.001 Berlekamp-Massey（BM / 最小线性递推）

Berlekamp–Massey 从域上序列前缀恢复最短常系数线性递推。维护当前递推 C 与上一次导致阶数增长的递推 B：算每项的预测差 d，非零时用错位缩放的 B 消去它；仅当前缀长度达到 $2L$ 才把阶数提升到 $n+1-L$ 并记下这次的 B。常用于只会造前若干项、却要求远项的题。

```
// 依赖 1.1.008 静态取模类
template <class T> vector<T> BM(const vector<T> &s)
{
    // s 是已知序列前缀；返回最短 rec，使 s[n]=sum(rec[i-1]*s[n-i])。
    vector<T> c, old;
    int len = 0, gap = 1;
    T last = 1;
    for (int n = 0; n < (int)s.size(); n++)
    {
        T d = s[n];
        for (int i = 1; i <= len; i++)
        {
            d += c[i - 1] * s[n - i];
        }
        if (d == T{})
        {
            gap++;
            continue;
```

```
        }
        auto pre = c;
        T q = -d / last;
        if ((int)c.size() < (int)old.size() + gap)
        {
            c.resize(old.size() + gap);
        }
        c[gap - 1] += q;
        for (int i = 0; i < (int)old.size(); i++)
        {
            c[i + gap] += q * old[i];
        }
        if (2 * len <= n)
        {
            len = n + 1 - len;
            old = move(pre);
            last = d;
            gap = 1;
        }
        else
        {
            gap++;
        }
    }
    for (auto &x : c)
    {
        x = -x;
    }
    return c;
}
```

`BM(s)` 的系数类型须构成域，返回 `rec` 后按 `a[n]=sum(rec[i-1]*a[n-i])` 续推；Fibonacci 前缀 `0,1,1,2,3,5,8,13` 恢复 `{1,1}`，常数列 `7,7,7,7` 恢复 `{1}`。$O(NL)$ 时间（最坏 $O(N^2)$）、$O(L)$ 空间（N 样本数、L 阶数）。已知递推求远项用 10.4.003 Linear Recurrence 或 10.4.005 Bostan-Mori / 分式第 N 项；把递推写成关系多项式见 10.4.002 最短递推式。

下面用 Fibonacci、常数列和全零列核对系数方向与零阶递推：

```
signed main()
{
    assert((BM<Z>({0, 1, 1, 2, 3, 5, 8, 13}) == vector<Z>{1, 1}));
    assert((BM<Z>({7, 7, 7, 7}) == vector<Z>{1}));
    assert(BM<Z>({0, 0, 0, 0}).empty());
    cout << "OK\n";
    return 0;
}
```

10.4.002 最短递推式

把最短递推写成常数项为 1 的关系多项式 $Q(x)=1-r_0x-\cdots-r_{k-1}x^k$。它直接就是序列 OGF 的分母，与多项式取模、Bostan–Mori 都以此为接口。此条只在 BM 结果上包一层符号转换，不重复 BM 本体。

```cpp
template <class T> vector<T> shortRel(const vector<T> &s)
{
    // s 是序列前缀；返回最短关系多项式 q，满足 sum(q[i]*s[n-i])=0 且 q[0]=1。
    auto rec = BM(s);
    vector<T> q(rec.size() + 1);
    q[0] = 1;
    for (int i = 0; i < (int)rec.size(); i++)
    {
        q[i + 1] = -rec[i];
    }
    return q;
}
```

`shortRel(s)` 返回 `q={1,-rec[0], ... }`，依赖 10.4.001 Berlekamp-Massey（BM / 最小线性递推） 的 `BM`，成本即 BM 的 $O(NL)$。得到 Q 后配上分子可交 10.4.005 Bostan-Mori / 分式第 N 项 求任意项，构造分子分母见 10.4.004 递推式转分式。

10.4.003 Linear Recurrence

已知 k 项初值与 k 阶递推求第 n 项。特征多项式 $C(x)=x^k-r_0x^{k-1}-\cdots-r_{k-1}$，则 $a_n=\sum_i([x^i](x^n \bmod C))a_i$：用快速幂在模 C 意义下算 x^n，每次多项式平方或乘法后做 $O(k^2)$ 暴力取模。不负责从样本猜递推。

```cpp
template <class T> vector<T> recMul(const vector<T> &a, const vector<T> &b,
                                    const vector<T> &rec)
{
    // a、b 是次数小于 k 的系数；返回乘积模特征多项式后的 k 项系数。
    int k = rec.size();
    assert(k > 0 && (int)a.size() == k && (int)b.size() == k); // 调试检查，可删
    vector<T> c(2 * k - 1);
    for (int i = 0; i < k; i++)
    {
        for (int j = 0; j < k; j++)
        {
            c[i + j] += a[i] * b[j];
        }
    }
    for (int i = 2 * k - 2; i >= k; i--)
    {
        for (int j = 1; j <= k; j++)
        {
            c[i - j] += c[i] * rec[j - 1];
        }
    }
```

```
    c.resize(k);
    return c;
}

template <class T>
vector<T> recCoef(const vector<T> &rec, unsigned long long n)
{
    // 返回 x^n 模特征多项式的 k 个系数，系数下标对应 x^0..x^(k-1)。
    int k = rec.size();
    assert(k > 0); // 调试检查，可删
    vector<T> ans(k), x(k);
    ans[0] = T(1);
    if (k == 1)
    {
        x[0] = rec[0];
    }
    else
    {
        x[1] = T(1);
    }
    while (n)
    {
        if (n & 1)
        {
            ans = recMul(ans, x, rec);
        }
        n >>= 1;
        if (n)
        {
            x = recMul(x, x, rec);
        }
    }
    return ans;
}

template <class T> T linearRec(const vector<T> &init, const vector<T> &rec, unsigned long long
↪ n)
{
    // init 是至少 k 项初值，rec 定义 a[n]=sum(rec[i-1]*a[n-i])，n 是下标；返回 a[n]。
    int k = rec.size();
    assert(k > 0 && (int)init.size() >= k); // 调试检查，可删。
    if (n < (unsigned long long)k)
    {
        return init[n];
    }
    auto ans = recCoef(rec, n);
    T res{};
    for (int i = 0; i < k; i++)
    {
        res += ans[i] * init[i];
    }
    return res;
```

```
}
```

`linearRec(init,rec,n)` 的 `rec` 方向须与 10.4.001 Berlekamp-Massey（BM / 最小线性递推）一致，`n` 从 0 起，只把 `init` 的前 k 项视为初值；`init={0,1}, rec={1,1}` 时第 $0,1,10,50$ 项为 $0,1,55,12586269025$。$O(k^2 \log n)$ 时间、$O(k)$ 空间。k 大且有快速卷积时把暴力取模换成 NTT，或直接用 10.4.005 Bostan-Mori / 分式第 N 项 降到 $O(k \log k \log n)$。

下面核对 Fibonacci 的近项、远项以及 “多给的初值不覆盖递推” 这一接口语义：

```
signed main()
{
    assert(linearRec<long long>({0, 1}, {1, 1}, 0) == 0);
    assert(linearRec<long long>({0, 1}, {1, 1}, 10) == 55);
    assert(linearRec<long long>({0, 1}, {1, 1}, 50) == 12586269025LL);
    assert(linearRec<long long>({0, 1, 999}, {1, 1}, 2) == 1);
    cout << "OK\n";
    return 0;
}
```

10.4.004 递推式转分式

把递推及初值转成 OGF $A(x) = \frac{P(x)}{Q(x)}$。分母 $Q = 1 - \sum_i r_i x^{i+1}$，把 $A(x)Q(x)$ 截到前 k 项，高次因递推恒等而抵消，剩下的低次就是分子 P，于是 $A = \frac{P}{Q}$。得到 P, Q 后即可喂给 Bostan–Mori。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
pair<Poly, Poly> recFraction(const vector<Z> &init, const vector<Z> &rec)
{
    // init 是至少 k 项初值，rec 定义 a[n]=sum(rec[i-1]*a[n-i])；返回生成函数 P(x)/Q(x)。
    int k = rec.size();
    assert(k > 0 && (int)init.size() >= k); // 调试检查，可删。
    Poly q(k + 1);
    q[0] = 1;
    for (int i = 0; i < k; i++)
    {
        q[i + 1] = -rec[i];
    }
    Poly a(init.begin(), init.begin() + k);
    Poly p = (a * q).cut(k);
    p.norm();
    return {p, q};
}
```

`recFraction(init,rec)` 返回 `{p,q}`，用 10.2.001 多项式封装 的 `Z`、`Poly`，初值至少含 k 项。$O(M(k))$ 时间、$O(k)$ 空间。直接消费者是 10.4.005 Bostan-Mori / 分式第 N 项；只需关系多项式 Q 见 10.4.002 最短递推式。

10.4.005 Bostan-Mori / 分式第 N 项

直接求有理 FPS $\frac{P(x)}{Q(x)}$ 的第 n 项。令 $Q^-(x)=Q(-x)$，则 $Q(x)Q^-(x)$ 只含偶次项；由 $\frac{P}{Q}=\frac{PQ^-}{QQ^-}$，在分子 PQ^- 中取与 n 同奇偶的系数、分母取偶次系数并做 $x^2\mapsto x$，所求下标减半而答案不变，$\log n$ 轮后 $n=0$ 直接读 $\frac{p_0}{q_0}$。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.4.004 递推式转分式
pair<Poly, Poly> bmStep(Poly q)
{
    // q 的首尾系数非零；返回 {q(-x),v(x)}，其中 q(x)q(-x)=v(x^2)。
    assert(!q.empty() && q[0] != Z(0) && q.back() != Z(0)); // 调试检查，可删。
    Poly neg = q;
    for (int i = 1; i < (int)neg.size(); i += 2)
    {
        neg[i] = -neg[i];
    }
    Poly a = conv(move(q), neg);
    Poly nq((a.size() + 1) / 2);
    for (int i = 0; i < (int)nq.size(); i++)
    {
        nq[i] = a[2 * i];
    }
    return {move(neg), move(nq)};
}

Z bostanMori(Poly p, Poly q, unsigned long long n)
{
    // p、q 是分子分母且 q[0] 非零，n 是所求下标；返回 [x^n]p(x)/q(x)。
    p.norm();
    q.norm();
    assert(!q.empty() && q[0] != Z(0)); // 调试检查，可删。
    while (n)
    {
        auto [neg, nq] = bmStep(move(q));
        Poly a = conv(move(p), move(neg));
        Poly np;
        np.reserve((a.size() + 1) / 2);
        for (int i = n & 1; i < (int)a.size(); i += 2)
        {
            np.push_back(a[i]);
        }
        p = move(np);
        q = move(nq);
        n >>= 1;
    }
    return p.empty() ? Z(0) : p[0] / q[0];
}
```

`bostanMori(p,q,n)` 要求 `q[0]≠0`；$p=x$、$q=1-x-x^2$ 的第 10 项为 55（即 Fibonacci）。令

$k=\max(|p|,|q|)$，则为 $O(M(k)\log n)$ 时间、$O(k)$ 空间。输入若来自递推，先用 10.4.004 递推式转分式 构造 P,Q；阶数小时 10.4.003 Linear Recurrence 常数更小。

下面把 Fibonacci 递推转成分式，直接检查分子、分母和几个已知项：

```
signed main()
{
    auto [p, q] = recFraction({0, 1}, {1, 1});
    assert(p == Poly({0, 1}) && q == Poly({1, -1, -1}));
    assert(bostanMori(p, q, 0) == 0);
    assert(bostanMori(p, q, 10) == 55);
    assert(bostanMori(p, q, 50) == Z(12586269025LL));
    return 0;
}
```

10.4.006 有理生成函数远处连续区间

单次 `bostanMori` 适合一个远项；连续取同一有理 FPS 的远处区间时，逐项查询会重复做同样的下标折半。这里先把 $Q(0)$ 归一化为 1，并做快速长除 $P/Q=A+R/Q$。有限多项式 A 直接填入答案；严格真分式部分先由 `invTail(q,k)` 一次求出

$$[x^k]\frac{1}{Q},\dots,[x^{k+d-1}]\frac{1}{Q},\qquad d=\deg Q.$$

`invTail` 每轮使用 `bmStep` 的 $Q(x)Q(-x)=Q_1(x^2)$，把区间起点按奇偶映到约一半的位置；降到 $k\le d$ 后用一次 FPS 逆取得局部块，再沿保存的 $Q(-x)$ 逐层回推。令这段逆元组成 T，截取 $U=(QT)\bmod x^d$，则远处移位在商环中的状态可由 $S\equiv RU\pmod Q$ 表示，最后展开 $S/Q \bmod x^m$ 就得到严格真分式从第 k 项起的 m 项。整个过程复用公共 `bmStep`、`conv`、`fpsInvFast` 和 `polyDivmod`，没有另一套递推乘法或私有 NTT。

```
// 依赖 1.1.008 静态取模类
// 依赖 10.2.001 多项式封装
// 依赖 10.2.002 乘法、除法、取模
// 依赖 10.2.003 乘法逆元
// 依赖 10.4.005 Bostan-Mori / 分式第 N 项
Poly invTail(Poly q, unsigned long long k)
{
    // q 的次数 d>0 且首尾系数非零；返回 [x^k..x^(k+d-1)]1/q(x)。
    q.norm();
    assert(q.size() > 1 && q[0] != Z(0) && q.back() != Z(0)); // 调试检查，可删。
    int d = q.size() - 1;
    assert(d <= NTT_LIM); // 调试检查，可删。
    if (k <= (unsigned long long)d)
    {
        assert(k <= (unsigned long long)(NTT_LIM - d)); // 调试检查，可删。
    }
    else
    {
        assert(convFit(d + 1, 2 * d - 1)); // 调试检查，可删。
```

```
    }
    vector<pair<Poly, int>> st;
    st.reserve(64);
    while (k > (unsigned long long)d)
    {
        auto [neg, nq] = bmStep(move(q));
        int t = (k - (unsigned long long)d) & 1ULL;
        st.push_back({move(neg), t});
        q = move(nq);
        k = (k - (unsigned long long)d + t) / 2;
    }

    int s = k;
    Poly a = fpsInvFast(q, s + d);
    a.erase(a.begin(), a.begin() + s);
    while (!st.empty())
    {
        Poly neg = move(st.back().first);
        int t = st.back().second;
        st.pop_back();
        Poly b(2 * d - 1);
        for (int i = 0; i < d; i++)
        {
            b[2 * i] = a[i];
        }
        Poly f = conv(move(neg), move(b));
        int l = d - t;
        a = Poly(f.begin() + l, f.begin() + l + d);
    }
    return a;
}

Poly ratBlock(Poly p, Poly q, unsigned long long k, int m)
{
    // p/q 是有理 FPS，q[0] 非零；返回从第 k 项起的 m 项。
    // 设 L=log(1+k/d)，时间 O(M(|p|)+M(d)L+M(d+m))，空间 O(|p|+dL+d+m)。
    // 所有实际卷积长度须不超过公共 NTT 的 2^23 上限。
    assert(m >= 0); // 调试检查，可删。
    q.norm();
    assert(!q.empty() && q[0] != Z(0)); // 调试检查，可删。
    if (!m)
    {
        return {};
    }
    assert(k <= numeric_limits<unsigned long long>::max() -
                    (unsigned long long)(m - 1)); // 调试检查，可删。
    p.norm();
    Z iv = q[0].inv();
    p *= iv;
    q *= iv;

    if (q.size() == 1)
```

```
    {
        Poly ans(m);
        if (k < (unsigned long long)p.size())
        {
            int n = min<unsigned long long>(m, (unsigned long long)p.size() - k);
            copy_n(p.begin() + (size_t)k, n, ans.begin());
        }
        return ans;
    }

    auto [a, r] = polyDivmod(move(p), q);
    Poly ans(m);
    if (k < (unsigned long long)a.size())
    {
        int n = min<unsigned long long>(m, (unsigned long long)a.size() - k);
        for (int i = 0; i < n; i++)
        {
            ans[i] = a[(size_t)k + i];
        }
    }

    int d = q.size() - 1;
    if (r.empty())
    {
        return ans;
    }
    assert(m <= NTT_LIM && convFit(d + 1, d)); // 调试检查，可删。
    Poly t = invTail(q, k);
    Poly u = conv(q, move(t));
    u.resize(d);
    Poly v = conv(move(u), move(r));
    Poly s = polyDivmod(move(v), q).second;
    assert(convFit(s.size(), m)); // 调试检查，可删。
    Poly iq = fpsInvFast(q, m);
    Poly b = conv(move(s), move(iq));
    b.resize(m);
    ans += b;
    return ans;
}
```

`ratBlock(p,q,k,m)` 返回 $[x^k]\frac{p(x)}{q(x)},\dots,[x^{k+m-1}]\frac{p(x)}{q(x)}$；`q[0]` 必须非零，`m=0` 返回空，且 $k+m-1$ 不能发生无符号回绕。常数分母在归一化后直接从有限多项式截取，时间为 $O(|p|+m)$；非常数分母先长除，使严格真分式和有限多项式部分同时成立，例如 $p=1+x^3, q=1-x$ 的前六项为 $1,1,1,2,2,2$。设 $N=\max(|p|,d+1)$、$L=1+\log(1+k/d)$，一般时间为 $O(M(N)+M(d)L+M(d+m))$，额外空间为 $O(N+dL+d+m)$。所有实际卷积的 NTT 长度必须不超过 2^{23}。只求一个远项时 10.4.005 Bostan-Mori / 分式第 N 项 更短，连续区间才使用本条。

下面用 Fibonacci、重根分母、非严格真分式、常数分母和空区间作固定核对；模数类 `Z` 沿用本章公共前导：

```cpp
signed main()
{
    assert((ratBlock({0, 1}, {1, -1, -1}, 10, 4) ==
            vector<Z>{55, 89, 144, 233}));
    assert((ratBlock({1}, {1, -2, 1}, 100, 3) ==
            vector<Z>{101, 102, 103}));
    assert((ratBlock({1, 0, 0, 1}, {1, -1}, 0, 6) ==
            vector<Z>{1, 1, 1, 2, 2, 2}));
    assert((ratBlock({2, 4, 6}, {2}, 1, 3) ==
            vector<Z>{2, 3, 0}));
    assert(ratBlock({1}, {1, -1}, 0, 0).empty());
    cout << "OK\n";
    return 0;
}
```

11. 计算几何

11.1 二维基础对象

11.1.001 浮点误差与精确整数几何

统一整数几何与浮点几何的判号规则。浮点类型统一记为 `Real`，实际使用 `long double`；整数坐标的叉积应优先保持精确，只有交点、距离、角度和圆等需要除法或开方的计算才转为 `Real`。`sgn` 对整数精确判号，对 `Real` 使用相对误差，因而只依赖符号的算法可以直接写成 `Point<T>` 共用。坐标乘积可能超过 `long long` 时，由调用者在专用算法中显式转为 `__int128`。

```cpp
using Real = long double;
constexpr Real eps = 1E-12L;

template <class T> int sgn(T x)
{
    // x 是整数；返回 -1、0、1 表示负、零、正。
    return (x > 0) - (x < 0);
}

int sgn(Real x)
{
    // x 表示待判断的浮点数；返回 -1、0、1 表示负、近似零、正。
    Real t = eps * max<Real>(1, fabsl(x));
    return (x > t) - (x < -t);
}

template <class T> bool eq(T x, T y)
{
    // x、y 是整数；返回二者是否相等。
    return x == y;
}

bool eq(Real x, Real y)
{
    // x、y 表示两个浮点数；使用相对误差判断它们是否近似相等。
    return fabsl(x - y) <= eps * max<Real>(1, max(fabsl(x), fabsl(y)));
}
```

整数与浮点计算都调用 `sgn(x)` 或 `eq(x,y)`；传入整数时不会产生浮点转换。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.002 Point / Vector

表示二维点和二维向量，提供坐标读写、比较、加减与数乘。Point 同时承担点和向量语义，是本章其余二维模板的共同参数类型。点和向量都存坐标，但语义不同：点减点得到向量，点加向量得到点；比较、哈

希和浮点相等必须采用与坐标类型一致的规则。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
template <class T> struct Point
{
    T x; // x 保存横坐标。
    T y; // y 保存纵坐标。

    Point(const T &x = 0, const T &y = 0) : x(x), y(y)
    {
        // x、y 表示初始坐标；构造点或向量。
    }

    template <class U> explicit operator Point<U>() const
    {
        // U 表示目标坐标类型；返回逐坐标 C 风格转换后的点。
        return Point<U>((U)x, (U)y);
    }

    Point &operator+=(const Point &p)
    {
        // p 表示另一个向量；把 p 加到当前向量并返回自身。
        x += p.x;
        y += p.y;
        return *this;
    }

    Point &operator-=(const Point &p)
    {
        // p 表示另一个向量；从当前向量减去 p 并返回自身。
        x -= p.x;
        y -= p.y;
        return *this;
    }

    Point &operator*=(const T &k)
    {
        // k 表示缩放倍数；原地数乘并返回自身。
        x *= k;
        y *= k;
        return *this;
    }

    Point &operator/=(const T &k)
    {
        // k 表示非零除数；原地除法并返回自身。
        assert(k != T(0));
        x /= k;
        y /= k;
        return *this;
    }

```

```
51      Point operator-() const
52      {
53          // 无参数；返回反向向量。
54          return {-x, -y};
55      }
56
57      friend Point operator+(Point a, const Point &b)
58      {
59          // a、b 表示两个向量；返回逐坐标之和。
60          return a += b;
61      }
62
63      friend Point operator-(Point a, const Point &b)
64      {
65          // a、b 表示两个点或向量；返回 a-b。
66          return a -= b;
67      }
68
69      friend Point operator*(Point a, const T &k)
70      {
71          // a 表示向量，k 表示倍数；返回缩放后的向量。
72          return a *= k;
73      }
74
75      friend Point operator*(const T &k, Point a)
76      {
77          // k 表示倍数，a 表示向量；返回缩放后的向量。
78          return a *= k;
79      }
80
81      friend Point operator/(Point a, const T &k)
82      {
83          // a 表示向量，k 表示非零除数；返回缩放后的向量。
84          return a /= k;
85      }
86
87      friend bool operator<(const Point &a, const Point &b)
88      {
89          // a、b 表示两个点；返回按 x、y 的字典序比较结果。
90          return a.x != b.x ? a.x < b.x : a.y < b.y;
91      }
92
93      friend bool operator==(const Point &a, const Point &b)
94      {
95          // a、b 表示两个点；返回坐标是否精确相等。
96          return a.x == b.x && a.y == b.y;
97      }
98  };
99
100 using P = Point<Real>;
101
102 template <class T> bool eq(Point<T> a, Point<T> b)
```

```
{
    // a、b 表示两个点；整数坐标精确比较，浮点坐标按统一误差比较。
    return eq(a.x, b.x) && eq(a.y, b.y);
}
```

使用 `Point<int>` 保存整数点，使用 `P` 即 `Point<Real>` 保存需要除法或开方的浮点点；排序使用的比较运算始终精确比较坐标，误差相等判断显式调用 `eq(a,b)`。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.003 点积、叉积、长度、夹角、旋转

计算点积、叉积、模长、夹角和旋转。点积描述投影与夹角，叉积描述有向面积和左右关系，两者是直线、多边形与圆算法的基础。点积刻画投影和夹角，叉积符号刻画转向、平行四边形有向面积；旋转使用正交矩阵，有向夹角把叉积和点积交给 `atan2`，无需先求模长或限制输入范围。二维 `norm` 返回的是模长的平方（与三维 11.6.001 的 `norm` 同义），外接圆公式、投影参数、旋转卡壳的直径平方都按平方长度取用；原稿此处误写成 `return abs(a);`（返回模长）会破坏这些调用，已修正为 `return square(a);`。

```
// 依赖 11.1.002 Point / Vector
template <class T> T dot(const Point<T> &a, const Point<T> &b)
{
    // a、b 表示两个向量；返回点积。
    return a.x * b.x + a.y * b.y;
}

template <class T> T cross(const Point<T> &a, const Point<T> &b)
{
    // a、b 表示两个向量；返回有向叉积。
    return a.x * b.y - a.y * b.x;
}

template <class T> T cross(const Point<T> &a, const Point<T> &b, const Point<T> &c)
{
    // a、b、c 表示三个点；返回三角形 abc 的有向面积两倍。
    return cross(b - a, c - a);
}

template <class T> T square(const Point<T> &a)
{
    // a 表示向量；返回模长平方。
    return dot(a, a);
}

template <class T> Real abs(const Point<T> &a)
{
    // a 表示向量；返回其欧氏模长。
    return sqrtl((Real)square(a));
}
```

```cpp

template <class T> T norm(const Point<T> &a)
{
    // a 表示向量；返回模长平方。
    return square(a);
}

template <class T> Real angle(const Point<T> &a, const Point<T> &b)
{
    // a、b 是非零向量；返回 a 到 b 的有向夹角，范围为 (-pi,pi]。
    assert(a.x != T{} || a.y != T{}); // 调试检查，可删。
    assert(b.x != T{} || b.y != T{}); // 调试检查，可删。
    return atan2l(cross(a, b), dot(a, b));
}

template <class T> Point<Real> rotate(const Point<T> &a, Real t)
{
    // a 表示向量，t 表示逆时针旋转弧度；返回旋转后的浮点向量。
    return {a.x * cosl(t) - a.y * sinl(t), a.x * sinl(t) + a.y * cosl(t)};
}

template <class T> Point<T> rot(const Point<T> &a)
{
    // a 表示向量；返回逆时针旋转 90 度后的向量。
    return {-a.y, a.x};
}
```

`dot(a,b)` 与 `cross(a,b)` 分别返回点积和二维叉积标量，`cross(a,b,c)` 判断 $a \to b$ 到 $a \to c$ 的转向。`angle(a,b)` 用 `atan2l(cross(a,b),dot(a,b))` 返回从 a 到 b 的有向夹角，范围为 $(-\pi,\pi]$；普通无向夹角取 `fabsl(angle(a,b))`。这些函数直接返回坐标类型 `T`，调用者必须保证乘积与加减结果能由 `T` 表示；本书的 `Point<int>` 因 `#define int long long` 使用 64 位坐标。只有确实可能超过 64 位的专用算法才在调用处显式转为 `__int128`，普通几何调用不承担隐藏的宽类型开销。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用本条目精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.004 极角排序与象限分类

把向量按极角从小到大排序，并在同一射线上按距离排序。适用于扫描射线、按方向合并边、半平面预处理和角度统计。先按上半平面或象限分类，再用叉积比较同半平面向量方向；共线时用长度打破平局。排序比较器必须保持严格弱序，因此这里对整数和 `Real` 都使用坐标与叉积的原值比较，不在比较器中混入 `eps`。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> int half(Point<T> p)
{
    // p 是向量；返回其所在的极角半平面编号。
    return p.y < 0 || (p.y == 0 && p.x < 0);
```

```cpp
}

// 以原点为极点，按 [-pi, pi) 逆时针排序；同射线按距离升序。
template <class T> bool polarCmp(Point<T> a, Point<T> b)
{
    // a、b 是相对同一原点的向量；返回 a 是否应排在 b 前面。
    int x = half(a);
    int y = half(b);
    if (x != y)
    {
        return x < y;
    }
    auto c = cross(a, b);
    return c ? c > 0 : norm(a) < norm(b);
}

// 绕极点 o 排序。
template <class T> void polarSort(vector<Point<T>> &a, Point<T> o = {0, 0})
{
    // 参数 a 表示 要原地按极角排序的点或向量序列；o 表示 极角排序使用的原点；以 o 为原点原地按极角排序
    ↪ 点集，无返回值。
    sort(a.begin(),
         a.end(),
         [&](Point<T> x, Point<T> y)
         {
             return polarCmp(x - o, y - o);
         });
}
```

调用 `polarSort(a,o)` 按相对原点 `o` 的极角排序；零向量以及同射线点的先后规则见 `polarCmp`。整数坐标最稳；浮点坐标应来自稳定输入，不能把计算误差很大的近似方向直接交给排序器。排序 $O(n \log n)$ 时间、$O(\log n)$ 比较栈空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.005 Line / Ray / Segment

表示直线、射线与线段。Line 保存两个端点；浮点算法内部需要点加方向向量时使用 DirLine，并明确两种表示的转换。直线由一点和非零方向向量表示，射线额外限制参数非负，线段限制参数在 `[0,1]`；同一几何对象的边界条件不能混用。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> struct Line
{
    Point<T> a; // a 保存直线、射线或线段的起点。
    Point<T> b; // b 保存直线上的第二点；作为射线时方向为 b-a。

    Line(Point<T> a = {}, Point<T> b = {}) : a(a), b(b)
    {
```

```
        // a、b 表示两个端点；构造端点式直线。
    }
};

struct DirLine
{
    P p; // p 保存直线上一点。
    P v; // v 保存非零方向向量。

    P at(Real t) const
    {
        // t 表示方向参数；返回 p+v*t。
        return p + v * t;
    }
};

using L = DirLine;

template <class T> DirLine toDir(const Line<T> &l)
{
    // l 表示端点式直线；返回供浮点算法使用的点方向式直线。
    auto v = l.b - l.a;
    assert(l.a != l.b); // 调试检查，可删。
    return {(P)l.a, (P)v};
}
```

通常构造 Line{a,b} 表示端点式直线或线段；只有调用旧的浮点圆与半平面接口时用 toDir(l) 转为 DirLine。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.006 平行、垂直、共线、同侧/异侧

判断两条方向的平行、垂直、共线以及点在线的哪一侧。整数输入时这些关系可只用点积和叉积精确完成。平行由方向叉积为零判断，垂直由点积为零判断，共线还需点到直线方向的叉积为零；同侧异侧看两个有向叉积的符号乘积。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
template <class T> bool parallel(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条端点式直线；返回方向是否平行。
    return sgn(cross(a.b - a.a, b.b - b.a)) == 0;
}

template <class T> bool perp(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条端点式直线；返回方向是否垂直。
```

```
    return sgn(dot(a.b - a.a, b.b - b.a)) == 0;
}

template <class T> int side(const Line<T> &l, const Point<T> &p)
{
    // l 表示有向直线，p 表示查询点；返回叉积符号，正数在左侧。
    return sgn(cross(l.b - l.a, p - l.a));
}

bool parallel(L a, L b)
{
    // a、b 表示两条点方向式直线；返回方向是否平行。
    return sgn(cross(a.v, b.v)) == 0;
}
```

parallel(a,b)、perp(a,b) 判断方向关系；side(l,p) 返回正、零、负分别表示左侧、线上、右侧。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.007 直线交点、投影、对称点

求非平行直线交点、点在线上的投影和关于直线的对称点。退化直线或平行直线必须由调用者提前排除。非平行直线交点参数由两个叉积之比得到；投影参数是点差在方向上的点积除以方向长度平方，对称点是投影点的两倍减原点。

```
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> Point<Real> lineMeet(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条不平行直线；返回唯一交点。
    auto u = (P)a.a;
    auto v = (P)(a.b - a.a);
    auto p = (P)b.a;
    auto q = (P)(b.b - b.a);
    assert(sgn(cross(v, q)) != 0); // 调试检查，可删。
    return u + v * (cross(p - u, q) / cross(v, q));
}

template <class T> Point<Real> projection(const Line<T> &l, const Point<T> &p)
{
    // l 表示非退化直线，p 表示查询点；返回 p 在直线上的正交投影。
    auto a = (P)l.a;
    auto v = (P)(l.b - l.a);
    auto q = (P)p;
    assert(sgn(square(v)) != 0); // 调试检查，可删。
    return a + v * (dot(q - a, v) / square(v));
}

```

```cpp
template <class T> Point<Real> reflection(const Line<T> &l, const Point<T> &p)
{
    // l 表示非退化直线，p 表示查询点；返回 p 关于直线的对称点。
    return projection(l, p) * 2 - (P)p;
}

P isect(L a, L b)
{
    // a、b 表示两条不平行的点方向式直线；返回唯一交点。
    assert(sgn(cross(a.v, b.v)) != 0); // 调试检查，可删。
    return a.at(cross(b.p - a.p, b.v) / cross(a.v, b.v));
}

P proj(L l, P p)
{
    // l 表示非退化点方向式直线，p 表示查询点；返回正交投影。
    assert(sgn(square(l.v)) != 0); // 调试检查，可删。
    return l.at(dot(p - l.p, l.v) / square(l.v));
}

P refl(L l, P p)
{
    // l 表示非退化点方向式直线，p 表示查询点；返回关于直线的对称点。
    return proj(l, p) * 2 - p;
}

Real dis(L l, P p)
{
    // l 表示非退化点方向式直线，p 表示查询点；返回点到直线距离。
    assert(sgn(square(l.v)) != 0); // 调试检查，可删。
    return fabsl(cross(l.v, p - l.p)) / abs(l.v);
}
```

`lineMeet(a,b)` 要求方向叉积非零；`projection(l,p)` 与 `reflection(l,p)` 要求 `l.a≠l.b`。这些接口接受整数或浮点端点，但除法后的结果统一返回 `Point<Real>`。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.008 点到直线/线段距离

计算点到直线、射线或线段的最短距离。线段距离需要先判断投影是否落在端点之间，不能直接套点到直线公式。点到直线距离是叉积绝对值除以方向长度；点到线段先看投影参数，落在线段外时取到相应端点距离。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.005 Line / Ray / Segment
template <class T> Real distancePL(const Point<T> &p, const Line<T> &l)
```

```
{
    // p 表示点，l 表示非退化直线；返回点到无限直线的距离。
    assert(l.a != l.b); // 调试检查，可删。
    return fabsl((Real)cross(l.b - l.a, p - l.a)) / abs(l.b - l.a);
}

template <class T> Real distancePS(const Point<T> &p, const Line<T> &l)
{
    // p 表示点，l 表示线段；返回点到闭线段的最短距离。
    if (l.a == l.b)
    {
        return abs(p - l.a);
    }
    if (dot(p - l.a, l.b - l.a) <= 0)
    {
        return abs(p - l.a);
    }
    if (dot(p - l.b, l.a - l.b) <= 0)
    {
        return abs(p - l.b);
    }
    return distancePL(p, l);
}
```

distancePL(p,l) 把 l 当无限直线，distancePS(p,l) 把 l 当闭线段；退化线段应直接取点距。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.009 线段相交与距离

判断两条闭线段是否相交，并计算两线段最短距离。实现同时覆盖端点接触与共线重叠。线段相交先用包围盒排斥，再比较两端相对另一线段的叉积符号；不相交时两线段距离是四个端点到对方线段距离的最小值。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.005 Line / Ray / Segment
template <class T> bool onSegment(const Point<T> &p, const Line<T> &l)
{
    // p 表示查询点，l 表示闭线段；返回 p 是否在线段上。
    return sgn(cross(p - l.a, l.b - l.a)) == 0 && sgn(dot(p - l.a, p - l.b)) <= 0;
}

template <class T> bool segMeet(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条闭线段；返回它们是否至少有一个公共点。
    int c1 = sgn(cross(a.b - a.a, b.a - a.a));
    int c2 = sgn(cross(a.b - a.a, b.b - a.a));
    int c3 = sgn(cross(b.b - b.a, a.a - b.a));
```

```
    int c4 = sgn(cross(b.b - b.a, a.b - b.a));
    if (c1 == 0 && onSegment(b.a, a))
    {
        return true;
    }
    if (c2 == 0 && onSegment(b.b, a))
    {
        return true;
    }
    if (c3 == 0 && onSegment(a.a, b))
    {
        return true;
    }
    if (c4 == 0 && onSegment(a.b, b))
    {
        return true;
    }
    return ((c1 > 0) != (c2 > 0)) && ((c3 > 0) != (c4 > 0));
}

template <class T> Real distanceSS(const Line<T> &a, const Line<T> &b)
{
    // a、b 表示两条闭线段；返回线段间最短距离。
    if (segMeet(a, b))
    {
        return 0;
    }
    return min({distancePS(a.a, b), distancePS(a.b, b), distancePS(b.a, a), distancePS(b.b,
    ↪ a)});
}

template <class T> bool onSeg(Point<T> p, Point<T> a, Point<T> b)
{
    // p 表示查询点，a、b 表示线段端点；返回 p 是否在线段上。
    return onSegment(p, Line<T>{a, b});
}

template <class T> bool segIsect(Point<T> a, Point<T> b, Point<T> c, Point<T> d)
{
    // a、b 与 c、d 分别表示两条闭线段；返回是否相交。
    return segMeet(Line<T>{a, b}, Line<T>{c, d});
}

template <class T> Real segDis(Point<T> a, Point<T> b, Point<T> c, Point<T> d)
{
    // a、b 与 c、d 分别表示两条闭线段；返回最短距离。
    return distanceSS(Line<T>{a, b}, Line<T>{c, d});
}
```

segMeet(a,b) 含边界相交，distanceSS(a,b) 返回闭线段间距离；整数坐标可精确判断相交。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。

线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.010 线段中垂线

由两个不同点构造线段中垂线，供外心、等距轨迹和 Voronoi 边界使用。中点是直线上一点，原线段方向旋转 90 度得到中垂线方向；两端点重合时方向不确定。

```
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
optional<Line<Real>> midPerp(P a, P b)
{
    // a、b 表示两个点；重合时返回空，否则返回端点式中垂线。
    if (eq(a, b))
    {
        return nullopt;
    }
    auto m = (a + b) / 2;
    return Line<Real>{m, m + rot(b - a)};
}
```

调用 `midPerp(a,b)`；两点重合返回 `nullopt`，否则返回 `Line<Real>`。中点可能出现二分之一，因此本接口固定使用浮点点。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.011 四点组成矩形 / 正方形判定

判断四个无序点是否组成非退化矩形或正方形。通过六条两两距离避免依赖输入环形顺序。枚举六个两两距离平方并排序。矩形应有两组相等边和两条相等对角线且满足勾股关系；正方形进一步要求四条边相等。

```
// 依赖 11.1.002 Point / Vector
template <class T> int rectKind(const array<Point<T>, 4> &p)
{
    // p[0..3] 表示四个无序点；返回 0 非矩形、1 矩形、2 正方形。
    vector<T> d;
    for (int i = 0; i < 4; i++)
    {
        for (int j = i + 1; j < 4; j++)
        {
            d.push_back(square(p[i] - p[j]));
        }
    }
    sort(d.begin(), d.end());
    if (sgn(d[0]) == 0 || !eq(d[0], d[1]) || !eq(d[2], d[3]) || !eq(d[4], d[5]) || !eq(d[0] +
    ↪ d[2], d[4]))
    {
```

```
        return 0;
    }
    return eq(d[0], d[2]) ? 2 : 1;
}
```

调用 `rectKind(p)`，`p[0..3]` 是四个无序点；整数坐标精确判断，`Point<Real>` 按统一误差判断。返回 0、1、2 分别表示否、矩形、正方形。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.1.012 三角形内心、外心、垂心与旁心

计算非退化三角形的内心、外心、垂心和三个旁心，供圆、距离和三角形构造题直接使用。内心与旁心使用边长加权；外心由两条中垂线求交；垂心可由向量恒等式 $H = A + B + C - 2O$ 得到。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
array<P, 6> triCenters(P a, P b, P c)
{
    // a、b、c 表示逆时针或顺时针非共线三角形；依次返回内心、外心、垂心和三个旁心。
    auto x = abs(b - c);
    auto y = abs(c - a);
    auto z = abs(a - b);
    auto in = (a * x + b * y + c * z) / (x + y + z);
    auto d = 2 * cross(b - a, c - a);
    assert(sgn(d) != 0); // 调试检查，可删。
    auto bb = square(b - a);
    auto cc = square(c - a);
    P out{a.x + ((c.y - a.y) * bb - (b.y - a.y) * cc) / d, a.y + ((b.x - a.x) * cc - (c.x -
    ↪ a.x) * bb) / d};
    auto orth = a + b + c - out * 2;
    auto exa = (-a * x + b * y + c * z) / (-x + y + z);
    auto exb = (a * x - b * y + c * z) / (x - y + z);
    auto exc = (a * x + b * y - c * z) / (x + y - z);
    return {in, out, orth, exa, exb, exc};
}
```

调用 triCenters(a,b,c)，返回数组依次为内心、外心、垂心、a 对旁心、b 对旁心、c 对旁心；三点不得共线。$O(1)$ 时间和 $O(1)$ 额外空间。二维对象统一使用 11.1.002 Point / Vector 与 11.1.005 Line / Ray / Segment。整数方向关系优先用 11.1.003 点积、叉积、长度、夹角、旋转 精确判断；只有交点、距离和圆计算才转浮点。线段相交使用 11.1.009 线段相交与距离，不能用无限直线交点代替。

11.2 多边形

11.2.001 多边形面积、周长、重心

计算简单多边形的有向面积、周长和重心。有向面积还能判断顶点方向，重心公式要求面积非零。鞋带公式的叉积和是有向面积的两倍；整数点先用 `area2` 保留精确整数，需要通常意义的面积、周长或重心时再由接口显式返回 `Real`。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> T area2(const vector<Point<T>> &a)
{
    // a 是环形点列；返回多边形有向面积的两倍。
    T ans = 0;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        ans += cross(a[i], a[(i + 1) % n]);
    }
    return ans;
}

template <class T> Real area(const vector<Point<T>> &a)
{
    // a 是环形点列；返回多边形有向面积。
    return (Real)area2(a) / 2;
}

template <class T> Real perimeter(const vector<Point<T>> &a)
{
    // a 是环形点列；返回多边形周长。
    Real ans = 0;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        ans += abs(a[(i + 1) % n] - a[i]);
    }
    return ans;
}

// 非退化简单多边形的重心；顺逆时针均可。
template <class T> Point<Real> cen(const vector<Point<T>> &a)
{
    // a 是非退化简单多边形；返回面积形心。
    P ans{};
    Real s = 0;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        Real c = (Real)cross(a[i], a[(i + 1) % n]);
        ans += ((P)a[i] + (P)a[(i + 1) % n]) * c;
        s += c;
```

```
    }
    assert(sgn(s) != 0); // 调试检查，可删。
    return ans / (3 * s);
}
```

`area2(poly)` 返回与坐标同类型的有向面积两倍值，整数坐标时没有除法误差；`area(poly)`、`perimeter(poly)` 和 `cen(poly)` 分别返回 `Real` 面积、周长和重心。调用者须保证叉积和能由 `T` 表示。各接口均为 $O(n)$ 时间、$O(1)$ 额外空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.002 点在多边形内与有符号绕数

同时提供简单多边形点分类和一般闭合轮廓的有符号绕数。`inPoly` 对简单多边形用向右射线的奇偶交点数返回外部、边界、内部；`windingNum` 对可自交、可重复绕行的闭合点列，按向上穿越加一、向下穿越减一。两者都先判边界并采用纵坐标半开规则，因此零长度边、重复顶点和射线穿过顶点不会重复计数；绕数正负随轮廓方向反转。

```
// 依赖 11.1.002 Point / Vector
template <class T> int inPoly(const vector<Point<T>> &a, Point<T> p)
{
    // a 是简单多边形，p 是查询点；返回 0、1、2 表示外部、边界、内部。
    bool in = false;
    int n = (int)a.size();
    for (int i = 0; i < n; i++)
    {
        Point<T> x = a[i];
        Point<T> y = a[(i + 1) % n];
        if (onSeg(p, x, y))
        {
            return 1;
        }
        if (x.y > y.y)
        {
            swap(x, y);
        }
        if (sgn(x.y - p.y) <= 0 && sgn(p.y - y.y) < 0 && sgn(cross(x, y, p)) > 0)
        {
            in = !in;
        }
    }
    return in ? 2 : 0;
}

template <class T> pair<bool, int> windingNum(const vector<Point<T>> &a, Point<T> p)
{
    // a 是按顺序连接并首尾闭合的轮廓，p 是查询点；返回是否在边界上及有符号绕数，边界上时绕数置零。
    int ans = 0;
    int n = (int)a.size();
```

```
    for (int i = 0; i < n; i++)
    {
        Point<T> x = a[i];
        Point<T> y = a[(i + 1) % n];
        if (onSeg(p, x, y))
        {
            return {true, 0};
        }
        int sx = sgn(x.y - p.y);
        int sy = sgn(y.y - p.y);
        int c = sgn(cross(x, y, p));
        if (sx <= 0 && sy > 0 && c > 0)
        {
            ans++;
        }
        if (sy <= 0 && sx > 0 && c < 0)
        {
            ans--;
        }
    }
    return {false, ans};
}
```

`inPoly(a,p)` 返回 $0,1,2$ 表示外部、边界、内部，只用于按边界顺序给出的简单多边形；`windingNum(a,p)` 返回的 `first` 表示点是否在边界上，`second` 是有符号绕数，边界上固定为 0。点列会自动首尾相连，空点列绕数为 0；一次查询为 $O(n)$ 时间、$O(1)$ 额外空间。整条线段包含见 11.2.003 线段是否在多边形内，多个轮廓的覆盖层面积见 11.2.011 多个简单多边形的覆盖层面积。

11.2.003 线段是否在多边形内

判断整条闭线段是否位于简单多边形内部或边界上，特别处理穿过凹顶点和沿边界重合的情况。端点在多边形内还不充分：凹多边形中线段可能穿出再穿入。实现枚举线段与每条边的交，并在经过顶点时结合相邻边方向判断是否越过外部。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.009 线段相交与距离
// 依赖 11.2.002 点在多边形内与有符号绕数
bool segInPoly(const vector<P> &a, P s, P t)
{
    // a 是简单多边形，s、t 是线段端点；返回闭线段是否完全位于内部或边界。
    if (!inPoly(a, s) || !inPoly(a, t))
    {
        return false;
    }
    vector<Real> q{0, 1};
    P v = t - s;
    Real z = norm(v);
    if (!sgn(z))
    {
```

```
        return true;
    }
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        P x = a[i];
        P y = a[(i + 1) % n];
        P w = y - x;
        Real d = cross(v, w);
        if (sgn(d))
        {
            Real u = cross(x - s, w) / d;
            Real k = cross(x - s, v) / d;
            if (sgn(u) >= 0 && sgn(u - 1) <= 0 && sgn(k) >= 0 && sgn(k - 1) <= 0)
            {
                q.push_back(u);
            }
        }
        else if (!sgn(cross(v, x - s)))
        {
            q.push_back(clamp(dot(x - s, v) / z, Real(0), Real(1)));
            q.push_back(clamp(dot(y - s, v) / z, Real(0), Real(1)));
        }
    }
    sort(q.begin(), q.end());
    q.erase(unique(q.begin(),
                   q.end(),
                   [](Real x, Real y)
                   {
                       return !sgn(x - y);
                   }),
            q.end());
    for (int i = 0; i + 1 < (int)q.size(); i++)
    {
        P p = s + v * ((q[i] + q[i + 1]) / 2);
        if (!inPoly(a, p))
        {
            return false;
        }
    }
    return true;
}
```

调用 `segInPoly(poly,s,t)`。端点在内部并不足够，函数还会按所有交点分段检查中点；退化为点时按点包含处理。$O(n\log n)$ 时间、$O(n)$ 空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。可据此手算自查：L 形多边形 {(0,0),(4,0),(4,1),(1,1),(1,4),(0,4)}：线段 (0,0)-(4,1) 在内部或边界，线段 (0,4)-(4,0) 穿过缺口外部；期望前者为真、后者为假。

11.2.004 多边形切割

用一条有向直线裁剪多边形，只保留左半平面。连续应用即可完成凸多边形与若干半平面的交。逐边判断顶点在切割有向直线哪侧，保留目标侧顶点，并在跨侧边上插入交点；这是 Sutherland-Hodgman 的单半平面裁剪步骤。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.007 直线交点、投影、对称点
vector<P> cutPoly(const vector<P> &a, L l)
{
    // a 是环形点列，l 是有向边界；返回其左闭半平面内的裁剪结果。
    vector<P> b;
    int n = a.size();
    for (int i = 0; i < n; i++)
    {
        P x = a[i];
        P y = a[(i + 1) % n];
        int sx = sgn(cross(l.v, x - l.p));
        int sy = sgn(cross(l.v, y - l.p));
        if (sx >= 0)
        {
            b.push_back(x);
        }
        if (sx * sy < 0)
        {
            b.push_back(isect({x, y - x}, l));
        }
    }
    return b;
}
```

调用 `cutPoly(poly,l)`，`l` 是点方向式有向直线，返回其左侧与边界上的多边形；输入应按环形顺序给出。$O(n)$ 时间、$O(n)$ 结果空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.005 多边形的核

求简单多边形的核，即能看见多边形全部位置的点集。逆时针多边形的核是所有边左半平面的交。简单多边形的核是其每条有向边内侧半平面的交；按顶点方向确定内侧后连续做半平面裁剪，最终为空即不存在可见全多边形的点。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.2.001 多边形面积、周长、重心
```

```cpp
// 依赖 11.2.004 多边形切割
vector<P> polyKernel(const vector<P> &a)
{
    // a 是简单多边形；返回核多边形，核为空时返回空序列。
    if (a.size() < 3)
    {
        return {};
    }
    vector<P> b = a;
    Real s = area(a);
    assert(sgn(s) != 0); // 调试检查，可删。
    int n = a.size();
    for (int i = 0; i < n && !b.empty(); i++)
    {
        P x = a[i];
        P y = a[(i + 1) % n];
        if (s < 0)
        {
            swap(x, y);
        }
        b = cutPoly(b, {x, y - x});
    }
    return b;
}
```

`polyKernel(poly)` 接受顺时针或逆时针简单多边形，空返回值表示核为空。基础版逐边调用 11.2.004 多边形切割，$O(n^2)$ 时间、$O(n)$ 空间；边数很大时可改用 11.3.009 半平面交 的 $O(n \log n)$ 实现。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，一般非凸布尔运算不在当前模板范围。

11.2.006 Ear Clipping 三角剖分

把简单多边形拆成不相交三角形。Ear Clipping 容易实现并能恢复三角形编号，但最坏复杂度较高。Ear Clipping 维护当前多边形链，凸顶点且其耳三角形内部不含其他顶点时可剪下；每次删除一个耳，直到只剩最后一个三角形。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> vector<array<int, 3>> triangulate(const vector<Point<T>> &a)
{
    // a 是简单多边形；返回三角形对应的原顶点下标。
    int n = (int)a.size();
    if (n < 3)
    {
        return {};
    }
    T s = area2(a);
    assert(sgn(s) != 0); // 调试检查，可删。
    vector<int> p(n);
    iota(p.begin(), p.end(), 0);
```

```
    if (s < 0)
    {
        reverse(p.begin(), p.end());
    }
    bool changed = true;
    while (changed && p.size() > 3)
    {
        changed = false;
        int m = (int)p.size();
        for (int i = 0; i < m; i++)
        {
            int x = p[(i + m - 1) % m];
            int y = p[i];
            int z = p[(i + 1) % m];
            if (sgn(cross(a[x], a[y], a[z])) == 0)
            {
                p.erase(p.begin() + i);
                changed = true;
                break;
            }
        }
    }
    vector<array<int, 3>> ans;
    auto inside = [&](Point<T> q, Point<T> x, Point<T> y, Point<T> z)
    {
        return sgn(cross(x, y, q)) >= 0 && sgn(cross(y, z, q)) >= 0
            && sgn(cross(z, x, q)) >= 0;
    };
    while (p.size() > 3)
    {
        bool ok = false;
        int m = (int)p.size();
        for (int i = 0; i < m; i++)
        {
            int x = p[(i + m - 1) % m];
            int y = p[i];
            int z = p[(i + 1) % m];
            if (sgn(cross(a[x], a[y], a[z])) <= 0)
            {
                continue;
            }
            bool bad = false;
            for (int w : p)
            {
                if (w ≠ x && w ≠ y && w ≠ z && inside(a[w], a[x], a[y], a[z]))
                {
                    bad = true;
                    break;
                }
            }
            if (bad)
            {
```

```
                continue;
            }
            ans.push_back({x, y, z});
            p.erase(p.begin() + i);
            ok = true;
            break;
        }
        if (!ok)
        {
            return {};
        }
    }
    if (p.size() == 3)
    {
        ans.push_back({p[0], p[1], p[2]});
    }
    return ans;
}
```

调用 `triangulate(poly)`，输入必须是不自交、面积非零的简单多边形；返回的 `array<int,3>` 三维分别是原多边形三个顶点下标。连续共线的边界点会先被删除，因而不一定出现在三角形中；少于三个点返回空序列。最坏 $O(n^3)$ 时间、$O(n)$ 空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.007 多边形交、并

计算两个凸多边形的交与并面积，并给出小规模 Delaunay 三角剖分边集。非凸多边形布尔运算不属于当前接口。凸多边形交可连续半平面裁剪；一般多边形布尔运算需求交点、按参数切边并依据内外状态选择边段，不能只拼顶点集合。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.1.009 线段相交与距离
// 依赖 11.2.001 多边形面积、周长、重心
vector<P> convexIsect(vector<P> a, const vector<P> &b)
{
    // a、b 是逆时针凸包；返回包含退化情形的交集凸包。
    auto in = [&](P p, const vector<P> &q)
    {
        if (q.empty())
        {
            return false;
        }
        if (q.size() == 1)
        {
            return eq(p, q[0]);
```

```
        }
        if (q.size() == 2)
        {
            return onSeg(p, q[0], q[1]);
        }
        for (int i = 0; i < (int)q.size(); i++)
        {
            if (sgn(cross(q[i], q[(i + 1) % q.size()], p)) < 0)
            {
                return false;
            }
        }
        return true;
    };
    auto add = [&](vector<P> &q, P p)
    {
        for (P x : q)
        {
            if (eq(x, p))
            {
                return;
            }
        }
        q.push_back(p);
    };
    auto segSeg = [&](P x, P y, P u, P v)
    {
        vector<P> q;
        if (onSeg(x, u, v))
        {
            add(q, x);
        }
        if (onSeg(y, u, v))
        {
            add(q, y);
        }
        if (onSeg(u, x, y))
        {
            add(q, u);
        }
        if (onSeg(v, x, y))
        {
            add(q, v);
        }
        if (q.empty() && segIsect(x, y, u, v))
        {
            add(q, isect({x, y - x}, {u, v - u}));
        }
        return q;
    };
    auto segPoly = [&](P x, P y, const vector<P> &q)
    {
```

```
        if (q.empty())
        {
            return vector<P>{};
        }
        if (q.size() == 1)
        {
            return onSeg(q[0], x, y) ? vector<P>{q[0]} : vector<P>{};
        }
        if (q.size() == 2)
        {
            return segSeg(x, y, q[0], q[1]);
        }
        P d = y - x;
        Real lo = 0;
        Real hi = 1;
        for (int i = 0; i < (int)q.size(); i++)
        {
            P e = q[(i + 1) % q.size()] - q[i];
            Real c = cross(e, x - q[i]);
            Real k = cross(e, d);
            if (sgn(k) == 0)
            {
                if (sgn(c) < 0)
                {
                    return vector<P>{};
                }
                continue;
            }
            Real t = -c / k;
            if (k > 0)
            {
                lo = max(lo, t);
            }
            else
            {
                hi = min(hi, t);
            }
        }
        if (sgn(lo - hi) > 0)
        {
            return vector<P>{};
        }
        vector<P> r{x + d * lo};
        if (!eq(lo, hi))
        {
            r.push_back(x + d * hi);
        }
        return r;
    };
    auto clean = [&](vector<P> q)
    {
        vector<P> r;
```

```
        for (P p : q)
        {
            if (r.empty() || !eq(r.back(), p))
            {
                r.push_back(p);
            }
        }
        if (r.size() > 1 && eq(r.front(), r.back()))
        {
            r.pop_back();
        }
        if (r.size() > 2 && sgn(area(r)) == 0)
        {
            P x = *min_element(r.begin(), r.end());
            P y = *max_element(r.begin(), r.end());
            return eq(x, y) ? vector<P>{x} : vector<P>{x, y};
        }
        return r;
    };
    if (a.empty() || b.empty())
    {
        return {};
    }
    if (a.size() == 1)
    {
        return in(a[0], b) ? a : vector<P>{};
    }
    if (b.size() == 1)
    {
        return in(b[0], a) ? b : vector<P>{};
    }
    if (a.size() == 2)
    {
        return segPoly(a[0], a[1], b);
    }
    if (b.size() == 2)
    {
        return segPoly(b[0], b[1], a);
    }
    for (int i = 0; i < (int)b.size() && !a.empty(); i++)
    {
        P p = b[i];
        P q = b[(i + 1) % b.size()];
        vector<P> c;
        for (int j = 0; j < (int)a.size(); j++)
        {
            P x = a[j];
            P y = a[(j + 1) % a.size()];
            int sx = sgn(cross(p, q, x));
            int sy = sgn(cross(p, q, y));
            if (sx >= 0)
            {
```

```
                c.push_back(x);
            }
            if (sx * sy < 0)
            {
                c.push_back(isect({x, y - x}, {p, q - p}));
            }
        }
        a.swap(c);
    }
    return clean(a);
}

Real convexUnion(const vector<P> &a, const vector<P> &b)
{
    // a、b 是逆时针凸包；返回二者并集面积。
    return area(a) + area(b) - area(convexIsect(a, b));
}
```

`convexIsect(a,b)` 返回两个逆时针凸包的交，空包、点和线段也可直接传入；`convexUnion(a,b)` 由容斥返回并面积。两个非退化凸多边形相交最坏 $O(nm)$，退化情形 $O(n+m)$；Delaunay 单独见 11.3.011。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.008 多边形与圆面积交

计算圆与简单多边形的相交面积。把每条有向边对圆心的面积贡献分成三角形段和扇形段累加。把每条有向边与圆围成的面积贡献分段计算：圆内段用三角形叉积，圆外段用扇形角；沿多边形边累加有向贡献后取绝对值。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
Real circlePoly(const vector<P> &a, P o, Real r)
{
    // a 是环形点列，o、r 是圆心和非负半径；返回多边形与圆的相交面积。
    assert(sgn(r) >= 0); // 调试检查，可删。
    auto edge = [&](P a, P b)
    {
        P v = b - a;
        array<Real, 4> q{0, 1};
        int m = 2;
        Real aa = norm(v);
        Real bb = 2 * dot(a, v);
        Real cc = norm(a) - r * r;
        Real d = bb * bb - 4 * aa * cc;
        if (sgn(d) >= 0 && sgn(aa) != 0)
        {
            d = sqrtl(max(Real(0), d));
            for (Real t : {(-bb - d) / (2 * aa), (-bb + d) / (2 * aa)})
            {
                if (sgn(t) > 0 && sgn(t - 1) < 0)
```

```
                {
                    q[m++] = t;
                }
            }
        }
        for (int i = 1; i < m; i++)
        {
            for (int j = i; j > 0 && q[j] < q[j - 1]; j--)
            {
                swap(q[j], q[j - 1]);
            }
        }
        Real ans = 0;
        for (int i = 0; i + 1 < m; i++)
        {
            P x = a + v * q[i];
            P y = a + v * q[i + 1];
            P z = (x + y) / 2;
            if (sgn(norm(z) - r * r) <= 0)
            {
                ans += cross(x, y) / 2;
            }
            else
            {
                ans += atan2l(cross(x, y), dot(x, y)) * r * r / 2;
            }
        }
        return ans;
    };
    Real ans = 0;
    for (int i = 0, n = a.size(); i < n; i++)
    {
        ans += edge(a[i] - o, a[(i + 1) % n] - o);
    }
    return fabsl(ans);
}
```

调用 `circlePoly(poly,o,r)`；`poly` 按环形顺序给出，`o`、`r` 是圆心和非负半径，返回无符号相交面积。接口直接接收计算所需的三个量，不为章节先后顺序增加圆类型模板参数。$O(n)$ 时间、$O(1)$ 额外空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

把下面的固定样例接在本章公共前导、上述实现与 11.4.001 Circle 基本对象 之后即可快速检查：半径为 1 的圆完全位于边长为 4 的正方形内，答案应为 π。

```
signed main()
{
    vector<P> a{{-2, -2}, {2, -2}, {2, 2}, {-2, 2}};
    cout << fixed << setprecision(12) << circlePoly(a, {0, 0}, 1) << '\n';
}
```

```
// 期望输出：3.141592653590
```

11.2.009 Pick 定理与格点多边形

对格点简单多边形使用 Pick 定理计算内部格点数与边界格点数。坐标、面积两倍值和 gcd 均保持整数精确。顶点为格点的简单多边形满足 `A=I+B/2-1`；边界格点数由每条边的 `gcd(|dx|,|dy|)` 累加，面积应用叉积保持两倍整数形式。

```
pair<long long, long long> pick(const vector<pair<long long, long long>> &a)
{
    // a 是非退化格点简单多边形；返回内部、边界格点数。
    using i128 = __int128_t;
    int n = (int)a.size();
    assert(n >= 3); // 调试检查，可删。
    auto ab = [&](i128 x)
    {
        return x < 0 ? -x : x;
    };
    auto gcd128 = [&](i128 x, i128 y)
    {
        while (y != 0)
        {
            i128 r = x % y;
            x = y;
            y = r;
        }
        return x;
    };
    i128 s = 0;
    i128 b = 0;
    for (int i = 0; i < n; i++)
    {
        auto [x, y] = a[i];
        auto [u, v] = a[(i + 1) % n];
        s += (i128)x * v - (i128)y * u;
        i128 dx = ab((i128)x - u);
        i128 dy = ab((i128)y - v);
        b += gcd128(dx, dy);
    }
    s = ab(s);
    assert(s > 0); // 调试检查，可删。
    assert((s - b) % 2 == 0); // 调试检查，可删。
    i128 in = (s - b + 2) / 2;
    assert(in >= 0 && in <= LLONG_MAX && b <= LLONG_MAX); // 调试检查，可删。
    return {(long long)in, (long long)b};
}
```

调用 `pick(poly)`，返回 `{内部格点数，边界格点数}`；顶点必须构成面积非零的格点简单多边形，且两个答案均能用 `long long` 表示。鞋带项、坐标差、绝对值和最大公约数都在 `__int128` 中计算。$O(n)$ 时间、$O(1)$ 额外空间。只做点包含使用 11.2.002 点在多边形内与有符号绕数；整条线段包含必须用 11.2.003 线

段是否在多边形内。凸多边形交可用 11.2.007 多边形交、并，由半平面描述的凸区域改用 11.3.009 半平面交；一般非凸布尔运算不在当前模板范围。

11.2.010 三角形点集批量计数

给定候选顶点集 A 和待统计点集 B，预处理后回答大量“三个 A 中顶点构成的三角形严格包含多少个 B 点”。把三个顶点按 (y,x) 排序后，内部区域可由若干条按低端到高端定向的边左侧半平面相加减；`edgeL[u][v]` 只数纵坐标严格位于两端之间且在边左侧的点，`edgeM` 单独记录边界，横向穿过中间顶点时再用 `pointL`、`pointM` 补齐同高点。坐标绝对值不超过 10^9 时叉积绝对值不超过 8×10^{18}，这里显式使用 `long long`，不额外扩大全部表项。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
struct TriPointCnt
{
    vector<Point<long long>> a; // a 是候选顶点集 A。
    vector<int> pointL, pointM; // pointL[u]、pointM[u] 是与 a[u] 同高且在其左侧、与其重合的 B 点数。
    vector<vector<int>> edgeL, edgeM; // edgeL[u][v]、edgeM[u][v] 是纵坐标严格夹在有向边 u->v 两端之间、位于其左侧、线上的 B 点数。

    TriPointCnt(const vector<Point<long long>> &vertices,
                        const vector<Point<long long>> &points) : a(vertices)
    {
        // vertices 是候选顶点集 A（count 用其下标），points 是待统计点集 B；预处理同高点计数 pointL/pointM。
        // 并对每条低到高有向边统计纵坐标严格夹在两端之间、位于其左侧/线上的 B 点数 edgeL/edgeM。
        int n = a.size();
        pointL.assign(n, 0);
        pointM.assign(n, 0);
        edgeL.assign(n, vector<int>(n));
        edgeM.assign(n, vector<int>(n));
        for (int u = 0; u < n; u++)
        {
            for (auto p : points)
            {
                if (p.y == a[u].y)
                {
                    pointL[u] += p.x < a[u].x;
                    pointM[u] += p.x == a[u].x;
                }
            }
        }
        for (int u = 0; u < n; u++)
        {
            for (int v = 0; v < n; v++)
            {
                if (a[u].y >= a[v].y)
                {
                    continue;
```

```
                }
                for (auto p : points)
                {
                    if (a[u].y < p.y && p.y < a[v].y)
                    {
                        long long det = cross(a[u] - a[v], p - a[v]);
                        edgeL[u][v] += det < 0;
                        edgeM[u][v] += det == 0;
                    }
                }
            }
        }
    }

    bool lessYX(int u, int v) const
    {
        // u、v 是顶点下标；返回 a[u] 是否按 (y,x) 字典序严格小于 a[v]。
        return pair(a[u].y, a[u].x) < pair(a[v].y, a[v].x);
    }

    int count(int x, int y, int z) const
    {
        // x、y、z 是 A 中三个顶点下标；返回严格位于该三角形内部的 B 点个数，三点共线或重合时返回 0。
        int n = a.size();
        assert(0 <= x && x < n && 0 <= y && y < n && 0 <= z && z < n); // 调试检查，可删。
        if (lessYX(y, x))
        {
            swap(x, y);
        }
        if (lessYX(z, y))
        {
            swap(y, z);
        }
        if (lessYX(y, x))
        {
            swap(x, y);
        }
        long long det = cross(a[x] - a[z], a[y] - a[z]);
        if (det == 0)
        {
            return 0;
        }
        int ans = 0;
        if (a[x].y == a[y].y)
        {
            ans = edgeL[y][z] - edgeL[x][z] - edgeM[x][z];
        }
        else if (a[y].y == a[z].y)
        {
            ans = edgeL[x][z] - edgeL[x][y] - edgeM[x][y];
        }
        else if (det < 0)
```

```
        {
            ans = edgeL[x][z] - edgeL[y][z] - edgeM[y][z] -
                  edgeL[x][y] - edgeM[x][y] - pointL[y] - pointM[y];
        }
        else
        {
            ans = edgeL[x][y] + edgeL[y][z] + pointL[y] -
                  edgeL[x][z] - edgeM[x][z];
        }
        return ans;
    }
};
```

构造 `TriPointCnt q(A,B)` 后调用 `q.count(i,j,k)`；返回值不含三条边上的点，三点共线或重合时返回 0。若 $|A|=N,|B|=M$，预处理 $O(N^2M)$ 时间、$O(N^2)$ 空间，每次查询 $O(1)$；单次点包含可直接用 11.2.002 点在多边形内与有符号绕数。

下面用固定点集检查水平边、边界点、重复点与退化三角形：

```
signed main()
{
    using Q = Point<long long>;
    vector<Q> a{{0, 0}, {4, 0}, {0, 4}, {2, 0}, {0, 0}};
    vector<Q> b{{1, 1}, {2, 1}, {1, 2}, {2, 0}, {0, 0}, {1, 1}, {4, 4}};
    TriPointCnt q(a, b);
    assert(q.count(0, 1, 2) == 4);
    assert(q.count(0, 3, 2) == 2);
    assert(q.count(0, 3, 1) == 0);
    assert(q.count(0, 1, 4) == 0);
}
```

11.2.011 多个简单多边形的覆盖层面积

求多个无洞简单多边形“至少被 k 个覆盖”的面积。对每条逆时针源边，把所有其他边与其支撑直线的交点按参数排序；若某个原子段右侧已被其他代表多边形覆盖 c 次，而源多边形有 w 个完全重复副本，则该有向段依次是第 $c+1$ 至第 $c+w$ 层的边界，按 Green 公式向这一层区间加入 $\frac{1}{2}\operatorname{cross}(s,t)$。整数版用有理参数精确排序和合批，浮点版也只用原始值建立严格弱序，误差判断不进入比较器。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.2.001 多边形面积、周长、重心
struct Fraction128
{
    __int128 num; // 分子。
    __int128 den; // 始终为正的分母。

    Fraction128(__int128 num = 0, __int128 den = 1) : num(num), den(den)
    {
        // num、den 是分子和非零分母；构造符号规范化的有理数。
```

```
        assert(den != 0); // 调试检查，可删。
        if (den < 0)
        {
            this->num = -num;
            this->den = -den;
        }
    }

    friend bool operator<(Fraction128 a, Fraction128 b)
    {
        // a、b 是有理数；返回 a 是否严格小于 b，交叉乘积必须能由 __int128 表示。
        return a.num * b.den < b.num * a.den;
    }

    friend bool operator==(Fraction128 a, Fraction128 b)
    {
        // a、b 是有理数；返回二者是否严格相等，交叉乘积必须能由 __int128 表示。
        return a.num * b.den == b.num * a.den;
    }
};

__int128 covCross(Point<long long> a, Point<long long> b, Point<long long> c)
{
    // a、b 确定有向直线，c 是查询点；返回使用宽整数计算的叉积。
    __int128 x1 = (__int128)b.x - a.x;
    __int128 y1 = (__int128)b.y - a.y;
    __int128 x2 = (__int128)c.x - a.x;
    __int128 y2 = (__int128)c.y - a.y;
    return x1 * y2 - y1 * x2;
}

int covSide(Point<long long> a, Point<long long> b, Point<long long> p)
{
    // a、b 确定整数有向直线，p 是查询点；返回 p 所在侧的精确符号。
    return sgn(covCross(a, b, p));
}

int covSide(P a, P b, P p)
{
    // a、b 确定浮点有向直线，p 是查询点；按统一误差返回 p 所在侧的符号。
    return sgn(cross(a, b, p));
}

int covDir(const vector<Point<long long>> &p)
{
    // p 是整数多边形顶点；返回其有向双倍面积的精确符号。
    __int128 sum = 0;
    for (int i = 0; i < (int)p.size(); i++)
    {
        int j = (i + 1) % p.size();
        sum += (__int128)p[i].x * p[j].y - (__int128)p[i].y * p[j].x;
    }
```

```
    return sgn(sum);
}

int covDir(const vector<P> &p)
{
    // p 是浮点多边形顶点；按统一误差返回其有向双倍面积的符号。
    return sgn(area2(p));
}

Fraction128 covParam(Point<long long> a, Point<long long> b, Point<long long> p)
{
    // a、b 确定整数参数直线，p 是线上点；返回满足 p=a+t*(b-a) 的精确参数 t。
    __int128 x = (__int128)b.x - a.x;
    __int128 y = (__int128)b.y - a.y;
    __int128 u = (__int128)p.x - a.x;
    __int128 v = (__int128)p.y - a.y;
    return {u * x + v * y, x * x + y * y};
}

Real covParam(P a, P b, P p)
{
    // a、b 确定浮点参数直线，p 是线上点；返回满足 p=a+t*(b-a) 的参数 t。
    P v = b - a;
    return dot(p - a, v) / norm(v);
}

Fraction128 covMeet(Point<long long> a, Point<long long> b, Point<long long> c, Point<long
↪ long> d)
{
    // a、b 与 c、d 确定两条不平行整数直线；返回前一条直线交点的精确参数 t。
    __int128 ux = (__int128)b.x - a.x;
    __int128 uy = (__int128)b.y - a.y;
    __int128 vx = (__int128)d.x - c.x;
    __int128 vy = (__int128)d.y - c.y;
    __int128 wx = (__int128)c.x - a.x;
    __int128 wy = (__int128)c.y - a.y;
    return {wx * vy - wy * vx, ux * vy - uy * vx};
}

Real covMeet(P a, P b, P c, P d)
{
    // a、b 与 c、d 确定两条不平行浮点直线；返回前一条直线交点的参数 t。
    P u = b - a;
    P v = d - c;
    return cross(c - a, v) / cross(u, v);
}

Real covVal(Fraction128 x)
{
    // x 是精确有理参数；返回其 long double 近似值。
    return (Real)x.num / (Real)x.den;
}
```

```cpp

Real covVal(Real x)
{
    // x 是浮点参数；返回其原值。
    return x;
}

template <class T, class Key> vector<Real> covAreaImpl(vector<vector<Point<T>>> poly)
{
    // poly 是无洞简单逆时针多边形集；返回各覆盖层面积，Key 是事件参数类型。
    int n = (int)poly.size();
    map<vector<Point<T>>, int> weight;
    for (auto p : poly)
    {
        if (p.size() > 1 && p.front() == p.back())
        {
            p.pop_back();
        }
        assert(p.size() >= 3 && covDir(p) > 0); // 调试检查，可删。
        for (int i = 0; i < (int)p.size(); i++)
        {
            assert(!(p[i] == p[(i + 1) % p.size()])); // 调试检查，可删。
        }
        int pos = min_element(p.begin(), p.end()) - p.begin();
        rotate(p.begin(), p.begin() + pos, p.end());
        weight[p]++;
    }
    vector<vector<Point<T>>> q;
    vector<int> w;
    for (auto &[p, cnt] : weight)
    {
        q.push_back(p);
        w.push_back(cnt);
    }
    for (int i = 0; i < (int)q.size(); i++)
    {
        for (int j = i + 1; j < (int)q.size(); j++)
        {
            for (int x = 0; x < (int)q[i].size(); x++)
            {
                Point<T> a = q[i][x];
                Point<T> b = q[i][(x + 1) % q[i].size()];
                for (int y = 0; y < (int)q[j].size(); y++)
                {
                    Point<T> c = q[j][y];
                    Point<T> d = q[j][(y + 1) % q[j].size()];
                    if (covSide(a, b, c) != 0 || covSide(a, b, d) != 0)
                    {
                        continue;
                    }
                    T al, ar, bl, br;
                    if (a.x != b.x)
```

```
                    {
                        al = min(a.x, b.x);
                        ar = max(a.x, b.x);
                        bl = min(c.x, d.x);
                        br = max(c.x, d.x);
                    }
                    else
                    {
                        al = min(a.y, b.y);
                        ar = max(a.y, b.y);
                        bl = min(c.y, d.y);
                        br = max(c.y, d.y);
                    }
                    assert(!(max(al, bl) < min(ar, br))); // 不支持不同代表多边形正长度共边。
                }
            }
        }
    }
    vector<Real> diff(n + 1);
    for (int i = 0; i < (int)q.size(); i++)
    {
        for (int x = 0; x < (int)q[i].size(); x++)
        {
            Point<T> a = q[i][x];
            Point<T> b = q[i][(x + 1) % q[i].size()];
            Key zero = covParam(a, b, a);
            Key one = covParam(a, b, b);
            vector<pair<Key, int>> e{{zero, 0}, {one, 0}};
            for (int j = 0; j < (int)q.size(); j++)
            {
                if (i == j)
                {
                    continue;
                }
                for (int y = 0; y < (int)q[j].size(); y++)
                {
                    Point<T> c = q[j][y];
                    Point<T> d = q[j][(y + 1) % q[j].size()];
                    int sc = covSide(a, b, c);
                    int sd = covSide(a, b, d);
                    if (sc == 0 && sd == 0)
                    {
                        e.push_back({covParam(a, b, c), 0});
                        e.push_back({covParam(a, b, d), 0});
                        continue;
                    }
                    int delta = 0;
                    if (sc < 0 && sd >= 0)
                    {
                        delta = -w[j];
                    }
                    if (sc >= 0 && sd < 0)
```

```
                    {
                        delta = w[j];
                    }
                    if (!delta)
                    {
                        continue;
                    }
                    Key t;
                    if (sc == 0)
                    {
                        t = covParam(a, b, c);
                    }
                    else if (sd == 0)
                    {
                        t = covParam(a, b, d);
                    }
                    else
                    {
                        t = covMeet(a, b, c, d);
                    }
                    e.push_back({t, delta});
                }
            }
            sort(e.begin(),
                 e.end(),
                 [](const auto &p, const auto &r)
                 {
                     if (p.first < r.first)
                     {
                         return true;
                     }
                     if (r.first < p.first)
                     {
                         return false;
                     }
                     return p.second < r.second;
                 });
            int cnt = 0;
            Key last = e.front().first;
            for (int j = 0; j < (int)e.size();)
            {
                Key t = e[j].first;
                Key l = last < zero ? zero : last;
                Key r = one < t ? one : t;
                if (l < r)
                {
                    assert(0 <= cnt && cnt + w[i] <= n); // 调试检查，可删。
                    P pa = (P)a;
                    P u = (P)(b - a);
                    P s = pa + u * covVal(l);
                    P z = pa + u * covVal(r);
                    Real val = cross(s, z) / 2;
```

```
                    diff[cnt] += val;
                    diff[cnt + w[i]] -= val;
                }
                int k = j;
                int delta = 0;
                while (k < (int)e.size() && e[k].first == t)
                {
                    delta += e[k].second;
                    k++;
                }
                cnt += delta;
                last = t;
                j = k;
            }
        }
    }
    vector<Real> ans(n);
    Real sum = 0;
    for (int i = 0; i < n; i++)
    {
        sum += diff[i];
        ans[i] = sgn(sum) == 0 ? 0 : sum;
    }
    return ans;
}

vector<Real> covArea(vector<vector<Point<long long>>> poly)
{
    // poly 是整数简单逆时针多边形集；事件拓扑精确、面积近似，坐标绝对值须不超过 1E9。
    // 完全重复多边形须仅差循环移位；其余不同多边形不能有正长度共边。
    return covAreaImpl<long long, Fraction128>(move(poly));
}

vector<Real> covArea(vector<vector<P>> poly)
{
    // poly 是浮点简单逆时针多边形集；返回近似覆盖面积，重复须仅差循环移位且不同代表不能正长度共边。
    return covAreaImpl<Real, Real>(move(poly));
}
```

两个 `covArea` 重载都返回长度为多边形数的数组，`ans[k-1]` 表示至少覆盖 k 次的面积。每个多边形必须简单、无洞、顶点逆时针且无零长度边；完全重复只允许顶点序列相差循环移位，不同代表多边形不得有正长度共边，普通横截、相切、点接触和内含均可。整数坐标绝对值须不超过 10^9，此时拓扑事件精确而最终面积仍转为 `Real`；浮点版为近似拓扑，不适合近重合退化输入。设总边数为 E，时间 $O(E^2 \log E)$，额外空间 $O(E+n)$；只有两个凸多边形且只求交并面积时，改用 11.2.007 多边形交、并 更短。

11.2.012 直线平面嵌入的面枚举与对偶图

给定顶点整数坐标和已经画好的连通无向直线平面图，可以直接从边的几何顺序恢复各面，而不必手工提供面邻接。把每条无向边拆成两条有向半边，并在每个顶点把出半边按极角逆时针排序；对于半边 $u \to v$，在 v 处取反向半边 $v \to u$ 的顺时针相邻边作为后继，就会始终沿当前半边左侧面的边界前进。这个后继

关系是半边集合上的置换，其每个环对应一个面；沿环累加鞋带公式得到有符号双倍面积，有限面为正，外部面为最小值。原边两条半边所属面不同时就在两面之间加入带权对偶边，桥的两侧是同一面，因而不会产生对偶自环。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T, class W = long long> struct PlanarEmbed
{
    struct Edge
    {
        int u;    // u 是无向边的一个端点。
        int v;    // v 是无向边的另一个端点。
        W weight; // weight 是穿过该边时对应的对偶边权。
    };

    vector<Point<T>> point;            // point 保存各顶点的互异整数坐标。
    vector<Edge> edge;                 // edge 按加入顺序保存无自环、无重复的无向边。
    vector<array<int, 2>> halfEdge;    // halfEdge[h][0]、[1] 分别是半边 h 的起点、终点。
    vector<int> nextHalfEdge;          // nextHalfEdge[h] 是沿 h 左侧面边界继续走的下一条半边。
    vector<int> faceOfHalf;        // faceOfHalf[h] 是半边 h 左侧面的编号。
    vector<__int128> faceArea2;        // faceArea2[f] 是面 f 边界游走的有符号双倍面积。
    int outer = -1;                // outer 是外部面的编号。
    vector<vector<pair<int, W>>> dual; // dual[f] 的元素是相邻面编号和分隔边权，允许平行边。

    explicit PlanarEmbed(vector<Point<T>> point = {}) : point(move(point))
    {
        // point 是平面嵌入各顶点的互异整数坐标；建立尚未加入边的嵌入。
        static_assert(is_integral_v<T>);
    }

    void addEdge(int u, int v, const W &weight = W(1))
    {
        // u、v 是不同端点编号，weight 是穿过该边的代价；加入一条无向边，无返回值。
        assert(0 <= u && u < (int)point.size());   // 调试检查，可删。
        assert(0 <= v && v < (int)point.size());   // 调试检查，可删。
        assert(u != v && !(point[u] == point[v])); // 调试检查，可删。
        edge.push_back({u, v, weight});
    }

    void build()
    {
        // 无参数；枚举各半边左侧面并填写 nextHalfEdge、faceOfHalf、faceArea2、outer 与 dual，无返回值。
        int n = point.size();
        int m = edge.size();
        halfEdge.assign(2 * m, {});
        nextHalfEdge.assign(2 * m, -1);
        faceOfHalf.assign(2 * m, -1);
        faceArea2.clear();
        outer = -1;
        dual.clear();

        set<Point<T>> uniquePoint(point.begin(), point.end());
```

```
        assert((int)uniquePoint.size() == n); // 调试检查，可删。
        vector<vector<int>> around(n);
        set<pair<int, int>> uniqEdge;
        for (int i = 0; i < m; i++)
        {
            auto [u, v, weight] = edge[i];
            assert(0 <= u && u < n && 0 <= v && v < n); // 调试检查，可删。
            assert(u != v && !(point[u] == point[v]));  // 调试检查，可删。
            pair<int, int> key{min(u, v), max(u, v)};
            assert(uniqEdge.insert(key).second); // 调试检查，可删。
            halfEdge[2 * i] = {u, v};
            halfEdge[2 * i + 1] = {v, u};
            around[u].push_back(2 * i);
            around[v].push_back(2 * i + 1);
        }

        if (m == 0)
        {
            assert(n <= 1); // 多点输入仍要求连通。
            faceArea2.push_back(0);
            outer = 0;
            dual.resize(1);
            return;
        }

        vector<char> seen(n);
        queue<int> q;
        seen[0] = true;
        q.push(0);
        while (!q.empty())
        {
            int u = q.front();
            q.pop();
            for (int h : around[u])
            {
                int v = halfEdge[h][1];
                if (!seen[v])
                {
                    seen[v] = true;
                    q.push(v);
                }
            }
        }
        assert(count(seen.begin(), seen.end(), true) == n); // 调试检查，可删。

        auto dir = [&](int h)
        {
            // h 是半边编号；返回先提升到 __int128 再相减得到的方向向量。
            int u = halfEdge[h][0];
            int v = halfEdge[h][1];
            __int128 x = (__int128)point[v].x - (__int128)point[u].x;
            __int128 y = (__int128)point[v].y - (__int128)point[u].y;
```

```
            return pair<__int128, __int128>{x, y};
        };
        auto halfPlane = [](__int128 x, __int128 y)
        {
            // x、y 是非零方向向量坐标；返回极角排序使用的半平面编号。
            return y < 0 || (y == 0 && x < 0);
        };

        vector<int> position(2 * m);
        for (int u = 0; u < n; u++)
        {
            sort(around[u].begin(),
                 around[u].end(),
                 [&](int a, int b)
                 {
                     // a、b 是从 u 出发的半边编号；返回 a 是否在逆时针极角序中先于 b。
                     if (a == b)
                     {
                         return false;
                     }
                     auto [ax, ay] = dir(a);
                     auto [bx, by] = dir(b);
                     int ha = halfPlane(ax, ay);
                     int hb = halfPlane(bx, by);
                     if (ha != hb)
                     {
                         return ha < hb;
                     }
                     __int128 z = ax * by - ay * bx;
                     if (z != 0)
                     {
                         return z > 0;
                     }
                     return a < b;
                 });
            for (int i = 0; i < (int)around[u].size(); i++)
            {
                position[around[u][i]] = i;
                if (i == 0)
                {
                    continue;
                }
                auto [ax, ay] = dir(around[u][i - 1]);
                auto [bx, by] = dir(around[u][i]);
                if (halfPlane(ax, ay) == halfPlane(bx, by))
                {
                    assert(ax * by - ay * bx != 0); // 不允许两条边占用同一射线。
                }
            }
        }

        for (int h = 0; h < 2 * m; h++)
```

```
        {
            int v = halfEdge[h][1];
            int reverse = h ^ 1;
            int d = around[v].size();
            int pos = position[reverse];
            nextHalfEdge[h] = around[v][(pos + d - 1) % d];
        }

        for (int start = 0; start < 2 * m; start++)
        {
            if (faceOfHalf[start] != -1)
            {
                continue;
            }
            int face = faceArea2.size();
            int h = start;
            __int128 area = 0;
            do
            {
                assert(faceOfHalf[h] == -1); // 调试检查，可删。
                faceOfHalf[h] = face;
                auto [u, v] = halfEdge[h];
                area += (__int128)point[u].x * (__int128)point[v].y - (__int128)point[u].y *
    (__int128)point[v].x;
                h = nextHalfEdge[h];
            } while (h != start);
            faceArea2.push_back(area);
        }

        outer = min_element(faceArea2.begin(), faceArea2.end()) - faceArea2.begin();
        dual.assign(faceArea2.size(), {});
        for (int i = 0; i < m; i++)
        {
            int a = faceOfHalf[2 * i];
            int b = faceOfHalf[2 * i + 1];
            if (a == b)
            {
                continue;
            }
            dual[a].push_back({b, edge[i].weight});
            dual[b].push_back({a, edge[i].weight});
        }
    }
};
```

构造 `PlanarEmbed<T,W> embedding(points)`，按输入边顺序调用 `addEdge(u,v,weight)` 后执行 `build()`；第 i 条原边对应半边 $2i$ 与 $2i+1$，`nextHalfEdge` 可还原每个面边界，`faceOfHalf`、`faceArea2`、`outer` 与 `dual` 分别给出半边面号、有符号双倍面积、外部面和带权对偶邻接。无点或单点且无边时返回唯一的零面积外部面；其余输入要求顶点坐标互异、图连通、无自环与重复边，线段只能在公共端点相交，同一顶点的边不能占用同向射线，边内部也不能经过第三个顶点。代码断言连通性等低成本条件，不额外做 $O(E^2)$ 的线段合法性检查；所有坐标差都先提升到 `__int128` 再相减，方向叉积和面积总和仍须能由 `__int128` 表

示。设顶点数、边数为 V, E，时间为 $O((V+E)\log(V+E))$，额外空间为 $O(V+E)$；拿到对偶图后需要求最小穿越代价时使用 5.2.007 平面图对偶最短路。

11.3 凸包

11.3.001 Andrew / Graham 静态凸包

用 Andrew 算法求静态点集凸包。默认删除边上的中间共线点并返回逆时针严格凸包。按坐标排序去重后分别维护下凸壳和上凸壳，遇到不符合指定转向的末点就弹栈；是否保留共线边界点由叉积比较用严格还是非严格决定。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> vector<Point<T>> hull(vector<Point<T>> a)
{
    // a 是点集；返回删除边上共线中间点后的逆时针凸包。
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    if (a.size() <= 1)
    {
        return a;
    }
    vector<Point<T>> h;
    for (int t = 0; t < 2; t++)
    {
        int z = h.size();
        for (auto p : a)
        {
            while ((int)h.size() >= z + 2 && sgn(cross(h.back() - h[h.size() - 2], p - h.back())) <= 0)
            {
                h.pop_back();
            }
            h.push_back(p);
        }
        h.pop_back();
        reverse(a.begin(), a.end());
    }
    return h;
}

// 严格凸包的直径平方；输入需逆时针且无共线邻边。
```

`hull(p)` 返回逆时针严格凸包并删除边上中间共线点；整数点全程精确，`Point<Real>` 按统一误差判断转向，但排序与去重仍使用坐标精确比较。$O(n\log n)$ 时间、$O(n)$ 空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

把下面的固定样例接在本章公共前导与上述实现之后即可快速检查：重复点和边上的共线中间点都应被删除，最终只留下正方形四角。

```cpp
signed main()
{
    vector<P> a{{0, 0}, {1, 0}, {2, 0}, {2, 2}, {0, 2}, {0, 0}, {1, 1}};
    auto h = hull(a);
    cout << h.size() << '\n';
    for (P p : h)
    {
        cout << p.x << ' ' << p.y << '\n';
    }
}
// 期望依次输出四个点：(0,0)、(2,0)、(2,2)、(0,2)。
```

11.3.002 单点插入动态凸包

维护只插点的动态凸包。上壳和下壳分别按横坐标维护严格凸链；新点若不越过相邻壳边就丢弃，否则插入后向两侧删除转向不再严格的点。同一横坐标在上壳只保留最高点、下壳只保留最低点，因此包含查询也只需在两个集合中各找一次相邻点。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> struct HullChain
{
    int dir;          // dir=1 维护上壳，dir=-1 维护下壳。
    set<Point<T>> a;  // a 按 x、y 保存当前严格凸链。

    HullChain(int dir) : dir(dir)
    {
        // dir 取 1 或 -1；分别构造上壳或下壳。
        assert(abs(dir) == 1); // 调试检查，可删
    }

    T turn(Point<T> x, Point<T> y, Point<T> z) const
    {
        // x、y、z 是三个点；返回按当前上下壳方向修正后的转向。
        return dir * cross(x, y, z);
    }

    bool add(Point<T> p)
    {
        // p 是新点；若 p 改变当前凸链则插入并返回 true。
        auto it = a.lower_bound({p.x, numeric_limits<T>::lowest()});
        if (it != a.end() && it->x == p.x)
        {
            if (dir * (p.y - it->y) <= 0)
            {
                return false;
            }
            a.erase(it);
        }
        it = a.lower_bound({p.x, numeric_limits<T>::lowest()});
        if (it != a.begin() && it != a.end() && turn(*prev(it), *it, p) <= 0)
```

```
        {
            return false;
        }
        auto q = a.insert(p).first;
        while (q ≠ a.begin())
        {
            auto y = prev(q);
            if (y == a.begin() || turn(*prev(y), *y, *q) < 0)
            {
                break;
            }
            a.erase(y);
        }
        while (next(q) ≠ a.end())
        {
            auto y = next(q);
            auto z = next(y);
            if (z == a.end() || turn(*q, *y, *z) < 0)
            {
                break;
            }
            a.erase(y);
        }
        return true;
    }

    bool contains(Point<T> p) const
    {
        // p 是查询点；返回 p 是否不越过当前凸链。
        if (a.empty())
        {
            return false;
        }
        auto it = a.lower_bound({p.x, numeric_limits<T>::lowest()});
        if (it ≠ a.end() && it->x == p.x)
        {
            return dir * (p.y - it->y) <= 0;
        }
        if (it == a.begin() || it == a.end())
        {
            return false;
        }
        return turn(*prev(it), *it, p) <= 0;
    }
};

template <class T> struct DynHull
{
    HullChain<T> up{1};    // up 保存从左到右的上凸链。
    HullChain<T> down{-1}; // down 保存从左到右的下凸链。

    void add(Point<T> p)
```

```
    {
        // p 是新点；把 p 插入动态凸包。
        up.add(p);
        down.add(p);
    }

    bool contains(Point<T> p) const
    {
        // p 是查询点；返回 p 是否在当前凸包内部或边界上。
        return up.contains(p) && down.contains(p);
    }

    vector<Point<T>> points() const
    {
        // 无参数；返回逆时针排列的当前严格凸包点。
        vector<Point<T>> h(up.a.begin(), up.a.end());
        for (auto it = down.a.rbegin(); it != down.a.rend(); it++)
        {
            if (h.empty() || (*it != h.front() && *it != h.back()))
            {
                h.push_back(*it);
            }
        }
        return h;
    }
};
```

构造 `DynHull<int>` `h` 后用 `h.add(p)` 插入整数点，`h.contains(p)` 在 $O(\log n)$ 时间判断包含，`h.points()` 用 $O(h)$ 时间输出当前逆时针严格凸包。每个点至多插入、删除常数次，单次操作摊还 $O(\log n)$、总空间 $O(n)$。集合比较要求坐标值稳定，浮点计算得到的近似点应先转为可靠的离散键，不能直接依赖 `eps` 改变 `set` 顺序。

11.3.003 点在凸包内

在严格逆时针凸包中判断点位于外部、边界还是内部。扇形二分把单次查询降到对数时间。凸多边形固定一个顶点后可按叉积二分查询点所在扇形，再判断点是否在对应三角形；边界、顶点和多边形退化为线段时需单独分类。

```
// 依赖 11.1.002 Point / Vector
template <class T> bool inConvex(const vector<Point<T>> &a, Point<T> p)
{
    // a 是逆时针严格凸包，p 是查询点；边界视为内部。
    int n = (int)a.size();
    if (n == 0)
    {
        return false;
    }
    if (n == 1)
    {
```

```
        return eq(a[0], p);
    }
    if (n == 2)
    {
        return onSeg(p, a[0], a[1]);
    }
    if (sgn(cross(a[0], a[1], p)) < 0 || sgn(cross(a[0], a.back(), p)) > 0)
    {
        return false;
    }
    int l = 1;
    int r = n - 1;
    while (r - l > 1)
    {
        int m = (l + r) / 2;
        if (sgn(cross(a[m] - a[0], p - a[0])) >= 0)
        {
            l = m;
        }
        else
        {
            r = m;
        }
    }
    int k = l;
    return sgn(cross(a[k], a[(k + 1) % n], p)) >= 0;
}
```

调用 `inConvex(h,p)`；`h` 必须是逆时针严格凸包，返回布尔值并把边界视为内部。$O(\log n)$ 时间、$O(1)$ 空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.004 凸包极点、切点、最近点查询

查询凸包的循环单峰极点、相对外点的两条切点和最近点。严格逆时针凸包相对外点的可见边形成一段连续链，两端就是切点；从任一切点到另一切点的链上，点到边的投影方向只改变一次，因此可再二分最近边。退化为点或线段时直接处理，查询点在凸包内时最近距离为 0。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.009 线段相交与距离
// 依赖 11.3.003 点在凸包内
template <class T, class F> int hullExtreme(const vector<Point<T>> &a, F f)
{
    // a 是严格逆时针凸包，f(x,y) 比较相邻方向；返回对应循环单峰的极值下标。
    int l = 0, r = (int)a.size() - 1, d = 1;
    if (f(a[r], a[l]))
    {
```

```
        swap(l, r);
        d = -1;
    }
    while (d * (r - l) > 1)
    {
        int m = (l + r) / 2;
        if (f(a[m], a[l]) && f(a[m], a[m - d]))
        {
            l = m;
        }
        else
        {
            r = m;
        }
    }
    return l;
}

template <class T> pair<int, int> hullTangents(const vector<Point<T>> &a, Point<T> p)
{
    // a 是严格逆时针凸包，p 是查询点；返回 p 的两个切点下标。
    int n = a.size();
    if (!n || inConvex(a, p))
    {
        return {-1, -1};
    }
    if (n <= 2)
    {
        return {0, n - 1};
    }
    int x = hullExtreme(a, [&](Point<T> u, Point<T> v)
    {
        return sgn(cross(p, v, u)) > 0;
    });
    int y = hullExtreme(a, [&](Point<T> u, Point<T> v)
    {
        return sgn(cross(p, u, v)) > 0;
    });
    return {x, y};
}

P nearSegment(P p, P a, P b)
{
    // p 是查询点，a、b 是不同端点；返回线段 ab 上离 p 最近的点。
    P v = b - a;
    Real t = clamp(dot(p - a, v) / norm(v), Real(0), Real(1));
    return a + v * t;
}

optional<pair<Real, P>> closestHull(const vector<P> &a, P p)
{
    // a 是严格逆时针凸包，p 是查询点；返回最短距离和一个最近点。
```

```
    int n = a.size();
    if (!n)
    {
        return nullopt;
    }
    if (inConvex(a, p))
    {
        return pair<Real, P>{0, p};
    }
    if (n == 1)
    {
        return pair<Real, P>{abs(p - a[0]), a[0]};
    }
    if (n == 2)
    {
        P q = nearSegment(p, a[0], a[1]);
        return pair<Real, P>{abs(p - q), q};
    }
    auto at = [&](int i)
    {
        return a[(i % n + n) % n];
    };
    auto chain = [&](int l, int r)
    {
        if (l > r)
        {
            r += n;
        }
        auto get = [&](int i)
        {
            P q = nearSegment(p, at(i), at(i + 1));
            return pair<Real, P>{abs(p - q), q};
        };
        auto ans = get(l);
        int side = sgn(dot(p - at(l), at(l + 1) - at(l)));
        while (l + 1 < r)
        {
            int m = (l + r) / 2;
            int cur = sgn(dot(p - at(m), at(m + 1) - at(m)));
            if (cur == side)
            {
                l = m;
            }
            else
            {
                r = m;
            }
        }
        auto q = get(l);
        if (q.first < ans.first)
        {
            ans = q;
```

```
        }
        return ans;
    };
    auto [x, y] = hullTangents(a, p);
    auto u = chain(x, y);
    auto v = chain(y, x);
    return u.first < v.first ? u : v;
}
```

调用 `closestHull(h,p)`；有答案时返回 `{最短距离, 最近点}`，空凸包返回 `nullopt`。非退化输入要求 `h` 是没有共线中间点的严格逆时针凸包；外点查询 $O(\log h)$，内部判断同样为 $O(\log h)$，额外空间 $O(1)$。若凸包保留了整段共线边，应先删去中间点，否则切点端点约定和单峰性都会改变。

单独使用 `hullExtreme` 时，比较器定义相邻点中第一点是否更优。例如查最大横坐标：

```
signed main()
{
    vector<Point<int>> h{{0, 0}, {2, 0}, {0, 1}};
    int p = hullExtreme(h, [](Point<int> x, Point<int> y)
    {
        return x.x > y.x;
    });
    assert(p == 1 && h[p].x == 2);
}
```

11.3.005 凸包切线

求凸包相对外部点的两条切线。切点二分已经由 11.3.004 凸包极点、切点、最近点查询 的 `hullTangents` 完整给出，本条只把两个切点转成直线，避免重复一套容易产生不同边界约定的二分。严格凸包上的可见边形成循环连续段，两条切线正好经过该段的两个端点。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.005 Line / Ray / Segment
template <class T> optional<array<Line<T>, 2>> hullTan(const vector<Point<T>> &a, Point<T> p)
{
    // a 是严格逆时针凸包，p 是查询点；外点返回两条切线，否则返回空。
    auto [x, y] = hullTangents(a, p);
    if (x < 0)
    {
        return nullopt;
    }
    return array<Line<T>, 2>{Line<T>{p, a[x]}, Line<T>{p, a[y]}};
}
```

`hullTangents(h,p)` 返回两个切点下标，`hullTan(h,p)` 返回对应点式直线；查询点在内部或边界时前者返回 `{-1,-1}`、后者返回空。严格凸包查询为 $O(\log h)$ 时间、$O(1)$ 空间；共线边的切点不唯一，若题目要求整段接触边，应在返回端点附近另行扩展相等方向。

11.3.006 旋转卡壳、凸包直径

在凸包上用旋转卡壳求最远点对和直径。卡壳指针随边方向单调移动，因此扫描为线性。凸包对踵点随枚举边单调前进；当下一点使边与点的叉积绝对值增大就移动指针，每对最远点会在某条支撑线方向被检查。

```cpp
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> T diameter2(const vector<Point<T>> &a)
{
    // a 是逆时针凸包；返回最远点对距离平方。
    int n = (int)a.size();
    if (n <= 1)
    {
        return 0;
    }
    if (n == 2)
    {
        return norm(a[0] - a[1]);
    }
    T ans = 0;
    int j = 1;
    for (int i = 0; i < n; i++)
    {
        while (sgn(cross(a[(i + 1) % n] - a[i], a[(j + 1) % n] - a[j])) > 0)
        {
            j = (j + 1) % n;
        }
        ans = max(ans, norm(a[i] - a[j]));
        ans = max(ans, norm(a[(i + 1) % n] - a[j]));
    }
    return ans;
}

template <class T> pair<int, int> farthestPair(const vector<Point<T>> &p)
{
    // p 是至少含两个点的点集；返回一对欧氏距离最大的原下标。
    assert(p.size() >= 2); // 调试检查，可删。
    map<Point<T>, int> id;
    for (int i = 0; i < (int)p.size(); i++)
    {
        id.try_emplace(p[i], i);
    }
    auto h = hull(p);
    if (h.size() == 1) return {0, 1};
    if (h.size() == 2) return {id[h[0]], id[h[1]]};
    int m = h.size(), j = 1;
    T best = -1;
    pair<int, int> ans{0, 1};
    auto update = [&](int x, int y)
    {
        T d = norm(h[x] - h[y]);
        if (best < d)
```

```
        {
            best = d;
            ans = {id[h[x]], id[h[y]]};
        }
    };
    for (int i = 0; i < m; i++)
    {
        while (cross(h[(i + 1) % m] - h[i],
                     h[(j + 1) % m] - h[j]) > 0)
        {
            j = (j + 1) % m;
        }
        update(i, j);
        update((i + 1) % m, j);
    }
    return ans;
}
```

调用 `diameter2(h)` 时 `h` 按逆时针给出且不得重复首点，并要求严格凸，返回最远点对距离平方，$O(h)$ 时间、$O(1)$ 额外空间。输入是按边界顺序给出的弱凸多边形时调用 `weakDiam2(h)`；它删除共线中间点，单独处理整图共线，并在平行支撑线并列时检查两个候选，$O(h)$ 时间、$O(h)$ 额外空间，顺逆时针均可。`farthestPair(p)` 接受至少两个、可重复且无需预建凸包的点，返回一对最远点的原下标；全相同点返回两个不同下标。它先以 $O(n \log n)$ 建凸包，再用 $O(h)$ 旋转卡壳，额外空间 $O(n)$。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

把下面的固定样例接在本章公共前导与上述实现之后即可快速检查：3×4 矩形的直径平方应为 25。

```
signed main()
{
    vector<P> h{{0, 0}, {3, 0}, {3, 4}, {0, 4}};
    cout << diameter2(h) << '\n';
}
// 期望输出：25
```

11.3.007 最大面积三角形、四边形

求凸包顶点组成的最大面积三角形或四边形。三角形可双指针，当前四边形基准版保留更高复杂度以便核对。固定凸包一条边时，对边面积最大的第三点沿边推进而单调移动；四边形可用两条对角线两侧最远点面积之和，避免枚举四点。

```
// 依赖 11.1.002 Point / Vector
template <class T> tuple<T, int, int, int> maxTriangle(const vector<Point<T>> &a)
{
    // a 是逆时针凸包；返回最大双倍面积和三个顶点下标。
    int n = (int)a.size();
    tuple<T, int, int, int> ans{0, -1, -1, -1};
    if (n < 3)
    {
```

```
        return ans;
    }
    auto area = [&](int i, int j, int k)
    {
        T s = cross(a[i % n], a[j % n], a[k % n]);
        return s < 0 ? -s : s;
    };
    for (int i = 0; i < n; i++)
    {
        int k = i + 2;
        for (int j = i + 1; j < n; j++)
        {
            if (k <= j)
            {
                k = j + 1;
            }
            while (k + 1 < i + n && sgn(area(i, j, k + 1) - area(i, j, k)) > 0)
            {
                k++;
            }
            T s = area(i, j, k);
            if (s > get<0>(ans))
            {
                ans = {s, i, j % n, k % n};
            }
        }
    }
    return ans;
}

template <class T> T maxQuad(const vector<Point<T>> &a)
{
    // a 是逆时针凸包；返回最大双倍面积。
    int n = (int)a.size();
    T ans = 0;
    auto area = [&](int i, int j, int k)
    {
        T s = cross(a[i], a[j], a[k]);
        return s < 0 ? -s : s;
    };
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            T x = 0;
            T y = 0;
            for (int k = i + 1; k < j; k++)
            {
                x = max(x, area(i, j, k));
            }
            for (int k = j + 1; k < n; k++)
            {
```

```
                y = max(y, area(i, j, k));
            }
            for (int k = 0; k < i; k++)
            {
                y = max(y, area(i, j, k));
            }
            ans = max(ans, x + y);
        }
    }
    return ans;
}
```

`maxTriangle(h)` 返回双倍面积和三个顶点下标，`maxQuad(h)` 返回双倍面积；输入必须是逆时针凸包。最大三角形 $O(h^2)$，当前最大四边形 $O(h^3)$。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.008 闵可夫斯基和

计算两个凸多边形的闵可夫斯基和。规范化顶点起点后按极角归并边向量，可用于平移碰撞和凸多边形距离。两个逆时针凸多边形的闵可夫斯基和，可从各自最低点出发按极角归并边向量并累加；结果再去除共线冗余点得到凸包。

```
// 依赖 11.1.002 Point / Vector
template <class T> vector<Point<T>> minkowski(vector<Point<T>> a, vector<Point<T>> b)
{
    // a、b 是两个点集；返回其凸包的闵可夫斯基和。
    a = hull(a);
    b = hull(b);
    if (a.empty() || b.empty())
    {
        return {};
    }
    if (a.size() <= 2 || b.size() <= 2)
    {
        vector<Point<T>> c;
        for (Point<T> x : a)
        {
            for (Point<T> y : b)
            {
                c.push_back(x + y);
            }
        }
        return hull(c);
    }
    auto edge = [](const vector<Point<T>> &p)
    {
        vector<Point<T>> e(p.size());
        for (int i = 0; i < (int)p.size(); i++)
```

```
        {
            e[i] = p[(i + 1) % p.size()] - p[i];
        }
        return e;
    };
    auto x = edge(a);
    auto y = edge(b);
    vector<Point<T>> c = {a[0] + b[0]};
    int i = 0;
    int j = 0;
    while (i < (int)x.size() || j < (int)y.size())
    {
        if (j == (int)y.size() || (i < (int)x.size() && sgn(cross(x[i], y[j])) > 0))
        {
            c.push_back(c.back() + x[i++]);
        }
        else if (i == (int)x.size() || sgn(cross(x[i], y[j])) < 0)
        {
            c.push_back(c.back() + y[j++]);
        }
        else
        {
            c.push_back(c.back() + x[i++] + y[j++]);
        }
    }
    c.pop_back();
    return hull(c);
}
```

调用 `minkowski(a,b)`；函数内部先对两组点建严格凸包，再按极角归并边向量。$O((n+m)\log(n+m))$ 时间、$O(n+m)$ 空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.009 半平面交

求一组浮点有向半平面的交。先按方向排序并去除同向冗余直线，再用双端队列维护有效边界。每条有向直线保留左侧。按极角排序后，同向直线只留约束更紧者；当队尾或队首交点落到新半平面外时弹出，对列最终相邻直线交点构成交多边形。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.006 平行、垂直、共线、同侧/异侧
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.2.001 多边形面积、周长、重心
struct HP
{
    P p; // p 保存边界直线上的一点。
    P v; // v 保存边界直线的方向。
```

```
    Real a; // a 保存 v 在 [0,2pi) 内的极角。

    HP(P p, P q) : p(p), v(q - p), a(atan2l(v.y, v.x))
    {
        // p、q 是边界直线上两个不同点；构造有向直线 p→q，保留左侧闭半平面，a 为方向极角并规范到 [0,2π)。
        assert(!eq(p, q)); // 调试检查，可删。
        if (a < 0)
        {
            a += 2 * acosl(-1);
        }
    }

    bool out(P q) const
    {
        // q 是查询点；返回其是否严格位于右侧。
        return sgn(cross(v, q - p)) < 0;
    }
};

optional<vector<P>> halfPlane(vector<HP> a)
{
    // a 中每条有向边保留左侧；空 optional 表示非空但无界。
    auto feasible = [&](vector<HP> q) -> optional<P>
    {
        mt19937 rng(0);
        shuffle(q.begin(), q.end(), rng);
        P x{};
        const Real inf = numeric_limits<Real>::infinity();
        for (int i = 0; i < (int)q.size(); i++)
        {
            if (!q[i].out(x))
            {
                continue;
            }
            Real lo = -inf;
            Real hi = inf;
            for (int j = 0; j < i; j++)
            {
                Real c = cross(q[j].v, q[i].p - q[j].p);
                Real k = cross(q[j].v, q[i].v);
                if (sgn(k) == 0)
                {
                    if (sgn(c) < 0)
                    {
                        return nullopt;
                    }
                    continue;
                }
                Real t = -c / k;
                if (k > 0)
                {
                    lo = max(lo, t);
```

```
                }
                else
                {
                    hi = min(hi, t);
                }
            }
            if (sgn(lo - hi) > 0)
            {
                return nullopt;
            }
            Real t = 0;
            if (isfinite(lo))
            {
                t = lo;
            }
            else if (isfinite(hi))
            {
                t = hi;
            }
            x = q[i].p + q[i].v * t;
        }
        return x;
    };
    if (!feasible(a))
    {
        return vector<P>{};
    }
    if (a.empty())
    {
        return nullopt;
    }
    vector<Real> ang;
    for (const HP &h : a)
    {
        ang.push_back(h.a);
    }
    sort(ang.begin(), ang.end());
    Real pi = acosl(-1);
    for (int i = 0; i < (int)ang.size(); i++)
    {
        Real x = ang[i];
        Real y = i + 1 < (int)ang.size() ? ang[i + 1] : ang[0] + 2 * pi;
        if (sgn(y - x - pi) >= 0)
        {
            return nullopt;
        }
    }
    sort(a.begin(), a.end(), [](const HP &x, const HP &y)
         {
             if (x.a != y.a)
             {
                 return x.a < y.a;
```

```
            }
            return cross(x.v, y.p - x.p) < 0;
        });
    vector<HP> b;
    for (HP h : a)
    {
        if (!b.empty() && parallel({b.back().p, b.back().v}, {h.p, h.v})
            && sgn(dot(b.back().v, h.v)) > 0)
        {
            if (h.out(b.back().p))
            {
                b.back() = h;
            }
            continue;
        }
        b.push_back(h);
    }
    if (b.size() > 1 && parallel({b.front().p, b.front().v}, {b.back().p, b.back().v})
        && sgn(dot(b.front().v, b.back().v)) > 0)
    {
        if (b.front().out(b.back().p))
        {
            b.back() = b.front();
        }
        b.erase(b.begin());
    }
    deque<HP> q;
    for (HP h : b)
    {
        while (q.size() > 1
               && h.out(isect({q[q.size() - 2].p, q[q.size() - 2].v},
                              {q.back().p, q.back().v})))
        {
            q.pop_back();
        }
        while (q.size() > 1
               && h.out(isect({q[0].p, q[0].v}, {q[1].p, q[1].v})))
        {
            q.pop_front();
        }
        q.push_back(h);
    }
    while (q.size() > 2
           && q.front().out(isect({q[q.size() - 2].p, q[q.size() - 2].v},
                                  {q.back().p, q.back().v})))
    {
        q.pop_back();
    }
    while (q.size() > 2
           && q.back().out(isect({q[0].p, q[0].v}, {q[1].p, q[1].v})))
    {
        q.pop_front();
```

```
    }
    if (q.size() < 3)
    {
        return vector<P>{};
    }
    vector<P> p;
    for (int i = 0; i < (int)q.size(); i++)
    {
        const HP &x = q[i];
        const HP &y = q[(i + 1) % q.size()];
        assert(!parallel({x.p, x.v}, {y.p, y.v})); // 调试检查，可删。
        P z = isect({x.p, x.v}, {y.p, y.v});
        if (p.empty() || !eq(p.back(), z))
        {
            p.push_back(z);
        }
    }
    if (p.size() > 1 && eq(p.front(), p.back()))
    {
        p.pop_back();
    }
    if (p.size() > 2 && sgn(area(p)) == 0)
    {
        P x = *min_element(p.begin(), p.end());
        P y = *max_element(p.begin(), p.end());
        return eq(x, y) ? vector<P>{x} : vector<P>{x, y};
    }
    return p;
}
```

把约束写成 `HP(p,q)`，表示有向直线 $p \to q$ 的左闭半平面。`halfPlane(lines)` 返回 `nullopt` 表示非空但无界，返回空向量表示空集，其余返回有界交的逆时针顶点；点和线段形式的有界交也会保留。可行性增量检查期望 $O(n)$、最坏 $O(n^2)$，边界构造 $O(n \log n)$，空间 $O(n)$。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

下面四组固定约束依次检查乱序矩形、同向冗余边、空集和无界半条带。

```
signed main()
{
    vector<HP> box{{{0, 1}, {0, 0}}, {{1, 0}, {1, 1}},
                   {{1, 1}, {0, 1}}, {{0, 0}, {1, 0}},
                   {{2, 0}, {2, 1}}};
    auto a = halfPlane(box);
    assert(a && eq(area(*a), 1));
    auto empty = halfPlane(vector<HP>{{{0, 0}, {0, 1}},
                                      {{1, 1}, {1, 0}}}); // x<=0 与 x>=1 无交。
    assert(empty && empty->empty());
    auto unbounded = halfPlane(vector<HP>{{{0, 1}, {0, 0}},
                                          {{1, 0}, {1, 1}},
                                          {{0, 0}, {1, 0}}});
```

```
    assert(!unbounded);
    cout << "OK\n";
}
```

11.3.010 整数半平面交

求若干整数线性不等式 `a*x+b*y+c>=0` 的公共区域。排序、删线和交点内外判定全部使用 `__int128`，适合不能接受浮点误差的凸多边形裁剪与线性约束可行域。返回边界直线和有理交点，不把非整数交点截断。把每条约束写成整数系数半平面。按边界方向极角排序，同向约束只保留更严格的一条；队列两端是否失效由三条直线系数的行列式直接判定，无需求出浮点交点。最终相邻边界的交点以两个分子和一个正分母保存，所以全程保持精确。

```
namespace integer_hpi
{
// ILine[0..2] 依次是整数半平面 a*x+b*y+c>=0 的系数 a、b、c。
using ILine = array<int, 3>;
// IRatPoint[0..2] 依次是有理点横坐标分子、纵坐标分子和公共正分母。
using IRatPoint = array<__int128, 3>;

int half(int x, int y)
{
    // x、y 是非零方向向量的坐标；返回它在极角排序中所属的半平面编号。
    return y < 0 || (y == 0 && x < 0);
}

__int128 det(const ILine &p, const ILine &q)
{
    // p、q 是两条半平面的系数；返回它们法向量的叉积。
    return (__int128)p[0] * q[1] - (__int128)p[1] * q[0];
}

bool sameDir(const ILine &p, const ILine &q)
{
    // p、q 是两条半平面；返回它们的边界方向是否相同，而不是相反。
    return det(p, q) == 0 && (__int128)p[0] * q[0] + (__int128)p[1] * q[1] > 0;
}

bool stricter(const ILine &p, const ILine &q)
{
    // p、q 必须同向；返回 p 是否比 q 的可行半平面更小。
    auto ab = [](__int128 x)
    {
        return x < 0 ? -x : x;
    };
    __int128 xp = p[0] ? ab(p[0]) : ab(p[1]);
    __int128 xq = q[0] ? ab(q[0]) : ab(q[1]);
    return (__int128)p[2] * xq < (__int128)q[2] * xp;
}

```

```cpp
bool compatible(const ILine &p, const ILine &q)
{
    // p、q 的法向量必须反向；返回两个闭半平面是否相容。
    __int128 d = (__int128)p[0] * p[0] + (__int128)p[1] * p[1];
    __int128 x = -(__int128)p[0] * p[2];
    __int128 y = -(__int128)p[1] * p[2];
    return (__int128)q[0] * x + (__int128)q[1] * y + (__int128)q[2] * d >= 0;
}

bool outside(const ILine &a, const ILine &b, const ILine &c)
{
    // a 是待检查半平面，b、c 是两条不平行边界；返回 b、c 交点是否严格落在 a 外。
    __int128 d = det(b, c);
    assert(d != 0); // 调试检查，可删。
    __int128 x = (__int128)b[1] * c[2] - (__int128)b[2] * c[1];
    __int128 y = (__int128)b[2] * c[0] - (__int128)b[0] * c[2];
    __int128 v = (__int128)a[0] * x + (__int128)a[1] * y + (__int128)a[2] * d;
    return v != 0 && ((v < 0) != (d < 0));
}

optional<vector<ILine>> halfPlane(vector<ILine> a)
{
    // a 是全部整数半平面；空 optional 表示非空但无界。
    for (auto &l : a)
    {
        assert(l[0] != 0 || l[1] != 0); // 调试检查，可删。
    }
    sort(a.begin(),
         a.end(),
         [](const ILine &p, const ILine &q)
         {
             int hp = half(p[1], -p[0]);
             int hq = half(q[1], -q[0]);
             if (hp != hq)
             {
                 return hp < hq;
             }
             auto d = det(p, q);
             if (d != 0)
             {
                 return d > 0;
             }
             if (sameDir(p, q))
             {
                 return stricter(p, q);
             }
             return false;
         });

    vector<ILine> b;
    for (auto l : a)
    {
```

```
        if (!b.empty() && sameDir(b.back(), l))
        {
            continue;
        }
        b.push_back(l);
    }

    if (b.empty())
    {
        return nullopt;
    }
    auto ab = [](__int128 x)
    {
        return x < 0 ? -x : x;
    };
    auto gcd128 = [&](__int128 x, __int128 y)
    {
        x = ab(x);
        y = ab(y);
        while (y != 0)
        {
            __int128 r = x % y;
            x = y;
            y = r;
        }
        return x;
    };
    map<pair<__int128, __int128>, ILine> dir;
    for (const ILine &l : b)
    {
        __int128 g = gcd128(l[0], l[1]);
        __int128 x = (__int128)l[0] / g;
        __int128 y = (__int128)l[1] / g;
        auto it = dir.find({-x, -y});
        if (it != dir.end() && !compatible(l, it->second))
        {
            return vector<ILine>{};
        }
        dir[{x, y}] = l;
    }
    for (int i = 0; i < (int)b.size(); i++)
    {
        if (det(b[i], b[(i + 1) % b.size()]) <= 0)
        {
            return nullopt;
        }
    }

    deque<ILine> q;
    for (auto l : b)
    {
        while (q.size() >= 2 && outside(l, q[q.size() - 2], q.back()))
```

```
        {
            q.pop_back();
        }
        while (q.size() >= 2 && outside(l, q[0], q[1]))
        {
            q.pop_front();
        }
        q.push_back(l);
    }
    while (q.size() >= 3 && outside(q.front(), q[q.size() - 2], q.back()))
    {
        q.pop_back();
    }
    while (q.size() >= 3 && outside(q.back(), q[0], q[1]))
    {
        q.pop_front();
    }
    if (q.size() < 3)
    {
        return vector<ILine>{};
    }
    return vector<ILine>{q.begin(), q.end()};
}

vector<IRatPoint> vertices(const vector<ILine> &q)
{
    // q 是 halfPlane 返回的有界边界；返回相邻边界交点的“横分子、纵分子、正分母”。
    vector<IRatPoint> p;
    auto ab = [](__int128 z)
    {
        return z < 0 ? -z : z;
    };
    auto gcd128 = [&](__int128 u, __int128 v)
    {
        u = ab(u);
        v = ab(v);
        while (v != 0)
        {
            __int128 r = u % v;
            u = v;
            v = r;
        }
        return u;
    };
    for (int i = 0; i < (int)q.size(); i++)
    {
        auto a = q[i];
        auto b = q[(i + 1) % q.size()];
        __int128 d = det(a, b);
        __int128 x = (__int128)a[1] * b[2] - (__int128)a[2] * b[1];
        __int128 y = (__int128)a[2] * b[0] - (__int128)a[0] * b[2];
        if (d < 0)
```

```
        {
            x = -x;
            y = -y;
            d = -d;
        }
        __int128 g = gcd128(gcd128(x, y), d);
        x /= g;
        y /= g;
        d /= g;
        IRatPoint z{x, y, d};
        if (p.empty() || p.back() != z)
        {
            p.push_back(z);
        }
    }
    if (p.size() > 1 && p.front() == p.back())
    {
        p.pop_back();
    }
    return p;
}
}
```

在 `integer_hpi` 命名空间中，把约束写成 `ILine{a,b,c}`，表示 `a*x+b*y+c>=0`。`halfPlane(lines)` 返回 `nullopt` 表示非空但无界，返回空向量表示空集，其余返回逆时针边界；再用 `vertices(*q)` 得到约分后的有理交点，点和线段形式的有界交会自动删除相邻重复点。系数三次乘积必须落在 `__int128` 范围内。$O(n \log n)$ 时间、$O(n)$ 空间；其中 n 为半平面数，排序后每条边至多入队和出队一次。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。可据此手算自查：四条约束 `{1,0,0}`、`{0,1,0}`、`{-1,0,2}`、`{0,-1,3}` 表示矩形 `[0,2]x[0,3]`；期望返回 4 条边，四个有理顶点分母均为 1，面积为 6。再加入 `{1,0,-4}` 后交集为空；只保留前三条时交集无界。

11.3.011 Delaunay 三角剖分 / Voronoi 图

分治合并左右两侧的 Delaunay 图。每个点的出边按旋转顺序组成双向循环链，合并时从共同下切线向上扫描；若候选边的对点落入外接圆，就立刻删除该边，再连接新的跨区边。这个结构既给出线性条数的平面边，也保留了恢复三角面和 Voronoi 对偶所需的嵌入顺序。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
using IP = Point<int>;
using IW = __int128_t;

struct DelaunayG
{
    struct Edge
    {
        int to = 0;           // to 保存有向边终点。
        int ccw = 0, cw = 0; // ccw、cw 是同起点边的逆时针、顺时针相邻边。
```

```
        int rev = 0;          // rev 保存反向边编号。
        bool on = false;      // on 表示边当前是否存在。
    };

    vector<IP> p;       // p 是按 x、y 排序且去重后的点。
    vector<Edge> e;     // e 保存可回收的旋转边结构。
    vector<int> free;   // free 保存已删除边的空闲编号。

    DelaunayG(vector<IP> p) : p(move(p))
    {
        // p 必须按字典序严格递增；构造 Delaunay 平面图。
        if (this->p.size() >= 2)
        {
            build(0, this->p.size());
        }
    }

    static bool inCircle(IP a, IP b, IP c, IP d)
    {
        // a、b、c、d 是整数点；返回 d 是否严格位于有向圆 abc 内。
        a -= d, b -= d, c -= d;
        auto cr = [](IP x, IP y)
        {
            return (IW)x.x * y.y - (IW)x.y * y.x;
        };
        auto sq = [](IP x)
        {
            return (IW)x.x * x.x + (IW)x.y * x.y;
        };
        IW z = cr(b, c) * sq(a) + cr(c, a) * sq(b) + cr(a, b) * sq(c);
        return z > 0;
    }

    int turn(int a, int b, int c) const
    {
        // a、b、c 是点下标；返回三点的精确转向。
        auto z = cross(p[b] - p[a], p[c] - p[a]);
        return (z > 0) - (z < 0);
    }

    int slot()
    {
        // 无参数；返回一个可写边编号。
        if (free.empty())
        {
            e.push_back({});
            return (int)e.size() - 1;
        }
        int x = free.back();
        free.pop_back();
        return x;
    }
```

```
64
65      pair<int, int> makeEdge(int u, int v)
66      {
67          // u、v 是端点；建立一对反向边并返回编号。
68          int a = slot(), b = slot();
69          e[a] = {v, a, a, b, true};
70          e[b] = {u, b, b, a, true};
71          return {a, b};
72      }
73
74      void eraseOne(int x)
75      {
76          // x 是有向边；从其起点的旋转链中删除它。
77          e[e[x].ccw].cw = e[x].cw;
78          e[e[x].cw].ccw = e[x].ccw;
79          e[x].on = false;
80      }
81
82      void eraseEdge(int x)
83      {
84          // x 是有向边；删除它及其反向边并回收编号。
85          int y = e[x].rev;
86          eraseOne(x);
87          eraseOne(y);
88          free.push_back(x);
89          free.push_back(y);
90      }
91
92      void insertCcw(int x, int at)
93      {
94          // x 是新边，at 是同起点边；把 x 插到 at 的逆时针一侧。
95          int y = e[at].ccw;
96          e[x].ccw = y, e[y].cw = x;
97          e[x].cw = at, e[at].ccw = x;
98      }
99
100     void insertCw(int x, int at)
101     {
102         // x 是新边，at 是同起点边；把 x 插到 at 的顺时针一侧。
103         int y = e[at].cw;
104         e[x].cw = y, e[y].ccw = x;
105         e[x].ccw = at, e[at].cw = x;
106     }
107
108     pair<int, int> next(int x) const
109     {
110         // x 是当前出边；返回逆时针相邻点及对应出边。
111         int v = e[x].to;
112         return {v, e[e[x].rev].ccw};
113     }
114
115     pair<int, int> prev(int x) const
```

```
    {
        // x 是当前出边；返回顺时针相邻点及对应出边。
        int y = e[x].cw;
        return {e[y].to, e[y].rev};
    }

    tuple<int, int, int, int> bottom(int a, int ea, int b, int eb) const
    {
        // a、b 在两个凸壳上，ea、eb 是出边；返回共同下切线状态。
        while (true)
        {
            auto [x, ex] = prev(ea);
            if (turn(b, a, x) > 0)
            {
                a = x, ea = ex;
                continue;
            }
            auto [y, ey] = next(eb);
            if (turn(a, b, y) < 0)
            {
                b = y, eb = ey;
                continue;
            }
            return {a, ea, b, eb};
        }
    }

    pair<int, int> extreme(int a, int ea, bool low) const
    {
        // a、ea 是凸壳入口，low 控制取最小或最大编号；返回极值点状态。
        pair<int, int> ans{a, ea};
        int u = a, x = ea;
        do
        {
            tie(u, x) = next(x);
            ans = low ? min(ans, pair{u, x}) : max(ans, pair{u, x});
        } while (x != ea);
        return ans;
    }

    pair<int, int> join(int a, int ea, int b, int eb)
    {
        // a、b 是左右 Delaunay 图入口；合并两侧并返回新外壳入口。
        tie(a, ea) = extreme(a, ea, false);
        tie(b, eb) = extreme(b, eb, true);
        auto [al, eal, bl, ebl] = bottom(a, ea, b, eb);
        auto [bu, ebu, au, eau] = bottom(b, eb, a, ea);
        ebl = e[ebl].cw;
        ebu = e[ebu].cw;
        auto [ab, ba] = makeEdge(al, bl);
        insertCw(ab, eal);
        insertCcw(ba, ebl);
```

```
        if (al == au)
        {
            eau = ab;
        }
        if (bl == bu)
        {
            ebu = ba;
        }
        int ap = al, eap = eal, bp = bl, ebp = ebl;
        while (ap ≠ au || bp ≠ bu)
        {
            int a2 = e[eap].to, b2 = e[ebp].to;
            int na = e[eap].ccw, nb = e[ebp].cw;
            if (eap ≠ eau && na ≠ ab)
            {
                int a1 = e[na].to;
                if (inCircle(p[ap], p[bp], p[a2], p[a1]))
                {
                    eraseEdge(eap);
                    eap = na;
                    continue;
                }
            }
            if (ebp ≠ ebu && nb ≠ ba)
            {
                int b1 = e[nb].to;
                if (inCircle(p[b2], p[ap], p[bp], p[b1]))
                {
                    eraseEdge(ebp);
                    ebp = nb;
                    continue;
                }
            }
            bool takeA = ebp == ebu;
            if (eap ≠ eau && ebp ≠ ebu)
            {
                if (turn(ap, bp, b2) < 0)
                {
                    takeA = true;
                }
                else if (turn(a2, ap, bp) < 0)
                {
                    takeA = false;
                }
                else
                {
                    takeA = inCircle(p[ap], p[bp], p[b2], p[a2]);
                }
            }
            if (takeA)
            {
                na = e[e[eap].rev].ccw;
```

```
                auto [x, y] = makeEdge(a2, bp);
                insertCw(x, na);
                insertCcw(y, ebp);
                eap = na, ap = a2;
            }
            else
            {
                nb = e[e[ebp].rev].cw;
                auto [x, y] = makeEdge(b2, ap);
                insertCcw(x, nb);
                insertCw(y, eap);
                ebp = nb, bp = b2;
            }
        }
        return {al, ab};
    }

    pair<int, int> build(int l, int r)
    {
        // l、r 是有序点的半开区间；递归构造并返回外壳入口。
        if (r - l == 2)
        {
            auto [x, y] = makeEdge(l, l + 1);
            return {l, x};
        }
        if (r - l == 3)
        {
            int u = l, v = l + 1, w = l + 2;
            auto [uv, vu] = makeEdge(u, v);
            auto [vw, wv] = makeEdge(v, w);
            int z = turn(u, v, w);
            if (!z)
            {
                insertCcw(vu, vw);
                return {u, uv};
            }
            auto [uw, wu] = makeEdge(u, w);
            if (z > 0)
            {
                insertCw(uv, uw), insertCw(vw, vu), insertCw(wu, wv);
                return {u, uv};
            }
            insertCcw(uv, uw), insertCcw(vw, vu), insertCcw(wu, wv);
            return {v, vu};
        }
        int m = (l + r) / 2;
        auto [a, ea] = build(l, m);
        auto [b, eb] = build(m, r);
        return join(a, ea, b, eb);
    }

    vector<pair<int, int>> edges() const
```

```
    {
        // 无参数；返回每条当前无向边一次。
        vector<pair<int, int>> ans;
        for (int i = 0; i < (int)e.size(); i++)
        {
            if (e[i].on && i > e[i].rev)
            {
                ans.push_back(minmax(e[i].to, e[e[i].rev].to));
            }
        }
        sort(ans.begin(), ans.end());
        return ans;
    }
};

struct DelaunayRes
{
    struct VorEdge
    {
        P a;     // a 是线段端点或射线、直线上的一点。
        P b;     // type=0 时 b 是另一端点，否则 b 是方向向量。
        int type; // type 为 0、1、2 时分别表示线段、射线、直线。
    };

    vector<pair<int, int>> edge; // edge 保存 Delaunay 无向边原下标。
    vector<array<int, 3>> tri;   // tri 保存逆时针三角面原下标。
    vector<P> center;            // center 保存与 tri 同序的外心。
    vector<VorEdge> vor;         // vor 保存完整 Voronoi 边。
};

DelaunayRes delaunay(const vector<IP> &input)
{
    // input 是整数点集；返回 Delaunay 三角剖分及其 Voronoi 对偶。
    vector<int> ord(input.size());
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(), ord.end(), [&](int x, int y)
    {
        return input[x] != input[y] ? input[x] < input[y] : x < y;
    });
    vector<IP> p;
    vector<int> id;
    for (int x : ord)
    {
        if (p.empty() || p.back() != input[x])
        {
            p.push_back(input[x]);
            id.push_back(x);
        }
    }
    DelaunayRes ans;
    if (p.size() < 2)
    {
```

```
        return ans;
    }
    DelaunayG g(p);
    auto es = g.edges();
    int n = p.size();
    vector<vector<int>> adj(n);
    for (auto [u, v] : es)
    {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    auto halfInt = [](IP x)
    {
        return x.y < 0 || (x.y == 0 && x.x < 0);
    };
    map<pair<int, int>, int> pos;
    for (int u = 0; u < n; u++)
    {
        sort(adj[u].begin(), adj[u].end(), [&](int x, int y)
        {
            IP a = p[x] - p[u], b = p[y] - p[u];
            int ha = halfInt(a), hb = halfInt(b);
            auto z = cross(a, b);
            return ha != hb ? ha < hb : (z ? z > 0 : norm(a) < norm(b));
        });
        for (int i = 0; i < (int)adj[u].size(); i++)
        {
            pos[{u, adj[u][i]}] = i;
        }
    }
    set<pair<int, int>> used;
    vector<array<int, 3>> tri;
    for (auto [s, t] : es)
    {
        for (auto [u0, v0] : {pair{s, t}, pair{t, s}})
        {
            if (used.contains({u0, v0}))
            {
                continue;
            }
            vector<int> face;
            int u = u0, v = v0;
            while (!used.contains({u, v}))
            {
                used.insert({u, v});
                face.push_back(u);
                int k = pos[{v, u}];
                int w = adj[v][(k - 1 + adj[v].size()) % adj[v].size()];
                u = v, v = w;
            }
            IW area = 0;
            for (int i = 0; i < (int)face.size(); i++)
```

```
            {
                area += cross(p[face[i]], p[face[(i + 1) % face.size()]]);
            }
            if (area > 0)
            {
                assert(face.size() == 3); // 调试检查，可删
                tri.push_back({face[0], face[1], face[2]});
            }
        }
    }
    map<pair<int, int>, vector<int>> side;
    for (auto e : es)
    {
        side[e];
        ans.edge.push_back(minmax(id[e.first], id[e.second]));
    }
    for (int i = 0; i < (int)tri.size(); i++)
    {
        auto x = tri[i];
        for (int k = 0; k < 3; k++)
        {
            side[minmax(x[k], x[(k + 1) % 3])].push_back(i);
        }
        P a = (P)p[x[0]], u = (P)(p[x[1]] - p[x[0]]), v = (P)(p[x[2]] - p[x[0]]);
        P c = a + (rot(u) * norm(v) - rot(v) * norm(u)) / (2 * cross(u, v));
        ans.center.push_back(c);
        ans.tri.push_back({id[x[0]], id[x[1]], id[x[2]]});
    }
    for (auto [e, f] : side)
    {
        if (f.size() == 2)
        {
            ans.vor.push_back({ans.center[f[0]], ans.center[f[1]], 0});
        }
        else if (f.size() == 1)
        {
            int u = e.first, v = e.second;
            auto x = tri[f[0]];
            bool forward = false;
            for (int k = 0; k < 3; k++)
            {
                forward |= x[k] == u && x[(k + 1) % 3] == v;
            }
            if (!forward)
            {
                swap(u, v);
            }
            P d{(Real)(p[v].y - p[u].y), (Real)(p[u].x - p[v].x)};
            ans.vor.push_back({ans.center[f[0]], d, 1});
        }
        else
        {
```

```
            P a = (P)p[e.first], b = (P)p[e.second];
            ans.vor.push_back({(a + b) / 2, rot(b - a), 2});
        }
    }
    sort(ans.edge.begin(), ans.edge.end());
    return ans;
}
```

`delaunay(p)` 要求整数坐标，返回 Delaunay 边、逆时针三角形、外心和 Voronoi 对偶；`VorEdge` 的 `type=0/1/2` 分别表示线段、射线和直线，后两者的 `b` 是方向向量。重复点只保留首次出现者；全部共线时三角形为空，Delaunay 边是相邻点链，Voronoi 输出相邻点中垂线。分治构造 $O(n \log n)$ 时间、$O(n)$ 边空间，圆内判定要求 $12D^4 < 2^{127}$，其中 D 是任意两坐标之差的最大绝对值。共圆时 Delaunay 对角线不唯一，本实现确定性选择其中一套。

```
signed main()
{
    vector<IP> p{{0, 0}, {2, 0}, {2, 2}, {0, 2}, {1, 1}};
    auto d = delaunay(p);
    assert(d.tri.size() == 4 && d.edge.size() == 8 && d.vor.size() == 8);
}
```

11.3.012 三角形四心（内心、外心、垂心、费马点）

汇总三角形内心、外心、垂心与费马点。前三者复用 11.1.012，费马点按最大角是否达到 120 度分情况。内心按三条对边长度加权；外心解两条等距线性方程；垂心使用 $H = A + B + C - 2O$；若某内角不小于 120°，费马点就是该顶点，否则连接向外构造的两个等边三角形顶点求交。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
P fermatPoint(P a, P b, P c)
{
    // a、b、c 表示非共线三角形顶点；返回到三顶点距离和最小的费马点。
    assert(sgn(cross(a, b, c)) != 0); // 调试检查，可删。
    const Real pi = acosl(-1);
    auto ang = [&](P x, P y, P z)
    {
        // x 表示角顶点，y、z 表示另外两点；返回夹角弧度。
        auto u = y - x;
        auto v = z - x;
        auto q = dot(u, v) / abs(u) / abs(v);
        return acosl(clamp(q, (Real)-1, (Real)1));
    };
    auto A = ang(a, b, c);
    auto B = ang(b, c, a);
    auto C = ang(c, a, b);
    if (sgn(A - 2 * pi / 3) >= 0)
    {
        return a;
    }
```

```
    if (sgn(B - 2 * pi / 3) >= 0)
    {
        return b;
    }
    if (sgn(C - 2 * pi / 3) >= 0)
    {
        return c;
    }
    auto x = abs(b - c) / sinl(A + pi / 3);
    auto y = abs(c - a) / sinl(B + pi / 3);
    auto z = abs(a - b) / sinl(C + pi / 3);
    return (a * x + b * y + c * z) / (x + y + z);
}
```

内心、外心、垂心调用 11.1.012 的 `triCenters`。费马点若某角不小于 120 度就是该顶点；否则先取三线坐标 $\csc(A+60^\circ):\csc(B+60^\circ):\csc(C+60^\circ)$，再分别乘对边长转换成重心坐标。$O(1)$ 时间和 $O(1)$ 额外空间。静态建包使用 11.3.001 Andrew / Graham 静态凸包，只插点且需要在线维护才考虑 11.3.002 单点插入动态凸包。最远点对使用 11.3.006 旋转卡壳、凸包直径，平移碰撞与凸多边形距离可转为 11.3.008 闵可夫斯基和。

11.3.013 凸包分层

凸包分层反复删除当前点集的全部凸包边界点。若每层重新 Andrew，最坏可达 $O(n^2\log n)$。下面先按横坐标把点放进完全二叉树，每个结点保存左右儿子上凸壳的公切桥；删除一个点后只重算其 $O(\log n)$ 个祖先，每次找新桥再花 $O(\log n)$。同时维护原点集的上凸壳与取反点集的上凸壳，就能拼出每一层的逆时针边界。

```
// 依赖 11.1.002 Point / Vector
struct UpperHull
{
    struct Node
    {
        int l = -1, r = -1, bl = -1, br = -1; // l、r 是子树最左存活点与最右存活点加一（r=-1 表示子
        ↪ 树为空），bl、br 是左右儿子上凸壳之间公切桥的端点。
    };

    int n, sz; // n 是点数，sz 是补齐到 2 的幂的叶子数。
    vector<Point<long long>> p; // p 是按坐标排序后的点。
    vector<Node> tr; // tr 是按堆式编号的线段树结点。
    vector<int> orig, pos;

    UpperHull(vector<Point<long long>> a) : n(a.size()), p(move(a))
    {
        // a 是互异整数点；建立包含全部点的动态上凸壳。
        orig.resize(n);
        iota(orig.begin(), orig.end(), 0);
        sort(orig.begin(), orig.end(), [&](int x, int y) { return p[x] < p[y]; });
        vector<Point<long long>> q(n);
        pos.resize(n);
```

```
        for (int i = 0; i < n; i++)
        {
            q[i] = p[orig[i]];
            pos[orig[i]] = i;
        }
        p.swap(q);
        sz = 1;
        while (sz < n)
        {
            sz <<= 1;
        }
        tr.resize(2 * sz);
        for (int i = 0; i < n; i++)
        {
            tr[sz + i] = {i, i + 1, i, i};
        }
        for (int i = sz - 1; i; i--)
        {
            pull(i);
        }
    }

    bool has(int x) const
    {
        // x 是线段树结点编号；返回该结点子树中是否还有未删除的点。
        return tr[x].r != -1;
    }

    void pull(int x)
    {
        // 这里的叉积还要与坐标相乘，局部扩为 __int128。
        auto cross128 = [](Point<long long> a, Point<long long> b)
        {
            return (__int128)a.x * b.y - (__int128)a.y * b.x;
        };
        if (!has(2 * x) && !has(2 * x + 1))
        {
            tr[x] = {};
            return;
        }
        if (!has(2 * x))
        {
            tr[x] = tr[2 * x + 1];
            return;
        }
        if (!has(2 * x + 1))
        {
            tr[x] = tr[2 * x];
            return;
        }
        int u = 2 * x, v = 2 * x + 1;
        long long split = p[tr[v].l].x;
```

```
        while (u < sz || v < sz)
        {
            if (u < sz && !has(2 * u))
            {
                u = 2 * u + 1;
                continue;
            }
            if (u < sz && !has(2 * u + 1))
            {
                u = 2 * u;
                continue;
            }
            if (v < sz && !has(2 * v))
            {
                v = 2 * v + 1;
                continue;
            }
            if (v < sz && !has(2 * v + 1))
            {
                v = 2 * v;
                continue;
            }
            int a = tr[u].bl, b = tr[u].br, c = tr[v].bl, d = tr[v].br;
            if (a != b && cross128(p[b] - p[a], p[c] - p[a]) > 0)
            {
                u = 2 * u;
            }
            else if (c != d && cross128(p[c] - p[b], p[d] - p[b]) > 0)
            {
                v = 2 * v + 1;
            }
            else if (a == b)
            {
                v = 2 * v;
            }
            else if (c == d)
            {
                u = 2 * u + 1;
            }
            else
            {
                __int128 x1 = cross128(p[b] - p[a], p[c] - p[a]);
                __int128 x2 = cross128(p[a] - p[b], p[d] - p[b]);
                if (x1 + x2 == 0 || x1 * p[d].x + x2 * p[c].x < (__int128)split * (x1 + x2))
                {
                    u = 2 * u + 1;
                }
                else
                {
                    v = 2 * v;
                }
            }
```

```
        }
        tr[x].l = tr[2 * x].l;
        tr[x].r = tr[2 * x + 1].r;
        tr[x].bl = tr[u].l;
        tr[x].br = tr[v].l;
    }

    void erase(int id)
    {
        // id 是原输入编号；从当前点集中删除该点。
        int x = sz + pos[id];
        tr[x] = {};
        for (x >>= 1; x; x >>= 1)
        {
            pull(x);
        }
    }

    vector<int> get() const
    {
        // 无参数；返回当前上凸壳从左到右的原输入编号。
        vector<int> ans;
        auto dfs = [&](auto &&self, int x, int l, int r) -> void
        {
            if (!has(x) || l >= r)
            {
                return;
            }
            if (x >= sz)
            {
                ans.push_back(tr[x].l);
                return;
            }
            if (!has(2 * x))
            {
                self(self, 2 * x + 1, l, r);
                return;
            }
            if (!has(2 * x + 1))
            {
                self(self, 2 * x, l, r);
                return;
            }
            if (r <= tr[x].bl)
            {
                self(self, 2 * x, l, r);
                return;
            }
            if (tr[x].br <= l)
            {
                self(self, 2 * x + 1, l, r);
                return;
```

```
            }
            self(self, 2 * x, l, tr[x].bl + 1);
            self(self, 2 * x + 1, tr[x].br, r);
        };
        if (n)
        {
            dfs(dfs, 1, 0, n);
        }
        for (auto &x : ans)
        {
            x = orig[x];
        }
        return ans;
    }
};

vector<vector<int>> convexLayers(const vector<Point<long long>> &p)
{
    // p 是互异且坐标绝对值不超过 1e9 的整数点；返回各层逆时针点编号。
    if (p.empty())
    {
        return {};
    }
    vector<Point<long long>> q;
    q.reserve(p.size());
    for (auto [x, y] : p)
    {
        assert(llabs(x) <= 1000000000 && llabs(y) <= 1000000000); // 调试检查，可删。
        q.push_back({-x, -y});
    }
    UpperHull up(p), down(q);
    vector<vector<int>> ans;
    int done = 0;
    while (done < (int)p.size())
    {
        auto a = down.get();
        auto b = up.get();
        if (a.size() == p.size() - done && b.size() == a.size())
        {
            // 剩余点全共线时两条壳相同，只保留其中一条。
        }
        else if (a.size() >= 2)
        {
            a.pop_back();
            b.pop_back();
            a.insert(a.end(), b.begin(), b.end());
            reverse(a.begin(), a.end());
        }
        ans.push_back(a);
        for (int x : a)
        {
            up.erase(x);
```

```
            down.erase(x);
            done++;
        }
    }
    return ans;
}
```

`convexLayers(p)` 返回每层原输入编号；同一凸包边上的全部共线点属于该层，点必须互异且坐标绝对值不超过 10^9。预处理 $O(n \log n)$，每次删除 $O(\log^2 n)$，总时间 $O(n \log^2 n)$、空间 $O(n)$。只求一层时直接用 11.3.001 Andrew / Graham 静态凸包，其代码更短。

下面用两层正方形和中心点检查层号；每层顶点在结果内的循环起点不作约定：

```
signed main()
{
    vector<Point<long long>> p{{0, 0}, {4, 0}, {4, 4}, {0, 4},
                               {1, 1}, {3, 1}, {3, 3}, {1, 3}, {2, 2}};
    auto lay = convexLayers(p);
    assert(lay.size() == 3);
    vector<int> bel(p.size());
    for (int i = 0; i < (int)lay.size(); i++)
    {
        for (int x : lay[i])
        {
            bel[x] = i + 1;
        }
    }
    assert(bel == vector<int>({1, 1, 1, 1, 2, 2, 2, 2, 3}));
}
```

11.3.014 点集最小 / 最大面积三角形

在同一点集中同时求不同下标三点的最小、最大双倍面积 $\left|\operatorname{cross}(b-a, c-a)\right|$。整数版扫描所有无向点对的方向：点按该方向法向投影排序，同一直线上的连续块在越过这一方向时整体反转；对一条基边，块外相邻点给出最小非零候选，排列两端给出最大候选，同方向事件必须整批处理。重复坐标或三点共线会使最小值为 0。浮点事件难以可靠判同向，因而单独保留直接枚举版本。

```
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
optional<pair<__int128, __int128>> triAreaRng2(const vector<Point<long long>> &a)
{
    // a 是整数点集；返回不同下标三点的最小、最大双倍面积，少于三个点时返回空。
    // 坐标绝对值不超过 1E9，以保证方向排序和面积乘积可由 __int128 表示。
    // 坐标差之间的叉积必须能由 __int128 表示。
    int n = (int)a.size();
    if (n < 3)
    {
        return nullopt;
    }
```

```
    vector<Point<long long>> p = a;
    sort(p.begin(), p.end());
    auto last = unique(p.begin(), p.end());
    bool dup = last != p.end();
    p.erase(last, p.end());
    if (p.size() < 3)
    {
        return pair<__int128, __int128>{0, 0};
    }
    int m = (int)p.size();
    struct Event
    {
        int u;      // 有向差向量的起点下标。
        int v;      // 有向差向量的终点下标。
        __int128 x; // 规范到上半平面后的横坐标差。
        __int128 y; // 规范到上半平面后的纵坐标差。
    };
    vector<Event> e;
    for (int i = 0; i < m; i++)
    {
        for (int j = i + 1; j < m; j++)
        {
            __int128 x = (__int128)p[j].x - p[i].x;
            __int128 y = (__int128)p[j].y - p[i].y;
            if (y < 0 || (y == 0 && x < 0))
            {
                x = -x;
                y = -y;
            }
            e.push_back({i, j, x, y});
        }
    }
    sort(e.begin(),
         e.end(),
         [](const Event &u, const Event &v)
         {
             __int128 c = u.x * v.y - u.y * v.x;
             if (c != 0)
             {
                 return c > 0;
             }
             return pair(u.u, u.v) < pair(v.u, v.v);
         });
    vector<int> ord(m), pos(m);
    iota(ord.begin(), ord.end(), 0);
    sort(ord.begin(),
         ord.end(),
         [&](int x, int y)
         {
             return tuple(p[x].y, p[x].x, x) < tuple(p[y].y, p[y].x, y);
         });
    for (int i = 0; i < m; i++)
```

```
    {
        pos[ord[i]] = i;
    }
    auto triArea = [&](int i, int j, int k)
    {
        __int128 x1 = (__int128)p[j].x - p[i].x;
        __int128 y1 = (__int128)p[j].y - p[i].y;
        __int128 x2 = (__int128)p[k].x - p[i].x;
        __int128 y2 = (__int128)p[k].y - p[i].y;
        __int128 s = x1 * y2 - y1 * x2;
        return s < 0 ? -s : s;
    };
    __int128 mn = 0;
    __int128 mx = 0;
    bool hasMin = dup;
    vector<int> fa(m, -1);
    auto find = [&](auto &&self, int x) -> int
    {
        return fa[x] == x ? x : fa[x] = self(self, fa[x]);
    };
    for (int l = 0; l < (int)e.size();)
    {
        int r = l + 1;
        while (r < (int)e.size() && e[l].x * e[r].y == e[l].y * e[r].x)
        {
            r++;
        }
        vector<int> touched;
        for (int i = l; i < r; i++)
        {
            for (int x : {e[i].u, e[i].v})
            {
                if (fa[x] == -1)
                {
                    fa[x] = x;
                    touched.push_back(x);
                }
            }
            int x = find(find, e[i].u);
            int y = find(find, e[i].v);
            if (x != y)
            {
                fa[x] = y;
            }
        }
        map<int, vector<int>> block;
        for (int x : touched)
        {
            block[find(find, x)].push_back(x);
        }
        for (int i = l; i < r; i++)
        {
```

```
            for (int x : {ord.front(), ord.back()})
            {
                if (x != e[i].u && x != e[i].v)
                {
                    mx = max(mx, triArea(e[i].u, e[i].v, x));
                }
            }
        }
        for (auto &[root, v] : block)
        {
            int x = m;
            int y = -1;
            for (int z : v)
            {
                x = min(x, pos[z]);
                y = max(y, pos[z]);
            }
            assert(y - x + 1 == (int)v.size()); // 调试检查，可删。
            if (v.size() >= 3)
            {
                mn = 0;
                hasMin = true;
            }
            else if (!hasMin || mn != 0)
            {
                int u = v[0];
                int z = v[1];
                for (int k : {x - 1, y + 1})
                {
                    if (0 <= k && k < m)
                    {
                        __int128 cur = triArea(u, z, ord[k]);
                        if (!hasMin || cur < mn)
                        {
                            mn = cur;
                            hasMin = true;
                        }
                    }
                }
            }
            reverse(ord.begin() + x, ord.begin() + y + 1);
            for (int k = x; k <= y; k++)
            {
                pos[ord[k]] = k;
            }
        }
        for (int x : touched)
        {
            fa[x] = -1;
        }
        l = r;
    }
```

```
    assert(hasMin); // 调试检查，可删。
    return pair<__int128, __int128>{mn, mx};
}

optional<pair<Real, Real>> triAreaRng2(const vector<P> &a)
{
    // a 是浮点点集；返回不同下标三点的最小、最大双倍面积，少于三个点时返回空。
    int n = (int)a.size();
    if (n < 3)
    {
        return nullopt;
    }
    Real mn = numeric_limits<Real>::infinity();
    Real mx = 0;
    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            for (int k = j + 1; k < n; k++)
            {
                Real s = fabsl(cross(a[i], a[j], a[k]));
                if (sgn(s) == 0)
                {
                    s = 0;
                }
                mn = min(mn, s);
                mx = max(mx, s);
            }
        }
    }
    return pair<Real, Real>{mn, mx};
}
```

整数重载接收 `vector<Point<long long>>` 并返回 `pair<__int128,__int128>`；坐标绝对值须不超过 10^9，少于三个下标返回 `nullopt`，重复下标坐标仍可同时选入三角形。其时间 $O(n^2 \log n)$、空间 $O(n^2)$。`Point<Real>` 重载返回 `pair<Real,Real>`，使用 $O(n^3)$ 时间和 $O(1)$ 额外空间。只求凸包最大三角形且坐标类型范围足够时可用 11.3.007 最大面积三角形、四边形；需要宽整数结果时使用本条。

11.4 圆

11.4.001 点、线、圆的位置关系

判断点与圆、直线与圆、两圆的相离、相切、相交或包含关系，为交点、切线和面积公式选择正确分支。点与圆比较距离平方和半径平方，直线与圆比较圆心到直线距离；两圆关系由圆心距与半径和、半径差比较，重合需单独状态。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
struct C
{
```

```
    P o; // o 保存圆心。
    Real r; // r 保存非负半径。
};

bool in(C c, P p)
{
    // c 是圆，p 是查询点；返回 p 是否在圆内或圆上。
    assert(sgn(c.r) >= 0); // 调试检查，可删。
    return sgn(abs(p - c.o) - c.r) <= 0;
}

// 0 相离，1 外切，2 相交，3 内切，4 内含，5 重合。
int circleRel(C a, C b)
{
    // a、b 是圆；返回二者的位置关系编号。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(a.o - b.o);
    if (!sgn(d) && !sgn(a.r - b.r))
    {
        return 5;
    }
    int x = sgn(d - a.r - b.r);
    int y = sgn(d - fabsl(a.r - b.r));
    if (x > 0)
    {
        return 0;
    }
    if (!x)
    {
        return 1;
    }
    if (y > 0)
    {
        return 2;
    }
    if (!y)
    {
        return 3;
    }
    return 4;
}
```

`C{o,r}` 保存圆心与半径；`in(c,p)` 判断点在圆内或圆上，`circleRel(a,b)` 返回编码 0 至 5，依次表示相离、外切、相交、内切、内含、重合。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.002 直线与圆、两圆交点

求直线与圆、两圆的全部交点。圆的基础类型和位置关系在 11.4.001 定义，本项只打印交点算法。直线圆交先把圆心投影到直线，再沿单位方向偏移弦半长；两圆交先在圆心连线上求公共弦中点，再沿垂线偏移。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.007 直线交点、投影、对称点
// 依赖 11.4.001 点、线、圆的位置关系
vector<P> lineCircle(L l, C c)
{
    // l 是直线，c 是圆；返回按直线参数排列的交点。
    assert(sgn(norm(l.v)) != 0 && sgn(c.r) >= 0); // 调试检查，可删。
    P h = proj(l, c.o);
    Real d = norm(h - c.o);
    if (sgn(d - c.r * c.r) > 0)
    {
        return {};
    }
    Real t = sqrtl(max(Real(0), c.r * c.r - d)) / abs(l.v);
    if (!sgn(t))
    {
        return {h};
    }
    return {h - l.v * t, h + l.v * t};
}

vector<P> circleIsect(C a, C b)
{
    // a、b 是圆；返回有限的公共点。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(b.o - a.o);
    if (sgn(d) == 0)
    {
        if (sgn(a.r) == 0 && sgn(b.r) == 0)
        {
            return {a.o};
        }
        return {};
    }
    if (sgn(d - a.r - b.r) > 0 || sgn(d - fabsl(a.r - b.r)) < 0)
    {
        return {};
    }
    Real x = (a.r * a.r - b.r * b.r + d * d) / (2 * d);
    Real h = sqrtl(max(Real(0), a.r * a.r - x * x));
    P v = (b.o - a.o) / d;
    P p = a.o + v * x;
    if (!sgn(h))
    {
        return {p};
    }
```

```
    return {p + rot(v) * h, p - rot(v) * h};
}
```

`lineCircle(l,c)` 与 `circleIsect(a,b)` 返回 0 至 2 个交点。重合的正半径圆有无穷多交点，以空结果表示并由 11.4.001 预先分类；重合的两个零半径圆只有一个公共点，返回该圆心。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.003 两圆相交面积

计算两圆公共部分面积。根据圆心距先处理相离与包含，再用两个扇形减三角形。两圆无交、内含时面积直接为零或小圆面积；相交时分别由余弦定理求圆心角，两段扇形面积减对应三角形面积后相加。

```
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
Real circleArea(C a, C b)
{
    // a、b 是圆；返回二者公共面积。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(a.o - b.o);
    if (sgn(d - a.r - b.r) >= 0)
    {
        return 0;
    }
    if (sgn(d - fabsl(a.r - b.r)) <= 0)
    {
        Real r = min(a.r, b.r);
        return acosl(-1) * r * r;
    }
    Real x = acosl(clamp((a.r * a.r + d * d - b.r * b.r) / (2 * a.r * d), Real(-1), Real(1)));
    Real y = acosl(clamp((b.r * b.r + d * d - a.r * a.r) / (2 * b.r * d), Real(-1), Real(1)));
    return a.r * a.r * x + b.r * b.r * y - a.r * d * sinl(x);
}
```

调用 `circleArea(a,b)` 返回两圆公共面积；半径必须非负。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.004 圆的切线与两圆公切线

求外点到圆的切点以及两圆公切线。重合圆有无穷多公切线，必须单独处理。点到圆切线由直角三角形求切点方向；两圆公切线可统一处理外切和内切的有符号半径，再按圆心距与半径差判断切线条数。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.4.001 点、线、圆的位置关系
vector<P> tangents(C c, P p)
```

```
{
    // c 是圆，p 是查询点；返回不同的切点。
    assert(sgn(c.r) >= 0); // 调试检查，可删。
    P v = p - c.o;
    Real d = norm(v);
    Real z = d - c.r * c.r;
    if (sgn(d) == 0)
    {
        return {};
    }
    if (sgn(c.r) == 0)
    {
        return {c.o};
    }
    if (sgn(z) < 0)
    {
        return {};
    }
    P q = c.o + v * (c.r * c.r / d);
    if (!sgn(z))
    {
        return {q};
    }
    P w = rot(v) * (c.r * sqrtl(z) / d);
    return {q + w, q - w};
}

// 返回两圆公切线在两圆上的切点对；重合圆有无穷多条，返回空。
vector<pair<P, P>> tangents(C a, C b)
{
    // a、b 是圆；返回每条不同公切线上的两个切点。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    if (sgn(a.r) == 0)
    {
        vector<pair<P, P>> ans;
        for (P q : tangents(b, a.o))
        {
            ans.push_back({a.o, q});
        }
        return ans;
    }
    if (sgn(b.r) == 0)
    {
        vector<pair<P, P>> ans;
        for (P q : tangents(a, b.o))
        {
            ans.push_back({q, b.o});
        }
        return ans;
    }
    P d = b.o - a.o;
    Real z = norm(d);
```

```
    if (!sgn(z))
    {
        return {};
    }
    vector<pair<P, P>> ans;
    for (int s : {-1, 1})
    {
        Real r = a.r - s * b.r;
        Real h2 = z - r * r;
        if (sgn(h2) < 0)
        {
            continue;
        }
        for (int t : {-1, 1})
        {
            if (!sgn(h2) && t == 1)
            {
                continue;
            }
            P v = (d * r + rot(d) * (sqrtl(max(Real(0), h2)) * t)) / z;
            ans.push_back({a.o + v * a.r, b.o + v * (s * b.r)});
        }
    }
    return ans;
}
```

调用 `tangents(c,p)` 或 `tangents(a,b)`，分别返回不同切点和不同公切线上的接触点对；重合的正半径圆返回空表示无穷多条。零半径圆按点退化：外部查询点只返回圆心一次，一个点圆与普通圆的公切线转成点到圆的切线，两个不同点圆返回连接二点的一组接触点。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.005 三点外接圆、三角形内切圆

构造直径圆、三点外接圆和非退化三角形的内切圆。外接圆圆心由两条垂直平分线确定；三点共线时 `circle(a,b,c)` 延续旧接口，返回覆盖三点的最小直径圆。内心按对边长加权，若双倍面积为 $\left|\operatorname{cross}(b-a, c-a)\right|$、周长为 p，则内切圆半径为该双倍面积除以 p；共线或重复点没有唯一内切圆。

```
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
C circle(P a, P b)
{
    // a、b 是直径端点；返回对应的圆。
    P o = (a + b) / 2;
    return {o, abs(a - o)};
}

C circle(P a, P b, P c)
{
```

```
    // a、b、c 是三个点；共线时返回覆盖三点的最小圆。
    P u = b - a;
    P v = c - a;
    Real d = 2 * cross(u, v);
    if (sgn(d) == 0)
    {
        if (norm(a - b) >= norm(a - c) && norm(a - b) >= norm(b - c))
        {
            return circle(a, b);
        }
        if (norm(a - c) >= norm(b - c))
        {
            return circle(a, c);
        }
        return circle(b, c);
    }
    P o = a + P{norm(u) * v.y - norm(v) * u.y, u.x * norm(v) - v.x * norm(u)} / d;
    return {o, abs(a - o)};
}

optional<C> incircle(P a, P b, P c)
{
    // a、b、c 是三角形顶点；返回内切圆，三点共线或有重复点时返回空。
    Real s = fabsl(cross(b - a, c - a));
    if (sgn(s) == 0)
    {
        return nullopt;
    }
    Real x = abs(b - c);
    Real y = abs(c - a);
    Real z = abs(a - b);
    Real sum = x + y + z;
    P o = (a * x + b * y + c * z) / sum;
    return C{o, s / sum};
}
```

`circle(a,b)` 返回以两点为直径端点的圆，`circle(a,b,c)` 返回三点外接圆或共线退化下的最小覆盖圆；`incircle(a,b,c)` 返回内切圆，退化三角形返回 `nullopt`。各接口均为 $O(1)$ 时间和 $O(1)$ 额外空间。覆盖任意点集使用 11.4.006 最小圆覆盖，不要把共线退化约定当作一般最小圆覆盖算法。

11.4.006 最小圆覆盖

求覆盖全部平面点的最小圆。随机增量算法在固定边界点后退化为两点或三点定圆。若当前最小圆不包含新点 p，则新圆边界必须经过 p；继续扫描旧点，若仍不包含 q，则边界还必须经过 q；第三层同理用三点外接圆。随机顺序使期望总工作量为线性。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.4.005 三点外接圆、三角形内切圆
C minCircle(vector<P> a, int seed = 0)
```

```
{
    // a 是点集，seed 是打乱种子；返回最小覆盖圆。
    mt19937 rng(seed);
    shuffle(a.begin(), a.end(), rng);
    C c{{0, 0}, 0};
    for (int i = 0; i < (int)a.size(); i++)
    {
        if (!in(c, a[i]))
        {
            c = {a[i], 0};
            for (int j = 0; j < i; j++)
            {
                if (!in(c, a[j]))
                {
                    c = circle(a[i], a[j]);
                    for (int k = 0; k < j; k++)
                    {
                        if (!in(c, a[k]))
                        {
                            c = circle(a[i], a[j], a[k]);
                        }
                    }
                }
            }
        }
    }
    return c;
}

string minCircleBd(const vector<Point<long long>> &p,
                                        unsigned long long seed = 0x9E3779B97F4A7C15ULL)
{
    // p 是整数点集，seed 是确定性洗牌种子；返回各点是否位于最小包围圆圆周的 0/1 串。
    struct Support
    {
        int size = 0; // size 是确定当前圆的边界点数（0 到 3）。
        array<Point<long long>, 3> p; // p[0..size) 是这些边界点。
    };
    auto cross128 = [](Point<long long> a, Point<long long> b)
    {
        return (__int128)a.x * b.y - (__int128)a.y * b.x;
    };
    auto norm128 = [](Point<long long> a)
    {
        return (__int128)a.x * a.x + (__int128)a.y * a.y;
    };
    auto diameter = [&](Point<long long> a, Point<long long> b, Point<long long> x)
    {
        Point<long long> u{2 * x.x - a.x - b.x, 2 * x.y - a.y - b.y};
        __int128 lhs = norm128(u), rhs = norm128(a - b);
        return lhs < rhs ? 1 : lhs == rhs ? 0 : -1;
    };
```

```
    auto contain = [&](const Support &s, Point<long long> x)
    {
        if (s.size == 0) return -1;
        if (s.size == 1) return x == s.p[0] ? 0 : -1;
        if (s.size == 2) return diameter(s.p[0], s.p[1], x);
        auto a = s.p[0] - x, b = s.p[1] - x, c = s.p[2] - x;
        __int128 det = norm128(a) * cross128(b, c) +
                       norm128(b) * cross128(c, a) +
                       norm128(c) * cross128(a, b);
        __int128 ori = cross128(s.p[1] - s.p[0], s.p[2] - s.p[0]);
        det = ori > 0 ? det : -det;
        return det > 0 ? 1 : det == 0 ? 0 : -1;
    };
    auto reduce = [&](Support &s)
    {
        if (s.size != 3 || cross128(s.p[1] - s.p[0], s.p[2] - s.p[0]) != 0) return;
        __int128 d01 = norm128(s.p[0] - s.p[1]);
        __int128 d12 = norm128(s.p[1] - s.p[2]);
        __int128 d02 = norm128(s.p[0] - s.p[2]);
        if (d12 >= d01 && d12 >= d02)
        {
            s.p[0] = s.p[1];
            s.p[1] = s.p[2];
        }
        else if (d02 >= d01)
        {
            s.p[1] = s.p[2];
        }
        s.size = 2;
    };
    vector<Point<long long>> q = p;
    mt19937_64 rng(seed);
    shuffle(q.begin(), q.end(), rng);
    Support s;
    for (int i = 0; i < (int)q.size(); i++)
    {
        if (contain(s, q[i]) >= 0) continue;
        s = {1, {q[i], {}, {}}};
        for (int j = 0; j < i; j++)
        {
            if (contain(s, q[j]) >= 0) continue;
            s = {2, {q[i], q[j], {}}};
            for (int k = 0; k < j; k++)
            {
                if (contain(s, q[k]) >= 0) continue;
                s = {3, {q[i], q[j], q[k]}};
                reduce(s);
            }
        }
    }
    string ans(p.size(), '0');
    for (int i = 0; i < (int)p.size(); i++)
```

```
    {
        ans[i] = contain(s, p[i]) == 0 ? '1' : '0';
    }
    return ans;
}
```

调用 `minCircle(a,seed)` 返回覆盖 `a` 中全部点的浮点最小圆；空点集返回圆心 (0,0)、半径 0。`minCircleBd(p,seed)` 用 2/3 个整数支撑点和 `__int128` 行列式精确判断圆内、圆外和圆周，返回与原输入同序的 0/1 串；它避免先求浮点圆心后用误差比较，适用于要求精确输出哪些整数点在最小圆圆周上的题目。两个接口在随机排列下期望 $O(n)$、最坏 $O(n^3)$ 时间，$O(n)$ 空间；整数版要求所有坐标差及其两倍仍可由 `long long` 表示，相关平方乘叉积可由 `__int128` 表示。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

下面用单点、共线点和正方形检查零半径、直径圆与外接圆分支：

```
signed main()
{
    C a = minCircle({P{3, -2}});
    assert(abs(a.o - P{3, -2}) < eps && fabsl(a.r) < eps);

    C b = minCircle({P{-2, 0}, P{0, 0}, P{4, 0}});
    assert(abs(b.o - P{1, 0}) < eps && fabsl(b.r - 3) < eps);

    C c = minCircle({P{-1, -1}, P{-1, 1}, P{1, 1}, P{1, -1}});
    assert(abs(c.o) < eps && fabsl(c.r - sqrtl(2)) < eps);
}
```

11.4.007 固定半径圆覆盖最多点

给定半径，求一个圆最多覆盖多少输入点。枚举一对点确定的候选圆心方向，再做角区间扫描。固定一个点在覆盖圆边界上，其他可覆盖点给出圆心极角区间；对跨越 `2pi` 的区间拆分后做角度扫描，最大重叠数加固定点即答案。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
int fixedCover(const vector<P> &a, Real r)
{
    // a 是点集，r 是固定半径；返回最多覆盖点数。
    assert(sgn(r) >= 0); // 调试检查，可删。
    int ans = a.empty() ? 0 : 1;
    const Real pi = acosl(-1);
    for (int i = 0; i < (int)a.size(); i++)
    {
        vector<pair<Real, int>> e;
        int same = 1;
        for (int j = 0; j < (int)a.size(); j++)
        {
            if (i == j)
            {
```

```
                continue;
            }
            Real d = abs(a[j] - a[i]);
            if (!sgn(d))
            {
                same++;
                continue;
            }
            if (sgn(d - 2 * r) > 0)
            {
                continue;
            }
            Real x = atan2l(a[j].y - a[i].y, a[j].x - a[i].x);
            Real y = acosl(clamp(d / (2 * r), Real(-1), Real(1)));
            Real l = x - y;
            Real h = x + y;
            while (l < 0)
            {
                l += 2 * pi;
                h += 2 * pi;
            }
            while (l >= 2 * pi)
            {
                l -= 2 * pi;
                h -= 2 * pi;
            }
            if (h <= 2 * pi)
            {
                e.push_back({l, 1});
                e.push_back({h, -1});
            }
            else
            {
                e.push_back({l, 1});
                e.push_back({2 * pi, -1});
                e.push_back({0, 1});
                e.push_back({h - 2 * pi, -1});
            }
        }
        sort(e.begin(),
             e.end(),
             [](auto x, auto y)
             {
                 return x.first != y.first ? x.first < y.first : x.second > y.second;
             });
        ans = max(ans, same);
        int cur = same;
        for (auto [x, v] : e)
        {
            cur += v;
            ans = max(ans, cur);
        }
```

```
    }
    return ans;
}
```

调用 `fixedCover(p,r)`；`r` 必须非负，返回某个半径为 `r` 的圆最多能覆盖的输入点数。$O(n^2 \log n)$ 时间、$O(n)$ 空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.008 圆反演与阿波罗尼斯圆

以圆心 O、半径 $R > 0$ 的圆做反演时，点 $P \neq O$ 的像满足 $\overrightarrow{OP'} = \frac{R^2}{|OP|^2}\overrightarrow{OP}$。过 O 的直线保持为直线，不过 O 的直线变为过 O 的圆；过 O 的正半径圆变为直线，其余圆仍为圆。实现直接使用解析公式，并用 `variant<Line<Real>,C>` 表达对象类型变化；不同定点 a, b 的阿波罗尼斯轨迹约定满足 $|X-a| = k|X-b|$。

```
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
using InvertObj = variant<Line<Real>, C>;

optional<P> invert(P p, C c)
{
    // p 是待反演点，c 是正半径反演圆；返回像点，圆心没有有限像。
    assert(sgn(c.r) > 0); // 调试检查，可删。
    P v = p - c.o;
    if (sgn(norm(v)) == 0)
    {
        return nullopt;
    }
    return c.o + v * (c.r * c.r / norm(v));
}

InvertObj invertLine(Line<Real> l, C c)
{
    // l 是待反演直线，c 是正半径反演圆；返回像直线或像圆。
    assert(!eq(l.a, l.b)); // 调试检查，可删。
    assert(sgn(c.r) > 0);  // 调试检查，可删。
    P v = l.b - l.a;
    P h = l.a + v * (dot(c.o - l.a, v) / norm(v));
    P d = h - c.o;
    if (sgn(norm(d)) == 0)
    {
        return Line<Real>{c.o, c.o + v};
    }
    P q = c.o + d * (c.r * c.r / norm(d));
    return C{(c.o + q) / 2, abs(q - c.o) / 2};
}

```

```
optional<InvertObj> invertCircle(C x, C c)
{
    // x 是待反演圆，c 是正半径反演圆；返回像直线或像圆，仅反演中心这个点圆返回空。
    assert(sgn(x.r) >= 0); // 调试检查，可删。
    assert(sgn(c.r) > 0);  // 调试检查，可删。
    P v = x.o - c.o;
    Real d2 = norm(v);
    if (sgn(x.r) == 0 && sgn(d2) == 0)
    {
        return nullopt;
    }
    Real k = c.r * c.r;
    Real z = d2 - x.r * x.r;
    if (sgn(z) == 0)
    {
        P p = c.o + v * (k / (2 * d2));
        return InvertObj{Line<Real>{p, p + rot(v)}};
    }
    return InvertObj{C{c.o + v * (k / z), k * x.r / fabsl(z)}};
}

optional<C> apollonius(P a, P b, Real k)
{
    // a、b 是不同定点，k 是到 a 的距离除以到 b 的距离；返回轨迹圆，k=1 的直线轨迹返回空。
    assert(!eq(a, b) && sgn(k) >= 0); // 调试检查，可删。
    if (sgn(k - 1) == 0)
    {
        return nullopt;
    }
    Real z = 1 - k * k;
    P o = (a - b * (k * k)) / z;
    return C{o, abs(a - b) * fabsl(k / z)};
}
```

`invert(p,c)`、`invertLine(l,c)` 和 `invertCircle(x,c)` 分别反演点、直线和圆，反演圆半径必须为正。反演中心没有有限像；位于反演中心的零半径源圆返回 `nullopt`，其余点圆以零半径像圆表示。`apollonius(a,b,k)` 要求 $a \neq b$、$k \geq 0$，并按 $|X-a| = k|X-b|$ 定义比值；$k = 1$ 时轨迹为垂直平分线，因返回类型只表示圆而返回 `nullopt`。所有接口均为 $O(1)$ 时间和 $O(1)$ 额外空间。

11.4.009 圆并与多圆覆盖层面积

计算非负半径圆集的圆并面积，或同时求 “至少被 k 个圆覆盖” 的全部层面积。对每个正半径圆，把其他圆在其圆周上覆盖的部分转成角区间并扫描覆盖数；覆盖数固定的一段圆弧通过 Green 公式贡献边界积分。完全相同的圆按下标把各自整圆边界分配到相邻覆盖层，同心内含、内外切和零半径圆都不产生错误角事件。

```
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
Real circleUnion(const vector<C> &a)
```

```
{
    // a 是非负半径圆集；返回至少被一个圆覆盖的面积。
    const Real pi = acosl(-1);
    Real ans = 0;
    for (int i = 0; i < (int)a.size(); i++)
    {
        assert(sgn(a[i].r) >= 0); // 调试检查，可删。
        vector<pair<Real, int>> e;
        bool cover = false;
        for (int j = 0; j < (int)a.size(); j++)
        {
            if (i == j)
            {
                continue;
            }
            Real d = abs(a[j].o - a[i].o);
            if (sgn(d + a[i].r - a[j].r) <= 0 && (sgn(a[i].r - a[j].r) < 0 || i > j))
            {
                cover = true;
                break;
            }
            if (sgn(d - a[i].r - a[j].r) >= 0 || sgn(d - fabsl(a[i].r - a[j].r)) <= 0)
            {
                continue;
            }
            Real x = atan2l(a[j].o.y - a[i].o.y, a[j].o.x - a[i].o.x);
            Real y = acosl(clamp((d * d + a[i].r * a[i].r - a[j].r * a[j].r) / (2 * d *
↪ a[i].r), Real(-1), Real(1)));
            Real l = x - y;
            Real r = x + y;
            while (l < 0)
            {
                l += 2 * pi;
                r += 2 * pi;
            }
            while (l >= 2 * pi)
            {
                l -= 2 * pi;
                r -= 2 * pi;
            }
            e.push_back({l, 1});
            e.push_back({min(r, 2 * pi), -1});
            if (r > 2 * pi)
            {
                e.push_back({0, 1});
                e.push_back({r - 2 * pi, -1});
            }
        }
        if (cover)
        {
            continue;
        }
```

```
        e.push_back({0, 0});
        e.push_back({2 * pi, 0});
        sort(e.begin(),
             e.end(),
             [](auto x, auto y)
             {
                 return x.first == y.first ? x.first < y.first : x.second > y.second;
             });
        int cnt = 0;
        Real last = 0;
        for (auto [x, v] : e)
        {
            if (!cnt)
            {
                P p = a[i].o + P{cosl(last), sinl(last)} * a[i].r;
                P q = a[i].o + P{cosl(x), sinl(x)} * a[i].r;
                Real t = x - last;
                ans += cross(p, q) / 2 + a[i].r * a[i].r * (t - sinl(t)) / 2;
            }
            cnt += v;
            last = x;
        }
    }
    return ans;
}

vector<Real> circleCov(const vector<C> &a)
{
    // a 是非负半径圆集；返回各覆盖层面积，ans[k-1] 表示至少被 k 个圆覆盖的面积。
    int n = (int)a.size();
    vector<Real> ans(n);
    const Real pi = acosl(-1);
    P shift = n ? a[0].o : P{};
    for (int i = 0; i < n; i++)
    {
        assert(sgn(a[i].r) >= 0); // 调试检查，可删。
        if (sgn(a[i].r) == 0)
        {
            continue;
        }
        vector<pair<Real, int>> e;
        int cnt = 0;
        for (int j = 0; j < n; j++)
        {
            assert(sgn(a[j].r) >= 0); // 调试检查，可删。
            if (i == j || sgn(a[j].r) == 0)
            {
                continue;
            }
            Real d = abs(a[j].o - a[i].o);
            if (eq(a[i].o, a[j].o) && eq(a[i].r, a[j].r))
            {
```

```
                cnt += j < i;
                continue;
            }
            if (sgn(d + a[i].r - a[j].r) <= 0)
            {
                cnt++;
                continue;
            }
            if (sgn(d - a[i].r - a[j].r) >= 0 || sgn(d - fabsl(a[i].r - a[j].r)) <= 0)
            {
                continue;
            }
            Real mid = atan2l(a[j].o.y - a[i].o.y, a[j].o.x - a[i].o.x);
            if (mid < 0)
            {
                mid += 2 * pi;
            }
            Real ang = acosl(clamp((d * d + a[i].r * a[i].r - a[j].r * a[j].r) / (2 * d *
↪  a[i].r), Real(-1), Real(1)));
            Real l = mid - ang;
            Real r = mid + ang;
            while (l < 0)
            {
                l += 2 * pi;
            }
            while (r >= 2 * pi)
            {
                r -= 2 * pi;
            }
            if (l < r)
            {
                e.push_back({l, 1});
                e.push_back({r, -1});
            }
            else
            {
                cnt++;
                e.push_back({r, -1});
                e.push_back({l, 1});
            }
        }
        sort(e.begin(),
             e.end(),
             [](auto x, auto y)
             {
                 return x.first != y.first ? x.first < y.first : x.second < y.second;
             });
        auto add = [&](Real l, Real r)
        {
            // l、r 是当前圆弧的起止角；把该弧的格林积分加入当前覆盖层。
            if (l == r)
            {
```

```
                return;
            }
            assert(0 <= cnt && cnt < n); // 调试检查，可删。
            P o = a[i].o - shift;
            Real sum =
                a[i].r * a[i].r * (r - l) + o.x * a[i].r * (sinl(r) - sinl(l)) - o.y * a[i].r *
↪  (cosl(r) - cosl(l));
            ans[cnt] += sum / 2;
        };
        Real last = 0;
        for (int j = 0; j < (int)e.size();)
        {
            int k = j;
            int delta = 0;
            Real x = e[j].first;
            add(last, x);
            while (k < (int)e.size() && e[k].first == x)
            {
                delta += e[k].second;
                k++;
            }
            cnt += delta;
            last = x;
            j = k;
        }
        add(last, 2 * pi);
    }
    for (Real &x : ans)
    {
        if (sgn(x) == 0)
        {
            x = 0;
        }
    }
    return ans;
}
```

`circleUnion(a)` 是只求第一层的短接口；`circleCov(a)` 返回长度为圆数的数组，`ans[k-1]` 表示至少覆盖 k 次的面积。输入圆半径均须非负，零半径圆不贡献面积。两者都使用 `Real` 数值计算，结果允许浮点误差；n 个圆的时间 $O(n^2 \log n)$，逐圆复用事件数组时额外空间 $O(n)$。固定半径只求最多覆盖点数使用 11.4.007 固定半径圆覆盖最多点。

11.4.010 最近圆对

求两圆边界之间的最小间隙，并在圆集合中找最近的一对。当前版本为确定性枚举基准，直接枚举全部圆对；距离定义是圆周间距还是圆心距必须在接口固定。若需更低复杂度，须另行证明按圆心分治时中线带的筛选条件，不能把点对最近的常数邻居结论直接套到半径不同的圆上。

```cpp
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.4.001 点、线、圆的位置关系
Real circleGap(C a, C b)
{
    // a、b 是圆；返回两条圆周之间的最短距离。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(a.o - b.o);
    return max({Real(0), d - a.r - b.r, fabsl(a.r - b.r) - d});
}

optional<tuple<Real, int, int>> nearCircles(const vector<C> &a)
{
    // a 是圆集；返回最短圆周距离及对应下标。
    if (a.size() < 2)
    {
        return nullopt;
    }
    tuple<Real, int, int> ans{numeric_limits<Real>::infinity(), -1, -1};
    for (int i = 0; i < (int)a.size(); i++)
    {
        for (int j = i + 1; j < (int)a.size(); j++)
        {
            if (circleGap(a[i], a[j]) < get<0>(ans))
            {
                ans = {circleGap(a[i], a[j]), i, j};
            }
        }
    }
    return ans;
}
```

`circleGap(a,b)` 返回两条圆周间的非负最短距离；两圆相交或相切时为 0，一圆严格位于另一圆内时为半径差减圆心距。`nearCircles(a)` 返回最小圆周距离及圆下标，少于两个圆时返回 `nullopt`。$O(n^2)$ 时间、$O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.011 圆弧表示与圆弧重心

表示圆弧并计算均匀弧线的重心。弧长趋近零时应退化到端点，整圆弧需单独约定方向。圆弧由圆心、半径和有向起止角表示；弧长重心位于角平分方向，距圆心为 `r*sin(theta/2)/(theta/2)`，整圆极限回到圆心。

```cpp
// 依赖 11.4.001 点、线、圆的位置关系
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
P arcCentroid(C c, Real l, Real r)
{
    // c 是圆，l、r 是有向起止角；返回均匀弧线的重心。
    assert(sgn(c.r) >= 0); // 调试检查，可删。
```

```
    Real t = r - l;
    if (sgn(t) == 0)
    {
        return c.o + P{cosl(l), sinl(l)} * c.r;
    }
    Real z = 2 * c.r * sinl(t / 2) / t;
    return c.o + P{cosl((l + r) / 2), sinl((l + r) / 2)} * z;
}
```

调用 `arcCentroid(c,a,b)`，`a`、`b` 是均匀有向弧线的起止角；跨越 2π 时先按所需方向规范角度。零长度弧返回对应端点，整圆返回圆心。$O(1)$ 时间和 $O(1)$ 额外空间。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。

11.4.012 圆上整点

枚举圆心在整点、半径平方为整数的圆周整点。平移到圆心后问题变为求全部整数解 $x^2+y^2=n$；本条与 7.5.007 是同一知识在几何章的完整重复。枚举 $0 \le x \le \lfloor\sqrt{n}\rfloor$，检查 $n-x^2$ 是否为完全平方数 y^2；每组非负解通过交换坐标和改变符号生成全部象限解，最后用集合去重。若原圆心为整点 (a,b)，把结果统一平移为 $(a+x,b+y)$。

```
vector<pair<long long, long long>> circlePoints(long long n)
{
    // n 是非负半径平方；返回全部整数解。
    assert(n >= 0); // 调试检查，可删。
    set<pair<long long, long long>> s;
    for (long long x = 0; x == 0 || x <= n / x; x++)
    {
        long long y2 = n - x * x;
        long long y = sqrtl(y2);
        while (y + 1 <= y2 / (y + 1))
        {
            y++;
        }
        while (y > 0 && y > y2 / y)
        {
            y--;
        }
        if (y * y != y2)
        {
            continue;
        }
        for (int sx : {-1, 1})
        {
            for (int sy : {-1, 1})
            {
                s.insert({sx * x, sy * y});
                s.insert({sx * y, sy * x});
            }
```

```
        }
    }
    return {s.begin(), s.end()};
}
```

调用 `circlePoints(n)` 枚举 $x^2+y^2=n$ 的全部整数解。参数要求 `n>=0`；返回结果已去重并按二元组字典序排列。圆心为整点 `(a,b)` 时，再把每个结果平移 `(a,b)`。枚举 $O(\sqrt{n}\log k)$ 时间、$O(k)$ 空间，其中 k 为圆周整点数。圆的交点、位置关系、切线统一建立在 11.4.002 直线与圆、两圆交点 的 Circle 上。覆盖全部点且半径未知用 11.4.006 最小圆覆盖；半径固定、只最大化覆盖点数用 11.4.007 固定半径圆覆盖最多点，两者不能互换。可据此手算自查：`n=25` 时应返回 12 个点：坐标绝对值对为 `(0,5)`、`(3,4)` 及其交换和全部符号组合；`n=3` 返回空集，`n=0` 只返回 `(0,0)`。

11.5 距离、扫描线与特殊坐标

11.5.001 平面最近点对

用分治求平面最近点对，同时返回原下标。按 x 分治、按 y 合并，并只检查中线附近常数个候选点。按 x 坐标分成两半，递归求两侧答案后按 y 合并。跨中线候选只需保留横向距离小于当前答案的点；平面装箱性质保证每个点只向后检查常数个点。

```
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
template <class T> struct ClosestPair
{
    struct Q
    {
        Point<T> p; // p 保存二维点。
        int id; // id 保存原下标。
    };

    vector<Q> a; // a 保存递归中按 x 或 y 排序的点。
    vector<Q> b; // b 保存归并缓冲区。
    T ans; // ans 保存当前最小距离平方。
    pair<int, int> ids; // ids 保存对应的两个原下标。

    void upd(Q x, Q y)
    {
        // x、y 是两个带原下标的点；尝试更新答案。
        T d = norm(x.p - y.p);
        if (d < ans)
        {
            ans = d;
            ids = {x.id, y.id};
        }
    }

    void solve(int l, int r)
    {
        // 递归求 [l,r) 的最近点对，并把该段按 y 坐标归并。
```

```
        if (r - l <= 3)
        {
            for (int i = l; i < r; i++)
            {
                for (int j = l; j < i; j++)
                {
                    upd(a[i], a[j]);
                }
            }
            sort(a.begin() + l,
                 a.begin() + r,
                 [](Q x, Q y)
                 {
                     return x.p.y < y.p.y;
                 });
            return;
        }
        int m = (l + r) / 2;
        T x = a[m].p.x;
        solve(l, m);
        solve(m, r);
        merge(a.begin() + l,
              a.begin() + m,
              a.begin() + m,
              a.begin() + r,
              b.begin() + l,
              [](Q x, Q y)
              {
                  return x.p.y < y.p.y;
              });
        copy(b.begin() + l, b.begin() + r, a.begin() + l);
        vector<Q> s;
        for (int i = l; i < r; i++)
        {
            if ((a[i].p.x - x) * (a[i].p.x - x) > ans)
            {
                continue;
            }
            for (int j = (int)s.size() - 1; j >= 0; j--)
            {
                if ((a[i].p.y - s[j].p.y) * (a[i].p.y - s[j].p.y) > ans)
                {
                    break;
                }
                upd(a[i], s[j]);
            }
            s.push_back(a[i]);
        }
    }

    optional<pair<T, pair<int, int>>> work(const vector<Point<T>> &p)
    {
```

```
        // p 是输入点集；返回最小距离平方与一对原下标。
        if (p.size() < 2)
        {
            return nullopt;
        }
        a.resize(p.size());
        b.resize(p.size());
        for (int i = 0; i < (int)p.size(); i++)
        {
            a[i] = {p[i], i};
        }
        sort(a.begin(),
             a.end(),
             [](Q x, Q y)
             {
                 return x.p.x != y.p.x ? x.p.x < y.p.x : x.p.y < y.p.y;
             });
        for (int i = 1; i < (int)a.size(); i++)
        {
            if (a[i - 1].p == a[i].p)
            {
                return pair<T, pair<int, int>>{0, {a[i - 1].id, a[i].id}};
            }
        }
        ans = norm(a[0].p - a[1].p);
        ids = {a[0].id, a[1].id};
        solve(0, (int)a.size());
        return pair<T, pair<int, int>>{ans, ids};
    }
};
```

构造 `ClosestPair<T> q` 后调用 `q.work(points)`；少于两个点返回空，否则返回类型为 `T` 的距离平方和一对原下标。整数点全程精确，`Point<Real>` 返回浮点距离平方；调用者须保证坐标差及平方和能由 `T` 表示。排序后先识别重复点并直接返回 0，避免零距离条带退化。时间 $O(n \log n)$，空间 $O(n)$；曼哈顿度量见 11.5.002 最远曼哈顿距离。

下面的固定样例同时核对距离和原下标；重复点应返回原下标 0、2，跨分治最近对的下标对应距离必须为 1。

```
signed main()
{
    ClosestPair<Real> q;
    vector<P> a{{0, 0}, {4, 0}, {0, 0}, {2, 2}};
    auto x = q.work(a);
    assert((x && x->first == 0 && x->second == pair<int, int>{0, 2}));
    vector<P> b{{-3, 0}, {-1, 0}, {0, 0}, {4, 0}};
    auto y = q.work(b);
    assert(y && eq(y->first, norm(b[y->second.first] - b[y->second.second])));
    assert(eq(y->first, 1));
    cout << "OK\n";
}
```

11.5.002 最远曼哈顿距离

求点集中最远曼哈顿距离。二维时维护 x+y 与 x-y 的极值即可线性完成。曼哈顿最远距离可枚举每个坐标符号组合，把点映成线性式；任意点对距离等于某个符号下两映射值之差，二维只需四种符号。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> T farManhattan(const vector<Point<T>> &a)
{
    // a 是点集；返回最远点对的曼哈顿距离。
    if (a.empty())
    {
        return 0;
    }
    T lo1 = a[0].x + a[0].y, hi1 = lo1;
    T lo2 = a[0].x - a[0].y, hi2 = lo2;
    for (auto p : a)
    {
        T u = p.x + p.y, v = p.x - p.y;
        lo1 = min(lo1, u);
        hi1 = max(hi1, u);
        lo2 = min(lo2, v);
        hi2 = max(hi2, v);
    }
    return max(hi1 - lo1, hi2 - lo2);
}
```

调用 `farManhattan(points)` 返回与坐标同类型的最远曼哈顿距离。整数版本要求每个 $x+y$、$x-y$ 及最终距离都能由 `T` 表示；坐标接近完整 64 位边界时，在题目代码中显式改用更宽坐标。空集和单点集返回 0；$O(n)$ 时间、$O(1)$ 额外空间。欧氏最近点对见 11.5.001 平面最近点对。

11.5.003 曼哈顿距离与切比雪夫距离转换

在曼哈顿距离与切比雪夫距离之间转换坐标，便于把菱形范围变成轴对齐正方形。二维变换 `u=x+y,v=x-y` 把曼哈顿距离变成切比雪夫距离；逆变换要除以二，因此整数格上的奇偶性必须一致。

```cpp
// 依赖 11.1.002 Point / Vector
template <class T> T manhattan(Point<T> a, Point<T> b)
{
    // a、b 是两个点；返回曼哈顿距离。
    T x = a.x - b.x, y = a.y - b.y;
    return (x < 0 ? -x : x) + (y < 0 ? -y : y);
}

template <class T> T chebyshev(Point<T> a, Point<T> b)
{
    // a、b 是两个点；返回切比雪夫距离。
    T x = a.x - b.x, y = a.y - b.y;
    x = x < 0 ? -x : x;
    y = y < 0 ? -y : y;
    return max(x, y);
```

```cpp
}

// (x,y)->(x+y,x-y)：原坐标曼哈顿距离等于新坐标切比雪夫距离。
template <class T> Point<T> toCheb(Point<T> p)
{
    // p 是平面点；返回对应切比雪夫坐标。
    return {p.x + p.y, p.x - p.y};
}

// 逆变换；若使用整数坐标，u,v 必须同奇偶。
P fromCheb(P p)
{
    // p 是浮点切比雪夫坐标；返回原坐标。
    return {(p.x + p.y) / 2, (p.x - p.y) / 2};
}

Point<long long> fromCheb(Point<long long> p)
{
    // p 是同奇偶的整数切比雪夫坐标；返回精确原坐标。
    assert((p.x - p.y) % 2 == 0); // 调试检查，可删。
    return {(p.x + p.y) / 2, (p.x - p.y) / 2};
}
```

`manhattan`、`chebyshev` 与 `toCheb` 由整数和 `Real` 共用并返回坐标类型。逆变换的整数版要求两个新坐标同奇偶，`Real` 版直接除以二；整数调用必须保证坐标加减、绝对值与距离均能由类型 `T` 表示。各函数均为 $O(1)$；点集最远距离见 11.5.002 最远曼哈顿距离。

11.5.004 矩形面积并、周长并

计算轴对齐矩形面积并或周长并。核心是扫描线和线段树覆盖长度，直接复用数据结构章节的实现。按横坐标扫描矩形左右边，线段树维护纵轴覆盖长度、覆盖段数以及左右端是否被覆盖。条带面积为覆盖长度乘横向宽度，横边长度为覆盖段数的两倍乘横向宽度，事件前后覆盖长度之差贡献竖边。

```cpp
template <class T> struct Rectangle
{
    // x1、y1、x2、y2 依次是矩形的左、下、右、上边界。
    T x1, y1, x2, y2;
};

template <class T, class Acc = T> class RectUnion
{
    struct Event
    {
        T x, y1, y2; // x 是扫描线位置，[y1,y2) 是该事件覆盖的纵向区间。
        int d; // d 为加入 1 或删除 -1。
    };

    vector<T> ys; // ys 保存离散后的纵坐标。
    vector<int> cov, seg, lc, rc; // 覆盖标记、段数和端点状态。
    vector<Acc> len; // len 保存覆盖长度。
```

```
18
19      void pull(int p, int l, int r)
20      {
21          // p 是节点编号，[l,r) 是纵坐标下标区间；由覆盖标记和儿子信息更新节点。
22          if (cov[p])
23          {
24              len[p] = (Acc)ys[r] - (Acc)ys[l];
25              seg[p] = lc[p] = rc[p] = 1;
26          }
27          else if (r - l == 1)
28          {
29              len[p] = 0;
30              seg[p] = lc[p] = rc[p] = 0;
31          }
32          else
33          {
34              int ls = p * 2, rs = ls + 1;
35              len[p] = len[ls] + len[rs];
36              seg[p] = seg[ls] + seg[rs] - (rc[ls] && lc[rs]);
37              lc[p] = lc[ls];
38              rc[p] = rc[rs];
39          }
40      }
41
42      void add(int p, int l, int r, int ql, int qr, int d)
43      {
44          // p、[l,r) 是当前节点，[ql,qr) 是修改区间，d 是覆盖次数增量。
45          if (ql <= l && r <= qr)
46          {
47              cov[p] += d;
48              pull(p, l, r);
49              return;
50          }
51          int m = midpoint(l, r);
52          if (ql < m)
53          {
54              add(p * 2, l, m, ql, qr, d);
55          }
56          if (m < qr)
57          {
58              add(p * 2 + 1, m, r, ql, qr, d);
59          }
60          pull(p, l, r);
61      }
62
63      pair<Acc, Acc> calc(const vector<Rectangle<T>> &a)
64      {
65          // a 是输入矩形集合；返回值 first、second 依次为面积并和周长并。
66          vector<Event> ev;
67          ys.clear();
68          for (auto [x1, y1, x2, y2] : a)
69          {
```

```
            if (x1 >= x2 || y1 >= y2)
            {
                continue;
            }
            ev.push_back({x1, y1, y2, 1});
            ev.push_back({x2, y1, y2, -1});
            ys.push_back(y1);
            ys.push_back(y2);
        }
        if (ev.empty())
        {
            return {};
        }
        ranges::sort(ys);
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        ranges::sort(ev, {}, &Event::x);
        int n = ys.size() * 4;
        cov.assign(n, 0);
        seg.assign(n, 0);
        lc.assign(n, 0);
        rc.assign(n, 0);
        len.assign(n, 0);

        Acc ar = 0;
        Acc peri = 0;
        T px = ev[0].x;
        for (int i = 0; i < (int)ev.size();)
        {
            int j = i;
            while (j < (int)ev.size() && ev[j].x == ev[i].x)
            {
                j++;
            }
            Acc dx = (Acc)ev[i].x - (Acc)px;
            ar += len[1] * dx;
            peri += (Acc)2 * seg[1] * dx;

            // 同一横坐标先加入再删除，逐次长度变化之和恰为新旧覆盖集合的对称差长度。
            for (int d : {1, -1})
            {
                for (int k = i; k < j; k++)
                {
                    if (ev[k].d != d)
                    {
                        continue;
                    }
                    Acc old = len[1];
                    int l = lower_bound(ys.begin(), ys.end(), ev[k].y1) - ys.begin();
                    int r = lower_bound(ys.begin(), ys.end(), ev[k].y2) - ys.begin();
                    add(1, 0, (int)ys.size() - 1, l, r, d);
                    Acc delta = len[1] - old;
                    peri += delta < 0 ? -delta : delta;
```

```
                }
            }
            px = ev[i].x;
            i = j;
        }
        return {ar, peri};
    }

  public:
    Acc area(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的面积。
        return calc(a).first;
    }

    Acc perimeter(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；返回全部轴平行矩形并集的周长。
        return calc(a).second;
    }

    Acc solve(const vector<Rectangle<T>> &a)
    {
        // a 是输入矩形集合；兼容旧接口并返回矩形并集面积。
        return area(a);
    }
};
```

构造 `RectUnion<int> q` 使用 64 位坐标与累计；若坐标差、覆盖长度或面积可能超过 64 位，改用 `RectUnion<int, __int128>`。矩形字段依次为左、下、右、上边界，退化矩形会被忽略。n 个矩形产生 $2n$ 个事件，时间 $O(n \log n)$、空间 $O(n)$。至少二重覆盖见 3.3.015 扫描线求矩形至少二重覆盖面积。

11.5.005 六边形坐标系

表示六边形网格的轴坐标，计算网格距离并与二维显示坐标互换。六边形轴坐标用两个坐标并隐含第三坐标之和为零；相邻移动改变两个轴，距离是三个立方坐标绝对差之和的一半。

```
// 依赖 11.1.002 Point / Vector
struct Hex
{
    int q; // q 是第一轴坐标。
    int r; // r 是第二轴坐标。

    Hex operator+(Hex b) const
    {
        // b 是另一组轴坐标；返回逐分量相加的轴坐标（平移 b）。
        return {q + b.q, r + b.r};
    }
    Hex operator-(Hex b) const
    {
```

```
        // b 是另一组轴坐标；返回逐分量相减的轴坐标，即从 b 到当前格的位移。
        return {q - b.q, r - b.r};
    }
};

int hexDist(Hex a, Hex b = {})
{
    // a、b 是轴坐标；返回六边形网格距离。
    __int128 q = (__int128)a.q - b.q;
    __int128 r = (__int128)a.r - b.r;
    auto ab = [](__int128 x)
    {
        return x < 0 ? -x : x;
    };
    __int128 d = (ab(q) + ab(r) + ab(q + r)) / 2;
    assert(d <= LLONG_MAX); // 调试检查，可删。
    return (int)d;
}

P hexPoint(Hex a, Real size = 1)
{
    // a 是轴坐标，size 是单元边长；返回平面显示坐标。
    return {size * sqrtl(3) * (a.q + a.r / 2.L), size * 1.5L * a.r};
}
```

构造轴坐标 `Hex{q,r}`，第三轴隐含为 `-q-r`；`hexDist(a,b)` 求格距，`hexPoint(a,size)` 转为平面显示坐标。时间和额外空间均为 $O(1)$。

11.5.006 球面经纬度与球面距离

由经纬度计算球面大圆距离，并给出高维球面公式入口。它是球面几何而不是平面欧氏距离。经纬度先转为单位球三维向量，中心角由点积的反余弦或 haversine 公式得到，再乘球半径；小角度优先 haversine 以减少消减误差。

```
// 依赖 11.1.001 浮点误差与精确整数几何
Real sphereDist(Real lat1, Real lon1, Real lat2, Real lon2, Real r = 1)
{
    // lat1、lon1、lat2、lon2 是弧度，r 是非负球半径。
    assert(sgn(r) >= 0); // 调试检查，可删。
    Real x = sinl((lat2 - lat1) / 2);
    Real y = sinl((lon2 - lon1) / 2);
    Real h = x * x + cosl(lat1) * cosl(lat2) * y * y;
    return 2 * r * asinl(sqrtl(clamp(h, Real(0), Real(1))));
}
```

调用 `sphereDist(lat1,lon1,lat2,lon2,r)`，经纬度均使用弧度，`r` 是球半径。haversine 在两点很近时避免点积接近 1 造成的消减误差；对跖点附近仍受浮点舍入影响，代码把中间值限制在 $[0,1]$。时间、额外空间均为 $O(1)$；平面欧氏距离使用 11.1.002 Point / Vector。

11.5.007 多线段任意相交对

在线段集中找出任意一对有公共点的闭线段。扫描线按横坐标成批处理左端插入、竖直线查询和右端删除；活动集的比较器只由线段端点确定，在尚未发现交点时与当前纵向次序一致，不读取会在容器外变化的扫描坐标。每次相邻关系建立时立即判交，同横坐标端点、竖直线段和零长度线段另行批处理，因此严格交、端点接触和共线重叠都能返回。

```cpp
// 依赖 11.1.002 Point / Vector
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
// 依赖 11.1.005 Line / Ray / Segment
// 依赖 11.1.009 线段相交与距离
__int128 swCross(Point<long long> a, Point<long long> b, Point<long long> c)
{
    // a、b 确定有向直线，c 是查询点；返回使用宽整数计算的叉积。
    __int128 x1 = (__int128)b.x - a.x;
    __int128 y1 = (__int128)b.y - a.y;
    __int128 x2 = (__int128)c.x - a.x;
    __int128 y2 = (__int128)c.y - a.y;
    return x1 * y2 - y1 * x2;
}

Real swCross(P a, P b, P c)
{
    // a、b 确定有向直线，c 是查询点；返回浮点叉积。
    return cross(b - a, c - a);
}

bool swOnSeg(Point<long long> p, Line<long long> l)
{
    // p 是整数点，l 是整数闭线段；返回 p 是否在线段上，方向判定使用宽整数。
    return swCross(l.a, l.b, p) == 0 && min(l.a.x, l.b.x) <= p.x && p.x <= max(l.a.x, l.b.x) &&
           min(l.a.y, l.b.y) <= p.y && p.y <= max(l.a.y, l.b.y);
}

bool swMeet(Line<long long> a, Line<long long> b)
{
    // a、b 是整数闭线段；返回二者是否有公共点，叉积使用宽整数。
    __int128 c1 = swCross(a.a, a.b, b.a);
    __int128 c2 = swCross(a.a, a.b, b.b);
    __int128 c3 = swCross(b.a, b.b, a.a);
    __int128 c4 = swCross(b.a, b.b, a.b);
    if (c1 == 0 && swOnSeg(b.a, a))
    {
        return true;
    }
    if (c2 == 0 && swOnSeg(b.b, a))
    {
        return true;
    }
    if (c3 == 0 && swOnSeg(a.a, b))
    {
```

```
        return true;
    }
    if (c4 == 0 && swOnSeg(a.b, b))
    {
        return true;
    }
    return ((c1 > 0) != (c2 > 0)) && ((c3 > 0) != (c4 > 0));
}

bool swMeet(Line<Real> a, Line<Real> b)
{
    // a、b 是浮点闭线段；按统一浮点误差返回二者是否有公共点。
    return segMeet(a, b);
}

template <class T> optional<pair<int, int>> anySegMeet(vector<Line<T>> seg)
{
    // seg 是 long long 或 Real 闭线段集；返回任意一对相交线段的原下标，不存在时返回空。
    // 整数坐标绝对值须不超过 4E18；浮点活动次序使用原值严格比较。
    int n = (int)seg.size();
    vector<pair<Point<T>, int>> endpoint;
    for (int i = 0; i < n; i++)
    {
        endpoint.push_back({seg[i].a, i});
        endpoint.push_back({seg[i].b, i});
    }
    sort(endpoint.begin(), endpoint.end());
    for (int i = 0; i < (int)endpoint.size();)
    {
        int j = i;
        int who = -1;
        while (j < (int)endpoint.size() && endpoint[j].first == endpoint[i].first)
        {
            if (who != -1 && who != endpoint[j].second)
            {
                return pair<int, int>{min(who, endpoint[j].second), max(who,
                ↪ endpoint[j].second)};
            }
            who = endpoint[j].second;
            j++;
        }
        i = j;
    }
    vector<tuple<T, int, int>> event; // 三维依次为横坐标、0 插入/1 竖线查询/2 删除、线段下标。
    for (int i = 0; i < n; i++)
    {
        if (seg[i].b.x < seg[i].a.x)
        {
            swap(seg[i].a, seg[i].b);
        }
        if (seg[i].a.x == seg[i].b.x)
        {
```

```
            event.push_back({seg[i].a.x, 1, i});
        }
        else
        {
            event.push_back({seg[i].a.x, 0, i});
            event.push_back({seg[i].b.x, 2, i});
        }
    }
    sort(event.begin(), event.end());
    Point<T> probe{};
    auto cmp = [&](int i, int j)
    {
        if (i == j)
        {
            return false;
        }
        if (j == -1)
        {
            auto c = swCross(seg[i].a, seg[i].b, probe);
            return c > 0;
        }
        if (i == -1)
        {
            auto c = swCross(seg[j].a, seg[j].b, probe);
            return c <= 0;
        }
        if (seg[i].a.x >= seg[j].a.x)
        {
            auto c = swCross(seg[j].a, seg[j].b, seg[i].a);
            if (c != 0)
            {
                return c < 0;
            }
        }
        else
        {
            auto c = swCross(seg[i].a, seg[i].b, seg[j].a);
            if (c != 0)
            {
                return c > 0;
            }
        }
        return i < j;
    };
    set<int, decltype(cmp)> active(cmp);
    for (int l = 0; l < (int)event.size();)
    {
        int r = l + 1;
        while (r < (int)event.size() && get<0>(event[r]) == get<0>(event[l]))
        {
            r++;
        }
```

```
148         for (int i = l; i < r; i++)
149         {
150             auto [x, type, id] = event[i];
151             if (type != 0)
152             {
153                 continue;
154             }
155             auto it = active.insert(id).first;
156             if (it != active.begin())
157             {
158                 int j = *prev(it);
159                 if (swMeet(seg[id], seg[j]))
160                 {
161                     return pair<int, int>{min(id, j), max(id, j)};
162                 }
163             }
164             if (next(it) != active.end())
165             {
166                 int j = *next(it);
167                 if (swMeet(seg[id], seg[j]))
168                 {
169                     return pair<int, int>{min(id, j), max(id, j)};
170                 }
171             }
172         }
173         vector<int> vertical;
174         for (int i = l; i < r; i++)
175         {
176             if (get<1>(event[i]) == 1)
177             {
178                 vertical.push_back(get<2>(event[i]));
179             }
180         }
181         sort(vertical.begin(),
182              vertical.end(),
183              [&](int i, int j)
184              {
185                  T il = min(seg[i].a.y, seg[i].b.y);
186                  T ir = max(seg[i].a.y, seg[i].b.y);
187                  T jl = min(seg[j].a.y, seg[j].b.y);
188                  T jr = max(seg[j].a.y, seg[j].b.y);
189                  return tuple(il, ir, i) < tuple(jl, jr, j);
190              });
191         int who = -1;
192         T high{};
193         for (int id : vertical)
194         {
195             T low = min(seg[id].a.y, seg[id].b.y);
196             T curHigh = max(seg[id].a.y, seg[id].b.y);
197             if (who != -1 && swMeet(seg[id], seg[who]))
198             {
199                 return pair<int, int>{min(id, who), max(id, who)};
```

```
            }
            if (who == -1 || curHigh > high)
            {
                who = id;
                high = curHigh;
            }
            probe = {get<0>(event[l]), low};
            auto it = active.lower_bound(-1);
            if (it != active.begin())
            {
                int j = *prev(it);
                if (swMeet(seg[id], seg[j]))
                {
                    return pair<int, int>{min(id, j), max(id, j)};
                }
            }
            if (it != active.end() && swMeet(seg[id], seg[*it]))
            {
                return pair<int, int>{min(id, *it), max(id, *it)};
            }
        }
        for (int i = l; i < r; i++)
        {
            auto [x, type, id] = event[i];
            if (type != 2)
            {
                continue;
            }
            auto it = active.find(id);
            assert(it != active.end()); // 调试检查，可删。
            auto nx = next(it);
            if (it != active.begin() && nx != active.end())
            {
                int u = *prev(it);
                int v = *nx;
                if (swMeet(seg[u], seg[v]))
                {
                    return pair<int, int>{min(u, v), max(u, v)};
                }
            }
            active.erase(it);
        }
        l = r;
    }
    return nullopt;
}
```

`anySegMeet(seg)` 接收 `vector<Line<long long>>` 或 `vector<Line<Real>>`，返回任意相交线段的一对原下标，不存在时返回 `nullopt`。整数版在坐标绝对值不超过 4×10^{18} 时用 `__int128` 宽叉积；浮点版关系判断使用统一误差，但活动集和事件排序仍使用原始值严格比较。时间 $O(n\log n)$、空间 $O(n)$；只判断给定一对线段时直接用 11.1.009 线段相交与距离。

11.6 三维几何

11.6.001 三维点、向量、直线、平面

定义三维点、向量、直线和平面，并提供点积、叉积、模长和单位化，作为本节所有空间几何算法的共同底座；具体关系、相交、距离和体积函数放在各自主讲条目。三维点差得到向量，直线用点加方向参数表示，平面用点和法向量或一般式表示；零方向和零法向量不是合法对象。

```
// 依赖 11.1.001 浮点误差与精确整数几何
struct P3
{
    Real x; // x 保存横坐标。
    Real y; // y 保存纵坐标。
    Real z; // z 保存竖坐标。

    P3 operator+(P3 p) const
    {
        // p 是另一三维点或向量；返回逐分量之和。
        return {x + p.x, y + p.y, z + p.z};
    }
    P3 operator-(P3 p) const
    {
        // p 是另一三维点或向量；返回逐分量之差，即从 p 指向当前点的向量。
        return {x - p.x, y - p.y, z - p.z};
    }
    P3 operator*(Real k) const
    {
        // k 是实数标量；返回各分量乘以 k 的向量。
        return {x * k, y * k, z * k};
    }
    P3 operator/(Real k) const
    {
        // k 是非零实数标量；返回各分量除以 k 的向量。
        assert(sgn(k) != 0); // 调试检查，可删。
        return {x / k, y / k, z / k};
    }
};

Real dot(P3 a, P3 b)
{
    // a、b 是三维向量；返回点积 a·b。
    return a.x * b.x + a.y * b.y + a.z * b.z;
}
P3 cross(P3 a, P3 b)
{
    // a、b 是三维向量；返回右手系叉积 a×b，与 a、b 都垂直。
    return {a.y * b.z - a.z * b.y, a.z * b.x - a.x * b.z, a.x * b.y - a.y * b.x};
}
Real norm(P3 a)
{
    // a 是三维向量；返回模长平方 |a|^2。
```

```
    return dot(a, a);
}
Real abs(P3 a)
{
    // a 是三维向量；返回模长 |a|。
    return sqrtl(norm(a));
}
P3 unit(P3 a)
{
    // a 是非零三维向量；返回同方向单位向量 a/|a|。
    return a / abs(a);
}

struct L3
{
    P3 p; // p 是直线上的一点。
    P3 v; // v 是非零方向向量。
};

struct Plane
{
    P3 p; // p 是平面上的一点。
    P3 n; // n 是非零法向量。
};
```

`P3`、`L3`、`Plane` 分别表示三维点向量、点方向式直线、点法式平面；三维浮点接口与二维部分共用 `Real`。本条只提供底座运算。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.002 共线、共面、平行、垂直

统一判断三维点、直线和平面的共线、共面、包含、平行和垂直关系。共线由叉积为零判定，共面由混合积为零判定；直线方向与平面法向的叉积或点积分别给出平行、垂直关系。浮点判号先除以相关向量长度，使判断不随方向向量的普通倍数缩放而改变。整数点另用 `IP3`，把坐标差先提升到 `__int128`；`mixedSign(a,b,c,d)` 返回 $(b-a)\cdot((c-a)\times(d-a))$ 的精确符号。若四个整数点坐标绝对值不超过 10^{12}，混合积的中间量不超过 $6(2\times10^{12})^3<2^{127}$，因此 `collinearEx(IP3,IP3,IP3)`、`coplanarEx(IP3,IP3,IP3,IP3)` 和 `mixedSign` 不发生有符号溢出。精确版本使用独立名称，避免 `collinear({},{},{})` 这类既有浮点调用产生重载二义性。

```
// 依赖 11.6.001 三维点、向量、直线、平面
struct IP3
{
    long long x; // x 是整数点的第一维坐标。
    long long y; // y 是整数点的第二维坐标。
    long long z; // z 是整数点的第三维坐标。
};

int mixedSign(IP3 a, IP3 b, IP3 c, IP3 d)
```

```
{
    // a、b、c、d 是整数空间点；返回 (b-a)·((c-a)×(d-a)) 的精确符号。
    __int128 ux = (__int128)b.x - a.x;
    __int128 uy = (__int128)b.y - a.y;
    __int128 uz = (__int128)b.z - a.z;
    __int128 vx = (__int128)c.x - a.x;
    __int128 vy = (__int128)c.y - a.y;
    __int128 vz = (__int128)c.z - a.z;
    __int128 wx = (__int128)d.x - a.x;
    __int128 wy = (__int128)d.y - a.y;
    __int128 wz = (__int128)d.z - a.z;
    __int128 det = ux * (vy * wz - vz * wy) - uy * (vx * wz - vz * wx) + uz * (vx * wy - vy * wx);
    return (det > 0) - (det < 0);
}

bool collinearEx(IP3 a, IP3 b, IP3 c)
{
    // a、b、c 是整数空间点；返回三点是否共线，重合点视为共线。
    __int128 ux = (__int128)b.x - a.x;
    __int128 uy = (__int128)b.y - a.y;
    __int128 uz = (__int128)b.z - a.z;
    __int128 vx = (__int128)c.x - a.x;
    __int128 vy = (__int128)c.y - a.y;
    __int128 vz = (__int128)c.z - a.z;
    return uy * vz - uz * vy == 0 && uz * vx - ux * vz == 0 && ux * vy - uy * vx == 0;
}

bool coplanarEx(IP3 a, IP3 b, IP3 c, IP3 d)
{
    // a、b、c、d 是整数空间点；返回四点是否共面，退化点集视为共面。
    return mixedSign(a, b, c, d) == 0;
}

bool collinear(P3 a, P3 b, P3 c)
{
    // a、b、c 是三个空间点；返回三点是否共线，重合点视为共线。
    P3 u = b - a;
    P3 v = c - a;
    if (sgn(abs(u)) == 0 || sgn(abs(v)) == 0)
    {
        return true;
    }
    return sgn(abs(cross(u, v)) / abs(u) / abs(v)) == 0;
}

bool coplanar(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是四个空间点；返回四点是否共面，重合或共线退化视为共面。
    P3 u = b - a;
    P3 v = c - a;
    P3 w = d - a;
    if (sgn(abs(u)) == 0 || sgn(abs(v)) == 0 || sgn(abs(w)) == 0)
```

```
62      {
63          return true;
64      }
65      return sgn(dot(u, cross(v, w)) / abs(u) / abs(v) / abs(w)) == 0;
66  }
67
68  bool onLine3(P3 p, L3 l)
69  {
70      // p 是查询点，l 是非退化直线；返回 p 是否在 l 上。
71      assert(sgn(abs(l.v)) != 0); // 调试检查，可删。
72      P3 u = p - l.p;
73      if (sgn(abs(u)) == 0)
74      {
75          return true;
76      }
77      return sgn(abs(cross(u, l.v)) / abs(u) / abs(l.v)) == 0;
78  }
79
80  bool onSegment3(P3 p, P3 a, P3 b)
81  {
82      // p 是查询点，a、b 是闭线段端点；返回 p 是否在线段上，a=b 时线段是单点。
83      P3 v = b - a;
84      if (sgn(abs(v)) == 0)
85      {
86          return sgn(abs(p - a)) == 0;
87      }
88      if (!onLine3(p, {a, v}))
89      {
90          return false;
91      }
92      Real t = dot(p - a, v) / norm(v);
93      return t >= -eps && t <= 1 + eps;
94  }
95
96  bool onPlane3(P3 p, Plane s)
97  {
98      // p 是查询点，s 是非退化平面；返回 p 是否在 s 上。
99      assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
100     return sgn(dot(p - s.p, s.n) / abs(s.n)) == 0;
101 }
102
103 bool parallel(L3 a, L3 b)
104 {
105     // a、b 是两条非退化直线；返回二者方向是否平行。
106     assert(sgn(abs(a.v)) != 0 && sgn(abs(b.v)) != 0); // 调试检查，可删。
107     return sgn(abs(cross(a.v, b.v)) / abs(a.v) / abs(b.v)) == 0;
108 }
109
110 bool parallel(Plane a, Plane b)
111 {
112     // a、b 是两个非退化平面；返回二者是否平行。
113     assert(sgn(abs(a.n)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
```

```
    return sgn(abs(cross(a.n, b.n)) / abs(a.n) / abs(b.n)) == 0;
}

bool parallel(L3 a, Plane b)
{
    // a 是非退化直线，b 是非退化平面；返回直线方向是否平行于平面。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
    return sgn(dot(a.v, b.n) / abs(a.v) / abs(b.n)) == 0;
}

bool perp(L3 a, L3 b)
{
    // a、b 是两条非退化直线；返回二者方向是否垂直。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.v)) != 0); // 调试检查，可删。
    return sgn(dot(a.v, b.v) / abs(a.v) / abs(b.v)) == 0;
}

bool perp(Plane a, Plane b)
{
    // a、b 是两个非退化平面；返回二者法向量是否垂直。
    assert(sgn(abs(a.n)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
    return sgn(dot(a.n, b.n) / abs(a.n) / abs(b.n)) == 0;
}
```

`collinear` 把含重合点的三点视为共线，`coplanar` 把重合或共线退化视为共面；整数版要求各坐标绝对值不超过 10^{12}，浮点版使用统一误差。`onSegment3` 支持端点相同的单点线段。`onLine3`、`onPlane3` 以及 `parallel`、`perp` 的直线方向或平面法向必须非零，接口中用 `assert` 检查。所有函数均为 $O(1)$ 时间和 $O(1)$ 额外空间；求交见 11.6.003 直线与平面、两平面求交，最近点和关系分类见 11.6.010 投影、最近点与空间距离。

11.6.003 直线与平面、两平面求交

求直线与平面交点、两平面交线。平行或重合时返回空 optional，由调用者区分题目语义。直线平面交参数由法向量点积方程解出；两非平行平面交线方向是两法向叉积，再联立两个平面方程求线上一点。

```
// 依赖 11.6.001 三维点、向量、直线、平面
optional<P3> linePlane(L3 l, Plane s)
{
    // 参数 l 表示 待求交的三维直线；s 表示 待求交的平面；返回直线与平面的唯一交点；平行或重合时返回空。
    assert(sgn(abs(l.v)) != 0 && sgn(abs(s.n)) != 0); // 调试检查，可删。
    Real d = dot(l.v, s.n);
    if (sgn(d / abs(l.v) / abs(s.n)) == 0)
    {
        return nullopt;
    }
    return l.p + l.v * (dot(s.p - l.p, s.n) / d);
}

optional<L3> planePlane(Plane a, Plane b)
```

```
{
    // 参数 a 表示 第一个平面；b 表示 第二个平面；返回两平面的交线；平行或重合时返回空。
    assert(sgn(abs(a.n)) != 0 && sgn(abs(b.n)) != 0); // 调试检查，可删。
    P3 v = cross(a.n, b.n);
    Real d = norm(v);
    if (sgn(d / norm(a.n) / norm(b.n)) == 0)
    {
        return nullopt;
    }
    Real x = dot(a.n, a.p);
    Real y = dot(b.n, b.p);
    P3 p = cross(b.n * x - a.n * y, v) / d;
    return L3{p, v};
}
```

`linePlane` 与 `planePlane` 返回 `optional`；空值表示没有唯一交点或交线，平行与重合需由题意进一步区分。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.004 点线面距离与夹角

计算点到直线、点到平面、两异面直线的距离，以及向量夹角。点线距离由点差与方向叉积长度除以方向长度，点面距离由平面方程绝对值除以法向长度；夹角统一由规范化点积或叉积计算。

```
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.6.001 三维点、向量、直线、平面
Real dis(P3 p, L3 l)
{
    // p 是点，l 是直线；返回点线距离。
    assert(sgn(abs(l.v)) != 0); // 调试检查，可删。
    return abs(cross(p - l.p, l.v)) / abs(l.v);
}
Real dis(P3 p, Plane s)
{
    // p 是点，s 是平面；返回点面距离。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    return fabsl(dot(p - s.p, s.n)) / abs(s.n);
}
Real dis(L3 a, L3 b)
{
    // a、b 是直线；返回二者最短距离。
    assert(sgn(abs(a.v)) != 0 && sgn(abs(b.v)) != 0); // 调试检查，可删。
    P3 n = cross(a.v, b.v);
    if (sgn(abs(n) / abs(a.v) / abs(b.v)) == 0)
    {
        return dis(a.p, b);
    }
    return fabsl(dot(b.p - a.p, n)) / abs(n);
}
```

```
Real angle(P3 a, P3 b)
{
    // a、b 是非零向量；返回夹角弧度。
    assert(sgn(abs(a)) != 0 && sgn(abs(b)) != 0); // 调试检查，可删。
    return acosl(clamp(dot(a, b) / abs(a) / abs(b), Real(-1), Real(1)));
}
```

调用三个 `dis` 重载和 `angle`；夹角返回弧度，零向量不能用于夹角。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.005 空间线段与三角形相交

判断空间线段是否穿过三角形。先与三角形所在平面求交，再用同侧叉积判点在三角形内。线段与三角形交可用 Möller-Trumbore 解射线参数和重心坐标，再限制参数在线段范围；共面时需投影到主平面做二维相交。

```
// 依赖 11.1.009 线段相交与距离
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.1.003 点积、叉积、长度、夹角、旋转
bool segTri(P3 p, P3 q, P3 a, P3 b, P3 c)
{
    // p、q 是线段端点，a、b、c 是非退化三角形顶点。
    P3 v = q - p;
    P3 e1 = b - a;
    P3 e2 = c - a;
    P3 n = cross(e1, e2);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    P3 h = cross(v, e2);
    Real d = dot(e1, h);
    Real scale = abs(v) * abs(e1) * abs(e2);
    if (fabsl(d) <= eps * max(Real(1), scale))
    {
        if (sgn(dot(n, p - a) / abs(n)) != 0
            || sgn(dot(n, q - a) / abs(n)) != 0)
        {
            return false;
        }
        int k = 0;
        if (fabsl(n.y) > fabsl(n.x))
        {
            k = 1;
        }
        if (fabsl(n.z) > (k == 0 ? fabsl(n.x) : fabsl(n.y)))
        {
            k = 2;
        }
        auto proj = [&](P3 x)
```

```
        {
            if (k == 0)
            {
                return P{x.y, x.z};
            }
            if (k == 1)
            {
                return P{x.x, x.z};
            }
            return P{x.x, x.y};
        };
        P x = proj(p);
        P y = proj(q);
        P A = proj(a);
        P B = proj(b);
        P C = proj(c);
        auto inside = [&](P z)
        {
            int x1 = sgn(cross(A, B, z));
            int x2 = sgn(cross(B, C, z));
            int x3 = sgn(cross(C, A, z));
            return !(min({x1, x2, x3}) < 0 && max({x1, x2, x3}) > 0);
        };
        return inside(x) || inside(y) || segIsect(x, y, A, B)
            || segIsect(x, y, B, C) || segIsect(x, y, C, A);
    }
    Real z = 1 / d;
    P3 s = p - a;
    Real u = dot(s, h) * z;
    if (u < -eps || u > 1 + eps)
    {
        return false;
    }
    Real w = dot(v, cross(s, e1)) * z;
    if (w < -eps || u + w > 1 + eps)
    {
        return false;
    }
    Real t = dot(e2, cross(s, e1)) * z;
    return t >= -eps && t <= 1 + eps;
}
```

调用 `segTri(p,q,a,b,c)`；三角形必须非退化，返回值包含端点落在三角形边界的情形。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.006 三维凸包

构造三维点集凸包的外向三角面。随机增量维护当前面集：新点能从外侧看到的面全部删除，可见面和保留面交界处的有向边恰好组成地平线，只沿这些边连接新点即可。先把四个不共面的点换到最前面；全部点共面时没有三维面，返回空。

```cpp
// 依赖 11.6.001 三维点、向量、直线、平面
vector<array<int, 3>> hull3(vector<P3> p, int seed = 0)
{
    // p 是点集，seed 是固定洗牌种子；返回外侧朝向的三角面原下标。
    int n = p.size();
    vector<int> id(n);
    iota(id.begin(), id.end(), 0);
    mt19937 rng(seed);
    for (int i = n - 1; i > 0; i--)
    {
        int j = rng() % (i + 1);
        swap(p[i], p[j]);
        swap(id[i], id[j]);
    }
    auto vol = [&](array<int, 3> f, P3 q)
    {
        return dot(cross(p[f[1]] - p[f[0]], p[f[2]] - p[f[0]]), q - p[f[0]]);
    };
    int b = -1;
    for (int i = 1; i < n; i++)
    {
        if (sgn(abs(p[i] - p[0])) != 0)
        {
            b = i;
            break;
        }
    }
    if (b < 0)
    {
        return {};
    }
    swap(p[1], p[b]);
    swap(id[1], id[b]);
    b = -1;
    for (int i = 2; i < n; i++)
    {
        if (sgn(abs(cross(p[1] - p[0], p[i] - p[0]))) != 0)
        {
            b = i;
            break;
        }
    }
    if (b < 0)
    {
        return {};
```

```
    }
    swap(p[2], p[b]);
    swap(id[2], id[b]);
    b = -1;
    for (int i = 3; i < n; i++)
    {
        if (sgn(vol({0, 1, 2}, p[i])) != 0)
        {
            b = i;
            break;
        }
    }
    if (b < 0)
    {
        return {};
    }
    swap(p[3], p[b]);
    swap(id[3], id[b]);

    vector<array<int, 3>> f{{0, 1, 2}, {0, 2, 1}};
    vector<vector<int>> seen(n, vector<int>(n, -1));
    for (int i = 3; i < n; i++)
    {
        vector<array<int, 3>> keep, out;
        for (auto x : f)
        {
            if (sgn(vol(x, p[i])) > 0)
            {
                out.push_back(x);
                for (int k = 0; k < 3; k++)
                {
                    seen[x[k]][x[(k + 1) % 3]] = i;
                }
            }
            else
            {
                keep.push_back(x);
            }
        }
        for (auto x : out)
        {
            for (int k = 0; k < 3; k++)
            {
                int u = x[k], v = x[(k + 1) % 3];
                if (seen[v][u] != i)
                {
                    keep.push_back({u, v, i});
                }
            }
        }
        f.swap(keep);
    }
```

```
    for (auto &x : f)
    {
        for (int &u : x)
        {
            u = id[u];
        }
    }
    return f;
}
```

调用 `hull3(p,seed)` 返回外向三角面的原下标，固定种子默认 0；输入允许重复点，若无法找到四个不共面点则返回空。随机顺序下凸包面数为 $O(n)$，增量扫描期望 $O(n^2)$ 时间；有向边时间戳矩阵占 $O(n^2)$ 空间，适合竞赛中常见的几千点规模。大量共面壳点会选择其中一套三角剖分，不保证保留面内所有点；需要输出共面多边形面时应再按平面合并相邻三角面。

11.6.007 最小覆盖球

求覆盖全部三维点的最小球。随机增量时边界集最多四点，可分别由一至四点确定球。随机打乱点后增量维护覆盖球；遇到球外点时把它依次固定为球面点，并在更内层重建由二、三、四个边界点确定的球。

```
// 依赖 11.6.001 三维点、向量、直线、平面
struct Sphere
{
    P3 o; // o 保存球心。
    Real r; // r 保存非负半径。
};

bool in(Sphere s, P3 p)
{
    // s 是球，p 是查询点；返回 p 是否在球内或球面上。
    assert(sgn(s.r) >= 0); // 调试检查，可删。
    return sgn(abs(p - s.o) - s.r) <= 0;
}

optional<Sphere> sphere4(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是四点；共面时没有唯一外接球。
    P3 u = b - a;
    P3 v = c - a;
    P3 w = d - a;
    Real det = dot(u, cross(v, w));
    if (fabsl(det) <= eps * max(Real(1), abs(u) * abs(v) * abs(w)))
    {
        return nullopt;
    }
    P3 o = a + (cross(v, w) * norm(u) + cross(w, u) * norm(v) + cross(u, v) * norm(w)) / (2 *
    ↪ det);
    return Sphere{o, abs(o - a)};
}

```

```
optional<Sphere> sphere3(P3 a, P3 b, P3 c)
{
    // a、b、c 是三点；共线时没有唯一外接球。
    P3 u = b - a;
    P3 v = c - a;
    P3 w = cross(u, v);
    Real z = 2 * norm(w);
    if (sgn(abs(w)) == 0)
    {
        return nullopt;
    }
    P3 o = a + (cross(w, u) * norm(v) + cross(v, w) * norm(u)) / z;
    return Sphere{o, abs(o - a)};
}

Sphere boundSphere(const vector<P3> &p)
{
    // p 含至多四个边界点；返回覆盖它们的最小球。
    if (p.empty())
    {
        return {{}, 0};
    }
    Sphere ans{{}, numeric_limits<Real>::infinity()};
    int n = p.size();
    auto add = [&](optional<Sphere> s)
    {
        if (!s || s->r >= ans.r)
        {
            return;
        }
        for (P3 x : p)
        {
            if (!in(*s, x))
            {
                return;
            }
        }
        ans = *s;
    };
    for (int i = 0; i < n; i++)
    {
        add(Sphere{p[i], 0});
    }
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < i; j++)
        {
            add(Sphere{(p[i] + p[j]) / 2, abs(p[i] - p[j]) / 2});
        }
    }
    for (int i = 0; i < n; i++)
    {
```

```
        for (int j = 0; j < i; j++)
        {
            for (int k = 0; k < j; k++)
            {
                add(sphere3(p[i], p[j], p[k]));
            }
        }
    }
    if (n == 4)
    {
        add(sphere4(p[0], p[1], p[2], p[3]));
    }
    return ans;
}

Sphere minSphere(vector<P3> p, unsigned seed = 712367)
{
    // p 是点集，seed 是打乱种子；返回最小覆盖球。
    mt19937 rng(seed);
    shuffle(p.begin(), p.end(), rng);
    Sphere s{{}, -1};
    for (int i = 0; i < (int)p.size(); i++)
    {
        if (s.r >= 0 && in(s, p[i]))
        {
            continue;
        }
        s = {p[i], 0};
        for (int j = 0; j < i; j++)
        {
            if (in(s, p[j]))
            {
                continue;
            }
            s = boundSphere({p[i], p[j]});
            for (int k = 0; k < j; k++)
            {
                if (in(s, p[k]))
                {
                    continue;
                }
                s = boundSphere({p[i], p[j], p[k]});
                for (int l = 0; l < k; l++)
                {
                    if (!in(s, p[l]))
                    {
                        s = boundSphere({p[i], p[j], p[k], p[l]});
                    }
                }
            }
        }
    }
```

```
    if (p.empty())
    {
        s = {{}, 0};
    }
    return s;
}
```

调用 `minSphere(p,seed)`，返回 `Sphere{o,r}`；空点集返回原点和半径 0。随机排列期望 $O(n)$、最坏 $O(n^4)$ 时间，$O(n)$ 空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.008 四面体体积与内切球

计算四面体体积与内切球。体积来自标量三重积，内心是按对面面积加权的顶点平均。四面体有向体积是三条边混合积的六分之一；内心按四个对面三角形面积加权顶点，内切球半径为三倍体积除以表面积。

```
// 依赖 11.1.001 浮点误差与精确整数几何
// 依赖 11.6.001 三维点、向量、直线、平面
Real tetraVolume(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是四面体顶点；返回无符号体积。
    return fabsl(dot(b - a, cross(c - a, d - a))) / 6;
}

pair<P3, Real> tetraSphere(P3 a, P3 b, P3 c, P3 d)
{
    // a、b、c、d 是非退化四面体顶点；返回内心和内切球半径。
    Real x = abs(cross(b - a, c - a));
    Real y = abs(cross(b - a, d - a));
    Real z = abs(cross(c - a, d - a));
    Real w = abs(cross(c - b, d - b));
    Real s = x + y + z + w;
    assert(sgn(s) != 0 && sgn(tetraVolume(a, b, c, d)) != 0); // 调试检查，可删。
    P3 o = (d * x + c * y + b * z + a * w) / s;
    return {o, 3 * tetraVolume(a, b, c, d) / (s / 2)};
}
```

调用 `tetraVolume` 与 `tetraSphere`；后者返回内心和内切球半径，四点必须构成非退化四面体。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.009 高维球体积、超球坐标与立体角

计算高维球体积、超球坐标相关量和立体角。公式依赖 Gamma 函数，数值范围与精度必须单独检查。d 维单位球体积满足 `V_d=pi^(d/2)/Gamma(d/2+1)`；超球坐标的体积元含逐层正弦幂，立体角是单位球面上对应区域测度。

```cpp
// 依赖 11.1.001 浮点误差与精确整数几何
Real sphereVolume(Real r, int n = 3)
{
    // r 是非负半径，n 是非负维数；返回 n 维球体积。
    assert(sgn(r) >= 0 && n >= 0); // 调试检查，可删。
    if (n == 0)
    {
        return 1;
    }
    if (sgn(r) == 0)
    {
        return 0;
    }
    Real x = Real(n) / 2 * logl(acosl(-1)) + n * logl(r)
        - lgammal(Real(n) / 2 + 1);
    return expl(x);
}
```

调用 `sphereVolume(r,n)`；`r` 与 `n` 必须非负，0 维球体积按约定为 1。函数在对数域使用 `lgammal`，最后一次 `expl` 仍可能在真实结果超出 `Real` 范围时得到无穷。$O(1)$ 时间和 $O(1)$ 额外空间。所有空间算法先使用 11.6.001 三维点、向量、直线、平面 的 P3/L3/Plane。小规模凸包可用 11.6.006 三维凸包 枚举版，规模大或存在大量共面点时应换可靠增量实现；覆盖球使用 11.6.007 最小覆盖球，不能用平面最小圆。

11.6.010 投影、最近点与空间距离

点到直线 l 的投影参数为 $t = \frac{(p-l.p)\cdot l.v}{\|l.v\|^2}$，点到线段时再把 t 截到 $[0,1]$；两条非平行直线由两个正交条件组成的二元方程求最近点对，据距离区分相交与异面。三个平面有唯一交点当且仅当 $D = n_a\cdot(n_b\times n_c) \neq 0$，Cramer 公式直接给出交点。本节依赖 11.6.002 共线、共面、平行、垂直、11.6.003 直线与平面、两平面求交 和 11.6.004 点线面距离与夹角。`lineLine3` 允许直线退化，任一方向为零时返回 `LineRel3::Degenerate`；平行或重合直线没有唯一最近点见证，故 `closest` 为空。

```cpp
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.004 点线面距离与夹角
struct PointSeg3
{
    P3 point;      // point 是线段上的最近点。
    Real t;        // t 满足 point=a+(b-a)*t，范围为 [0,1]。
    Real distance; // distance 是查询点到线段的距离。
};

struct ClosestPair3
{
    P3 onA;        // onA 是第一个对象上的最近点。
    P3 onB;        // onB 是第二个对象上的最近点。
    Real tA;       // tA 是 onA 在第一个对象参数式中的参数。
    Real tB;       // tB 是 onB 在第二个对象参数式中的参数。
    Real distance; // distance 是两个对象之间的距离。
};

```

```cpp
enum class LineRel3
{
    Intersecting,
    Skew,
    Parallel,
    Coincident,
    Degenerate
};

struct LineClosest3
{
    LineRel3 relation;          // relation 是两条直线的空间关系。
    Real distance;                   // distance 是两条直线的最短距离，所有关系下都有效。
    optional<ClosestPair3> closest; // closest 在相交、异面或退化点线关系下保存唯一最近点对。
};

P3 projection(L3 l, P3 p)
{
    // l 是非退化直线，p 是查询点；返回 p 在 l 上的正交投影。
    assert(norm(l.v) != 0); // 调试检查，可删。
    return l.p + l.v * (dot(p - l.p, l.v) / norm(l.v));
}

P3 projection(Plane s, P3 p)
{
    // s 是非退化平面，p 是查询点；返回 p 在 s 上的正交投影。
    assert(norm(s.n) != 0); // 调试检查，可删。
    return p - s.n * (dot(p - s.p, s.n) / norm(s.n));
}

PointSeg3 pointSeg3(P3 p, P3 a, P3 b)
{
    // p 是查询点，a、b 是闭线段端点；返回最近点、线段参数和距离。
    P3 v = b - a;
    Real t = 0;
    if (norm(v) != 0)
    {
        t = clamp(dot(p - a, v) / norm(v), Real(0), Real(1));
    }
    P3 q = a + v * t;
    return {q, t, abs(p - q)};
}

LineClosest3 lineLine3(L3 a, L3 b)
{
    // a、b 是两条可退化直线；返回关系、距离以及存在唯一解时的最近点对。
    Real aa = norm(a.v);
    Real bb = norm(b.v);
    if (aa == 0 && bb == 0)
    {
        Real distance = abs(a.p - b.p);
        return {LineRel3::Degenerate, distance, ClosestPair3{a.p, b.p, 0, 0, distance}};
```

```
    }
    if (aa == 0)
    {
        Real t = dot(a.p - b.p, b.v) / bb;
        P3 q = b.p + b.v * t;
        Real distance = abs(a.p - q);
        return {LineRel3::Degenerate, distance, ClosestPair3{a.p, q, 0, t, distance}};
    }
    if (bb == 0)
    {
        Real s = dot(b.p - a.p, a.v) / aa;
        P3 q = a.p + a.v * s;
        Real distance = abs(q - b.p);
        return {LineRel3::Degenerate, distance, ClosestPair3{q, b.p, s, 0, distance}};
    }
    P3 n = cross(a.v, b.v);
    Real d = norm(n);
    if (d == 0)
    {
        LineRel3 relation = onLine3(a.p, b) ? LineRel3::Coincident : LineRel3::Parallel;
        Real distance = relation == LineRel3::Coincident ? 0 : dis(a.p, b);
        return {relation, distance, nullopt};
    }
    P3 w = b.p - a.p;
    Real s = dot(cross(w, b.v), n) / d;
    Real t = dot(cross(w, a.v), n) / d;
    P3 x = a.p + a.v * s;
    P3 y = b.p + b.v * t;
    Real distance = abs(x - y);
    LineRel3 relation = sgn(distance) == 0 ? LineRel3::Intersecting : LineRel3::Skew;
    return {relation, distance, ClosestPair3{x, y, s, t, distance}};
}

ClosestPair3 segSeg3(P3 a, P3 b, P3 c, P3 d)
{
    // a、b 与 c、d 是两条闭线段端点；返回一组最近点、两段参数和距离。
    P3 u = b - a;
    P3 v = d - c;
    P3 w = a - c;
    Real aa = norm(u);
    Real ab = dot(u, v);
    Real bb = norm(v);
    Real aw = dot(u, w);
    Real bw = dot(v, w);
    Real s = 0;
    Real t = 0;
    if (aa == 0 && bb == 0)
    {
        s = t = 0;
    }
    else if (aa == 0)
    {
```

```
        t = clamp(bw / bb, Real(0), Real(1));
    }
    else if (bb == 0)
    {
        s = clamp(-aw / aa, Real(0), Real(1));
    }
    else
    {
        P3 n = cross(u, v);
        Real det = norm(n);
        if (det != 0)
        {
            s = clamp(dot(cross(c - a, v), n) / det, Real(0), Real(1));
        }
        t = (ab * s + bw) / bb;
        if (t < 0)
        {
            t = 0;
            s = clamp(-aw / aa, Real(0), Real(1));
        }
        else if (t > 1)
        {
            t = 1;
            s = clamp((ab - aw) / aa, Real(0), Real(1));
        }
    }
    P3 x = a + u * s;
    P3 y = c + v * t;
    ClosestPair3 ans{x, y, s, t, abs(x - y)};
    auto take = [&](P3 p, P3 q, Real ta, Real tb)
    {
        Real distance = abs(p - q);
        if (distance < ans.distance)
        {
            ans = {p, q, ta, tb, distance};
        }
    };
    PointSeg3 acd = pointSeg3(a, c, d);
    PointSeg3 bcd = pointSeg3(b, c, d);
    PointSeg3 cab = pointSeg3(c, a, b);
    PointSeg3 dab = pointSeg3(d, a, b);
    take(a, acd.point, 0, acd.t);
    take(b, bcd.point, 1, bcd.t);
    take(cab.point, c, cab.t, 0);
    take(dab.point, d, dab.t, 1);
    return ans;
}

Real angle(L3 a, L3 b)
{
    // a、b 是两条非退化无向直线；返回范围 [0,pi/2] 的较小夹角弧度。
    assert(norm(a.v) != 0 && norm(b.v) != 0); // 调试检查，可删。
```

```
    Real x = fabsl(dot(a.v, b.v) / abs(a.v) / abs(b.v));
    return acosl(clamp(x, Real(0), Real(1)));
}

Real angle(L3 a, Plane b)
{
    // a 是非退化直线，b 是非退化平面；返回范围 [0,pi/2] 的夹角弧度。
    assert(norm(a.v) != 0 && norm(b.n) != 0); // 调试检查，可删。
    Real x = fabsl(dot(a.v, b.n) / abs(a.v) / abs(b.n));
    return asinl(clamp(x, Real(0), Real(1)));
}

Real angle(Plane a, Plane b)
{
    // a、b 是两个非退化无向平面；返回范围 [0,pi/2] 的较小夹角弧度。
    assert(norm(a.n) != 0 && norm(b.n) != 0); // 调试检查，可删。
    Real x = fabsl(dot(a.n, b.n) / abs(a.n) / abs(b.n));
    return acosl(clamp(x, Real(0), Real(1)));
}

optional<P3> planesMeet3(Plane a, Plane b, Plane c)
{
    // a、b、c 是三个非退化平面；返回唯一公共点，不唯一或不存在时返回空。
    assert(norm(a.n) != 0 && norm(b.n) != 0 && norm(c.n) != 0); // 调试检查，可删。
    Real det = dot(a.n, cross(b.n, c.n));
    if (det == 0)
    {
        return nullopt;
    }
    Real x = dot(a.n, a.p);
    Real y = dot(b.n, b.p);
    Real z = dot(c.n, c.p);
    P3 num = cross(b.n, c.n) * x + cross(c.n, a.n) * y + cross(a.n, b.n) * z;
    return num * (1 / det);
}
```

调用 `projection`、`pointSeg3`、`lineLine3` 或 `segSeg3` 取得投影与最近点信息；两个线段接口都允许端点重合，此时退化段按单点处理。`angle` 的三个重载返回无向较小夹角，`planesMeet3` 仅在三平面有唯一公共点时返回值。所有接口均为 $O(1)$ 时间和 $O(1)$ 额外空间；投影与夹角接口要求相应直线、平面非退化，只求标量距离或向量夹角时直接使用 11.6.004 点线面距离与夹角 更短。

11.6.011 射线与 AABB 查询

轴对齐包围盒可在三个坐标轴上分别解参数不等式，再把三个闭区间与查询参数域取交；射线使用 $[0,\infty)$，线段使用 $[0,1]$，因此盒内射线起点的进入参数固定为 0。平面查询同样只接受非负参数。本节的射线接口显式允许零方向，但此时 `rayPlane3` 和 `rayAABB3` 都返回 `RayState3::Degenerate`，不把静止点当作普通射线命中。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
struct Ray3
{
    P3 p; // p 是射线起点。
    P3 v; // v 是射线方向，查询函数允许零方向并返回 Degenerate。
};

struct AABB3
{
    P3 lo; // lo 是三个坐标的闭区间下界。
    P3 hi; // hi 是三个坐标的闭区间上界。
};

struct ParamPoint3
{
    Real t;   // t 是参数式 p+v*t 中的参数。
    P3 point; // point 是参数 t 对应的空间点。
};

struct ParamRange3
{
    Real enter; // enter 是命中闭集的最小参数。
    Real exit;  // exit 是离开闭集的最大参数。
    P3 first;   // first 是 enter 对应的点。
    P3 last;    // last 是 exit 对应的点。
};

enum class RayState3
{
    None,
    Hit,
    Degenerate
};

struct RayPtQuery3
{
    RayState3 state; // state 表示未命中、命中或零方向退化。
    ParamPoint3 hit;      // hit 只在 state=Hit 时有效。
};

struct RayIntQuery3
{
    RayState3 state; // state 表示未命中、命中或零方向退化。
    ParamRange3 hit;   // hit 只在 state=Hit 时有效。
};

bool in(AABB3 b, P3 p)
{
    // b 是合法闭包围盒，p 是查询点；返回 p 是否在盒内或边界上。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    auto inside = [&](Real x, Real l, Real r)
```

```
    {
        return (x > l || eq(x, l)) && (x < r || eq(x, r));
    };
    return inside(p.x, b.lo.x, b.hi.x) && inside(p.y, b.lo.y, b.hi.y) && inside(p.z, b.lo.z,
    ↪ b.hi.z);
}

optional<ParamRange3> aabbRange3(P3 p, P3 v, AABB3 b, Real lower, Real upper)
{
    // p+v*t 是参数直线，b 是合法闭包围盒，[lower,upper] 是参数域；返回命中参数区间，未命中返回空。
    assert(lower <= upper);                                                    // 调试检查，可删。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    array<Real, 3> origin{p.x, p.y, p.z};
    array<Real, 3> dir{v.x, v.y, v.z};
    array<Real, 3> lo{b.lo.x, b.lo.y, b.lo.z};
    array<Real, 3> hi{b.hi.x, b.hi.y, b.hi.z};
    if (v.x == 0 && v.y == 0 && v.z == 0)
    {
        if (!in(b, p))
        {
            return nullopt;
        }
        return ParamRange3{lower, upper, p, p};
    }
    Real enter = lower;
    Real exit = upper;
    for (int i = 0; i < 3; i++)
    {
        if (dir[i] == 0)
        {
            bool outside = (origin[i] < lo[i] && !eq(origin[i], lo[i])) || (origin[i] > hi[i]
            ↪ && !eq(origin[i], hi[i]));
            if (outside)
            {
                return nullopt;
            }
            continue;
        }
        Real x = (lo[i] - origin[i]) / dir[i];
        Real y = (hi[i] - origin[i]) / dir[i];
        if (x > y)
        {
            swap(x, y);
        }
        enter = max(enter, x);
        exit = min(exit, y);
        if (enter > exit && !eq(enter, exit))
        {
            return nullopt;
        }
    }
    if (enter > exit)
```

```
    {
        Real t = (enter + exit) / 2;
        enter = exit = t;
    }
    return ParamRange3{enter, exit, p + v * enter, p + v * exit};
}

RayPtQuery3 rayPlane3(Ray3 r, Plane s)
{
    // r 是可退化射线，s 是非退化平面；返回闭平面的首次非负命中。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    Real d = dot(r.v, s.n);
    if (d == 0)
    {
        if (onPlane3(r.p, s))
        {
            return {RayState3::Hit, {0, r.p}};
        }
        return {RayState3::None, {}};
    }
    Real t = dot(s.p - r.p, s.n) / d;
    if (t < 0)
    {
        return {RayState3::None, {}};
    }
    return {RayState3::Hit, {t, r.p + r.v * t}};
}

RayIntQuery3 rayAABB3(Ray3 r, AABB3 b)
{
    // r 是可退化射线，b 是合法闭包围盒；返回盒内的非负参数区间。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    auto hit = aabbRange3(r.p, r.v, b, 0, numeric_limits<Real>::infinity());
    if (!hit)
    {
        return {RayState3::None, {}};
    }
    return {RayState3::Hit, *hit};
}

optional<ParamRange3> segmentAABB3(P3 a, P3 b, AABB3 box)
{
    // a、b 是闭线段端点，box 是合法闭包围盒；返回参数域 [0,1] 内的命中区间，未命中返回空。
    P3 v = b - a;
```

```
    if (v.x == 0 && v.y == 0 && v.z == 0)
    {
        if (!in(box, a))
        {
            return nullopt;
        }
        return ParamRange3{0, 1, a, a};
    }
    return aabbRange3(a, v, box, 0, 1);
}

bool sphereAABB3(Sphere s, AABB3 b)
{
    // s 是非负半径闭球，b 是合法闭包围盒；返回两个闭集是否相交。
    assert(sgn(s.r) >= 0);                                          // 调试检查，可删。
    assert(b.lo.x <= b.hi.x && b.lo.y <= b.hi.y && b.lo.z <= b.hi.z); // 调试检查，可删。
    P3 q{clamp(s.o.x, b.lo.x, b.hi.x), clamp(s.o.y, b.lo.y, b.hi.y), clamp(s.o.z, b.lo.z,
    ↪ b.hi.z)};
    return in(b, q) && in(s, q);
}
```

`RayState3::None`、`Hit`、`Degenerate` 分别表示未命中、命中和零方向射线；射线与平面共面时首次命中为 $t = 0$。`aabbRange3` 与 `segmentAABB3` 未命中时返回 `nullopt`，退化线段若位于盒内则返回完整参数域 $[0, 1]$；`sphereAABB3` 通过逐坐标夹取得盒内最近点判断闭球与闭盒是否相交。包围盒必须满足各维下界不大于上界，球半径必须非负。所有查询均为 $O(1)$ 时间和 $O(1)$ 额外空间，并依赖 11.6.007 最小覆盖球 的 `Sphere` 及 11.6.010 投影、最近点与空间距离 的基础接口。

11.6.012 空间三角形与平面裁剪

空间三角形先以法向量确定承载平面，再用同向边叉积判断闭集包含关系；最近点若投影落在内部就用重心坐标表示，否则比较三条边，三角形与平面之交则按顶点符号分类为空、点、线段或整三角形。除仅计算面积的接口外，三角形参数必须非退化；`clipPolygon3` 只支持顶点有序的共面凸多边形，并保留裁剪平面法向正侧的闭半空间。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
// 依赖 11.6.004 点线面距离与夹角
// 依赖 11.6.010 投影、最近点与空间距离
// 依赖 11.6.011 射线与 AABB 查询
enum class TriPlaneType
{
    Empty,
    Point,
    Segment,
    Triangle
};

struct TriPlaneRes3
{
```

```
    TriPlaneType type; // type 是空、点、线段或整三角形四类交集。
    vector<P3> points;       // points 按 type 保存 0、1、2 或 3 点；整三角形保持输入序，线段点按三条边枚举序。
};

struct TriClose3
{
    P3 point;                   // point 是三角形闭集上的最近点。
    array<Real, 3> barycentric; // barycentric 依次是三个顶点的重心坐标。
    Real distance;              // distance 是查询点到三角形的距离。
};

bool inTri3(P3 p, P3 a, P3 b, P3 c)
{
    // p 是查询点，a、b、c 是非退化三角形；返回 p 是否在三角形闭集内。
    P3 n = cross(b - a, c - a);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    if (!onPlane3(p, {a, n}))
    {
        return false;
    }
    Real x = dot(cross(b - a, p - a), n) / abs(b - a) / abs(n);
    Real y = dot(cross(c - b, p - b), n) / abs(c - b) / abs(n);
    Real z = dot(cross(a - c, p - c), n) / abs(a - c) / abs(n);
    return sgn(x) >= 0 && sgn(y) >= 0 && sgn(z) >= 0;
}

TriClose3 pointTri3(P3 p, P3 a, P3 b, P3 c)
{
    // p 是查询点，a、b、c 是非退化三角形；返回最近点、重心坐标和距离。
    P3 u = b - a;
    P3 v = c - a;
    P3 n = cross(u, v);
    assert(sgn(abs(n)) != 0); // 调试检查，可删。
    P3 q = projection(Plane{a, n}, p);
    if (inTri3(q, a, b, c))
    {
        P3 w = q - a;
        Real uu = norm(u);
        Real uv = dot(u, v);
        Real vv = norm(v);
        Real uw = dot(u, w);
        Real vw = dot(v, w);
        Real d = uu * vv - uv * uv;
        Real y = (vv * uw - uv * vw) / d;
        Real z = (uu * vw - uv * uw) / d;
        return {q, {1 - y - z, y, z}, abs(p - q)};
    }
    PointSeg3 ab = pointSeg3(p, a, b);
    PointSeg3 bc = pointSeg3(p, b, c);
    PointSeg3 ca = pointSeg3(p, c, a);
    TriClose3 ans{ab.point, {1 - ab.t, ab.t, 0}, ab.distance};
```

```
    if (bc.distance < ans.distance)
    {
        ans = {bc.point, {0, 1 - bc.t, bc.t}, bc.distance};
    }
    if (ca.distance < ans.distance)
    {
        ans = {ca.point, {ca.t, 0, 1 - ca.t}, ca.distance};
    }
    return ans;
}

TriPlaneRes3 triPlane3(P3 a, P3 b, P3 c, Plane s)
{
    // a、b、c 是非退化三角形，s 是非退化平面；返回完整交集分类与顶点。
    assert(sgn(abs(cross(b - a, c - a))) != 0 && sgn(abs(s.n)) != 0); // 调试检查，可删。
    array<P3, 3> p{a, b, c};
    array<Real, 3> d;
    array<int, 3> side;
    for (int i = 0; i < 3; i++)
    {
        d[i] = dot(p[i] - s.p, s.n) / abs(s.n);
        side[i] = sgn(d[i]);
    }
    if (side[0] == 0 && side[1] == 0 && side[2] == 0)
    {
        return {TriPlaneType::Triangle, {a, b, c}};
    }
    vector<P3> out;
    auto add = [&](P3 q)
    {
        for (P3 x : out)
        {
            if (sgn(abs(x - q)) == 0)
            {
                return;
            }
        }
        out.push_back(q);
    };
    for (int i = 0; i < 3; i++)
    {
        int j = (i + 1) % 3;
        if (side[i] == 0)
        {
            add(p[i]);
        }
        if (side[i] * side[j] < 0)
        {
            add(p[i] + (p[j] - p[i]) * (d[i] / (d[i] - d[j])));
        }
    }
    if (out.empty())
```

```
    {
        return {TriPlaneType::Empty, {}};
    }
    if (out.size() == 1)
    {
        return {TriPlaneType::Point, out};
    }
    assert(out.size() == 2); // 调试检查，可删。
    return {TriPlaneType::Segment, out};
}

vector<P3> clipPolygon3(const vector<P3> &poly, Plane s)
{
    // poly 是有序共面凸多边形，s 是非退化裁剪平面；返回法向正侧闭半空间内的顶点序列。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    if (poly.empty())
    {
        return {};
    }
    vector<P3> out;
    int n = (int)poly.size();
    for (int i = 0; i < n; i++)
    {
        P3 a = poly[i];
        P3 b = poly[(i + 1) % n];
        Real x = dot(a - s.p, s.n) / abs(s.n);
        Real y = dot(b - s.p, s.n) / abs(s.n);
        int sx = sgn(x);
        int sy = sgn(y);
        if (sx >= 0)
        {
            out.push_back(a);
        }
        if (sx * sy < 0)
        {
            out.push_back(a + (b - a) * (x / (x - y)));
        }
    }
    vector<P3> clean;
    for (P3 p : out)
    {
        if (clean.empty() || sgn(abs(clean.back() - p)) != 0)
        {
            clean.push_back(p);
        }
    }
    if (clean.size() > 1 && sgn(abs(clean.front() - clean.back())) == 0)
    {
        clean.pop_back();
    }
    return clean;
}
```

```
171
172 Real triArea3(P3 a, P3 b, P3 c)
173 {
174     // a、b、c 是三角形顶点；返回无符号面积，退化时返回 0。
175     return abs(cross(b - a, c - a)) / 2;
176 }
177
178 Real polygonArea3(const vector<P3> &poly)
179 {
180     // poly 是顶点有序的共面简单多边形；返回无符号面积，少于三个点时返回 0。
181     if (poly.size() < 3)
182     {
183         return 0;
184     }
185     P3 sum{};
186     for (int i = 1; i + 1 < (int)poly.size(); i++)
187     {
188         sum = sum + cross(poly[i] - poly[0], poly[i + 1] - poly[0]);
189     }
190     return abs(sum) / 2;
191 }
192
193 RayPtQuery3 rayTriangle3(Ray3 r, P3 a, P3 b, P3 c)
194 {
195     // r 是可退化射线，a、b、c 是非退化三角形；返回三角形闭集的首次命中。
196     P3 n = cross(b - a, c - a);
197     assert(sgn(abs(n)) != 0); // 调试检查，可删。
198     if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
199     {
200         return {RayState3::Degenerate, {}};
201     }
202     Real d = dot(r.v, n);
203     if (d != 0)
204     {
205         Real t = dot(a - r.p, n) / d;
206         if (t < 0)
207         {
208             return {RayState3::None, {}};
209         }
210         P3 p = r.p + r.v * t;
211         if (!inTri3(p, a, b, c))
212         {
213             return {RayState3::None, {}};
214         }
215         return {RayState3::Hit, {t, p}};
216     }
217     if (!onPlane3(r.p, {a, n}))
218     {
219         return {RayState3::None, {}};
220     }
221     array<P3, 3> p{a, b, c};
222     Real enter = 0;
```

```
    Real exit = numeric_limits<Real>::infinity();
    for (int i = 0; i < 3; i++)
    {
        int j = (i + 1) % 3;
        P3 edge = p[j] - p[i];
        Real scale = abs(edge) * abs(n);
        Real x = dot(cross(edge, r.p - p[i]), n) / scale;
        Real y = dot(cross(edge, r.v), n) / scale;
        if (y == 0)
        {
            if (sgn(x) < 0)
            {
                return {RayState3::None, {}};
            }
            continue;
        }
        Real t = -x / y;
        if (y > 0)
        {
            enter = max(enter, t);
        }
        else
        {
            exit = min(exit, t);
        }
        if (enter > exit && !eq(enter, exit))
        {
            return {RayState3::None, {}};
        }
    }
    return {RayState3::Hit, {enter, r.p + r.v * enter}};
}
```

调用 `inTri3`、`pointTri3`、`triPlane3` 和 `rayTriangle3` 完成常数规模查询；零方向射线返回 `RayState3::Degenerate`。`polygonArea3` 只接受顶点有序的共面简单多边形。`triArea3` 为 $O(1)$，含 m 个顶点的 `clipPolygon3` 与 `polygonArea3` 均为 $O(m)$ 时间，前者使用 $O(m)$、后者使用 $O(1)$ 额外空间，其余接口均为 $O(1)$ 时间和空间；射线首次命中沿用 11.6.011 射线与 AABB 查询，只判断闭线段与三角形是否相交可用 11.6.005 空间线段与三角形相交。

11.6.013 球、空间圆与球冠 / 球带

直线与球面之交先投影球心并由弦长恢复至多两个参数，平面与球面之交返回空间圆，两球关系则由球心距和半径和、半径差分类；球冠与球带使用旋转体截面积分公式。所有 `Sphere`、`Circle3` 及面积体积接口都要求半径非负；`raySphere3` 只保留非负参数，所以射线从球内出发时只返回出口交点。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
// 依赖 11.6.004 点线面距离与夹角
// 依赖 11.6.010 投影、最近点与空间距离
```

```cpp
// 依赖 11.6.011 射线与 AABB 查询
struct Circle3
{
    P3 o;   // o 是空间圆圆心。
    P3 n;   // n 是圆所在平面的非零法向量。
    Real r; // r 是非负半径，0 表示单点圆。
};

enum class SphereRel3
{
    Separate,
    ExtTangent,
    Intersecting,
    IntTangent,
    Contained,
    Coincident
};

struct SpherePair3
{
    SphereRel3 relation; // relation 是两个球面的相对关系。
    optional<Circle3> circle; // circle 在相切或相交时保存交圆，相切时半径为 0。
};

struct RayPtsQuery3
{
    RayState3 state;     // state 表示未命中、命中或零方向退化。
    vector<ParamPoint3> hits; // hits 在命中时按非负参数递增保存球面交点。
};

vector<ParamPoint3> lineSphere3(L3 l, Sphere s)
{
    // l 是非退化直线，s 是非负半径球；返回按参数递增的 0 至 2 个球面交点。
    assert((l.v.x != 0 || l.v.y != 0 || l.v.z != 0) && s.r >= 0); // 调试检查，可删。
    Real vv = norm(l.v);
    Real middle = dot(s.o - l.p, l.v) / vv;
    P3 foot = l.p + l.v * middle;
    Real d2 = norm(foot - s.o);
    Real radius2 = s.r * s.r;
    Real tol = eps * max(d2, radius2);
    if (d2 > radius2 + tol)
    {
        return {};
    }
    Real chord2 = radius2 - d2;
    if (fabsl(chord2) <= tol)
    {
        return {{middle, foot}};
    }
    Real half = sqrtl(max(Real(0), chord2) / vv);
    Real x = middle - half;
    Real y = middle + half;
```

```
    return {{x, l.p + l.v * x}, {y, l.p + l.v * y}};
}

RayPtsQuery3 raySphere3(Ray3 r, Sphere s)
{
    // r 是可退化射线，s 是非负半径球；返回所有非负参数的球面交点。
    assert(s.r >= 0); // 调试检查，可删。
    if (r.v.x == 0 && r.v.y == 0 && r.v.z == 0)
    {
        return {RayState3::Degenerate, {}};
    }
    vector<ParamPoint3> out;
    for (ParamPoint3 hit : lineSphere3({r.p, r.v}, s))
    {
        if (hit.t < 0)
        {
            continue;
        }
        out.push_back(hit);
    }
    if (out.empty())
    {
        return {RayState3::None, {}};
    }
    return {RayState3::Hit, out};
}

optional<Circle3> planeSphere3(Plane p, Sphere s)
{
    // p 是非退化平面，s 是非负半径球；返回交圆，相离时返回空，相切时半径为 0。
    assert(sgn(abs(p.n)) != 0 && sgn(s.r) >= 0); // 调试检查，可删。
    Real h = dot(s.o - p.p, p.n) / abs(p.n);
    if (fabsl(h) > s.r && !eq(fabsl(h), s.r))
    {
        return nullopt;
    }
    P3 o = projection(p, s.o);
    Real r2 = max(Real(0), s.r * s.r - h * h);
    return Circle3{o, unit(p.n), sqrtl(r2)};
}

SpherePair3 spherePair3(Sphere a, Sphere b)
{
    // a、b 是两个非负半径球；返回球面关系及相切或相交时的交圆。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(b.o - a.o);
    Real sum = a.r + b.r;
    Real diff = fabsl(a.r - b.r);
    if (eq(d, Real(0)) && eq(a.r, b.r))
    {
        return {SphereRel3::Coincident, nullopt};
    }
```

```
    if (d < diff && !eq(d, diff))
    {
        return {SphereRel3::Contained, nullopt};
    }
    SphereRel3 relation;
    if (eq(d, diff))
    {
        relation = SphereRel3::IntTangent;
    }
    else if (d > sum && !eq(d, sum))
    {
        return {SphereRel3::Separate, nullopt};
    }
    else if (eq(d, sum))
    {
        relation = SphereRel3::ExtTangent;
    }
    else
    {
        relation = SphereRel3::Intersecting;
    }
    P3 u = (b.o - a.o) / d;
    Real x = (d * d + a.r * a.r - b.r * b.r) / (2 * d);
    Real r2 = max(Real(0), a.r * a.r - x * x);
    return {relation, Circle3{a.o + u * x, u, sqrtl(r2)}};
}

vector<ParamPoint3> lineCircle3(L3 l, Circle3 c)
{
    // l 是非退化直线，c 是法向非零的空间圆；返回按直线参数递增的交点。
    assert((l.v.x != 0 || l.v.y != 0 || l.v.z != 0) && sgn(abs(c.n)) != 0 && c.r >= 0); // 调试
    ↪ 检查，可删。
    Real d = dot(l.v, c.n);
    if (d != 0)
    {
        Real t = dot(c.o - l.p, c.n) / d;
        P3 p = l.p + l.v * t;
        Real d2 = norm(p - c.o);
        Real radius2 = c.r * c.r;
        if (fabsl(d2 - radius2) > eps * max(d2, radius2))
        {
            return {};
        }
        return {{t, p}};
    }
    if (sgn(dot(l.p - c.o, c.n) / abs(c.n)) != 0)
    {
        return {};
    }
    return lineSphere3(l, {c.o, c.r});
}

```

```
Real sphereLens3(Sphere a, Sphere b)
{
    // a、b 是两个非负半径闭球；返回二者公共部分体积。
    assert(sgn(a.r) >= 0 && sgn(b.r) >= 0); // 调试检查，可删。
    Real d = abs(b.o - a.o);
    Real sum = a.r + b.r;
    Real diff = fabsl(a.r - b.r);
    if (eq(d, Real(0)) || d < diff || eq(d, diff))
    {
        return sphereVolume(min(a.r, b.r), 3);
    }
    if (d > sum || eq(d, sum))
    {
        return 0;
    }
    const Real pi = acosl(-1);
    Real h = sum - d;
    Real q = d * d + 2 * d * sum - 3 * (a.r - b.r) * (a.r - b.r);
    return pi * h * h * q / (12 * d);
}

Real sphereArea(Real r)
{
    // r 是非负球半径；返回三维球面面积。
    assert(r >= 0); // 调试检查，可删。
    return 4 * acosl(-1) * r * r;
}

Real capArea(Real r, Real h)
{
    // r 是非负球半径，h 是范围 [0,2r] 的球冠高；返回球冠曲面面积。
    assert(r >= 0 && (h > 0 || eq(h, Real(0))) && (h < 2 * r || eq(h, 2 * r))); // 调试检查，可删。
    h = clamp(h, Real(0), 2 * r);
    return max(Real(0), 2 * acosl(-1) * r * h);
}

Real capVolume(Real r, Real h)
{
    // r 是非负球半径，h 是范围 [0,2r] 的球冠高；返回球冠体积。
    assert(r >= 0 && (h > 0 || eq(h, Real(0))) && (h < 2 * r || eq(h, 2 * r))); // 调试检查，可删。
    h = clamp(h, Real(0), 2 * r);
    return max(Real(0), acosl(-1) * h * h * (r - h / 3));
}

Real zoneArea(Real r, Real low, Real high)
{
    // r 是非负球半径，[low,high] 是相对球心轴坐标区间；返回球带曲面面积。
    assert(r >= 0 && (low > -r || eq(low, -r)) && (low < high || eq(low, high)) &&
           (high < r || eq(high, r))); // 调试检查，可删。
    low = clamp(low, -r, r);
    high = clamp(high, -r, r);
    if (low > high)
```

```
    {
        low = high = (low + high) / 2;
    }
    return max(Real(0), 2 * acosl(-1) * r * (high - low));
}

Real zoneVol3(Real r, Real low, Real high)
{
    // r 是非负球半径，[low,high] 是相对球心轴坐标区间；返回两截面间体积。
    assert(r >= 0 && (low > -r || eq(low, -r)) && (low < high || eq(low, high)) &&
           (high < r || eq(high, r))); // 调试检查，可删。
    low = clamp(low, -r, r);
    high = clamp(high, -r, r);
    if (low > high)
    {
        low = high = (low + high) / 2;
    }
    auto primitive = [&](Real z)
    {
        return r * r * z - z * z * z / 3;
    };
    return max(Real(0), acosl(-1) * (primitive(high) - primitive(low)));
}
```

调用 `lineSphere3`、`raySphere3`、`planeSphere3`、`spherePair3` 或 `lineCircle3` 查询边界交集；球冠高满足 $0 \le h \le 2r$，球带轴坐标满足 $-r \le low \le high \le r$。`sphereLens3` 返回两闭球公共体积，其余公式接口分别返回球面、球冠或球带的面积与体积。所有接口均为 $O(1)$ 时间和 $O(1)$ 额外空间，交点向量至多含两个元素；实现依赖 11.6.009 高维球体积、超球坐标与立体角 的球体积及 11.6.011 射线与 AABB 查询 的射线结果类型。

11.6.014 共面基底、凸包与网格截面

非退化平面可构造右手正交基，把共面点投到二维后复用 11.3.001 二维凸包；闭合三角网格的体积由各有向面与固定原点组成的四面体代数体积求和，截面则把同一网格边上的交点按共享边编号合并并沿面定向连成环。网格必须是闭合、方向一致、共享顶点使用同一编号的二流形；截平面必须处于一般位置，经过网格顶点、边或面时 `meshSection3` 返回 `nullopt`。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.002 共线、共面、平行、垂直
struct PlaneBasis3
{
    P3 o; // o 是局部二维坐标原点。
    P3 u; // u 是局部 x 轴单位向量。
    P3 v; // v 是局部 y 轴单位向量。
    P3 n; // n 是满足 cross(u,v)=n 的单位法向量。
};

struct MeshLoop3
{
```

```
    vector<P3> points; // points 是不重复首点的有向截面环。
    Real area;   // area 是沿截平面法向观察的有向面积。
    Real perimeter;    // perimeter 是截面环周长。
};

PlaneBasis3 planeBasis3(Plane s)
{
    // s 是非退化平面；返回以 s.p 为原点的右手正交基。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    P3 n = unit(s.n);
    P3 axis;
    if (fabsl(n.x) <= fabsl(n.y) && fabsl(n.x) <= fabsl(n.z))
    {
        axis = {1, 0, 0};
    }
    else if (fabsl(n.y) <= fabsl(n.z))
    {
        axis = {0, 1, 0};
    }
    else
    {
        axis = {0, 0, 1};
    }
    P3 u = unit(cross(axis, n));
    P3 v = cross(n, u);
    return {s.p, u, v, n};
}

P planeCoord3(PlaneBasis3 b, P3 p)
{
    // b 是右手正交平面基，p 是空间点；返回 p 的局部二维投影坐标。
    P3 q = p - b.o;
    return {dot(q, b.u), dot(q, b.v)};
}

P3 planePoint3(PlaneBasis3 b, P p)
{
    // b 是右手正交平面基，p 是局部二维坐标；返回对应的空间平面点。
    return b.o + b.u * p.x + b.v * p.y;
}

vector<P3> coplanarHull(const vector<P3> &points, Plane s)
{
    // points 是 s 上的点集，s 是非退化平面；返回沿 s.n 观察逆时针的凸包顶点。
    PlaneBasis3 basis = planeBasis3(s);
    vector<P> projected;
    map<P, int> original;
    for (int i = 0; i < (int)points.size(); i++)
    {
        assert(onPlane3(points[i], s)); // 调试检查，可删。
        P q = planeCoord3(basis, points[i]);
        projected.push_back(q);
```

```
        if (!original.count(q))
        {
            original[q] = i;
        }
    }
    vector<P3> out;
    for (P q : hull(projected))
    {
        out.push_back(points[original[q]]);
    }
    return out;
}

Real meshVolume3(const vector<P3> &points, const vector<array<int, 3>> &faces)
{
    // points 是网格顶点，faces 每项三维依次是按一致外向顺序的三个顶点下标；返回封闭网格体积。
    if (faces.empty())
    {
        return 0;
    }
    assert(!points.empty() && 0 <= faces[0][0] && faces[0][0] < (int)points.size()); // 调试检
    ↪ 查，可删。
    P3 o = points[faces[0][0]];
    Real sum = 0;
    for (auto f : faces)
    {
        for (int x : f)
        {
            assert(0 <= x && x < (int)points.size()); // 调试检查，可删。
        }
        P3 a = points[f[0]] - o;
        P3 b = points[f[1]] - o;
        P3 c = points[f[2]] - o;
        assert(sgn(abs(cross(b - a, c - a))) != 0); // 调试检查，可删。
        sum += dot(a, cross(b, c));
    }
    return fabsl(sum) / 6;
}

optional<vector<MeshLoop3>> meshSection3(const vector<P3> &points, const vector<array<int, 3>>
↪ &faces, Plane s)
{
    // points 是网格顶点，faces 每项三维依次是按一致外向顺序的三个顶点下标，s 是截平面；返回截面环，退
    ↪ 化时返回空。
    assert(sgn(abs(s.n)) != 0); // 调试检查，可删。
    vector<Real> distance(points.size());
    vector<int> side(points.size());
    for (int i = 0; i < (int)points.size(); i++)
    {
        distance[i] = dot(points[i] - s.p, s.n) / abs(s.n);
        side[i] = sgn(distance[i]);
        if (side[i] == 0)
```

```
        {
            return nullopt;
        }
    }
    map<pair<int, int>, int> id; // id 把无向网格边映射到唯一截面点编号。
    vector<P3> nodePoint;        // nodePoint 保存各截面点的空间坐标。
    vector<int> next;            // next 保存有向截面环中的后继点编号。
    vector<int> indegree;        // indegree 保存各截面点在有向截面图中的入度。
    auto getNode = [&](int a, int b)
    {
        pair<int, int> edge = minmax(a, b);
        auto it = id.find(edge);
        if (it != id.end())
        {
            return it->second;
        }
        Real t = distance[a] / (distance[a] - distance[b]);
        int z = (int)nodePoint.size();
        id[edge] = z;
        nodePoint.push_back(points[a] + (points[b] - points[a]) * t);
        next.push_back(-1);
        indegree.push_back(0);
        return z;
    };
    for (auto f : faces)
    {
        for (int x : f)
        {
            assert(0 <= x && x < (int)points.size()); // 调试检查，可删。
        }
        P3 fn = cross(points[f[1]] - points[f[0]], points[f[2]] - points[f[0]]);
        if (sgn(abs(fn)) == 0)
        {
            return nullopt;
        }
        vector<pair<int, int>> cut;
        for (int i = 0; i < 3; i++)
        {
            int a = f[i];
            int b = f[(i + 1) % 3];
            if (side[a] * side[b] < 0)
            {
                cut.push_back({a, b});
            }
        }
        if (cut.empty())
        {
            continue;
        }
        if (cut.size() != 2)
        {
            return nullopt;
```

```
        }
        int a = getNode(cut[0].first, cut[0].second);
        int b = getNode(cut[1].first, cut[1].second);
        P3 dir = cross(s.n, fn);
        Real turn = dot(nodePoint[b] - nodePoint[a], dir);
        if (sgn(turn / abs(nodePoint[b] - nodePoint[a]) / abs(dir)) == 0)
        {
            return nullopt;
        }
        if (turn < 0)
        {
            swap(a, b);
        }
        if (next[a] != -1 || indegree[b] != 0)
        {
            return nullopt;
        }
        next[a] = b;
        indegree[b]++;
    }
    for (int i = 0; i < (int)nodePoint.size(); i++)
    {
        if (next[i] == -1 || indegree[i] != 1)
        {
            return nullopt;
        }
    }
    PlaneBasis3 basis = planeBasis3(s);
    vector<char> used(nodePoint.size());
    vector<MeshLoop3> out;
    for (int start = 0; start < (int)nodePoint.size(); start++)
    {
        if (used[start])
        {
            continue;
        }
        vector<P3> loop;
        int cur = start;
        while (!used[cur])
        {
            used[cur] = true;
            loop.push_back(nodePoint[cur]);
            cur = next[cur];
        }
        if (cur != start || loop.size() < 3)
        {
            return nullopt;
        }
        Real area2 = 0;
        Real perimeter = 0;
        for (int i = 1; i + 1 < (int)loop.size(); i++)
        {
```

```
            area2 += dot(cross(loop[i] - loop[0], loop[i + 1] - loop[0]), basis.n);
        }
        for (int i = 0; i < (int)loop.size(); i++)
        {
            int j = (i + 1) % loop.size();
            perimeter += abs(loop[j] - loop[i]);
        }
        out.push_back({loop, area2 / 2, perimeter});
    }
    return out;
}
```

调用 `planeCoord3` 与 `planePoint3` 在局部二维坐标和空间点之间转换，`coplanarHull` 返回沿给定法向观察的逆时针凸包；`meshSection3` 的空 `optional` 表示退化输入，含空向量的有效结果表示平面未切中网格。平面基与坐标转换为 $O(1)$ 时间和空间；设点数、面数和参与截面的边数分别为 V、F、E，共面凸包为 $O(V\log V)$ 时间和 $O(V)$ 额外空间，体积为 $O(F)$ 时间和 $O(1)$ 额外空间，截面为 $O(V+F\log E)$ 时间和 $O(V+E)$ 额外空间。本节依赖 11.3.001 Andrew / Graham 静态凸包 和 11.6.012 空间三角形与平面裁剪。

11.6.015 反射、二面角与球面多边形

点 p 关于平面的反射是 $p'=2\,\mathrm{projection}(p)-p$；有向二面角把两个面法向投到公共边的正交补，再按公共边右手方向从第一投影转向第二投影，两个投影都必须非零。对单位球面方向 a,b,c，令 $y=a\cdot(b\times c)$、$x=1+a\cdot b+b\cdot c+c\cdot a$，短大圆弧三角形的立体角为 $\Omega=2\,\mathrm{atan2}(y,x)$；一般简单球面多边形由 Gauss–Bonnet 得到 $A/R^2\equiv 2\pi-\sum_i\tau_i\pmod{4\pi}$，其中 τ_i 是有向顶点转角。顶点必须位于同一正半径球面，边取短大圆弧且相邻点不得对跖；允许非凸和跨越半球，但不允许自交或把边替成长大圆弧。返回模 $4\pi R^2$ 的主值有向面积，范围为 $(-2\pi R^2,2\pi R^2]$。

```
// 依赖 11.6.001 三维点、向量、直线、平面
// 依赖 11.6.004 点线面距离与夹角
// 依赖 11.6.010 投影、最近点与空间距离
P3 reflection(Plane s, P3 p)
{
    // s 是非退化平面，p 是查询点；返回 p 关于 s 的镜像点。
    P3 q = projection(s, p);
    return q * 2 - p;
}

Real signedDih3(P3 edge, P3 na, P3 nb)
{
    // edge 是非零有向边，两法向的正交投影非零；返回绕 edge 按右手方向从 A 到 B 的 [-pi,pi] 角。
    assert(sgn(abs(edge)) != 0 && sgn(abs(na)) != 0 && sgn(abs(nb)) != 0); // 调试检查，可删。
    P3 e = unit(edge);
    P3 a = na - e * dot(na, e);
    P3 b = nb - e * dot(nb, e);
    assert(sgn(abs(a)) != 0 && sgn(abs(b)) != 0); // 调试检查，可删。
    return atan2l(dot(e, cross(a, b)), dot(a, b));
}
```

```
21
22  Real dihedral3(P3 edge, P3 na, P3 nb)
23  {
24      // edge 是非零公共边，na、nb 是两面非零法向；返回范围 [0,pi] 的无向二面角。
25      return fabsl(signedDih3(edge, na, nb));
26  }
27
28  P3 fromLatLon3(Real lat, Real lon, Real r = 1)
29  {
30      // lat、lon 是纬度和经度弧度，r 是非负半径；返回球心在原点的球面点。
31      const Real pi = acosl(-1);
32      assert((lat > -pi / 2 || eq(lat, -pi / 2)) && (lat < pi / 2 || eq(lat, pi / 2)) &&
33             r >= 0); // 调试检查，可删。
34      Real z = cosl(lat);
35      return {r * z * cosl(lon), r * z * sinl(lon), r * sinl(lat)};
36  }
37
38  pair<Real, Real> toLatLon3(P3 p)
39  {
40      // p 是以原点为球心的非零球面点；返回纬度与范围 [-pi,pi] 的经度弧度。
41      assert(p.x != 0 || p.y != 0 || p.z != 0); // 调试检查，可删。
42      Real xy = hypotl(p.x, p.y);
43      Real lon = xy == 0 ? 0 : atan2l(p.y, p.x);
44      return {atan2l(p.z, xy), lon};
45  }
46
47  Real solidAngle3(P3 a, P3 b, P3 c)
48  {
49      // a、b、c 是非零方向且两两不反向；返回三条短大圆弧围成的主值有向立体角。
50      assert((a.x != 0 || a.y != 0 || a.z != 0) && (b.x != 0 || b.y != 0 || b.z != 0) &&
51             (c.x != 0 || c.y != 0 || c.z != 0)); // 调试检查，可删。
52      auto normalize = [&](P3 p)
53      {
54          Real len = abs(p);
55          return P3{p.x / len, p.y / len, p.z / len};
56      };
57      a = normalize(a);
58      b = normalize(b);
59      c = normalize(c);
60      assert(norm(a + b) != 0 && norm(b + c) != 0 && norm(c + a) != 0); // 调试检查，可删。
61      Real y = dot(a, cross(b, c));
62      Real x = 1 + dot(a, b) + dot(b, c) + dot(c, a);
63      return 2 * atan2l(y, x);
64  }
65
66  Real spherePoly3(const vector<P3> &poly, Sphere s)
67  {
68      // poly 是球面简单多边形顶点，s 是正半径球；返回 (-2pi*r^2,2pi*r^2] 内的主值有向面积。
69      assert(s.r > 0); // 调试检查，可删。
70      auto normalize = [&](P3 p)
71      {
72          Real len = abs(p);
```

```
        return P3{p.x / len, p.y / len, p.z / len};
    };
    vector<P3> dir;
    for (P3 p : poly)
    {
        P3 q = p - s.o;
        Real len = abs(q);
        assert(fabsl(len - s.r) <= eps * max(len, s.r)); // 调试检查，可删。
        P3 u = normalize(q);
        if (dir.empty() || norm(dir.back() - u) != 0)
        {
            dir.push_back(u);
        }
    }
    if (dir.size() > 1 && norm(dir.front() - dir.back()) == 0)
    {
        dir.pop_back();
    }
    if (dir.size() < 3)
    {
        return 0;
    }
    for (int i = 0; i < (int)dir.size(); i++)
    {
        int j = (i + 1) % dir.size();
        assert(norm(dir[i] + dir[j]) != 0); // 调试检查，可删。
    }
    Real turn = 0;
    int n = (int)dir.size();
    for (int i = 0; i < n; i++)
    {
        P3 pre = dir[(i + n - 1) % n];
        P3 cur = dir[i];
        P3 next = dir[(i + 1) % n];
        P3 ni = normalize(cross(pre, cur));
        P3 no = normalize(cross(cur, next));
        P3 vin = cross(ni, cur);
        P3 vout = cross(no, cur);
        turn += atan2l(dot(cur, cross(vin, vout)), dot(vin, vout));
    }
    const Real pi = acosl(-1);
    Real ans = turn > 0 ? 2 * pi - turn : -2 * pi - turn;
    if (turn == 0)
    {
        ans = 2 * pi;
    }
    while (ans <= -2 * pi)
    {
        ans += 4 * pi;
    }
    while (ans > 2 * pi)
    {
```

```
        ans -= 4 * pi;
    }
    return ans * s.r * s.r;
}
```

调用 `reflection`、`signedDih3` 与 `dihedral3` 处理平面和二面角，后两者分别返回 $[-\pi,\pi]$ 和 $[0,\pi]$ 内的弧度；`fromLatLon3` 与 `toLatLon3` 在球面点和纬经度弧度之间转换，位于两极且横向分量为零时经度约定返回 0。`solidAngle3` 返回三条短大圆弧围成的主值有向立体角。`spherePoly3` 对 n 个顶点使用 $O(n)$ 时间和 $O(n)$ 额外空间，其余接口均为 $O(1)$ 时间和空间，并依赖 11.6.007 最小覆盖球 的 `Sphere` 和 11.6.010 投影、最近点与空间距离 的投影接口。

12. 动态规划与优化

本章遵循统一类型约定：下标、状态编号、容量和普通计数使用 `int`（公共前导中的 `#define int long long` 会在提交中将其提升为 `long long`）；可能超过 32 位的数值、代价和答案显式使用 `long long`。只有确实依赖 64 位机器字或无符号范围的位集、插头状态、汉诺塔步数、数位 DP 计数和凸包差值比较保留 `unsigned long long`，不得将这些类型扩散到普通变量。

12.1 背包

12.1.001 01 背包

01 背包让每件物品至多用一次，状态 $f[j]$ 是恰好占用容量 j 时的最大价值，转移 $f[j] \leftarrow \max(f[j],\ f[j-w]+v)$。关键在容量必须倒序枚举：倒序时 `f[j-w]` 尚未在本轮被同一件物品更新，仍代表”未放入该物品”的旧状态；一旦改成正序，`f[j-w]` 可能已含这件物品，就退化成可重复取的完全背包。不可达容量用 $-\infty$（这里取 `-INF`）标记并在转移前判掉，避免把无效状态当作价值 0 向后传播。

```
vector<int> zeroOnePack(int m, const vector<pair<int, int>> &a)
{
    // m 是容量，a 的每项是物品体积和价值；返回每个恰好容量的最大价值，不可达为 -INF。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v] : a)
    {
        assert(w > 0); // 调试检查，可删。
        for (int j = m; j >= w; j--)
        {
            if (f[j - w] != -INF)
            {
                chmax(f[j], f[j - w] + v);
            }
        }
    }
    return f;
}
```

调用 `zeroOnePack(m,a)`，容量非负、物品体积为正，且所有可达价值和须能由 `int` 表示；返回的 `f[j]` 是恰好装满容量 j 的最优值，若求容量不超过 m 的答案取 `max(f)`。时间 $O(nm)$、空间 $O(m)$。容量循环方向是这一族的分水岭：可重复取时改用 12.1.002 完全背包 的正序，件数有限用 12.1.003 多重背包，组内互斥用 12.1.006 分组背包，物品间存在选取依赖时不能拍平成普通物品、须走 12.1.007 有依赖背包。

12.1.002 完全背包

完全背包每种物品可取任意次，状态与转移和 01 背包完全相同，只有容量改成正序枚举：正序时 `f[j-w]` 可能已在本轮被同种物品更新过，于是”再加一件”自然叠加，等价于该物品用无限次。体积必须严格为正，否则 `j-w>=j`、正序循环没有推进方向甚至自我引用。同样用 `-INF` 标记不可达并在转移前判掉。

```cpp
vector<int> completePack(int m, const vector<pair<int, int>> &a)
{
    // m 是容量，a 的每项是物品体积和价值且每种可取任意次；返回各恰好容量的最大价值。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v] : a)
    {
        assert(w > 0); // 调试检查，可删。
        for (int j = w; j <= m; j++)
        {
            if (f[j - w] != -INF)
            {
                chmax(f[j], f[j - w] + v);
            }
        }
    }
    return f;
}
```

调用 `completePack(m,a)`，语义与 12.1.001 一致但每种可无限取；容量非负、物品体积为正，且所有可达价值和须能由 `int` 表示。时间 $O(nm)$、空间 $O(m)$。当同一种物品有明确件数上限时它会超额取用，应换 12.1.003 多重背包；混合出现多类物品见 12.1.004 混合背包。

12.1.003 多重背包

多重背包给每种物品一个件数上限 c。直接把它拆成 c 件 01 物品会带来 $O(c)$ 因子，这里用二进制拆分：把 c 分成 $1, 2, 4, \ldots$ 直到剩余，每组当作一件体积价值成倍的 01 物品，用 $O(\log c)$ 组就能凑出 $0 \sim c$ 之间任意取用件数。拆出的每组仍按 01 背包倒序更新，因此天然不会超额。件数极大时可换成按容量余数分组的单调队列优化（见 12.1.009），二进制拆分胜在实现短、常数稳。

```cpp
vector<int> boundedPack(int m, const vector<array<int, 3>> &a)
{
    // m 是容量；a[i][0..2] 依次是体积、价值和最多可选件数；返回各恰好容量的最大价值。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v, c] : a)
    {
        assert(w > 0 && c >= 0); // 调试检查，可删。
        int k = 1;
        while (c > 0)
        {
            int z = min(k, c);
            c -= z;
            if (z <= m / w)
            {
                int ww = w * z;
                int vv = v * z;
```

```
                for (int j = m; j >= ww; j--)
                {
                    if (f[j - ww] != -INF)
                    {
                        chmax(f[j], f[j - ww] + vv);
                    }
                }
            }
            if (c > 0)
            {
                k = k <= c / 2 ? k * 2 : c;
            }
        }
    }
    return f;
}
```

调用 `boundedPack(m,a)`，`a[i][0..2]` 依次是体积、价值、件数，要求体积为正、件数非负。时间 $O(m\sum_i \log(c_i+1))$、空间 $O(m)$。一个小样例：容量 10、物品 `{3,4,2}` 与 `{2,3,3}`，恰好装满容量 10 的最大价值为 14。件数无上限时退回 12.1.002 完全背包，与 01、完全物品混杂时用 12.1.004 混合背包 统一处理。

12.1.004 混合背包

混合背包在同一份输入里同时出现 01、完全、多重三类物品，做法是所有物品共享同一个 f 数组，但逐物品按其类型选定更新方式：完全物品正序、01 物品倒序（相当于件数为 1 的多重）、多重物品做二进制拆分后倒序。这里用件数字段 -1 / 0 / 正数作类型标记；每一类的循环方向必须在处理该物品时就地确定，不能全局统一，否则完全与 01 会互相污染。

```
vector<int> mixedPack(int m, const vector<array<int, 3>> &a)
{
    // m 是容量；a[i] 是体积、价值、件数，件数 -1/0/正数分别表示 01/完全/多重物品。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto [w, v, c] : a)
    {
        assert(w > 0 && c >= -1); // 调试检查，可删。
        if (c == 0)
        {
            for (int j = w; j <= m; j++)
            {
                if (f[j - w] != -INF)
                {
                    chmax(f[j], f[j - w] + v);
                }
            }
            continue;
        }
```

```
        if (c == -1)
        {
            c = 1;
        }
        int k = 1;
        while (c > 0)
        {
            int z = min(k, c);
            c -= z;
            if (z <= m / w)
            {
                int ww = w * z;
                int vv = v * z;
                for (int j = m; j >= ww; j--)
                {
                    if (f[j - ww] != -INF)
                    {
                        chmax(f[j], f[j - ww] + vv);
                    }
                }
            }
            if (c > 0)
            {
                k = k <= c / 2 ? k * 2 : c;
            }
        }
    }
    return f;
}
```

调用 `mixedPack(m,a)`，第三维为 `-1`、`0`、正数分别表示 01、完全、多重物品。最坏时间 $O(m\sum_i \log(c_i+1))$、空间 $O(m)$，其中完全物品每种只花 $O(m)$。若全部物品同类，直接用对应的 12.1.001 / 12.1.002 / 12.1.003 更清晰；组内互斥仍须 12.1.006 分组背包，本函数不处理分组语义。

12.1.005 二维费用背包

二维费用背包每件物品同时消耗两种资源、只可选一次，状态升为 $f[x][y]$——恰好用掉第一维费用 x、第二维费用 y 时的最大价值，转移 $f[x][y] \leftarrow \max(f[x][y],\ f[x-a][y-b]+v)$。它就是把 01 背包的倒序思想推到两维：两层容量都必须倒序，才能保证同一件物品在本轮至多被计入一次。函数最后遍历整张表取最大值，得到” 两维都不超上限” 的答案。

```
int knapsack2(int cap1, int cap2, const vector<array<int, 3>> &a)
{
    // cap1、cap2 是两维容量；a[i][0..2] 依次是第一费用、第二费用、价值；返回不超容量的最大总价值。
    assert(cap1 >= 0 && cap2 >= 0); // 调试检查，可删。
    vector f(cap1 + 1, vector<int>(cap2 + 1, -INF));
    f[0][0] = 0;
    for (auto [x, y, v] : a)
    {
```

```
        assert(x >= 0 && y >= 0 && (x > 0 || y > 0)); // 调试检查，可删。
        for (int i = cap1; i >= x; i--)
        {
            for (int j = cap2; j >= y; j--)
            {
                if (f[i - x][j - y] != -INF)
                {
                    chmax(f[i][j], f[i - x][j - y] + v);
                }
            }
        }
    }
    int ans = 0;
    for (const auto &row : f)
    {
        for (int x : row)
        {
            chmax(ans, x);
        }
    }
    return ans;
}
```

调用 `knapsack2(cap1,cap2,a)`，每个 `array<int,3>` 的三维依次是第一费用、第二费用、价值，返回允许两维都有剩余时的最大价值。时间 $O(nAB)$、空间 $O(AB)$，其中 n 为物品数、A, B 为两维容量上限。一个样例：两维容量 $(4,5)$、物品 `{(2,3,6),(2,2,5),(3,2,7)}`，答案为 11。这是普通 01 背包 12.1.001 的直接推广，费用维数继续增加时按同样的倒序方式再加层即可，但空间随维数指数增长；它与依赖背包 12.1.007、子集和 12.1.010 是不同问题，不要混用。

12.1.006 分组背包

分组背包要求每组至多选一件。诀窍是每组开始前先把当前 f 快照成 `h`，组内每件物品都只从旧数组 `f` 转移、写进新数组 `h`，处理完整组再 `swap`。这样组内任何两件都来自同一份进组前状态，天然互斥；若图省事在同一数组里连续更新，本组前一件的结果会喂给后一件，导致同组多件被同时选中。允许整组不选，因为初始的 `h=f` 已保留” 不选本组” 的状态。

```
vector<int> groupPack(int m, const vector<vector<pair<int, int>>> &a)
{
    // m 是容量，a[i] 是第 i 组全部“体积、价值”物品；每组至多选一件，返回各恰好容量最大价值。
    assert(m >= 0); // 调试检查，可删。
    vector<int> f(m + 1, -INF);
    f[0] = 0;
    for (auto &g : a)
    {
        auto h = f;
        for (auto [w, v] : g)
        {
            assert(w > 0); // 调试检查，可删。
            for (int j = w; j <= m; j++)
```

```
            {
                if (f[j - w] != -INF)
                {
                    chmax(h[j], f[j - w] + v);
                }
            }
        }
        f.swap(h);
    }
    return f;
}
```

调用 `groupPack(m,a)`，外层每个 `a[i]` 是一组 (体积, 价值) 物品，返回各恰好容量的最大价值，允许整组不选。时间 $O(m\sum_i |G_i|)$、空间 $O(m)$。当每组只有一件时它退化成 01 背包 12.1.001；若组之间还存在” 选子必选父” 的层级约束，分组模型无法表达，须改用 12.1.007 有依赖背包。

12.1.007 有依赖背包

有依赖背包（金明的预算方案型）中物品构成父子依赖森林：选任何非根物品前必须先选它的父物品。树形 DP 用一个虚根 `rt` 把所有依赖树根挂在一起；真实节点只有在自身已选的状态上才能合并非空儿子状态，从而不会出现 “没选父亲却选了后代”。完整实现统一放在 6.5.004 树上依赖背包，本条只复用该接口，不再维护第二份同义代码。

`treeDepPack` 的接口为 `template<class Acc = long long> vector<optional<Acc>> treeDepPack(const vector<int> &w, const vector<Acc> &val, const vector<int> &par, int cap)`：`w[u]` 是正体积，`val[u]` 是价值，`par[u]` 是父节点编号，依赖树根用 `-1` 表示，`par` 必须构成森林，且 `cap >= 0`。返回长度为 `cap + 1` 的数组，其中第 `c` 项是恰好占用容量 `c` 且满足全部父子依赖的最大价值；不可达容量为 `nullopt`。求容量不超过 `cap` 的答案时只在有值的元素中取最大值。默认累计类型是 `long long`；当价值和可能超出 64 位时，显式传入 `treeDepPack<__int128>` 和 `vector<__int128>`。所有可达状态及其加法中间结果必须能由 `Acc` 表示。

```
// 依赖 6.5.004 树上依赖背包
signed main()
{
    vector<__int128> val{3, 5, 8};
    auto f = treeDepPack<__int128>(vector<int>{2, 2, 3}, val,
                                   vector<int>{-1, 0, 0}, 5);
    assert(f[5] && *f[5] == 11);
}
```

合并儿子的 max-plus 卷积保守时间为 $O(nC^2)$、额外空间为 $O(nC)$，其中 n 是物品数、$C = cap$；依赖链的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。样例中体积 `{2,2,3}`、价值 `{3,5,8}`、父节点 `{-1,0,0}`、容量 5 时最大值为 `11`（选点 `0` 与 `2`）。没有父子依赖时用 12.1.001 01 背包；只有组内至多选一时用 12.1.006 分组背包；树边只表示合并顺序、并不限制是否选父节点时用 12.1.009 树上背包及单调队列优化。

12.1.008 背包方案数与具体方案

这条在求 01 背包最优值的同时统计最优方案数并恢复一组具体方案。方案数是把最优值 DP 里的 `max` 换成”当价值相等时累加方案数”：`f[j]` 被严格刷新则方案数继承 `cnt[j-w]`，与旧值打平则 `cnt[j]+=cnt[j-w]`，对 `mod` 取模。方案恢复靠一条前驱链——每次严格刷新 `f[j]` 时把 (物品下标, 上一状态在 `path` 中的位置) 压进 `path`、并让 `at[j]` 指向它，最后从最优容量沿 `prev` 回溯即得被选物品序列。注意方案计数按物品编号区分、天然是”组合”意义（每件物品处理一次），若要”排列”数需改变枚举顺序。

```
struct PackResult
{
    // val 是不超过容量时的最优总价值。
    long long val;
    // ways 是模 mod 的最优方案数。
    int ways;
    // pick 是恢复出的一组物品下标，按输入顺序排列。
    vector<int> pick;
};

PackResult packPlan(int cap, const vector<pair<int, long long>> &a, int mod = 1000000007)
{
    // cap 是容量上限，a 是各 01 物品的体积与价值，mod 是计数模数；返回最优价值、方案数及一组方案。
    struct Path
    {
        // item 是本步选择的物品下标，pre 是上一步前驱节点编号。
        int item;
        int pre;
    };
    assert(cap >= 0 && mod > 0); // 调试检查，可删。
    int n = a.size();
    vector<long long> f(cap + 1, -INF);
    vector<int> cnt(cap + 1), at(cap + 1, -1);
    vector<Path> path;
    f[0] = 0;
    cnt[0] = 1;
    for (int i = 0; i < n; i++)
    {
        auto [w, v] = a[i];
        assert(w > 0); // 调试检查，可删。
        for (int j = cap; j >= w; j--)
        {
            if (f[j - w] == -INF)
            {
                continue;
            }
            long long x = f[j - w] + v;
            if (x > f[j])
            {
                f[j] = x;
                cnt[j] = cnt[j - w];
                path.push_back({i, at[j - w]});
```

```
                at[j] = path.size() - 1;
            }
            else if (x == f[j] && x > -INF)
            {
                cnt[j] = addMod(cnt[j], cnt[j - w], mod);
            }
        }
    }
    long long val = *max_element(f.begin(), f.end());
    int ways = 0;
    int last = -1;
    for (int j = 0; j <= cap; j++)
    {
        if (f[j] == val)
        {
            ways = addMod(ways, cnt[j], mod);
            last = at[j];
        }
    }
    vector<int> pick;
    while (last >= 0)
    {
        pick.push_back(path[last].item);
        last = path[last].pre;
    }
    reverse(pick.begin(), pick.end());
    return {val, ways, pick};
}
```

调用 `packPlan(cap,a,mod)`，容量非负、体积为正、模数为正，`a` 每项是 (体积, 价值)；返回 `PackResult{val,ways,pick}`，分别是最优价值、模 `mod` 的最优方案数和一组按输入顺序排列的物品下标。时间 $O(nC)$；状态数组占 $O(C)$，持久前驱节点最坏占 $O(nC)$。只要最优值不需要方案信息时直接用 12.1.00101 背包 更省内存；要把恢复空间严格压到 $O(C)$，可分治重算转移，而不能继续永久保存每次刷新。

12.1.009 树上背包及单调队列优化

这条给出树上背包中一个可被单调队列优化的清晰模型：每个树节点挂一类有件数上限的物品，先在本点线性完成这类多重背包，再把子树答案卷积合并上来。多重背包的单调队列优化按体积余数 r 把容量分成若干链 $r, r+w, r+2w, \dots$，在每条链上件数上限就化成滑动窗口长度：维护 `pre[x]-j*v`（旧价值减去到当前格所需件数的线性代价）的窗口最大值，窗口宽度即件数上限 cnt，于是本点转移降到 $O(C)$。子树之间仍是普通 max-plus 卷积。注意本函数的树边只决定合并顺序，不表示父子选取依赖，这与 12.1.007 不同。

```
template <class Acc = long long>
vector<optional<Acc>> treePack(const vector<vector<int>> &g,
                               const vector<array<long long, 3>> &a,
                               int root,
                               int cap)
{
```

```
    // a[u] 是 {体积, 价值, 件数}; 返回恰用各容量的最大价值, nullopt 表示不可达。
    int n = g.size();
    assert(n > 0 && (int)a.size() == n && 0 <= root && root < n && cap >= 0); // 调试检查, 可删
    auto dfs = [&](auto &&self, int u, int p) -> vector<optional<Acc>>
    {
        int w = a[u][0];
        Acc v = (Acc)a[u][1];
        int cnt = a[u][2];
        assert(w > 0 && cnt >= 0); // 调试检查, 可删
        vector<optional<Acc>> pre(cap + 1), f(cap + 1);
        pre[0] = (Acc)0;
        for (int r = 0; r < w && r <= cap; r++)
        {
            deque<int> q;
            for (int j = 0, x = r; x <= cap; j++, x += w)
            {
                while (!q.empty() && q.front() < j - cnt)
                {
                    q.pop_front();
                }
                if (pre[x])
                {
                    Acc cur = *pre[x] - (Acc)j * v;
                    while (!q.empty())
                    {
                        int y = r + q.back() * w;
                        Acc old = *pre[y] - (Acc)q.back() * v;
                        if (old > cur)
                        {
                            break;
                        }
                        q.pop_back();
                    }
                    q.push_back(j);
                }
                if (!q.empty())
                {
                    int y = r + q.front() * w;
                    f[x] = *pre[y] + (Acc)(j - q.front()) * v;
                }
            }
        }
        for (int x : g[u])
        {
            if (x == p)
            {
                continue;
            }
            auto h = self(self, x, u);
            auto nf = f;
            for (int i = 0; i <= cap; i++)
            {
```

```
                for (int j = 1; i + j <= cap; j++)
                {
                    if (f[i] && h[j])
                    {
                        Acc cur = *f[i] + *h[j];
                        if (!nf[i + j] || *nf[i + j] < cur)
                        {
                            nf[i + j] = cur;
                        }
                    }
                }
            }
            f.swap(nf);
        }
        return f;
    };
    return dfs(dfs, root, -1);
}
```

调用 `treePack(g,a,root,cap)`，`g` 必须是覆盖全部点的非空无向树，`a[u][0..2]` 是点 u 的物品体积、价值、件数上限；返回的 `f[c]` 是恰用容量 c 的最大价值，`nullopt` 表示不可达。默认状态、队列评分与卷积和都用 `long long`，必须保证这些中间值可表示；需要宽范围时调用 `treePack<__int128>`。单点树、物品 `{2,3,3}`、容量 7 时，容量 $0,2,4,6$ 的答案为 $0,3,6,9$。保守时间 $O(nC^2)$、额外空间 $O(nC)$；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。单节点的有界物品步骤是 $O(C)$。非树上的单个多重背包用 12.1.003 多重背包，单调队列原理见 12.3.001 单调队列优化。

12.1.010 布尔母函数 / 有界子集和可达性

有界子集和判定问题：每种权值 w_i 最多用 cnt_i 次，问某个目标和是否可达。二进制拆分能做，但这里用一个更巧的 $O(kS)$ 布尔母函数：`rem[s]` 记录”和 s 可达时，当前这种权值还剩几次可用”。处理权值 w_i 时正序扫 s——若 s 在上一轮已可达就把额度重置为 cnt_i（本轮一次没用）；否则若 $s-w_i$ 可达且那里还有剩余次数，就从它继承并消耗一次（`rem[s]=rem[s-w_i]-1`），两者都不满足则 s 不可达置 -1。这样无需拆分就限制了每种的使用次数。

```
bool boundSubset(const vector<int> &cnt, const vector<int> &w, int sum)
{
    // cnt、w 分别是每类物品数量和正权值，sum 是目标和；返回目标是否可达。
    assert(cnt.size() == w.size() && sum >= 0); // 调试检查，可删。
    for (int i = 0; i < (int)w.size(); i++)
    {
        assert(cnt[i] >= 0 && w[i] > 0); // 调试检查，可删。
    }
    vector<int> rem(sum + 1, -1);
    rem[0] = 0;
    for (int i = 0; i < (int)w.size(); i++)
    {
        for (int s = 0; s <= sum; s++)
        {
            if (rem[s] >= 0)
```

```
            {
                rem[s] = cnt[i];
            }
            else if (s < w[i] || rem[s - w[i]] <= 0)
            {
                rem[s] = -1;
            }
            else
            {
                rem[s] = rem[s - w[i]] - 1;
            }
        }
    }
    return rem[sum] >= 0;
}
```

调用 `boundSubset(cnt,w,sum)`，`cnt[i]`、`w[i]` 分别是第 i 种物品的非负数量与正权值、下标一一对应，`sum` 是目标和，返回布尔是否可达（原调用说明误写成 `array<int,2>` 输入与” 可达表” 返回，与函数签名不符，已按实际签名更正）。时间 $O(kS)$、空间 $O(S)$，k 为种类数、S 为目标和。只判可达性而不限件数时，用 12.2.008 子集和 Bitset 优化 的按字并行更快；需要方案数或最优价值则退回普通 12.1.003 多重背包。

12.1.011 基础区间 DP

区间 DP 以区间长度为阶段，$dp_{l,r}$ 表示半开区间 $[l,r)$ 的最优值，由所有分割点 $k\in(l,r)$ 的两段合并而来；记录最优分割点 `cut[l][r]` 后可以递归还原方案。这里给出三个基本模型：

- 石子合并：每次合并相邻两堆，代价为两堆之和，$dp_{l,r}=\min_k(dp_{l,k}+dp_{k,r})+\sum_{i=l}^{r-1}a_i$。环形把数组复制一倍，答案是所有长度为 n 的窗口中的最小值。
- 矩阵链乘：第 i 个矩阵为 $p_i\times p_{i+1}$，$dp_{l,r}=\min_k(dp_{l,k}+dp_{k,r}+p_lp_kp_r)$。
- 括号补全：$dp_{l,r}$ 是使 $[l,r)$ 合法的最少插入数；两端匹配时可取 $dp_{l+1,r-1}$，再与所有分割取最小值。

```
namespace interval_dp
{
struct Result
{
    __int128 cost = 0;        // cost 是最优总代价。
    int start = 0;            // start 是环形时最优的断环起点，链形为 0。
    vector<vector<int>> cut; // cut[l][r] 是区间 [l,r) 最优的分割点，长度为 1 的区间为 -1。

    vector<array<int, 3>> plan(int l, int r) const
    {
        // 按后序返回区间 [l,r) 的所有合并 {x,m,y}，表示把 [x,m) 与 [m,y) 合并。
        vector<array<int, 3>> res;
        auto dfs = [&](auto &&self, int x, int y) -> void
        {
            if (y - x <= 1)
            {
```

```
                return;
            }
            int m = cut[x][y];
            self(self, x, m);
            self(self, m, y);
            res.push_back({x, m, y});
        };
        dfs(dfs, l, r);
        return res;
    }
};

Result stones(vector<int> a, bool ring = false)
{
    // a 是各堆石子数（可为负），ring 表示首尾相邻；每次合并相邻两堆、代价为两堆之和，
    // 返回最小总代价与方案。环形把数组复制一倍后取所有长度为 n 的窗口。O(n^3)。
    int n = a.size();
    if (!n)
    {
        return {};
    }
    if (ring)
    {
        auto b = a;
        a.insert(a.end(), b.begin(), b.end());
    }
    int m = a.size();
    vector<__int128> pre(m + 1);
    for (int i = 0; i < m; i++)
    {
        pre[i + 1] = pre[i] + a[i];
    }
    vector<vector<__int128>> dp(m, vector<__int128>(m + 1)); // dp[l][r] 是合并 [l,r) 的最小代
↪   价。
    Result res;
    res.cut.assign(m, vector<int>(m + 1, -1));
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= m; l++)
        {
            int r = l + len;
            for (int k = l + 1; k < r; k++)
            {
                __int128 v = dp[l][k] + dp[k][r] + pre[r] - pre[l];
                // 求最大总代价：把这里的 v < dp[l][r] 和下面环形窗口的 dp[l][l + n] < res.cost 都改
↪                   成 >。
                if (k == l + 1 || v < dp[l][r])
                {
                    dp[l][r] = v;
                    res.cut[l][r] = k;
                }
            }
```

```
        }
    }
    res.cost = dp[0][n];
    for (int l = 1; ring && l < n; l++)
    {
        if (dp[l][l + n] < res.cost)
        {
            res.cost = dp[l][l + n];
            res.start = l;
        }
    }
    return res;
}

Result matrixChain(const vector<int> &p)
{
    // p 有 n+1 个正整数，第 i 个矩阵是 p[i] x p[i+1]；返回最少标量乘法次数与加括号方案。O(n^3)。
    assert(!p.empty()); // 调试检查，可删。
    int n = p.size() - 1;
    Result res;
    res.cut.assign(n, vector<int>(n + 1, -1));
    vector<vector<__int128>> dp(n, vector<__int128>(n + 1));
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= n; l++)
        {
            int r = l + len;
            for (int k = l + 1; k < r; k++)
            {
                __int128 v = dp[l][k] + dp[k][r] + (__int128)p[l] * p[k] * p[r];
                if (k == l + 1 || v < dp[l][r])
                {
                    dp[l][r] = v;
                    res.cut[l][r] = k;
                }
            }
        }
    }
    if (n)
    {
        res.cost = dp[0][n];
    }
    return res;
}

int brackets(const string &s)
{
    // s 只含 ()[]{}；返回最少插入多少个括号能使其合法。O(n^3)。
    // 只有一种括号时用贪心即可：未匹配的右括号数 + 剩余左括号数，O(n)。
    int n = s.size();
    vector<vector<int>> dp(n + 1, vector<int>(n + 1)); // dp[l][r] 是区间 [l,r) 的答案。
    for (int i = 0; i < n; i++)
```

```
    {
        dp[i][i + 1] = 1;
    }
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= n; l++)
        {
            int r = l + len;
            dp[l][r] = len;
            int i = string("()[]{}").find(s[l]); // 左括号在偶数位，其右括号紧随其后；找不到为 -1。
            if (i >= 0 && i % 2 == 0 && "()[]{}"[i + 1] == s[r - 1])
            {
                dp[l][r] = dp[l + 1][r - 1];
            }
            for (int k = l + 1; k < r; k++)
            {
                dp[l][r] = min(dp[l][r], dp[l][k] + dp[k][r]);
            }
        }
    }
    return dp[0][n];
}
}
```

`stones(a,ring)` 允许负数堆，返回最小代价、环形时最优窗口的起点 `start` 以及分割表；`plan(l,r)` 按后序给出合并步骤 $\{x, m, y\}$，环形时传入 `plan(start,start+n)`。`matrixChain(p)` 要求 p 非空且元素为正，`brackets(s)` 只接受 `()[]{}`。三者都是 $O(n^3)$ 时间、$O(n^2)$ 空间，代价用 `__int128` 累加。改求最大代价时按代码注释同步修改两处比较。

堆大小非负时石子合并满足四边形不等式，可以用 12.3.002 四边形不等式优化 降到 $O(n^2)$；有负数时该条件不成立，只能用这里的 $O(n^3)$。只有一种括号时贪心即可：未匹配的右括号数加上剩余左括号数。下面的环形样例中首尾两个 1 相邻，环形答案比链形更小：

```
signed main()
{
    auto s = interval_dp::stones({1, 9, 9, 1});
    auto r = interval_dp::stones({1, 9, 9, 1}, true);
    assert(s.cost == 40 && r.cost == 33 && r.start == 1);
    assert((s.plan(0, 4) == vector<array<int, 3>>{{0, 1, 2}, {2, 3, 4}, {0, 2, 4}}));
    assert(interval_dp::matrixChain({10, 30, 5, 60}).cost == 4500);
    assert(interval_dp::brackets("([)]") == 2);
}
```

12.2 常见 DP

12.2.001 数位 DP

数位 DP 从高位到低位扫描整数、统计 $[0, n]$ 中满足数位约束的个数。这份实现把” 约束” 抽象成一个自动机：调用者提供状态转移 `next(q,d,ns)` 和终态判定 `accept(q,st)`，本体只负责管理两个通用维

度——`lim`（是否仍贴着上界 n，贴界时最高位只能枚举到 n 的对应位）和 `st`（是否已经填过非零位，即是否走出前导零）。每读一位，就对 $(\mathrm{lim}, \mathrm{st}, q)$ 的计数向 $(\mathrm{nl}, \mathrm{ns}, \mathrm{nq})$ 分摊。前导零必须靠 `st` 单独标记而不能当普通的数字 0，否则形如” 数位递增” “无前导零段” 这类约束会算错；`lim` 维度保证不会数超过 n。

```cpp
template<class Next, class Accept>
unsigned long long digitDP(long long n, int m, int s, Next next, Accept accept)
{
    // n 是区间右端点，m 是状态数，s 是初态；next 转移状态，accept 判定终态；返回被接受的整数个数。
    if (n < 0)
    {
        return 0;
    }
    assert(m > 0 && 0 <= s && s < m); // 调试检查，可删。
    string a = to_string(n);
    vector dp(2, vector(2, vector<unsigned long long>(m)));
    dp[1][0][s] = 1;
    for (char c : a)
    {
        vector ndp(2, vector(2, vector<unsigned long long>(m)));
        for (int lim = 0; lim < 2; lim++)
        {
            for (int st = 0; st < 2; st++)
            {
                for (int q = 0; q < m; q++)
                {
                    for (int d = 0; d <= (lim ? c - '0' : 9); d++)
                    {
                        int nl = lim && d == c - '0';
                        int ns = st || d;
                        int nq = next(q, d, ns);
                        assert(0 <= nq && nq < m); // 调试检查，可删。
                        ndp[nl][ns][nq] += dp[lim][st][q];
                    }
                }
            }
        }
        dp.swap(ndp);
    }
    unsigned long long ans = 0;
    for (int lim = 0; lim < 2; lim++)
    {
        for (int st = 0; st < 2; st++)
        {
            for (int q = 0; q < m; q++)
            {
                if (accept(q, st))
                {
                    ans += dp[lim][st][q];
                }
            }
```

```
        }
    }
    return ans;
}
```

调用 `digitDP(n,m,s,next,accept)`：`n` 是区间右端点（$n < 0$ 返回 0），`m` 是正状态数，`s` 是初态；`next(q,d,ns)` 返回合法新状态，`accept(q,st)` 判断终态，其中 `ns`、`st` 表示是否已经读入非零位。返回类型能表示 $[0, 2^{63}-1]$ 的全部 2^{63} 个整数；求 $[l, r]$ 用两次调用之差。时间 $O(10m \log n)$、空间 $O(m)$。集合状态压缩用 12.2.002 状压 DP，棋盘轮廓局部转移用 12.2.011 插头 DP。

下面固定检查 “不含数字 4” 和数字 0 的 `st` 语义：

```
signed main()
{
    auto nextf = [](int q, int d, int ns)
    {
        // q 是当前状态，d 是本位数字，ns 表示读入后是否已开始；返回新状态。
        (void)ns;
        return q == 1 || d == 4;
    };
    auto no4 = [](int q, int st)
    {
        // q 是终态，st 表示是否已开始；返回该整数是否不含数字 4。
        (void)st;
        return q == 0;
    };
    assert(digitDP(49, 2, 0, nextf, no4) == 36);
    auto same = [](int q, int d, int ns)
    {
        // 这个单状态自动机不改变状态。
        (void)d;
        (void)ns;
        return q;
    };
    auto started = [](int q, int st)
    {
        // 只接受已经读入非零位的整数，因此排除数字 0。
        (void)q;
        return st;
    };
    assert(digitDP(0, 1, 0, same, started) == 0);
    auto all = [](int q, int st)
    {
        // 接受所有终态，包括始终未开始的数字 0。
        (void)q;
        (void)st;
        return true;
    };
    assert(digitDP(numeric_limits<long long>::max(), 1, 0, same, all)
           == (1ULL << 63));
    cout << "OK\n";
```

```
}
```

12.2.002 状压 DP

状压 DP 把”规模较小的集合中哪些元素已用”编码成位掩码，适合旅行商、集合划分、匹配这类未来只依赖已选集合而非顺序的问题。下例是最经典的旅行商：$dp[s][u]$ 表示已访问集合恰为 s、当前停在 u 的最小费用，转移枚举下一个未访问点 v：$dp[s\mid 2^v][v] \leftarrow dp[s][u]+w[u][v]$，最后在全集上加回到起点 0 的边取最小。$n \le 22$ 时状态数 $2^n n$ 尚可承受；复杂度必须按实际枚举的状态数与转移数算，不能笼统写成 2^n。

```cpp
// 知识点 12.2.002：旅行商状态压缩 DP。
namespace bitmask_dp
{
int tsp(const vector<vector<int>> &w)
{
    // w 是完整有向图权值方阵；返回从 0 出发访问所有点并回到 0 的最小费用。
    int n = w.size();
    assert(1 <= n && n <= 22); // 调试检查，可删。
    for (auto &row : w)
    {
        assert((int)row.size() == n); // 调试检查，可删。
    }
    if (n == 1)
    {
        return 0;
    }
    vector<vector<int>> dp(1LL << n, vector<int>(n));
    vector<vector<char>> ok(1LL << n, vector<char>(n));
    dp[1][0] = 0;
    ok[1][0] = true;
    for (int s = 1; s < (1LL << n); s++)
    {
        for (int u = 0; u < n; u++)
        {
            if (!(s >> u & 1) || !ok[s][u])
            {
                continue;
            }
            for (int v = 0; v < n; v++)
            {
                if (s >> v & 1)
                {
                    continue;
                }
                int ns = s | 1LL << v;
                int nd = dp[s][u] + w[u][v];
                if (!ok[ns][v] || nd < dp[ns][v])
                {
                    dp[ns][v] = nd;
```

```
                    ok[ns][v] = true;
                }
            }
        }
    }
    int ans = 0;
    bool found = false;
    for (int u = 0; u < n; u++)
    {
        if (ok.back()[u])
        {
            int cur = dp.back()[u] + w[u][0];
            if (!found || cur < ans)
            {
                ans = cur;
                found = true;
            }
        }
    }
    return ans;
}
}
```

调用 `bitmask_dp::tsp(w)`，`w` 是 $1 \le n \le 22$ 的完整有向图权值方阵，所有路径费用须能由 `int` 表示；返回从 0 出发遍历所有点再回到 0 的最小费用，单点图返回 0，不读取自环权值。时间 $O(2^n n^2)$、空间 $O(2^n n)$。集合划分可在此框架上改成枚举子集；整数各位约束用 12.2.001 数位 DP，只判子集和可达性时用 12.2.008 子集和 Bitset 优化。

12.2.003 棋盘分割

棋盘分割把矩形棋盘沿格线递归切成 k 个子矩形，最小化各块元素和的平方和。状态是当前子矩形四条边界与块数，转移枚举最后一刀及两侧块数分配；$k = 1$ 时代价是整块和的平方。记忆值用 `optional` 区分不可达状态，不拿某个可能被合法答案达到的整数充当无穷大。

```
// 知识点 12.2.003：矩形棋盘切成 k 块的最小平方和。
namespace board_part
{
template <class Acc = long long>
Acc splitBoard(const vector<vector<int>> &a, int parts)
{
    // a 是非空棋盘，parts 是块数；返回最小平方和。
    static_assert(numeric_limits<Acc>::is_integer && numeric_limits<Acc>::is_signed &&
                  sizeof(Acc) >= sizeof(long long));
    assert(!a.empty() && !a[0].empty()); // 调试检查，可删。
    int n = a.size();
    int m = a[0].size();
    assert(1 <= parts && parts <= n * m); // 调试检查，可删。
    vector<vector<Acc>> s(n + 1, vector<Acc>(m + 1));
    for (int i = 0; i < n; i++)
```

```
    {
        assert((int)a[i].size() == m); // 调试检查，可删。
        for (int j = 0; j < m; j++)
        {
            s[i + 1][j + 1] = s[i][j + 1] + s[i + 1][j] - s[i][j] + a[i][j];
        }
    }
    auto sum = [&](int x1, int y1, int x2, int y2)
    {
        // 四个参数定义半开子矩形；返回元素和。
        return s[x2][y2] - s[x1][y2] - s[x2][y1] + s[x1][y1];
    };
    // Key[0..4] 依次是上、左、下、右边界与块数。
    using Key = array<int, 5>;
    map<Key, optional<Acc>> memo;
    auto dfs = [&](auto &&self, int x1, int y1, int x2, int y2, int k) -> optional<Acc>
    {
        // self 是递归自身，四坐标定义矩形，k 是块数；返回最小平方和。
        Key key{x1, y1, x2, y2, k};
        if (auto it = memo.find(key); it != memo.end())
        {
            return it->second;
        }
        if (k == 1)
        {
            Acc x = sum(x1, y1, x2, y2);
            return memo[key] = x * x;
        }
        if (k > (x2 - x1) * (y2 - y1))
        {
            return memo[key] = nullopt;
        }
        optional<Acc> ans;
        for (int x = x1 + 1; x < x2; x++)
        {
            for (int p = 1; p < k; p++)
            {
                auto l = self(self, x1, y1, x, y2, p);
                auto r = self(self, x, y1, x2, y2, k - p);
                if (l && r && (!ans || *l + *r < *ans))
                {
                    ans = *l + *r;
                }
            }
        }
        for (int y = y1 + 1; y < y2; y++)
        {
            for (int p = 1; p < k; p++)
            {
                auto l = self(self, x1, y1, x2, y, p);
                auto r = self(self, x1, y, x2, y2, k - p);
                if (l && r && (!ans || *l + *r < *ans))
```

```
                {
                    ans = *l + *r;
                }
            }
        }
        return memo[key] = ans;
    };
    return *dfs(dfs, 0, 0, n, m, parts);
}
}
```

`splitBoard(a,parts)` 返回合法切分的最小平方和；需要宽累计时调用 `splitBoard<__int128>`。调用者须保证二维前缀和、任意区域和的平方及候选代价之和都能由 `Acc` 表示。内部 `optional` 只区分某个子矩形无法切出指定块数的状态，不暴露到公开接口。状态数 $O(n^2m^2k)$，最坏时间 $O(n^3m^2k^2+n^2m^3k^2)$，空间 $O(n^2m^2k+nm)$。

12.2.004 两台机器工作调度

两台相同机器并行做一批独立任务、最小化完工时间（makespan），本质是把总工时尽量对半分：设一台机器的负载为 s，另一台就是 $\text{sum}-s$，完工时间是二者较大者 $\max(s,\text{sum}-s)$，s 越接近 $\text{sum}/2$ 越优。于是问题化成”哪些负载 s 可由任务子集凑出”的 01 子集和可达性 DP，`dp[s]` 用布尔标记，最后在所有可达 s 上取 $\max(s,\text{sum}-s)$ 的最小值。

```
// 知识点 12.2.004：独立任务分配给两台机器的最短完工时间。
namespace twoMachSched
{
int twoMachines(const vector<int> &a)
{
    // a 是各任务的非负耗时；返回两台相同机器并行完成全部任务的最小 makespan。
    __int128 total = 0;
    for (int x : a)
    {
        assert(x >= 0); // 调试检查，可删。
        total += x;
    }
    // 本算法按总耗时分配数组，故总耗时还必须能作为容器下标。
    // 调试检查，可删。
    assert(total <= numeric_limits<int>::max()
           && total < (__int128)numeric_limits<size_t>::max());
    int sum = (int)total;
    vector<char> dp(sum + 1);
    dp[0] = 1;
    for (int x : a)
    {
        for (int s = sum; s >= x; s--)
        {
            dp[s] |= dp[s - x];
        }
    }
```

```
    int ans = sum;
    for (int s = 0; s <= sum; s++)
    {
        if (dp[s])
        {
            ans = min(ans, max(s, sum - s));
        }
    }
    return ans;
}
}
```

调用 `twoMachSched::twoMachines(a)`,`a` 是各任务的非负耗时,返回两台相同机器并行的最小 makespan。时间 $O(nS)$、空间 $O(S)$，S 为总耗时。子集和可达性判定本身可用 12.2.008 子集和 Bitset 优化 把常数压到 $1/64$；机器数超过两台时对半分不再成立，得转多维背包或状压。

12.2.005 找硬币问题

找硬币问题的关键是先分清模型：硬币是否可重复、顺序是否计入、要最少枚数还是方案数或只判可达，三者的初值与合并运算各不相同。下例是最常见的一种——面值无限、顺序无关，同时求最少枚数与无序方案数：外层枚举面值、内层容量正序（完全背包语义，一种面值可反复用），`best[s]=\min(best[s],best[s-x]+1)` 取最少枚数、`ways[s]+=ways[s-x]` 累加方案数。外层枚举面值而非金额，保证每种组合只按面值升序统计一次，得到的是无序方案数（若把两层循环对调则统计有序序列数）。

```
// 知识点 12.2.005：完全背包找硬币的最少枚数与方案数。
namespace coin_problem
{
array<int, 2> coinChange(const vector<int> &coin, int target, int mod)
{
    // coin 是不同正面值，target 是目标金额，mod 是方案数模数；返回最少枚数和无序方案数，无解时第一维为
    ↪ -1。
    assert(target >= 0 && mod > 0); // 调试检查，可删。
    set<int> seen;
    for (int x : coin)
    {
        assert(x > 0 && seen.insert(x).second); // 调试检查，可删。
    }
    const int inf = numeric_limits<int>::max() / 4;
    vector<int> best(target + 1, inf);
    vector<int> ways(target + 1);
    best[0] = 0;
    ways[0] = 1;
    for (int x : coin)
    {
        for (int s = x; s <= target; s++)
        {
            if (best[s - x] != inf)
            {
                best[s] = min(best[s], best[s - x] + 1);
```

```
            }
            ways[s] = addMod(ways[s], ways[s - x], mod);
        }
    }
    return {best[target] == inf ? -1 : best[target], ways[target]};
}
}
```

调用 `coin_problem::coinChange(coin,target,mod)`，`coin` 是不同正面值、`target` 是目标金额、`mod` 是方案数模数，返回 `{最少枚数，无序方案数}`，无解时第一维为 -1。时间 $O(kS)$、空间 $O(S)$，k 为面值数、S 为金额。硬币有件数上限时正序不再成立，改用 12.1.003 多重背包 的二进制拆分或 12.1.010 的剩余次数 DP；只关心某金额能否凑出用 12.2.008 子集和 Bitset 优化。

12.2.006 汉诺塔

经典三柱汉诺塔的解是递归的：“移 n 个盘 = 先把上 $n-1$ 个移到辅助柱，再移最大盘，再把 $n-1$ 个移回”，总步数 2^n-1。本条不做递归模拟，而是 $O(1)$ 回答” 第 k 步移哪个盘、从哪到哪”：观察到第 k 步移动的盘号等于 k 二进制末尾零的个数加一（`countr_zero(k)+1`），因为盘 i 每 2^i 步动一次；而某个盘每次移动的方向固定（由 n — disk 的奇偶决定顺时针或逆时针），出现次数 `before` 决定它当前在哪根柱上。多柱（Frame–Stewart）版本步数不再是 2^n-1，该公式不适用。

```
// 知识点 12.2.006：汉诺塔第 k 步移动。
namespace hanoi
{
array<int, 3> kthMove(int n, unsigned long long k)
{
    // n 是盘数，k 从 1 开始且不超过 2^n-1；返回盘号、起始柱和目标柱，柱编号为 0..2。
    assert(1 <= n && n < 64 && 1 <= k && k < (1ULL << n)); // 调试检查，可删。
    int disk = countr_zero(k) + 1;
    unsigned long long before = (k >> disk) + 1;
    int dir = ((n - disk) & 1) ? 1 : -1;
    int from = (dir * (int)(before - 1) % 3 + 3) % 3;
    int to = (from + dir + 3) % 3;
    return {disk, from, to};
}
}
```

调用 `hanoi::kthMove(n,k)`，n 是盘数（$1 \le n < 64$）、k 从 1 开始且不超过 2^n-1，返回 `{盘号，起始柱，目标柱}`，柱编号 $0 \dots 2$。时间与空间 $O(1)$。要输出整段移动序列时逐 k 调用即得，总输出量 2^n-1 是指数级、复杂度须按输出规模计而非递归层数；多柱最优步数问题不在此列。

12.2.007 最长上升/公共子序列

最长上升子序列（LIS）与最长公共子序列（LCS）状态和复杂度不同。LIS 维护 “长度为 $k+1$ 的上升子序列的最小末值” `tail[k]`，对每个元素用 `lower_bound` 找到应替换的位置；`at[k]` 记录该末值来自哪个下标，`pre[i]` 记录方案前驱，从最后一个末值即可恢复一条严格 LIS。LCS 是两序列前缀 DP，其完整实现统一放在 4.5.001 最长公共子序列，本条不再复制。

```cpp
template<class T>
vector<T> lis(const vector<T> &a)
{
    // a 是输入序列；返回一条严格最长上升子序列。
    int n = a.size();
    vector<T> tail;
    vector<int> at;
    vector<int> pre(n, -1);
    for (int i = 0; i < n; i++)
    {
        int k = lower_bound(tail.begin(), tail.end(), a[i]) - tail.begin();
        if (k > 0)
        {
            pre[i] = at[k - 1];
        }
        if (k == (int)tail.size())
        {
            tail.push_back(a[i]);
            at.push_back(i);
        }
        else
        {
            tail[k] = a[i];
            at[k] = i;
        }
    }
    if (at.empty())
    {
        return {};
    }
    vector<T> ans;
    for (int x = at.back(); x != -1; x = pre[x])
    {
        ans.push_back(a[x]);
    }
    reverse(ans.begin(), ans.end());
    return ans;
}
```

`lis(a)` 返回一条严格最长上升子序列，空输入返回空序列；把 `lower_bound` 改成 `upper_bound` 可求最长不下降子序列。时间 $O(n\log n)$、额外空间 $O(n)$。普通 LCS、Bitset LCS 和 SCS 使用 4.5.001 最长公共子序列的 `lcs`、`lcsBitset` 与 `scs`，不要在本章复制实现。

12.2.008 子集和 Bitset 优化

子集和可达性判定用一个位集表示” 哪些和可达”，加入体积 w 时执行 `bits |= bits << w`——第 s 位为 1 表示和 s 可达，左移 w 位再或回去，就等价于” 每个原可达和再加一件 w “。这样把布尔 DP 的每字 64 个状态并行处理，常数降到 $1/64$。`SubsetSum` 手写了跨 `unsigned long long` 字的移位（`q=w>>6` 是整字偏移、`r=w&63` 是字内偏移，跨字时补上前一字被移出的高位），并在每次 `add` 后把超过上限 n 的高位掩掉。注意这条 `bits |= bits<<w` 是 01 物品语义（每次调用只加一件）；完全背包那种” 同一件反复

取” 不能靠单次移位表达，需在字内做前缀 or。

```
struct SubsetSum
{
    // n 是维护的最大子集和。
    int n;
    // a 的第 i 位表示子集和 i 当前是否可达。
    vector<unsigned long long> a;

    static size_t words(int n)
    {
        // n 是最大子集和；返回位集所需的机器字数。
        assert(n >= 0); // 调试检查，可删。
        return (size_t)(n / 64 + 1);
    }

    SubsetSum(int n) : n(n), a(words(n))
    {
        // n 是维护的最大子集和；初始时只有和 0 可达。
        a[0] = 1;
    }

    void add(int x)
    {
        // x 是新加入的一件 01 物品的非负重量；更新可达和。
        assert(x >= 0); // 调试检查，可删。
        int q = x >> 6, r = x & 63;
        for (int i = (int)a.size() - 1; i >= q; i--)
        {
            unsigned long long z = a[i - q] << r;
            if (r && i > q)
            {
                z |= a[i - q - 1] >> (64 - r);
            }
            a[i] |= z;
        }
        if ((n & 63) != 63)
        {
            a.back() &= (1ULL << ((n & 63) + 1)) - 1;
        }
    }

    bool has(int x) const
    {
        // 参数 x 表示 要查询是否可达的子集和；返回目标子集和 x 当前是否可达。
        return 0 <= x && x <= n && (a[x >> 6] >> (x & 63) & 1);
    }
};
```

用 `SubsetSum ss(n)` 建对象（n 是需维护的最大和），`ss.add(x)` 加入一件体积 x，`ss.has(x)` 查询和 x 是否可达。加入 k 件的总时间 $O\left(\frac{kS}{64}\right)$、空间 $O\left(\frac{S}{64}\right)$，$S$ 为上限。它是 12.2.004 两台机器工作调度 等可达性问题的加速器；需要方案数或最优价值时它不适用，回到普通 01 背包 12.1.001；有件数上限的可达

性判定用 12.1.010。

下面检查跨越 64 位块的移位与末字掩码：

```
signed main()
{
    SubsetSum s(65);
    for (int x : {1, 63, 64})
    {
        s.add(x);
    }
    assert(s.has(0) && s.has(64) && s.has(65) && !s.has(62));
}
```

12.2.009 树形 DP 与动态 DP

树上最大权独立集的静态递推是 $f_{u,0}=\sum_v \max(f_{v,0},f_{v,1})$、$f_{u,1}=w_u+\sum_v f_{v,0}$，其中 $0/1$ 表示不选或选择 u。代码用递归 lambda 在回溯时合并子树状态；链形树的递归深度为 $O(n)$，实际可承受深度取决于评测环境的线程栈。支持单点修改的动态版本要把重链转移写成 max-plus 矩阵并维护矩阵积，完整实现见 6.4.005 动态 DP，本章不重复打印其 `GlobalTree` 底层。

```
template <class Acc = long long>
Acc treeMWIS(const vector<vector<int>> &g, const vector<Acc> &w, int root = 0)
{
    // g 是无向树，w 是点权，root 是根；返回允许空集的最大权独立集权值。
    int n = g.size();
    assert((int)w.size() == n); // 调试检查，可删。
    if (n == 0)
    {
        return 0;
    }
    assert(0 <= root && root < n); // 调试检查，可删。
    int deg = 0;
    for (int u = 0; u < n; u++)
    {
        deg += g[u].size();
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
        }
    }
    assert(deg == 2 * (n - 1)); // 调试检查，可删。
    vector<int> par(n, -2);
    int seen = 0;
    auto dfs = [&](auto &&self, int u, int p) -> array<Acc, 2>
    {
        par[u] = p;
        seen++;
        array<Acc, 2> f{0, w[u]};
        for (int v : g[u])
```

```
        {
            if (v == p)
            {
                continue;
            }
            assert(par[v] == -2); // 调试检查，可删。
            auto h = self(self, v, u);
            f[0] += max(h[0], h[1]);
            f[1] += h[0];
        }
        return f;
    };
    auto f = dfs(dfs, root, -1);
    assert(seen == n); // 调试检查，可删。
    return max(f[0], f[1]);

}
```

调用 `treeMWIS(g,w,root)` 求静态树的最大权独立集，空树返回 0，非空输入必须是覆盖全部点的无向树；默认点权、DP 状态与返回值均为 `long long`，必须保证所有子树状态能装入 `Acc`。需要宽答案时调用 `treeMWIS<__int128>` 并传入 `vector<__int128>` 点权。时间和额外空间均为 $O(n)$。需要修改点权时使用 6.4.005 动态 DP 的 `DynamicMWIS`：预处理 $O(n \log n)$、单点修改 $O(\log n)$、查询 $O(1)$。其他树形状态不能只替换矩阵元素，必须重新证明链上转移满足结合律。

12.2.010 不同子序列计数

统计一个序列的不同子序列数量（含空串则加一）。设 sum 为” 当前所有不同子序列数”（初值 1，代表空串），逐元素处理：加入元素 x 时，理论上每个已有子序列后面接一个 x 都产生新串，于是 sum 翻倍；但若 x 之前出现过，上次处理 x 那一刻已经统计过” 以 x 结尾” 的那批串，它们会被重复计入，须减去 x 上次出现前的 sum。故递推 $\text{sum} \leftarrow 2\,\text{sum} - \text{last}[x]$，其中 `last[x]` 记 x 上次处理前的 sum。全程对 `mod` 取模，减法先加 `mod` 防负。

```
// 返回不同非空子序列个数，值相同的元素视为相同字符。
int distinctSub(const vector<long long> &a, int mod = 998244353)
{
    // a 是输入序列，mod 是正模数；返回不同非空子序列数量模 mod。
    assert(mod > 0); // 调试检查，可删。
    unordered_map<long long, int> last;
    int sum = 1 % mod;
    for (long long x : a)
    {
        int old = last[x];
        int twice = sum >= mod - sum ? sum - (mod - sum) : sum + sum;
        int next = twice - old;
        if (next < 0)
        {
            next += mod;
        }
        last[x] = sum;
```

```
        sum = next;
    }
    return sum == 0 ? mod - 1 : sum - 1;
}
```

调用 `distinctSub(a,mod)`，`a` 是输入序列（值相同即视为同字符），返回不同非空子序列个数模 `mod`（末尾 -1 去掉空串）。期望 $O(n)$ 时间、$O(k)$ 空间，k 为不同值数；用 `unordered_map` 存 `last`，哈希退化时最坏 $O(n^2)$，值域小时可换定长数组稳定到 $O(n)$。它统计的是” 去重后的子序列数”，与不去重的子序列计数（直接 2^n）是不同问题，别混用。

12.2.011 插头 DP

沿棋盘扫描线（轮廓线）逐格转移，状态只保存轮廓上每个位置的插头——即回路/路径穿过该格线的连通信息。相邻状压 DP 只记「有无」，插头 DP 还要记连通性，用于铺砖、哈密顿回路、连通块构造这类要求整体连通的问题。连通性用括号表示法编码：轮廓上有 $m+1$ 个插头，每个取值 $\{0$ 无，1 左括号(，2 右括号$)\}$，同一条链的两端记成一对匹配括号，2 bit 一个插头压进一个整数。

下例是最经典的 URAL1519 型问题：用一条闭合回路恰好覆盖棋盘全部空格，求方案数。处理格 (i,j) 时看它的左插头 $L=\text{plug}[j]$ 与上插头 $U=\text{plug}[j+1]$，按二者取值分情形转移（下插头写回 $\text{plug}[j]$、右插头写回 $\text{plug}[j+1]$）：

- $L=U=0$：本格是回路拐角「，新建一对插头，下 $=($、右 $=)$；
- 恰一个非空：回路直行或拐弯，把该插头改道向右或向下（两种转移）；
- $L=U=1$（两左括号）：两段合并，需把 U 匹配的) 改成 (；
- $L=U=2$（两右括号）：把 L 匹配的 (改成)；
- $L=2,U=1$ $()$：两条不同链首尾相连，直接消去两插头；
- $L=1,U=2$ $(())$：相邻括号互为配对，闭合成回路——只有在最后一个空格、且闭合后轮廓再无其它插头时才计入答案（否则会漏成多条回路）。

障碍格要求左、上插头都为空。每行处理完要求最右插头为空，再整体左移一位对齐下一行。

```
// 插头 DP（括号表示法）：统计用一条闭合回路覆盖棋盘全部空格的方案数（URAL1519 型）。
// 轮廓线上有 m+1 个插头，每个取值 {0 无, 1 左括号'(', 2 右括号')'}，2 bit 编码。
template <class Acc = long long>
struct PlugDP
{
    int n; // n 是棋盘行数。
    int m; // m 是棋盘列数。
    vector<vector<int>> a; // a[i][j] 为 1 表示障碍，0 表示必须覆盖的空格。

    PlugDP(vector<vector<int>> g) : n(g.size()), m(0), a(move(g))
    {
        // g 是矩形棋盘，1 表示障碍，0 表示必须覆盖的空格；初始化棋盘。
        assert(n > 0 && !a[0].empty()); // 调试检查，可删。
        m = a[0].size();
        assert(2 * (m + 1) <= 64); // 调试检查，可删。
        for (int i = 0; i < n; i++)
        {
```

```
            assert((int)a[i].size() == m); // 调试检查，可删。
        }
    }

    int get(unsigned long long s, int p) const
    {
        // s 是插头状态，p 是插头位置；返回该位置的 0/1/2 编码。
        return (s >> (2 * p)) & 3;
    }

    unsigned long long set(unsigned long long s, int p, int v) const
    {
        // s 是插头状态，p 是插头位置，v 是新的 0/1/2 编码；返回修改后的状态。
        return (s & ~(3ULL << (2 * p))) | ((unsigned long long)v << (2 * p));
    }

    Acc solve()
    {
        // 无参数；返回覆盖全部空格且只有一条闭合回路的方案数。
        int ex = -1;
        int ey = -1;
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < m; j++)
            {
                if (!a[i][j])
                {
                    ex = i;
                    ey = j;
                }
            }
        }
        if (ex < 0)
        {
            return 0;
        }

        map<unsigned long long, Acc> cur;
        cur[0] = 1;
        Acc ans = 0;
        for (int i = 0; i < n; i++)
        {
            map<unsigned long long, Acc> nxt;
            for (auto [s, c] : cur)
            {
                nxt[s << 2] += c;
            }
            cur = nxt;
            for (int j = 0; j < m; j++)
            {
                map<unsigned long long, Acc> nx;
                for (auto [s, c] : cur)
```

```
                {
                    int L = get(s, j), U = get(s, j + 1);
                    if (a[i][j])                        // 障碍：左、上插头必须都空
                    {
                        if (!L && !U)
                        {
                            nx[s] += c;
                        }
                        continue;
                    }
                    if (!L && !U)                       // ┌：新建一对插头
                    {
                        if (i + 1 < n && j + 1 < m)
                        {
                            nx[set(set(s, j, 1), j + 1, 2)] += c;
                        }
                    }
                    else if (!L || !U)                  // 直行/拐弯：单插头可向右或向下
                    {
                        int t = L ? L : U;
                        if (j + 1 < m)
                        {
                            nx[set(set(s, j, 0), j + 1, t)] += c;
                        }
                        if (i + 1 < n)
                        {
                            nx[set(set(s, j, t), j + 1, 0)] += c;
                        }
                    }
                    else if (L == 1 && U == 1)          // ( ( ：找 U 的配对 ) 改成 (
                    {
                        unsigned long long t = set(set(s, j, 0), j + 1, 0);
                        int cnt = 1;
                        for (int k = j + 2; k <= m; k++)
                        {
                            int v = get(t, k);
                            if (v == 1)
                            {
                                cnt++;
                            }
                            else if (v == 2)
                            {
                                cnt--;
                                if (cnt == 0)
                                {
                                    t = set(t, k, 1);
                                    break;
                                }
                            }
                        }
                        nx[t] += c;
                    }
```

```
                    else if (L == 2 && U == 2)          // ) ) : 找 L 的配对 ( 改成 )
                    {
                        unsigned long long t = set(set(s, j, 0), j + 1, 0);
                        int cnt = 1;
                        for (int k = j - 1; k >= 0; k--)
                        {
                            int v = get(t, k);
                            if (v == 2)
                            {
                                cnt++;
                            }
                            else if (v == 1)
                            {
                                cnt--;
                                if (cnt == 0)
                                {
                                    t = set(t, k, 2);
                                    break;
                                }
                            }
                        }
                        nx[t] += c;
                    }
                    else if (L == 2 && U == 1)          // ) ( : 两条链合并，两插头消去
                    {
                        nx[set(set(s, j, 0), j + 1, 0)] += c;
                    }
                    else                                // ( ) : 相邻配对，闭合回路
                    {
                        // 仅当此为末空格、且闭合后轮廓再无其它插头（整条回路收尾）才计入
                        if (i == ex && j == ey && set(set(s, j, 0), j + 1, 0) == 0)
                        {
                            ans += c;
                        }
                    }
                }
                cur = nx;
            }
            map<unsigned long long, Acc> f;
            for (auto [s, c] : cur)
            {
                if (!get(s, m))
                {
                    f[s] += c;
                }
            }
            cur = f;
        }
        return ans;
    }
};
```

用 `PlugDP q(g)` 构造非空矩形棋盘，1 表示障碍、0 表示必须覆盖的空格，宽度须满足 $m \le 31$；默认 `q.solve()` 返回 `long long` 方案数，必须保证方案数累加能装入 `Acc`，可能超过 64 位时使用 `PlugDP<__int128>`。全为障碍时返回 0。状态数随轮廓宽度指数增长，应把较小的一维放在宽度方向。铺砖、指定端点路径和最优权值需要重新定义插头转移，不能只改计数合并运算。

下面用三张全空棋盘的已知答案和一张带障碍图检查主要转移：

```
signed main()
{
    vector<array<int, 3>> chk{{2, 3, 1}, {3, 3, 0}, {4, 4, 6}};
    for (auto [n, m, want] : chk)
    {
        vector<vector<int>> g(n, vector<int>(m));
        assert(PlugDP(g).solve() == want);
    }
    vector<vector<int>> g{{0, 0, 0, 0}, {0, 1, 1, 0}, {0, 0, 0, 0}};
    assert(PlugDP(g).solve() == 1);
    cout << "OK\n";
}
```

设轮廓合法状态数为 S，时间 $O(nmS)$、额外空间 $O(S)$。

12.3 DP 优化

12.3.001 单调队列优化

当转移是 $f[i] = c_i + \min_{i-k \le j < i} f[j]$ 时，只需维护最近 k 个 $f[j]$ 的最小值。双端队列按 f 值单调递增：队首过期就弹出，新状态入队前弹掉所有不优于它的队尾；每个下标至多进出各一次，便把朴素 $O(nk)$ 降到 $O(n)$。通用的窗口最值、单调栈和最大矩形实现见 2.2.008 单调栈 / 单调队列，本条只保留 DP 转移。

```
vector<int> windowDP(const vector<int> &c, int k)
{
    // c[i] 是进入状态 i 的代价，k 是前驱窗口长度；返回 f[0]=c[0] 的最小代价 DP。
    if (c.empty())
    {
        return {};
    }
    assert(k > 0); // 调试检查，可删。
    int n = c.size();
    vector<int> f(n);
    deque<int> q;
    f[0] = c[0];
    q.push_back(0);
    for (int i = 1; i < n; i++)
    {
        while (!q.empty() && q.front() < i - k)
        {
            q.pop_front();
        }
```

```
        f[i] = f[q.front()] + c[i];
        while (!q.empty() && f[q.back()] >= f[i])
        {
            q.pop_back();
        }
        q.push_back(i);
    }
    return f;
}
```

`windowDP(c,k)` 返回上述递推的全部状态，空代价序列返回空；非空时 $k>0$，所有状态值须能由 `int` 表示。时间 $O(n)$、额外空间 $O(n)$。代价满足四边形不等式的区间合并用 12.3.002 四边形不等式优化，最优决策随位置单调用 12.3.003 分治优化，转移写成直线求最值用 12.3.004 斜率优化 / 李超树优化。

12.3.002 四边形不等式优化

半开区间合并递推为 $dp[l][r]=\min_{l<k<r}\left(dp[l][k]+dp[k][r]\right)+w(l,r)$，单元素基例 $dp[i][i+1]=0$。若代价 w 满足四边形不等式（$w(a,c)+w(b,d)\le w(a,d)+w(b,c)$，$a\le b\le c\le d$）且区间单调，则最优决策点满足 $\mathrm{opt}[l][r-1]\le\mathrm{opt}[l][r]\le\mathrm{opt}[l+1][r]$。Knuth 优化把断点限制在这两个旧决策之间，总枚举量降到 $O(n^2)$；使用前必须证明条件。

```
// 知识点 12.3.002：满足 Knuth 条件的区间 DP。
namespace quad_opt
{
template <class Acc = long long, class Cost>
Acc knuth(int n, Cost w)
{
    // n 是序列长度，w(l,r) 返回半开区间代价；返回 dp[0][n]，调用者必须证明 Knuth 单调性条件。
    assert(n >= 0); // 调试检查，可删。
    vector<vector<Acc>> dp(n + 1, vector<Acc>(n + 1));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1));
    for (int i = 0; i < n; i++)
    {
        opt[i][i] = i;
        opt[i][i + 1] = i + 1;
    }
    opt[n][n] = n;
    for (int len = 2; len <= n; len++)
    {
        for (int l = 0; l + len <= n; l++)
        {
            int r = l + len;
            optional<Acc> val;
            int lo = max(l + 1, opt[l][r - 1]);
            int hi = min(r - 1, opt[l + 1][r]);
            for (int k = lo; k <= hi; k++)
            {
                Acc cur = dp[l][k] + dp[k][r] + (Acc)w(l, r);
                if (!val || cur < *val)
```

```
                {
                    val = cur;
                    opt[l][r] = k;
                }
            }
            assert(val); // Knuth 条件下该区间至少有一个候选。
            dp[l][r] = *val;
        }
    }
    return dp[0][n];
}
}
```

`knuth(n,w)` 默认返回 `long long` 的 $dp[0][n]$；$n = 0$ 或 1 返回 0。调用者须证明 Knuth 条件，并保证区间代价和候选加法能由 `Acc` 表示；需要宽状态时调用 `knuth<__int128>`。局部 `optional` 只表示本区间尚未枚举候选，因此最优值恰等于类型上界也不会和哨兵混淆。时间、空间均为 $O(n^2)$。

下面检查空序列、单元素与三堆石子的已知最优值：

```
signed main()
{
    auto cost = [](const vector<int> &a)
    {
        int n = a.size();
        vector<int> pre(n + 1);
        for (int i = 0; i < n; i++)
        {
            pre[i + 1] = pre[i] + a[i];
        }
        auto w = [&](int l, int r)
        {
            return pre[r] - pre[l];
        };
        return quad_opt::knuth(n, w);
    };
    assert(cost({}) == 0);
    assert(cost({7}) == 0);
    assert(cost({1, 2, 3}) == 9);
    assert(cost({4, 3, 3, 4}) == 28);
}
```

12.3.003 分治优化

分治优化针对分层转移 $dp[i] = \min_{j<i}\left(\mathrm{pre}[j] + w(j,i)\right)$，当本层最优决策点 $\mathrm{opt}(i)$ 随 i 单调不减时适用。做法是分治处理下标区间 $[l,r]$：先算中点 mid，在给定候选决策区间 $[ql,qr]$ 里暴力找出它的最优决策 $best$；由单调性，$[l,mid-1]$ 的最优决策都 $\le best$、$[mid+1,r]$ 的都 $\ge best$，于是递归时把候选区间分别收窄到 $[ql,best]$ 和 $[best,qr]$。每层递归的决策枚举量之和是 $O(n)$、递归 $O(\log n)$ 层，故代价函数被调用 $O(n\log n)$ 次。它与 Knuth 的区别：这里是”一层依赖上一层”的转移，Knuth 是”区间拆两段”的转移。

```cpp
// 知识点 12.3.003：决策单调的分治优化。
namespace dc_dp
{
template <class Acc = long long, class Cost>
vector<Acc> divideLayer(const vector<Acc> &pre, int n, Cost w)
{
    // pre[j] 是上一层状态，n 是本层最大下标，w(j,i) 是转移代价；返回本层 dp，调用者必须保证最优决策单调。
    assert(n >= 0 && (int)pre.size() == n + 1); // 调试检查，可删。
    vector<Acc> dp(n + 1);
    auto solve = [&](auto &&self, int l, int r, int ql, int qr) -> void
    {
        // self 是递归自身，[l,r] 是待算区间，[ql,qr] 是候选决策范围；填充 dp，无返回值。
        if (l > r)
        {
            return;
        }
        int mid = l + (r - l) / 2;
        int best = ql;
        optional<Acc> val;
        for (int j = ql; j <= min(qr, mid - 1); j++)
        {
            Acc cur = pre[j] + (Acc)w(j, mid);
            if (!val || cur < *val)
            {
                val = cur;
                best = j;
            }
        }
        assert(val); // 候选区间非空，且 pre 中没有不可达状态。
        dp[mid] = *val;
        self(self, l, mid - 1, ql, best);
        self(self, mid + 1, r, best, qr);
    };
    solve(solve, 1, n, 0, n - 1);
    return dp;
}
}
```

`divideLayer(pre,n,w)` 要求 `pre[0..n-1]` 全部是可达状态，且最优决策单调；默认返回 `long long` 的本层结果。若上一层含不可达状态，应先过滤候选或另用 `optional` 状态，不能拿类型上界传入本函数。所有转移和须能由 `Acc` 表示；需要宽状态时调用 `divideLayer<__int128>`。单层调用代价函数 $O(n \log n)$ 次，额外空间 $O(n + \log n)$。

下面检查空层、单点、并列决策与平方代价的已知结果：

```cpp
signed main()
{
    auto solve = [](const vector<int> &s, const vector<int> &pre)
    {
        int n = (int)s.size() - 1;
```

```
        auto w = [&](int j, int i)
        {
            long long d = s[i] - s[j];
            return d * d;
        };
        return dc_dp::divideLayer(pre, n, w);
    };
    assert(solve({0}, {0}) == vector<long long>({0}));
    assert(solve({0, 5}, {7, 0}) == vector<long long>({0, 32}));
    assert(solve({0, 1, 2, 3}, {0, 0, 0, 0}) == vector<long long>({0, 1, 1, 1}));
}
```

12.3.004 斜率优化 / 李超树优化

把转移 $dp[i] = \min_j(a_j x_i + b_j)$ 中的每个决策看成直线即可做斜率优化。若斜率按不增顺序加入、查询横坐标也不减，维护下凸壳的双端队列就能均摊 $O(1)$：新线使队尾交点次序逆转时弹尾，查询时第二条线不劣于第一条便弹头。求最大值时只有在斜率、截距和函数值都能安全取反时，才把每条线同时取反、复用本最小值凸壳并对非空查询结果取反；这样斜率加入顺序也随取反自然变成原斜率不减，不能只把查询比较改成 `>=`，遇到最小负数要完整反向比较。斜率或查询无序时改用 3.7.001 李超线段树，本章不复制数据结构实现。

```
template <class Acc = long long>
struct MonoHull
{
    // 当前凸壳按最小函数值维护。改求最大值可在 k、b 与函数值都能安全取反时同时取反，
    // 继续调用本结构，最后把非空 query 结果取反；否则必须完整反向交点与比较判定。
    using U128 = unsigned __int128;

    struct Line
    {
        // k、b 是直线 y=kx+b 的斜率和截距。
        int k;
        int b;

        Acc get(int x) const
        {
            // x 是查询横坐标；返回直线在 x 处的函数值。
            return (Acc)k * x + b;
        }
    };
    // q 是按交点顺序排列的下凸壳，last 是上一次查询横坐标。
    deque<Line> q;
    optional<int> last;

    static pair<bool, U128> product(int a, int b, int c, int d)
    {
        // 返回 (a-b)(c-d) 的符号与绝对值，true 表示负数。
        auto diff = [](int x, int y)
        {
```

```
            if (x >= y)
            {
                return pair{false, (unsigned long long)x - (unsigned long long)y};
            }
            return pair{true, (unsigned long long)y - (unsigned long long)x};
        };
        auto x = diff(a, b), y = diff(c, d);
        return {x.first != y.first, (U128)x.second * y.second};
    }

    static bool less(pair<bool, U128> a, pair<bool, U128> b)
    {
        // 比较两个符号-绝对值表示的整数。
        if (a.first != b.first)
        {
            return a.first;
        }
        return a.first ? a.second > b.second : a.second < b.second;
    }

    void add(int k, int b)
    {
        // k、b 是新直线的斜率和截距；要求斜率按不增顺序加入。
        assert(q.empty() || k <= q.back().k); // 调试检查，可删。
        if (!q.empty() && k == q.back().k)
        {
            if (q.back().b <= b)
            {
                return;
            }
            q.pop_back();
        }
        Line c{k, b};
        while (q.size() >= 2)
        {
            Line a = q[q.size() - 2];
            Line d = q.back();
            auto l = product(d.b, a.b, d.k, c.k);
            auto r = product(c.b, d.b, a.k, d.k);
            if (less(l, r))
            {
                break;
            }
            q.pop_back();
        }
        q.push_back(c);
    }

    optional<Acc> query(int x)
    {
        // x 是不小于上次查询的横坐标；无直线时返回空，否则返回最小函数值。
        assert(!last || *last <= x); // 调试检查，可删。
```

```
        last = x;
        while (q.size() >= 2 && q[1].get(x) <= q[0].get(x))
        {
            q.pop_front();
        }
        if (q.empty())
        {
            return nullopt;
        }
        return q.front().get(x);
    }
};
```

用 `MonoHull h` 建空凸壳，斜率不增插入、横坐标不减查询。函数值默认用 `long long`，需要宽答案时使用 `MonoHull<__int128>`；交点比较把完整 64 位差的绝对值乘积放进 `unsigned __int128`，不会溢出有符号 128 位。无直线时返回空；插入和查询均摊 $O(1)$，空间 $O(q)$。顺序不单调时改用 3.7.001 李超线段树。

12.3.005 WQS 二分

WQS 二分（又称带权二分 / Aliens trick）处理「恰好取 K 个」且最优值关于 K 下凸的问题。设 $g(K)$ 为恰取 K 个的最优代价，g 下凸。去掉个数限制、给每次「取」附加一个罚 λ，做无约束最优化

$$h(\lambda) = \min_{c} \left(g(c) + \lambda c \right).$$

λ 越大越抑制取用，最优个数 $c^*(\lambda)$ 单调不增；二分 λ 使 $c^* = K$，再由凸共轭关系

$$g(K) = \max_{\lambda} \left(h(\lambda) - \lambda K \right)$$

反推真实代价。由于斜率 $g(c) - g(c+1)$ 为整数，整数 λ 足以命中每个凸包顶点。两个实现要点：

- check 的 DP 在同代价时按「段数最多」打破平局，使 $c^*(\lambda)$ 单调、且能覆盖凸包边上的每个整点；
- 恢复答案时取「最多段数仍 $\geq K$ 的最大 λ」。若误取「段数 $\leq K$ 的最小 λ」，会选到相邻凸包边、令 $h(\lambda) - \lambda K$ 偏小（实测偏小 1）。

下例把正数数组恰好分成 K 个连续非空段，最小化 $\sum(\text{段和})^2$。罚值上界不再写死：设总和为 S，当 $\lambda = S^2 + 1$ 时，任何至少两段的含罚代价都严格大于单段方案，因此它一定跨过所有 $K > 1$ 的目标边界。

```
template <class Acc = long long>
Acc wqsPartition(const vector<int> &pre, int K)
{
    // pre 是正数数组的前缀和，K 是目标段数；返回恰分 K 段的最小平方和。
    static_assert(numeric_limits<Acc>::is_integer && numeric_limits<Acc>::is_signed &&
                  sizeof(Acc) >= sizeof(long long));
    int n = (int)pre.size() - 1;
    assert(n >= 1 && 1 <= K && K <= n && pre[0] == 0); // 调试检查，可删。
    for (int i = 1; i <= n; i++)
```

```
    {
        assert(pre[i] > pre[i - 1]); // 调试检查，可删。
    }
    Acc total = pre[n];
    if (K == 1)
    {
        return total * total;
    }
    auto check = [&](Acc lambda)
    {
        // lambda 是每段罚值；返回含罚最优代价及同代价下的最多段数。
        vector<Acc> dp(n + 1);
        vector<int> cnt(n + 1);
        for (int i = 1; i <= n; i++)
        {
            bool found = false;
            for (int j = 0; j < i; j++)
            {
                Acc seg = (Acc)pre[i] - pre[j];
                Acc cur = dp[j] + seg * seg + lambda;
                int c = cnt[j] + 1;
                if (!found || cur < dp[i] || (cur == dp[i] && c > cnt[i]))
                {
                    dp[i] = cur;
                    cnt[i] = c;
                    found = true;
                }
            }
        }
        return pair<Acc, int>{dp[n], cnt[n]};
    };
    Acc lo = 0;
    Acc hi = total * total + 1;
    Acc best = 0;
    while (lo <= hi)
    {
        Acc mid = lo + (hi - lo) / 2;
        if (check(mid).second >= K)
        {
            best = mid;
            lo = mid + 1;
        }
        else
        {
            hi = mid - 1;
        }
    }
    return check(best).first - best * K;
}
```

`wqsPartition(pre,K)` 默认用 `long long` 返回恰好 K 段的最小平方和，需要宽代价时调用 `wqsPartition<__int128>`。调用者须保证 S^2+1、每个 `dp[j]+seg^2+lambda`、二分边界及最终的 `best*K` 都能由

Acc 表示。当前 check 为 $O(n^2)$，总时间 $O(n^2 \log S)$、额外空间 $O(n)$；接入斜率优化后可进一步降低检查复杂度。

下面检查一段、每点单独成段和一组最优切分不在端点的例子：

```
signed main()
{
    assert(wqsPartition(vector<int>{0, 5}, 1) == 25);
    assert(wqsPartition(vector<int>{0, 1, 2, 3, 4}, 4) == 4);
    assert(wqsPartition(vector<int>{0, 3, 4, 8, 9, 14}, 3) == 66);
}
```

12.3.006 SMAWK 全单调矩阵优化

当转移代价形成全单调矩阵（特别是 Monge 矩阵）时，SMAWK 在线性时间 $O(H + W)$ 内求出 H 行、W 列矩阵每行的最优列。回调 `better(r,x,y)` 表示第 `r` 行的 `y` 是否优于 `x`；相等时应返回 `false` 以保留最左最优列。它比本章分治优化的 $O(H \log W)$ 更快，但要求更强：所有行的最优列必须单调，且候选列的比较满足全单调性。

```
template <class F>
vector<int> smawk(int rows, int cols, F better)
{
    // rows、cols 是矩阵行列数；better(row, oldCol, newCol) 表示新列严格更优；返回每行最左最优列。
    assert(rows >= 0 && cols > 0);
    if (rows == 0)
    {
        return {};
    }
    auto solve = [&](auto &&self, const vector<int> &rs, const vector<int> &cs) -> vector<int>
    {
        vector<int> reduced;
        reduced.reserve(cs.size());
        for (int col : cs)
        {
            while (!reduced.empty())
            {
                int row = rs[(int)reduced.size() - 1];
                if (!better(row, reduced.back(), col))
                {
                    break;
                }
                reduced.pop_back();
            }
            if ((int)reduced.size() < (int)rs.size())
            {
                reduced.push_back(col);
            }
        }
        vector<int> odd;
        for (int i = 1; i < (int)rs.size(); i += 2)
```

```
        {
            odd.push_back(rs[i]);
        }
        vector<int> ans(rs.size());
        if (!odd.empty())
        {
            vector<int> oa = self(self, odd, reduced);
            for (int i = 0; i < (int)oa.size(); i++)
            {
                ans[2 * i + 1] = oa[i];
            }
        }
        int p = 0;
        for (int i = 0; i < (int)rs.size(); i += 2)
        {
            int lim = (i + 1 == (int)rs.size() ? reduced.back() : ans[i + 1]);
            int best = reduced[p];
            while (reduced[p] < lim)
            {
                ++p;
                if (better(rs[i], best, reduced[p]))
                {
                    best = reduced[p];
                }
            }
            ans[i] = best;
        }
        return ans;
    };
    vector<int> rs(rows), cs(cols);
    iota(rs.begin(), rs.end(), 0);
    iota(cs.begin(), cs.end(), 0);
    return solve(solve, rs, cs);
}
```

空间复杂度为 $O(H+W)$（递归栈计入 $O(\log H)$）。若仅能证明最优列单调而不能证明全单调，应使用 `monoMin` 或 `divideLayer`；误用 SMAWK 会产生错误答案。

```
signed main()
{
    vector<vector<int>> a{{0, 4, 9}, {1, 1, 5}, {4, 2, 2}, {8, 3, 1}};
    auto p = smawk(4, 3, [&](int r, int x, int y) { return a[r][y] < a[r][x]; });
    assert((p == vector<int>{0, 0, 1, 2}));
}
```

12.3.007 Slope Trick

Slope Trick 维护分段线性凸函数

$$f(x)=mn+\sum_{a\in L}\max(a-x,0)+\sum_{b\in R}\max(x-b,0),$$

其中 L 用大根堆、R 用小根堆保存拐点，并保持 $\max L \le \min R$，于是 f 在 $[\max L, \min R]$ 上取到最小值 mn。加入 $\max(a-x,0)$ 时，若 $a > \min R$，最小值增加 $a - \min R$；之后把 a 放入 R、再把 R 的最小拐点移到 L，恢复不变量。$\max(x-a,0)$ 对称处理。`dl`、`dr` 是 L、R 整体平移量的懒标记。

常用的函数变换都只改堆或标记：$f(x) \leftarrow \min_{x-b \le y \le x-a} f(y)$ 等价于左半段右移 a、右半段右移 b；前缀最小值 $\min_{y \le x} f(y)$ 是删去右侧上升段，后缀最小值对称。

```
struct SlopeTrick
{
    // 维护分段线性凸函数 f(x) = mn + sum_{a in L} max(a-x,0) + sum_{b in R} max(x-b,0),
    // 且 max L <= min R, [max L, min R] 即最小值区间。dl、dr 是 L、R 整体的平移量（懒标记）。
    priority_queue<int> l;                          // l 存 L 中各拐点减去 dl。
    priority_queue<int, vector<int>, greater<>> r;  // r 存 R 中各拐点减去 dr。
    int dl = 0, dr = 0, mn = 0;                     // mn 是 f 的最小值。

    void addLeft(int a)
    {
        // 给 f 加上 max(a-x,0)。
        if (!r.empty())
        {
            mn += max(0LL, a - (r.top() + dr));
        }
        r.push(a - dr);
        int v = r.top() + dr;
        r.pop();
        l.push(v - dl);
    }

    void addRight(int a)
    {
        // 给 f 加上 max(x-a,0)。
        if (!l.empty())
        {
            mn += max(0LL, l.top() + dl - a);
        }
        l.push(a - dl);
        int v = l.top() + dl;
        l.pop();
        r.push(v - dr);
    }

    void addAbs(int a)
    {
        // 给 f 加上 |x-a|。
        addLeft(a);
        addRight(a);
    }

    void shift(int a, int b)
    {
        // a<=b; f(x) <- min_{x-b<=y<=x-a} f(y)，即左半段右移 a、右半段右移 b；a==b 时为平移。
        assert(a <= b); // 调试检查，可删。
```

```
        dl += a;
        dr += b;
    }

    void prefixMin()
    {
        // f(x) <- min_{y<=x} f(y)：去掉右侧上升段。
        r = {};
    }

    void suffixMin()
    {
        // f(x) <- min_{y>=x} f(y)：去掉左侧下降段。
        l = {};
    }

    pair<optional<int>, optional<int>> argmin() const
    {
        // 返回最小值区间的左右端点，无界一侧为空。
        optional<int> x, y;
        if (!l.empty())
        {
            x = l.top() + dl;
        }
        if (!r.empty())
        {
            y = r.top() + dr;
        }
        return {x, y};
    }

    int eval(int x) const
    {
        // 返回 f(x)，O(|L|+|R|) 并复制堆，仅用于调试或少量查询。
        auto a = l;
        auto b = r;
        int ans = mn;
        for (; !a.empty(); a.pop())
        {
            ans += max(0LL, a.top() + dl - x);
        }
        for (; !b.empty(); b.pop())
        {
            ans += max(0LL, x - b.top() - dr);
        }
        return ans;
    }
};
```

公开成员 `mn` 是当前最小值；`addLeft`、`addRight`、`addAbs` 加入一次项，`shift(a,b)` 要求 $a \le b$，`prefixMin`、`suffixMin` 取前后缀最小值，`argmin()` 返回最小值区间的两个端点（无界一侧为空），`eval(x)` 复制两个堆求函数值，只用于调试或少量查询。加入与平移为 $O(\log n)$，`argmin` 为 $O(1)$，`eval` 为 $O(n \log n)$；

n 是加入的拐点数，所有值须在 `long long` 范围内。

典型模型是“把数列改成满足单调或差分约束的最小代价”。例如单调不降化：设 $f_i(x)$ 是前 i 项且 $b_i = x$ 的最小代价，则 $f_i(x) = \min_{y \le x} f_{i-1}(y) + |x - a_i|$，每读入 a_i 先取前缀最小值再加入 $|x - a_i|$。值域小时也可以直接做 $O(nV)$ 的 DP；其他凸性结构见 12.3.004 斜率优化 / 李超树优化 与 12.3.005 WQS 二分。

```
signed main()
{
    vector<int> a{3, 1, 4, 1, 5, 9, 2, 6}; // 改成单调不降的最小代价 sum |a_i-b_i|。
    SlopeTrick f;
    for (int x : a)
    {
        f.prefixMin();
        f.addAbs(x);
    }
    auto [l, r] = f.argmin();
    assert(f.mn == 12 && *l == 6 && *r == 6 && f.eval(0) == 31);
}
```

13. 博弈论、概率与杂项数学

13.1 博弈论

13.1.001 Bash 与扩展 Bash

单堆 n 颗石子、每次取 1 到 m 颗、取走最后一颗者胜，这就是 Bash 博弈。把” 轮到某人时必输” 的局面称为 P 局面，其余为 N 局面：$n=0$ 是 P 局面（无子可取即已输），而 n 是 $m+1$ 的倍数时先手不论取几颗，对手都能补足到下一个 $m+1$ 的倍数，始终把 P 局面还给先手，直至 0，故 $n \bmod (m+1)=0$ 恰为 P 局面。反之余数非零时先手一步取到最近的倍数即可反制。要点是这条结论只在” 连续可取 $1..m$、正常取胜” 规则下成立，一旦改成不连续可取集合或最后取者负，必须回到必败态递推或 SG 重新求周期，见 13.1.008 取石子游戏。

```cpp
bool bashWin(int n, int m)
{
    // n 是当前石子数，m 是单步最多取石数；返回先手是否必胜。
    assert(n >= 0 && m >= 1); // 调试检查，可删。
    if (m == numeric_limits<int>::max())
    {
        return n != 0;
    }
    return n % (m + 1) != 0;
}
```

`bashWin(n, m)` 要求 $n \geq 0$、$m \geq 1$，返回先手是否必胜，$m = LLONG_MAX$ 时单独按 `n!=0` 处理，其他情况用 64 位取模；$O(1)$ 时间与空间。多堆 Bash 是独立子游戏之和，须先对每堆求 SG 值（单堆 Bash 的 SG 值即 $n \bmod (m+1)$）再异或，见 13.1.004 SG；取数集合不连续时改用 13.1.008 取石子游戏 的通用 SG 表。

13.1.002 Nim、Anti-Nim、Nim-K

普通 Nim 每步减少一堆的任意正数颗石子，无法行动者负。先手必胜当且仅当异或和 $X=\bigoplus_i a_i \neq 0$：异或和为零时任一步都会使它非零；非零时可选 $a_i \oplus X < a_i$，把该堆减到 $a_i \oplus X$。

Anti-Nim 改为取走最后一颗者负。若所有堆均不超过 1，忽略零堆后，偶数个非空堆为胜、奇数个为负；为与递推一致，无步终局记为胜。存在大于 1 的堆时仍按异或和判胜，但恢复具体走法时须处理即将只剩单位堆的情况。

Nim-K 每步任选 1 到 k 堆，各减少任意正数，无法行动者负。其必败条件是每个二进制位上的 1 数量均为 $k+1$ 的倍数；此结论用于判胜，不能把这些余数当作一般 SG 值。

```cpp
// 依赖 13.1.001 Bash 与扩展 Bash
bool nimWin(const vector<unsigned long long> &a)
{
    // 参数 a 表示 普通 Nim 各堆的非负石子数；返回 Nim 局面的先手是否必胜。
    unsigned long long x = 0;
    for (unsigned long long v : a)
```

```
    {
        x ^= v;
    }
    return x != 0;
}

bool antiNimWin(const vector<unsigned long long> &a)
{
    // 参数 a 表示 反 Nim 各堆的非负石子数；返回反 Nim 局面的先手是否必胜。
    bool allOne = true;
    unsigned long long x = 0;
    int one = 0;
    for (unsigned long long v : a)
    {
        allOne &= v <= 1;
        x ^= v;
        one += v == 1;
    }
    return allOne ? one % 2 == 0 : x != 0;
}

bool nimKWin(const vector<unsigned long long> &a, int k)
{
    // a 是各堆石子数，k>=1 是每步最多可减少的堆数；返回正常规则下先手是否必胜。
    assert(k >= 1);
    // 无符号类型保留完整 64 位堆大小，并承载 k+1。
    unsigned long long mod = (unsigned long long)k + 1;
    for (int b = 0; b < 64; b++)
    {
        unsigned long long cnt = 0;
        for (auto x : a)
        {
            cnt += x >> b & 1;
        }
        if (cnt % mod != 0) return true;
    }
    return false;
}
```

三个接口都接受完整 64 位无符号堆大小。`nimWin(a)`、`antiNimWin(a)` 为 $O(n)$ 时间；`nimKWin(a,k)` 要求 $k \geq 1$，为 $O(64n)$ 时间，均用 $O(1)$ 额外空间。空堆集合在普通 Nim、Nim-K 中为负，在 Anti-Nim 中为胜。判必胜后要还原到具体一步时，对 Nim 找满足 $a_i \oplus X < a_i$ 的堆并取到 $a_i \oplus X$（X 为总异或）。只能移动石子而非取走的阶梯型规则见 13.1.003 阶梯 Nim，任意状态图上的一般合成见 13.1.004 SG。

13.1.003 阶梯 Nim

阶梯 Nim 把石子摆在编号 $0, 1, 2, \ldots$ 的台阶上，每步可把某台阶上任意正数颗石子移到下一级，移到第 0 级即“落地”不再参与。只有奇数编号台阶的石子异或和决定胜负：从奇数级移到偶数级对应 Nim 取石；对手从偶数级移到奇数级后，先手可把等量石子再下移一级抵消。因此先手必胜当且仅当奇数下标石子数的异或和非零。

```cpp
bool stairNimWin(const vector<unsigned long long> &a)
{
    // 参数 a 表示 从第 0 层开始记录的各阶梯石子数；返回阶梯 Nim 局面的先手是否必胜。
    unsigned long long x = 0;
    for (int i = 1; i < (int)a.size(); i += 2)
    {
        x ^= a[i];
    }
    return x != 0;
}
```

`stairNimWin(a)` 中 `a[i]` 为第 i 级台阶石子数，返回先手是否必胜，$O(n)$ 时间、$O(1)$ 额外空间。它本质是把奇数下标当作 13.1.002 Nim 的堆再异或；若移动规则不是” 下一级” 而是任意 DAG 上的转移，则退回 13.1.004 SG 逐状态求 mex。

13.1.004 SG、Anti-SG、Multi-SG、Every-SG

SG 理论适用于有限、完全信息、无随机因素的公平正常规则博弈：双方可选走法相同，无法行动者负。DAG 状态满足

$$\mathrm{SG}(x) = \mathrm{mex}\{\mathrm{SG}(y) : x \to y\},$$

其中 mex 为集合外最小非负整数，终局为 0，SG 为零恰为必败。独立子游戏之和每步只动一个子游戏，整体 SG 为各子游戏 SG 的异或。

Anti-SG 改为最后行动者负，一般不能只由正常规则 SG 判胜；13.1.002 Anti-Nim 的单位堆判定只适用于对应取石规则。Multi-SG 若允许一次拆成多个独立子游戏，应先异或每个后继局面的各分量 SG，再对全部合法后继取 mex。Every-SG 的名称常指每回合同时推进所有未结束子游戏；必须明确谁行动、哪些分量必须移动和何时终止，再按完整联合状态递推，不能直接异或或只看步数奇偶。

代码按逆拓扑序计算。状态 x 的 mex 至多为出度，使用状态编号作标记时间戳，所有 mex 扫描次数之和不超过边数。

```cpp
// 依赖 13.1.001 Bash 与扩展 Bash
vector<int> sg(const vector<vector<int>> &to)
{
    // to[x] 是状态 x 的后继编号表；返回每个 DAG 状态的 Sprague-Grundy 值。
    int n = to.size();
    vector<int> deg(n);
    for (int x = 0; x < n; x++)
    {
        for (int y : to[x])
        {
            assert(0 <= y && y < n); // 调试检查，可删。
            deg[y]++;
        }
    }
    queue<int> q;
    for (int x = 0; x < n; x++)
    {
        if (deg[x] == 0)
```

```
        {
            q.push(x);
        }
    }
    vector<int> ord;
    while (!q.empty())
    {
        int x = q.front();
        q.pop();
        ord.push_back(x);
        for (int y : to[x])
        {
            deg[y]--;
            if (deg[y] == 0)
            {
                q.push(y);
            }
        }
    }
    assert((int)ord.size() == n); // 调试检查，可删。
    reverse(ord.begin(), ord.end());
    vector<int> f(n), seen(n + 1, -1);
    for (int x : ord)
    {
        for (int y : to[x])
        {
            seen[f[y]] = x; // 用状态编号作时间戳，无需逐点清空。
        }
        int g = 0;
        while (seen[g] == x)
        {
            g++;
        }
        f[x] = g;
    }
    return f;
}
```

`sg(to)` 要求后继编号合法且状态图为 DAG，返回各状态的 SG。设状态数 V、转移数 E，时间 $O(V+E)$，额外空间 $O(V)$，重复边允许存在。有环且无限对局为平时使用 13.1.010 有环博弈的胜负平逆推；单堆固定取法使用 13.1.008 取石子游戏。

13.1.005 Wythoff 博弈

Wythoff 博弈有两堆石子 (a,b)，每步可从任一堆取任意正数，或从两堆同时取相同正数，取到最后者胜。其 P 局面为 $(\lfloor k\varphi\rfloor,\lfloor k\varphi\rfloor+k)$ 及交换，$k\ge 0$，$\varphi=\frac{1+\sqrt{5}}{2}$。两列 $\lfloor k\varphi\rfloor$ 与 $\lfloor k\varphi^2\rfloor$ 构成互补 Beatty 序列。令较小堆为 a、差为 $k=b-a$，无需计算可能超出 `int` 的 $\lfloor k\varphi\rfloor$；直接用精确等价条件 $a^2-ak\le k^2<(a+1)^2-(a+1)k$ 判断当前 a 是否就是该冷位置的较小堆。

```cpp
// 知识点 13.1.005：Wythoff 两堆取石博弈。
namespace wythoff_game
{
bool firstWins(int a, int b)
{
    // a、b 是非负石子数；返回先手是否必胜。
    assert(a >= 0 && b >= 0); // 调试检查，可删。
    if (a > b)
    {
        swap(a, b);
    }
    int k = b - a;
    __int128 x = a;
    __int128 d = k;
    __int128 z = d * d;
    bool cold = x * x - x * d <= z
                && z < (x + 1) * (x + 1) - (x + 1) * d;
    return !cold;
}
}
```

`wythoff_game::firstWins(a,b)` 接受完整非负 `int` 范围内的两堆石子数，返回先手是否必胜，$O(1)$ 时间与空间；判定全程在 `__int128` 中完成，不依赖浮点近似，也不要求冷位置坐标能由 `int` 表示。其他移动规则回到 13.1.004 SG。下面检查前几组冷局面、对称性和大数热局面：

```cpp
signed main()
{
    for (auto [a, b] : vector<pair<int, int>>{{0, 0}, {1, 2}, {3, 5}, {4, 7}, {6, 10}, {8, 13}})
    {
        assert(!wythoff_game::firstWins(a, b));
        assert(!wythoff_game::firstWins(b, a));
    }
    assert(wythoff_game::firstWins(1, 1));
    assert(wythoff_game::firstWins(2, 3));
    assert(wythoff_game::firstWins(numeric_limits<int>::max(),
                                   numeric_limits<int>::max()));
}
```

13.1.006 Fibonacci 博弈

单堆 $n > 0$ 颗石子，先手首次只能取 1 到 $n-1$ 颗，此后每步至多取对方上一步的两倍，取走最后一颗者胜。Zeckendorf 定理将每个正整数唯一分解为若干不相邻 Fibonacci 数之和，数列取 $1, 2, 3, 5, \ldots$。令 $z(n)$ 为最小分解项，完整状态 (n, m) 表示剩余 n 颗、当前最多可取 m 颗，则必败当且仅当 $m < z(n)$。

当 $m \geq z(n)$ 时取走 $z(n)$：若未取空，剩余最小项大于 $2z(n)$，对手进入必败态；当 $m < z(n)$ 时，任何合法取数 t 都满足 $z(n-t) \leq 2t$，故对手有必胜策略。初始上限为 $n-1$，因此恰好 Fibonacci 数为必败态；其余局面取最小分解项即可。剩余石子数不必是 Fibonacci 数，例如 12 首取 1 后剩余 11，关键是对手上限变为 2。

```cpp
// 知识点 13.1.006：Fibonacci 取石的最优首取量。
namespace fib_game
{
int zeckMin(int n)
{
    // n 是正石子数；返回 Zeckendorf 分解中的最小 Fibonacci 项。
    assert(n > 0); // 调试检查，可删。
    vector<int> f = {1, 2};
    while (f.back() <= n - f[f.size() - 2])
    {
        f.push_back(f.back() + f[f.size() - 2]);
    }
    int last = 0;
    for (auto it = f.rbegin(); it != f.rend(); ++it)
    {
        if (*it <= n)
        {
            n -= *it;
            last = *it;
        }
    }
    return last;
}
}
```

`fib_game::zeckMin(n)` 要求 $n > 0$，返回其 Zeckendorf 分解的最小 Fibonacci 项：该值等于 n 时说明 n 本身是 Fibonacci 数（先手必败），否则它就是必胜首取量。Fibonacci 数按 φ^k 指数增长，故枚举到 n 只需 $O(\log n)$ 项，时间与空间均 $O(\log n)$。这是单堆特殊规则，与依赖异或合成的 13.1.002 Nim、13.1.004 SG 不同一类，规则变化时应把上一步取数或当前上限纳入状态，不能套用固定取法集合的周期结论。

13.1.007 树上删边、无向图删边游戏

Green Hackenbush 的所有边均可由双方删除：固定一个地面根，每步删一条边，随后失去地面连通性的边全部脱落，无法行动者负。初始不接地的部分不参与；多个地面点应先合并成一个根。树上由 Colon Principle 得到 $\mathrm{SG}(u) = \bigoplus_v (\mathrm{SG}(v) + 1)$，其中 v 枚举子节点，叶子为 0；各分支在根处独立，每条悬挂边使其子游戏的 nimber 加 1。

一般无向图使用融合原理：同一环上的顶点可合并而不改变 SG。反复融合后，每个边双连通分量收缩成一点，内部边全部成为自环，桥构成森林。每个自环只能删一次，局部贡献为分量内边数的奇偶 p_u，故根所在树满足

$$\mathrm{SG}(u) = p_u \oplus \bigoplus_{v \in \mathrm{child}(u)} (\mathrm{SG}(v) + 1).$$

这里统计的是内部边数，不是点数；偶环的局部贡献为 0，奇环为 1。`graphSg` 复用 5.3.004 边双连通分量的 `EdgeBCC`；只复制 `treeSg` 时可省去该依赖。

```cpp
// 依赖 5.3.004 割边与边双连通分量
// 知识点 13.1.007：树与一般无向图删边 Green Hackenbush 的 SG 值。
namespace edge_del
{
int treeSg(const vector<vector<int>> &g, int root = 0)
{
    // g 是无向树，root 是地面根；返回整个局面的 SG 值。
    int n = g.size();
    assert(n > 0 && 0 <= root && root < n); // 调试检查，可删。
    int deg = 0;
    for (int u = 0; u < n; u++)
    {
        for (int v : g[u])
        {
            assert(0 <= v && v < n); // 调试检查，可删。
            deg++;
        }
    }
    assert(deg == 2 * (n - 1)); // 调试检查，可删。
    vector<int> par(n, -2);
    int seen = 0;
    auto dfs = [&](auto &&self, int u, int p) -> int
    {
        // self 是递归自身，u 是当前点，p 是父点；返回接地子树的 SG 值。
        par[u] = p;
        seen++;
        int ans = 0;
        for (int v : g[u])
        {
            if (v == p)
            {
                continue;
            }
            assert(par[v] == -2); // 调试检查，可删。
            ans ^= self(self, v, u) + 1;
        }
        return ans;
    };
    int ans = dfs(dfs, root, -1);
    assert(seen == n); // 调试检查，可删。
    return ans;
}

int graphSg(int n, const vector<array<int, 2>> &ed, int root = 0)
{
    // n 是点数，ed[i] 两维为无向边端点（每条边只列一次），root 是唯一地面根。
    // 返回 Green Hackenbush 的 SG；允许重边、自环，与根不连通的部分不参与。
    assert(n > 0 && 0 <= root && root < n);
    EdgeBCC bcc(n);
    for (auto [u, v] : ed)
    {
        bcc.addEdge(u, v);
```

```
    }
    bcc.build();
    auto bel = bcc.comps();
    int cnt = *max_element(bel.begin(), bel.end()) + 1;
    vector<vector<int>> tr(cnt);
    vector<int> val(cnt); // 每个边双内部融合后的自环数奇偶，即局部 SG。
    for (auto [u, v] : ed)
    {
        int a = bel[u], b = bel[v];
        if (a == b)
        {
            val[a] ^= 1;
        }
        else
        {
            tr[a].push_back(b);
            tr[b].push_back(a);
        }
    }
    auto dfs = [&](auto &&self, int u, int p) -> int
    {
        // self 是递归自身，u 是边双编号，p 是父边双；返回带局部自环的接地子树 SG。
        int ans = val[u];
        for (int v : tr[u])
        {
            if (v != p) ans ^= self(self, v, u) + 1;
        }
        return ans;
    };
    return dfs(dfs, bel[root], -1);
}
}
```

`edge_del::treeSg(g,root)` 接受连通无向树，每条边在两端邻接表各列一次；时间与额外空间均为 $O(n)$。`graphSg(n,ed,root)` 要求 $n > 0$，每条无向边只列一次，允许自环、重边和非连通图；时间与额外空间均为 $O(n + m)$，其中 m 是边数。两者均使用 $[0, n)$ 编号并返回整局 SG，非零即先手必胜；递归深度最坏为 $O(n)$，实际可承受规模取决于评测线程栈。多个独立接地游戏用 13.1.004 SG 异或合成，删点行走规则改用 13.1.011。

```
signed main()
{
    // 无参数；检查奇偶环、桥下悬挂的奇环与自环，返回 0。
    assert(edge_del::graphSg(3, {{0, 1}, {1, 2}, {2, 0}}) == 1);
    assert(edge_del::graphSg(4, {{0, 1}, {1, 2}, {2, 3}, {3, 0}}) == 0);
    assert(edge_del::graphSg(4, {{0, 1}, {1, 2}, {2, 3}, {3, 1}}) == 2);
    assert(edge_del::graphSg(1, {{0, 0}, {0, 0}}) == 0);
}
```

13.1.008 取石子游戏

减法游戏（Subtraction Game）是最一般的单堆取石模型：给定允许取走的正数集合 S，每步从堆里取走 S 中某个数量，无法行动者负。其 SG 值由 $\mathrm{SG}(x)=\mathrm{mex}\{\mathrm{SG}(x-d):d\in S,\ d\le x\}$ 逐点递推，$\mathrm{SG}(0)=0$，先手必胜当且仅当 $\mathrm{SG}(n)\neq 0$。有限取数集合的 SG 序列最终一定进入周期（状态转移只依赖前 $\max S$ 项），但周期长度须由算出的表验证，不能凭规则臆断。因单点 SG 值不超过后继个数 $|S|$，判 mex 时只需大小 $|S|+1$ 的标记数组。这份 `subGameSG` 是 Bash、限取集合等 13.1.001 Bash 无法用简单模运算覆盖的规则的通用求解器。

```cpp
// 依赖 13.1.001 Bash 与扩展 Bash
namespace stone_game
{
vector<int> subGameSG(int n, const vector<int> &take)
{
    // n 是最大石子数，take 是每次允许取走的正数；返回 0..n 的 SG 值。
    assert(n >= 0);
    vector<int> step = take;
    sort(step.begin(), step.end());
    step.erase(unique(step.begin(), step.end()), step.end());
    for (int d : step)
    {
        assert(d > 0);
    }
    vector<int> sg(n + 1);
    for (int x = 1; x <= n; x++)
    {
        vector<char> seen(step.size() + 1);
        for (int d : step)
        {
            if (d <= x && sg[x - d] < (int)seen.size())
            {
                seen[sg[x - d]] = true;
            }
        }
        while (sg[x] < (int)seen.size() && seen[sg[x]])
        {
            sg[x]++;
        }
    }
    return sg;
}
}
```

`stone_game::subGameSG(n,take)` 要求 $n\ge 0$ 且每种取数为正；函数先去重，再返回 $0..n$ 的 SG 表。设去重后取数种类为 k，时间 $O(nk+k\log k)$、空间 $O(n+k)$。多堆合成时异或各堆 SG；连续取 $1..m$ 时直接用 13.1.001 Bash。

13.1.009 64 位 nim-product

有限 nimber 在异或和 nim-product 下构成特征为 2 的域。直接按 mex 定义相乘不可用；利用 Conway 域塔，先把 $[0, 2^{16})$ 的非零元素写成一个原始元的幂，再把 32 位和 64 位元素分别视为二次扩域元素做 Karatsuba 式合并。`basis` 给出该原始元乘法在二进制基下的线性映射，指数表中的偏移 3、6 分别完成乘 2^{15} 及其平方；这些常数属于 nimber 域结构，不是普通有限域任取的不可约多项式。

```
struct NimProduct64
{
    inline static array<int, 131075> ex{}; // ex[i] 是 16 位 nimber 原始元的 i 次幂，周期 65535，尾部延长供指数相加后直接查表。
    inline static array<int, 65536> lg{}; // lg[x] 是非零元 x 关于原始元的离散对数。
    inline static bool ready = false; // ready 表示两张表是否已建立。

    static void init()
    {
        // 无参数；首次调用时建 16 位 nimber 原始元的幂表 ex（周期 65535，尾部延长供指数和取用）与离散对数表 lg。
        if (ready)
        {
            return;
        }
        ready = true;
        const array<int, 16> basis = {
            10279, 15417, 35722, 52687, 44124, 62628, 15661, 5686,
            3862, 1323, 334, 647, 61560, 20636, 4267, 8445
        };
        array<int, 65536> nxt{};
        for (int i = 0; i < 16; i++)
        {
            for (int s = 0; s < (1LL << i); s++)
            {
                nxt[s | 1LL << i] = nxt[s] ^ basis[i];
            }
        }
        ex[0] = 1;
        for (int i = 0; i < 65534; i++)
        {
            ex[i + 1] = nxt[ex[i]];
        }
        for (int i = 0; i < 65535; i++)
        {
            lg[ex[i]] = i;
            ex[65535 + i] = ex[i];
        }
        for (int i = 0; i < 5; i++)
        {
            ex[131070 + i] = ex[i];
        }
    }

```

```
    static int p16(int a, int b, int shift = 0)
    {
        // a、b 是 16 位 nimber，须已 init；返回 a*b*g^shift（g 为原始元），a 或 b 为 0 时返回 0。
        // shift 是附加指数：3 即再乘 nimber 2^15，6 即再乘其平方。
        if (!a || !b)
        {
            return 0;
        }
        return ex[(int)lg[a] + lg[b] + shift];
    }

    static int p32high(int a, int b)
    {
        // a、b 是 32 位 nimber，须已 init；返回 nim 积 a*b*2^31，供 64 位合并时的低半部分使用。
        int al = a & 65535, ah = a >> 16;
        int bl = b & 65535, bh = b >> 16;
        int x = p16(al, bl, 3);
        int y = p16(ah, bh, 6);
        int z = p16(al ^ ah, bl ^ bh, 3);
        return (int)(y ^ z) << 16 | p16(z ^ x, 1, 3);
    }

    static int p32(int a, int b)
    {
        // a、b 是 32 位 nimber，须已 init；按 16 位子域上的二次扩域做 Karatsuba 合并，返回 nim 积
        ↪ a*b。
        int al = a & 65535, ah = a >> 16;
        int bl = b & 65535, bh = b >> 16;
        int c = p16(al, bl);
        return (int)(p16(al ^ ah, bl ^ bh) ^ c) << 16 |
               (p16(ah, bh, 3) ^ c);
    }

    static int product32(int a, int b)
    {
        // a、b 是 32 位 nimber；自动初始化后返回它们的积。
        init();
        return p32(a, b);
    }

    static unsigned long long product(unsigned long long a, unsigned long long b)
    {
        // a、b 是 64 位 nimber；自动初始化后按 32 位子域的二次扩域合并，返回 nim 积 a*b。
        init();
        int al = a & 0xffffffffULL, ah = a >> 32;
        int bl = b & 0xffffffffULL, bh = b >> 32;
        int c = p32(al, bl);
        return (unsigned long long)(p32(al ^ ah, bl ^ bh) ^ c) << 32 |
               (p32high(ah, bh) ^ c);
    }
};

```

```cpp
unsigned long long nimProduct(unsigned long long a, unsigned long long b)
{
    // a、b 是 64 位 nimber；返回 nim 积 a*b，不要求调用方预先初始化。
    return NimProduct64::product(a, b);
}

int nimProduct32(int a, int b)
{
    // a、b 是 32 位 nimber；返回乘积，不要求调用方预先初始化。
    return NimProduct64::product32(a, b);
}
```

`nimProduct(a,b)` 返回两个 64 位 nimber 的积；输入只占 32 位时用 `nimProduct32(a,b)`，它会自行初始化并直接调用更短的 32 位扩域路径。`p16`、`p32`、`p32high` 是要求已经初始化的底层接口，不应从题目入口直接调用。nim 和仍用 `a^b`，不能把这里的乘法拿去替代模乘。首次调用用 $O(2^{16})$ 时间建表，之后每次乘法为 $O(1)$。两张常驻表共 196611 个元素：性能版每项 2 字节，约 384 KiB；正文默认整数宏下每项 8 字节，约 1.50 MiB。初始化另有 65536 项临时表。正文的 32 位分块接口要求开启整数宏；需要 SG 理论回看 13.1.004 SG、Anti-SG、Multi-SG、Every-SG。

下面检查零元、单位元与低阶 nimber 乘法；完整随机校验放在本地测试文件：

```cpp
signed main()
{
    assert(nimProduct(0, 123456789) == 0);
    assert(nimProduct(1, 123456789) == 123456789);
    assert(nimProduct(2, 2) == 3);
    assert(nimProduct(2, 3) == 1);
    assert(nimProduct(3, 3) == 2);
}
```

13.1.010 有环博弈的胜负平逆推

有限有向图的顶点表示完整局面，双方轮流选边，无出边者负，无限对局为平。顶点必须包含影响走法的全部信息；双方可用边不同时，须把当前行动方也编码进状态。结果相对于该状态的当前行动者。

从无出边的必败态开始沿反图传播：存在一个必败后继则当前必胜；全部后继均为必胜则当前必败。队列耗尽后仍未分类的点至少有一个未分类后继，且没有必败后继，双方都能避免失败并维持无限对局，因此为平。环上可以存在必胜、必败点，不能把所有含环分量直接判平。

```cpp
vector<int> gameGraph(const vector<vector<int>> &g)
{
    // g[u] 是轮到行动者处于状态 u 时的全部后继；无步者负，无限对局平。
    // 返回各状态对当前行动者的结果：1 必胜、-1 必败、0 平局；编号为 [0,g.size())。
    int n = g.size();
    vector<vector<int>> rev(n);
    vector<int> left(n), ans(n);
    queue<int> q;
    for (int u = 0; u < n; u++)
    {
```

```
        left[u] = g[u].size(); // 尚未证实为对手必胜的走法数，重边也计数。
        for (int v : g[u])
        {
            assert(0 <= v && v < n);
            rev[v].push_back(u);
        }
        if (left[u] == 0)
        {
            ans[u] = -1;
            q.push(u);
        }
    }
    while (!q.empty())
    {
        int v = q.front();
        q.pop();
        for (int u : rev[v])
        {
            if (ans[u] != 0) continue;
            if (ans[v] == -1)
            {
                ans[u] = 1;
            }
            else if (--left[u] == 0)
            {
                ans[u] = -1;
            }
            else continue;
            q.push(u);
        }
    }
    return ans;
}
```

`gameGraph(g)` 返回按顶点顺序排列的结果：1 必胜、-1 必败、0 平局。允许自环、重边和空图，时间与额外空间均为 $O(V+E)$。终局若由其他规则决定，应先按题意重建状态转移与终局；本接口不处理随机转移。有 DAG 且需合成独立公平游戏时用 13.1.004 SG。

```
signed main()
{
    // 无参数；检查终局、必胜前驱、自环与可逃出环的状态，返回 0。
    vector<vector<int>> g = {{}, {0}, {2}, {2, 0}, {1, 3}};
    assert((gameGraph(g) == vector<int>{-1, 1, 0, 1, -1}));
}
```

13.1.011 无向图删点行走与匹配博弈

局面为无向图与当前点 s。每步走到相邻点，再删除原点及其关联边，无法行动者负；与 13.1.007 删边脱落 的规则不同。记 $\nu(G)$ 为最大匹配大小，则

$$s \text{ 必胜} \iff \nu(G-s) < \nu(G) \iff s \text{ 被每个最大匹配覆盖.}$$

若 s 为必胜起点，取任意最大匹配 M，沿匹配边走到 t。删除 s 后，$M-\{st\}$ 是不覆盖 t 的最大匹配，故对手必败。若存在不覆盖 s 的最大匹配，则删去 s 后它仍最大；每个邻点 t 都必须被该图的所有最大匹配覆盖，否则可加上 st 超过原最大值。因此任意首步都让对手必胜。

实现复用 5.7.001 一般图最大匹配的 `Blossom`，至多求两次最大匹配，直接返回原匹配中 s 的搭档作为必胜首步。

```cpp
// 依赖 5.7.001 一般图最大匹配（Blossom）
optional<int> vertexGame(int n, const vector<array<int, 2>> &ed, int start)
{
    // n 是点数，ed[i] 两维为简单无向边端点，start 是当前点；每步走到邻点并删掉原点。
    // 返回一条必胜首步的目标点；先手必败时返回空。无法行动者负。
    assert(n > 0 && 0 <= start && start < n);
    Blossom all(n), cut(n);
    for (auto [u, v] : ed)
    {
        assert(0 <= u && u < n && 0 <= v && v < n && u != v);
        all.add(u, v);
        if (u != start && v != start) cut.add(u, v);
    }
    int best = all.work();
    if (all.match[start] == -1 || cut.work() == best) return nullopt;
    return all.match[start];
}
```

`vertexGame(n,ed,start)` 要求 $n>0$、$0\le start<n$，`ed[i]` 为简单无向边的两个端点，每条边只列一次，允许孤立点和非连通图。必败返回 `nullopt`，必胜返回一个合法目标点编号；时间 $O(n^3)$，额外空间 $O(n+m)$。二分图且要查询全部起点时用 13.1.012。若规则是第一步任意选点删除，此后每步必须删上一步所删点的邻点，则先手必败当且仅当原图有完美匹配；它的初始选点阶段不能直接当作已固定当前点。

```cpp
signed main()
{
    // 无参数；三点路径从中间出发必胜，从端点出发必败，返回 0。
    vector<array<int, 2>> ed = {{0, 1}, {1, 2}};
    auto to = vertexGame(3, ed, 1);
    assert(to && (*to == 0 || *to == 2));
    assert(!vertexGame(3, ed, 0));
    assert(!vertexGame(1, {}, 0));
}
```

13.1.012 二分图博弈与全部必胜起点

沿用 13.1.011 无向图删点行走 的规则；二分图可一次求出全部被每个最大匹配覆盖的点。先用 5.6.003 Hopcroft–Karp 求任意最大匹配 M。从未匹配左点出发，交替经过非匹配边、匹配边：可达左点均可通

过翻转路径成为未匹配点，故为必败；若某左点在另一最大匹配中未匹配，两匹配的对称差必给出这样的交替路径，因此不可达左点恰为必胜起点。交换左右部后同理求右侧。

5.6.007 最小点覆盖的 `minVtxCover` 已实现交替搜索，其返回的左点表正是本侧不可达点。下面复用该函数两次，第二次反向邻接表并交换匹配数组；不能直接把一次最小点覆盖中的全部点都判为必胜。

```cpp
// 依赖 5.6.003 Hopcroft-Karp
// 依赖 5.6.007 最小点覆盖、最小边覆盖
pair<vector<int>, vector<int>> bipGame(const vector<vector<int>> &g, int nr)
{
    // g[u] 是左点 u 相邻的右点编号，nr 是右部点数；左右部均从 0 编号。
    // 每步走到邻点并删原点，无法行动者负；返回全部必胜起点的 {左点表，右点表}，各自递增。
    assert(nr >= 0);
    int nl = g.size();
    HopcroftKarp hk(nl, nr);
    vector<vector<int>> rev(nr);
    for (int u = 0; u < nl; u++)
    {
        for (int v : g[u])
        {
            hk.addEdge(u, v);
            rev[v].push_back(u);
        }
    }
    hk.matching();
    auto ml = hk.leftMatch(), mr = hk.matchR();
    // 点覆盖构造的左部结果恰为本侧交替搜索不可达点，即所有最大匹配均覆盖的点。
    auto l = minVtxCover(g, ml, mr).first;
    auto r = minVtxCover(rev, mr, ml).first;
    return {move(l), move(r)};
}
```

`bipGame(g,nr)` 中左右部各自从 0 编号，`g[u]` 只存右点编号，要求 $nr \geq 0$ 且每个邻点在 $[0, nr)$。返回 `{左侧必胜点表，右侧必胜点表}`，均递增；允许空侧、孤立点和重复邻边。设总点数为 V、邻接表项数为 E，时间 $O((V+E)\sqrt{V+1})$，额外空间 $O(V+E)$，匹配递归深度最坏 $O(V)$。要返回单个起点的一条必胜首步，可在原最大匹配中取其搭档；一般图则用 13.1.011。

```cpp
signed main()
{
    // 无参数；两个左点共用一个右点时只有右点必胜，返回 0。
    auto [l, r] = bipGame({{0}, {0}}, 1);
    assert(l.empty() && r == vector<int>{0});
    auto [a, b] = bipGame({{}, {}}, 0);
    assert(a.empty() && b.empty());
}
```

13.2 概率与分布

13.2.001 常见概率公式

这几条公式都建立在同一个事件空间上。条件概率 $P(A\mid B)=\dfrac{P(A\cap B)}{P(B)}$ 要求 $P(B)>0$；把样本空间按互斥且完备的 $\{B_i\}$ 划分，得全概率公式 $P(A)=\sum_i P(A\mid B_i)P(B_i)$；两者结合即 Bayes 公式 $P(B_i\mid A)=\dfrac{P(A\mid B_i)P(B_i)}{\sum_j P(A\mid B_j)P(B_j)}$，把先验 $P(B_i)$ 与似然 $P(A\mid B_i)$ 更新为后验。实现里 `bayes` 传入的 `like` 就是各假设下的似然（不必是概率、无需归一），分母 z 为归一化权重（离散概率似然时为证据概率，连续似然时为证据密度），`assert` 挡住 $z=0$ 的退化。要点是概率为零不等于事件不可能（连续分布上单点概率恒为零），且独立性 $P(A\cap B)=P(A)P(B)$ 是额外假设、不能默认成立。

```cpp
// 知识点 13.2.001：条件概率、Bayes 与全概率公式。
namespace prob_formula
{
long double condProb(long double pab, long double pb)
{
    // pab 是 P(AnB)，pb 是 P(B)>0；返回 P(A|B)。
    assert(0 <= pab && pab <= pb && pb > 0 && pb <= 1); // 调试检查，可删。
    return pab / pb;
}

vector<long double> bayes(const vector<long double> &prior, const vector<long double> &like)
{
    // prior 是先验概率，like 是各假设下证据似然；返回观察证据后的后验分布。
    assert(prior.size() == like.size()); // 调试检查，可删。
    assert(!prior.empty()); // 调试检查，可删。
    long double z = 0;
    long double sum = 0;
    for (int i = 0; i < (int)prior.size(); i++)
    {
        assert(prior[i] >= 0 && like[i] >= 0); // 调试检查，可删。
        sum += prior[i];
        z += prior[i] * like[i];
    }
    assert(fabsl(sum - 1) <= 1E-12L); // 调试检查，可删。
    assert(z > 0); // 调试检查，可删。
    vector<long double> ans(prior.size());
    for (int i = 0; i < (int)prior.size(); i++)
    {
        ans[i] = prior[i] * like[i] / z;
    }
    return ans;
}
}
```

`prob_formula::condProb(pab,pb)` 要求 $0\le P(A\cap B)\le P(B)\le 1$ 且 $P(B)>0$；`bayes(prior,like)` 要求非空先验为非负且和为 1，似然非负，证据总权重为正。返回后验分布，时间与输出空间 $O(n)$。随机变量矩见 13.2.003 随机变量期望与方差。

13.2.002 常见概率分布

每种分布由参数、支持集和归一化概率确定。Bernoulli 与二项分布要求 $0 \le p \le 1$、$n \in \mathbb{Z}_{\ge 0}$；几何分布要求 $0 < p \le 1$；负二项要求 $r \in \mathbb{Z}_{>0}$、$0 < p \le 1$；Poisson 要求 $\lambda \ge 0$，$\lambda = 0$ 时退化为 $X = 0$。超几何要求 $N \in \mathbb{Z}_{>0}$、$0 \le K, n \le N$，且 $\max(0, n-N+K) \le k \le \min(n, K)$；表中方差式用于 $N > 1$，$N = 1$ 时方差为 0。连续均匀分布要求 $a < b$，指数分布要求 $\lambda > 0$，正态分布要求 $\sigma > 0$。

离散分布：

分布	参数	$P(X = k)$
Bernoulli	p	$p^k(1-p)^{1-k},\ k \in \{0,1\}$
二项	n, p	$\binom{n}{k} p^k (1-p)^{n-k},\ 0 \le k \le n$
几何（首次成功）	p	$(1-p)^{k-1} p,\ k \ge 1$
负二项（第 r 次成功）	r, p	$\binom{k-1}{r-1} p^r (1-p)^{k-r},\ k \ge r$
Poisson	λ	$\dfrac{e^{-\lambda}\lambda^k}{k!},\ k \ge 0$
超几何	N, K, n	$\dfrac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$

分布	$E[X]$	$\operatorname{Var}(X)$
Bernoulli	p	$p(1-p)$
二项	np	$np(1-p)$
几何	$\frac{1}{p}$	$\frac{1-p}{p^2}$
负二项	$\frac{r}{p}$	$\frac{r(1-p)}{p^2}$
Poisson	λ	λ
超几何	$\frac{nK}{N}$	$\dfrac{nK(N-K)(N-n)}{N^2(N-1)}$

连续分布（f 为密度）：

分布	参数	$f(x)$
均匀	a, b	$\frac{1}{b-a},\ x \in [a, b]$
指数	λ	$\lambda e^{-\lambda x},\ x \ge 0$
正态	μ, σ^2	$\dfrac{1}{\sigma\sqrt{2\pi}} e^{-(x-\mu)^2/(2\sigma^2)}$

分布	$E[X]$	$\operatorname{Var}(X)$
均匀	$\frac{a+b}{2}$	$\frac{(b-a)^2}{12}$
指数	$\frac{1}{\lambda}$	$\frac{1}{\lambda^2}$
正态	μ	σ^2

二项概率使用对数 Gamma 计算，避免组合数上溢与幂下溢先形成 $\infty \cdot 0$：

```cpp
long double binomialPmf(int n, int k, long double p)
{
    // n、k、p 是试验数、成功数和单次成功率；返回二项分布 P(X=k)。
    assert(n >= 0 && 0 <= p && p <= 1); // 调试检查，可删。
    if (k < 0 || k > n)
    {
        return 0;
    }
    if (p == 0)
    {
        return k == 0;
    }
    if (p == 1)
    {
        return k == n;
    }
    long double z = lgammal((long double)n + 1) - lgammal((long double)k + 1)
                  - lgammal((long double)(n - k) + 1);
    z += k * logl(p) + (n - k) * log1pl(-p);
    return expl(z);
}

array<long double, 2> geomMoments(long double p)
{
    // p 是每次成功概率；返回几何分布（首次成功试验数）的 {期望, 方差}。
    assert(0 < p && p <= 1); // 调试检查，可删。
    return {1 / p, (1 - p) / (p * p)};
}
```

`binomialPmf(n,k,p)` 对 $k \notin [0,n]$ 返回 0，$p \in \{0,1\}$ 时直接返回退化分布的概率，其余情形用 `lgammal` 求对数后再取指数，按固定精度数学库调用计为 $O(1)$ 时间、$O(1)$ 空间。大 n 时对数 Gamma 相减会损失精度，极小概率仍可能下溢；`geomMoments(p)` 要求 $0 < p \le 1$，返回首次成功所需试验次数的期望 $1/p$ 与方差 $(1-p)/p^2$。

13.2.003 随机变量期望与方差

离散随机变量满足 $E[X]=\sum_i p_i x_i$、$\operatorname{Var}(X)=\sum_i p_i (x_i-E[X])^2$。直接计算 $E[X^2]-E[X]^2$ 会让两个平方量级的大数相减，在大均值、低方差数据上丢失全部有效位；代码改用带权 Welford，逐步更新总权重、均值和中心二阶矩。期望恒线性而方差含协方差项，只有协方差为零时方差才能直接相加。

```cpp
// 依赖 13.1.001 Bash 与扩展 Bash
namespace exp_var
{
array<long double, 2> moments(const vector<long double> &x, const vector<long double> &p)
{
    // x 是取值，p 是对应概率且总和为 1；返回 {期望, 方差}。
    assert(x.size() == p.size());
    assert(!x.empty());
    long double sum = 0;
```

```
    long double mean = 0;
    long double m2 = 0;
    for (int i = 0; i < (int)x.size(); i++)
    {
        assert(p[i] >= 0);
        if (p[i] == 0)
        {
            continue;
        }
        long double next = sum + p[i];
        long double d = x[i] - mean;
        mean += d * p[i] / next;
        m2 += p[i] * d * (x[i] - mean);
        sum = next;
    }
    assert(fabsl(sum - 1) <= 1E-12L);
    return {mean, max((long double)0, m2 / sum)};
}
}
```

`exp_var::moments(x, p)` 要求 `x` 与 `p` 等长、`p` 之和为 1（代码用 `assert` 容差校验），返回 `{期望，方差}`，$O(n)$ 时间、$O(1)$ 额外空间。各标准分布的期望方差闭式见 13.2.002 常见概率分布；状态相互依赖、无法直接求和时应建立条件期望方程（全期望公式 $E[X] = E[E[X \mid Y]]$）逐步消元，这类递推常配合 13.2.001 常见概率公式 的全概率展开。

13.2.004 随机算法与 Xorshift

Xorshift 用移位异或推进非零 64 位状态，本实现采用 $13, 7, 17$ 参数。完整周期包含 1 到 $2^{64} - 1$，不包含零；无符号自然溢出与确切位宽是算法的一部分。

令 $M = 2^{64} - 1$，把输出减一得到 $[0, M)$，只接受前 $L = M - (M \bmod n)$ 个值再取模。因为 $n \mid L$，每个余数有相同数量的原像；若误按包含零的 2^{64} 个值计算阈值，会引入偏差。这只消除区间映射偏差，不保证伪随机序列独立。

```
// 依赖 13.1.001 Bash 与扩展 Bash
struct RNG
{
    // s 是 xorshift 伪随机数生成器的非零状态。
    unsigned long long s;

    RNG(unsigned long long seed = chrono::steady_clock::now().time_since_epoch().count()) :
    ↪ s(seed ? seed : 1)
    {
        // seed 是随机种子；0 会替换为 1。
    }

    unsigned long long next()
    {
        // 无参数；推进随机数状态并返回下一个随机数。
```

```
        s ^= s << 13;
        s ^= s >> 7;
        s ^= s << 17;
        return s;
    }

    unsigned long long operator()(unsigned long long n)
    {
        // n 是正上界；返回 [0,n) 内的伪随机整数；无符号类型用于 64 位移位与自然溢出。
        assert(n > 0); // 调试检查，可删。
        const auto mx = numeric_limits<unsigned long long>::max();
        unsigned long long lim = mx - mx % n;
        unsigned long long x;
        do
        {
            x = next() - 1; // 非零输出平移到 [0,2^64-2]，共有 2^64-1 个值。
        } while (x >= lim);
        return x % n;
    }
};
```

`RNG rng(seed)` 用指定种子构造，零种子替换为 1；默认取时钟计数。`next()` 为 $O(1)$ 时间，返回非零机器字。`rng(n)` 要求 $1 \le n \le 2^{64}-1$，返回 $[0,n)$ 内伪随机整数；在独立均匀非零输出模型下接受率大于 $1/2$，期望 $O(1)$ 时间，额外空间 $O(1)$，不承诺固定最坏调用次数。固定种子可复现，但有界采样结果随本次区间修正而改变。需标准库发生器时用 `mt19937_64` 配合 `uniform_int_distribution`；概率分析见 13.2.006 概率界与集中不等式。

13.2.005 期望 DP 与随机游走

期望 DP 把“从状态 u 出发直到终止的代价”写成

$$E_u = c_u + \sum_v p_{u,v} E_v,$$

其中 c_u 是每次访问状态 u 时支付的期望即时代价（包括终点的代价），$p_{u,v}$ 是转移概率。若转移图无环，可按逆拓扑序直接计算；有环时把所有方程移到一侧，使用高斯消元，或利用单调性与吸收条件做迭代。线性期望允许逐项求和，即使事件不独立也有 $E[\sum X_i] = \sum E[X_i]$。

```
vector<long double> expectedDAG(const vector<vector<pair<int, long double>>> &g,
                                 const vector<long double> &cost,
                                 const vector<int> &order)
{
    // g[u] 是从 u 指向后继 v 的 {v, 概率}；cost[u] 是访问 u 时支付的期望即时代价；
    // order 是全部顶点的逆拓扑排列；返回各状态期望总代价，缺失概率表示直接结束。
    assert(g.size() == cost.size() && order.size() == g.size());
    vector<long double> e(cost.size());
    for (int u : order)
    {
        e[u] = cost[u];
        for (auto [v, p] : g[u])
```

```
        {
            e[u] += p * e[v];
        }
    }
    return e;
}
```

`order` 必须是全部顶点的逆拓扑排列，概率非负、每个状态概率和至多为 1。`expectedDAG` 要求每条边都从当前状态指向已经在 `order` 中计算的后继，缺失概率表示直接终止。时间复杂度为 $O(|V|+|E|)$，空间复杂度为 $O(|V|)$。有环随机游走需要将方程组交给 9.2.002 列主元高斯消元，并先确认吸收概率为 1；否则期望可能不存在。

```
signed main()
{
    // 无参数；验证逆拓扑期望递推，返回 0。
    vector<vector<pair<int, long double>>> g(3);
    g[0] = {{1, 0.5L}, {2, 0.5L}};
    vector<long double> e = expectedDAG(g, {1, 3, 0}, {2, 1, 0});
    assert(fabsl(e[0] - 2.5L) < 1E-12L);
}
```

13.2.006 概率界与集中不等式

这些界用于选择重复次数、抽样规模和分析随机算法；独立性必须来自概率模型，不能由“用了随机数”推出。概率上界超过 1 时取 1 即可。

对任意事件 $A_1,\dots,A_m$，并集界不要求独立：

$$P\left(\bigcup_i A_i\right)\le\sum_i P(A_i).$$

若需同时控制 $m\ge 1$ 个错误事件总概率不超过 η，让每个事件概率不超过 η/m 即足够。

若 $X\ge 0$、$E[X]<\infty$ 且 $t>0$，Markov 不等式为

$$P(X\ge t)\le\frac{E[X]}{t}.$$

它来自 $t1_{X\ge t}\le X$。对 $(X-E[X])^2$ 应用同式，得到有限方差下的 Chebyshev 不等式：

$$P(|X-E[X]|\ge t)\le\frac{\operatorname{Var}(X)}{t^2},\qquad t>0.$$

独立性不是这两个界的前提；若用方差可加性计算右侧，则须另核对协方差。

对相互独立的 Bernoulli 变量 X_i，令 $X=\sum_i X_i$、$\mu=E[X]>0$，各成功率可不同。Chernoff 的常用乘法界为

$$\begin{aligned}
P(X\ge(1+\delta)\mu)&\le\left(\frac{e^{\delta}}{(1+\delta)^{1+\delta}}\right)^{\mu},&\delta>0,\\
P(X\le(1-\delta)\mu)&\le e^{-\mu\delta^2/2},&0<\delta<1,\\
P(|X-\mu|\ge\delta\mu)&\le 2e^{-\mu\delta^2/3},&0<\delta\le 1.
\end{aligned}$$

从 $P(X\ge a)\le e^{-ta}E[e^{tX}]$ $(t>0)$ 出发，独立性把矩母函数分解为乘积，再选择使界最小的 t。$\mu=0$ 时上述 Bernoulli 和恒为零，直接处理。

若相互独立的实随机变量满足 $a_i\le X_i\le b_i$，记 $D=\sum_i (b_i-a_i)^2>0$，Hoeffding 给出

$$P\left(\left|\sum_i X_i - E\left[\sum_i X_i\right]\right|\ge t\right)\le 2\exp\left(-\frac{2t^2}{D}\right),\qquad t>0.$$

$D=0$ 时变量之和为常数。特别地，N 次独立 $[0,1]$ 样本的均值 $\bar X$ 满足

$$P(|\bar X - E[\bar X]|\ge \varepsilon)\le 2e^{-2N\varepsilon^2}.$$

给定 $\varepsilon>0$、$0<\eta<1$，取 $N\ge \lceil \ln(2/\eta)/(2\varepsilon^2)\rceil$ 即足以把失败概率控制为 η。Bernoulli 和的相对偏差优先看 Chernoff；一般有界独立和的绝对偏差用 Hoeffding；只知均值或方差时分别用 Markov、Chebyshev。本条为公式，无须另封装代码；期望计算见 13.2.003 随机变量期望与方差。

13.2.007 指示变量、尾和公式与集齐问题

对事件 A 定义指示变量 1_A，有 $E[1_A]=P(A)$。若计数 $X=\sum_i 1_{A_i}$，则

$$E[X]=\sum_i P(A_i),$$

不要求事件独立。任意有限个有限方差变量满足

$$\operatorname{Var}\left(\sum_i X_i\right)=\sum_i \operatorname{Var}(X_i)+2\sum_{i<j}\operatorname{Cov}(X_i,X_j).$$

两两独立足以令协方差为零；期望的线性性无需该条件。

对非负整数随机变量 T，可逐结果写为 $T=\sum_{k\ge 0}1_{T>k}$，交换非负求和得到尾和公式：

$$E[T]=\sum_{k=0}^{\infty}P(T>k)=\sum_{k=1}^{\infty}P(T\ge k).$$

非负实随机变量相应满足 $E[T]=\int_0^\infty P(T>t)\,\mathrm{d}t$，两式允许值为 $+\infty$。对可积变量，按互斥完备事件 B_i 分类可用全期望 $E[X]=\sum_i P(B_i)E[X\mid B_i]$，零概率项跳过。

集齐问题中，每次独立等概率抽取 $n\ge 1$ 种物品之一，已收集 i 种时，等到新种类的试验数服从成功率 $(n-i)/n$ 的几何分布。因此集齐全部种类的期望为

$$E[T]=\sum_{i=0}^{n-1}\frac{n}{n-i}=nH_n.$$

不等概率且各 $p_i>0$ 时，由并集指示变量的容斥与尾和得到

$$E[T]=\sum_{\emptyset\ne S\subseteq\{1,\dots,n\}}\frac{(-1)^{|S|+1}}{\sum_{i\in S}p_i}.$$

给定某个必需种类概率为零时，集齐时间为无穷。该容斥式直接枚举为 $O(n2^n)$，预计算子集概率和可降到 $O(2^n)$，空间 $O(2^n)$；等概率情形直接用调和和，勿展开子集。

期望不是可靠的截止时间。等概率模型下，$m \ge 0$ 次抽取仍缺少某种的概率满足

$$P(T > m) \le n(1-1/n)^m \le n e^{-m/n}.$$

给定 $0 < \eta < 1$，取 $m \ge \lceil n \ln(n/\eta) \rceil$ 足以保证失败概率至多 η。这里先限制每种从未出现的概率，再用 13.2.006 并集界，无需把 “缺少各类” 视为独立事件；几何分布约定见 13.2.002 常见概率分布。

13.3 常用公式表

13.3.001 三角恒等式、导数与不定积分表

角度采用弧度制，以下实数公式均在函数有定义且分母非零的开区间内使用，积分省略 $+C$。实数幂 x^a 对一般实数 a 取 $x > 0$；指数底数 $a > 0$，对数还要求 $a \ne 1, x > 0$。

三角恒等式：

$$\sin^2 x + \cos^2 x = 1, \quad 1 + \tan^2 x = \sec^2 x, \quad 1 + \cot^2 x = \csc^2 x$$

$$\begin{aligned} \sin(x \pm y) &= \sin x \cos y \pm \cos x \sin y, \\ \cos(x \pm y) &= \cos x \cos y \mp \sin x \sin y, \\ \tan(x \pm y) &= \frac{\tan x \pm \tan y}{1 \mp \tan x \tan y}. \end{aligned}$$

$$\begin{aligned} \sin 2x &= 2 \sin x \cos x, \\ \cos 2x &= \cos^2 x - \sin^2 x = 2\cos^2 x - 1 = 1 - 2\sin^2 x, \\ \tan 2x &= \frac{2\tan x}{1 - \tan^2 x}. \end{aligned}$$

$$\sin^2 \tfrac{x}{2} = \frac{1 - \cos x}{2}, \quad \cos^2 \tfrac{x}{2} = \frac{1 + \cos x}{2}, \quad \tan \tfrac{x}{2} = \frac{1 - \cos x}{\sin x} = \frac{\sin x}{1 + \cos x}$$

积化和差：$\sin x \cos y = \tfrac{1}{2}[\sin(x+y) + \sin(x-y)]$，$\cos x \cos y = \tfrac{1}{2}[\cos(x+y) + \cos(x-y)]$，$\sin x \sin y = \tfrac{1}{2}[\cos(x-y) - \cos(x+y)]$。

和差化积：$\sin x + \sin y = 2 \sin \tfrac{x+y}{2} \cos \tfrac{x-y}{2}$，$\cos x + \cos y = 2 \cos \tfrac{x+y}{2} \cos \tfrac{x-y}{2}$，$\cos x - \cos y = -2 \sin \tfrac{x+y}{2} \sin \tfrac{x-y}{2}$。

万能代换 $t = \tan \tfrac{x}{2}$：$\sin x = \dfrac{2t}{1+t^2}$，$\cos x = \dfrac{1-t^2}{1+t^2}$，$\tan x = \dfrac{2t}{1-t^2}$。

导数表（法则：$(uv)' = u'v + uv'$，$\left(\tfrac{u}{v}\right)' = \dfrac{u'v - uv'}{v^2}$，$[f(g(x))]' = f'(g)\,g'$）。基本初等函数与三角：

f	f'	f	f'	f	f'
x^a	ax^{a-1}	a^x	$a^x \ln a$	$\log_a x$	$\dfrac{1}{x \ln a}$
e^x	e^x	$\ln x$	$\dfrac{1}{x}$		
$\sin x$	$\cos x$	$\cos x$	$-\sin x$	$\tan x$	$\sec^2 x$
$\cot x$	$-\csc^2 x$	$\sec x$	$\sec x \tan x$	$\csc x$	$-\csc x \cot x$

反三角、双曲与反双曲导数：arcsin, arccos 要求 $|x| < 1$，arcsec, arccsc 要求 $|x| > 1$；arccosh 要求 $x > 1$，arctanh 要求 $|x| < 1$，arccoth 要求 $|x| > 1$，arcsech 要求 $0 < x < 1$，arccsch, coth, csch 要求 $x \neq 0$。反余切取值为 $(0, \pi)$，反正割取值为 $[0, \pi] \setminus \{\pi/2\}$，反余割取值为 $[-\pi/2, \pi/2] \setminus \{0\}$。

f	f'	f	f'
$\arcsin x$	$\dfrac{1}{\sqrt{1-x^2}}$	$\arccos x$	$-\dfrac{1}{\sqrt{1-x^2}}$
$\arctan x$	$\dfrac{1}{1+x^2}$	$\operatorname{arccot} x$	$-\dfrac{1}{1+x^2}$
$\operatorname{arcsec} x$	$\dfrac{1}{\lvert x\rvert\sqrt{x^2-1}}$	$\operatorname{arccsc} x$	$-\dfrac{1}{\lvert x\rvert\sqrt{x^2-1}}$
$\sinh x$	$\cosh x$	$\cosh x$	$\sinh x$
$\tanh x$	$\operatorname{sech}^2 x$	$\coth x$	$-\operatorname{csch}^2 x$
$\operatorname{sech} x$	$-\operatorname{sech} x \tanh x$	$\operatorname{csch} x$	$-\operatorname{csch} x \coth x$
$\operatorname{arcsinh} x$	$\dfrac{1}{\sqrt{x^2+1}}$	$\operatorname{arccosh} x$	$\dfrac{1}{\sqrt{x^2-1}}$
$\operatorname{arctanh} x$	$\dfrac{1}{1-x^2}$	$\operatorname{arccoth} x$	$\dfrac{1}{1-x^2}$
$\operatorname{arcsech} x$	$-\dfrac{1}{x\sqrt{1-x^2}}$	$\operatorname{arccsch} x$	$-\dfrac{1}{\lvert x\rvert\sqrt{1+x^2}}$

不定积分表使用分部积分 $\int u\,dv = uv - \int v\,du$。基本表中 $1/(a^2+x^2)$ 要求 $a \neq 0$；根式分母要求根内严格为正，对数自变量非零。

被积函数	原函数	被积函数	原函数
$x^a\ (a \neq -1)$	$\dfrac{x^{a+1}}{a+1}$	$\dfrac{1}{x}$	$\ln\lvert x\rvert$
e^x	e^x	a^x	$\dfrac{a^x}{\ln a}$
$\sin x$	$-\cos x$	$\cos x$	$\sin x$
$\sec^2 x$	$\tan x$	$\csc^2 x$	$-\cot x$
$\dfrac{1}{a^2+x^2}$	$\dfrac{1}{a}\arctan\dfrac{x}{a}$	$\dfrac{1}{\sqrt{1-x^2}}$	$\arcsin x$
$\dfrac{1}{\sqrt{x^2 \pm a^2}}$	$\ln\left\lvert x + \sqrt{x^2 \pm a^2}\right\rvert$	$\ln x$	$x \ln x - x$

二次三项式 $ax^2 + bx + c\ (a > 0)$：

$$\int \frac{\mathrm{d}x}{ax^2+bx+c} = \begin{cases} \dfrac{2}{\sqrt{4ac-b^2}} \arctan \dfrac{2ax+b}{\sqrt{4ac-b^2}}, & b^2 < 4ac, \\ -\dfrac{2}{2ax+b}, & b^2 = 4ac, \\ \dfrac{1}{\sqrt{b^2-4ac}} \ln \left| \dfrac{2ax+b-\sqrt{b^2-4ac}}{2ax+b+\sqrt{b^2-4ac}} \right|, & b^2 > 4ac. \end{cases}$$

$$\int \frac{x\,\mathrm{d}x}{ax^2+bx+c} = \frac{1}{2a}\ln|ax^2+bx+c| - \frac{b}{2a}\int \frac{\mathrm{d}x}{ax^2+bx+c}$$

以下两式要求 $a>0$、根式内严格为正；第二式还要求 $b^2+4ac>0$。第一式对数内为零的退化分支须另算：若 $b^2=4ac$ 且 $2ax+b<0$，原函数为 $-\ln|2ax+b|/\sqrt{a}$。

$$\int \frac{\mathrm{d}x}{\sqrt{ax^2+bx+c}} = \frac{1}{\sqrt{a}}\ln\left|2ax+b+2\sqrt{a}\sqrt{ax^2+bx+c}\right|,$$
$$\int \frac{\mathrm{d}x}{\sqrt{c+bx-ax^2}} = \frac{1}{\sqrt{a}}\arcsin\frac{2ax-b}{\sqrt{b^2+4ac}}.$$

$$\int \frac{\mathrm{d}x}{\sqrt{(x-a)(b-x)}} = 2\arcsin\sqrt{\frac{x-a}{b-a}},$$
$$\int \sqrt{(x-a)(b-x)}\,\mathrm{d}x = \frac{2x-a-b}{4}\sqrt{(x-a)(b-x)} + \frac{(b-a)^2}{4}\arcsin\sqrt{\frac{x-a}{b-a}}.$$

上式要求 $a<b$ 且 $a<x<b$；端点处按单侧连续延拓。

三角函数：

$$\int \tan x\,\mathrm{d}x = -\ln|\cos x|,$$
$$\int \cot x\,\mathrm{d}x = \ln|\sin x|,$$
$$\int \sec x\,\mathrm{d}x = \ln|\sec x+\tan x|,$$
$$\int \csc x\,\mathrm{d}x = \ln\left|\tan\frac{x}{2}\right|.$$

$$\int \sec x\tan x\,\mathrm{d}x = \sec x,$$
$$\int \csc x\cot x\,\mathrm{d}x = -\csc x,$$
$$\int \sin^2 x\,\mathrm{d}x = \frac{x}{2}-\frac{1}{4}\sin 2x,$$
$$\int \cos^2 x\,\mathrm{d}x = \frac{x}{2}+\frac{1}{4}\sin 2x.$$

降次递推（前两式要求 $n\neq 0$；$n=0$ 时被积函数为 1，原函数为 x）：

$$\int \sin^n x\,\mathrm{d}x = -\frac{1}{n}\sin^{n-1} x\cos x + \frac{n-1}{n}\int \sin^{n-2} x\,\mathrm{d}x,$$
$$\int \cos^n x\,\mathrm{d}x = \frac{1}{n}\cos^{n-1} x\sin x + \frac{n-1}{n}\int \cos^{n-2} x\,\mathrm{d}x.$$

$$\int \cos^m x\sin^n x\,\mathrm{d}x = \frac{1}{m+n}\cos^{m-1} x\sin^{n+1} x + \frac{m-1}{m+n}\int \cos^{m-2} x\sin^n x\,\mathrm{d}x, \qquad m+n\neq 0.$$

有理三角式先令 $t=\tan(x/2)$。下式要求 $a\neq 0$，并避开被积函数与换元的奇点：

$$\int \frac{\mathrm{d}x}{a+b\sin x} = \begin{cases} \dfrac{2}{\sqrt{a^2-b^2}}\arctan\dfrac{a\tan\frac{x}{2}+b}{\sqrt{a^2-b^2}}, & a^2>b^2, \\ -\dfrac{2}{a\tan\frac{x}{2}+b}, & a^2=b^2,\ (a,b)\neq(0,0), \\ \dfrac{1}{\sqrt{b^2-a^2}}\ln\left|\dfrac{a\tan\frac{x}{2}+b-\sqrt{b^2-a^2}}{a\tan\frac{x}{2}+b+\sqrt{b^2-a^2}}\right|, & a^2<b^2. \end{cases}$$

当 $a=0, b\neq 0$ 时，$\int \mathrm{d}x/(b\sin x) = \ln|\tan(x/2)|/b$；$a=b=0$ 时无定义。

当 $ab\neq 0$ 时，

$$\int \frac{\mathrm{d}x}{a^2\cos^2 x + b^2\sin^2 x} = \frac{1}{ab}\arctan\left(\frac{b}{a}\tan x\right).$$

当 $a\neq 0$ 时，

$$\int x\sin ax\,\mathrm{d}x = \frac{1}{a^2}\sin ax - \frac{1}{a}x\cos ax,$$
$$\int x\cos ax\,\mathrm{d}x = \frac{1}{a^2}\cos ax + \frac{1}{a}x\sin ax.$$

当 $a=0$ 时，两式的原函数分别为 0 与 $\frac{x^2}{2}$。

反三角函数要求 $a>0$；前两式在 $|x|<a$ 上求导，端点取连续值：

$$\int \arcsin\frac{x}{a}\,\mathrm{d}x = x\arcsin\frac{x}{a} + \sqrt{a^2-x^2},$$
$$\int \arccos\frac{x}{a}\,\mathrm{d}x = x\arccos\frac{x}{a} - \sqrt{a^2-x^2},$$
$$\int \arctan\frac{x}{a}\,\mathrm{d}x = x\arctan\frac{x}{a} - \frac{a}{2}\ln(a^2+x^2).$$

指数函数：

$$\int xe^{ax}\,\mathrm{d}x = \frac{1}{a^2}(ax-1)e^{ax},$$
$$\int x^n e^{ax}\,\mathrm{d}x = \frac{1}{a}x^n e^{ax} - \frac{n}{a}\int x^{n-1}e^{ax}\,\mathrm{d}x, \qquad a\neq 0.$$

当 $a=0$ 时，第一式原函数为 $\frac{x^2}{2}$，第二式退化为普通幂函数积分。

$$\int e^{ax}\sin bx\,\mathrm{d}x = \frac{e^{ax}(a\sin bx - b\cos bx)}{a^2+b^2},$$
$$\int e^{ax}\cos bx\,\mathrm{d}x = \frac{e^{ax}(b\sin bx + a\cos bx)}{a^2+b^2}, \qquad a^2+b^2>0.$$

当 $a=b=0$ 时，两式的原函数分别为 0 与 x。

对数函数要求 $x>0$，第二式另要求 $x\neq 1$：

$$\int \ln x\,\mathrm{d}x = x\ln x - x,$$
$$\int \frac{\mathrm{d}x}{x\ln x} = \ln|\ln x|,$$
$$\int x^n \ln x\,\mathrm{d}x = \frac{x^{n+1}}{n+1}\left(\ln x - \frac{1}{n+1}\right), \qquad n\neq -1.$$

当 $n=-1$ 时，$\int x^{-1}\ln x\,\mathrm{d}x = \frac{1}{2}(\ln x)^2$；另有 $\int (\ln x)^n\,\mathrm{d}x = x(\ln x)^n - n\int(\ln x)^{n-1}\,\mathrm{d}x$。

C++ 标准库 `<cmath>` 提供相关特殊函数，依本章前导省略命名空间前缀。下列 Beta 积分要求 $x,y>0$；指数尾积分要求 $x>0$；椭圆半轴满足 $a\geq b>0$：$\int_0^1 t^{x-1}(1-t)^{y-1}\mathrm{d}t = B(x,y) =$ `beta(x,y)`；$\Gamma(x) =$ `tgamma(x)`；指数积分 $\mathrm{Ei}(x) =$ `expint(x)`，于是 $\int_x^\infty \frac{e^{-t}}{t}\mathrm{d}t = -\mathrm{Ei}(-x) =$ `-expint(-x)`；$\zeta(x) =$ `riemann_zeta(x)`；$\frac{2}{\sqrt{\pi}}\int_0^x e^{-t^2}\mathrm{d}t =$ `erf(x)`；第一 / 第二类椭圆积分 `ellint_1(k,phi)`、`ellint_2(k,phi)`（椭圆周长 $=4a$ `ellint_2(e,pi/2)`，离心率 $e=\sqrt{1-b^2/a^2}$）。

多项式的逐项求导/积分可直接对系数操作：

```cpp
vector<long double> derivative(const vector<long double> &a)
{
    // a[i] 是 x^i 系数；返回导函数系数，常数项导数为 {0}。
    if (a.size() <= 1)
    {
        return {0};
    }
    vector<long double> b(a.size() - 1);
    for (int i = 1; i < (int)a.size(); i++)
    {
        b[i - 1] = i * a[i];
    }
    return b;
}

```

```cpp
vector<long double> integral(const vector<long double> &a, long double c = 0)
{
    // a[i] 是 x^i 系数，c 是积分常数；返回一个原函数的系数。
    vector<long double> b(a.size() + 1);
    b[0] = c;
    for (int i = 0; i < (int)a.size(); i++)
    {
        b[i + 1] = a[i] / (i + 1);
    }
    return b;
}
```

`derivative(a)` 与 `integral(a,c)` 都按低次在前的系数表操作，分别返回导函数与常数项为 c 的原函数，时间与空间均为 $O(n)$；常数多项式的导数约定为 $\{0\}$。

13.3.002 调和数与幂和

用于估算复杂度、化简整式求和。$\gamma \approx 0.5772156649$ 为 Euler-Mascheroni 常数。

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},$$
$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$
$$\sum_{i=1}^{n} i^3 = \left(\frac{n(n+1)}{2}\right)^2.$$

$$\sum_{i=1}^{n} i^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}, \quad \sum_{i=1}^{n} i^5 = \frac{n^2(n+1)^2(2n^2+2n-1)}{12}$$

一般幂和是关于 n 的 $k+1$ 次多项式，可用 Bernoulli 数（8.2.007）闭式或对 $k+2$ 个点拉格朗日求值：

$$\sum_{i=0}^{n-1} i^k = \frac{1}{k+1} \sum_{j=0}^{k} \binom{k+1}{j} B_j \, n^{k+1-j}$$

调和数与整除分块的渐近式（右侧仅用于估计，不作精确值）：

$$H_n = \sum_{i=1}^{n} \frac{1}{i} = \ln n + \gamma + \frac{1}{2n} - \frac{1}{12n^2} + O(n^{-4}),$$
$$\sum_{i=1}^{n} \left\lfloor \frac{n}{i} \right\rfloor = n \ln n + (2\gamma - 1)n + O(\sqrt{n}).$$

两条精确调和恒等式为

$$\sum_{k=1}^{n} H_k = (n+1)H_n - n, \qquad \sum_{k=1}^{n} kH_k = \frac{n(n+1)}{2}H_n - \frac{n(n-1)}{4}.$$

```cpp
array<__int128, 3> powerSums(int n)
{
    // n 是非负整数；返回 1..n 的一次、二次、三次幂和。
    assert(0 <= n && n <= 5107605666LL); // 调试检查，可删。
    __int128 x = n;
    __int128 s1 = x * (x + 1) / 2;
    __int128 a = x;
    __int128 b = x + 1;
    __int128 c = 2 * x + 1;
    if (a % 2 == 0)
    {
        a /= 2;
    }
    else
    {
        b /= 2;
    }
    if (a % 3 == 0)
    {
        a /= 3;
    }
    else if (b % 3 == 0)
    {
        b /= 3;
    }
    else
    {
        c /= 3;
    }
    return {s1, a * b * c, s1 * s1};
}

long double harmonic(int n)
{
    // n 是非负整数；返回直接累加得到的第 n 个调和数。
    assert(n >= 0); // 调试检查，可删。
    long double ans = 0;
    for (int i = 1; i <= n;)
    {
        ans += 1.0L / i;
        if (i == n)
        {
            break;
        }
        i++;
    }
    return ans;
}
```

`powerSums(n)` 用 `__int128` 返回前三个精确幂和，要求 $0 \le n \le 5107605666$；该上界恰保证最大的三次幂和不超过 $2^{127}-1$，二次幂和先约分再乘，避免中间式先溢出。时间、空间均为 $O(1)$。`harmonic(n)` 直接累加，时间 $O(n)$、额外空间 $O(1)$；大 n 只需近似时用上面的渐近式。

13.3.003 三角形、四边形与圆公式

设三角形三边 a,b,c 分别对应角 A,B,C，半周长为 $s=(a+b+c)/2$，面积为 S，内外接圆半径为 r,R。以下三角形公式要求三边为正且严格满足三角不等式：

$$\frac{a}{\sin A}=\frac{b}{\sin B}=\frac{c}{\sin C}=2R,\qquad c^2=a^2+b^2-2ab\cos C$$

$$S=\frac{ab\sin C}{2}=\sqrt{s(s-a)(s-b)(s-c)}=rs=\frac{abc}{4R}$$

量	公式
a 边中线 m_a	$\frac{1}{2}\sqrt{2b^2+2c^2-a^2}=\frac{1}{2}\sqrt{b^2+c^2+2bc\cos A}$
A 角平分线 t_a	$\frac{\sqrt{bc((b+c)^2-a^2)}}{b+c}=\frac{2bc\cos(A/2)}{b+c}$
a 边高 h_a	$b\sin C=c\sin B=\sqrt{b^2-\left(\frac{a^2+b^2-c^2}{2a}\right)^2}$
内切圆半径 r	$\frac{S}{s}=4R\sin\frac{A}{2}\sin\frac{B}{2}\sin\frac{C}{2}=\sqrt{\frac{(s-a)(s-b)(s-c)}{s}}=s\tan\frac{A}{2}\tan\frac{B}{2}\tan\frac{C}{2}$
外接圆半径 R	$\frac{abc}{4S}=\frac{a}{2\sin A}=\frac{b}{2\sin B}=\frac{c}{2\sin C}$

凸四边形边长依次为 a,b,c,d，对角线为 D_1,D_2，对角线夹角为 α，两条对角线中点距离为 M，则

$$a^2+b^2+c^2+d^2=D_1^2+D_2^2+4M^2,\qquad S=\frac{1}{2}D_1D_2\sin\alpha.$$

若四边形内接于圆，另有 Ptolemy 定理与 Brahmagupta 公式

$$ac+bd=D_1D_2,\qquad S=\sqrt{(s-a)(s-b)(s-c)(s-d)},$$

其中 $s=\frac{1}{2}(a+b+c+d)$。

正 $n\ge 3$ 边形的中心角 $\alpha=\frac{2\pi}{n}$、内角 $\beta=\frac{(n-2)\pi}{n}$。边长与面积满足

$$a=2\sqrt{R^2-r^2}=2R\sin\frac{\alpha}{2}=2r\tan\frac{\alpha}{2},$$

$$S=\frac{nar}{2}=nr^2\tan\frac{\alpha}{2}=\frac{nR^2\sin\alpha}{2}=\frac{na^2}{4\tan(\alpha/2)}.$$

下表取半径 $r>0$、劣弧圆心角 $0\le\theta\le\pi$；l 为该弧长度，a 为弦长，h 为劣弓高：

量	公式
弧长	$l=r\theta$
弦长	$a=2\sqrt{2rh-h^2}=2r\sin\frac{\theta}{2}$
弓高	$h=r-\sqrt{r^2-\frac{a^2}{4}}=r\left(1-\cos\frac{\theta}{2}\right)=\frac{a}{2}\tan\frac{\theta}{4}$
扇形面积	$S_1=\frac{rl}{2}=\frac{r^2\theta}{2}$
弓形面积	$S_2=\frac{rl-a(r-h)}{2}=\frac{r^2(\theta-\sin\theta)}{2}$

退化（三点共线）时 $S=0$，代入前先验三角不等式，`acos` 参数需 `clamp` 到 $[-1,1]$ 防浮点越界：

```cpp
// 依赖 13.1.001 Bash 与扩展 Bash
array<long double, 3> triangle(long double a, long double b, long double c)
{
    // a、b、c 是非退化三角形边长；返回 {面积，c 对应角（弧度），外接圆半径}。
    assert(a > 0 && b > 0 && c > 0 && a + b > c && a + c > b && b + c > a);
    long double s = (a + b + c) / 2;
    long double area = sqrtl(s * (s - a) * (s - b) * (s - c));
    long double angle = acosl(clamp((a * a + b * b - c * c) / (2 * a * b),
                                    -1.0L, 1.0L));
    return {area, angle, a * b * c / (4 * area)};
}
```

`triangle(a,b,c)` 返回面积、角 C 与外接圆半径；表中所有角均用弧度。代码为 $O(1)$，接近退化时应使用与输入尺度匹配的浮点误差策略。

13.3.004 常用面积、体积公式

平面面积还常用：平行四边形 ah，梯形 $\frac{1}{2}(a+b)h$，圆 πr^2，椭圆 πab。三角形、正多边形、扇形与弓形的完整公式见 13.3.003 三角形、四边形与圆公式。下表的圆柱、圆锥和圆台均指直圆体；A,A_1,A_2 是底面积，p,p_1,p_2 是底面周长，h 是垂高，l 是斜高或母线。

立体	侧面积 L	全表面积 T	体积 V
直棱柱	ph	$ph+2A$	Ah
正棱锥	$\frac{1}{2}pl$	$\frac{1}{2}pl+A$	$\frac{1}{3}Ah$
正棱台	$\frac{1}{2}(p_1+p_2)l$	$\frac{1}{2}(p_1+p_2)l+A_1+A_2$	$\frac{h}{3}(A_1+A_2+\sqrt{A_1A_2})$
圆柱（半径 r）	$2\pi rh$	$2\pi r(h+r)$	πr^2h
圆锥（底半径 r）	πrl	$\pi r(l+r)$	$\frac{1}{3}\pi r^2h$
圆台（底半径 r_1,r_2）	$\pi(r_1+r_2)l$	$\pi(r_1+r_2)l+\pi(r_1^2+r_2^2)$	$\frac{\pi h}{3}(r_1^2+r_1r_2+r_2^2)$
球（半径 R）	—	$4\pi R^2$	$\frac{4}{3}\pi R^3$
球台（球半径 R，底圆半径 r_1,r_2）	$2\pi Rh$	$\pi(2Rh+r_1^2+r_2^2)$	$\frac{\pi h}{6}(3r_1^2+3r_2^2+h^2)$

立体	侧面积 L	全表面积 T	体积 V
球扇形（球半径 R，球冠高 h、底圆半径 r_0）	—	$\pi R(2h + r_0)$	$\frac{2}{3}\pi R^2 h$

圆锥母线 $l = \sqrt{h^2 + r^2}$，圆台母线 $l = \sqrt{h^2 + (r_1 - r_2)^2}$。斜高、垂高、侧面积与全表面积不可混用；所有长度必须先统一单位。常用圆体的关键量：

```cpp
// 依赖 13.1.001 Bash 与扩展 Bash
array<long double, 4> roundBodies(long double r, long double h, long double r2)
{
    // r 是圆/球半径及圆台第一底半径，h 是圆台高，r2 是另一底半径；返回 {圆面积，球面积，球体积，圆台体积}。
    assert(r >= 0 && h >= 0 && r2 >= 0);
    long double pi = acosl(-1.0L);
    return {pi * r * r, 4 * pi * r * r,
            4 * pi * r * r * r / 3,
            pi * h * (r * r + r * r2 + r2 * r2) / 3};
}
```

`roundBodies(r,h,r2)` 只返回圆面积、球面积、球体积和圆台体积，用于快速计算；其余量直接按表代入。函数为 $O(1)$，输入须为非负有限实数。

13.3.005 日期公式：Zeller / 基姆拉尔森

由公历年月日直接算星期时，可把 1、2 月视作上一年的 13、14 月；更通用的做法是先把日期映射为相对 1970-01-01 的纪元日数，再由日数模 7 求星期。完整、唯一的 `Date` 实现放在 14.1.005 日期与时间：它提供闰年、月天数、日期与纪元日互转及星期接口。本条只保留公式入口，避免两份日期边界和注释漂移。

13.3.006 常用常数与 NTT / 哈希素数表

常数：$\pi = \arccos(-1)$，$e = \exp(1)$，黄金比 $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.6180339887$，Euler-Mascheroni $\gamma \approx 0.5772156649$，$\ln 2 \approx 0.6931471806$。

NTT 友好素数 $p = c \cdot 2^k + 1$，原根 g，可做长度至多 2^k 的变换：

p	分解	g	最大 2^k
998244353	$119 \cdot 2^{23} + 1$	3	2^{23}
1004535809	$479 \cdot 2^{21} + 1$	3	2^{21}
469762049	$7 \cdot 2^{26} + 1$	3	2^{26}
167772161	$5 \cdot 2^{25} + 1$	3	2^{25}
1224736769	$73 \cdot 2^{24} + 1$	3	2^{24}
2013265921	$15 \cdot 2^{27} + 1$	31	2^{27}
2281701377	$17 \cdot 2^{27} + 1$	3	2^{27}

需要更大变换长度（如大位宽 NTT、超长卷积或挑三模数）时可参考下表，$p = r \cdot 2^k + 1$，g 为最小原根：

p	r	k	g
17	1	4	3
97	3	5	5
193	3	6	5
257	1	8	3
7681	15	9	17
12289	3	12	11
40961	5	13	3
65537	1	16	3
786433	3	18	10
5767169	11	19	3
7340033	7	20	3
23068673	11	21	3
104857601	25	22	3
167772161	5	25	3
469762049	7	26	3
998244353	119	23	3
1004535809	479	21	3
2013265921	15	27	31
2281701377	17	27	3

较大的模数单独列出，避免打印时与前四列挤在一起：

p	r	k	g
3221225473	3	30	5
75161927681	35	31	3
77309411329	9	33	7
206158430209	3	36	22
2061584302081	15	37	7
2748779069441	5	39	3
6597069766657	3	41	5
39582418599937	9	42	5
79164837199873	9	43	5
263882790666241	15	44	7
1231453023109121	35	45	3
1337006139375617	19	46	3
3799912185593857	27	47	5
4222124650659841	15	48	19
7881299347898369	7	50	6
31525197391593473	7	52	3
180143985094819841	5	55	6

p	r	k	g
1945555039024054273	27	56	5
4179340454199820289	29	57	3

任意模数卷积常取 $\{998244353, 1004535809, 469762049\}$ 三模数 CRT 合并。设三模乘积为 P；无符号恢复要求真实卷积系数落在 $[0, P)$，有符号对称恢复要求绝对值严格小于 $P/2$。若输入系数绝对值分别不超过 A, B，还须核对 $\min(n, m)AB < P/2$，不能只凭“十亿级系数”判断安全。

字符串/表哈希大素数：单哈希用 10^9+7、10^9+9、998244353；需抗 hack 的双哈希取 Mersenne 素数 $2^{31}-1=2147483647$ 与 $2^{61}-1=2305843009213693951$（后者乘法用 `__int128` 或快速取模）。基数取大于字符集的随机奇数（如 131、13331 或运行时随机）。

```cpp
constexpr long double pi = 3.14159265358979323846264338327950288L;
constexpr long double e = 2.71828182845904523536028747135266249L;
constexpr long double phi = 1.61803398874989484820458683436563811L;
// ntt[i] 三维依次是质数、原根、可用的最大二次幂指数。
constexpr array<array<int, 3>, 7> ntt = {{{998244353, 3, 23},
                                          {1004535809, 3, 21},
                                          {469762049, 3, 26},
                                          {167772161, 3, 25},
                                          {1224736769, 3, 24},
                                          {2013265921, 31, 27},
                                          {2281701377, 3, 27}}};
```

常用数量级只保留明确标注的上下界附近素数；需要随机素数时仍应运行素性测试，不能把速查表当成随机源。

k	不小于 10^k 的常用素数	小于 10^{k+1} 的靠后素数
2	101	997
3	1009	9973
4	10007	99991
5	100003	999983
6	1000003	9999991
7	10000019	99999989
8	100000007	999999937
9	1000000007	9999999967

13.3.007 数论数量级与极值表

赛场估复杂度、判溢出用。除注明外均指 64 位有符号范围；细表 $k=19$ 一行超出该范围，需无符号 64 位或更宽整数。计算某具体上界的最大约数个数或反素数本身用 7.2.007 因数最多的数。

Stirling 估计 $\ln(n!) = n\ln n - n + \frac{1}{2}\ln(2\pi n) + O(1/n)$；阶乘增长极快，64 位内最大阶乘是 $20! = 2432902008176640000$（$21!$ 溢出），$13!$ 已超 32 位。

上界 N	$\pi(N)$ 素数个数	$\max_{n\le N} d(n)$
10^3	168	32
10^6	78498	240
10^9	50847534	1344
10^{12}	37607912018	6720
10^{18}	$\approx 2.47\times 10^{16}$	103680

约数个数最大值的部分达到者为

$$840,\quad 720720,\quad 735134400,$$

分别对应前三行；10^{18} 对应的达到者是

$$897612484786617600.$$

上界 N	$\max_{n\le N}\omega(n)$	最大素数间隙
10^3	4	20
10^4	5	36
10^5	6	72
10^6	7	114
10^7	8	154
10^8	8	220
10^9	9	282
10^{10}	10	354
10^{11}	10	464
10^{12}	11	540
10^{13}	12	674
10^{14}	12	804
10^{15}	13	906
10^{16}	13	1132
10^{17}	14	1220
10^{18}	15	1442

ω 上界即前若干最小素数之积不超过 N：$2\cdot 3\cdots 23 = 223092870 < 10^9$（9 个），前 15 个素数之积 $\approx 6.15\times 10^{17} < 10^{18}$。表中素数间隙指相邻素数 $p<q\le N$ 的最大 $q-p$，因此只有两个端点都不超过 N 的间隙才计入。约数和与欧拉函数的极值须分别分析，不能由约数个数表推出。

逐数量级细表：10^k 前 / 后第一个素数（以相对 10^k 的偏移给出，如 10^9 前第一个素数是 10^9-63），以及 $\le 10^k$ 的 $\max\omega(n)$、$\max d(n)$。挑「贴着 10^k 的素数模数」或估极值时直接查：

k	10^k 前第一质数	10^k 后第一质数	$\max \omega$	$\max d$	k	10^k 前第一质数	10^k 后第一质数	$\max \omega$	$\max d$
1	-3	$+1$	2	4	11	-23	$+3$	10	4032
2	-3	$+1$	3	12	12	-11	$+39$	11	6720
3	-3	$+9$	4	32	13	-29	$+37$	12	10752
4	-27	$+7$	5	64	14	-27	$+31$	12	17280
5	-9	$+3$	6	128	15	-11	$+37$	13	26880
6	-17	$+3$	7	240	16	-63	$+61$	13	41472
7	-9	$+19$	8	448	17	-3	$+3$	14	64512
8	-11	$+7$	8	768	18	-11	$+3$	15	103680
9	-63	$+7$	9	1344	19	-39	$+51$	15	161280
10	-33	$+19$	10	2304					

13.3.008 裂项相消、有限差分与几何级数

裂项相消先把通项写成相邻两项之差。对整数 $a \le b$，最基本的望远镜求和为

$$\sum_{k=a}^{b} \bigl(f(k+1)-f(k)\bigr) = f(b+1)-f(a).$$

下列分式要求求和范围内的分母非零；给出的有限和均取整数 $n \ge 1$：

$$\begin{aligned}
\frac{1}{k(k+1)} &= \frac{1}{k}-\frac{1}{k+1},\\
\sum_{k=1}^{n} \frac{1}{k(k+1)} &= 1-\frac{1}{n+1},\\
\frac{1}{k(k+1)(k+2)} &= \frac{1}{2}\left(\frac{1}{k(k+1)}-\frac{1}{(k+1)(k+2)}\right),\\
\sum_{k=1}^{n} \frac{1}{k(k+1)(k+2)} &= \frac{1}{4}-\frac{1}{2(n+1)(n+2)}.
\end{aligned}$$

阶乘分母也常直接裂成相邻两项：

$$\frac{k}{(k+1)!} = \frac{1}{k!}-\frac{1}{(k+1)!}, \qquad \sum_{k=1}^{n} \frac{k}{(k+1)!} = 1-\frac{1}{(n+1)!}.$$

有限几何级数及其加权式如下。对整数 $n \ge 0$，令 $m = n+1$。当 $q \ne 1$ 时，定义

$$\begin{aligned}
P_2(m,q) &= m^2(1-q)^2 + 2mq(1-q) + q(1+q),\\
P_3(m,q) &= m^3(1-q)^3 + 3m^2q(1-q)^2\\
&\quad + 3mq(1+q)(1-q) + q(1+4q+q^2),\\
P_4(m,q) &= m^4(1-q)^4 + 4m^3q(1-q)^3\\
&\quad + 6m^2q(1+q)(1-q)^2\\
&\quad + 4mq(1+4q+q^2)(1-q)\\
&\quad + q(1+11q+11q^2+q^3).
\end{aligned}$$

则二、三、四阶加权和为

$$\begin{aligned}
\sum_{x=0}^{n} x^2 q^x &= \frac{q(1+q) - q^m P_2(m,q)}{(1-q)^3},\\
\sum_{x=0}^{n} x^3 q^x &= \frac{q(1+4q+q^2) - q^m P_3(m,q)}{(1-q)^4},\\
\sum_{x=0}^{n} x^4 q^x &= \frac{q(1+11q+11q^2+q^3) - q^m P_4(m,q)}{(1-q)^5}.
\end{aligned}$$

零阶与一阶式为

$$\sum_{x=0}^{n} q^x = \frac{1-q^m}{1-q}, \qquad \sum_{x=0}^{n} x q^x = \frac{q - mq^m + (m-1)q^{m+1}}{(1-q)^2}.$$

在模意义下，上述闭式还要求 $1-q$ 可逆。若 $q=1$，改用

$$\begin{aligned}
\sum_{x=0}^{n} x^2 &= \frac{n(n+1)(2n+1)}{6},\\
\sum_{x=0}^{n} x^3 &= \left(\frac{n(n+1)}{2}\right)^2,\\
\sum_{x=0}^{n} x^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}.
\end{aligned}$$

这些式子可从有限几何级数反复求导并乘以 q 得到。有限差分定义为 $\Delta f(k) = f(k+1) - f(k)$；对整数 $m \ge 0$，有

$$\Delta^m f(0) = \sum_{j=0}^{m} (-1)^{m-j} \binom{m}{j} f(j).$$

若 f 是次数小于 m 的多项式，右侧为 0；这可以直接化简交错二项式和，也可用于差分表和 Newton 展开。组合数相关的交错幂和见 8.2.005 Stirling 数，含 $\lfloor n/i \rfloor$ 的和优先按 7.3.009 整除分块枚举 $O(\sqrt{n})$ 个商值块。

13.3.009 泛型复数与三次单位根代数

`Complex<T>` 表示 $a+bi$，其中 $i^2=-1$，共轭与范数分别为 $\overline{z}=a-bi$、$N(z)=z\overline{z}=a^2+b^2$；因此 $N(z)$ 可逆时有 $z^{-1}=\overline{z}/N(z)$。`Complex3<T>` 表示 $a+b\omega$，其中 $\omega^2+\omega+1=0$，把乘积中的 ω^2 替换为 $-1-\omega$，可得 $(a+b\omega)(c+d\omega)=(ac-bd)+(ad+bc-bd)\omega$；这里的共轭把 ω 换成 ω^2，故 $\overline{a+b\omega}=(a-b)-b\omega$、$N(a+b\omega)=a^2-ab+b^2$。两种类型都只要求底层 `T` 提供实际调用所需的环或域运算，适合复用模整数、有理数和浮点类型。

```
template <class T> struct Complex
{
    T a; // 实部。
    T b; // 虚部的系数。

    Complex(const T &a_ = T(), const T &b_ = T()) : a(a_), b(b_)
    {
        // a_ 是实部，b_ 是虚部的系数；构造 a_+b_*i。
    }

    Complex operator+(const Complex &rhs) const
    {
        // rhs 是加数；返回当前复数与 rhs 的和。
        return {a + rhs.a, b + rhs.b};
    }

    Complex operator-(const Complex &rhs) const
    {
        // rhs 是减数；返回当前复数减 rhs。
        return {a - rhs.a, b - rhs.b};
    }

    Complex operator*(const Complex &rhs) const
    {
        // rhs 是乘数；返回当前复数与 rhs 的积。
        return {a * rhs.a - b * rhs.b, a * rhs.b + b * rhs.a};
    }

    Complex operator/(const Complex &rhs) const
    {
        // rhs 是除数；返回当前复数除以 rhs，要求 rhs 的范数在 T 中可逆。
        return *this * rhs.inv();
    }

    Complex &operator+=(const Complex &rhs)
    {
        // rhs 是加数；把当前复数增加 rhs 并返回自身引用。
        return *this = *this + rhs;
    }

    Complex &operator-=(const Complex &rhs)
    {
        // rhs 是减数；把当前复数减去 rhs 并返回自身引用。
```

```
        return *this = *this - rhs;
    }

    Complex &operator*=(const Complex &rhs)
    {
        // rhs 是乘数；把当前复数乘以 rhs 并返回自身引用。
        return *this = *this * rhs;
    }

    Complex &operator/=(const Complex &rhs)
    {
        // rhs 是除数；把当前复数除以 rhs 并返回自身引用，要求 rhs 的范数在 T 中可逆。
        return *this = *this / rhs;
    }

    bool operator==(const Complex &rhs) const
    {
        // rhs 是待比较复数；返回两个系数是否分别相等。
        return a == rhs.a && b == rhs.b;
    }

    bool operator!=(const Complex &rhs) const
    {
        // rhs 是待比较复数；返回两个复数是否不等。
        return !(*this == rhs);
    }

    bool isZero() const
    {
        // 返回当前复数是否为加法零元。
        return a == T() && b == T();
    }

    Complex conj() const
    {
        // 返回当前复数的共轭。
        return {a, -b};
    }

    T norm() const
    {
        // 返回当前复数与其共轭之积 a^2+b^2。
        return a * a + b * b;
    }

    Complex inv() const
    {
        // 返回当前复数的乘法逆元，要求范数在 T 中非零且可逆。
        T n = norm();
        assert(n != T());
        return {a / n, -b / n};
    }
```

```cpp
};

template <class T> Complex<T> conj(const Complex<T> &z)
{
    // z 是待共轭复数；返回 z 的共轭。
    return z.conj();
}

template <class T> struct Complex3
{
    T a; // 常数项的系数。
    T b; // 三次单位根 omega 的系数。

    Complex3(const T &a_ = T(), const T &b_ = T()) : a(a_), b(b_)
    {
        // a_ 是常数项系数，b_ 是 omega 的系数；构造 a_+b_*omega。
    }

    Complex3 operator+(const Complex3 &rhs) const
    {
        // rhs 是加数；返回当前元素与 rhs 的和。
        return {a + rhs.a, b + rhs.b};
    }

    Complex3 operator-(const Complex3 &rhs) const
    {
        // rhs 是减数；返回当前元素减 rhs。
        return {a - rhs.a, b - rhs.b};
    }

    Complex3 operator*(const Complex3 &rhs) const
    {
        // rhs 是乘数；返回按 omega^2=-1-omega 化简后的积。
        T bd = b * rhs.b;
        return {a * rhs.a - bd, a * rhs.b + b * rhs.a - bd};
    }

    Complex3 operator/(const Complex3 &rhs) const
    {
        // rhs 是除数；返回当前元素除以 rhs，要求 rhs 的范数在 T 中可逆。
        return *this * rhs.inv();
    }

    Complex3 &operator+=(const Complex3 &rhs)
    {
        // rhs 是加数；把当前元素增加 rhs 并返回自身引用。
        return *this = *this + rhs;
    }

    Complex3 &operator-=(const Complex3 &rhs)
    {
        // rhs 是减数；把当前元素减去 rhs 并返回自身引用。
```

```
148         return *this = *this - rhs;
149     }
150
151     Complex3 &operator*=(const Complex3 &rhs)
152     {
153         // rhs 是乘数；把当前元素乘以 rhs 并返回自身引用。
154         return *this = *this * rhs;
155     }
156
157     Complex3 &operator/=(const Complex3 &rhs)
158     {
159         // rhs 是除数；把当前元素除以 rhs 并返回自身引用，要求 rhs 的范数在 T 中可逆。
160         return *this = *this / rhs;
161     }
162
163     bool operator==(const Complex3 &rhs) const
164     {
165         // rhs 是待比较元素；返回两个系数是否分别相等。
166         return a == rhs.a && b == rhs.b;
167     }
168
169     bool operator≠(const Complex3 &rhs) const
170     {
171         // rhs 是待比较元素；返回两个元素是否不等。
172         return !(*this == rhs);
173     }
174
175     bool isZero() const
176     {
177         // 返回当前元素是否为加法零元。
178         return a == T() && b == T();
179     }
180
181     Complex3 conj() const
182     {
183         // 返回把 omega 替换为 omega^2 后的共轭元素。
184         return {a - b, -b};
185     }
186
187     T norm() const
188     {
189         // 返回当前元素与其共轭之积 a^2-a*b+b^2。
190         return a * a - a * b + b * b;
191     }
192
193     Complex3 inv() const
194     {
195         // 返回当前元素的乘法逆元，要求范数在 T 中非零且可逆。
196         T n = norm();
197         assert(n ≠ T());
198         return {(a - b) / n, -b / n};
199     }
```

```
};

template <class T> Complex3<T> conj(const Complex3<T> &z)
{
    // z 是待共轭元素；返回 z 的共轭。
    return z.conj();
}
```

直接以两个系数构造对象；每次运算都是 $O(1)$ 时间和空间，数值范围由 `T` 负责。`inv()`、`operator/` 和 `operator/=` 不仅要求范数非零，还要求该范数在 `T` 中存在可用的乘法逆元；模整数应工作在相应可逆元上，有理数和浮点数可直接使用，而原生整数除法会截断，所以整数实例通常只用于加、减、乘、共轭与范数。复数没有与代数结构相容的自然全序，本实现不定义 `<` 或 `>`；若题意确实按模长排序，应显式比较 `norm()`，只需要普通浮点复数时也可直接使用标准库 `complex<T>`。

14. 其他实用模板

14.1 其他实用模板

14.1.001 高精度整数（BigInt）

带符号任意精度整数，内部按 10^9 一块、低位块在前存放绝对值，符号单独记在 `s` 里（取 $-1,0,1$）。加减法在同号时走 `addAbs`、异号时先比较绝对值再走 `subAbs`；乘法是 $O(nm)$ 的分块卷积，单项乘积加旧块与进位严格小于 $10^{18}+2\times10^9$，用 `unsigned long long` 即可。除法 `divmod` 从最高块到最低块逐位定商——每步把当前余数左移一块，再二分出最大的 $q_i\in[0,B)$ 使 `mulAbs(d,q_i)` 不超过余数，商向零取整、余数与被除数同号。每次运算后都要 `norm` 删掉最高端的零块并把空值的符号归零，这是比较与 `str` 输出正确的前提；忘记归一化会让 0 带上残留符号、进而破坏 `operator<`。

```
struct BigInt
{
    // B 是每个低位块使用的进制 10^9。
    static constexpr int B = 1000000000;
    // a 按低位在前保存绝对值的 10^9 进制块。
    vector<int> a;
    // s 保存符号：-1、0、1 分别表示负数、零、正数。
    int s = 0;

    BigInt(long long x = 0)
    {
        // x 是待转换的 64 位整数；构造同值高精度整数。
        *this = x;
    }
    BigInt(const string &x)
    {
        // x 是带可选单个正负号的十进制整数串；构造同值高精度整数。
        assign(x);
    }

    BigInt &operator=(long long x)
    {
        // x 是待转换的 64 位整数；返回赋值后的当前对象。
        a.clear();
        s = (x > 0) - (x < 0);
        unsigned long long y = x < 0 ? (unsigned long long)(-(x + 1)) + 1 : (unsigned long
        ↪  long)x;
        while (y)
        {
            a.push_back(y % B);
            y /= B;
        }
        return *this;
    }

    BigInt &assign(const string &x)
```

```
    {
        // x 是带可选单个正负号且至少含一位数字的十进制整数串；返回赋值后的当前对象。
        a.clear();
        int p = 0;
        s = 1;
        if (!x.empty() && (x[0] == '-' || x[0] == '+'))
        {
            s = x[0] == '-' ? -1 : 1;
            p = 1;
        }
        assert(p < (int)x.size()); // 调试检查，可删。
        for (int i = p; i < (int)x.size(); i++)
        {
            assert('0' <= x[i] && x[i] <= '9'); // 调试检查，可删。
        }
        for (int r = (int)x.size(); r > p; r -= 9)
        {
            int l = max(p, r - 9);
            int v = 0;
            for (int i = l; i < r; i++)
            {
                v = v * 10 + x[i] - '0';
            }
            a.push_back(v);
        }
        norm();
        return *this;
    }

    string str() const
    {
        // 无参数；返回当前高精度整数的十进制字符串。
        if (!s)
        {
            return "0";
        }
        string x = s < 0 ? "-" : "";
        x += to_string(a.back());
        for (int i = (int)a.size() - 2; i >= 0; i--)
        {
            string t = to_string(a[i]);
            x += string(9 - t.size(), '0') + t;
        }
        return x;
    }

    void norm()
    {
        // 删除最高端零块并同步零值符号，无返回值。
        while (!a.empty() && !a.back())
        {
            a.pop_back();
```

```
        }
        if (a.empty())
        {
            s = 0;
        }
    }

    static int cmpAbs(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；按 |x| 与 |y| 的大小返回 -1、0 或 1。
        if (x.a.size() != y.a.size())
        {
            return x.a.size() < y.a.size() ? -1 : 1;
        }
        for (int i = (int)(x.a.size()) - 1; i >= 0; i--)
        {
            if (x.a[i] != y.a[i])
            {
                return x.a[i] < y.a[i] ? -1 : 1;
            }
        }
        return 0;
    }

    friend bool operator==(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回二者是否相等。
        return x.s == y.s && x.a == y.a;
    }
    friend bool operator<(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否小于 y。
        if (x.s != y.s)
        {
            return x.s < y.s;
        }
        if (!x.s)
        {
            return false;
        }
        int c = cmpAbs(x, y);
        return x.s > 0 ? c < 0 : c > 0;
    }
    friend bool operator!=(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回二者是否不相等。
        return !(x == y);
    }
    friend bool operator>(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否大于 y。
        return y < x;
```

```
    }
    friend bool operator<=(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否不大于 y。
        return !(y < x);
    }
    friend bool operator>=(const BigInt &x, const BigInt &y)
    {
        // x、y 是待比较整数；返回 x 是否不小于 y。
        return !(x < y);
    }

    BigInt operator-() const
    {
        // 无参数；返回当前整数的相反数。
        BigInt x = *this;
        x.s = -x.s;
        return x;
    }

    static BigInt addAbs(const BigInt &x, const BigInt &y)
    {
        // x、y 是两个整数；返回 |x|+|y|。
        BigInt z;
        z.s = 1;
        int n = max(x.a.size(), y.a.size());
        z.a.resize(n);
        long long c = 0;
        for (int i = 0; i < n; i++)
        {
            c += (i < (int)(x.a.size()) ? x.a[i] : 0);
            c += (i < (int)(y.a.size()) ? y.a[i] : 0);
            z.a[i] = c % B;
            c /= B;
        }
        if (c)
        {
            z.a.push_back(c);
        }
        z.norm();
        return z;
    }

    // 要求 |x| >= |y|。
    static BigInt subAbs(const BigInt &x, const BigInt &y)
    {
        // x、y 满足 |x⊳⊲y|；返回 |x|-|y|。
        BigInt z;
        z.s = 1;
        z.a.resize(x.a.size());
        long long c = 0;
        for (int i = 0; i < (int)(x.a.size()); i++)
```

```
        {
            c = (long long)(x.a[i]) - c - (i < (int)(y.a.size()) ? y.a[i] : 0);
            if (c < 0)
            {
                c += B;
                z.a[i] = c;
                c = 1;
            }
            else
            {
                z.a[i] = c;
                c = 0;
            }
        }
        z.norm();
        return z;
    }

    BigInt &operator+=(const BigInt &y)
    {
        // y 是加数；把 y 加到当前整数并返回当前对象。
        if (!y.s)
        {
            return *this;
        }
        if (!s)
        {
            return *this = y;
        }
        if (s == y.s)
        {
            int t = s;
            *this = addAbs(*this, y);
            s = t;
        }
        else if (cmpAbs(*this, y) >= 0)
        {
            int t = s;
            *this = subAbs(*this, y);
            if (a.size())
            {
                s = t;
            }
        }
        else
        {
            int t = y.s;
            *this = subAbs(y, *this);
            if (a.size())
            {
                s = t;
            }
```

```
        }
        return *this;
    }
    BigInt &operator-=(const BigInt &y)
    {
        // y 是减数；从当前整数减去 y 并返回当前对象。
        return *this += -y;
    }

    BigInt &operator*=(const BigInt &y)
    {
        // y 是乘数；把当前整数乘以 y 并返回当前对象。
        if (!s || !y.s)
        {
            return *this = 0;
        }
        vector<int> c(a.size() + y.a.size());
        for (int i = 0; i < (int)a.size(); i++)
        {
            unsigned long long v = 0;
            for (int j = 0; j < (int)(y.a.size()) || v; j++)
            {
                unsigned long long w = c[i + j] + v;
                if (j < (int)(y.a.size()))
                {
                    w += (unsigned long long)a[i] * y.a[j];
                }
                c[i + j] = w % B;
                v = w / B;
            }
        }
        a = move(c);
        s *= y.s;
        norm();
        return *this;
    }

    static BigInt mulAbs(const BigInt &x, int y)
    {
        // x 是整数，y 满足 0<=y<B；返回 |x|*y。
        BigInt z;
        if (!y || !x.s)
        {
            return z;
        }
        z.s = 1;
        z.a.resize(x.a.size());
        unsigned long long c = 0;
        for (int i = 0; i < (int)x.a.size(); i++)
        {
            unsigned long long w = c + (unsigned long long)x.a[i] * y;
            z.a[i] = w % B;
```

```
            c = w / B;
        }
        if (c)
        {
            z.a.push_back(c);
        }
        return z;
    }

    // 商向 0 取整，余数与被除数同号；除数不能为 0。
    static pair<BigInt, BigInt> divmod(const BigInt &x, const BigInt &y)
    {
        // x 是被除数，y 是非零除数；返回向零取整的商和与 x 同号的余数。
        assert(y.s); // 调试检查，可删。
        BigInt q, r, d = y;
        d.s = 1;
        q.a.resize(x.a.size());
        q.s = x.s * y.s;
        for (int i = (int)(x.a.size()) - 1; i >= 0; i--)
        {
            r.a.insert(r.a.begin(), x.a[i]);
            r.s = 1;
            r.norm();
            int l = 0, h = B - 1;
            while (l < h)
            {
                int m = l + (h - l + 1) / 2;
                if (cmpAbs(mulAbs(d, m), r) <= 0)
                {
                    l = m;
                }
                else
                {
                    h = m - 1;
                }
            }
            q.a[i] = l;
            r = subAbs(r, mulAbs(d, l));
        }
        q.norm();
        if (r.a.size())
        {
            r.s = x.s;
        }
        return {q, r};
    }

    BigInt &operator/=(const BigInt &y)
    {
        // y 是非零除数；把当前整数改成向零取整的商并返回当前对象。
        return *this = divmod(*this, y).first;
    }
```

```
    BigInt &operator%=(const BigInt &y)
    {
        // y 是非零除数；把当前整数改成与原值同号的余数并返回当前对象。
        return *this = divmod(*this, y).second;
    }

    friend BigInt operator+(BigInt x, const BigInt &y)
    {
        // x、y 是加数；返回 x+y。
        return x += y;
    }
    friend BigInt operator-(BigInt x, const BigInt &y)
    {
        // x 是被减数，y 是减数；返回 x-y。
        return x -= y;
    }
    friend BigInt operator*(BigInt x, const BigInt &y)
    {
        // x、y 是乘数；返回 x*y。
        return x *= y;
    }
    friend BigInt operator/(BigInt x, const BigInt &y)
    {
        // x 是被除数，y 是非零除数；返回向零取整的商。
        return x /= y;
    }
    friend BigInt operator%(BigInt x, const BigInt &y)
    {
        // x 是被除数，y 是非零除数；返回与 x 同号的余数。
        return x %= y;
    }
};
```

用 `BigInt x(十进制字符串)` 或 `BigInt(long long)` 构造；字符串必须是可选单个正负号后接至少一位十进制数字，否则抛出 `invalid_argument`。四则与取模直接用重载的 `+ - * / %`，除数必须非零；输出调用 `str()`，本类不重载流运算符。该类适合常规十进制中等位数；百万位十进制题使用下方 `HexBigInt::fromDecimal` / `toDecimal`，把十进制分组转换到 Karatsuba 与 Burnikel–Ziegler 路径。设两数块数为 n, m：比较与加减为 $O(\max(n,m))$，乘法为 $O(nm)$，当前逐块二分除法为 $O(n(n+m)\log B)$，额外空间为 $O(n+m)$。需要幂或进制转换见 14.1.006 高精度进制转换与快速幂，只求向下取整平方根见 14.1.016 高精度整数平方根。

除法逐位定商和跨块进位最易写错。下面检查 10^9 分块边界和一组固定四则答案；长位数压力测试放在本地测试文件：

```
signed main()
{
    assert(BigInt::addAbs(0, 0).str() == "0");
    assert((BigInt("999999999") + 1).str() == "1000000000");
    assert((BigInt("1000000000") - 1).str() == "999999999");

    BigInt x("1234567890123"), y("987654321");
```

```
    assert((x + y).str() == "12355555444444");
    assert((x - y).str() == "1233580235802");
    assert((x * y).str() == "12193263112478341714 83");
    assert((x / y).str() == "1249");
    assert((x % y).str() == "987643194");
    assert((-x / y).str() == "-1249");
}
```

14.1.002 分数类与有理数还原

有理数类用 64 位分子和正分母保存唯一既约表示。构造时统一分母符号并约分；乘除先交叉约分，既能减小中间量，也省去额外的大整数包装。比较时的交叉乘法有较高溢出风险，只在这一处局部使用 `__int128`。标题所说的“有理数还原”若指由近似值反推最简分数，还需另配 Stern–Brocot 树或连分数逼近，本条只负责精确运算。

```
struct Frac
{
    long long x, y; // x、y 是既约分子和正分母。

    Frac(long long x = 0, long long y = 1) : x(x), y(y)
    {
        // x、y 是分子和非零分母；构造既约分数 x/y。
        assert(y); // 调试检查，可删。
        if (y < 0)
        {
            x = -x;
            y = -y;
        }
        long long g = gcd(llabs(x), y);
        this->x = x / g;
        this->y = y / g;
    }

    friend Frac operator+(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回 a+b。
        long long g = gcd(a.y, b.y);
        return {a.x * (b.y / g) + b.x * (a.y / g), a.y / g * b.y};
    }
    friend Frac operator-(const Frac &a, const Frac &b)
    {
        // a 是被减数，b 是减数；返回 a-b。
        long long g = gcd(a.y, b.y);
        return {a.x * (b.y / g) - b.x * (a.y / g), a.y / g * b.y};
    }
    friend Frac operator*(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回 a*b。
        long long g = gcd(llabs(a.x), b.y);
        long long h = gcd(llabs(b.x), a.y);
```

```
        return {a.x / g * (b.x / h), a.y / h * (b.y / g)};
    }
    friend Frac operator/(const Frac &a, const Frac &b)
    {
        // a 是被除数，b 是非零除数；返回 a/b。
        assert(b.x); // 调试检查，可删。
        long long g = gcd(llabs(a.x), llabs(b.x));
        long long h = gcd(a.y, b.y);
        return {a.x / g * (b.y / h), a.y / h * (b.x / g)};
    }
    friend bool operator==(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回二者是否相等。
        return a.x == b.x && a.y == b.y;
    }
    friend bool operator<(const Frac &a, const Frac &b)
    {
        // a、b 是两个分数；返回 a 是否小于 b。
        return (__int128)a.x * b.y < (__int128)b.x * a.y;
    }
};
```

用 `Frac(x,y)` 构造，其中 $y \neq 0$；随后直接使用 `+ - * / < ==`。调用者须保证取绝对值、分母变号和四则中间量能由 `long long` 表示；接近完整 64 位边界时改用 14.1.001 高精度整数（BigInt）。构造与四则各做常数次欧几里得算法，最坏 $O(\log M)$ 时间、$O(1)$ 空间，M 是操作数绝对值的最大值；比较为 $O(1)$。Python 等价物见 14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO。

下面用可手算的固定值覆盖负分母、四则和比较：

```
signed main()
{
    assert(Frac(2, -4) == Frac(-1, 2));
    assert(Frac(1, 6) + Frac(1, 3) == Frac(1, 2));
    assert(Frac(5, 6) - Frac(1, 4) == Frac(7, 12));
    assert(Frac(-7, 3) < Frac(-2, 1));
    assert(Frac(2, 3) * Frac(9, 4) == Frac(3, 2));
    assert(Frac(2, 3) / Frac(4, 9) == Frac(3, 2));
    cout << "OK\n";
    return 0;
}
```

14.1.003 表达式解析：中缀转后缀、表达式树

递归下降求值：`expr` 处理最低优先级的 $+$, $-$，`term` 处理 $*$, $/$，`atom` 迭代读完连续一元正负号，再处理括号或整数。三层调用自然实现优先级，二元运算的循环实现左结合。各层用 `bool` 区分成功与失败，`eval` 还要求完整消费输入，因此空串、残缺表达式、多余字符、除零和 64 位中间结果溢出都返回 `nullopt`，不会越界或产生未定义行为。

```
struct Expr
{
    // s 保存待求值表达式。
    string s;
    // p 保存下一个待读字符的位置。
    int p = 0;

    void skip()
    {
        // 无参数；把读位置 p 向后移过 s 中连续的空白字符。
        while (p < (int)s.size() && isspace((unsigned char)s[p]))
        {
            p++;
        }
    }

    static bool take(__int128 v, long long &x)
    {
        // v 是中间结果，x 接收 64 位值；可表示时写入 x 并返回 true。
        if (v < LLONG_MIN || v > LLONG_MAX)
        {
            return false;
        }
        x = (long long)v;
        return true;
    }

    bool expr(long long &x)
    {
        // x 接收当前位置起的加减表达式值；语法或数值失败时返回 false。
        if (!term(x))
        {
            return false;
        }
        skip();
        while (p < (int)s.size() && (s[p] == '+' || s[p] == '-'))
        {
            char op = s[p++];
            long long y;
            if (!term(y))
            {
                return false;
            }
            __int128 v = op == '+' ? (__int128)x + y : (__int128)x - y;
            if (!take(v, x))
            {
                return false;
            }
            skip();
        }
        return true;
    }
```

```
53
54      bool term(long long &x)
55      {
56          // x 接收当前位置起的乘除项值；语法或数值失败时返回 false。
57          if (!atom(x))
58          {
59              return false;
60          }
61          skip();
62          while (p < (int)s.size() && (s[p] == '*' || s[p] == '/'))
63          {
64              char op = s[p++];
65              long long y;
66              if (!atom(y) || (op == '/' && !y))
67              {
68                  return false;
69              }
70              skip();
71              __int128 v = op == '*' ? (__int128)x * y : (__int128)x / y;
72              if (!take(v, x))
73              {
74                  return false;
75              }
76          }
77          return true;
78      }
79
80      bool atom(long long &x)
81      {
82          // x 接收当前位置起的一元表达式值；语法或数值失败时返回 false。
83          skip();
84          bool neg = false;
85          while (p < (int)s.size() && (s[p] == '+' || s[p] == '-'))
86          {
87              neg ^= s[p] == '-';
88              p++;
89              skip();
90          }
91          if (p >= (int)s.size())
92          {
93              return false;
94          }
95          if (s[p] == '(')
96          {
97              p++;
98              if (!expr(x) || p >= (int)s.size() || s[p] != ')')
99              {
100                 return false;
101             }
102             p++;
103             return !neg || take(-(__int128)x, x);
104         }
```

```
        if (s[p] < '0' || s[p] > '9')
        {
            return false;
        }
        __int128 v = 0;
        while (p < (int)s.size() && s[p] >= '0' && s[p] <= '9')
        {
            v = v * 10 + s[p++] - '0';
            if (v > (__int128)LLONG_MAX + 1)
            {
                return false;
            }
        }
        if (neg)
        {
            v = -v;
        }
        return take(v, x);
    }

    optional<long long> eval(const string &t)
    {
        // t 是待求值表达式；成功时返回整数值，语法或数值失败时返回空。
        s = t;
        p = 0;
        long long x;
        if (!expr(x))
        {
            return nullopt;
        }
        skip();
        if (p != (int)s.size())
        {
            return nullopt;
        }
        return x;
    }
};
```

`Expr::eval(t)` 支持 `+ - * /`、括号、连续一元正负号和任意空白，成功时返回整数值，失败时返回空。除法向零截断。长度为 n 的表达式用 $O(n)$ 时间，递归栈为 $O(d)$，其中 d 是括号嵌套深度；一元符号已迭代处理，不增加递归层数。精确有理数中间值可改用 14.1.002 分数类与有理数还原，任意精度整数可改用 14.1.001 高精度整数（BigInt）。

下面用固定表达式分别覆盖优先级、左结合除法、运算符后一元负号、连续一元符号、嵌套括号和失败分支：

```
signed main()
{
    Expr e;
    assert(e.eval("2+3*4") == 14);
```

```
    assert(e.eval("8/3*3") == 6);
    assert(e.eval("2*-3") == -6);
    assert(e.eval("--5") == 5);
    assert(e.eval("-(3+4)*2") == -14);
    assert(e.eval("((2+3)*(4-1))") == 15);
    assert(e.eval("-9223372036854775808") == LLONG_MIN);
    assert(!e.eval(""));
    assert(!e.eval("1+"));
    assert(!e.eval("(1+2"));
    assert(!e.eval("1/0"));
    assert(!e.eval("1a"));
    cout << "OK\n";
    return 0;
}
```

14.1.004 德州扑克牌型判断（五张牌与多手牌最佳五张）

`pokerScore` 把五张牌压成一个可字典序比较的评分数组 `z`：`z[0]` 是牌型等级（8 同花顺、7 四条、6 葫芦、5 同花、4 顺子、3 三条、2 两对、1 一对、0 高牌），其后各维依次填决胜的关键点数，于是两手牌直接按 `array` 字典序比大小即可。点数用 $2-14$（A 记 14），先降序排序，并特判把 A-5-4-3-2 的“轮子顺”改写成 $5,4,3,2,1$ 使最小顺子也能被检出；`flush` 看五张花色是否全同，`straight` 看相邻差是否恒为 1，再用按出现次数降序的 `(cnt,rank)` 对区分四条 / 葫芦 / 三条 / 两对 / 一对。判定顺序必须“同花顺 → 四条 → 葫芦 →……”从强到弱，否则同花顺会被同花或顺子截胡。七张或更多牌取最佳五张时，在外层枚举五张子集分别调用本函数取字典序最大评分即可，本条只实现五张的核心比较。

```
array<int, 6> pokerScore(vector<int> r, vector<int> suit)
{
    // r、suit 依次给出五张牌的点数与花色；返回可直接字典序比较的六元评分。
    assert(r.size() == 5 && suit.size() == 5); // 调试检查，可删。
    set<pair<int, int>> card;
    for (int i = 0; i < 5; i++)
    {
        assert(2 <= r[i] && r[i] <= 14); // 调试检查，可删。
        assert(0 <= suit[i] && suit[i] < 4); // 调试检查，可删。
        card.insert({r[i], suit[i]});
    }
    assert(card.size() == 5); // 调试检查，可删。
    sort(r.begin(), r.end(), greater<>());
    if (r == vector<int>{14, 5, 4, 3, 2})
    {
        r = {5, 4, 3, 2, 1};
    }
    bool flush = count(suit.begin(), suit.end(), suit[0]) == 5;
    bool straight = true;
    for (int i = 1; i < 5; i++)
    {
        straight &= r[i - 1] == r[i] + 1;
    }
    map<int, int, greater<>> f;
```

```
    for (int x : r)
    {
        f[x]++;
    }
    vector<pair<int, int>> a;
    for (auto [x, c] : f)
    {
        a.push_back({c, x});
    }
    sort(a.begin(), a.end(), greater<>());
    array<int, 6> z{};
    if (straight && flush)
    {
        z[0] = 8;
        z[1] = r[0];
    }
    else if (a[0].first == 4)
    {
        z[0] = 7;
        z[1] = a[0].second;
        z[2] = a[1].second;
    }
    else if (a[0].first == 3 && a[1].first == 2)
    {
        z[0] = 6;
        z[1] = a[0].second;
        z[2] = a[1].second;
    }
    else if (flush)
    {
        z[0] = 5;
        copy(r.begin(), r.end(), z.begin() + 1);
    }
    else if (straight)
    {
        z[0] = 4;
        z[1] = r[0];
    }
    else if (a[0].first == 3)
    {
        z[0] = 3;
        z[1] = a[0].second;
        vector<int> q;
        for (auto [c, x] : a)
        {
            if (c == 1)
            {
                q.push_back(x);
            }
        }
        sort(q.begin(), q.end(), greater<>());
        copy(q.begin(), q.end(), z.begin() + 2);
```

```
    }
    else if (a[0].first == 2 && a[1].first == 2)
    {
        z[0] = 2;
        z[1] = max(a[0].second, a[1].second);
        z[2] = min(a[0].second, a[1].second);
        z[3] = a[2].second;
    }
    else if (a[0].first == 2)
    {
        z[0] = 1;
        z[1] = a[0].second;
        vector<int> q;
        for (auto [c, x] : a)
        {
            if (c == 1)
            {
                q.push_back(x);
            }
        }
        sort(q.begin(), q.end(), greater<>());
        copy(q.begin(), q.end(), z.begin() + 2);
    }
    else
    {
        copy(r.begin(), r.end(), z.begin() + 1);
    }
    return z;
}
```

调用 `pokerScore(r,suit)`，两向量必须按相同次序描述五张互不重复的实体牌，点数在 $[2,14]$，花色在 $[0,4)$；返回数组可直接用 `<`、`>` 比较。单手判定为 $O(1)$ 时间和空间；七张牌枚举 $\binom{7}{5}=21$ 个子集仍为 $O(1)$。胜率估计需在外层采样，不属于牌型判定。

下面固定检查顺子同花、A-5 轮子顺、葫芦、两对踢脚和普通高牌；完整牌型覆盖放在本地测试文件：

```
signed main()
{
    assert((pokerScore({14, 13, 12, 11, 10}, {0, 0, 0, 0, 0})
            == array<int, 6>{8, 14, 0, 0, 0, 0}));
    assert((pokerScore({4, 4, 4, 2, 2}, {0, 1, 2, 0, 1})
            == array<int, 6>{6, 4, 2, 0, 0, 0}));
    assert((pokerScore({14, 5, 4, 3, 2}, {0, 1, 2, 3, 0})
            == array<int, 6>{4, 5, 0, 0, 0, 0}));
    assert((pokerScore({14, 14, 2, 2, 9}, {0, 1, 0, 1, 2})
            == array<int, 6>{2, 14, 2, 9, 0, 0}));
    assert((pokerScore({14, 11, 9, 5, 2}, {0, 1, 2, 3, 0})
            == array<int, 6>{0, 14, 11, 9, 5, 2}));
    cout << "OK\n";
    return 0;
}
```

14.1.005 日期与时间换算

公历日期与“距 1970-01-01 的天数”互转，核心是 Howard Hinnant 的纪元–年–日算法：`days()` 先把 ≤ 2 月的月份并入上一年（`yy=y-(m<=2)`），以 400 年为一个纪元（恰 146097 天）算出纪元编号 `era`、纪元内年 `yo` 与内日 `dy`，合成后减 719468 把原点对齐到 1970-01-01；`fromDays` 是它逐步的逆变换。闰年按公历“四年一闰、百年不闰、四百年再闰”判定，月天数查表并对二月补闰日——所以任何日期加减都必须先转成连续日序，绝不能按固定 365 天硬算。星期由 `(days()%7+11)%7` 给出（0 为周日），$+11$ 是把可能为负的取模结果掰回 $[0,7)$ 的偏移。

```
struct Date
{
    // y 保存公历年份。
    int y;
    // m 保存公历月份。
    int m;
    // d 保存公历月内日期。
    int d;

    static bool leap(int y)
    {
        // y 是公历年份；返回该年是否为闰年。
        return y % 4 == 0 && (y % 100 != 0 || y % 400 == 0);
    }

    static int monthDays(int y, int m)
    {
        // y、m 是公历年份和月份；返回该月的天数。
        static constexpr int a[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
        assert(1 <= m && m <= 12); // 调试检查，可删。
        return a[m] + (m == 2 && leap(y));
    }

    bool valid() const
    {
        // 无参数；返回当前年月日是否为合法公历日期。
        return 1 <= m && m <= 12 && 1 <= d && d <= monthDays(y, m);
    }

    // 与 1970-01-01 相差的天数，可为负。
    long long days() const
    {
        // 无参数；返回当前日期与 1970-01-01 相差的天数。
        assert(valid()); // 调试检查，可删。
        __int128 yy = (__int128)y - (m <= 2);
        __int128 era = (yy >= 0 ? yy : yy - 399) / 400;
        __int128 yo = yy - era * 400;
        __int128 mp = m + (m > 2 ? -3 : 9);
        __int128 dy = (153 * mp + 2) / 5 + d - 1;
        __int128 de = yo * 365 + yo / 4 - yo / 100 + dy;
        __int128 z = era * 146097 + de - 719468;
        assert(LLONG_MIN <= z && z <= LLONG_MAX); // 调试检查，可删。
```

```
        return (long long)z;
    }

    // days() 的逆变换。
    static Date fromDays(long long z)
    {
        // z 是相对 1970-01-01 的天数；返回对应公历日期。
        __int128 zz = (__int128)z + 719468;
        __int128 era = (zz >= 0 ? zz : zz - 146096) / 146097;
        __int128 de = zz - era * 146097;
        __int128 yo = (de - de / 1460 + de / 36524 - de / 146096) / 365;
        __int128 yy = yo + era * 400;
        __int128 dy = de - (365 * yo + yo / 4 - yo / 100);
        __int128 mp = (5 * dy + 2) / 153;
        __int128 dd = dy - (153 * mp + 2) / 5 + 1;
        __int128 mm = mp + (mp < 10 ? 3 : -9);
        yy += mm <= 2;
        return {(long long)yy, (long long)mm, (long long)dd};
    }

    // 周日为 0，周一为 1，…，周六为 6。
    int weekday() const
    {
        // 无参数；返回当前日期的星期编号，周日为 0。
        return (int)((days() % 7 + 11) % 7);
    }
};
```

构造 `Date{y,m,d}` 后可用 `valid()` 判断日期是否合法；`days()` 要求日期合法，并要求相对 1970-01-01 的日序可由 `long long` 表示。`Date::fromDays(z)` 接受任意 64 位日序，`weekday()` 返回 $[0,7)$，周日为 0。所有接口均为 $O(1)$ 时间和空间。Python 的 `date`、`timedelta` 见 14.1.014 Python：datetime、Fraction、递归深度与文件 IO。

下面固定检查纪元、世纪闰年和日期往返：

```
signed main()
{
    assert((Date{1970, 1, 1}.days() == 0));
    assert((Date{1970, 1, 1}.weekday() == 4));
    assert(!Date::leap(1900));
    assert(Date::leap(2000));
    long long z = Date{2000, 2, 29}.days();
    Date d = Date::fromDays(z);
    assert(d.y == 2000 && d.m == 2 && d.d == 29);
    assert((Date{2024, 2, 29}.valid() && !Date{2023, 2, 29}.valid()));
}
```

14.1.006 高精度进制转换与快速幂

两个只依赖字符串或大整数的小工具。`convertBase` 在 $2-62$ 进制间转换任意长带符号整数，数码严格按 `0-9A-Za-z` 映射到 $0-61$，因此大小写在 37 进制以上含义不同；它反复对目标进制做整型短除，每次余数逆序拼成答案，零统一输出为 `0`。`bigPow` 是 `BigInt` 上的二进制快速幂——按指数 `e` 的二进制位决定是否把当前底数乘入答案、同时底数自乘，乘法次数从 e 降到 $O(\log e)$，底层乘法复用 14.1.001 高精度整数（BigInt）；`convertBase` 本身不用 `BigInt`。

```
// 依赖 14.1.001 高精度整数（BigInt）
string convertBase(string s, int from, int to)
{
    // s 是字符序 0-9A-Za-z 下带可选正负号的 from 进制整数；返回同字符序、无前导零的 to 进制字符串。
    assert(!s.empty()); // 调试检查，可删。
    assert(2 <= from && from <= 62 && 2 <= to && to <= 62); // 调试检查，可删。
    const string digit = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
    bool neg = s[0] == '-';
    int p = neg || s[0] == '+';
    assert(p < (int)s.size()); // 调试检查，可删。
    vector<int> a;
    for (; p < (int)s.size(); p++)
    {
        size_t pos = digit.find(s[p]);
        assert(pos != string::npos && pos < (size_t)from); // 调试检查，可删。
        int x = pos;
        a.push_back(x);
    }
    while (a.size() > 1 && a[0] == 0)
    {
        a.erase(a.begin());
    }
    string ans;
    while (!(a.size() == 1 && a[0] == 0))
    {
        vector<int> b;
        int rem = 0;
        for (int x : a)
        {
            int cur = rem * from + x;
            if (!b.empty() || cur / to)
            {
                b.push_back(cur / to);
            }
            rem = cur % to;
        }
        ans.push_back(digit[rem]);
        a = b.empty() ? vector<int>{0} : move(b);
    }
    if (ans.empty())
    {
        return "0";
    }
```

```
    reverse(ans.begin(), ans.end());
    return neg ? "-" + ans : ans;
}

BigInt bigPow(BigInt a, int e)
{
    // a 是任意精度整数，e 是非负整数；返回 a^e，依赖 14.1.001 的 BigInt 乘法。
    assert(e >= 0); // 调试检查，可删。
    BigInt ans = 1;
    while (e)
    {
        if (e & 1)
        {
            ans *= a;
        }
        e >>= 1;
        if (e)
        {
            a *= a;
        }
    }
    return ans;
}
```

`convertBase(s,from,to)` 要求 `s` 除可选首字符 `+`、`-` 外至少有一位合法数码；`from`、`to` 均在 $[2,62]$，输出沿用同一字符序并去掉前导零。先引入 14.1.001 高精度整数（BigInt） 后可调用 `bigPow(a,e)`，其中 $e \geq 0$。设转换途中数值最多 D 位，反复短除用 $O(D^2)$ 时间、$O(D)$ 空间；快速幂做 $O(\log e)$ 次高精度乘法。`convertBase("-z",62,10)` 返回 `-61`，`convertBase("61",10,62)` 返回 `z`，`bigPow(BigInt("12"),10).str()` 返回 `61917364224`。

14.1.007 9×9 数独字典序最小解

标准 9×9 数独求解，用三个位掩码 `row`/`col`/`box` 各记一行、一列、一宫已用的数字（第 v 位表示数字 v，常量 `1022` 即二进制 `1111111110`、圈定候选 $1\text{—}9$）。要得到标题承诺的字典序最小解，`dfs` 必须严格按行优先顺序逐格填、每格从最小候选试起、找到首解即停：先固定的靠前格因此拿到尽可能小的值，整盘按行读出才最小。原实现此处改用了最少候选数（MRV）启发式挑格，能快速求出某个合法解却不保证字典序最小——空盘下它给出的第 4 行是 `231674895`，而真正字典序最小解的第 4 行为 `214365897`；下面已改回行优先扫描，参数 `z` 也回归 “下一个待检查格编号” 的本义。`solve` 先把初盘已填数字灌入三个掩码并顺带检测冲突，再调 `dfs`。

```
struct Sudoku
{
    // a 保存 9x9 棋盘，0 表示空格，1..9 表示已填数字。
    array<array<int, 9>, 9> a{};
    // row[i] 的第 v 位表示第 i 行是否已使用数字 v。
    array<int, 9> row{};
    // col[j] 的第 v 位表示第 j 列是否已使用数字 v。
    array<int, 9> col{};
```

```
    // box[b] 的第 v 位表示第 b 个 3x3 宫是否已使用数字 v。
    array<int, 9> box{};

    bool dfs(int z = 0)
    {
        // 参数 z 表示 按行优先扫描时下一个待检查的格编号；从该格起继续深度优先搜索，成功返回 true。
        while (z < 81 && a[z / 9][z % 9])
        {
            z++;
        }
        if (z == 81)
        {
            return true;
        }
        int x = z / 9;
        int y = z % 9;
        int b = x / 3 * 3 + y / 3;
        int m = ~(row[x] | col[y] | box[b]) & 1022;
        while (m)
        {
            int v = countr_zero((unsigned)m);
            m &= m - 1;
            a[x][y] = v;
            row[x] |= 1 << v;
            col[y] |= 1 << v;
            box[b] |= 1 << v;
            if (dfs(z + 1))
            {
                return true;
            }
            row[x] ^= 1 << v;
            col[y] ^= 1 << v;
            box[b] ^= 1 << v;
            a[x][y] = 0;
        }
        return false;
    }

    bool solve()
    {
        // 无参数；验证初盘后求字典序最小完整数独，成功返回 true，矛盾时返回 false。
        row.fill(0);
        col.fill(0);
        box.fill(0);
        for (int i = 0; i < 9; i++)
        {
            for (int j = 0; j < 9; j++)
            {
                assert(0 <= a[i][j] && a[i][j] <= 9); // 调试检查，可删。
                if (a[i][j])
                {
                    int b = i / 3 * 3 + j / 3;
```

```
                    int q = 1 << a[i][j];
                    if ((row[i] | col[j] | box[b]) & q)
                    {
                        return false;
                    }
                    row[i] |= q;
                    col[j] |= q;
                    box[b] |= q;
                }
            }
        }
        return dfs();
    }
};
```

填好标准 9×9 初盘后调用 `solve()`，其中 0 表示空格，非零格必须在 $[1,9]$。成功时 `a` 就地改成按行展开后字典序最小的完整解；取值合法但已有行、列或宫冲突时返回 `false`。设空格数为 k，最坏 $O(9^k)$ 时间、$O(k)$ 递归栈。需要更快地求任意一组解时可改用 Dancing Links；同类构造见 14.1.018 N 皇后构造解。

14.1.008 装箱问题

装箱是 NP-hard，这里给三档实现。`firstFitDec` 是首次适应下降近似：物品按体积降序，逐个放进第一个装得下的箱（`rem` 按开箱顺序存各箱剩余容量），放不下就开新箱，简单但当前线性找箱为 $O(n^2)$。`firstFit` 保留原输入顺序，复用 3.3.001 单点修改线段树与摘要二分的 `Seg<MaxInfo>`：每个叶子是一个箱子的剩余容量，区间摘要取最大值，`findFirst` 找最早还能容纳当前物品的箱子；尚未使用的叶子初值也是箱容量，所以第一次走到它就等价于开新箱。`exactBins` 是 $O(n2^n)$ 的子集 DP 求精确最少箱数：`dp[s]` 记“已装物品集合为 s”时按字典序最优的二元组 $(\text{已用箱数}, \text{末箱已用容量})$，转移把某个未装物品 i 塞进当前末箱（装得下）或新开一箱、取字典序最小。以 $(\text{箱数}, \text{末箱负载})$ 为最小化目标是这套 DP 正确的关键：箱数相同再压低末箱负载，才能给后续物品留最大空隙。

```
// 依赖 3.3.001 单点修改线段树与摘要二分
// 知识点 14.1.008：装箱问题的近似与小规模精确解。
namespace bin_packing
{
int firstFitDec(vector<int> a, int cap)
{
    // a 是物品体积，cap 是箱容量；返回降序首次适应使用的箱数。
    assert(cap >= 0); // 调试检查，可删。
    for (int x : a)
    {
        assert(0 <= x && x <= cap); // 调试检查，可删。
    }
    sort(a.begin(), a.end(), greater<>());
    vector<int> rem;
    for (int x : a)
    {
        int p = 0;
        while (p < (int)rem.size() && rem[p] < x)
        {
```

```
            p++;
        }
        if (p == (int)rem.size())
        {
            rem.push_back(cap);
        }
        rem[p] -= x;
    }
    return rem.size();
}

pair<int, vector<int>> firstFit(const vector<int> &a, int cap)
{
    // 按输入顺序放置，返回使用箱数与每件物品所在的 0 下标箱号。
    assert(cap >= 0); // 调试检查，可删
    int n = (int)a.size();
    vector<int> rem(n, cap), box;
    vector<MaxInfo> init(n, MaxInfo(cap));
    Seg<MaxInfo> st(init);
    box.reserve(n);
    int used = 0;
    for (int x : a)
    {
        assert(0 <= x && x <= cap); // 调试检查，可删
        int p = st.findFirst(0, n, [&](MaxInfo z) { return z.val >= x; });
        assert(p != -1); // 前 i 件物品至多占用 i 个箱子
        rem[p] -= x;
        st.set(p, MaxInfo(rem[p]));
        used = max(used, p + 1);
        box.push_back(p);
    }
    return {used, box};
}

int exactBins(const vector<int> &a, int cap)
{
    // a 是至多 24 个物品体积，cap 是箱容量；返回最少箱数。
    int n = (int)a.size();
    assert(0 <= n && n <= 24); // 调试检查，可删。
    assert(cap >= 0); // 调试检查，可删。
    for (int x : a)
    {
        assert(0 <= x && x <= cap); // 调试检查，可删。
    }
    // State[0..1] 依次是已用箱数与最后一个箱的已用容量。
    using State = array<int, 2>;
    State inf{n + 1, 0};
    vector<State> dp(1LL << n, inf);
    dp[0] = {1, 0};
    for (int s = 0; s < (1LL << n); s++)
    {
        for (int i = 0; i < n; i++)
```

```
        {
            if (s >> i & 1)
            {
                continue;
            }
            State cur = dp[s];
            if (a[i] <= cap - cur[1])
            {
                cur[1] += a[i];
            }
            else
            {
                cur[0]++;
                cur[1] = a[i];
            }
            dp[s | 1LL << i] = min(dp[s | 1LL << i], cur);
        }
    }
    return n == 0 ? 0 : dp.back()[0];
}
}
```

`firstFitDec(a,cap)` 返回 FFD 的近似箱数；`firstFit(a,cap)` 返回按输入顺序 First-Fit 使用的箱数及每件物品的 0 下标箱号；二者都要求物品体积落在 $[0,cap]$，但排序会改变放置顺序，结果不能混称。当前 FFD 为 $O(n^2)$，线段树 First-Fit 为 $O(n\log n)$ 时间、$O(n)$ 空间。`exactBins(a,cap)` 要求物品数 ≤ 24，以 $O(n2^n)$ 时间、$O(2^n)$ 空间返回精确最少箱数。物品多且必须保持到达顺序时用 `firstFit`，允许重排且只需简单近似时用 FFD，$n\le 24$ 要精确最优用 `exactBins`；若还要在容量约束下优化物品价值则退化为背包，属第 12 章动态规划范畴。

14.1.009 选数与网格路径计数

只向下或向右走的网格路径计数。无障碍时从 $(0,0)$ 到 $(h-1,w-1)$ 有 $\binom{h+w-2}{h-1}$ 条路径。有障碍时把障碍与终点按坐标排序，令 `dp[i]` 为到第 i 个点且不经过任何更早障碍的路径数；先计全部路径，再减去每个能到达它的障碍 j 所截获的 $dp_j\binom{(x_i-x_j)+(y_i-y_j)}{x_i-x_j}$。组合数复用第 8 章代表实现，本条只保留容斥转移。

```
// 知识点 14.1.009：障碍点网格最短路径计数。
namespace grid_count
{
template<class T, class C>
T gridPaths(int h, int w, vector<array<int, 2>> ban, C &comb)
{
    // h、w 是网格尺寸，ban[i] 是障碍坐标，comb 提供 C(n,k)；返回模 T::mod() 的路径数。
    assert(h > 0 && w > 0); // 调试检查，可删。
    assert(h - 1 < T::mod() && w - 1 < T::mod() - (h - 1)); // 调试检查，可删。
    for (auto p : ban)
    {
        assert(0 <= p[0] && p[0] < h && 0 <= p[1] && p[1] < w); // 调试检查，可删。
    }
```

```
    sort(ban.begin(), ban.end());
    ban.erase(unique(ban.begin(), ban.end()), ban.end());
    array<int, 2> goal{h - 1, w - 1};
    if (binary_search(ban.begin(), ban.end(), goal))
    {
        return T(0);
    }
    ban.push_back(goal);
    sort(ban.begin(), ban.end());
    int goalPos = lower_bound(ban.begin(), ban.end(), goal) - ban.begin();
    vector<T> dp(ban.size());
    for (int i = 0; i < (int)ban.size(); i++)
    {
        auto [x, y] = ban[i];
        dp[i] = comb.C(x + y, x);
        for (int j = 0; j < i; j++)
        {
            auto [u, v] = ban[j];
            if (u <= x && v <= y)
            {
                dp[i] -= dp[j] * comb.C(x - u + y - v, x - u);
            }
        }
    }
    return dp[goalPos];
}
}
```

先引入 1.1.008 静态取模类的 `MInt<P>` 与 8.1.001 阶乘、逆阶乘与组合数的 `Comb<T>`，再以 `Comb<Z> c; grid_count::gridPaths<Z>(h,w,ban,c)` 调用。要求 $h, w > 0$、障碍均在网格内，且 $h + w - 2 < P$，从而所有阶乘在素模 P 下可逆；重复障碍会自动去重，终点是障碍时返回 0。设去重后有 k 个障碍，预处理与 DP 共用 $O(h + w + k^2)$ 时间、$O(h + w + k)$ 空间。网格较小而障碍很多时改用 $O(hw)$ 的逐格递推。

14.1.010 坐标类

二维点、向量、直线和线段基本操作统一使用 11.1.002 Point / Vector 的 `Point<T>`、`Line<T>`、点积、叉积、投影、交点与距离接口，本章不再复制同一实现。该代表条目要求直线方向向量非零；求两直线唯一交点前还须确认它们不平行。

浮点坐标有两套不能混用的关系：排序、`set` 和 `map` 的比较必须采用原始坐标的精确字典序，以保持严格弱序；判断几何重合、共线或方向时才用 `eps` 判号。整点的精确共线与格点计数应直接使用整数坐标和宽化叉积，含角度、投影或距离时再使用浮点版本。所有基础点线操作均为 $O(1)$ 时间和空间。

14.1.011 Python：基础语法、容器、字符串、数学库

Python 3 竞赛常用写法速查。`tokens()` 一次性读入并按空白切分（`sys.stdin.buffer.read().split()` 是大数据量下最快的读法），其余是列表推导、`bisect` 二分、`Counter`/`defaultdict`/`deque`、`heapq` 小根堆

（模拟大根堆时压相反数）、字符串 `split`/`join`/切片，以及 `math` 的 `gcd`/`lcm`/`isqrt`。要点在于 Python 整数自动扩展精度、不会溢出，列表与字符串切片会复制（$O(n)$），集合与字典平均 $O(1)$。

```python
import heapq
import math
import sys
from bisect import bisect_left, bisect_right
from collections import Counter, defaultdict, deque


def tokens():
    # 无参数；读取标准输入剩余内容，返回按空白切分的字节串列表。
    return sys.stdin.buffer.read().split()


# a 是整数表；切片会复制，推导式生成新表。
a = [1, 3, 3, 8]
b = [x * x for x in a if x & 1]
lo = bisect_left(a, 3)
hi = bisect_right(a, 3)

# cnt 是频率表，pos 按值保存出现位置，q 是双端队列。
cnt = Counter(a)
pos = defaultdict(list)
for i, x in enumerate(a):
    pos[x].append(i)
q = deque(a)
q.appendleft(0)

# hp 是小根堆；要模拟大根堆时压入相反数。
hp = a[:]
heapq.heapify(hp)
mn = heapq.heappop(hp)

# 字符串不可修改；split、join 和切片都会产生新字符串。
s = "acm-template"
parts = s.split("-")
joined = "/".join(parts)
rev = s[::-1]

# Python 整数不限位数；数学库函数的返回类型按标准库定义。
g = math.gcd(84, 30)
lcm = math.lcm(84, 30)
root = math.isqrt(10**100)
ans = sum(b)
sys.stdout.write(str(ans) + "\n")
```

整段按 Python 3 运行，示例变量只为展示接口，提交时保留需要的部分即可；`tokens()` 读取剩余全部标准输入。列表末尾追加均摊 $O(1)$、排序 $O(n \log n)$、二分 $O(\log n)$、字典与集合平均 $O(1)$，切片 / 拼接 / 整体读入 $O(n)$。数据结构与流、多项式的 Python 版见 14.1.012 Python：并查集、线段树、最大流、费用流、FFT、NTT，日期与精确分数见 14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO。

14.1.012 Python：并查集、线段树、最大流、费用流、FFT、NTT

Python 版的并查集、点修改区间和线段树、Dinic 最大流、SPFA 增广的最小费用最大流、FFT 与 NTT，供允许 Python 且规模适中的题直接取用。`SegTree` 把内部叶数补到 $N = \text{bit_ceil}(n)$，使用长度 $2N$ 的数组；Dinic 的阻塞流搜索使用显式路径栈，不受 Python 递归深度限制；流量、距离与费用直接使用任意精度整数，不设置有限“无穷”哨兵。

```python
import cmath
from collections import deque


class DSU:
    def __init__(self, n):
        # n 是点数；建立 n 个单点集合，无返回值。
        self.fa = list(range(n))
        self.siz = [1] * n

    def find(self, x):
        # x 是点编号；返回 x 所在集合的根并做路径压缩。
        while x != self.fa[x]:
            self.fa[x] = self.fa[self.fa[x]]
            x = self.fa[x]
        return x

    def merge(self, x, y):
        # x、y 是点编号；合并成功返回 True，原本连通返回 False。
        x, y = self.find(x), self.find(y)
        if x == y:
            return False
        if self.siz[x] < self.siz[y]:
            x, y = y, x
        self.fa[y] = x
        self.siz[x] += self.siz[y]
        return True


class SegTree:
    def __init__(self, a):
        # a 是初始数列；建立点修改、区间和线段树，无返回值。
        self.n = 1
        while self.n < len(a):
            self.n <<= 1
        self.tr = [0] * (2 * self.n)
        self.tr[self.n:self.n + len(a)] = a
        for p in range(self.n - 1, 0, -1):
            self.tr[p] = self.tr[p << 1] + self.tr[p << 1 | 1]

    def set(self, p, x):
        # p 是 0 下标位置，x 是新值；完成单点赋值，无返回值。
        p += self.n
        self.tr[p] = x
```

```
        while p > 1:
            p >>= 1
            self.tr[p] = self.tr[p << 1] + self.tr[p << 1 | 1]

    def prod(self, l, r):
        # l、r 定义半开区间；返回该区间元素和。
        l += self.n
        r += self.n
        x = y = 0
        while l < r:
            if l & 1:
                x += self.tr[l]
                l += 1
            if r & 1:
                r -= 1
                y += self.tr[r]
            l >>= 1
            r >>= 1
        return x + y


class Dinic:
    def __init__(self, n):
        # n 是点数；g 保存边的终点、反向边编号和剩余容量。
        self.n = n
        self.g = [[] for _ in range(n)]

    def add(self, u, v, c):
        # u、v 是端点，c 是非负容量；加入一条有向边，无返回值。
        assert 0 <= u < self.n and 0 <= v < self.n
        assert c >= 0
        a = [v, len(self.g[v]), c]
        b = [u, len(self.g[u]), 0]
        self.g[u].append(a)
        self.g[v].append(b)

    def flow(self, s, t):
        # s、t 是源汇；返回最大流并修改残量网络。
        assert 0 <= s < self.n and 0 <= t < self.n and s != t
        ans = 0
        while True:
            dep = [-1] * self.n
            dep[s] = 0
            q = deque([s])
            while q:
                u = q.popleft()
                for v, _, c in self.g[u]:
                    if c and dep[v] < 0:
                        dep[v] = dep[u] + 1
                        q.append(v)
            if dep[t] < 0:
                return ans
```

```
            cur = [0] * self.n

            def send():
                # 无参数；在当前层次图中用显式栈找到一条增广路并返回流量。
                path_v = [s]
                path_e = []
                while path_v:
                    u = path_v[-1]
                    if u == t:
                        f = min(self.g[x][i][2] for x, i in path_e)
                        for x, i in path_e:
                            e = self.g[x][i]
                            e[2] -= f
                            self.g[e[0]][e[1]][2] += f
                        return f
                    while cur[u] < len(self.g[u]):
                        e = self.g[u][cur[u]]
                        if e[2] and dep[e[0]] == dep[u] + 1:
                            break
                        cur[u] += 1
                    if cur[u] == len(self.g[u]):
                        path_v.pop()
                        if path_e:
                            x, _ = path_e.pop()
                            cur[x] += 1
                        continue
                    path_e.append((u, cur[u]))
                    path_v.append(self.g[u][cur[u]][0])
                return 0

            while True:
                f = send()
                if not f:
                    break
                ans += f


class MinCostFlow:
    def __init__(self, n):
        # n 是点数；g 保存终点、反向边编号、容量和单位费用。
        self.n = n
        self.g = [[] for _ in range(n)]

    def add(self, u, v, c, w):
        # u、v 是端点，c 是容量，w 是单位费用；加入有向边，无返回值。
        assert 0 <= u < self.n and 0 <= v < self.n
        assert c >= 0
        a = [v, len(self.g[v]), c, w]
        b = [u, len(self.g[u]), 0, -w]
        self.g[u].append(a)
        self.g[v].append(b)

```

```
    def flow(self, s, t, lim=None):
        # s、t 是源汇，lim 是可选流量上限；返回实际流量与最小费用。
        n = self.n
        assert 0 <= s < n and 0 <= t < n and s != t
        assert lim is None or lim >= 0
        total = 0
        cost = 0
        while lim is None or total < lim:
            dis = [None] * n
            pre = [None] * n
            inq = [False] * n
            depth = [0] * n
            dis[s] = 0
            q = deque([s])
            inq[s] = True
            while q:
                u = q.popleft()
                inq[u] = False
                for i, e in enumerate(self.g[u]):
                    v, _, c, w = e
                    nd = dis[u] + w
                    if c and (dis[v] is None or nd < dis[v]):
                        dis[v] = nd
                        pre[v] = (u, i)
                        depth[v] = depth[u] + 1
                        if depth[v] >= n:
                            raise ValueError("source-reachable negative cycle")
                        if not inq[v]:
                            inq[v] = True
                            q.append(v)
            if pre[t] is None:
                break
            f = None if lim is None else lim - total
            v = t
            while v != s:
                u, i = pre[v]
                c = self.g[u][i][2]
                f = c if f is None else min(f, c)
                v = u
            v = t
            while v != s:
                u, i = pre[v]
                e = self.g[u][i]
                e[2] -= f
                self.g[v][e[1]][2] += f
                cost += f * e[3]
                v = u
            total += f
        return total, cost


def fft(a, inv=False):
```

```
    # a 是长度为二次幂的复数表，inv 表示逆变换；原地修改 a，无返回值。
    n = len(a)
    if n == 0 or n & (n - 1):
        raise ValueError("FFT length must be a nonzero power of two")
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]
    length = 2
    while length <= n:
        wn = cmath.exp((2j if inv else -2j) * cmath.pi / length)
        for l in range(0, n, length):
            w = 1
            half = length >> 1
            for i in range(l, l + half):
                x = a[i]
                y = a[i + half] * w
                a[i] = x + y
                a[i + half] = x - y
                w *= wn
        length <<= 1
    if inv:
        for i in range(n):
            a[i] /= n


def conv_fft(a, b):
    # a、b 是实数系数表；返回普通卷积的浮点近似值。
    if not a or not b:
        return []
    n = 1
    while n < len(a) + len(b) - 1:
        n <<= 1
    x = list(map(complex, a)) + [0j] * (n - len(a))
    y = list(map(complex, b)) + [0j] * (n - len(b))
    fft(x)
    fft(y)
    for i in range(n):
        x[i] *= y[i]
    fft(x, True)
    return [x[i].real for i in range(len(a) + len(b) - 1)]


def ntt(a, inv=False, mod=998244353, root=3):
    # a 是模 mod 系数表，inv 表示逆变换，root 是原根；原地修改 a。
    n = len(a)
    supported = {(998244353, 3), (1004535809, 3), (469762049, 3)}
```

```
    if (mod, root) not in supported:
        raise ValueError("unsupported NTT modulus or primitive root")
    if n == 0 or n & (n - 1) or (mod - 1) % n:
        raise ValueError("invalid NTT length")
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j & bit:
            j ^= bit
            bit >>= 1
        j ^= bit
        if i < j:
            a[i], a[j] = a[j], a[i]
    length = 2
    while length <= n:
        wn = pow(root, (mod - 1) // length, mod)
        if inv:
            wn = pow(wn, mod - 2, mod)
        for l in range(0, n, length):
            w = 1
            half = length >> 1
            for i in range(l, l + half):
                x = a[i]
                y = a[i + half] * w % mod
                a[i] = (x + y) % mod
                a[i + half] = (x - y) % mod
                w = w * wn % mod
        length <<= 1
    if inv:
        ni = pow(n, mod - 2, mod)
        for i in range(n):
            a[i] = a[i] * ni % mod


def conv_ntt(a, b, mod=998244353, root=3):
    # a、b 是模 mod 系数表；返回精确模卷积，模数与原根必须在受支持表中。
    if not a or not b:
        return []
    n = 1
    while n < len(a) + len(b) - 1:
        n <<= 1
    x = [v % mod for v in a] + [0] * (n - len(a))
    y = [v % mod for v in b] + [0] * (n - len(b))
    ntt(x, mod=mod, root=root)
    ntt(y, mod=mod, root=root)
    for i in range(n):
        x[i] = x[i] * y[i] % mod
    ntt(x, True, mod, root)
    return x[:len(a) + len(b) - 1]
```

所有下标从 0 起。`Dinic.flow(s,t)` 要求端点合法、$s \neq t$、容量非负；显式栈实现不受递归深度限制。`MinCostFlow.flow(s,t,lim)` 中 `lim=None` 表示尽量增广，要求每轮残量图不存在源可达负费用环，检测

到时抛出 `ValueError`。`conv_fft` 返回浮点近似，只有调用者另行证明每个系数误差小于 $\frac{1}{2}$ 时才能舍入成精确整数；无条件精确结果应使用 `conv_ntt` 或多模 NTT 加 CRT。当前 NTT 支持 $(998244353, 3)$、$(1004535809, 3)$、$(469762049, 3)$，并要求补零长度为非零二次幂且整除 $mod - 1$。并查集单次均摊 $O(\alpha(n))$，线段树单次 $O(\log n)$、空间 $O(N)$，其中 $N = \text{bit_ceil}(n)$；Dinic 为 $O(V^2E)$，SPFA 费用流最坏 $O(FVE)$，FFT/NTT 为 $O(n \log n)$。

下面用固定答案检查主要接口；长链和大整数压力测试放在本地测试文件：

```
d = DSU(4)
assert d.merge(0, 1) and not d.merge(1, 0)
st = SegTree([1, 2, 3, 4])
assert st.prod(1, 4) == 9
st.set(2, 10)
assert st.prod(0, 4) == 17

flow = Dinic(4)
for e in [(0, 1, 3), (0, 2, 2), (1, 2, 1), (1, 3, 2), (2, 3, 4)]:
    flow.add(*e)
assert flow.flow(0, 3) == 5

empty = Dinic(4)
empty.add(0, 1, 7)
empty.add(2, 3, 9)
assert empty.flow(0, 3) == 0

mcf = MinCostFlow(3)
mcf.add(0, 1, 2, 10**6)
mcf.add(1, 2, 2, -3)
assert mcf.flow(0, 2) == (2, 2 * (10**6 - 3))

got = conv_fft([1, 2, 3], [4, -1])
assert all(abs(x - y) < 1e-8 for x, y in zip(got, [4, 7, 10, -3]))
assert conv_ntt([1, 2, 3], [4, 5]) == [4, 13, 22, 15]
```

14.1.013 Python：Splay

维护可重复整数的伸展树（Python 版），支持插入、删一个、按排名取值 `kth` 与查询严格小于给定值的元素数 `rank`。用平行数组 `ch/fa/val/cnt/siz` 存节点、第 0 项当空哨兵；每次查找命中或触底后都把相应节点 `_splay` 旋到根，摊还掉最坏深度。`_rotate` 单旋后必须先 `_pull` 儿子再 `_pull` 自己以自底向上重算子树大小，`_splay` 按 zig-zig / zig-zag 分类决定先旋父还是先旋子——次序错会破坏平衡性的摊还保证。

```
class Splay:
    def __init__(self):
        # 无参数；建立空伸展树，各列表第 0 项表示空节点。
        self.ch = [[0, 0]]
        self.fa = [0]
        self.val = [0]
        self.cnt = [0]
        self.siz = [0]
```

```
        self.rt = 0
        self.free = []

    def _new(self, x, f):
        # x 是键值，f 是父节点；创建节点并返回编号。
        if self.free:
            u = self.free.pop()
            self.ch[u] = [0, 0]
            self.fa[u] = f
            self.val[u] = x
            self.cnt[u] = 1
            self.siz[u] = 1
            return u
        self.ch.append([0, 0])
        self.fa.append(f)
        self.val.append(x)
        self.cnt.append(1)
        self.siz.append(1)
        return len(self.val) - 1

    def _recycle(self, u):
        # u 是已从树中断开的节点；清空并回收其编号，无返回值。
        self.ch[u] = [0, 0]
        self.fa[u] = 0
        self.cnt[u] = 0
        self.siz[u] = 0
        self.free.append(u)

    def _pull(self, u):
        # u 是节点编号；根据儿子和重复计数重算子树大小，无返回值。
        self.siz[u] = self.siz[self.ch[u][0]] + self.siz[self.ch[u][1]] + self.cnt[u]

    def _rotate(self, x):
        # x 是待上旋节点；完成一次旋转并维护父子关系，无返回值。
        y = self.fa[x]
        z = self.fa[y]
        d = int(self.ch[y][1] == x)
        b = self.ch[x][d ^ 1]
        if z:
            self.ch[z][int(self.ch[z][1] == y)] = x
        else:
            self.rt = x
        self.fa[x] = z
        self.ch[x][d ^ 1] = y
        self.fa[y] = x
        self.ch[y][d] = b
        if b:
            self.fa[b] = y
        self._pull(y)
        self._pull(x)

    def _splay(self, x, goal=0):
```

```
        # x 是目标节点，goal 是目标父亲；把 x 旋到 goal 下，无返回值。
        while self.fa[x] != goal:
            y = self.fa[x]
            z = self.fa[y]
            if z != goal:
                if (self.ch[y][1] == x) == (self.ch[z][1] == y):
                    self._rotate(y)
                else:
                    self._rotate(x)
            self._rotate(x)

    def _find(self, x):
        # x 是键值；若存在则伸展到根并返回节点，否则伸展最后访问点并返回 0。
        u = self.rt
        last = 0
        while u and self.val[u] != x:
            last = u
            u = self.ch[u][x > self.val[u]]
        if u:
            self._splay(u)
        elif last:
            self._splay(last)
        return u

    def insert(self, x):
        # x 是键值；插入一个 x，无返回值。
        if not self.rt:
            self.rt = self._new(x, 0)
            return
        u = self.rt
        while True:
            if self.val[u] == x:
                self.cnt[u] += 1
                self._pull(u)
                self._splay(u)
                return
            d = int(x > self.val[u])
            if not self.ch[u][d]:
                v = self._new(x, u)
                self.ch[u][d] = v
                self._pull(u)
                self._splay(v)
                return
            u = self.ch[u][d]

    def erase(self, x):
        # x 是键值；删除一个 x，成功返回 True，不存在返回 False。
        u = self._find(x)
        if not u:
            return False
        if self.cnt[u] > 1:
            self.cnt[u] -= 1
```

```
            self._pull(u)
            return True
        l, r = self.ch[u]
        if not l:
            self.rt = r
            if r:
                self.fa[r] = 0
            self._recycle(u)
            return True
        self.rt = l
        self.fa[l] = 0
        v = l
        while self.ch[v][1]:
            v = self.ch[v][1]
        self._splay(v)
        self.ch[v][1] = r
        if r:
            self.fa[r] = v
        self._pull(v)
        self._recycle(u)
        return True

    def kth(self, k):
        # k 是从 0 开始的排名；返回对应键值，越界返回 None。
        if k < 0 or k >= self.siz[self.rt]:
            return None
        u = self.rt
        while True:
            left = self.siz[self.ch[u][0]]
            if k < left:
                u = self.ch[u][0]
            elif k < left + self.cnt[u]:
                self._splay(u)
                return self.val[u]
            else:
                k -= left + self.cnt[u]
                u = self.ch[u][1]

    def rank(self, x):
        # x 是查询值；返回严格小于 x 的元素个数，并伸展最后访问节点。
        u = self.rt
        last = 0
        ans = 0
        while u:
            last = u
            if x <= self.val[u]:
                u = self.ch[u][0]
            else:
                ans += self.siz[self.ch[u][0]] + self.cnt[u]
                u = self.ch[u][1]
        if last:
            self._splay(last)
```

```
        return ans
```

先构造 `Splay()`：`insert(x)` 插一个，`erase(x)` 删一个，`kth(k)` 取第 k 小，`rank(x)` 返回严格小于 x 的元素数。各操作均摊 $O(\log n)$、单次最坏 $O(n)$；空闲编号会复用，数组空间为 $O(M)$，其中 M 是历史上同时存在的不同键数峰值，不随累计插入次数增长。`sortedcontainers.SortedList` 是第三方库，只有评测环境已安装时才能替代本实现；C++ 平衡树见第 3 章数据结构。

下面用一组短操作检查重复计数、排名、删除和编号复用：

```
t = Splay()
for x in [3, 1, 3, 2]:
    t.insert(x)
assert [t.kth(i) for i in range(4)] == [1, 2, 3, 3]
assert t.rank(3) == 2
assert t.erase(3) and [t.kth(i) for i in range(3)] == [1, 2, 3]
assert t.erase(3) and not t.erase(3)
for x in range(1000):
    t.insert(x + 10)
    assert t.erase(x + 10)
assert len(t.val) == 4
```

14.1.014 Python：`datetime`、`Fraction`、递归深度与文件 IO

Python 的日期差、精确分数、递归深度与文本文件 IO 汇总。`day_diff` 用 `datetime.date` 相减取天数；`Fraction` 自动约分、保持精确有理数；`read_text`/`write_text` 显式按 UTF-8 读写。一个常见坑：`sys.setrecursionlimit` 只抬高解释器的递归计数阈值，并不扩大 C 层系统栈，深达几万层的递归仍可能段错误，真正的深搜要改成显式栈或加大线程栈运行。

```
import sys
from datetime import date, datetime, timedelta
from fractions import Fraction
from pathlib import Path


def day_diff(y1, m1, d1, y2, m2, d2):
    # 两组三元组是公历年月日；返回第二个日期减第一个日期的天数。
    return (date(y2, m2, d2) - date(y1, m1, d1)).days


def read_text(path):
    # path 是文件路径；按 UTF-8 读取并返回完整文本。
    return Path(path).read_text(encoding="utf-8")


def write_text(path, text):
    # path 是文件路径，text 是内容；按 UTF-8 覆盖写入，无返回值。
    Path(path).write_text(text, encoding="utf-8")


```

```python
# 提高限制只改变 Python 检查阈值，不能保证系统栈足够。
sys.setrecursionlimit(1_000_000)

# 日期时间可解析、格式化和加减时间段。
now = datetime.strptime("2026-07-23 12:30:00", "%Y-%m-%d %H:%M:%S")
tomorrow = now + timedelta(days=1)
stamp = tomorrow.strftime("%Y/%m/%d %H:%M:%S")
delta = day_diff(2026, 7, 1, 2026, 7, 23)

# Fraction 始终保持精确分子分母并自动约分。
q = Fraction(2, 3) + Fraction(5, 7)
num, den = q.num, q.den

# 标准输入输出优先使用缓冲接口；文件 IO 只在题目明确要求时使用。
data = sys.stdin.buffer.readline().split()
sys.stdout.write(f"{num}/{den} {delta}\n")
```

文件函数显式用 UTF-8；竞赛标准 IO 通常直接用 `sys.stdin.buffer` 和 `sys.stdout.write`。日期与分数基本运算对机器字长输入视为 $O(1)$，读写长度 L 的文件 $O(L)$ 时间与空间，大整数分数运算随分子分母位数增长。C++ 端的日期换算见 14.1.005 日期与时间换算，精确分数见 14.1.002 分数类与有理数还原，基础语法见 14.1.011 Python：基础语法、容器、字符串、数学库。

14.1.015 完全图部分边染色的 1-因子扩展尝试

本条尝试在不改已有颜色的前提下，把 K_n 的合法 m-边染色补成 K_{m+1} 的 1-因子分解。最终每种颜色都须是大小 $\frac{m+1}{2}$ 的完美匹配，故 m 必为奇数。代码逐点加入新顶点，并用一次二分图匹配为该点的全部新边选色；已经确定的选择不回溯到先前顶点，因此成功时一定给出合法扩展，失败只表示这套逐点策略没有找到，不能证明不存在扩展。

```cpp
// 知识点 14.1.015：把部分完全图边染色扩展成完整 1-因子分解。
namespace factor_ext
{
optional<vector<vector<int>>> extendFactor(vector<vector<int>> a, int m)
{
    // a 是部分染色矩阵，m 是颜色数；成功时返回完整染色，本策略失败时返回空。
    auto n = (int)a.size();
    if (m <= 0 || m % 2 == 0 || n > m + 1)
    {
        return nullopt;
    }
    for (auto &r : a)
    {
        if ((int)r.size() != n)
        {
            return nullopt;
        }
    }
    vector<vector<char>> used(n, vector<char>(m));
    vector<int> ec(m);
```

```
    for (int i = 0; i < n; i++)
    {
        if (a[i][i] != -1)
        {
            return nullopt;
        }
        for (int j = i + 1; j < n; j++)
        {
            auto c = a[i][j];
            if (c < 0 || c >= m || a[j][i] != c)
            {
                return nullopt;
            }
            if (used[i][c] || used[j][c])
            {
                return nullopt;
            }
            used[i][c] = true;
            used[j][c] = true;
            ec[c]++;
        }
    }

    auto cap = (m + 1) / 2;
    while (n < m + 1)
    {
        for (int c = 0; c < m; c++)
        {
            if (n - ec[c] > cap)
            {
                return nullopt;
            }
        }

        vector<vector<int>> g(m);
        for (int c = 0; c < m; c++)
        {
            for (int v = 0; v < n; v++)
            {
                if (!used[v][c])
                {
                    g[c].push_back(v);
                }
            }
            if (n - ec[c] < cap)
            {
                for (int v = n; v < m; v++)
                {
                    g[c].push_back(v);
                }
            }
        }
```

```
73
74          vector<int> mt(m, -1);
75          vector<int> ord(m);
76          iota(ord.begin(), ord.end(), 0);
77          stable_sort(ord.begin(),
78                      ord.end(),
79                      [&](int x, int y)
80                      {
81                          return g[x].size() < g[y].size();
82                      });
83          auto dfs = [&](auto &&self, int c, vector<char> &vis) -> bool
84          {
85              for (auto v : g[c])
86              {
87                  if (vis[v])
88                  {
89                      continue;
90                  }
91                  vis[v] = true;
92                  if (mt[v] == -1 || self(self, mt[v], vis))
93                  {
94                      mt[v] = c;
95                      return true;
96                  }
97              }
98              return false;
99          };
100         for (auto c : ord)
101         {
102             vector<char> vis(m);
103             if (!dfs(dfs, c, vis))
104             {
105                 return nullopt;
106             }
107         }
108
109         for (auto &r : a)
110         {
111             r.push_back(-1);
112         }
113         a.push_back(vector<int>(n + 1, -1));
114         used.push_back(vector<char>(m));
115         for (int v = 0; v < n; v++)
116         {
117             auto c = mt[v];
118             a[v][n] = c;
119             a[n][v] = c;
120             used[v][c] = true;
121             used[n][c] = true;
122             ec[c]++;
123         }
124         n++;
```

```
    }
    return a;
}
} // 命名空间 factor_ext
```

`extendFactor(a,m)` 的输入是对称颜色矩阵，主对角线为 -1，已有边色在 $[0,m)$ 内且同色边不共端点。成功时返回完整染色；若输入非法或当前贪心顺序失败则返回空，后一种情况不代表数学上无解。逐点匈牙利增广最坏 $O(m^4)$ 时间，空间 $O(m^2)$。

下面直接检查 $m=3$ 的固定完整染色，并保留一组当前贪心返回空的合法前缀；后者只覆盖失败分支，不表示数学上无解：

```
signed main()
{
    vector<vector<int>> want{{-1, 2, 1, 0}, {2, -1, 0, 1},
                             {1, 0, -1, 2}, {0, 1, 2, -1}};
    assert(factor_ext::extendFactor({}, 3) == optional(want));
    vector<vector<int>> bad{
        {-1, 0, 1, 2},
        {0, -1, 2, 1},
        {1, 2, -1, 0},
        {2, 1, 0, -1}
    };
    assert(!factor_ext::extendFactor(bad, 5));
}
```

14.1.016 高精度整数平方根

任意长非负十进制整数的向下取整平方根，全程走字符串高精度、不依赖机器整数宽度。用的是竖式逐位开方（不是二分）：把数字从高位起每两位一组“落下”，维护当前已得根 `root` 与余数 `rem`；每组令 `base = 20*root`，从 9 往下试最大的数字 d 使 $(base+d)\cdot d \le rem$——这正是 $(10\,root+d)^2-100\,root^2$ 的展开，把它从 `rem` 减掉并令 `root\gets 10\,root+d`。配套的 `trimNum/cmpNum/mulSmall/addSmall/subNum` 是够用的最小字符串算术（乘、加的第二操作数限定在小常数范围内）。结束时 `root` 满足 $root^2 \le n < (root+1)^2$。

```
string trimNum(string s)
{
    // 参数 s 是非空十进制数字串；返回删除多余前导零后的规范表示。
    auto p = s.find_first_not_of('0');
    return p == string::npos ? "0" : s.substr(p);
}

int cmpNum(const string &a, const string &b)
{
    // 参数 a、b 是规范非负十进制数；返回 -1、0、1 表示 a 与 b 的大小关系。
    if (a.size() != b.size())
    {
        return a.size() < b.size() ? -1 : 1;
    }
```

```
    if (a == b)
    {
        return 0;
    }
    return a < b ? -1 : 1;
}

string mulSmall(const string &a, int b)
{
    // 参数 a 是规范非负十进制数，b 在 [0,100]；返回精确乘积。
    if (b == 0 || a == "0")
    {
        return "0";
    }
    string c;
    int carry = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--)
    {
        auto v = (a[i] - '0') * b + carry;
        c.push_back(char('0' + v % 10));
        carry = v / 10;
    }
    while (carry)
    {
        c.push_back(char('0' + carry % 10));
        carry /= 10;
    }
    reverse(c.begin(), c.end());
    return c;
}

string addSmall(string a, int b)
{
    // 参数 a 是规范非负十进制数，b 在 [0,99]；返回精确和。
    for (int i = (int)a.size() - 1; i >= 0 && b; i--)
    {
        auto v = a[i] - '0' + b;
        a[i] = char('0' + v % 10);
        b = v / 10;
    }
    while (b)
    {
        a.insert(a.begin(), char('0' + b % 10));
        b /= 10;
    }
    return trimNum(a);
}

string subNum(string a, const string &b)
{
    // 参数 a、b 是规范非负十进制数且 a>=b；返回精确差。
    assert(cmpNum(a, b) >= 0); // 调试检查，可删。
```

```
    int borrow = 0;
    int j = (int)b.size() - 1;
    for (int i = (int)a.size() - 1; i >= 0; i--)
    {
        auto v = a[i] - '0' - borrow - (j >= 0 ? b[j] - '0' : 0);
        borrow = v < 0;
        if (borrow)
        {
            v += 10;
        }
        a[i] = char('0' + v);
        j--;
    }
    return trimNum(a);
}

string bigSqrt(string s)
{
    // 参数 s 是无前导符号的非负十进制整数；返回 floor(sqrt(s)) 的十进制表示。
    assert(!s.empty()); // 调试检查，可删。
    for (auto c : s)
    {
        assert('0' <= c && c <= '9'); // 调试检查，可删。
    }
    s = trimNum(s);
    if (s == "0")
    {
        return "0";
    }
    if (s.size() & 1)
    {
        s = "0" + s;
    }
    string root = "0";
    string rem = "0";
    for (int i = 0; i < (int)s.size(); i += 2)
    {
        rem = trimNum(rem + s.substr(i, 2));
        auto base = mulSmall(root, 20);
        int d = 9;
        while (cmpNum(mulSmall(addSmall(base, d), d), rem) > 0)
        {
            d--;
        }
        rem = subNum(rem, mulSmall(addSmall(base, d), d));
        root = addSmall(mulSmall(root, 10), d);
    }
    return root;
}
```

`bigSqrt(s)` 的 `s` 是不带符号的非负十进制串（允许前导零），返回规范化（无前导零）的 $\lfloor\sqrt{s}\rfloor$。设十进制位数为 D，当前实现 $O(D^2)$ 时间、$O(D)$ 空间。数值落在 `long long` 内时直接用整型 `sqrtl` 加一两

步修正更快；要完整大整数四则见 14.1.001 高精度整数（BigInt），本条刻意用轻量字符串算术，免得为一次开方引入整套 `BigInt`。

竖式试商和落下 `00` 分组最易写错。下面覆盖零、平方边界和一组长数：

```
signed main()
{
    assert(bigSqrt("0") == "0");
    assert(bigSqrt("9999") == "99");
    assert(bigSqrt("15241578753238836750495351562536198787501905199875019052100")
           == "123456789012345678901234567890");
}
```

14.1.017 一元二次方程实根

实系数一元二次方程 $ax^2+bx+c=0$ 的全部实根，兼容退化成一次方程。次数按归一化前的精确零判断；随后把系数除以最大绝对值以控制判别式量级，要求原来非零的最高有效系数归一化后仍非零，这一可表示比例取决于评测机的 `long double`。二次情形用数值稳定的 $q=-\frac{1}{2}\left(b+\operatorname{sgn}(b)\sqrt{\Delta}\right)$，两根取 q/a 与 c/q，避免直接公式中的相消。

```
vector<long double> quadRoots(long double a, long double b, long double c)
{
    // 参数 a、b、c 是方程 ax^2+bx+c=0 的系数；返回升序实根，重根只返回一次，无实根返回空表。
    assert(isfinite(a) && isfinite(b) && isfinite(c)); // 调试检查，可删。
    bool hasA = a != 0, hasB = b != 0;
    long double scale = max({fabsl(a), fabsl(b), fabsl(c)});
    if (scale == 0)
    {
        return {};
    }
    a /= scale;
    b /= scale;
    c /= scale;
    assert((!hasA || a != 0) && (!hasB || b != 0)); // 调试检查，可删。
    if (!hasA)
    {
        return !hasB ? vector<long double>{} : vector<long double>{-c / b};
    }
    long double bb = b * b;
    long double ac4 = 4 * a * c;
    long double d = fmal(-4 * a, c, bb);
    long double tol = 64 * numeric_limits<long double>::epsilon()
                      * (fabsl(bb) + fabsl(ac4));
    if (d < -tol)
    {
        return {};
    }
    if (fabsl(d) <= tol)
    {
        return {-b / (2 * a)};
```

```
    }
    long double q = -0.5L * (b + copysignl(sqrtl(d), b));
    vector<long double> ans{q / a, c / q};
    sort(ans.begin(), ans.end());
    return ans;
}
```

`quadRoots(a,b,c)` 要求三个系数有限，且原来非零的最高有效系数除以最大绝对值后仍可表示为非零；MSVC 的 `long double` 通常等同 `double`，可用比例范围会小于 GCC x86 的扩展精度。函数返回升序实根，重根只保留一次；恒等式与无解的一次退化都返回空。时间、空间均为 $O(1)$。

下面检查一次退化、重根、无实根、相消明显的两根，以及同一方程在不同数量级缩放下的结果：

```
bool near(long double x, long double y)
{
    // x、y 是待比较浮点数；返回二者是否在相对误差 1e-15 内相等。
    return fabsl(x - y) <= 1E-15L * max({1.0L, fabsl(x), fabsl(y)});
}

signed main()
{
    assert(quadRoots(0, 2, -4) == vector<long double>{2});
    assert(quadRoots(0, 0, 1).empty());
    assert(quadRoots(1, 0, 1).empty());
    auto one = quadRoots(1, -2, 1);
    assert(one.size() == 1 && near(one[0], 1));
    for (long double k : {1E-200L, 1.0L, 1E200L})
    {
        auto r = quadRoots(k, -3 * k, 2 * k);
        assert(r.size() == 2 && near(r[0], 1) && near(r[1], 2));
    }
    auto r = quadRoots(1, -1E20L, 1);
    assert(r.size() == 2 && near(r[0], 1E-20L) && near(r[1], 1E20L));
    cout << "OK\n";
    return 0;
}
```

14.1.018 N 皇后构造解

$O(n)$ 直接构造一个 n 皇后方案（不是回溯枚举全部解），返回每行皇后所在列。$n=1$ 平凡，$n=2,3$ 无解返回空表，$n\ge 4$ 按 $n \bmod 6$ 分三类摆放：奇数 n 先解 $n-1$ 再把最后一行放到最后一列；偶数且 $n \bmod 6 \neq 2$ 用一套先偶列后奇列的等差排布；$n \bmod 6 = 2$ 用另一套带环移位的排布。三类构造都保证任意两皇后不同列、且行差绝对值不等于列差绝对值（即不同对角线），因此天然合法。

```
vector<int> nQueens(int n)
{
    // 参数 n 是棋盘边长；返回每行皇后的 0 下标列号，n=2 或 3 时返回空表。
    assert(n >= 1); // 调试检查，可删。
    if (n == 1)
```

```
    {
        return {0};
    }
    if (n < 4)
    {
        return {};
    }
    vector<int> p(n);
    auto gen = [&](auto &&self, int m, vector<int> &q) -> void
    {
        if (m & 1)
        {
            vector<int> t(m - 1);
            self(self, m - 1, t);
            copy(t.begin(), t.end(), q.begin());
            q[m - 1] = m - 1;
        }
        else if (m % 6 != 2)
        {
            for (int i = 1; i <= m / 2; i++)
            {
                q[i - 1] = 2 * i - 1;
            }
            for (int i = m / 2 + 1; i <= m; i++)
            {
                q[i - 1] = 2 * i - m - 2;
            }
        }
        else
        {
            for (int i = 1; i <= m / 2; i++)
            {
                q[i - 1] = (2 * i + m / 2 - 3) % m;
            }
            for (int i = m / 2 + 1; i <= m; i++)
            {
                q[i - 1] = m - 1 - (2 * (m - i + 1) + m / 2 - 3) % m;
            }
        }
    };
    gen(gen, n, p);
    return p;
}
```

`nQueens(n)` 要求 $n \geq 1$，返回第 i 行皇后的 0 下标列号；无解的 $n = 2, 3$ 返回空向量。构造用 $O(n)$ 时间和空间。只需一个可行解时使用本条；枚举全部方案或附加约束时改用 DFS 与位运算剪枝。同类约束满足问题见 14.1.007 9×9 数独字典序最小解。

下面检查最小有解、无解边界和一个常用规模：

```cpp
signed main()
{
    assert(nQueens(1) == vector<int>({0}));
    assert(nQueens(2).empty() && nQueens(3).empty());
    assert(nQueens(4) == vector<int>({1, 3, 0, 2}));
    assert(nQueens(8) == vector<int>({3, 5, 7, 1, 6, 0, 2, 4}));
}
```

14.1.019 十六进制百万位大整数

普通 14.1.001 高精度整数 面向十进制日常运算；百万级输入不能先逐块转换到二进制再转换回来。`BigInt` 直接以 10^9 为基数保存十进制块，短乘法使用竖式，长乘法复用 10.1.004 任意模数卷积 的三模 CRT 与 radix-4 NTT，除法使用 Newton 逆；它是十进制大整数题的接口。百万级十六进制位则使用更紧凑的二进制 limb：`HexBigInt` 每个 `uint32_t` 保存 8 个十六进制位，符号与绝对值分开；加减线性，乘法在短块用竖式、长块用 Karatsuba，除法在短块用规格化长除、长块用 Burnikel–Ziegler 的 $2n/n$ 与 $3n/2n$ 递归。两类结果都统一删除高零，零永远不带负号；除法向零截断，余数与被除数同号。

```cpp
// 每个 limb 固定保存 32 位；进位、移位、除法和 CRT 重建依赖该布局。
struct HexBigInt
{
    using It = vector<uint32_t>::iterator;
    using CIt = vector<uint32_t>::const_iterator;

    bool neg = false; // neg 表示负数，零恒为非负。
    vector<uint32_t> a; // a 是绝对值的 32 位小端 limb，无高位零，零为空数组。

    HexBigInt() = default;
    HexBigInt(long long x)
    {
        // x 是任意 64 位有符号整数（含 LLONG_MIN）；按符号与绝对值的 32 位小端 limb 构造。
        neg = x < 0;
        uint64_t y = neg ? 0 - (uint64_t)x : (uint64_t)x;
        while (y)
        {
            a.push_back((uint32_t)y);
            y >>= 32;
        }
    }
    HexBigInt(const string &s)
    {
        // s 是带可选 +/- 号、至少一位数字的十六进制串（大小写均可）；解析为对应大整数。
        assign(s);
    }
    HexBigInt(bool sgn, vector<uint32_t> v) : neg(sgn), a(move(v))
    {
        // sgn 为真表示负数，v 是绝对值的 32 位小端 limb；构造后去高零，值为零时强制非负。
        trim(this->a);
        if (this->a.empty())
        {
```

```
            this->neg = false;
        }
    }

    HexBigInt &assign(const string &s)
    {
        // s 是带可选 +/- 号、至少一位数字的十六进制串（0-9A-Fa-f）；覆盖当前值并返回 *this，零非负。
        assert(!s.empty()); // 调试检查，可删。
        int p = 0;
        neg = s[0] == '-';
        if (s[0] == '-' || s[0] == '+')
        {
            p++;
        }
        assert(p < (int)s.size()); // 调试检查，可删。
        a.clear();
        for (int r = s.size(); r > p; r -= 8)
        {
            int l = max(p, r - 8);
            uint32_t x = 0;
            for (int i = l; i < r; i++)
            {
                x <<= 4;
                if ('0' <= s[i] && s[i] <= '9')
                {
                    x |= s[i] - '0';
                }
                else if ('A' <= s[i] && s[i] <= 'F')
                {
                    x |= s[i] - 'A' + 10;
                }
                else
                {
                    assert('a' <= s[i] && s[i] <= 'f'); // 调试检查，可删。
                    x |= s[i] - 'a' + 10;
                }
            }
            a.push_back(x);
        }
        trim(a);
        if (a.empty())
        {
            neg = false;
        }
        return *this;
    }

    string toHex() const
    {
        // 无参数；返回大写十六进制串，负数带前导 -，无前导零，零返回 "0"。
        if (a.empty())
        {
```

```
            return "0";
        }
        const char *digit = "0123456789ABCDEF";
        string s = neg ? "-" : "";
        for (int i = a.size() - 1; i >= 0; i--)
        {
            string t(8, '0');
            uint32_t x = a[i];
            for (int j = 7; j >= 0; j--)
            {
                t[j] = digit[x & 15];
                x >>= 4;
            }
            if (i == (int)a.size() - 1)
            {
                t.erase(0, t.find_first_not_of('0'));
            }
            s += t;
        }
        return s;
    }

    bool isZero() const
    {
        // 无参数；返回当前值是否为零（limb 数组为空）。
        return a.empty();
    }
    HexBigInt operator-() const
    {
        // 无参数；返回相反数，零仍保持非负。
        HexBigInt x = *this;
        if (!x.a.empty())
        {
            x.neg = !x.neg;
        }
        return x;
    }

    friend bool operator==(const HexBigInt &x, const HexBigInt &y)
    {
        return x.neg == y.neg && x.a == y.a;
    }
    friend bool operator<(const HexBigInt &x, const HexBigInt &y)
    {
        // x、y 是规范大整数；返回按有符号数值比较 x<y 是否成立。
        if (x.neg != y.neg)
        {
            return x.neg;
        }
        int c = cmp(x.a, y.a);
        return x.neg ? c > 0 : c < 0;
    }
```

```
137     friend bool operator≠(const HexBigInt &x, const HexBigInt &y)
138     {
139         return !(x == y);
140     }
141     friend bool operator>(const HexBigInt &x, const HexBigInt &y)
142     {
143         // x、y 是规范大整数；返回 x>y 是否成立（即 y<x）。
144         return y < x;
145     }
146     friend bool operator<=(const HexBigInt &x, const HexBigInt &y)
147     {
148         return !(y < x);
149     }
150     friend bool operator>=(const HexBigInt &x, const HexBigInt &y)
151     {
152         return !(x < y);
153     }
154 
155     friend HexBigInt operator+(const HexBigInt &x, const HexBigInt &y)
156     {
157         // x、y 是规范大整数；返回 x+y：同号相加绝对值，异号用较大绝对值减较小绝对值。
158         if (x.neg == y.neg)
159         {
160             return {x.neg, addVec(x.a, y.a)};
161         }
162         int c = cmp(x.a, y.a);
163         if (c <= 0)
164         {
165             return {c ? y.neg : false, subVec(y.a, x.a)};
166         }
167         return {x.neg, subVec(x.a, y.a)};
168     }
169     friend HexBigInt operator-(const HexBigInt &x, const HexBigInt &y)
170     {
171         // x、y 是规范大整数；返回 x-y，即 x+(-y)。
172         return x + (-y);
173     }
174     friend HexBigInt operator*(const HexBigInt &x, const HexBigInt &y)
175     {
176         // x、y 是规范大整数；返回 x*y，积为零时非负。
177         auto z = mulVec(x.a, y.a);
178         return {z.empty() ? false : x.neg ≠ y.neg, move(z)};
179     }
180     friend pair<HexBigInt, HexBigInt> divmod(const HexBigInt &x, const HexBigInt &y)
181     {
182         // x 是被除数，y 是非零除数；返回 {商，余数}，商向零截断，余数与 x 同号（零非负）。
183         assert(!y.a.empty()); // 调试检查，可删：除数不得为零。
184         auto [q, r] = divVec(x.a, y.a);
185         return {{q.empty() ? false : x.neg ≠ y.neg, move(q)},
186                 {r.empty() ? false : x.neg, move(r)}};
187     }
188     friend HexBigInt operator/(const HexBigInt &x, const HexBigInt &y)
```

```
    {
        // x 是被除数，y 是非零除数；返回向零截断的商。
        return divmod(x, y).first;
    }
    friend HexBigInt operator%(const HexBigInt &x, const HexBigInt &y)
    {
        // x 是被除数，y 是非零除数；返回与 x 同号的余数 x-(x/y)*y。
        return divmod(x, y).second;
    }

    HexBigInt &operator+=(const HexBigInt &x)
    {
        return *this = *this + x;
    }
    HexBigInt &operator-=(const HexBigInt &x)
    {
        return *this = *this - x;
    }
    HexBigInt &operator*=(const HexBigInt &x)
    {
        return *this = *this * x;
    }
    HexBigInt &operator/=(const HexBigInt &x)
    {
        return *this = *this / x;
    }
    HexBigInt &operator%=(const HexBigInt &x)
    {
        return *this = *this % x;
    }
    friend ostream &operator<<(ostream &out, const HexBigInt &x)
    {
        // out 是输出流，x 是大整数；写入 x.toHex() 并返回 out。
        return out << x.toHex();
    }

  private:
    static constexpr int naiveDiv = 64; // naiveDiv 是改用竖式除法的规模阈值（limb 数）。

    static void trim(vector<uint32_t> &x)
    {
        // x 是小端 limb 数组；原地弹掉最高位的零 limb，零变为空数组。
        while (!x.empty() && x.back() == 0)
        {
            x.pop_back();
        }
    }
    static bool less(CIt x, CIt xe, CIt y, CIt ye)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间，长度可不同且允许高位零；返回无符号值 x<y 是否成立。
        int nx = xe - x, ny = ye - y;
        while (nx && x[nx - 1] == 0)
```

```
        {
            nx--;
        }
        while (ny && y[ny - 1] == 0)
        {
            ny--;
        }
        if (nx != ny)
        {
            return nx < ny;
        }
        for (int i = nx - 1; i >= 0; i--)
        {
            if (x[i] != y[i])
            {
                return x[i] < y[i];
            }
        }
        return false;
    }
    static int cmp(const vector<uint32_t> &x, const vector<uint32_t> &y)
    {
        // x、y 是已去高零的小端 limb 数组；返回无符号比较结果 -1（x<y）、0（相等）、1（x>y）。
        if (x == y)
        {
            return 0;
        }
        return less(x.begin(), x.end(), y.begin(), y.end()) ? -1 : 1;
    }

    static void add(CIt x, CIt xe, CIt y, CIt ye, It z, It ze)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间；把无符号和 x+y 写入 z 的低 max(|x|,|y|) 个 limb，有进位
        ↪ 时再写一位。
        // z 至少长 max(|x|,|y|)，产生最高进位时须再多一位；z 中更高的位不被改动。
        if (xe - x < ye - y)
        {
            swap(x, y);
            swap(xe, ye);
        }
        int nx = xe - x, ny = ye - y;
        assert(nx <= ze - z); // 调试检查，可删。
        uint32_t carry = 0;
        for (int i = 0; i < ny; i++)
        {
            uint64_t v = (uint64_t)x[i] + y[i] + carry;
            z[i] = v;
            carry = v >> 32;
        }
        for (int i = ny; i < nx; i++)
        {
            uint64_t v = (uint64_t)x[i] + carry;
```

```
            z[i] = v;
            carry = v >> 32;
        }
        if (carry)
        {
            assert(z + nx < ze); // 调试检查，可删。
            z[nx] = carry;
        }
    }
    static void addTo(It x, It xe, CIt y, CIt ye)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间且 |x|≥|y|；原地令 x+=y，要求最终进位能在 x 的长度内吸收。
        int nx = xe - x, ny = ye - y;
        uint32_t carry = 0;
        for (int i = 0; i < ny; i++)
        {
            uint64_t v = (uint64_t)x[i] + y[i] + carry;
            x[i] = v;
            carry = v >> 32;
        }
        for (int i = ny; carry && i < nx; i++)
        {
            uint64_t v = (uint64_t)x[i] + carry;
            x[i] = v;
            carry = v >> 32;
        }
        assert(!carry); // 调试检查，可删。
    }
    static void subFrom(It x, It xe, CIt y, CIt ye)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间且 |x|≥|y|；原地令 x-=y，要求 x 的值不小于 y（无最终借位）。
        int nx = xe - x, ny = ye - y;
        int32_t borrow = 0;
        for (int i = 0; i < ny; i++)
        {
            int64_t v = (int64_t)x[i] - y[i] + borrow;
            x[i] = v;
            borrow = v >> 32;
        }
        for (int i = ny; borrow && i < nx; i++)
        {
            int64_t v = (int64_t)x[i] + borrow;
            x[i] = v;
            borrow = v >> 32;
        }
        assert(!borrow); // 调试检查，可删。
    }
    static vector<uint32_t> addVec(const vector<uint32_t> &x,
                                   const vector<uint32_t> &y)
    {
        // x、y 是小端 limb 数组；返回去高零后的无符号和 x+y。
        vector<uint32_t> z(max(x.size(), y.size()) + 1);
```

```
        add(x.begin(), x.end(), y.begin(), y.end(), z.begin(), z.end());
        trim(z);
        return z;
    }
    static vector<uint32_t> subVec(const vector<uint32_t> &x,
                                   const vector<uint32_t> &y)
    {
        // x、y 是去高零的小端 limb 数组且 x≥y；返回去高零后的无符号差 x-y。
        assert(cmp(x, y) >= 0); // 调试检查，可删。
        vector<uint32_t> z = x;
        subFrom(z.begin(), z.end(), y.begin(), y.end());
        trim(z);
        return z;
    }

    static void mulNaive(CIt x, CIt xe, CIt y, CIt ye, It z, It ze)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间；竖式乘法把无符号积 x*y 写入 z。
        // z 长度恰为 |x|+|y|，且低 |y| 个 limb 须为零（通常整体清零）。
        int nx = xe - x, ny = ye - y;
        assert(ze - z == nx + ny); // 调试检查，可删。
        for (int i = 0; i < nx; i++)
        {
            uint32_t carry = 0;
            for (int j = 0; j < ny; j++)
            {
                uint64_t v = (uint64_t)x[i] * y[j] + carry + z[i + j];
                z[i + j] = v;
                carry = v >> 32;
            }
            z[i + ny] = carry;
        }
    }
    static void mul(CIt x, CIt xe, CIt y, CIt ye, It z, It ze)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间，z 长度恰为 |x|+|y| 且须预先清零；把无符号积 x*y 写入 z。
        // 较短因子不超过 128 个 limb 时用竖式，否则用 Karatsuba（长短悬殊时先把长因子对半拆开）。
        if (xe - x < ye - y)
        {
            swap(x, y);
            swap(xe, ye);
        }
        int nx = xe - x, ny = ye - y;
        assert(ze - z == nx + ny); // 调试检查，可删。
        if (ny <= 128)
        {
            mulNaive(x, xe, y, ye, z, ze);
            return;
        }
        int n = (nx + 1) / 2;
        if (ny <= n)
        {
```

```
396             mul(x, x + n, y, ye, z, z + n + ny);
397             vector<uint32_t> carry(z + n, z + n + ny);
398             fill(z + n, z + n + ny, 0);
399             mul(x + n, xe, y, ye, z + n, ze);
400             addTo(z + n, ze, carry.begin(), carry.end());
401             return;
402         }
403         mul(x, x + n, y, y + n, z, z + 2 * n);
404         mul(x + n, xe, y + n, ye, z + 2 * n, ze);
405         vector<uint32_t> x1(n + 1), y1(n + 1), mid(2 * n + 2);
406         add(x, x + n, x + n, xe, x1.begin(), x1.end());
407         add(y, y + n, y + n, ye, y1.begin(), y1.end());
408         mul(x1.begin(), x1.end(), y1.begin(), y1.end(), mid.begin(), mid.end());
409         subFrom(mid.begin(), mid.end(), z, z + 2 * n);
410         subFrom(mid.begin(), mid.end(), z + 2 * n, ze);
411         trim(mid);
412         addTo(z + n, ze, mid.begin(), mid.end());
413     }
414     static vector<uint32_t> mulVec(const vector<uint32_t> &x,
415                                    const vector<uint32_t> &y)
416     {
417         // x、y 是去高零的小端 limb 数组；返回去高零后的无符号积，任一为空（零）时返回空。
418         if (x.empty() || y.empty())
419         {
420             return {};
421         }
422         vector<uint32_t> z(x.size() + y.size());
423         mul(x.begin(), x.end(), y.begin(), y.end(), z.begin(), z.end());
424         trim(z);
425         return z;
426     }
427
428     static void leftShift(It x, It xe, int s)
429     {
430         // [x,xe) 是小端 limb 区间，0<=s<32；原地整体左移 s 位，要求最高 limb 被移出的位全为零。
431         assert(0 <= s && s < 32); // 调试检查，可删。
432         if (!s)
433         {
434             return;
435         }
436         int back = 32 - s;
437         uint32_t carry = 0;
438         for (auto it = x; it != xe; ++it)
439         {
440             uint32_t v = (*it << s) | carry;
441             carry = *it >> back;
442             *it = v;
443         }
444         assert(!carry); // 调试检查，可删。
445     }
446     static void shiftR(It x, It xe, int s)
447     {
```

```
448         // [x,xe) 是小端 limb 区间，0<=s<32；原地整体右移 s 位，要求最低 limb 的低 s 位为零（值能被
            ↪ 2^s 整除）。
449         assert(0 <= s && s < 32); // 调试检查，可删。
450         if (!s)
451         {
452             return;
453         }
454         int back = 32 - s;
455         uint32_t carry = 0;
456         for (auto it = xe; it != x;)
457         {
458             --it;
459             uint32_t v = (*it >> s) | carry;
460             carry = *it << back;
461             *it = v;
462         }
463         assert(!carry); // 调试检查，可删。
464     }
465
466     static void divNaive(CIt x, CIt xe, CIt y, CIt ye,
467                          It q, It qe, It r, It re)
468     {
469         // [x,xe) 是被除数、[y,ye) 是除数（最高 limb 非零）的小端 limb 区间；规格化长除法把商写入 q、
            ↪ 余数写入 r。
470         // 只写商与余数的有效低位，q 须能容纳商、r 至少 |y| 个 limb，二者须预先清零。
471         int nx = xe - x, ny = ye - y;
472         if (ny == 1)
473         {
474             uint64_t carry = 0;
475             for (int i = nx - 1; i >= 0; i--)
476             {
477                 uint64_t v = (carry << 32) | x[i];
478                 q[i] = v / y[0];
479                 carry = v - (uint64_t)q[i] * y[0];
480             }
481             r[0] = carry;
482             return;
483         }
484         if (max(nx, ny) <= 2)
485         {
486             uint64_t xv = x[0] | (nx == 2 ? (uint64_t)x[1] << 32 : 0);
487             uint64_t yv = y[0] | (ny == 2 ? (uint64_t)y[1] << 32 : 0);
488             uint64_t qv = xv / yv, rv = xv % yv;
489             if (q != qe)
490             {
491                 q[0] = qv;
492                 if (q + 1 != qe)
493                 {
494                     q[1] = qv >> 32;
495                 }
496             }
497             r[0] = rv;
```

```
            if (r + 1 != re)
            {
                r[1] = rv >> 32;
            }
            return;
        }
        if (less(x, xe, y, ye))
        {
            copy(x, min(xe, x + (ye - y)), r);
            return;
        }
        int shift = countl_zero(y[ny - 1]);
        vector<uint32_t> a(nx + (countl_zero(x[nx - 1]) < shift));
        vector<uint32_t> b(y, ye);
        copy(x, xe, a.begin());
        leftShift(a.begin(), a.end(), shift);
        leftShift(b.begin(), b.end(), shift);
        uint32_t top = b.back();
        vector<uint32_t> qv(a.size() - b.size() + 1);
        vector<uint32_t> rv(a.end() - b.size(), a.end());
        for (int i = qv.size() - 1; i >= 0; i--)
        {
            if (rv.size() == b.size())
            {
                if (!less(rv.begin(), rv.end(), b.begin(), b.end()))
                {
                    qv[i] = 1;
                    rv = subVec(rv, b);
                }
            }
            else if (rv.size() > b.size())
            {
                uint64_t top2 = ((uint64_t)rv.back() << 32) | rv[rv.size() - 2];
                uint64_t z = min<uint64_t>(UINT32_MAX, top2 / top);
                vector<uint32_t> bz = mulVec(b, vector<uint32_t>{(uint32_t)z});
                while (less(rv.begin(), rv.end(), bz.begin(), bz.end()))
                {
                    z--;
                    bz = subVec(bz, b);
                }
                rv = subVec(rv, bz);
                while (!less(rv.begin(), rv.end(), b.begin(), b.end()))
                {
                    z++;
                    rv = subVec(rv, b);
                }
                qv[i] = z;
            }
            if (i)
            {
                rv.insert(rv.begin(), a[i - 1]);
            }
```

```
            trim(rv);
        }
        shiftR(rv.begin(), rv.end(), shift);
        trim(qv);
        trim(rv);
        copy(qv.begin(), qv.end(), q);
        copy(rv.begin(), rv.end(), r);
    }

    static void div2n1n(CIt x, CIt xe, CIt y, CIt ye,
                        It q, It qe, It r, It re)
    {
        // x 长 2n、y 长 n（divide 规格化后的除数），要求 x 的高 n 个 limb 小于 y；q 长 n、r 长 n+1，均须预先清零。
        // 把 x/y 的商写入 q、余数写入 r；n 为奇数或不超过 naiveDiv 时用长除法，否则拆成两次 3n/2n 除法。
        int n = ye - y;
        assert(xe - x == 2 * n && qe - q == n && re - r == n + 1); // 调试检查，可删。
        assert(less(x + n, xe, y, ye)); // 调试检查，可删。
        if (n % 2 || n <= naiveDiv)
        {
            divNaive(x, xe, y, ye, q, qe, r, re);
            return;
        }
        int h = n / 2;
        vector<uint32_t> r1(n + h + 1);
        copy(x, x + h, r1.begin());
        div3n2n(x + h, xe, y, ye, q + h, qe, r1.begin() + h, r1.end());
        div3n2n(r1.begin(), r1.end() - 1, y, ye, q, q + h, r, re);
    }
    static void div3n2n(CIt x, CIt xe, CIt y, CIt ye,
                        It q, It qe, It r, It re)
    {
        // x 长 3n、y 长 2n，要求 x 的高 2n 个 limb 小于 y；q 长 n、r 长 2n+1，均须预先清零。
        // 用高位 2n/n 除法（或全 1）估商，再反复 q--、r+=y 修正，把 x/y 的商写入 q、余数写入 r。
        int n = (ye - y) / 2;
        assert(xe - x == 3 * n && qe - q == n && re - r == 2 * n + 1); // 调试检查，可删。
        assert(less(x + n, xe, y, ye)); // 调试检查，可删。
        vector<uint32_t> d(2 * n);
        if (less(x + 2 * n, xe, y + n, ye))
        {
            div2n1n(x + n, xe, y + n, ye, q, qe, r + n, re);
            mul(q, qe, y, y + n, d.begin(), d.end());
        }
        else
        {
            fill(q, qe, UINT32_MAX);
            add(x + n, x + 2 * n, y + n, ye, r + n, re);
            copy(y, y + n, d.begin() + n);
            subFrom(d.begin(), d.end(), y, y + n);
        }
        copy(x, x + n, r);
```

```
        while (less(r, re, d.begin(), d.end()))
        {
            addTo(r, re, y, ye);
            vector<uint32_t> one{1};
            subFrom(q, qe, one.begin(), one.end());
        }
        subFrom(r, re, d.begin(), d.end());
    }

    static void divide(CIt x, CIt xe, CIt y, CIt ye,
                       It q, It qe, It r, It re)
    {
        // [x,xe)、[y,ye) 是小端 limb 区间（y 非空且最高 limb 非零），q 长 |x|-|y|+1、r 长 |y|，均须预先清零。
        // 把 x/y 的商写入 q、余数写入 r；较短时用长除法，否则补零规格化后逐块调用 2n/n 除法。
        int nx = xe - x, ny = ye - y;
        assert(ny > 0 && qe - q == nx - ny + 1 && re - r == ny); // 调试检查，可删。
        if (min(nx - ny, ny) <= naiveDiv)
        {
            divNaive(x, xe, y, ye, q, qe, r, re);
            return;
        }
        int blocks = (ny + naiveDiv - 1) / naiveDiv;
        if (blocks > 1)
        {
            blocks = bit_ceil((unsigned long long)blocks);
        }
        int n = (ny + blocks - 1) / blocks * blocks;
        int digit = n - ny;
        int shift = countl_zero(y[ny - 1]);
        vector<uint32_t> a(nx + digit + (countl_zero(x[nx - 1]) <= shift));
        vector<uint32_t> b(n), rem(n + 1), z(2 * n);
        copy(x, xe, a.begin() + digit);
        copy(y, ye, b.begin() + digit);
        leftShift(a.begin() + digit, a.end(), shift);
        leftShift(b.begin() + digit, b.end(), shift);
        int t = max<long long>(2, (a.size() + n - 1) / n);
        copy(a.begin() + (t - 2) * n, a.end(), z.begin());
        int upper = qe - (q + (t - 2) * n);
        if (upper < n)
        {
            vector<uint32_t> qq(n);
            div2n1n(z.begin(), z.end(), b.begin(), b.end(),
                    qq.begin(), qq.end(), rem.begin(), rem.end());
            copy(qq.begin(), qq.begin() + upper, q + (t - 2) * n);
        }
        else
        {
            div2n1n(z.begin(), z.end(), b.begin(), b.end(),
                    q + (t - 2) * n, q + (t - 1) * n, rem.begin(), rem.end());
        }
        for (int i = t - 3; i >= 0; i--)
```

```
        {
            copy(a.begin() + i * n, a.begin() + (i + 1) * n, z.begin());
            copy(rem.begin(), rem.begin() + n, z.begin() + n);
            fill(rem.begin(), rem.end(), 0);
            div2n1n(z.begin(), z.end(), b.begin(), b.end(),
                    q + i * n, q + (i + 1) * n, rem.begin(), rem.end());
        }
        copy(rem.begin() + digit, rem.begin() + digit + (re - r), r);
        shiftR(r, re, shift);
    }

    static pair<vector<uint32_t>, vector<uint32_t>> divVec(
        const vector<uint32_t> &x, const vector<uint32_t> &y)
    {
        // x、y 是去高零的小端 limb 数组且 y 非空；返回去高零后的无符号 {商, 余数}，x<y 时为 {空,x}。
        assert(!y.empty()); // 调试检查，可删。
        if (cmp(x, y) < 0)
        {
            return {{}, x};
        }
        vector<uint32_t> q(x.size() - y.size() + 1), r(y.size());
        divide(x.begin(), x.end(), y.begin(), y.end(),
               q.begin(), q.end(), r.begin(), r.end());
        trim(q);
        trim(r);
        return {q, r};
    }
};
```

用 `HexBigInt x(s)` 从带可选正负号的十六进制串构造，`toHex()` 或流输出返回大写规范串；支持比较、`+ - * / %` 与 `divmod`。`HexBigInt::fromDecimal(s)` 与 `toDecimal()` 把十进制按 10^9 分组，用幂分治转换到二进制 limb，输出时按同一幂表递归除法恢复分组，因此十进制百万位题也复用快速乘除。输入须至少含一位合法数字，除数不得为零。设 limb 数为 $n \ge m$，加减 $O(n)$，Karatsuba 乘法约 $O(n^{\log_2 3})$，Burnikel–Ziegler 除法约 $O(M(n)\log n)$，进制转换增加同阶分治合并/拆分开销，空间由递归临时块决定；百万位目标依赖这些次平方路径，短十进制日常题继续用 14.1.001 高精度整数。

下面只检查固定短算术与跨块进位；Karatsuba、Burnikel–Ziegler 和长链进位放在本地测试文件：

```
signed main()
{
    assert((HexBigInt("FFFFFFFF") + HexBigInt(1)).toHex() == "100000000");
    assert((HexBigInt("1234") * HexBigInt("10")).toHex() == "12340");
    auto [q0, r0] = divmod(HexBigInt("123456789ABCDEF"), HexBigInt("1000"));
    assert(q0.toHex() == "123456789ABC" && r0.toHex() == "DEF");

    assert((HexBigInt("FFFFFFFFFFFFFFFF") + HexBigInt(1)).toHex()
           == "10000000000000000");
}
```

14.1.020 任意精度定点与十进制数

把十进制数表示为 $x = \texttt{man} \times 10^{-\texttt{scale}}$，其中 `man` 是带符号 `BigInt`，`scale` 是非负小数位数；`normalize` 会删去尾数的十进制末尾零，使相等比较可以直接检查规范表示。这一个类型统一覆盖动态精度定点数与常规任意精度十进制数，`str(fixed)` 还能固定输出小数位。全局精度 p 约束解析与四则运算保留的小数位数，舍弃部分按绝对值四舍五入；加减先对齐小数位，乘法相乘尾数，除法先补 $p+4$ 位保护位再做整数除法，平方根则把被开方数缩放成整数后调用 14.1.016 高精度整数平方根。实现依赖 14.1.001 高精度整数（BigInt），支持普通十进制串、比较、`+ - * / %`、`trunc`、`pow` 与 `sqrt`，不解析科学计数法。

```
// 依赖 14.1.016 高精度整数平方根
struct Decimal
{
    // man 是带符号十进制尾数，scale 是小数点后的十进制位数。
    BigInt man = 0;
    int scale = 0;
    // prec 是乘除、幂和平方根保留的小数位数。
    inline static int prec = 50;

    Decimal() = default;

    Decimal(long long x) : man(x)
    {
        // x 是 64 位整数；构造同值十进制数。
    }

    explicit Decimal(const BigInt &x) : man(x)
    {
        // x 是任意精度整数；构造同值十进制数。
    }

    explicit Decimal(const string &s)
    {
        // s 是带可选正负号和小数点的十进制串；构造按当前精度舍入后的十进制数。
        assign(s);
    }

    static void setPrecision(int digits)
    {
        // digits 是后续运算保留的小数位数；修改全局动态精度，无返回值。
        assert(digits >= 0);
        prec = digits;
    }

    static int getPrecision()
    {
        // 无参数；返回当前动态精度。
        return prec;
    }

    Decimal &assign(const string &s)
    {
```

```
        // s 是带可选正负号和至多一个小数点的十进制串；解析、舍入并返回当前对象。
        assert(!s.empty());
        int p = 0, sign = 1;
        if (s[p] == '+' || s[p] == '-')
        {
            sign = s[p] == '-' ? -1 : 1;
            p++;
        }
        assert(p < (int)s.size());
        string digit;
        bool dot = false;
        scale = 0;
        for (; p < (int)s.size(); p++)
        {
            if (s[p] == '.')
            {
                assert(!dot);
                dot = true;
                continue;
            }
            assert('0' <= s[p] && s[p] <= '9');
            digit.push_back(s[p]);
            if (dot)
            {
                scale++;
            }
        }
        assert(!digit.empty());
        man = BigInt((sign < 0 ? "-" : "") + digit);
        round(prec);
        normalize();
        return *this;
    }

    string str(int fixed = -1) const
    {
        // fixed 小于 0 时删除末尾零，否则固定输出 fixed 位小数；返回十进制字符串。
        assert(fixed >= -1);
        Decimal x = *this;
        if (fixed >= 0)
        {
            x.round(fixed);
        }
        if (!x.man.s)
        {
            return fixed > 0 ? "0." + string(fixed, '0') : "0";
        }
        bool neg = x.man.s < 0;
        string s = x.man.str();
        if (neg)
        {
            s.erase(s.begin());
```

```
        }
        if ((int)s.size() <= x.scale)
        {
            s = string(x.scale + 1 - s.size(), '0') + s;
        }
        string whole = s.substr(0, s.size() - x.scale);
        string fraction = x.scale ? s.substr(s.size() - x.scale) : "";
        if (fixed >= 0)
        {
            fraction += string(fixed - fraction.size(), '0');
        }
        string ans = (neg ? "-" : "") + whole;
        if (!fraction.empty())
        {
            ans += "." + fraction;
        }
        return ans;
    }

    void round(int digits)
    {
        // digits 是要保留的小数位数；按绝对值五入舍去其余位，无返回值。
        assert(digits >= 0);
        if (scale > digits)
        {
            man = drop10(man, scale - digits, true);
            scale = digits;
        }
        normalize();
    }

    Decimal trunc() const
    {
        // 无参数；返回向零取整后的整数值。
        Decimal ans;
        ans.man = drop10(man, scale, false);
        ans.normalize();
        return ans;
    }

    Decimal abs() const
    {
        // 无参数；返回当前数的绝对值。
        Decimal ans = *this;
        if (ans.man.s < 0)
        {
            ans.man.s = 1;
        }
        return ans;
    }

    bool isZero() const
```

```
147     {
148         // 无参数；返回当前数是否为零。
149         return man.s == 0;
150     }
151 
152     static Decimal pow(Decimal base, int exponent)
153     {
154         // base 是底数，exponent 是非负整数指数；返回按当前精度计算的幂。
155         assert(exponent >= 0);
156         Decimal ans(1);
157         while (exponent)
158         {
159             if (exponent & 1)
160             {
161                 ans *= base;
162             }
163             exponent >>= 1;
164             if (exponent)
165             {
166                 base *= base;
167             }
168         }
169         return ans;
170     }
171 
172     Decimal sqrt() const
173     {
174         // 无参数；当前数须非负，返回按当前精度四舍五入的平方根。
175         assert(man.s >= 0);
176         if (!man.s)
177         {
178             return Decimal(0);
179         }
180         int work = max(prec + 4, (scale + 1) / 2 + 2);
181         int exponent = 2 * work - scale;
182         assert(exponent >= 0);
183         Decimal ans;
184         ans.man = BigInt(bigSqrt(shift10(man, exponent).str()));
185         ans.scale = work;
186         ans.round(prec);
187         return ans;
188     }
189 
190     Decimal operator-() const
191     {
192         // 无参数；返回当前数的相反数。
193         Decimal ans = *this;
194         ans.man = -ans.man;
195         return ans;
196     }
197 
198     Decimal &operator+=(const Decimal &other)
```

```
    {
        // other 是加数；把它加到当前数并返回当前对象。
        return *this = *this + other;
    }

    Decimal &operator-=(const Decimal &other)
    {
        // other 是减数；从当前数减去它并返回当前对象。
        return *this = *this - other;
    }

    Decimal &operator*=(const Decimal &other)
    {
        // other 是乘数；把当前数乘以它并返回当前对象。
        return *this = *this * other;
    }

    Decimal &operator/=(const Decimal &other)
    {
        // other 是非零除数；把当前数除以它并返回当前对象。
        return *this = *this / other;
    }

    Decimal &operator%=(const Decimal &other)
    {
        // other 是非零除数；把当前数改为向零取商对应的余数并返回当前对象。
        return *this = *this % other;
    }

    friend Decimal operator+(const Decimal &a, const Decimal &b)
    {
        // a、b 是加数；返回按当前精度舍入的和。
        int scale = max(a.scale, b.scale);
        Decimal ans;
        ans.man = shift10(a.man, scale - a.scale) + shift10(b.man, scale - b.scale);
        ans.scale = scale;
        ans.round(prec);
        return ans;
    }

    friend Decimal operator-(const Decimal &a, const Decimal &b)
    {
        // a 是被减数，b 是减数；返回按当前精度舍入的差。
        return a + (-b);
    }

    friend Decimal operator*(const Decimal &a, const Decimal &b)
    {
        // a、b 是乘数；返回按当前精度舍入的积。
        Decimal ans;
        ans.man = a.man * b.man;
        ans.scale = a.scale + b.scale;
```

```
        ans.round(prec);
        return ans;
    }

    friend Decimal operator/(const Decimal &a, const Decimal &b)
    {
        // a 是被除数，b 是非零除数；返回按当前精度舍入的商。
        assert(b.man.s);
        if (!a.man.s)
        {
            return Decimal(0);
        }
        int sign = a.man.s * b.man.s;
        BigInt num = absInt(a.man), den = absInt(b.man);
        int work = prec + 4;
        int exponent = work + b.scale - a.scale;
        if (exponent >= 0)
        {
            num = shift10(num, exponent);
        }
        else
        {
            den = shift10(den, -exponent);
        }
        Decimal ans;
        ans.man = num / den;
        ans.man.s *= sign;
        ans.scale = work;
        ans.round(prec);
        return ans;
    }

    friend Decimal operator%(const Decimal &a, const Decimal &b)
    {
        // a 是被除数，b 是非零除数；返回 a-trunc(a/b)*b。
        assert(b.man.s);
        BigInt num = a.man, den = b.man;
        if (b.scale >= a.scale)
        {
            num = shift10(num, b.scale - a.scale);
        }
        else
        {
            den = shift10(den, a.scale - b.scale);
        }
        Decimal quotient(num / den);
        return a - quotient * b;
    }

    friend bool operator==(const Decimal &a, const Decimal &b)
    {
        // a、b 是待比较十进制数；返回二者是否相等。
```

```
303         return a.scale == b.scale && a.man == b.man;
304     }
305
306     friend bool operator≠(const Decimal &a, const Decimal &b)
307     {
308         // a、b 是待比较十进制数；返回二者是否不等。
309         return !(a == b);
310     }
311
312     friend bool operator<(const Decimal &a, const Decimal &b)
313     {
314         // a、b 是待比较十进制数；返回 a 是否小于 b。
315         int scale = max(a.scale, b.scale);
316         return shift10(a.man, scale - a.scale) < shift10(b.man, scale - b.scale);
317     }
318
319     friend bool operator>(const Decimal &a, const Decimal &b)
320     {
321         // a、b 是待比较十进制数；返回 a 是否大于 b。
322         return b < a;
323     }
324
325     friend bool operator<=(const Decimal &a, const Decimal &b)
326     {
327         // a、b 是待比较十进制数；返回 a 是否不大于 b。
328         return !(b < a);
329     }
330
331     friend bool operator>=(const Decimal &a, const Decimal &b)
332     {
333         // a、b 是待比较十进制数；返回 a 是否不小于 b。
334         return !(a < b);
335     }
336
337     friend ostream &operator<<(ostream &out, const Decimal &x)
338     {
339         // out 是输出流，x 是待输出十进制数；写入去掉末尾零的表示并返回输出流。
340         return out << x.str();
341     }
342
343     friend istream &operator>>(istream &in, Decimal &x)
344     {
345         // in 是输入流，x 接收十进制数；读取一个字符串并返回输入流。
346         string s;
347         in >> s;
348         if (in)
349         {
350             x.assign(s);
351         }
352         return in;
353     }
354
```

```
355   private:
356     static BigInt absInt(BigInt x)
357     {
358         // x 是任意精度整数；返回其绝对值。
359         if (x.s < 0)
360         {
361             x.s = 1;
362         }
363         return x;
364     }
365 
366     static BigInt shift10(BigInt x, int digits)
367     {
368         // x 是任意精度整数，digits 是非负十进制位数；返回 x*10^digits。
369         assert(digits >= 0);
370         if (!x.s || !digits)
371         {
372             return x;
373         }
374         int sign = x.s;
375         x.s = 1;
376         int blocks = digits / 9, rest = digits % 9;
377         if (rest)
378         {
379             int factor = 1;
380             for (int i = 0; i < rest; i++)
381             {
382                 factor *= 10;
383             }
384             x = BigInt::mulAbs(x, factor);
385         }
386         x.a.insert(x.a.begin(), blocks, 0);
387         x.s = sign;
388         return x;
389     }
390 
391     static BigInt drop10(BigInt x, int digits, bool nearest)
392     {
393        // x 是任意精度整数，digits 是舍去位数；nearest 表示是否绝对值五入，返回向零截断或舍入后的整数。
394         assert(digits >= 0);
395         if (!x.s || !digits)
396         {
397             return x;
398         }
399         int sign = x.s;
400         x.s = 1;
401         int blocks = digits / 9, rest = digits % 9;
402         int roundDig = 0;
403         if (rest && blocks < (int)x.a.size())
404         {
405             int factor = 1;
406             for (int i = 1; i < rest; i++)
```

```
            {
                factor *= 10;
            }
            roundDig = x.a[blocks] / factor % 10;
        }
        else if (!rest && blocks && blocks - 1 < (int)x.a.size())
        {
            roundDig = x.a[blocks - 1] / 100000000;
        }
        if (blocks >= (int)x.a.size())
        {
            x = 0;
        }
        else
        {
            x.a.erase(x.a.begin(), x.a.begin() + blocks);
            if (rest)
            {
                int divisor = 1;
                for (int i = 0; i < rest; i++)
                {
                    divisor *= 10;
                }
                unsigned long long carry = 0;
                for (int i = (int)x.a.size() - 1; i >= 0; i--)
                {
                    unsigned long long cur = carry * BigInt::B + x.a[i];
                    x.a[i] = cur / divisor;
                    carry = cur % divisor;
                }
            }
            x.norm();
            if (x.s)
            {
                x.s = sign;
            }
        }
        if (nearest && roundDig >= 5)
        {
            x += BigInt(sign);
        }
        return x;
    }

    void normalize()
    {
        // 无参数；删除尾数与小数位中的十进制末尾零，无返回值。
        man.norm();
        if (!man.s)
        {
            scale = 0;
            return;
```

```
        }
        while (scale >= 9 && !man.a.empty() && man.a[0] == 0)
        {
            man.a.erase(man.a.begin());
            scale -= 9;
        }
        man.norm();
        while (scale && man.a[0] % 10 == 0)
        {
            man = drop10(man, 1, false);
            scale--;
        }
    }
};
```

先用 `Decimal::setPrecision(p)` 设置 $p \ge 0$,再用 `Decimal(s)` 构造;`str()` 删除小数末尾零,`str(fixed)` 固定输出 `fixed` 位并按同一规则舍入。`pow(base,e)` 要求 $e \ge 0$，`sqrt()` 要求当前值非负，除法和取模要求除数非零；`%` 使用向零取商，满足 $a = \operatorname{trunc}(a/b)b + (a \bmod b)$。精度是全局状态，临时修改时应保存并恢复旧值。设参与运算的整数位与保留小数位合计至多 D，当前 `BigInt` 下加减、比较为 $O(D)$，乘除、取模与平方根最坏为 $O(D^2)$，`pow` 为 $O(D^2 \log e)$，额外空间为 $O(D)$；每一步都会舍入，连续运算不保证实数域中的结合律。

下面用固定答案检查符号、舍入、取模、幂和平方根:

```
signed main()
{
    Decimal::setPrecision(6);
    assert((Decimal("1.25") + Decimal("-0.5")).str() == "0.75");
    assert((Decimal("1.2") * Decimal("3.4")).str() == "4.08");
    assert((Decimal(1) / Decimal(8)).str(6) == "0.125000");
    assert((Decimal(2) / Decimal(3)).str(6) == "0.666667");
    assert((Decimal("-5.5") % Decimal(2)).str() == "-1.5");
    assert(Decimal::pow(Decimal("1.5"), 3).str() == "3.375");

    Decimal::setPrecision(30);
    assert(Decimal(2).sqrt().str(30) == "1.41421356237309504880168872421O");
}
```

14.1.021 高精度圆周率：Machin 与 Chudnovsky

Machin 公式把圆周率化为 $\pi = 16\arctan\frac{1}{5} - 4\arctan\frac{1}{239}$，其中 $\arctan x = \sum_{k\ge0}(-1)^k\frac{x^{2k+1}}{2k+1}$ 在本条的 $|x| < 1$ 下收敛；`atanRecip(q)` 逐项递推，直到新项在当前精度下舍入为零。Chudnovsky 版本使用 $\pi = \frac{426880\sqrt{10005}}{S}$，其中 $S = \sum_{k\ge0}\frac{(6k)!(13591409+545140134k)}{(3k)!(k!)^3(-262537412640768000)^k}$，相邻项的阶乘比可约成代码中的整数递推，每项约增加十四位正确数字。两个接口都临时把 14.1.020 任意精度定点与十进制数 的精度提高八位，求和结束后再舍入到目标位数。

```
// 依赖 14.1.020 任意精度定点与十进制数
Decimal atanRecip(int q)
{
```

```
    // q 是大于 1 的正整数；返回按当前 Decimal 精度计算的 arctan(1/q)。
    assert(q > 1);
    Decimal x = Decimal(1) / Decimal(q);
    Decimal square = x * x;
    Decimal term = x;
    Decimal ans(0);
    for (int k = 0;; k++)
    {
        Decimal add = term / Decimal(2 * k + 1);
        if (add.isZero())
        {
            break;
        }
        ans += add;
        term = -term * square;
    }
    return ans;
}

Decimal piMachin(int digits)
{
    // digits 是结果保留的小数位数；返回 Machin 公式计算并四舍五入的圆周率。
    assert(0 <= digits && digits <= numeric_limits<int>::max() - 8);
    int old = Decimal::getPrecision();
    Decimal::setPrecision(digits + 8);
    Decimal ans = Decimal(16) * atanRecip(5) - Decimal(4) * atanRecip(239);
    ans.round(digits);
    Decimal::setPrecision(old);
    return ans;
}

Decimal piChudnovsky(int digits)
{
    // digits 是结果保留的小数位数；返回 Chudnovsky 级数计算并四舍五入的圆周率。
    assert(0 <= digits && digits <= numeric_limits<int>::max() - 8);
    int old = Decimal::getPrecision();
    Decimal::setPrecision(digits + 8);
    BigInt mul = 1;
    BigInt power = 1;
    int linear = 13591409;
    Decimal sum(linear);
    for (int k = 1;; k++)
    {
        int x = 6 + 12 * (k - 1);
        mul *= BigInt(x * x * x - 16 * x);
        mul /= BigInt(k * k * k);
        linear += 545140134;
        power *= BigInt("-262537412640768000");
        Decimal term(mul * BigInt(linear));
        term /= Decimal(power);
        if (term.isZero())
        {
```

```
            break;
        }
        sum += term;
    }
    Decimal ans = Decimal(426880) * Decimal(10005).sqrt() / sum;
    ans.round(digits);
    Decimal::setPrecision(old);
    return ans;
}
```

`piMachin(digits)` 与 `piChudnovsky(digits)` 要求 $0 \le digits \le$ `numeric_limits` $<$ `int` $>::$ `max`$()-8$，返回保留 `digits` 位小数的圆周率并恢复原精度。八位保护位适合竞赛常见位数，但不构成任意规模下的严格误差证明；需要正确舍入时应增加保护位并验证误差界。令 $P = digits + 8$，当前二次复杂度大整数下二者最坏均为 $O(P^3)$ 时间、$O(P)$ 空间；Chudnovsky 约用 $P/14$ 项，通常更快，极高精度改用二分拆分。

下面用已知的四十位小数分别核对两种公式：

```
signed main()
{
    const string expected = "3.1415926535897932384626433832795028841972";
    assert(piMachin(40).str(40) == expected);
    assert(piChudnovsky(40).str(40) == expected);
}
```

15. ACM 常用英文单词

15.1 按字母索引

15.1.001 A

English	中文释义
abbreviation	[数学] 约分;
activity on edge	AOE 网
activity on vertex	AOV 网
add, subtract, multiply and divide	加减乘除
adjacency list	邻接表
adjacency multilist	邻接多重表
adjacency matrix	邻接矩阵
adjacent sequence elements	相邻的元素串
adjacent vertex	相邻顶点
algebraic term	代数项
alphabetical order	字典序
alternately rise and fall	交替上升和下降
Ambiguous	模糊不清
ancestor	祖先
anticlockwise	逆时针
Approximate String Matching	模糊匹配
Arbitrary Precision Arithmetic	高精度计算
arc	弧
arithmetic mean	算数平均值
arithmetic progression	等差数列
geometric progression	等比数列
array	数组
articulation point	连接点
ascending lexicographical order	词典顺序升序排列
ascending order	升序
descending order	降序
aspect ratio	固定长宽比
assemble	组合
assess	评定，评估
assigned	指定的，赋值的
augmenting path graph	增广路径图
augmenting path	增广路径
average search length	平均查找长度
average temperature	顺时针
axis axes	轴

15.1.002 B

English	中文释义
balance merging sort	平衡归并排序
balance two-way merging	二路平衡归并排序
Bandwidth Reduction	带宽压缩
banlanced binary tree	平衡二叉树
base	底边；幂的底数
biconnected graph	重连通图
bidirectional	双向的
binary search tree	二叉查找树
binary search	二分查找
binary sort tree	二叉排序树
binary	二进制
bipartite graph	二部图
Bishop	主教（象）只斜走。格数不限不能越子。每方有两象，黑白格各占 1 个
blank string	空白（空格）串
block search	分块查找
boundary	界限

15.1.003 C

English	中文释义
calculate	计算
Calendrical Calculations	日期
carpet	地毯
chariot	战车（中国象棋）
checkmate	(国际象棋）将死; 输棋，将死; 败局; 败北，挫败;
circular linked list	循环链表
cirular queue	循环队列
Clique	最大团
clockwise order	顺时针方向顺序
Coefficient	系数，率，程度
Collinear	共线的
column major order	以列为主的顺序分配
columns	列
Combinatorial Problems	组合问题
comma	逗号
common superstring	公共父串
compile	编译，汇编
complete binary tree	完全二叉树

English	中文释义
complete graph	完全图
composite numbers	合数
Computational Geometry	计算几何
concave	凹的
connected component	连通分量
Connected Components	连通分支
consecutive	连续的
constant	n 常数，常量 adj 不变的，始终如一的，持续不断的
Constrained and Unconstrained Optimization	最值问题
Convex Hull	凸包
coordinates	坐标
corrupt	腐烂，破坏
counterclockwise（anticlockwise ）	逆时针
critical path	关键路径
Cryptography	密码
Cube root	立方根

15.1.004 D

English	中文释义
D is rounded to 2 decimal places	D 是精确到小数点后 2 位
Data Structures	基本数据结构
data type	数据类型
decimal	n 小数，十进制；adj 小数的，十进制的
decision tree	判定树
Deck	甲板
define	定义，明确，使规定
deformed	变形的
Denominator	分母
denote	代表; 指代; 预示; 意思是; 标志；象征
dense graph	稠密图
Deployed	部署
depth	深度
deque(double-ended queue)	双端列表
descentdant	子孙
destination	终点
Determinants and Permanents	行列式
Diagonal	对角
diagonally	斜对角线的
dial	钟面，拨打
dialing	拨号音打电话，拨电话号码

English	中文释义
Dictionaries	字典
difference	差
digital analysis method	数字分析法
digital search tree	数字查找树
digit	位数; 数字
digraph(directed graph)	有向图
Dimensiona	尺寸
diminishing increment sort	随小增量排序
direct access file	直接存取文件
directed acyclic graph	有向无环图
directory structure	目录结构
directory	（计算机文件或程序的）目录; 指导的咨询的; 管理的
discrete Fourier transform	离散傅里叶变换
disjoint	不相交的
Distinct values	独一无二的值
distinct	不同的；独一无二的
division method	除法
divisor	因子; 分母
doubly linked list	双向链表
doubly linked tree	双链树
Drawing Graphs Nicely	图的描绘
Drawing Trees	树的描绘
duplicated	复制；打印的
duplicates	复制品

15.1.005 E

English	中文释义
Edge and Vertex Connectivity	割边/割点
Edge Coloring	边染色
embed	插入
enable	启用
Entry	进口
equation	方程式; 等式
equivalent equation	同解方程; 等价方程
equivalent	相等的，等效的
estimate	预测
Eulerian Cycle / Chinese Postman Euler	回路/中国邮路
evaluate	评价，估价
evaluated	求…的值
even	偶数的

English	中文释义
excluding	排除，拒绝，驱逐; 除...外，不包括
execute	执行，完成
executed	执行的；生效的
exponent	指数；幂
external sort	外部排序

15.1.006 F

English	中文释义
Facility	设备，设施
factorial	阶乘; 因子的，阶乘的
Factoring and Primality Testing	因子分解/质数判定
Feedback Edge/Vertex Set	最大无环子图
Finite State Machine Minimization	有穷自动机简化
fixed-aggregate data type	固定聚合数据类型
foggiest idea	概念
folding method	折叠法
follow by	跟随，其后
forest	森林
formula	公式
fraction	分数; 小部分
front	队头
full binary tree	满二叉树

15.1.007 G

English	中文释义
gcd （greatest common divisor）	最大公约数
generalized list	广义表
Generating Graphs	图的生成
Generating Partitions	划分生成
Generating Permutations	排列生成
Generating Subsets	子集生成
geometric progression	等比数列
grabh	图
Graph Data Structures	图
Graph Isomorphism	同构
Graph Partition	图的划分
Graph Problems — hard	图论-NP 问题
Graph Problems — polynomial	图论-多项式算法

English	中文释义
greatest integer	最大整数
grid	网格；方格；（地图上的）坐标方格

15.1.008 H

English	中文释义
Hamiltonian Cycle Hamilton	回路
hash search	散列查找
hash table	散列表
head node	头结点
head pointer	头指针
heap sort	堆排序
horizontal or vertical direction	水平和垂直方向
horizontally	adv 水平地
horizontal	adj 水平的
Huffman tree	哈夫曼树

15.1.009 I

English	中文释义
Identifier	标识符，识别码
immediate predecessor	直接前趋
immediate successor	直接后继
immediately allocating method	直接定址法
improper fraction	假分数
in the range of	在...范围内
in the shape of a cross	十字形
incident edge	关联边
indegree	入度
indent	缩进
indentical	相同的
Independent Set	独立集
indexed file	索引文件
indexed non-sequential file	索引非顺序文件
indexed sequential file	顺序
indicating	adj 指示的，标志的
inequality	不等式
infinite	无限的
initial	adj 最初的，词首的，开始的 n 首字母
initial node	初始结点

English	中文释义
initialization	n 初始化，赋初值
initialize	v 初始化
inorder traversal	中序遍历
insertion sort	插入排序
insertion	插入
integer	整数
Interior	内部，本质
internal sort	内部排序
Interpret	解释，执行
intersect	v 相交，交叉
intersection	横切; 相交; 交叉点，交叉线; [数] 交集;
Intersection Detection	碰撞测试
intersect	相交
intervals	间隔时间; 间隔；区间
Invade	侵略
invalid	无效的
inverted file	倒排文件
irreparably	不能恢复地

15.1.010 J

English	中文释义
Job Scheduling	工程安排
justified	合理的，合法化的

15.1.011 K

English	中文释义
Kd-Trees	线段树
Knapsack Problem	背包问题
Knight	骑士（马）每步棋先横走或直走一格，然后再往外斜走一格；或者先斜走一格，最后再往外横走或竖走一格（即走 “日” 字)。可以越子，没有象棋中的 “蹩马腿” 限制。

15.1.012 L

English	中文释义
lcm (Least Common Multiple)	最小公倍数
left or right-justified	左对齐 or 右对齐
lexicographically	字典序
like terms ,similar terms	同类项
linear algebra	线性代数
linear equation	线性方程
linear linked list	线性链表
Linear Programming	线性规划
linear structure	线性结构
link field	链域
linked list	链表
literal coefficient	字母系数
logarithm	对数
logical structure	逻辑结构
Longest Common Substring	最长公共子串
loop	环

15.1.013 M

English	中文释义
Maintaining Line Arrangements	平面分割
master file	主文件
Matching	匹配
Matrix Multiplication	矩阵乘法
maximum matching	最大匹配
meadow	草坪
mean	平均值
Medial-Axis Transformation	中轴变换
Median and Selection	中位数
memorable	值得纪念的; 显著的，难忘的; 重大的，著名的
merge sort	归并排序
mid-square method	平方取中法
minimal	adj 最小限度的
minimal volume	最小体积
minimum(cost)spanning tree	最小（代价）生成树
mixed number	带分数
mod	v 求余
modulus	n 系数，模数
Motion Planning	运动规划
motion	多边形
multi-dimentional array	多维数组

English	中文释义
multilinked list	多重链表
multilist file	多重链表文件
multiple	adj 多重的多样的，许多的 n 倍数
multiplication	乘法
municipal	市政的

15.1.014 N

English	中文释义
Nearest Neighbor Search	最近点对查询
negative , positive	负，正
Network Flow	网络流
no special punctuation symbols or spacingrules	无特殊标点符号或间距的规则
non-intersecting	非相交的; 不相交的;
nonlinear structure	非线性结构
notation	标记
numerator	分子
numerical coefficient	数字系数
Numerical Problems	数值问题

15.1.015 O

English	中文释义
Obesity	肥胖
octal (binhex)	adj 八进制的 (十六进制)
odd and even	奇和偶
optimal	最佳的
optimally	最佳
Orbit	轨道
ordered pair	有序对
ordered tree	有序树
Ordinal	有次序的
original equation	原方程
origin	原点
orthogonal list	十字链表
Out degree	出度
Over brim	溢出
overflow	上溢
Overlapping	覆盖

English	中文释义
ox	牛

15.1.016 P

English	中文释义
palindrome	回文
palindromic	回文的
parallel	平行的
parity property	奇偶性
partical order	偏序
Pawn	禁卫军（兵）只能向前直走，每次只能走一格。但走第一步时，可以走一格或两格。兵的吃子方法与行棋方向不一样，它是直走斜吃，即如果兵的斜进一格内有对方棋子，就可以吃掉它而占据该格
phyical structure	物理结构
Pipe	管道
Planarity Detection and Embedding	平面性检测和嵌入
ploygon-shaped faces/ polygon	多边形
ployphase merging sort	多步归并排序
Point Location	位置查询
pointer field	指针域
Polygon Partitioning	多边形分割
positive and negative integers	正整数和负整数
postorder traversal	后序遍历
precision	n 精密，精确度精确
predecessor	前趋
prefix	前缀
preorder traversal	先序遍历
prime	质数
Priority Queues	优先队列
proceed	运行
process	v 加工，处理 n 程序，进程
process a sequence of n distinct integers	处理一串 n 个不同的整数
profile	轮廓
proper fraction	真分数
proportional	成比例的
Protrusions	凸起物
Pyramid	金字塔，渐增

15.1.017 Q

English	中文释义
quadrant	象限，四分之一圆
Queen	皇后横、直、斜都可以走，步数不受限制，但不能越子
quotient	商

15.1.018 R

English	中文释义
radix sort	基数排序
Random Number Generation	随机数生成
random number method	随机数法
Range Search	范围查询
rat, ox, tiger, rabbit, dragon, snake,horse, sheep, monkey, rooster, dog pig	十二生肖
rate of convergence	收敛速度
rear	队尾
rectangular	矩形的，成直角的
Relates	叙述，讲述
replacement selection sort	置换选择排序
respectively	各自的，分别的，独自的
robustness	鲁棒性
Rook	战车横竖均可以走，步数不受限，不能斜走。除王车易位外不能越子。
rooster	鸡
root sign	根号
round()	四舍五入（当取舍位为 5 时，若取舍位数前的小数为奇数则直接舍弃，若为偶数则向上取舍）
rounded to n decimal places	精确到小数点后 n 位
row major order	以行为主的顺序分配
Rows and columns	行与列

15.1.019 S

English	中文释义
Satisfiability	可满足性
scenario	方案；(可能发生的) 情况；
search (sequential search)	线性查找（顺序查找）
searching	查找，线索
segment	段，分段; 分割；环节; 部分; 划分；
selection sort	选择排序
semicolon	分号

English	中文释义
sequence	顺序，序列，连续
serial	连续的; 连载的; 顺序排列的;
series	连续的同类事物，系列
Set and String Problems	集合与串的问题
Set Cover	集合覆盖
Set Data Structures	集合
Set Packing	集合配置
Shape Similarity	相似多边形
Shell	贝壳，脱壳
shelter	遮蔽物
Shortest Common Superstring	最短公共父串
Shortest Path	最短路径
simple cycle	简单回路
simple path	简单路径
Simplifying Polygons	多边形化简
simultaneously	同时的
single linked list	单链表
sink	汇点
solution	解决方案
Solving Linear Equations	线性方程组
source	源点
spanning forest	生成森林
spanning tree	生成树
spares graph	稀疏图
sparse matrix	稀疏矩阵
specify	指定
square root	平方根
square	平方，正方形，广场，方格
Squared	平方
Stack Overflow	堆栈溢出通常是您的程序陷入了无穷递归，或递归嵌套层数过多。
statistical	统计的
Steiner Tree Steiner	树
stem	词根
String Matching	模式匹配
strongly connected graph	强连通图
subgraph	子图
subsequent	随后的，后来的
substring	子串
subtree	子树
successor	后继
sufficient	充足的；足够的；
suffix	后缀

English	中文释义
Supervisor	监督人
symmetric matrix	对称矩阵

15.1.020 T

English	中文释义
tail pointer	尾指针
terminal node	终端结点
Text Compression	压缩
threaded binary tree	线索二叉树
times	乘
Topological Sorting	拓扑排序
toss	扔（硬币）
Transitive Closure and Reduction	传递闭包
transposed matrix	转置矩阵
traversal of tree	树的遍历
traversing binary tree	遍历二叉树
traversing graph	遍历图
tree index	树型索引
triangle	三角形
triangle inequality	三角不等式
Triangulation	三角剖分
triple	三倍的，三方的，三部分的; 增至三倍; 三倍的数 [量]; 三个一组;
Tromino	三格骨牌
Troop	军队，组群
truangular matrix	三角矩阵
two adjacent sequence elements	两个相邻的元素串
two-dimensional array	二维数组
two-dimensional	维数

15.1.021 U

English	中文释义
ultimate	基本的，终极的
unconnected graph	非连通图
underflow	下溢
undigraph(undirected graph)	无向图
union	并集
unique identifier	唯一的标识符
unordered pair	无序对

English	中文释义
unordered tree	无序树
uppercase	大写字母盘; 以大写字母印刷; 大写字母的
uppercase（Capital）	大写字母
（Lowercase letters）	（小写字母）

15.1.022 V

English	中文释义
variable-aggregate data type	可变聚合数据类型
variable	变量
Vertex Coloring	点染色
Vertex Cover	点覆盖
vertex	n 顶点，最高点
vertical	n 垂直线，垂直面 adj 垂直的，顶点的
volume	数量，容量
Voronoi Diagrams Voronoi	图
vulnerable	容易受到攻击的

15.1.023 W

English	中文释义
weakly connected graph	弱连通图
weight	权
weighted average	加权平均值
weighted graph	加权图
wooden planks	木板